GROUPEMENT INTERBANCAIRE MONÉTIQUE DE L'AFRIQUE CENTRALE
Banque des États de l'Afrique Centrale (BEAC)
Département Exploitation et Supervision (DES)

PLAN D'IMPLÉMENTATION COMPLET

LOT 1 : MODULE DE GESTION DE LA CERTIFICATION

MS-HOMOLOGATION (HOM-01 à HOM-24) · MS-CERTIFICATION (CER-01 à CER-15)

Écrans DES-08 à DES-12 et POR-03 · backend et frontend

Socle GIMAC : JHipster, Kafka, Keycloak, OPA, Ceph WORM, Prometheus, Kubernetes · outbox transactionnelle · CloudEvents · CRUD universel

Avec tests unitaires · tests d'intégration · critères Gherkin · validation cURL et Playwright via la passerelle

Référence	**PI-GIMAC-01 v1.0**
Livrable	Livrable 5 : plan d'implémentation (sprint backlog exécutable) du lot 1
Documents sources	Carnet de produit CP-GIMAC-01 v2.0 ; cahier de conception CC-GIMAC-01 v1.1 ; système de conception DS-GIMAC-01 v1.0 ; état du socle au 29 septembre 2026
Contenu	Convention transversale, Phase 0, 39 récits backend traités de bout en bout, 7 fragments frontend, intégrations transverses, cartographie, récapitulatif et Definition of Done en 40 points
Statut	Soumis à validation du responsable de produit
Établi par	**KOAGNE ISSA RYAN**, Ingénieur **NGOUNOU STEVE DELORS**, Ingénieur
Sous la supervision de	**Mme EKOUTOU GUILEINE** **M. TABANGOLI Anthony** **M. SOSSY Martial**

GIMAC, siège social, Yaoundé, République du Cameroun
Plateforme intégrée de suivi, de supervision et de dé-silotage des activités du DES
Plan d'implémentation du lot 1, version 1.0, octobre 2026

Fiche d'identification du document

Rubrique	Valeur
Titre	Plan d'implémentation, lot 1 : module de gestion de la certification
Référence	PI-GIMAC-01 v1.0
Version	Version 1.0 (première édition du plan d'implémentation du lot 1)
Date d'émission	Octobre 2026
Établi par	KOAGNE ISSA RYAN, Ingénieur ; NGOUNOU STEVE DELORS, Ingénieur
Organisme	Groupement Interbancaire Monétique de l'Afrique Centrale (GIMAC), Département Exploitation et Supervision
Destinataires	Responsable de produit, équipe de développement (backend et frontend), RSSI, responsable conformité, chef du DES
Périmètre	MS-HOMOLOGATION (24 récits, 118 points), MS-CERTIFICATION (15 récits, 76 points), écrans DES-08 à DES-12 et POR-03 (environ 60 points)
Documents amont	CCT-GIMAC-01 v1.1 ; CA-GIMAC-01 v1.1 ; CC-GIMAC-01 v1.1 ; CP-GIMAC-01 v2.0 ; DS-GIMAC-01 v1.0
Document aval	Rapport de recette du lot 1 ; plan d'implémentation du lot 2 (incidents et monitoring)
Classification	Interne GIMAC ; aucune donnée nominative, aucune adresse d'infrastructure, aucun secret

Historique des versions

Version	Date	Auteur	Modifications
1.0	Octobre 2026	Équipe projet	Première édition : convention transversale, Phase 0, 39 récits backend, 7 fragments frontend, intégrations transverses, cartographie, récapitulatif, Definition of Done

Mode de lecture

Le document se lit dans l'ordre d'exécution. La convention transversale fixe les règles communes ; la Phase 0 prépare le socle ; chaque récit est ensuite traité de bout en bout, dans l'ordre d'implémentation, avec ses cinq sections : A. implémentation, B. tests unitaires, C. tests d'intégration, D. tests Gherkin, E. tests cURL ou Playwright via la passerelle. Les fragments frontend suivent le même schéma par écran. Les chapitres de fin consolident les contrats, les fichiers, le calendrier, la Definition of Done, les risques et les prompts d'amorçage de l'agent codeur.

Les extraits de code sont des références d'implémentation : ils fixent les noms, les signatures, les transitions et les contrôles attendus, et s'écrivent dans les fichiers indiqués en première

ligne de chaque extrait. Le dépôt fait foi pour les noms déjà générés par le socle ; le tableau de correspondance produit en Phase 0 fait le lien avec le présent plan.

Sommaire

Fiche d'identification du document....2
Historique des versions....2
Mode de lecture....2
Sommaire....3
CONVENTION TRANSVERSALE │ Lot 1 : module de gestion de la certification....6
Objet du lot 1....6
Schéma appliqué à chaque récit....6
Principes qui ne se négocient pas....7
Hypothèses sur le socle, à confirmer à l'étape 0.1....8
Ordre d'exécution et jalons....8
Utilisateurs de test et jeu de données....9
PHASE 0 │ Préparation du socle pour le lot 1....10
0.1. Relevé du socle....10
0.2. Modèle de données de référence (JDL)....10
0.3. Corrections Liquibase après génération....13
0.4. Classes communes de messagerie....15
0.5. Sujets Kafka, schémas et rebut....17
0.6. Permissions, politiques OPA et routes de la passerelle....17
0.7. Activation des composants inactifs nécessaires au lot....19
0.8. Arborescence hexagonale greffée sur le squelette généré....20
0.9. Compilation, exécution et scripts de test....21
MS-HOMOLOGATION │ Récits HOM-01 à HOM-24 : suivi de l'homologation des participants....22
HOM-01 │ Publier une version du référentiel d'opérations par type de projet....23
HOM-02 │ Contrôler l'acyclicité et la cohérence du référentiel avant publication....33
HOM-03 │ Migrer de façon contrôlée les liaisons en cours vers une nouvelle version du référentiel....40
HOM-04 │ Ouvrir une liaison pour une institution et un type de projet....50
HOM-05 │ Dériver le périmètre contractualisé des services souscrits....58
HOM-06 │ Geler une liaison sur changement de statut de l'institution....66

HOM-07 │ Instancier les opérations d'une liaison par tri topologique....73
HOM-08 │ Attribuer le responsable principal de chaque opération....81
HOM-09 │ Déposer la preuve attendue d'une opération....88
HOM-10 │ Clore une opération selon la règle des quatre éléments....95
HOM-11 │ Rouvrir une opération close sur décision motivée....104
HOM-12 │ Suivre la vérification du code QR marchand statique et dynamique....109
HOM-13 │ Reprendre en lecture seule les résultats des campagnes du simulateur....114
HOM-14 │ Tolérer l'indisponibilité du simulateur de certification....122
HOM-15 │ Rapprocher les scénarios repris des opérations et isoler les scénarios inconnus 129
HOM-16 │ Garantir la non-intrusion vis-à-vis du simulateur de certification....135
HOM-17 │ Calculer les couvertures de certification et de mise en service par service....141
HOM-18 │ Détecter l'aptitude à la certification et saisir MS-CERTIFICATION....146
HOM-19 │ Refléter dans la liaison les décisions du circuit de certification....151
HOM-20 │ Piloter les liaisons dans un tableau de bord....156
HOM-21 │ Notifier les échéances des opérations....161
HOM-22 │ Exporter le dossier de preuves d'une liaison....165
HOM-23 │ Déposer une preuve depuis le portail des participants....171
HOM-24 │ Suivre sa liaison depuis le portail des participants....176
MS-CERTIFICATION │ Récits CER-01 à CER-15 : certification, quitus et mise en service 181
CER-01 │ Ouvrir le circuit de certification d'une liaison apte....182
CER-02 │ Signer le quitus technique après réauthentification FIDO2....191
CER-03 │ Signer en parallèle les quitus sécurité et conformité....199
CER-04 │ Signer le quitus de la Direction Générale et autoriser la mise en service....205
CER-05 │ Refuser un quitus et renvoyer la liaison en homologation....211
CER-06 │ Rendre caducs les quitus antérieurs lorsque le socle d'évaluation change....216
CER-07 │ Contrôler l'impartialité des signataires....221
CER-08 │ Contrôler la complétude du dossier de mise en service....226
CER-09 │ Lire la fiche de changement sans jamais écrire dans le dispositif de gestion des changements....234
CER-10 │ Consigner le procès-verbal de mise en service dans la fenêtre de changement 240
CER-11 │ Sceller et horodater les quitus et le dossier de mise en service....245

CER-12 │ Émettre l'attestation de certification au format PDF/A-3 signée PAdES.....251
CER-13 │ Consulter sa certification et télécharger l'attestation depuis le portail.....259
CER-14 │ Tenir le registre des certifications délivrées et suivre leur renouvellement.....264
CER-15 │ Révoquer ou suspendre une attestation de certification.....269
INTERFACE │ Socle d'interface du lot et écrans DES-08 à DES-12 et POR-03.....275
F0 │ Socle d'interface du lot 1 : descripteurs, composants de domaine et hooks.....276
DES-12 │ Référentiels d'opérations.....284
DES-08 │ Liaisons et parcours.....290
DES-09 │ Vue d'une liaison.....295
DES-10 │ Clore une opération.....302
DES-11 │ Dossier de mise en service.....308
POR-03 │ Ma liaison (portail des participants).....316
INTÉGRATIONS TRANSVERSES │ Observabilité, résilience et sécurité du lot 1.....322
Métriques métier exposées à la pile d'observabilité.....322
Résilience des appels sortants.....323
Politique réseau et non-intrusion.....324
Contrôles de sécurité de fin d'itération.....325
CARTOGRAPHIE │ Contrats REST, événements et permissions du lot 1.....326
Contrats de MS-HOMOLOGATION.....326
Contrats de MS-CERTIFICATION.....330
RÉCAPITULATIF │ Fichiers, ordre d'implémentation, calendrier et Definition of Done....334
Fichiers à créer ou modifier dans MS-HOMOLOGATION.....334
Fichiers à créer ou modifier dans MS-CERTIFICATION.....336
Fichiers à créer dans l'application d'interface.....338
Ordre d'implémentation.....338
Calendrier indicatif.....339
Definition of Done du lot 1 (40 points).....339
Registre des risques du lot.....341
Prompts d'amorçage pour l'agent codeur.....342
Conclusion.....343

CONVENTION TRANSVERSALE │ Lot 1 : module de gestion de la certification

Objet du lot 1

Le socle de la plateforme est opérationnel de bout en bout : quatorze squelettes de services, la passerelle et ses treize routes, le fournisseur d'identité, le moteur de politiques en refus par défaut, le dépôt objet à verrou WORM, le bus d'événements, la pile d'observabilité, le masquage des journaux et la chaîne de construction (1 356 tests unitaires, 5 476 tests d'intégration, 48 contrôles de bout en bout, rendu Kubernetes de 111 objets). Le lot 1 spécialise les deux premiers microservices métier, MS-HOMOLOGATION puis MS-CERTIFICATION, et livre leurs écrans, de façon à obtenir en priorité le module de gestion de la certification : référentiel d'opérations, liaisons, règle des quatre éléments, reprise en lecture seule du simulateur, couvertures, circuit des quatre quitus, dossier de mise en service, scellement et attestation.

Le présent plan est le sprint backlog exécutable de ce lot. Il découle du carnet de produit CP-GIMAC-01 v2.0 (récits HOM-01 à HOM-24 et CER-01 à CER-15), du cahier de conception CC-GIMAC-01 v1.1 (classes de conception du domaine 3, algorithmes 6.30 à 6.38) et du système de conception DS-GIMAC-01 v1.0 (écrans DES-08 à DES-12 et POR-03, CRUD universel). Chaque récit y est traité de bout en bout selon un schéma unique : implémentation par étapes avec le code, tests unitaires, tests d'intégration, critères d'acceptation Gherkin, validation cURL ou Playwright à travers la passerelle.

Schéma appliqué à chaque récit

Couche	Outil ou cadre	Objectif
Implémentation backend	Java 21, Spring Boot 3, JHipster (squelettes du socle), JPA et Liquibase, MapStruct	Code de production organisé en couches hexagonales greffées sur le squelette généré
Messagerie	Kafka, CloudEvents 1.0, outbox transactionnelle, consommateur idempotent	Émission fiable et consommation sans doublon ; MS-AUDIT consomme tout
Sécurité	Keycloak (JWT RS256 relayé par la passerelle), OPA (refus par défaut), `@PreAuthorize` par permission, réauthentification FIDO2 par MS-IAM	Double contrôle passerelle et service ; signatures uniquement après réauthentification forte
Tests unitaires	JUnit 5, Mockito, AssertJ	Services de domaine purs (gardes, automates, calculs, tri topologique) sans conteneur
Tests d'intégration	@SpringBootTest, Testcontainers (PostgreSQL 16, Kafka), MockMvc, WireMock	Chaîne REST, persistance, outbox, consommateurs, tiers simulés
Tests Gherkin	Cucumber (CucumberIT généré par JHipster), features en français	Un scénario exécutable par scénario de la fiche du carnet
Tests cURL	cURL via la passerelle avec jeton	Validation fonctionnelle de bout en

Couche	Outil ou cadre	Objectif
	Keycloak d'un utilisateur de test par rôle	bout, refus sans jeton, refus par OPA, cloisonnement
Implémentation frontend	React 18, TypeScript strict, Vite, TanStack Query, react-hook-form et zod, i18next, CSS modules et jetons `--gm-*`	Descripteurs de ressource et composants de domaine ; pages réduites à l'appel des composants universels
Tests frontend	Vitest et Testing Library, Storybook et axe-core, Playwright (BDD et bout en bout)	Quatre états de chaque écran, actions permises, accessibilité WCAG 2.2 AA, parcours complets via la passerelle

Principes qui ne se négocient pas

Non-intrusion

La plateforme lit, rapproche, trace et notifie. Elle n'écrit jamais dans le simulateur de certification ni dans le dispositif de gestion des changements, ne crée aucun jeu de test, ne recalcule aucun verdict. Les ports sortants vers ces tiers ne comportent aucune méthode d'écriture, la politique réseau limite les flux sortants à la lecture, et un test d'architecture interdit toute méthode HTTP autre que GET dans leurs adaptateurs.

Règle des quatre éléments

Une opération n'est close que lorsque sont réunis un responsable, une preuve scellée en WORM, un jeton d'horodatage RFC 3161 et une validation par un tiers distinct du responsable et du déposant. La garde de clôture nomme l'élément manquant ; elle est un service de domaine pur, testé sans conteneur.

Quatre quitus ordonnés

Technique par le chef du DES, puis sécurité par le RSSI et conformité par le responsable conformité en parallèle, puis Direction Générale. Chaque quitus est une décision signée par le coffre de clés après réauthentification FIDO2, immuable, horodatée, portant l'empreinte du socle d'évaluation ; un changement du socle rend caducs les quitus antérieurs.

Outbox et idempotence

Toute émission d'événement s'écrit dans la table `outbox_event` dans la transaction de l'agrégat ; toute consommation passe par `IdempotentConsumer.handleOnce`. Un événement invalide va au rebut (`rejected_record` et sujet `.dlq`), jamais à la poubelle.

Aucune donnée de carte

Aucun dépôt de pièce n'est stocké par MS-HOMOLOGATION ni MS-CERTIFICATION : tout passe par MS-DOCUMENT, qui applique l'analyse antivirale et le contrôle de masquage, et renvoie une référence, une empreinte et l'état scellé. Les journaux des deux services sont couverts par le masquage du socle.

Hypothèses sur le socle, à confirmer à l'étape 0.1

Hypothèse	Conséquence si elle est fausse
Les services `mshomologation` et `mscertification` existent, générés par JHipster avec les paquets `org.gimac.des.homologation` et `org.gimac.des.certification`	Adapter les chemins de paquet ; le plan n'utilise que des chemins relatifs
Les entités du JDL de conception ont été générées ; les noms peuvent différer de ceux de la Phase 0	Le dépôt fait foi pour ce qui existe ; les entités et champs manquants sont ajoutés par les changelogs de la Phase 0
Une bibliothèque commune `gimac-common-messaging` fournit `OutboxEvent`, `OutboxPublisher`, `OutboxRelay`, `IdempotentConsumer`, `CloudEventEnvelope`	Si elle est absente, les classes de l'étape 0.4 sont créées dans chaque service, à l'identique
La passerelle expose déjà `/bff/des/**` et `/bff/portail/**` et relaie le jeton vers les services	Sinon, ajouter les quatre routes de l'étape 0.6
MS-IAM fournit la réauthentification FIDO2 (IAM-07), la recherche de collaborateurs habilités et les délégations ; MS-PARTICIPANT fournit la fiche institution et les services souscrits ; MS-DOCUMENT fournit le dépôt, la vérification d'empreinte et la restitution journalisée	Chaque appel passe par un port avec un adaptateur de substitution activé en profil `dev` tant que le récit fournisseur n'est pas livré
Le jeton de service inter-services, la file de rebut Kafka et le mTLS sont présents dans le socle mais inactifs	Le lot 1 les active pour ses seuls besoins (étape 0.7) sans en modifier la conception

Ordre d'exécution et jalons

Le lot est découpé en trois itérations qui suivent l'ordre des dépendances du carnet. Les récits sont pris dans l'ordre indiqué ; un récit n'est commencé que lorsque ses dépendances internes au lot sont fusionnées et que ses dépendances externes disposent d'un adaptateur de substitution.

Itération	Semaines	Backend	Frontend	Jalon de sortie
Phase 0	Semaine 1	Relevé du socle, JDL, Liquibase, sujets Kafka, politiques OPA, routes, activation du jeton de service et du rebut, structure hexagonale	F0 : descripteurs, composants de domaine, client HTTP, `SignDialog`	Les deux services démarrent avec leur schéma complet ; la chaîne de tests du lot est verte à vide
Itération A	Semaines 2 à 4	HOM-01, HOM-02, HOM-04, HOM-05, HOM-06, HOM-07, HOM-08, HOM-09, HOM-10, HOM-11, HOM-12	DES-12, DES-08, DES-09, DES-10	Une liaison est ouverte, ses opérations instanciées, une preuve déposée et une opération close selon la règle des quatre éléments, de bout en bout par

Itération	Semaines	Backend	Frontend	Jalon de sortie
				l'interface
Itération B	Semaines 5 à 8	HOM-13, HOM-14, HOM-15, HOM-16, HOM-17, HOM-18, HOM-19, HOM-20, HOM-22, HOM-23, HOM-24, CER-01 à CER-05, CER-07 à CER-13	DES-11, POR-03	Une liaison déclarée apte parcourt le circuit des quatre quitus, le dossier de mise en service atteint l'état attesté, l'attestation est téléchargée depuis le portail
Itération C	Semaines 9 à 10	HOM-03, HOM-21, CER-06, CER-14, CER-15	Compléments DES-11 (registre, demandes de statut)	Migration de référentiel, échéances, caducité, registre et révocation ; recette du lot

Capacité de référence : 110 points par sprint de deux semaines pour l'équipe complète. Le lot représente 118 points pour MS-HOMOLOGATION, 76 points pour MS-CERTIFICATION et environ 60 points d'interface, soit 254 points, ce qui correspond à cinq sprints de deux semaines si l'itération A et l'itération B sont menées en parallèle par deux binômes (backend et frontend). Un calendrier indicatif est proposé au chapitre de récapitulation ; il est ajusté à la revue de fin de Phase 0.

Utilisateurs de test et jeu de données

Les utilisateurs de test sont désignés par leur rôle et créés dans le domaine Keycloak `gimac` par le script du socle : `test-ingenieur-homologation`, `test-chef-des`, `test-rssi`, `test-conformite`, `test-dg`, `test-correspondant-a` (institution « Établissement de paiement A ») et `test-correspondant-b` (institution « Banque B »). Le jeu de données de recette comprend deux versions publiées du référentiel (carte interbancaire et paiement mobile), une liaison par type de projet, un simulateur de substitution qui expose trois campagnes dont une contient un scénario inconnu, un dispositif de gestion des changements de substitution avec une fiche approuvée et une fiche en attente, et une autorité d'horodatage de substitution conforme RFC 3161. Aucune donnée réelle d'institution, de personne ou d'infrastructure n'entre dans ce jeu.

PHASE 0 | Préparation du socle pour le lot 1

Objectif

Vérifier l'état du socle, aligner le modèle de données des deux services sur le JDL de référence, créer les sujets Kafka, les permissions et les politiques du lot, ouvrir les routes de la passerelle, activer le jeton de service et la file de rebut, greffer l'arborescence hexagonale sur les squelettes, puis compiler et exécuter les deux services à vide avant toute ligne de code métier. La Phase 0 dure une semaine et se termine par une revue où les hypothèses de la convention transversale sont confirmées ou corrigées.

0.1. Relevé du socle

On démarre l'environnement de développement, on rejoue les contrôles du socle et l'on relève les noms réels des paquets, des entités, des routes et des sujets afin de confirmer les hypothèses de la convention transversale.

```
# Démarrage de l'environnement et rejeu des 48 contrôles du socle
cd infra/docker-compose && ./gimac-dev.sh demarrer && ./tester-socle.sh

# Relevé des paquets de base et des entités déjà générées dans les deux services
for s in mshomologation mscertification; do
  echo "== $s"
  find services/$s/src/main/java -maxdepth 4 -type d -name domain
  ls services/$s/src/main/java/org/gimac/des/*/domain/ 2>/dev/null
  ls services/$s/src/main/resources/config/liquibase/changelog/
done

# Relevé des routes de la passerelle et des sujets Kafka existants
grep -n "uri:\|Path=" services/gateway/src/main/resources/config/application*.yml
docker compose exec kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server
kafka:9092 --list
```

Règle du relevé

Ce qui existe dans le dépôt fait foi. Un nom d'entité, de champ ou de route qui diffère du présent plan est reporté dans un tableau de correspondance placé en tête du dépôt du lot (`docs/lot1/correspondance.md`) ; le plan n'est pas réécrit, le tableau fait le lien.

0.2. Modèle de données de référence (JDL)

Le JDL ci-dessous est la référence du lot. Il est comparé au JDL du socle ; les entités et champs manquants sont ajoutés par régénération incrémentale (`jhipster jdl lot1-homologation.jdl --skip-fake-data --force` sur les seules entités nouvelles) ou, pour un champ isolé, par un changelog Liquibase manuel. Les types JSON sont déclarés `TextBlob` dans le JDL puis convertis en `jsonb` à l'étape 0.3.

```
// jdl/lot1-homologation.jdl
application { config { baseName mshomologation, applicationType microservice,
packageName org.gimac.des.homologation,
  databaseType sql, prodDatabaseType postgresql, devDatabaseType postgresql,
authenticationType oauth2,
  serverPort 8087, messageBroker kafka, cacheProvider no, enableHibernateCache false,
skipClient true } entities * }
```

```
enum TypeProjet { CARTE_INTERBANCAIRE, PAIEMENT_MOBILE, DELEGATAIRE }
enum EtatVersionReferentiel { BROUILLON, PUBLIEE, RETIREE }
enum Segment { TRONC_COMMUN_AMONT, BRANCHE, TRONC_COMMUN_AVAL }
enum Partie { DES, PARTICIPANT }
enum NatureCouverture { CERTIFICATION, MISE_EN_SERVICE, AUCUNE }
enum EtatLiaison { OUVERTE, EN_HOMOLOGATION, GELEE, APTE_CERTIFICATION,
MISE_EN_SERVICE_AUTORISEE, CERTIFIEE, CLOSE }
enum EtatScopeItem { A_ENGAGER, EN_COURS, COUVERT, HORS_PERIMETRE }
enum EtatOperation { VERROUILLEE, ELIGIBLE, PREUVE_DEPOSEE,
VALIDEE_HORODATAGE_EN_ATTENTE, CLOSE, ROUVERTE, HORS_PERIMETRE }
enum Verdict { FAVORABLE, DEFAVORABLE, NON_EXECUTE }
enum QualificationScenario { A_QUALIFIER, A_DECLARER_AU_REFERENTIEL, HORS_PARCOURS }
enum EtatDisjoncteur { FERME, OUVERT, DEMI_OUVERT }
enum EtatMigration { SIMULEE, APPROUVEE, EN_COURS, TERMINEE, INTERROMPUE }

entity ReferentialVersion { typeProjet TypeProjet required, numero Integer required,
libelle String required maxlength(200),
  etat EtatVersionReferentiel required, dateCreation Instant required,
datePublication Instant,
  ordreTopologique TextBlob, versionSourceId UUID }
entity ReferenceOperation { code String required maxlength(40), libelle String
required maxlength(200), phase String required maxlength(60),
  segment Segment required, partie Partie required, serviceCode String maxlength(40),
natureCouverture NatureCouverture required,
  preuveAttendue TextBlob, idScenario String maxlength(80), predecesseurs TextBlob,
fonctionResponsable String maxlength(60), grille TextBlob }
entity Liaison { typeProjet TypeProjet required, institutionId UUID required,
institutionNom String maxlength(200), etat EtatLiaison required,
  chefDeProjetId UUID required, banquePartenaireId UUID, dateOuverture Instant
required, motifEtat String maxlength(500),
  cycleAptitude Integer required, ordreInstanciation TextBlob, etatAvantGel
TextBlob }
entity ContractualScope { dateDerivation Instant required, sourceEventId String
maxlength(80) }
entity ScopeItem { serviceCode String required maxlength(40), serviceLibelle String
maxlength(200), etat EtatScopeItem required }
entity OperationInstance { code String required maxlength(40), libelle String
maxlength(200), segment Segment, partie Partie,
  serviceCode String maxlength(40), etat EtatOperation required, degreEntrant Integer
required, echeance LocalDate, dateEtat Instant, grilleReponses TextBlob }
entity Assignment { responsableId UUID required, responsableNom String
maxlength(200), partie Partie required, motif String maxlength(500),
  dateAttribution Instant required, actif Boolean required }
entity EvidenceReference { documentRef String required maxlength(120), empreinte
String required maxlength(64), version Integer required,
  typePiece String required maxlength(60), deposantId UUID required, dateDepot
Instant required, scelle Boolean required, valide Boolean }
entity ClosureRecord { validateurId UUID required, dateValidation Instant required,
empreinteCloture String required maxlength(64),
  jetonHorodatage TextBlob, dateHorodatage Instant, actif Boolean required,
motifReouverture String maxlength(500), dateReouverture Instant }
entity CampaignResult { campagneId String required maxlength(80), executionId String
required maxlength(80), idScenario String required maxlength(80),
  verdict Verdict required, dateExecution Instant required, versionScenario String
maxlength(40), referenceTrace String maxlength(120),
  empreinteReponse String required maxlength(64), dateReprise Instant required }
entity UnmatchedScenario { idScenario String required maxlength(80), campagneId
String maxlength(80), qualification QualificationScenario required,
  motif String maxlength(500), dateDetection Instant required }
entity Coverage { serviceCode String required maxlength(40), nature NatureCouverture
required, numerateur Integer, denominateur Integer,
  taux BigDecimal, calculable Boolean required, motifNonCalculable String
maxlength(200), dateCalcul Instant required }
entity EvaluationBase { empreinte String required maxlength(64), cycle Integer
required, dateCalcul Instant required, composition TextBlob }
entity EvidenceFileExport { manifeste TextBlob, empreinteManifeste String
maxlength(64), documentRef String maxlength(120), dateExport Instant,
  demandeurId UUID, socleEmpreinte String maxlength(64), etat String maxlength(30) }
```

```
entity ReplayCursor { campagneId String maxlength(80), liaisonId UUID, position
String maxlength(120), dateDernierSucces Instant,
  echecsConsecutifs Integer, etatDisjoncteur EtatDisjoncteur }
entity RejectedRecord { source String maxlength(60), chargeUtile TextBlob, empreinte
String maxlength(64), motif String maxlength(500),
  dateRejet Instant, relance Boolean }
entity ReferentialMigration { versionSourceId UUID, versionCibleId UUID, etat
EtatMigration, rapport TextBlob, dateDemande Instant,
  decideurId UUID, liaisonsRetenues TextBlob }
entity DeadlineNotification { operationId UUID required, seuil String required
maxlength(30), dateEnvoi Instant required }

relationship ManyToOne { ReferenceOperation{version required} to ReferentialVersion,
Liaison{version required} to ReferentialVersion,
  ScopeItem{scope required} to ContractualScope, OperationInstance{liaison required}
to Liaison,
  OperationInstance{referenceOperation required} to ReferenceOperation,
Assignment{operation required} to OperationInstance,
  EvidenceReference{operation required} to OperationInstance, ClosureRecord{operation
required} to OperationInstance,
  CampaignResult{liaison required} to Liaison, CampaignResult{operation} to
OperationInstance, UnmatchedScenario{liaison required} to Liaison,
  Coverage{liaison required} to Liaison, EvaluationBase{liaison required} to Liaison,
EvidenceFileExport{liaison required} to Liaison }
relationship OneToOne { ContractualScope{liaison required} to Liaison }
dto * with mapstruct
service * with serviceImpl
paginate Liaison, OperationInstance, CampaignResult, UnmatchedScenario with
pagination
```

```
// jdl/lot1-certification.jdl
application { config { baseName mscertification, applicationType microservice,
packageName org.gimac.des.certification,
  databaseType sql, prodDatabaseType postgresql, devDatabaseType postgresql,
authenticationType oauth2,
  serverPort 8088, messageBroker kafka, cacheProvider no, enableHibernateCache false,
skipClient true } entities * }

enum NatureQuitus { TECHNIQUE, SECURITE, CONFORMITE, DIRECTION_GENERALE }
enum EtatQuitus { ATTENDU, SIGNE_FAVORABLE, SIGNE_DEFAVORABLE, CADUC, ANNULE,
NON_REQUIS }
enum EtatCircuit { OUVERT, INTERROMPU, ACHEVE }
enum EtatDossier { OUVERT, AUTORISE, MIS_EN_SERVICE, COMPLET, ATTESTE }
enum ConditionCode { COUVERTURE_CERTIFICATION, COUVERTURE_MISE_EN_SERVICE,
QUATRE_QUITUS, FICHE_CHANGEMENT_APPROUVEE, PROCES_VERBAL, EMPREINTES_VERIFIEES }
enum EtatAttestation { EN_VIGUEUR, SUSPENDUE, REVOQUEE, REMPLACEE }
enum EtatScellement { EN_ATTENTE, SCELLE, REJETE }
enum TypeDemandeStatut { REVOCATION, SUSPENSION, LEVEE }

entity CertificationCircuit { liaisonId UUID required, institutionId UUID, cycle
Integer required, socleEmpreinte String required maxlength(64),
  etat EtatCircuit required, dateOuverture Instant, dateFin Instant }
entity Quitus { nature NatureQuitus required, etat EtatQuitus required, dateAttendu
Instant, motifCaducite String maxlength(500) }
entity SignedDecision { decisionFavorable Boolean required, motif String required
maxlength(1000), signataireId UUID required,
  signataireRole String maxlength(60), dateSignature Instant required, socleEmpreinte
String required maxlength(64),
  empreinteDecision String required maxlength(64), signature TextBlob required,
operationsDesignees TextBlob, servicesDesignes TextBlob,
  referenceAudit String maxlength(120) }
entity GoLiveDossier { liaisonId UUID required, institutionId UUID, etat EtatDossier
required, dateOuverture Instant, dateAutorisation Instant,
  dateMiseEnService Instant, dateCompletude Instant, etatScellement EtatScellement,
referenceFicheChangement String maxlength(120) }
entity CompletenessCondition { code ConditionCode required, satisfaite Boolean
required, source String maxlength(60), detail String maxlength(500),
  dateVerification Instant }
entity ChangeRecordSnapshot { reference String required maxlength(120),
```

```
etatApprobation String required maxlength(40), fenetreDebut Instant,
  fenetreFin Instant, dateLecture Instant required, empreinteReponse String
maxlength(64) }
entity GoLiveReport { documentRef String required maxlength(120), empreinte String
required maxlength(64), heureBascule Instant required,
  dansFenetre Boolean required, deposantId UUID, dateDepot Instant, regularise
Boolean }
entity TimestampToken { objetType String required maxlength(20), objetId UUID
required, empreinte String required maxlength(64),
  nonce String required maxlength(64), jeton TextBlob required, dateJeton Instant
required, verifie Boolean required, autoriteChaine String maxlength(500) }
entity Attestation { numero String required unique maxlength(30), institutionId UUID,
typeProjet String maxlength(30), servicesCertifies TextBlob,
  dateEmission Instant, socleEmpreinte String maxlength(64), documentRef String
maxlength(120), empreinteDocument String maxlength(64),
  etat EtatAttestation, referenceAudit String maxlength(120) }
entity CertificationRegisterEntry { numeroAttestation String required maxlength(30),
institutionId UUID required, typeProjet String maxlength(30),
  services TextBlob, dateEmission Instant, echeanceRenouvellement LocalDate, etat
EtatAttestation, precedenteId UUID, dernierSeuilNotifie String maxlength(30) }
entity AttestationStatusDecision { typeDemande TypeDemandeStatut required, motif
String required maxlength(1000), pieceRef String maxlength(120),
  demandeurId UUID required, approbateurId UUID, dateDemande Instant required,
dateApprobation Instant, etat String maxlength(30),
  avisDocumentRef String maxlength(120), conditionLevee String maxlength(500) }

relationship ManyToOne { Quitus{circuit required} to CertificationCircuit,
CompletenessCondition{dossier required} to GoLiveDossier,
  ChangeRecordSnapshot{dossier required} to GoLiveDossier,
AttestationStatusDecision{attestation required} to Attestation }
relationship OneToOne { SignedDecision{quitus required} to Quitus,
GoLiveDossier{circuit required} to CertificationCircuit,
  GoLiveReport{dossier required} to GoLiveDossier, Attestation{dossier required} to
GoLiveDossier }
dto * with mapstruct
service * with serviceImpl
paginate CertificationRegisterEntry, Attestation with pagination
```

0.3. Corrections Liquibase après génération

Trois familles de corrections sont appliquées dans un changelog dédié par service, car JHipster génère des colonnes `TEXT` pour les blobs de texte, aucun index métier et aucune contrainte d'unicité composite.

```
<!--
mshomologation/src/main/resources/config/liquibase/changelog/20261005000001_lot1_post
_generation.xml -->
<databaseChangeLog xmlns="http://www.liquibase.org/xml/ns/dbchangelog"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.liquibase.org/xml/ns/dbchangelog
http://www.liquibase.org/xml/ns/dbchangelog/dbchangelog-latest.xsd">

  <!-- 1. Colonnes JSON : TEXT vers jsonb (interrogeables, indexables) -->
  <changeSet id="lot1-jsonb" author="lot1">
    <sql>ALTER TABLE referential_version ALTER COLUMN ordre_topologique TYPE jsonb
USING ordre_topologique::jsonb;
         ALTER TABLE reference_operation ALTER COLUMN predecesseurs TYPE jsonb USING
predecesseurs::jsonb;
         ALTER TABLE reference_operation ALTER COLUMN preuve_attendue TYPE jsonb
USING preuve_attendue::jsonb;
         ALTER TABLE reference_operation ALTER COLUMN grille TYPE jsonb USING
grille::jsonb;
         ALTER TABLE liaison ALTER COLUMN ordre_instanciation TYPE jsonb USING
ordre_instanciation::jsonb;
         ALTER TABLE liaison ALTER COLUMN etat_avant_gel TYPE jsonb USING
etat_avant_gel::jsonb;
```

```
        ALTER TABLE operation_instance ALTER COLUMN grille_reponses TYPE jsonb USING
grille_reponses::jsonb;
        ALTER TABLE evaluation_base ALTER COLUMN composition TYPE jsonb USING
composition::jsonb;
        ALTER TABLE evidence_file_export ALTER COLUMN manifeste TYPE jsonb USING
manifeste::jsonb;
        ALTER TABLE referential_migration ALTER COLUMN rapport TYPE jsonb USING
rapport::jsonb;
        ALTER TABLE referential_migration ALTER COLUMN liaisons_retenues TYPE jsonb
USING liaisons_retenues::jsonb;</sql>
  </changeSet>

  <!-- 2. Unicités métier -->
  <changeSet id="lot1-unicites" author="lot1">
    <addUniqueConstraint tableName="referential_version" columnNames="type_projet,
numero" constraintName="uk_version_type_numero"/>
    <addUniqueConstraint tableName="reference_operation" columnNames="version_id,
code" constraintName="uk_refop_version_code"/>
    <addUniqueConstraint tableName="operation_instance" columnNames="liaison_id,
code" constraintName="uk_opinst_liaison_code"/>
    <addUniqueConstraint tableName="campaign_result" columnNames="liaison_id,
campagne_id, execution_id" constraintName="uk_result_campagne_execution"/>
    <addUniqueConstraint tableName="coverage" columnNames="liaison_id, service_code,
nature" constraintName="uk_coverage_service_nature"/>
    <addUniqueConstraint tableName="deadline_notification" columnNames="operation_id,
seuil" constraintName="uk_deadline_operation_seuil"/>
    <sql>CREATE UNIQUE INDEX uk_liaison_active ON liaison (institution_id,
type_projet)
        WHERE etat NOT IN ('CLOSE');</sql>
  </changeSet>

  <!-- 3. Index de lecture (tableau de bord, graphe, reprise) -->
  <changeSet id="lot1-index" author="lot1">
    <createIndex tableName="operation_instance"
indexName="idx_opinst_liaison_etat"><column name="liaison_id"/><column
name="etat"/></createIndex>
    <createIndex tableName="operation_instance"
indexName="idx_opinst_echeance"><column name="echeance"/></createIndex>
    <createIndex tableName="campaign_result" indexName="idx_result_scenario"><column
name="liaison_id"/><column name="id_scenario"/><column
name="date_execution"/></createIndex>
    <createIndex tableName="assignment" indexName="idx_assignment_actif"><column
name="operation_id"/><column name="actif"/></createIndex>
    <sql>CREATE INDEX idx_refop_predecesseurs ON reference_operation USING gin
(predecesseurs);</sql>
  </changeSet>

  <!-- 4. Immuabilité des enregistrements de clôture actifs et des versions publiées
(défense en profondeur) -->
  <changeSet id="lot1-immuabilite" author="lot1">
    <sql splitStatements="false">
      CREATE OR REPLACE FUNCTION refuse_update_published() RETURNS trigger AS $$
      BEGIN IF OLD.etat = 'PUBLIEE' AND NEW.etat = 'PUBLIEE' THEN
        RAISE EXCEPTION 'version publiee immuable (%)', OLD.id; END IF; RETURN NEW;
END; $$ LANGUAGE plpgsql;
      CREATE TRIGGER trg_version_immuable BEFORE UPDATE ON referential_version
        FOR EACH ROW EXECUTE FUNCTION refuse_update_published();
    </sql>
  </changeSet>
</databaseChangeLog>
```

Le changelog homologue de `mscertification` convertit en `jsonb` les colonnes `operations_designees`, `services_designes`, `services_certifies`, `services`, ajoute les unicités `(circuit_id, nature)` sur `quitus`, `(liaison_id, cycle)` sur `certification_circuit`, `(objet_type, objet_id)` sur `timestamp_token`, la séquence `attestation_numero_seq` et un déclencheur qui refuse toute mise à jour d'une ligne de `signed_decision`.

Sur les entités Java, les colonnes converties reçoivent l'annotation `@JdbcTypeCode(SqlTypes.JSON) @Column(columnDefinition = "jsonb")`.

0.4. Classes communes de messagerie

Les classes ci-dessous sont fournies par la bibliothèque commune du socle ; si le relevé montre qu'elles en sont absentes, elles sont créées à l'identique dans `infrastructure/messaging` de chaque service. Tous les fragments du plan les appellent sans les réécrire.

```
// infrastructure/messaging/outbox/OutboxEvent.java
@Entity @Table(name = "outbox_event")
public class OutboxEvent {
    public enum Status { PENDING, SENT, FAILED }
    @Id private UUID id = UUID.randomUUID();
    @Column(nullable = false) private String aggregateType;
    @Column(nullable = false) private String aggregateId;
    @Column(nullable = false) private String eventType;      // ex.
homologation.operation-closed.v1
    @JdbcTypeCode(SqlTypes.JSON) @Column(columnDefinition = "jsonb", nullable =
false) private String payload;
    @Column(nullable = false) private Instant createdAt = Instant.now();
    @Enumerated(EnumType.STRING) @Column(nullable = false) private Status status =
Status.PENDING;
    private Instant sentAt; private int attempts;
    private String correlationId; private String institutionId;
    // accesseurs omis
}
```

```
// infrastructure/messaging/outbox/OutboxPublisher.java
@Component
public class OutboxPublisher {
    private final OutboxEventRepository repo; private final ObjectMapper mapper;
private final CorrelationContext ctx;
    public OutboxPublisher(OutboxEventRepository repo, ObjectMapper mapper,
CorrelationContext ctx) { this.repo = repo; this.mapper = mapper; this.ctx = ctx; }

    /** À appeler dans la transaction de l'agrégat : l'événement part avec elle ou
pas du tout. */
    @Transactional(propagation = Propagation.MANDATORY)
    public UUID publish(String aggregateType, UUID aggregateId, String eventType,
Object payload) {
        OutboxEvent e = new OutboxEvent();
        e.setAggregateType(aggregateType); e.setAggregateId(aggregateId.toString());
e.setEventType(eventType);
        try { e.setPayload(mapper.writeValueAsString(payload)); } catch
(JsonProcessingException ex) { throw new IllegalArgumentException("charge utile non
serialisable", ex); }
        e.setCorrelationId(ctx.correlationId());
e.setInstitutionId(ctx.institutionId().orElse(null));
        return repo.save(e).getId();
    }
}
```

```
// infrastructure/messaging/outbox/OutboxRelay.java
@Component
public class OutboxRelay {
    private static final int BATCH = 100;
    private final OutboxEventRepository repo; private final KafkaTemplate<String,
String> kafka;
    private final ObjectMapper mapper; private final TopicResolver topics; private
final String source;

    public OutboxRelay(OutboxEventRepository repo, KafkaTemplate<String, String>
```

```
kafka, ObjectMapper mapper,
                       TopicResolver topics, @Value("${spring.application.name}")
String app) {
        this.repo = repo; this.kafka = kafka; this.mapper = mapper; this.topics =
topics; this.source = "/gimac/des/" + app;
    }

    /** Relais toutes les 500 ms ; FOR UPDATE SKIP LOCKED permet plusieurs réplicas
sans doublon. */
    @Scheduled(fixedDelayString = "${gimac.outbox.delay:500}")
    @Transactional
    public void relay() {
        for (OutboxEvent e : repo.lockPendingBatch(BATCH)) {
            try {
                String envelope = mapper.writeValueAsString(CloudEventEnvelope.of(e,
source));
                kafka.send(topics.topicFor(e.getEventType()), e.getAggregateId(),
envelope).get(5, TimeUnit.SECONDS);
                e.setStatus(OutboxEvent.Status.SENT); e.setSentAt(Instant.now());
            } catch (Exception ex) {
                e.setAttempts(e.getAttempts() + 1);
                if (e.getAttempts() >= 10) e.setStatus(OutboxEvent.Status.FAILED); //
alerte MS-MONITORING par métrique
            }
        }
    }
}
```

```
// infrastructure/messaging/outbox/OutboxEventRepository.java
public interface OutboxEventRepository extends JpaRepository<OutboxEvent, UUID> {
    @Query(value = "SELECT * FROM outbox_event WHERE status = 'PENDING' ORDER BY
created_at LIMIT :n FOR UPDATE SKIP LOCKED", nativeQuery = true)
    List<OutboxEvent> lockPendingBatch(@Param("n") int n);
}
```

```
// infrastructure/messaging/CloudEventEnvelope.java
public record CloudEventEnvelope(String specversion, String id, String source, String
type, String subject, String time,
                                 String datacontenttype, String correlationid, String
institutionid, JsonNode data) {
    public static CloudEventEnvelope of(OutboxEvent e, String source) {
        return new CloudEventEnvelope("1.0", e.getId().toString(), source,
e.getEventType(), e.getAggregateId(),
            e.getCreatedAt().toString(), "application/json", e.getCorrelationId(),
e.getInstitutionId(), Json.tree(e.getPayload()));
    }
}
```

```
// infrastructure/messaging/consumer/IdempotentConsumer.java
@Component
public class IdempotentConsumer {
    private final ProcessedEventRepository processed;
    public IdempotentConsumer(ProcessedEventRepository processed) { this.processed =
processed; }

    /** Exécute le traitement une seule fois par (eventId, groupe) ; un doublon est
acquitté sans effet. */
    @Transactional
    public boolean handleOnce(String eventId, String consumerGroup, Runnable handler)
{
        if (processed.existsByEventIdAndConsumerGroup(eventId, consumerGroup)) return
false;
        handler.run();
        processed.save(new ProcessedEvent(eventId, consumerGroup, Instant.now()));
        return true;
    }
}
```

```
// infrastructure/messaging/TopicResolver.java
@Component
public class TopicResolver {
    /** homologation.operation-closed.v1 vers homologation.operation ;
certification.quitus-signed.v1 vers certification.quitus */
    public String topicFor(String eventType) {
        String[] parts = eventType.split("\\.");
        String aggregate = parts[1].substring(0, parts[1].indexOf('-'));
        return parts[0] + "." + aggregate;
    }
}
```

Tables associées, à ajouter au changelog de l'étape 0.3 si elles n'existent pas : `outbox_event` (index partiel `WHERE status = 'PENDING'` sur `created_at`), `processed_event` (clé unique `event_id, consumer_group`).

0.5. Sujets Kafka, schémas et rebut

```
# infra/kafka/creer-sujets-lot1.sh : sujets d'agrégat, 6 partitions, rétention 30
jours, plus le sujet de rebut de chaque sujet consommé
for t in homologation.referential homologation.liaison homologation.operation
homologation.campaign homologation.coverage \
         homologation.notification certification.circuit certification.quitus
certification.dossier certification.attestation \
         certification.notification; do
  kafka-topics.sh --bootstrap-server "${KAFKA_BOOTSTRAP}" --create --if-not-exists --
topic "$t" \
    --partitions 6 --replication-factor 3 --config retention.ms=2592000000 --config
min.insync.replicas=2
done
for t in participant.subscription participant.status document.document iam.user
iam.role homologation.liaison \
         homologation.operation homologation.coverage certification.quitus
certification.dossier certification.attestation; do
  kafka-topics.sh --bootstrap-server "${KAFKA_BOOTSTRAP}" --create --if-not-exists --
topic "$t.dlq" --partitions 3 --replication-factor 3
done
```

Les schémas JSON des charges utiles sont versionnés dans `contracts/events/<type>.schema.json` et vérifiés par un test de contrat dans chaque service producteur et consommateur (bibliothèque `networknt/json-schema-validator`). Le tableau des événements du lot figure au chapitre de cartographie.

0.6. Permissions, politiques OPA et routes de la passerelle

Les permissions du lot sont déclarées dans MS-IAM (référentiel des permissions par rôle) et vérifiées deux fois : par OPA à la passerelle, par `@PreAuthorize` dans le service. Extrait de la politique du lot, à ajouter au paquet `gimac.authz` du socle :

```
# infra/opa/policies/lot1_homologation.rego
package gimac.authz

import future.keywords.if
import future.keywords.in

default allow := false

# Correspondance chemin et méthode vers permission requise (extrait)
required_permission := p if {
    some rule in [
        {"m": "POST", "re": "^/bff/des/homologation/referentiels/[^/]+/publication$",
```

```
"p": "HOM_REFERENTIEL_PUBLISH"},
        {"m": "POST", "re": "^/bff/des/homologation/liaisons$",
"p": "HOM_LIAISON_OPEN"},
        {"m": "POST", "re": "^/bff/des/homologation/operations/[^/]+/validation$",
"p": "HOM_OPERATION_VALIDATE"},
        {"m": "POST", "re": "^/bff/des/homologation/operations/[^/]+/reouverture$",
"p": "HOM_OPERATION_REOPEN"},
        {"m": "POST", "re":
"^/bff/des/certification/circuits/[^/]+/quitus/technique$",           "p":
"CER_QUITUS_TECHNIQUE_SIGN"},
        {"m": "POST", "re":
"^/bff/des/certification/circuits/[^/]+/quitus/securite$",            "p":
"CER_QUITUS_SECURITE_SIGN"},
        {"m": "POST", "re":
"^/bff/des/certification/circuits/[^/]+/quitus/conformite$",          "p":
"CER_QUITUS_CONFORMITE_SIGN"},
        {"m": "POST", "re": "^/bff/des/certification/circuits/[^/]+/quitus/direction-
generale$",  "p": "CER_QUITUS_DG_SIGN"},
        {"m": "GET",  "re": "^/bff/portail/homologation/",   "p": "HOM_PORTAL_READ"},
        {"m": "POST", "re": "^/bff/portail/homologation/operations/[^/]+/preuves$",
"p": "HOM_PORTAL_SUBMIT"},
        {"m": "GET",  "re": "^/bff/portail/certification/",  "p": "CER_PORTAL_READ"},
    ]
    rule.m == input.method
    regex.match(rule.re, input.path)
    p := rule.p
}

allow if {
    required_permission in input.permissions
}

# Le portail exige de plus la présence d'une institution dans le jeton
allow if {
    startswith(input.path, "/bff/portail/")
    required_permission in input.permissions
    input.institution_id != ""
}
```

Routes de la passerelle (Spring Cloud Gateway) à ajouter aux treize routes existantes :

```
# gateway/src/main/resources/config/application-lot1.yml
spring.cloud.gateway.routes:
  - id: des-homologation
    uri: lb://mshomologation
    predicates: [ "Path=/bff/des/homologation/**" ]
    filters: [ "RewritePath=/bff/des/homologation/(?<p>.*), /api/v1/${p}",
"TokenRelay=", "OpaAuthorize=" ]
  - id: portail-homologation
    uri: lb://mshomologation
    predicates: [ "Path=/bff/portail/homologation/**" ]
    filters: [ "RewritePath=/bff/portail/homologation/(?<p>.*), /api/v1/portail/$
{p}", "TokenRelay=", "OpaAuthorize=" ]
  - id: des-certification
    uri: lb://mscertification
    predicates: [ "Path=/bff/des/certification/**" ]
    filters: [ "RewritePath=/bff/des/certification/(?<p>.*), /api/v1/${p}",
"TokenRelay=", "OpaAuthorize=" ]
  - id: portail-certification
    uri: lb://mscertification
    predicates: [ "Path=/bff/portail/certification/**" ]
    filters: [ "RewritePath=/bff/portail/certification/(?<p>.*), /api/v1/portail/$
{p}", "TokenRelay=", "OpaAuthorize=" ]
```

Dans chaque service, la configuration de sécurité générée est complétée pour exiger la permission `INTERNAL_<SERVICE>_READ` sur les routes internes et refuser toute route du portail sans claim `institution_id` :

```
// config/SecurityConfiguration.java (complément)
http.authorizeHttpRequests(a -> a
    .requestMatchers("/api/v1/liaisons/*/socle-evaluation",
"/api/v1/liaisons/*/intervenants",

"/api/v1/liaisons/*/dossier-de-preuves/interne/**").hasAuthority("INTERNAL_HOMOLOGATI
ON_READ")
    .requestMatchers("/api/v1/portail/**").access(new
InstitutionClaimPresentAuthorizationManager())
    .requestMatchers("/api/v1/**").authenticated()
    .requestMatchers("/management/health/**", "/management/prometheus").permitAll());
```

0.7. Activation des composants inactifs nécessaires au lot

Composant du socle	Usage dans le lot 1	Activation
Jeton de service (client Keycloak par service, `client_credentials`)	Appels internes MS-CERTIFICATION vers MS-HOMOLOGATION (socle d'évaluation, intervenants, export du dossier de preuves), MS-HOMOLOGATION vers MS-PARTICIPANT et MS-DOCUMENT	Créer les clients `mshomologation-service` et `mscertification-service` dans le domaine `gimac`, leur attribuer les permissions `INTERNAL_*_READ`, configurer `spring.security.oauth2.client.registration.service-token` dans chaque service, secrets fournis par le coffre du cluster
File de rebut Kafka	Événements invalides des consommateurs du lot (schéma, empreinte divergente)	Activer `DeadLetterPublishingRecoverer` avec `DefaultErrorHandler` (trois tentatives, retard 1 s, 5 s, 25 s) vers `<sujet>.dlq` ; écran de consultation dans la console d'administration hors lot
mTLS sortant	Lecture du simulateur et du dispositif de gestion des changements (adaptateurs réels)	Monter le magasin de certificats client par secret Kubernetes ; le profil `dev` utilise les adaptateurs de substitution sans mTLS
Jeton fantôme, administration Keycloak	Non requis par le lot 1	Restent inactifs ; récits GW et IAM correspondants

```
// infrastructure/client/ServiceTokenInterceptor.java : ajoute le jeton de service
aux appels internes
@Component
public class ServiceTokenInterceptor implements ClientHttpRequestInterceptor {
    private final OAuth2AuthorizedClientManager manager;
    public ServiceTokenInterceptor(OAuth2AuthorizedClientManager manager)
{ this.manager = manager; }
    @Override public ClientHttpResponse intercept(HttpRequest req, byte[] body,
ClientHttpRequestExecution exec) throws IOException {
        OAuth2AuthorizedClient client =
manager.authorize(OAuth2AuthorizeRequest.withClientRegistrationId("service-
token").principal("service").build());
        req.getHeaders().setBearerAuth(client.getAccessToken().getTokenValue());
        req.getHeaders().set("X-Correlation-Id", CorrelationContext.current());
        return exec.execute(req, body);
    }
}
```

```
// config/KafkaErrorHandlingConfiguration.java : file de rebut avec réessais bornés
@Configuration
```

```
public class KafkaErrorHandlingConfiguration {
    @Bean
    public DefaultErrorHandler kafkaErrorHandler(KafkaTemplate<String, String>
template) {
        DeadLetterPublishingRecoverer recoverer = new
DeadLetterPublishingRecoverer(template,
            (record, ex) -> new TopicPartition(record.topic() + ".dlq",
record.partition() % 3));
        return new DefaultErrorHandler(recoverer, new
ExponentialBackOffWithMaxRetries(3) {{ setInitialInterval(1000);
setMultiplier(5); }});
    }
}
```

0.8. Arborescence hexagonale greffée sur le squelette généré

```
mshomologation/src/main/java/org/gimac/des/homologation/
├── config/                          # généré JHipster (sécurité, Kafka, Liquibase) +
compléments 0.6 et 0.7
├── domain/
│   ├── model/                       # entités JPA générées (ReferentialVersion,
Liaison, OperationInstance, ...)
│   ├── enumeration/                 # énumérations JDL
│   ├── service/                     # TopologicalSorter, ClosureGuard,
CoverageCalculator, EvaluationBaseHasher, ...
│   └── event/                       # records des charges utiles d'événements
├── application/
│   ├── port/in/                     # PublishReferentialUseCase, OpenLiaisonUseCase,
CloseOperationUseCase, ...
│   ├── port/out/                    # ParticipantReaderPort, DocumentDepositPort,
TimestampAuthorityPort, SimulatorReaderPort, ReauthenticationPort
│   └── usecase/                     # implémentations transactionnelles
├── infrastructure/
│   ├── persistence/                 # repositories Spring Data générés +
spécifications et projections
│   ├── messaging/outbox/            # 0.4
│   ├── messaging/consumer/          # consommateurs idempotents
│   ├── client/                      # adaptateurs réels (*Adapter) et de
substitution (*StubAdapter)
│   ├── scheduling/                  # reprise du simulateur, échéances, relais
d'outbox
│   └── security/                    # CorrelationContext, extraction des claims,
InstitutionClaimPresentAuthorizationManager
└── web/rest/                        # ressources REST générées et ajoutées, DTO,
mappers, ProblemDetail
```

Sélection des adaptateurs par propriété, un port à la fois :

```
# src/main/resources/config/application-dev.yml (extrait)
gimac:
  adapters:
    participant-reader: stub      # real | stub
    document-deposit: real        # MS-DOCUMENT est livré dans le socle
    reauthentication: stub        # IAM-07 : passe à real dès livraison
    simulator-reader: stub
    timestamp-authority: stub
    key-vault-signing: stub
  homologation:
    replay: { period: PT5M, failure-threshold: 5, half-open-after: PT2M }
    deadlines: { warning-days: 5, overdue-repeat-days: 7 }
  certification:
    change-record: { freshness: PT15M }
    renewal: { notice-days: 90 }
```

```
// config/AdaptersConfiguration.java : une fabrique par port
```

```
@Configuration
public class AdaptersConfiguration {
    @Bean @ConditionalOnProperty(name = "gimac.adapters.participant-reader",
havingValue = "real")
    ParticipantReaderPort participantReaderReal(RestClient.Builder b,
ServiceTokenInterceptor t,
                                                @Value("$
{gimac.services.participant.url}") String url) {
        return new
ParticipantReaderAdapter(b.baseUrl(url).requestInterceptor(t).build());
    }
    @Bean @ConditionalOnProperty(name = "gimac.adapters.participant-reader",
havingValue = "stub", matchIfMissing = true)
    ParticipantReaderPort participantReaderStub() { return new
ParticipantReaderStubAdapter(); }
    // même schéma pour chaque port sortant
}
```

0.9. Compilation, exécution et scripts de test

```
# Régénération incrémentale puis construction des deux services
cd services/mshomologation && jhipster jdl ../../jdl/lot1-homologation.jdl --skip-
fake-data --force --skip-install && ./mvnw -Pdev clean verify
cd ../mscertification    && jhipster jdl ../../jdl/lot1-certification.jdl --skip-
fake-data --force --skip-install && ./mvnw -Pdev clean verify

# Démarrage à vide et contrôle de santé (via la passerelle, avec jeton)
cd ../../infra/docker-compose && ./gimac-dev.sh demarrer mshomologation
mscertification
source ./scripts/token.sh test-chef-des
curl -s -o /dev/null -w "%{http_code}\n"
"${GATEWAY_URL}/bff/des/homologation/liaisons"                                  # 401
sans jeton
curl -s -H "Authorization: Bearer ${TOKEN}"
"${GATEWAY_URL}/bff/des/homologation/liaisons?page=0&size=1" | jq .         # 200,
page vide
curl -s -H "Authorization: Bearer ${TOKEN}"
"${GATEWAY_URL}/bff/des/certification/registre?page=0&size=1" | jq .        # 200,
page vide
```

```
# scripts/token.sh : obtention du jeton d'un utilisateur de test (les mots de passe
viennent de l'environnement)
#!/usr/bin/env bash
set -euo pipefail
USER_NAME="${1:?utilisateur de test requis}"
PASS_VAR="PASSWORD_$(echo "${USER_NAME}" | tr 'a-z-' 'A-Z_')"
export TOKEN=$(curl -s -X POST "${KEYCLOAK_URL}/realms/gimac/protocol/openid-
connect/token" \
  -d "grant_type=password" -d "client_id=tester-cli" -d "client_secret=$
{TESTER_CLIENT_SECRET}" \
  -d "username=${USER_NAME}" -d "password=${!PASS_VAR}" | jq -r .access_token)
[ "${TOKEN}" != "null" ] || { echo "jeton non obtenu pour ${USER_NAME}" >&2; exit
1; }
```

Sortie de la Phase 0

Les deux services compilent, démarrent, exposent leur santé, refusent sans jeton et répondent avec un jeton valide à travers la passerelle ; les tables du JDL sont créées avec leurs unicités et index ; les sujets Kafka et leurs files de rebut existent ; les permissions du lot sont connues de MS-IAM et d'OPA ; les tests générés sont verts ; le tableau de correspondance des noms est publié. La revue de fin de Phase 0 confirme ou corrige les hypothèses de la convention transversale avant l'itération A.

MS-HOMOLOGATION — Récits HOM-01 à HOM-24 : suivi de l'homologation des participants

Le microservice tient le référentiel d'opérations versionné par type de projet, ouvre et suit les liaisons des participants sur leur périmètre contractualisé, instancie et débloque les opérations par tri topologique, clôt chaque opération selon la règle des quatre éléments, reprend en lecture seule les résultats du simulateur de certification sans jamais les recalculer, calcule les couvertures de certification et de mise en service par service et saisit MS-CERTIFICATION lorsque la liaison est apte. Les vingt-quatre récits sont traités ci-dessous dans l'ordre des features du carnet ; l'ordre d'exécution effectif figure au récapitulatif.

HOM-01 | Publier une version du référentiel d'opérations par type de projet

Élément	Valeur
Points	5
Priorité	Must Have
Sprint cible	Sprint 3
Écrans	DES-12
Dépendances	SOC-04, SOC-05, IAM-04, AUD-01, IHM-06

Objectif

Livrer le cycle de vie des versions du référentiel d'opérations (brouillon, publiée, retirée) pour les trois types de projet : création ou dérivation d'un brouillon, édition opération par opération, publication qui numérote, date et fige la version puis émet `homologation.referential-published.v1` par l'outbox. Résultat vérifiable : une écriture sur une version publiée renvoie 409 `HOM_REFERENTIAL_FROZEN` avec la proposition d'un brouillon dérivé, et une publication incomplète renvoie 409 `HOM_REFERENTIAL_INCOMPLETE` avec la liste des opérations et des attributs manquants.

A. Implémentation

A1. Tables `referential_version` et `reference_operation`

Le récit introduit les deux tables du référentiel. Les attributs d'une opération restent facultatifs en base pour qu'un brouillon puisse être saisi progressivement ; la complétude est contrôlée à la publication (A3). Les opérations transverses (signature de la convention, par exemple) sont rattachées au code de service réservé `TRANSVERSE`, toujours inclus dans le périmètre (HOM-05).

```
<!-- resources/config/liquibase/changelog/20261005000100_hom01_referentiel.xml (en-
tête JHipster conservé) -->
<databaseChangeLog>
  <changeSet id="hom-01-referentiel" author="mshomologation">
    <createTable tableName="referential_version">
      <column name="id" type="uuid"><constraints primaryKey="true"/></column>
      <column name="type_projet" type="varchar(32)"><constraints
nullable="false"/></column>
      <column name="numero" type="integer"/>
      <column name="libelle" type="varchar(200)"><constraints
nullable="false"/></column>
      <column name="etat" type="varchar(16)"><constraints nullable="false"/></column>
      <column name="date_creation" type="timestamptz"><constraints
nullable="false"/></column>
      <column name="date_publication" type="timestamptz"/>
      <column name="ordre_topologique" type="jsonb"/>
      <column name="version_source_id" type="uuid"/>
    </createTable>
    <!-- numérotation continue par type ; deux publications concurrentes échouent sur
cette contrainte -->
    <addUniqueConstraint tableName="referential_version" columnNames="type_projet,
numero"
                         constraintName="ux_referential_version_type_numero"/>
    <createTable tableName="reference_operation">
      <column name="id" type="uuid"><constraints primaryKey="true"/></column>
      <column name="version_id" type="uuid">
        <constraints nullable="false" foreignKeyName="fk_refop_version"
references="referential_version(id)"/>
```

```
      </column>
      <column name="code" type="varchar(40)"><constraints nullable="false"/></column>
      <column name="libelle" type="varchar(250)"/>
      <column name="phase" type="varchar(60)"/>
      <column name="segment" type="varchar(24)"/>
      <column name="partie" type="varchar(16)"/>
      <column name="service_code" type="varchar(40)"/>
      <column name="nature_couverture" type="varchar(20)"/>
      <column name="preuve_attendue" type="jsonb"/>
      <column name="id_scenario" type="varchar(60)"/>
      <column name="predecesseurs" type="jsonb" defaultValueComputed="'[]'::jsonb"/>
      <column name="fonction_responsable" type="varchar(60)"/>
      <column name="grille" type="jsonb"/>
    </createTable>
    <addUniqueConstraint tableName="reference_operation" columnNames="version_id,
code"
                         constraintName="ux_reference_operation_version_code"/>
  </changeSet>
</databaseChangeLog>
```

A2. Entités, garde d'immuabilité et erreur métier

La garde `assertModifiable()` est le point de contrôle unique des écritures ; un écouteur JPA la rappelle avant toute insertion, mise à jour ou suppression d'une opération, de sorte qu'aucun chemin de code ne contourne l'immuabilité. Publier une nouvelle version d'un type retire la précédente : elle reste lisible et continue de servir les liaisons qui y sont rattachées jusqu'à leur migration (HOM-03), mais n'est plus proposée à l'ouverture d'une liaison (HOM-04). L'exception `HomologationException` est posée ici pour tout le service ; l'`ExceptionTranslator` généré la convertit en `ProblemDetail` RFC 9457.

```
// domain/HomologationException.java
public class HomologationException extends RuntimeException {
    private final String code;
    private final int status;
    private final Map<String, Object> details;

    private HomologationException(String code, int status, String message,
Map<String, Object> details) {
        super(message);
        this.code = code; this.status = status; this.details = Map.copyOf(details);
    }
    /** 409 : règle métier ou transition interdite. */
    public static HomologationException conflict(String code, String message,
Map<String, Object> details) {
        return new HomologationException(code, 409, message, details);
    }
    /** 503 : tiers indisponible, sans valeur par défaut. */
    public static HomologationException unavailable(String code, String message) {
        return new HomologationException(code, 503, message, Map.of());
    }
    public static HomologationException notFound(String code, Object id) {
        return new HomologationException(code, 404, "Ressource introuvable",
Map.of("id", String.valueOf(id)));
    }
    public String code() { return code; }
    public int status() { return status; }
    public Map<String, Object> details() { return details; }
}
```

```
// domain/model/ReferentialVersion.java (extrait : accesseurs générés omis)
@Entity
@Table(name = "referential_version")
public class ReferentialVersion {
    @Id private UUID id;
```

```
    @Enumerated(EnumType.STRING) private TypeProjet typeProjet;
    private Integer numero;
    private String libelle;
    @Enumerated(EnumType.STRING) private EtatVersionReferentiel etat;
    private Instant dateCreation;
    private Instant datePublication;
    @JdbcTypeCode(SqlTypes.JSON) @Column(columnDefinition = "jsonb")
    private List<String> ordreTopologique;
    private UUID versionSourceId;
    @OneToMany(mappedBy = "version", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<ReferenceOperation> operations = new ArrayList<>();

    public static ReferentialVersion nouveauBrouillon(TypeProjet type, String
libelle, Instant instant) {
        ReferentialVersion v = new ReferentialVersion();
        v.id = UUID.randomUUID(); v.typeProjet = type; v.libelle = libelle;
        v.etat = EtatVersionReferentiel.BROUILLON; v.dateCreation = instant;
        return v;
    }

    /** Dérivation : copie profonde des opérations d'une version publiée dans un
nouveau brouillon. */
    public static ReferentialVersion deriverDe(ReferentialVersion source, String
libelle, Instant instant) {
        ReferentialVersion v = nouveauBrouillon(source.typeProjet, libelle, instant);
        v.versionSourceId = source.id;
        source.operations.forEach(op -> v.operations.add(op.copierVers(v)));
        return v;
    }

    /** Garde unique : seule une version à l'état BROUILLON accepte une écriture. */
    public void assertModifiable() {
        if (etat != EtatVersionReferentiel.BROUILLON) {
            throw HomologationException.conflict("HOM_REFERENTIAL_FROZEN",
                "La version " + numero + " est figée ; un brouillon dérivé doit être
créé",
                Map.of("versionId", id.toString(),
                       "brouillonDerivePropose", Map.of("methode", "POST", "chemin",
"/referentiels",
                                                        "versionSourceId",
id.toString())));
        }
    }

    public ReferenceOperation upsertOperation(String code, ReferenceOperationData
data) {
        assertModifiable();
        ReferenceOperation op = operations.stream().filter(o ->
o.getCode().equals(code)).findFirst()
            .orElseGet(() -> { ReferenceOperation n =
ReferenceOperation.nouvelle(this, code); operations.add(n); return n; });
        op.appliquer(data);
        return op;
    }

    /** Fige la version : numéro, date et ordre topologique calculé par le contrôle
de HOM-02. */
    public void publier(int numeroAttribue, Instant instant, List<String> ordre) {
        assertModifiable();
        this.numero = numeroAttribue;
        this.datePublication = instant;
        this.ordreTopologique = List.copyOf(ordre);
        this.etat = EtatVersionReferentiel.PUBLIEE;
    }

    public void retirer() {
        if (etat == EtatVersionReferentiel.PUBLIEE) etat =
EtatVersionReferentiel.RETIREE;
    }
```

```
}
```

```
// domain/model/ReferenceOperationImmutabilityListener.java
// Déclaré par @EntityListeners sur ReferenceOperation : défense en profondeur de la
garde.
public class ReferenceOperationImmutabilityListener {
    @PrePersist @PreUpdate @PreRemove
    void verifier(ReferenceOperation operation) {
        operation.getVersion().assertModifiable();
    }
}
```

A3. Contrôle de complétude (service de domaine pur)

Le contrôle parcourt toutes les opérations et restitue, par code, la liste des attributs absents. L'identifiant de scénario n'est exigé que pour une opération de nature `CERTIFICATION`, car c'est lui qui permet le rapprochement des résultats du simulateur (HOM-15).

```
// domain/service/ReferentialCompletenessChecker.java
public final class ReferentialCompletenessChecker {

    public record MissingAttributes(String code, List<String> attributs) {}

    public List<MissingAttributes> check(Collection<ReferenceOperation> operations) {
        List<MissingAttributes> manquants = new ArrayList<>();
        for (ReferenceOperation op : operations) {
            List<String> absents = new ArrayList<>();
            if (vide(op.getLibelle())) absents.add("libelle");
            if (vide(op.getPhase())) absents.add("phase");
            if (op.getSegment() == null) absents.add("segment");
            if (op.getPartie() == null) absents.add("partie");
            if (vide(op.getServiceCode())) absents.add("serviceCode");
            if (op.getNatureCouverture() == null) absents.add("natureCouverture");
            if (op.getPreuveAttendue() == null || op.getPreuveAttendue().isEmpty())
absents.add("preuveAttendue");
            if (vide(op.getFonctionResponsable()))
absents.add("fonctionResponsable");
            if (op.getNatureCouverture() == NatureCouverture.CERTIFICATION &&
vide(op.getIdScenario())) absents.add("idScenario");
            if (!absents.isEmpty()) manquants.add(new MissingAttributes(op.getCode(),
List.copyOf(absents)));
        }
        manquants.sort(Comparator.comparing(MissingAttributes::code));
        return manquants;
    }

    private static boolean vide(String s) { return s == null || s.isBlank(); }
}
```

A4. Cas d'utilisation transactionnel et émission par l'outbox

La publication verrouille la version (`SELECT ... FOR UPDATE`), contrôle la complétude, délègue le contrôle du graphe à `ReferentialGraphValidator` (HOM-02), retire la version publiée précédente du même type, attribue le numéro suivant et écrit l'événement dans l'outbox au sein de la même transaction. Le premier argument de `publish` est le nom d'agrégat qui détermine le sujet (`homologation.referential`).

```
// application/usecase/ReferentialLifecycleService.java
@Service
@Transactional
@RequiredArgsConstructor
public class ReferentialLifecycleService implements ManageReferentialUseCase {
```

```
    private final ReferentialVersionRepository versions;
    private final ReferentialCompletenessChecker completeness;
    private final ReferentialGraphValidator graphValidator;   // HOM-02
    private final OutboxPublisher outboxPublisher;
    private final Clock clock;

    @Override
    public ReferentialVersion creerBrouillon(TypeProjet type, String libelle, UUID
versionSourceId) {
        Instant now = clock.instant();
        ReferentialVersion brouillon = versionSourceId == null
            ? ReferentialVersion.nouveauBrouillon(type, libelle, now)
            : ReferentialVersion.deriverDe(charger(versionSourceId), libelle, now);
        return versions.save(brouillon);
    }

    @Override
    public ReferenceOperation enregistrerOperation(UUID versionId, String code,
ReferenceOperationData data) {
        return versions.findByIdForUpdate(versionId)
            .orElseThrow(() ->
HomologationException.notFound("HOM_REFERENTIAL_NOT_FOUND", versionId))
            .upsertOperation(code, data);
    }

    @Override
    public ReferentialVersion publier(UUID versionId) {
        ReferentialVersion v = versions.findByIdForUpdate(versionId)
            .orElseThrow(() ->
HomologationException.notFound("HOM_REFERENTIAL_NOT_FOUND", versionId));
        v.assertModifiable();
        var manquants = completeness.check(v.getOperations());
        if (!manquants.isEmpty()) {
            throw HomologationException.conflict("HOM_REFERENTIAL_INCOMPLETE",
                "Publication refusée : opérations incomplètes", Map.of("operations",
manquants));
        }
        List<String> ordre = graphValidator.validate(v.noeuds());   // lève
HOM_REFERENTIAL_GRAPH_INVALID
        versions.findFirstByTypeProjetAndEtat(v.getTypeProjet(),
EtatVersionReferentiel.PUBLIEE)
            .ifPresent(ReferentialVersion::retirer);
        int numero = versions.maxNumero(v.getTypeProjet()).orElse(0) + 1;
        v.publier(numero, clock.instant(), ordre);
        outboxPublisher.publish("referential", v.getId().toString(),
"homologation.referential-published.v1",
            new ReferentialPublishedPayload(v.getId(), v.getTypeProjet(), numero,
v.getDatePublication(),
                                            v.getOperations().size(),
v.getVersionSourceId()));
        return v;
    }

    private ReferentialVersion charger(UUID id) {
        return versions.findById(id).orElseThrow(() ->
HomologationException.notFound("HOM_REFERENTIAL_NOT_FOUND", id));
    }
}
```

A5. Ressource REST

Méthode et chemin	Permission	Réponse
`POST /referentiels` (brouillon, `versionSourceId` facultatif pour une dérivation)	`HOM_REFERENTIEL_EDIT`	201, 409
`PUT /referentiels/{id}/operations`	`HOM_REFERENTIEL_EDIT`	200, 409

Méthode et chemin	Permission	Réponse
`/{code}`		`HOM_REFERENTIAL_FROZEN`
`POST /referentiels/{id}/publication`	`HOM_REFERENTIEL_PUBLISH`	200, 409
`GET /referentiels?type=`	`HOM_REFERENTIEL_READ`	200

```
// web/rest/ReferentialResource.java
@RestController
@RequestMapping("/api/v1/referentiels")
@RequiredArgsConstructor
public class ReferentialResource {

    private final ManageReferentialUseCase useCase;
    private final ReferentialVersionRepository versions;
    private final ReferentialMapper mapper;   // calcule transitionsPermises et
actionsPermises selon l'état

    @PostMapping
    @PreAuthorize("hasAuthority('HOM_REFERENTIEL_EDIT')")
    public ResponseEntity<ReferentialVersionDTO> creer(@Valid @RequestBody
CreateDraftRequest req) {
        ReferentialVersion v = useCase.creerBrouillon(req.typeProjet(),
req.libelle(), req.versionSourceId());
        return ResponseEntity.created(URI.create("/api/v1/referentiels/" +
v.getId())).body(mapper.toDto(v));
    }

    @PutMapping("/{id}/operations/{code}")
    @PreAuthorize("hasAuthority('HOM_REFERENTIEL_EDIT')")
    public ReferenceOperationDTO enregistrer(@PathVariable UUID id, @PathVariable
String code,
                                             @Valid @RequestBody
ReferenceOperationData data) {
        return mapper.toDto(useCase.enregistrerOperation(id, code, data));
    }

    @PostMapping("/{id}/publication")
    @PreAuthorize("hasAuthority('HOM_REFERENTIEL_PUBLISH')")
    public ReferentialVersionDTO publier(@PathVariable UUID id) {
        return mapper.toDto(useCase.publier(id));
    }

    @GetMapping
    @PreAuthorize("hasAuthority('HOM_REFERENTIEL_READ')")
    public List<ReferentialVersionDTO> lister(@RequestParam("type") TypeProjet type)
{
        return
versions.findByTypeProjetOrderByDateCreationDesc(type).stream().map(mapper::toDto).to
List();
    }
}
```

Pour un brouillon, `actionsPermises` vaut `["EDITER", "CONTROLER", "PUBLIER"]` ; pour une version publiée ou retirée, `["DERIVER"]`, ce qui permet à l'écran DES-12 de proposer la dérivation sans recalculer la règle.

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
// src/test/java/org/gimac/des/homologation/domain/model/ReferentialVersionTest.java
class ReferentialVersionTest {

    private static final Instant T0 = Instant.parse("2026-10-05T08:00:00Z");
```

```
    @Test
    @DisplayName("HOM-01 : toute écriture sur une version publiée est refusée avec
proposition de brouillon dérivé")
    void refuseEcritureSurVersionPubliee() {
        ReferentialVersion v =
ReferentialVersion.nouveauBrouillon(TypeProjet.CARTE_INTERBANCAIRE, "Carte 2026",
T0);
        v.upsertOperation("CI-AMONT-01",
ReferenceOperationSamples.complete(Segment.TRONC_COMMUN_AMONT));
        v.publier(1, T0.plusSeconds(60), List.of("CI-AMONT-01"));

        assertThatThrownBy(() -> v.upsertOperation("CI-AMONT-01",
ReferenceOperationSamples.complete(Segment.TRONC_COMMUN_AMONT)))
            .isInstanceOfSatisfying(HomologationException.class, ex -> {
                assertThat(ex.code()).isEqualTo("HOM_REFERENTIAL_FROZEN");
                assertThat(ex.details()).containsKey("brouillonDerivePropose");
            });
    }

    @Test
    @DisplayName("HOM-01 : la complétude nomme l'opération et ses attributs
manquants")
    void completudeNommeLesAttributsManquants() {
        ReferentialVersion v =
ReferentialVersion.nouveauBrouillon(TypeProjet.PAIEMENT_MOBILE, "Mobile 2026", T0);
        v.upsertOperation("PM-BR-04",
ReferenceOperationSamples.complete(Segment.BRANCHE).sansPartie().sansPreuveAttendue()
);

        var manquants = new
ReferentialCompletenessChecker().check(v.getOperations());

        assertThat(manquants).singleElement().satisfies(m -> {
            assertThat(m.code()).isEqualTo("PM-BR-04");
            assertThat(m.attributs()).containsExactly("partie", "preuveAttendue");
        });
    }
}
```

```
//
src/test/java/org/gimac/des/homologation/application/usecase/ReferentialLifecycleServ
iceTest.java
@ExtendWith(MockitoExtension.class)
class ReferentialLifecycleServiceTest {

    @Mock ReferentialVersionRepository versions;
    @Mock ReferentialGraphValidator graphValidator;
    @Mock OutboxPublisher outboxPublisher;
    ReferentialLifecycleService service;

    @BeforeEach
    void init() {
        service = new ReferentialLifecycleService(versions, new
ReferentialCompletenessChecker(), graphValidator,
            outboxPublisher, Clock.fixed(Instant.parse("2026-10-05T08:00:00Z"),
ZoneOffset.UTC));
    }

    @Test
    @DisplayName("HOM-01 : la publication numérote à la suite, retire la version
précédente et écrit l'outbox")
    void publicationNumeroteRetireEtPublie() {
        ReferentialVersion precedente =
ReferentialVersionSamples.publiee(TypeProjet.DELEGATAIRE, 2);
        ReferentialVersion brouillon =
ReferentialVersionSamples.brouillonComplet(TypeProjet.DELEGATAIRE);

when(versions.findByIdForUpdate(brouillon.getId())).thenReturn(Optional.of(brouillon)
);
```

```
        when(graphValidator.validate(anyList())).thenReturn(List.of("DL-AMONT-01",
"DL-BR-01", "DL-AVAL-01"));
        when(versions.findFirstByTypeProjetAndEtat(TypeProjet.DELEGATAIRE,
EtatVersionReferentiel.PUBLIEE))
            .thenReturn(Optional.of(precedente));
        when(versions.maxNumero(TypeProjet.DELEGATAIRE)).thenReturn(Optional.of(2));

        ReferentialVersion publiee = service.publier(brouillon.getId());

        assertThat(publiee.getNumero()).isEqualTo(3);
        assertThat(publiee.getEtat()).isEqualTo(EtatVersionReferentiel.PUBLIEE);
        assertThat(precedente.getEtat()).isEqualTo(EtatVersionReferentiel.RETIREE);
        verify(outboxPublisher).publish(eq("referential"),
eq(brouillon.getId().toString()),
            eq("homologation.referential-published.v1"),
any(ReferentialPublishedPayload.class));
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

La classe de base posée par ce premier récit du service déclare les conteneurs et sert à tous les tests d'intégration de MS-HOMOLOGATION.

```
// src/test/java/org/gimac/des/homologation/AbstractHomologationIT.java
@SpringBootTest
@AutoConfigureMockMvc
@Testcontainers
@ActiveProfiles("test")   // gimac.adapters.*=stub : adaptateurs de substitution
public abstract class AbstractHomologationIT {

    @Container @ServiceConnection
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:16-
alpine");
    @Container @ServiceConnection
    static KafkaContainer kafka = new KafkaContainer("apache/kafka:3.8.0");

    @Autowired protected MockMvc mvc;
    @Autowired protected JdbcTemplate jdbc;
    @Autowired protected HomologationFixtures fixtures;   // jeux de données de test
(src/test)

    protected static RequestPostProcessor utilisateur(UUID id, String... permissions)
{
        return jwt().jwt(j -> j.subject(id.toString()))
                    .authorities(Arrays.stream(permissions).map(SimpleGrantedAuthorit
y::new).toList());
    }

    protected Consumer<String, String> consommateur(String sujet) {
        var props = KafkaTestUtils.consumerProps(kafka.getBootstrapServers(), "it-" +
UUID.randomUUID(), "true");
        Consumer<String, String> c = new DefaultKafkaConsumerFactory<>(props, new
StringDeserializer(),
            new StringDeserializer()).createConsumer();
        c.subscribe(List.of(sujet));
        return c;
    }
}
```

```
// src/test/java/org/gimac/des/homologation/web/rest/ReferentialPublicationIT.java
class ReferentialPublicationIT extends AbstractHomologationIT {

    private static final UUID INGENIEUR = UUID.fromString("00000000-0000-0000-0000-
00000000e001");
```

```
    @Test
    @DisplayName("HOM-01 : la publication écrit l'outbox puis l'événement parvient au
sujet lu par MS-AUDIT")
    void publicationOutboxPuisSujet() throws Exception {
        UUID versionId = fixtures.brouillonComplet(TypeProjet.PAIEMENT_MOBILE);

        mvc.perform(post("/api/v1/referentiels/{id}/publication", versionId)
                .with(utilisateur(INGENIEUR, "HOM_REFERENTIEL_PUBLISH")))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.etat").value("PUBLIEE"))
            .andExpect(jsonPath("$.numero").value(1))
            .andExpect(jsonPath("$.actionsPermises").value(contains("DERIVER")));

        assertThat(jdbc.queryForObject(
            "select count(*) from outbox_event where aggregate_id = ? and event_type
= ?", Integer.class,
            versionId.toString(), "homologation.referential-
published.v1")).isEqualTo(1);

        try (Consumer<String, String> c = consommateur("homologation.referential")) {
            var recus = KafkaTestUtils.getRecords(c,
Duration.ofSeconds(10)).records("homologation.referential");
            assertThat(recus).anySatisfy(r -> {                  // d'autres tests
publient sur le même sujet
                assertThat(r.key()).isEqualTo(versionId.toString());
                assertThat(JsonPath.<String>read(r.value(),
"$.type")).isEqualTo("homologation.referential-published.v1");
            });
        }

        mvc.perform(put("/api/v1/referentiels/{id}/operations/{code}", versionId,
"PM-AMONT-01")
                .with(utilisateur(INGENIEUR, "HOM_REFERENTIEL_EDIT"))
                .contentType(MediaType.APPLICATION_JSON).content(fixtures.operationJs
on("PM-AMONT-01")))
            .andExpect(status().isConflict())
            .andExpect(jsonPath("$.code").value("HOM_REFERENTIAL_FROZEN"));
    }
}
```

L'inscription effective au registre d'audit est vérifiée en recette (scénario D) par consultation de MS-AUDIT : le rôle du service s'arrête à la remise de l'événement sur son sujet.

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: HOM-01 Publication d'une version du référentiel d'opérations

  Contexte:
    Étant donné un utilisateur authentifié avec le rôle "INGENIEUR_HOMOLOGATION"

  Plan du scénario: Nominal : publication d'une version du référentiel
    Étant donné un brouillon de référentiel du type "<type>" dont chaque opération
porte un code, un libellé, une phase, un segment, ses prédécesseurs, la partie à qui
elle incombe, le service rattaché, la nature de couverture et la preuve attendue par
catégorie de participant
    Quand l'ingénieur d'homologation demande la publication du brouillon
    Alors une version numérotée et datée est créée à l'état "PUBLIEE" et devient
immuable
    Et l'événement "homologation.referential-published.v1" est émis puis inscrit au
registre d'audit

    Exemples:
      | type                |
      | CARTE_INTERBANCAIRE |
      | PAIEMENT_MOBILE     |
      | DELEGATAIRE         |
```

```
  Scénario: Alternatif : évolution d'une version publiée
    Étant donné une version publiée du référentiel "CARTE_INTERBANCAIRE"
    Quand un ingénieur tente de modifier l'opération "CI-AMONT-01" de cette version
    Alors la réponse a le statut 409 et le code "HOM_REFERENTIAL_FROZEN"
    Et la réponse propose la création d'un brouillon dérivé de cette version

  Scénario: Erreur : opération incomplète
    Étant donné un brouillon dont l'opération "PM-BR-04" n'indique ni la partie à qui
elle incombe ni la preuve attendue
    Quand la publication est demandée
    Alors la réponse a le statut 409 et le code "HOM_REFERENTIAL_INCOMPLETE"
    Et la réponse restitue pour "PM-BR-04" les attributs manquants "partie" et
"preuveAttendue"
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-ingenieur-homologation
BASE="${GATEWAY_URL}/bff/des/homologation"

# 1. Créer un brouillon de référentiel de type paiement mobile
VERSION_ID=$(curl -s -X POST "${BASE}/referentiels" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" \
  -d '{"typeProjet":"PAIEMENT_MOBILE","libelle":"Paiement mobile 2026"}' | jq -r .id)

# 2. Saisir une opération du tronc commun amont
curl -s -X PUT "${BASE}/referentiels/${VERSION_ID}/operations/PM-AMONT-01" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" \
  -d '{"libelle":"Signature de la
convention","phase":"Adhésion","segment":"TRONC_COMMUN_AMONT",
       "partie":"PARTICIPANT","serviceCode":"TRANSVERSE","natureCouverture":"AUCUNE",
       "preuveAttendue":
{"ETABLISSEMENT_PAIEMENT":"CONVENTION_SIGNEE"},"predecesseurs":[],
       "fonctionResponsable":"DIRECTION"}' | jq '{code, segment}'

# 3. Publier : attendu etat=PUBLIEE et numero attribué
curl -s -X POST "${BASE}/referentiels/${VERSION_ID}/publication" \
  -H "Authorization: Bearer ${TOKEN}" | jq '{etat, numero, datePublication,
actionsPermises}'

# 4. Refus : modification d'une version publiée, attendu 409 HOM_REFERENTIAL_FROZEN
curl -s -o /tmp/hom01.json -w "%{http_code}\n" -X PUT "${BASE}/referentiels/$
{VERSION_ID}/operations/PM-AMONT-01" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" -d
'{"libelle":"Modification"}'
jq '{code, details}' /tmp/hom01.json

# 5. Refus par permission : un correspondant ne publie pas, attendu 403
source ./scripts/token.sh test-correspondant-a
curl -s -o /dev/null -w "%{http_code}\n" -X POST "${BASE}/referentiels/$
{VERSION_ID}/publication" \
  -H "Authorization: Bearer ${TOKEN}"
```

HOM-02 | Contrôler l'acyclicité et la cohérence du référentiel avant publication

Élément	Valeur
Points	5
Priorité	Must Have
Sprint cible	Sprint 3
Écrans	DES-12
Dépendances	HOM-01

Objectif

Refuser la publication d'un brouillon dont le graphe des prédécesseurs comporte un code dupliqué, un prédécesseur inconnu, une incohérence entre segments ou un cycle, en restituant les codes fautifs et, pour un cycle, la suite ordonnée des codes qui le composent ; enregistrer l'ordre topologique de référence de toute version acceptée. Le même contrôle est exposé à blanc (`POST /referentiels/{id}/controle`) et le graphe par couches alimente la vue `DagView` de DES-12.

A. Implémentation

A1. Tri de Kahn avec restitution du cycle (service de domaine pur)

Le tri calcule les degrés entrants, extrait à chaque pas le plus petit code prêt (file de priorité : l'ordre obtenu est déterministe et reproductible), et calcule la couche de chaque nœud (plus long chemin depuis une source) pour la vue graphique. Si des nœuds restent non traités, chacun d'eux garde au moins un prédécesseur non traité : en remontant les prédécesseurs résiduels depuis le plus petit code résiduel, on retombe nécessairement sur un nœud déjà vu, ce qui isole un cycle. Le cycle est restitué dans le sens « a pour prédécesseur », en commençant par son plus petit code et en répétant ce code à la fin (`A, B, C, A`). Complexité : O((V + E) log V), soit quelques millisecondes pour cinq cents opérations.

```
// domain/service/TopologicalSorter.java
public final class TopologicalSorter {

    /** Ordre complet et couches si le graphe est acyclique ; sinon cycle restitué et
ordre vide. */
    public record Result(List<String> ordre, Map<String, Integer> couches,
List<String> cycle) {
        public boolean acyclique() { return cycle.isEmpty(); }
    }

    /**
     * @param predecesseurs pour chaque code, ses prédécesseurs ; précondition : tout
prédécesseur est une clé
     *                      (vérifiée par ReferentialGraphValidator avant l'appel).
     */
    public Result sort(Map<String, ? extends Collection<String>> predecesseurs) {
        Map<String, Integer> degre = new HashMap<>();
        Map<String, List<String>> successeurs = new HashMap<>();
        predecesseurs.forEach((code, preds) -> {
            degre.merge(code, 0, Integer::sum);
            for (String p : new LinkedHashSet<>(preds)) {          // un prédécesseur
répété ne compte qu'une fois
                degre.merge(code, 1, Integer::sum);
                successeurs.computeIfAbsent(p, k -> new ArrayList<>()).add(code);
            }
```

```
        });
        PriorityQueue<String> prets = new PriorityQueue<>();
        degre.forEach((code, d) -> { if (d == 0) prets.add(code); });

        List<String> ordre = new ArrayList<>(degre.size());
        Map<String, Integer> couches = new HashMap<>();
        while (!prets.isEmpty()) {
            String n = prets.poll();
            ordre.add(n);
            int couche = couches.computeIfAbsent(n, k -> 0);
            for (String s : successeurs.getOrDefault(n, List.of())) {
                couches.merge(s, couche + 1, Math::max);
                if (degre.merge(s, -1, Integer::sum) == 0) prets.add(s);
            }
        }
        if (ordre.size() == degre.size()) {
            return new Result(List.copyOf(ordre), Map.copyOf(couches), List.of());
        }
        return new Result(List.of(), Map.of(), extraireCycle(predecesseurs, degre));
    }

    private List<String> extraireCycle(Map<String, ? extends Collection<String>>
preds, Map<String, Integer> degre) {
        String courant = degre.entrySet().stream().filter(e -> e.getValue() > 0)
            .map(Map.Entry::getKey).min(Comparator.naturalOrder()).orElseThrow();
        Map<String, Integer> rang = new HashMap<>();
        List<String> chemin = new ArrayList<>();
        while (!rang.containsKey(courant)) {
            rang.put(courant, chemin.size());
            chemin.add(courant);
            courant = preds.get(courant).stream().filter(p -> degre.get(p) > 0)
                .min(Comparator.naturalOrder()).orElseThrow();
        }
        List<String> cycle = new ArrayList<>(chemin.subList(rang.get(courant),
chemin.size()));
        Collections.rotate(cycle, -cycle.indexOf(Collections.min(cycle)));
        cycle.add(cycle.get(0));                                    // fermeture
explicite : A, B, C, A
        return List.copyOf(cycle);
    }
}
```

A2. Contrôle de cohérence complet du référentiel

Le validateur enchaîne quatre contrôles et produit un rapport qui recense toutes les anomalies (et non la première seulement) : unicité des codes, existence de chaque prédécesseur, compatibilité des segments, acyclicité. Règle de segments retenue : le rang d'un prédécesseur (tronc commun amont = 0, branche = 1, tronc commun aval = 2) ne dépasse jamais celui de l'opération. Une opération du tronc commun amont ne peut donc dépendre que du tronc commun amont, ce qui garantit que ce tronc reste identique pour tous les types de projet. Le tri de Kahn s'exécute sur les seuls arcs valides, afin qu'un prédécesseur inconnu ne masque pas un cycle.

```
// domain/service/ReferentialGraphValidator.java
public final class ReferentialGraphValidator {

    public enum TypeAnomalie { CODE_DUPLIQUE, PREDECESSEUR_INCONNU,
SEGMENT_INCOHERENT, CYCLE }
    public record Anomalie(TypeAnomalie type, List<String> codes, String motif) {}
    public record Rapport(List<Anomalie> anomalies, List<String> ordre, Map<String,
Integer> couches) {
        public boolean conforme() { return anomalies.isEmpty(); }
    }

    private final TopologicalSorter sorter = new TopologicalSorter();
```

```
    public Rapport controler(List<NoeudReferentiel> noeuds) {
        List<Anomalie> anomalies = new ArrayList<>();
        Map<String, NoeudReferentiel> parCode = new TreeMap<>();
        for (NoeudReferentiel n : noeuds) {
            if (parCode.putIfAbsent(n.code(), n) != null) {
                anomalies.add(new Anomalie(TypeAnomalie.CODE_DUPLIQUE,
List.of(n.code()), "Code présent plusieurs fois"));
            }
        }
        Map<String, List<String>> arcsValides = new TreeMap<>();
        for (NoeudReferentiel n : parCode.values()) {
            List<String> valides = new ArrayList<>();
            for (String p : n.predecesseurs()) {
                NoeudReferentiel pred = parCode.get(p);
                if (pred == null) {
                    anomalies.add(new Anomalie(TypeAnomalie.PREDECESSEUR_INCONNU,
List.of(n.code(), p),
                        "Prédécesseur absent de la version"));
                    continue;
                }
                if (pred.segment().ordinal() > n.segment().ordinal()) {   // ordre de
l'énumération Segment : AMONT, BRANCHE, AVAL
                    anomalies.add(new Anomalie(TypeAnomalie.SEGMENT_INCOHERENT,
List.of(n.code(), p),
                        "Une opération du segment " + n.segment() + " ne peut
succéder à une opération du segment " + pred.segment()));
                }
                valides.add(p);
            }
            arcsValides.put(n.code(), valides);
        }
        TopologicalSorter.Result tri = sorter.sort(arcsValides);
        if (!tri.acyclique()) {
            anomalies.add(new Anomalie(TypeAnomalie.CYCLE, tri.cycle(), "Cycle dans
le graphe des prédécesseurs"));
        }
        return new Rapport(List.copyOf(anomalies), tri.ordre(), tri.couches());
    }

    /** Port interne invoqué par la publication (HOM-01) : renvoie l'ordre de
référence ou refuse. */
    public List<String> validate(List<NoeudReferentiel> noeuds) {
        Rapport r = controler(noeuds);
        if (!r.conforme()) {
            throw HomologationException.conflict("HOM_REFERENTIAL_GRAPH_INVALID",
                "Publication refusée : graphe des prédécesseurs non conforme",
Map.of("anomalies", r.anomalies()));
        }
        return r.ordre();
    }
}
```

```
// domain/service/NoeudReferentiel.java
public record NoeudReferentiel(String code, String libelle, Segment segment,
List<String> predecesseurs) {}
```

Les deux services purs sont déclarés comme beans dans `config/DomainServicesConfiguration.java` (`@Bean ReferentialGraphValidator`, `@Bean TopologicalSorter`, `@Bean ReferentialCompletenessChecker`) ; ils ne portent aucune annotation Spring. `ReferentialVersion.noeuds()` projette les opérations de la version en `NoeudReferentiel`.

A3. Contrôle à blanc et graphe par couches (ressource REST)

Le contrôle à blanc ne modifie rien et répond toujours 200 avec le rapport, y compris sur une version publiée. Le graphe renvoie les nœuds avec leur couche et les arcs, dans le format consommé par `DagView`.

Méthode et chemin	Permission	Réponse
`POST /referentiels/{id}/controle`	`HOM_REFERENTIEL_EDIT`	200 : rapport (`conforme`, `anomalies`, `ordre`)
`GET /referentiels/{id}/graphe`	`HOM_REFERENTIEL_READ`	200 : `noeuds[{code, libelle, segment, couche}]`, `arcs[{de, vers}]`, `anomalies`

```
// web/rest/ReferentialGraphResource.java
@RestController
@RequestMapping("/api/v1/referentiels/{id}")
@RequiredArgsConstructor
public class ReferentialGraphResource {

    private final ReferentialVersionRepository versions;
    private final ReferentialGraphValidator validator;

    @PostMapping("/controle")
    @PreAuthorize("hasAuthority('HOM_REFERENTIEL_EDIT')")
    public ReferentialGraphValidator.Rapport controler(@PathVariable UUID id) {
        return validator.controler(charger(id).noeuds());
    }

    @GetMapping("/graphe")
    @PreAuthorize("hasAuthority('HOM_REFERENTIEL_READ')")
    public GrapheDTO graphe(@PathVariable UUID id) {
        List<NoeudReferentiel> noeuds = charger(id).noeuds();
        var rapport = validator.controler(noeuds);
        return new GrapheDTO(
            noeuds.stream().map(n -> new GrapheDTO.Noeud(n.code(), n.libelle(),
n.segment(),
                rapport.couches().get(n.code()))).toList(),                    //
couche nulle si le graphe est cyclique
            noeuds.stream().flatMap(n -> n.predecesseurs().stream().map(p -> new
GrapheDTO.Arc(p, n.code()))).toList(),
            rapport.anomalies());
    }

    private ReferentialVersion charger(UUID id) {
        return versions.findById(id).orElseThrow(() ->
HomologationException.notFound("HOM_REFERENTIAL_NOT_FOUND", id));
    }
}
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
//
src/test/java/org/gimac/des/homologation/domain/service/ReferentialGraphValidatorTest
.java
class ReferentialGraphValidatorTest {

    private final ReferentialGraphValidator validator = new
ReferentialGraphValidator();

    static NoeudReferentiel n(String code, Segment s, String... preds) {
        return new NoeudReferentiel(code, code, s, List.of(preds));
    }
```

```
    static Stream<Arguments> cyclesDeReference() {
        return Stream.of(
            Arguments.of("boucle simple", List.of(n("A", Segment.BRANCHE, "A")),
List.of("A", "A")),
            Arguments.of("cycle de longueur deux", List.of(n("A", Segment.BRANCHE,
"B"), n("B", Segment.BRANCHE, "A")),
                         List.of("A", "B", "A")),
            Arguments.of("cycle long", List.of(n("A", Segment.BRANCHE, "B"), n("B",
Segment.BRANCHE, "C"),
                         n("C", Segment.BRANCHE, "A"), n("D", Segment.BRANCHE, "A")),
List.of("A", "B", "C", "A")));
    }

    @ParameterizedTest(name = "{0}")
    @MethodSource("cyclesDeReference")
    @DisplayName("HOM-02 : le cycle est détecté et restitué sous forme de suite
ordonnée de codes")
    void cycleRestitue(String cas, List<NoeudReferentiel> noeuds, List<String>
cycleAttendu) {
        var rapport = validator.controler(noeuds);
        assertThat(rapport.conforme()).isFalse();
        assertThat(rapport.anomalies()).filteredOn(a -> a.type() ==
TypeAnomalie.CYCLE)
            .singleElement().extracting(Anomalie::codes).isEqualTo(cycleAttendu);
    }

    @Test
    @DisplayName("HOM-02 : un prédécesseur de branche pour une opération du tronc
commun amont est incohérent")
    void segmentIncoherent() {
        var rapport = validator.controler(List.of(n("DL-AMONT-03",
Segment.TRONC_COMMUN_AMONT, "DL-BR-02"),
                                                  n("DL-BR-02", Segment.BRANCHE)));
        assertThat(rapport.anomalies()).singleElement().satisfies(a -> {
            assertThat(a.type()).isEqualTo(TypeAnomalie.SEGMENT_INCOHERENT);
            assertThat(a.codes()).containsExactly("DL-AMONT-03", "DL-BR-02");
        });
    }

    @RepeatedTest(20)
    @DisplayName("HOM-02 : tout graphe accepté admet un ordre topologique vérifié arc
par arc")
    void ordreVerifie(RepetitionInfo info) {
        List<NoeudReferentiel> dag = GraphSamples.dagAleatoire(120, 0.05,
info.getCurrentRepetition());  // graine fixe
        var rapport = validator.controler(dag);
        assertThat(rapport.conforme()).isTrue();
        Map<String, Integer> position = IntStream.range(0,
rapport.ordre().size()).boxed()
            .collect(Collectors.toMap(rapport.ordre()::get, i -> i));
        dag.forEach(op -> op.predecesseurs().forEach(p ->
            assertThat(position.get(p)).as("%s avant %s", p,
op.code()).isLessThan(position.get(op.code()))));
    }

    @Test
    @DisplayName("HOM-02 : un référentiel de cinq cents opérations est contrôlé en
moins d'une seconde")
    void chargeCinqCentsOperations() {
        List<NoeudReferentiel> dag = GraphSamples.dagAleatoire(500, 0.02, 42);
        assertTimeoutPreemptively(Duration.ofSeconds(1), () ->
validator.controler(dag));
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
// src/test/java/org/gimac/des/homologation/web/rest/ReferentialControlIT.java
class ReferentialControlIT extends AbstractHomologationIT {

    private static final UUID INGENIEUR = UUID.fromString("00000000-0000-0000-0000-
00000000e001");

    @Test
    @DisplayName("HOM-02 : un brouillon cyclique est signalé à blanc puis sa
publication est refusée sans outbox")
    void cycleSignalePuisPublicationRefusee() throws Exception {
        UUID versionId = fixtures.brouillonAvecCycle(TypeProjet.DELEGATAIRE, "DL-BR-
01", "DL-BR-02", "DL-BR-03");

        mvc.perform(post("/api/v1/referentiels/{id}/controle",
versionId).with(utilisateur(INGENIEUR, "HOM_REFERENTIEL_EDIT")))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.conforme").value(false))
            .andExpect(jsonPath("$.anomalies[0].type").value("CYCLE"))
            .andExpect(jsonPath("$.anomalies[0].codes").value(contains("DL-BR-01",
"DL-BR-02", "DL-BR-03", "DL-BR-01")));

        mvc.perform(post("/api/v1/referentiels/{id}/publication",
versionId).with(utilisateur(INGENIEUR, "HOM_REFERENTIEL_PUBLISH")))
            .andExpect(status().isConflict())
            .andExpect(jsonPath("$.code").value("HOM_REFERENTIAL_GRAPH_INVALID"));

        assertThat(jdbc.queryForObject("select etat from referential_version where id
= ?", String.class, versionId))
            .isEqualTo("BROUILLON");
        assertThat(jdbc.queryForObject("select count(*) from outbox_event where
aggregate_id = ?", Integer.class,
            versionId.toString())).isZero();
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: HOM-02 Contrôle d'acyclicité et de cohérence du référentiel

  Contexte:
    Étant donné un utilisateur authentifié avec le rôle "INGENIEUR_HOMOLOGATION"

  Scénario: Nominal : graphe acyclique accepté
    Étant donné un brouillon dont les prédécesseurs forment un graphe orienté sans
cycle
    Quand le contrôle de publication s'exécute
    Alors un ordre topologique de référence est calculé et enregistré avec la version
    Et la publication se poursuit jusqu'à l'état "PUBLIEE"

  Scénario: Alternatif : prédécesseur incohérent
    Étant donné un brouillon délégataire dont l'opération "DL-AMONT-03" du tronc
commun amont déclare pour prédécesseur l'opération "DL-BR-02" de la branche
délégataire
    Quand le contrôle de publication s'exécute
    Alors la publication est refusée avec l'anomalie "SEGMENT_INCOHERENT"
    Et les codes "DL-AMONT-03" et "DL-BR-02" sont restitués

  Scénario: Erreur : cycle détecté
    Étant donné trois opérations "DL-BR-01", "DL-BR-02" et "DL-BR-03" dont chacune
désigne la suivante comme prédécesseur, la dernière désignant la première
    Quand le contrôle de publication s'exécute
    Alors la publication est refusée
    Et le cycle est restitué sous la forme de la suite ordonnée "DL-BR-01, DL-BR-02,
DL-BR-03, DL-BR-01"
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-ingenieur-homologation
BASE="${GATEWAY_URL}/bff/des/homologation"
VERSION_ID="${VERSION_ID:?identifiant d'un brouillon de recette}"

# 1. Contrôle à blanc : conformité, anomalies et ordre topologique
curl -s -X POST "${BASE}/referentiels/${VERSION_ID}/controle" \
  -H "Authorization: Bearer ${TOKEN}" | jq '{conforme, anomalies,
premiers: .ordre[0:5]}'

# 2. Graphe par couches pour la vue DagView
curl -s "${BASE}/referentiels/${VERSION_ID}/graphe" \
  -H "Authorization: Bearer ${TOKEN}" | jq '[.noeuds[] | {code, couche}] |
group_by(.couche) | map(length)'

# 3. Refus : publication d'un brouillon cyclique, attendu 409
HOM_REFERENTIAL_GRAPH_INVALID
curl -s -o /tmp/hom02.json -w "%{http_code}\n" -X POST "${BASE}/referentiels/$
{VERSION_CYCLIQUE_ID}/publication" \
  -H "Authorization: Bearer ${TOKEN}"
jq '.details.anomalies[] | select(.type == "CYCLE") | .codes' /tmp/hom02.json

# 4. Refus par permission : un correspondant ne lance pas le contrôle, attendu 403
source ./scripts/token.sh test-correspondant-a
curl -s -o /dev/null -w "%{http_code}\n" -X POST "${BASE}/referentiels/$
{VERSION_ID}/controle" \
  -H "Authorization: Bearer ${TOKEN}"
```

HOM-03 | Migrer de façon contrôlée les liaisons en cours vers une nouvelle version du référentiel

Élément	Valeur
Points	8
Priorité	Should Have
Sprint cible	Sprint 5
Écrans	DES-12, DES-08
Dépendances	HOM-01, HOM-07, HOM-11, NOT-14

Objectif

Permettre au chef du DES de simuler l'impact d'une nouvelle version publiée sur les liaisons rattachées à la version précédente, puis d'appliquer la migration aux seules liaisons retenues et migrables : opérations conservées avec leur état et leurs preuves, opérations ajoutées instanciées par tri de Kahn, opérations retirées archivées, opérations closes dont la preuve attendue a changé rouvertes. Résultat vérifiable : une migration interrompue reprend sans doublon d'opération, et une liaison engagée dans le circuit de quitus est exclue avec son motif.

A. Implémentation

A1. Table `referential_migration`

La migration est un agrégat à part entière : elle porte le différentiel et le rapport d'impact figés au moment de la simulation, la décision (liaisons retenues, décideur) et la progression. Automate : `SIMULEE` → `APPROUVEE` → `EN_COURS` → `TERMINEE`, avec `INTERROMPUE` → `EN_COURS` pour la reprise.

```
<!-- resources/config/liquibase/changelog/20261102000100_hom03_migration.xml (en-tête
JHipster conservé) -->
<databaseChangeLog>
  <changeSet id="hom-03-migration" author="mshomologation">
    <createTable tableName="referential_migration">
      <column name="id" type="uuid"><constraints primaryKey="true"/></column>
      <column name="version_source_id" type="uuid">
        <constraints nullable="false" foreignKeyName="fk_migration_source"
references="referential_version(id)"/>
      </column>
      <column name="version_cible_id" type="uuid">
        <constraints nullable="false" foreignKeyName="fk_migration_cible"
references="referential_version(id)"/>
      </column>
      <column name="etat" type="varchar(16)"><constraints nullable="false"/></column>
      <column name="rapport" type="jsonb"><constraints nullable="false"/></column>
      <column name="date_demande" type="timestamptz"><constraints
nullable="false"/></column>
      <column name="decideur_id" type="uuid"><constraints nullable="false"/></column>
      <column name="liaisons_retenues" type="jsonb"
defaultValueComputed="'[]'::jsonb"/>
    </createTable>
    <!-- le planificateur ne balaie que les migrations à exécuter -->
    <createIndex tableName="referential_migration"
indexName="ix_migration_a_executer">
      <column name="etat"/><column name="date_demande"/>
    </createIndex>
  </changeSet>
```

```
</databaseChangeLog>
```

A2. Différentiel entre deux versions (service de domaine pur)

Le différentiel est calculé par code d'opération. Une opération présente dans les deux versions est `CONSERVEE` si tous ses attributs sont identiques et `MODIFIEE` sinon, avec la liste des attributs modifiés ; les prédécesseurs sont comparés comme un ensemble, leur ordre de saisie étant sans portée. La comparaison parcourt les composantes du record `Definition`, de sorte qu'un attribut ajouté au référentiel entre automatiquement dans le différentiel.

```
// domain/service/ReferentialDiff.java
public final class ReferentialDiff {

    public enum Nature { CONSERVEE, AJOUTEE, RETIREE, MODIFIEE }

    public record Ecart(String code, Nature nature, Set<String> attributsModifies) {
        public boolean preuveAttendueModifiee() { return
attributsModifies.contains("preuveAttendue"); }
    }

    /** Définition d'une opération de référence, indépendante de la version qui la
porte. */
    public record Definition(String code, String libelle, String phase, Segment
segment, Partie partie,
                             String serviceCode, NatureCouverture natureCouverture,
                             Map<String, String> preuveAttendue, String idScenario,
Set<String> predecesseurs,
                             String fonctionResponsable, String grille) {}

    public List<Ecart> comparer(Collection<Definition> source, Collection<Definition>
cible) {
        Map<String, Definition> s = index(source);
        Map<String, Definition> c = index(cible);
        SortedSet<String> codes = new TreeSet<>(s.keySet());
        codes.addAll(c.keySet());
        List<Ecart> ecarts = new ArrayList<>(codes.size());
        for (String code : codes) {
            Definition avant = s.get(code), apres = c.get(code);
            if (avant == null) ecarts.add(new Ecart(code, Nature.AJOUTEE, Set.of()));
            else if (apres == null) ecarts.add(new Ecart(code, Nature.RETIREE,
Set.of()));
            else {
                Set<String> modifies = attributsModifies(avant, apres);
                ecarts.add(new Ecart(code, modifies.isEmpty() ? Nature.CONSERVEE :
Nature.MODIFIEE, modifies));
            }
        }
        return List.copyOf(ecarts);
    }

    private static Set<String> attributsModifies(Definition a, Definition b) {
        Set<String> modifies = new TreeSet<>();
        for (RecordComponent rc : Definition.class.getRecordComponents()) {
            try {
                if (!Objects.equals(rc.getAccessor().invoke(a),
rc.getAccessor().invoke(b))) modifies.add(rc.getName());
            } catch (ReflectiveOperationException e) {
                throw new IllegalStateException(e);
            }
        }
        return modifies;
    }

    private static Map<String, Definition> index(Collection<Definition> defs) {
        return defs.stream().collect(Collectors.toMap(Definition::code, d -> d));
    }
```

```
}
```

A3. Planification de l'impact et règle d'exclusion (service de domaine pur)

Seules les liaisons aux états `OUVERTE` et `EN_HOMOLOGATION` sont migrables. Une liaison apte, autorisée à la mise en service ou certifiée est engagée dans le circuit de quitus : elle est exclue et ne redevient migrable qu'après `homologation.liaison-returned.v1` (retour en homologation). Une liaison gelée est exclue jusqu'à son dégel (HOM-06). Seules les opérations closes dont la preuve attendue a changé sont rouvertes ; une opération non close conserve son état et la nouvelle preuve attendue s'applique à son prochain dépôt.

```
// domain/service/MigrationPlanner.java
public final class MigrationPlanner {

    public record ImpactLiaison(UUID liaisonId, String institutionNom, String
motifExclusion,
                                List<String> conservees, List<String> ajoutees,
List<String> retirees,
                                List<String> modifiees, List<String> aRouvrir) {}

    public Optional<String> motifExclusion(EtatLiaison etat) {
        return switch (etat) {
            case OUVERTE, EN_HOMOLOGATION -> Optional.empty();
            case APTE_CERTIFICATION, MISE_EN_SERVICE_AUTORISEE, CERTIFIEE ->
Optional.of(
                "HOM_MIGRATION_LIAISON_IN_QUITUS : circuit de quitus ouvert ; un
retour en homologation est préalable");
            case GELEE -> Optional.of("HOM_MIGRATION_LIAISON_FROZEN : liaison gelée ;
un dégel est préalable");
            case CLOSE -> Optional.of("HOM_MIGRATION_LIAISON_CLOSED : liaison
close");
        };
    }

    public ImpactLiaison evaluer(UUID liaisonId, String institutionNom, EtatLiaison
etat,
                                 Map<String, EtatOperation> etatsParCode,
List<ReferentialDiff.Ecart> ecarts) {
        Map<ReferentialDiff.Nature, List<String>> parNature =
ecarts.stream().collect(Collectors.groupingBy(
            ReferentialDiff.Ecart::nature, () -> new
EnumMap<>(ReferentialDiff.Nature.class),
            Collectors.mapping(ReferentialDiff.Ecart::code, Collectors.toList())));
        List<String> aRouvrir = ecarts.stream()
            .filter(e -> e.nature() == ReferentialDiff.Nature.MODIFIEE &&
e.preuveAttendueModifiee())
            .map(ReferentialDiff.Ecart::code)
            .filter(code -> etatsParCode.get(code) == EtatOperation.CLOSE)
            .toList();
        return new ImpactLiaison(liaisonId, institutionNom,
motifExclusion(etat).orElse(null),
            parNature.getOrDefault(ReferentialDiff.Nature.CONSERVEE, List.of()),
            parNature.getOrDefault(ReferentialDiff.Nature.AJOUTEE, List.of()),
            parNature.getOrDefault(ReferentialDiff.Nature.RETIREE, List.of()),
            parNature.getOrDefault(ReferentialDiff.Nature.MODIFIEE, List.of()),
aRouvrir);
    }
}
```

A4. Simulation, approbation et application transactionnelle par liaison

La simulation fige le différentiel et le rapport d'impact dans la migration (`SIMULEE`). L'approbation enregistre les liaisons retenues : celles que le rapport déclare non migrables sont écartées avec leur motif et renvoyées dans la réponse. L'application est confiée au

planificateur : il réclame une migration `APPROUVEE` ou `INTERROMPUE` par `FOR UPDATE SKIP LOCKED`, puis traite chaque liaison dans sa propre transaction (`REQUIRES_NEW`). Une liaison est donc migrée entièrement ou pas du tout ; une liaison déjà rattachée à la version cible est ignorée, ce qui rend la reprise idempotente même après l'arrêt brutal d'une instance. Une opération retirée reste liée à l'opération de référence de l'ancienne version : elle est archivée, et les requêtes des opérations actives filtrent sur la version de la liaison.

```
// application/usecase/ReferentialMigrationService.java
@Service
@Transactional
@RequiredArgsConstructor
public class ReferentialMigrationService implements MigrateReferentialUseCase {

    private final ReferentialVersionRepository versions;
    private final LiaisonRepository liaisons;
    private final OperationInstanceRepository operations;
    private final ReferentialMigrationRepository migrations;
    private final ReferentialDiff diff;
    private final MigrationPlanner planner;
    private final Clock clock;

    @Override
    public ReferentialMigration simuler(UUID versionCibleId, UUID versionSourceId,
UUID decideurId) {
        ReferentialVersion cible = charger(versionCibleId), source =
charger(versionSourceId);
        if (cible.getEtat() != EtatVersionReferentiel.PUBLIEE ||
source.getTypeProjet() != cible.getTypeProjet()
                || source.getNumero() >= cible.getNumero()) {
            throw HomologationException.conflict("HOM_MIGRATION_VERSIONS_INVALID",
                "La cible doit être une version publiée plus récente du même type de
projet", Map.of());
        }
        List<ReferentialDiff.Ecart> ecarts = diff.comparer(source.definitions(),
cible.definitions());
        List<MigrationPlanner.ImpactLiaison> impacts =
liaisons.findByVersionIdAndEtatNot(source.getId(), EtatLiaison.CLOSE)
            .stream()
            .map(l -> planner.evaluer(l.getId(), l.getInstitutionNom(), l.getEtat(),
operations.etatsParCode(l.getId()), ecarts))
            .toList();
        return migrations.save(ReferentialMigration.simulee(source.getId(),
cible.getId(),
            new RapportMigration(ecarts, impacts), decideurId, clock.instant()));
    }

    @Override
    public ReferentialMigration approuver(UUID versionCibleId, UUID migrationId,
Set<UUID> retenues, UUID decideurId) {
        ReferentialMigration m = migrations.findByIdForUpdate(migrationId)
            .filter(x -> x.getVersionCibleId().equals(versionCibleId))
            .orElseThrow(() ->
HomologationException.notFound("HOM_MIGRATION_NOT_FOUND", migrationId));
        m.approuver(retenues, decideurId, clock.instant());   // écarte avec motif
les liaisons non migrables
        return m;
    }

    private ReferentialVersion charger(UUID id) {
        return versions.findById(id).orElseThrow(() ->
HomologationException.notFound("HOM_REFERENTIAL_NOT_FOUND", id));
    }
}
```

```
// infrastructure/scheduling/MigrationRunner.java
@Component
```

```
@RequiredArgsConstructor
public class MigrationRunner {

    private final ReferentialMigrationRepository migrations;   // claimNext() : etat
in (APPROUVEE, INTERROMPUE) ... for update skip locked
    private final LiaisonMigrator migrator;
    private final Clock clock;

    @Scheduled(fixedDelayString = "${gimac.migration.poll-interval:PT30S}")
    @Transactional
    public void executerSuivante() {
        migrations.claimNext().ifPresent(m -> {
            m.demarrer(clock.instant());
            for (UUID liaisonId : m.liaisonsRetenuesOrdonnees()) {
                try {
                    m.consigner(liaisonId, migrator.migrer(m.getId(),
liaisonId));   // transaction propre à la liaison
                } catch (RuntimeException e) {
                    m.interrompre(liaisonId, e.getMessage(), clock.instant());   //
reprise au passage suivant
                    return;
                }
            }
            m.terminer(clock.instant());
        });
    }
}
```

```
// application/usecase/LiaisonMigrator.java
@Service
@RequiredArgsConstructor
public class LiaisonMigrator {

    static final String MOTIF_MIGRATION = "migration de référentiel";

    private final ReferentialMigrationRepository migrations;
    private final ReferentialVersionRepository versions;
    private final LiaisonRepository liaisons;
    private final OperationInstanceRepository operations;
    private final MigrationPlanner planner;
    private final ReopenOperationUseCase reopen;            // HOM-11
    private final OperationGraphService operationGraph;     // HOM-07
    private final CoverageRecalculationPort coverage;       // HOM-17
    private final NotificationRequester notifications;      // HOM-05
    private final OutboxPublisher outboxPublisher;

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public ResultatLiaison migrer(UUID migrationId, UUID liaisonId) {
        ReferentialMigration m = migrations.findById(migrationId).orElseThrow();
        Liaison l = liaisons.findByIdForUpdate(liaisonId).orElseThrow();
        if (l.getVersion().getId().equals(m.getVersionCibleId())) {
            return ResultatLiaison.dejaMigree();                                  //
reprise : aucun doublon
        }
        Optional<String> exclusion = planner.motifExclusion(l.getEtat());      //
l'état a pu changer depuis l'approbation
        if (exclusion.isPresent()) return ResultatLiaison.exclue(exclusion.get());

        ReferentialVersion cible =
versions.findById(m.getVersionCibleId()).orElseThrow();
        Map<String, ReferenceOperation> refCible = cible.operationsParCode();
        List<OperationInstance> rouvertes = new ArrayList<>();
        for (ReferentialDiff.Ecart e : m.getRapport().ecarts()) {
            switch (e.nature()) {
                case CONSERVEE, MODIFIEE -> {
                    OperationInstance op =
operations.findByLiaisonIdAndCode(liaisonId, e.code()).orElseThrow();
                    op.rattacher(refCible.get(e.code()));       // état,
attributions, preuves et clôtures conservés
```

```
                    if (e.preuveAttendueModifiee() && op.getEtat() ==
EtatOperation.CLOSE) {
                        reopen.reopenBySystem(op.getId(), MOTIF_MIGRATION);   //
publie operation-reopened.v1
                        rouvertes.add(op);
                    }
                }
                case AJOUTEE -> operations.findByLiaisonIdAndCode(liaisonId,
e.code()).ifPresentOrElse(
                    archivee -> archivee.rattacher(refCible.get(e.code())),   // code
réintroduit : instance réactivée
                    () -> operations.save(OperationInstance.instancier(l,
refCible.get(e.code()))));
                case RETIREE -> { /* archivée : l'instance reste liée à l'opération
de l'ancienne version */ }
            }
        }
        l.changerVersion(cible);
        operationGraph.applyScope(l);        // HOM-07 : états et degrés entrants
recalculés par tri de Kahn
        coverage.recalculer(liaisonId);      // HOM-17 : émet coverage-changed.v1 si
une couverture varie
        if (!rouvertes.isEmpty()) {
            notifications.demander("HOM_OPERATION_ROUVERTE_MIGRATION",
                Destinataires.responsablesEtChefDeProjet(rouvertes,
l.getChefDeProjetId()),
                Map.of("liaisonId", liaisonId, "codes",
rouvertes.stream().map(OperationInstance::getCode).toList()));
        }
        outboxPublisher.publish("liaison", liaisonId.toString(),
"homologation.liaison-migrated.v1",
            new LiaisonMigratedPayload(liaisonId, m.getVersionSourceId(),
cible.getId(), migrationId,
                rouvertes.stream().map(OperationInstance::getCode).toList()));
        return ResultatLiaison.migree(rouvertes.size());
    }
}
```

A5. Ressource REST et export du rapport

Méthode et chemin	Permission	Réponse
`POST /referentiels/{id}/migrations/simulation` (`{id}` : version cible ; corps `versionSourceId`)	`HOM_REFERENTIEL_MIGRATE`	201 : migration `SIMULEE` et rapport
`POST /referentiels/{id}/migrations` (corps `migrationId`, `liaisonsRetenues`)	`HOM_REFERENTIEL_MIGRATE`	202 : migration `APPROUVEE` et liaisons écartées
`GET /migrations/{id}/rapport` (`application/json` ou `text/csv`)	`HOM_REFERENTIEL_READ`	200

```
// web/rest/ReferentialMigrationResource.java
@RestController
@RequestMapping("/api/v1")
@RequiredArgsConstructor
public class ReferentialMigrationResource {

    private final MigrateReferentialUseCase useCase;
    private final ReferentialMigrationRepository migrations;
    private final CurrentUser currentUser;                 // extraction du jeton
(socle)
    private final MigrationReportCsvWriter csvWriter;

    @PostMapping("/referentiels/{id}/migrations/simulation")
```

```
    @PreAuthorize("hasAuthority('HOM_REFERENTIEL_MIGRATE')")
    public ResponseEntity<MigrationDTO> simuler(@PathVariable UUID id, @Valid
@RequestBody SimulationRequest req) {
        ReferentialMigration m = useCase.simuler(id, req.versionSourceId(),
currentUser.id());
        return ResponseEntity.created(URI.create("/api/v1/migrations/" + m.getId() +
"/rapport")).body(MigrationDTO.of(m));
    }

    @PostMapping("/referentiels/{id}/migrations")
    @PreAuthorize("hasAuthority('HOM_REFERENTIEL_MIGRATE')")
    public ResponseEntity<MigrationDTO> approuver(@PathVariable UUID id, @Valid
@RequestBody ApprovalRequest req) {
        return ResponseEntity.accepted()
            .body(MigrationDTO.of(useCase.approuver(id, req.migrationId(),
req.liaisonsRetenues(), currentUser.id())));
    }

    @GetMapping(value = "/migrations/{id}/rapport", produces =
MediaType.APPLICATION_JSON_VALUE)
    @PreAuthorize("hasAuthority('HOM_REFERENTIEL_READ')")
    public RapportMigration rapport(@PathVariable UUID id) {
        return charger(id).getRapport();
    }

    @GetMapping(value = "/migrations/{id}/rapport", produces = "text/csv")
    @PreAuthorize("hasAuthority('HOM_REFERENTIEL_READ')")
    public ResponseEntity<String> rapportCsv(@PathVariable UUID id) {
        return ResponseEntity.ok()
            .header(HttpHeaders.CONTENT_DISPOSITION, "attachment; filename=rapport-
migration-" + id + ".csv")
            .body(csvWriter.ecrire(charger(id).getRapport()));   // une ligne par
liaison et par nature d'écart
    }

    private ReferentialMigration charger(UUID id) {
        return migrations.findById(id).orElseThrow(() ->
HomologationException.notFound("HOM_MIGRATION_NOT_FOUND", id));
    }
}
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
// src/test/java/org/gimac/des/homologation/domain/service/ReferentialDiffTest.java
class ReferentialDiffTest {

    private final ReferentialDiff diff = new ReferentialDiff();

    @Test
    @DisplayName("HOM-03 : le différentiel classe chaque code en conservée, ajoutée,
retirée ou modifiée")
    void differentielQuatreNatures() {
        var a = DefinitionSamples.def("PM-AMONT-01");
        var b = DefinitionSamples.def("PM-BR-02");
        var c = DefinitionSamples.def("PM-BR-03");
        var bModifiee = DefinitionSamples.avecPreuve(b,
Map.of("ETABLISSEMENT_PAIEMENT", "RAPPORT_PARAMETRAGE_V2"));
        var d = DefinitionSamples.def("PM-AVAL-04");

        var ecarts = diff.comparer(List.of(a, b, c), List.of(a, bModifiee, d));

        assertThat(ecarts).extracting(ReferentialDiff.Ecart::code,
ReferentialDiff.Ecart::nature).containsExactly(
            tuple("PM-AMONT-01", Nature.CONSERVEE), tuple("PM-AVAL-04",
Nature.AJOUTEE),
            tuple("PM-BR-02", Nature.MODIFIEE), tuple("PM-BR-03", Nature.RETIREE));
```

```
assertThat(ecarts.get(2).attributsModifies()).containsExactly("preuveAttendue");
    }

    @Test
    @DisplayName("HOM-03 : un simple réordonnancement des prédécesseurs ne constitue
pas une modification")
    void predecesseursComparesCommeEnsemble() {
        var avant = DefinitionSamples.avecPredecesseurs("PM-AVAL-01", Set.of("PM-BR-
01", "PM-BR-02"));
        var apres = DefinitionSamples.avecPredecesseurs("PM-AVAL-01", new
LinkedHashSet<>(List.of("PM-BR-02", "PM-BR-01")));
        assertThat(diff.comparer(List.of(avant), List.of(apres))).singleElement()
            .extracting(ReferentialDiff.Ecart::nature).isEqualTo(Nature.CONSERVEE);
    }
}
```

```
// src/test/java/org/gimac/des/homologation/domain/service/MigrationPlannerTest.java
class MigrationPlannerTest {

    private final MigrationPlanner planner = new MigrationPlanner();

    @Test
    @DisplayName("HOM-03 : une liaison apte, engagée dans le circuit de quitus, est
exclue avec son motif")
    void liaisonEnCircuitDeQuitusExclue() {
        assertThat(planner.motifExclusion(EtatLiaison.APTE_CERTIFICATION))
            .hasValueSatisfying(m ->
assertThat(m).startsWith("HOM_MIGRATION_LIAISON_IN_QUITUS"));
        assertThat(planner.motifExclusion(EtatLiaison.EN_HOMOLOGATION)).isEmpty();
    }

    @Test
    @DisplayName("HOM-03 : seule une opération close dont la preuve attendue change
est à rouvrir")
    void reouvertureLimiteeAuxOperationsCloses() {
        var ecarts = List.of(new ReferentialDiff.Ecart("PM-BR-02", Nature.MODIFIEE,
Set.of("preuveAttendue")),
                             new ReferentialDiff.Ecart("PM-BR-05", Nature.MODIFIEE,
Set.of("preuveAttendue")));
        var impact = planner.evaluer(UUID.randomUUID(), "Établissement de paiement
A", EtatLiaison.EN_HOMOLOGATION,
            Map.of("PM-BR-02", EtatOperation.CLOSE, "PM-BR-05",
EtatOperation.ELIGIBLE), ecarts);
        assertThat(impact.aRouvrir()).containsExactly("PM-BR-02");
        assertThat(impact.motifExclusion()).isNull();
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
//
src/test/java/org/gimac/des/homologation/application/usecase/ReferentialMigrationIT.j
ava
class ReferentialMigrationIT extends AbstractHomologationIT {

    private static final UUID CHEF_DES = UUID.fromString("00000000-0000-0000-0000-
00000000c001");

    @Autowired MigrationRunner runner;
    @SpyBean CoverageRecalculationPort coverage;

    @Test
    @DisplayName("HOM-03 : une migration interrompue reprend sans doublon
d'opération")
    void migrationInterrompuePuisReprise() throws Exception {
        var versions =
fixtures.deuxVersionsPubliees(TypeProjet.PAIEMENT_MOBILE);   // v1 puis v2 : 1
```

```
ajoutée, 1 retirée, 1 modifiée
        UUID l1 = fixtures.liaisonEnHomologation(versions.v1(), "Établissement de
paiement A");
        UUID l2 = fixtures.liaisonEnHomologation(versions.v1(), "Établissement de
paiement C");
        UUID l3 = fixtures.liaisonApte(versions.v1(), "Banque B");

        String simulation =
mvc.perform(post("/api/v1/referentiels/{id}/migrations/simulation", versions.v2())
                .with(utilisateur(CHEF_DES,
"HOM_REFERENTIEL_MIGRATE")).contentType(MediaType.APPLICATION_JSON)
                .content("{\"versionSourceId\":\"" + versions.v1() + "\"}"))
            .andExpect(status().isCreated()).andReturn().getResponse().getContentAsSt
ring();
        UUID migrationId = UUID.fromString(JsonPath.read(simulation, "$.id"));

        mvc.perform(post("/api/v1/referentiels/{id}/migrations", versions.v2())
                .with(utilisateur(CHEF_DES,
"HOM_REFERENTIEL_MIGRATE")).contentType(MediaType.APPLICATION_JSON)
                .content("{\"migrationId\":\"%s\",\"liaisonsRetenues\":
[\"%s\",\"%s\",\"%s\"]}".formatted(migrationId, l1, l2, l3)))
            .andExpect(status().isAccepted())
            .andExpect(jsonPath("$.liaisonsEcartees[0].liaisonId").value(l3.toString(
)));

        doThrow(new IllegalStateException("interruption
simulée")).doCallRealMethod().when(coverage).recalculer(l2);
        runner.executerSuivante();   // l1 migrée, l2 annulée : migration INTERROMPUE
        assertThat(etatMigration(migrationId)).isEqualTo("INTERROMPUE");

        runner.executerSuivante();   // reprise : l1 ignorée, l2 migrée
        assertThat(etatMigration(migrationId)).isEqualTo("TERMINEE");
        assertThat(jdbc.queryForList("select liaison_id, code from operation_instance
group by liaison_id, code having count(*) > 1"))
            .isEmpty();
        assertThat(jdbc.queryForObject("select count(*) from outbox_event where
event_type = 'homologation.liaison-migrated.v1' "
            + "and payload ->> 'migrationId' = ?", Integer.class,
migrationId.toString())).isEqualTo(2);
    }

    private String etatMigration(UUID id) {
        return jdbc.queryForObject("select etat from referential_migration where id =
?", String.class, id);
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: HOM-03 Migration contrôlée des liaisons vers une nouvelle version du
référentiel

  Contexte:
    Étant donné un utilisateur authentifié avec le rôle "CHEF_DES"

  Scénario: Nominal : simulation d'impact puis migration
    Étant donné une nouvelle version publiée du référentiel "PAIEMENT_MOBILE" et
trois liaisons en homologation sur la version précédente
    Quand le chef du DES lance la simulation puis approuve la migration de deux
liaisons
    Alors le rapport d'impact indique par liaison les opérations conservées,
ajoutées, retirées et modifiées
    Et les opérations conservées gardent leur état et leurs preuves, les ajoutées
sont instanciées par tri topologique, les retirées sont archivées sans suppression
    Et l'événement "homologation.liaison-migrated.v1" est émis pour chacune des deux
liaisons
```

```
  Scénario: Alternatif : preuve attendue modifiée
    Étant donné une opération close "PM-BR-02" dont la preuve attendue a changé dans
la nouvelle version
    Quand la migration s'applique
    Alors l'opération "PM-BR-02" est à l'état "ROUVERTE" avec le motif "migration de
référentiel"
    Et son responsable et le chef de projet de la liaison sont notifiés

  Scénario: Erreur : liaison engagée dans le circuit de quitus
    Étant donné une liaison de la Banque B déclarée apte dont le circuit de quitus
est ouvert
    Quand le chef du DES l'inclut dans une migration
    Alors la liaison est exclue de la migration avec le motif
"HOM_MIGRATION_LIAISON_IN_QUITUS"
    Et elle ne pourra être migrée qu'après un retour en homologation
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-chef-des
BASE="${GATEWAY_URL}/bff/des/homologation"

# 1. Simulation d'impact de la version cible V2 sur les liaisons de la version V1
MIGRATION_ID=$(curl -s -X POST
"${BASE}/referentiels/${V2_ID}/migrations/simulation" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" \
  -d "{\"versionSourceId\":\"${V1_ID}\"}" | tee /tmp/hom03-simulation.json | jq -
r .id)
jq '.rapport.impacts[] | {liaisonId, motifExclusion, ajoutees: (.ajoutees|length),
aRouvrir}' /tmp/hom03-simulation.json

# 2. Approbation de deux liaisons, attendu 202
curl -s -w "\n%{http_code}\n" -X POST "${BASE}/referentiels/${V2_ID}/migrations" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" \
  -d "{\"migrationId\":\"${MIGRATION_ID}\",\"liaisonsRetenues\":[\"${LIAISON_1}\",\"$
{LIAISON_2}\"]}"

# 3. Rapport consultable (JSON) et exportable (CSV)
curl -s "${BASE}/migrations/${MIGRATION_ID}/rapport" -H "Authorization: Bearer $
{TOKEN}" | jq '.impacts | length'
curl -s -H "Accept: text/csv" "${BASE}/migrations/${MIGRATION_ID}/rapport" -H
"Authorization: Bearer ${TOKEN}" -o rapport-migration.csv

# 4. Refus par permission : un ingénieur d'homologation ne décide pas la migration,
attendu 403
source ./scripts/token.sh test-ingenieur-homologation
curl -s -o /dev/null -w "%{http_code}\n" -X POST
"${BASE}/referentiels/${V2_ID}/migrations/simulation" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" -d
"{\"versionSourceId\":\"${V1_ID}\"}"
```

HOM-04 | Ouvrir une liaison pour une institution et un type de projet

Élément	Valeur
Points	5
Priorité	Must Have
Sprint cible	Sprint 3
Écrans	DES-08, DES-09
Dépendances	HOM-01, PAR-02, PAR-13, IAM-04

Objectif

Ouvrir une liaison qui rattache une institution à l'état « en homologation » dans MS-PARTICIPANT, un type de projet, la version en vigueur du référentiel et un chef de projet habilité, avec le cas échéant la banque partenaire qui porte le participant ; émettre `homologation.liaison-opened.v1`. Résultat vérifiable : une institution candidate ou une seconde liaison active pour le même couple institution et type de projet est refusée (409) sans création.

A. Implémentation

A1. Table `liaison` et unicité des liaisons actives

Une liaison est active tant qu'elle n'est pas close. L'index unique partiel garantit l'unicité institution et type de projet même en cas de requêtes concurrentes ; le cas d'utilisation vérifie la règle en amont pour renvoyer un motif lisible. Le portage est un attribut (`banque_partenaire_id`) et jamais un type de projet.

```
<!-- resources/config/liquibase/changelog/20261005000200_hom04_liaison.xml (en-tête
JHipster conservé) -->
<databaseChangeLog>
  <changeSet id="hom-04-liaison" author="mshomologation">
    <createTable tableName="liaison">
      <column name="id" type="uuid"><constraints primaryKey="true"/></column>
      <column name="type_projet" type="varchar(32)"><constraints
nullable="false"/></column>
      <column name="institution_id" type="uuid"><constraints
nullable="false"/></column>
      <column name="institution_nom" type="varchar(200)"><constraints
nullable="false"/></column>
      <column name="etat" type="varchar(32)"><constraints nullable="false"/></column>
      <column name="chef_de_projet_id" type="uuid"><constraints
nullable="false"/></column>
      <column name="banque_partenaire_id" type="uuid"/>
      <column name="date_ouverture" type="timestamptz"><constraints
nullable="false"/></column>
      <column name="motif_etat" type="varchar(500)"/>
      <column name="cycle_aptitude" type="integer"
defaultValueNumeric="0"><constraints nullable="false"/></column>
      <column name="ordre_instanciation" type="jsonb"/>
      <column name="etat_avant_gel" type="jsonb"/>
      <column name="version_id" type="uuid">
        <constraints nullable="false" foreignKeyName="fk_liaison_version"
references="referential_version(id)"/>
      </column>
    </createTable>
    <sql>
      CREATE UNIQUE INDEX ux_liaison_active ON liaison (institution_id, type_projet)
WHERE etat &lt;&gt; 'CLOSE';
```

```
    </sql>
    <createIndex tableName="liaison" indexName="ix_liaison_version"><column
name="version_id"/></createIndex>
  </changeSet>
</databaseChangeLog>
```

A2. Ports sortants vers MS-PARTICIPANT et MS-IAM, adaptateurs réel et de substitution

`ParticipantDirectoryPort` lit l'institution par l'interface interne `GET /api/v1/institutions/{id}` de MS-PARTICIPANT, avec le jeton de service de `mshomologation` (permission `INTERNAL_PARTICIPANT_READ`). En cas d'indisponibilité, l'ouverture échoue en 503 `HOM_PARTICIPANT_UNAVAILABLE` : aucun état par défaut n'est supposé. `CollaboratorDirectoryPort` lit un collaborateur auprès de MS-IAM (IAM-04) pour vérifier l'habilitation du chef de projet ; ses deux adaptateurs suivent exactement le même schéma. La sélection se fait par `gimac.adapters.participant=real|stub` et `gimac.adapters.iam=real|stub`.

```
// application/port/out/ParticipantDirectoryPort.java
public interface ParticipantDirectoryPort {
    /** Statuts lus tels que publiés par MS-PARTICIPANT (PAR-02) : CANDIDAT,
EN_HOMOLOGATION, ADHERENT, SUSPENDU, RETIRE. */
    record InstitutionView(UUID id, String nom, String statut, String categorie) {}
    InstitutionView getInstitution(UUID institutionId);
}

// application/port/out/CollaboratorDirectoryPort.java
public interface CollaboratorDirectoryPort {
    record CollaboratorView(UUID id, String nomAffiche, Set<String> roles, UUID
institutionId, Set<String> fonctions, boolean actif) {}
    Optional<CollaboratorView> findCollaborator(UUID userId);
}
```

```
// infrastructure/client/ParticipantRestAdapter.java
@Component
@ConditionalOnProperty(name = "gimac.adapters.participant", havingValue = "real")
public class ParticipantRestAdapter implements ParticipantDirectoryPort {

    private final RestClient client;

    public ParticipantRestAdapter(RestClient.Builder builder,
                                  @Value("${gimac.clients.participant.base-url}")
String baseUrl,
                                  ServiceTokenInterceptor serviceToken) {      //
jeton de service du socle
        this.client =
builder.baseUrl(baseUrl).requestInterceptor(serviceToken).build();
    }

    @Override
    public InstitutionView getInstitution(UUID institutionId) {
        try {
            return client.get().uri("/api/v1/institutions/{id}",
institutionId).retrieve()
                .onStatus(s -> s.value() == 404, (req, res) -> {
                    throw
HomologationException.conflict("HOM_LIAISON_INSTITUTION_UNKNOWN",
                        "Institution inconnue de MS-PARTICIPANT",
Map.of("institutionId", institutionId.toString()));
                })
                .body(InstitutionView.class);
        } catch (ResourceAccessException | HttpServerErrorException e) {
            throw HomologationException.unavailable("HOM_PARTICIPANT_UNAVAILABLE",
```

```
                "MS-PARTICIPANT indisponible ; l'ouverture est différée, aucun statut
n'est supposé");
        }
    }
}
```

```
// infrastructure/client/ParticipantStubAdapter.java
@Component
@ConditionalOnProperty(name = "gimac.adapters.participant", havingValue = "stub")
public class ParticipantStubAdapter implements ParticipantDirectoryPort {

    private final Map<UUID, InstitutionView> institutions = new
ConcurrentHashMap<>();

    /** Alimenté par les jeux de données de recette (profil dev) et par les tests
d'intégration. */
    public void enregistrer(InstitutionView institution)
{ institutions.put(institution.id(), institution); }

    @Override
    public InstitutionView getInstitution(UUID institutionId) {
        return Optional.ofNullable(institutions.get(institutionId)).orElseThrow(() ->
            HomologationException.conflict("HOM_LIAISON_INSTITUTION_UNKNOWN",
"Institution inconnue",
                Map.of("institutionId", institutionId.toString())));
    }
}
```

A3. Règles d'ouverture (service de domaine pur) et cas d'utilisation

La politique vérifie l'éligibilité de l'institution, la nature bancaire du porteur et l'habilitation du chef de projet (collaborateur actif du DES portant le rôle `INGENIEUR_HOMOLOGATION` ou `CHEF_DES`). Le cas d'utilisation lie la liaison à la version publiée en vigueur pour le type de projet ; cette version reste celle de la liaison jusqu'à une migration (HOM-03). La violation de l'index unique par une requête concurrente est traduite dans le même motif que le contrôle amont.

```
// domain/service/LiaisonOpeningPolicy.java
public final class LiaisonOpeningPolicy {

    private static final Set<String> ROLES_CHEF_DE_PROJET =
Set.of("INGENIEUR_HOMOLOGATION", "CHEF_DES");

    public void verifier(InstitutionView institution, InstitutionView porteur,
CollaboratorView chefDeProjet) {
        if (!"EN_HOMOLOGATION".equals(institution.statut())) {
            throw
HomologationException.conflict("HOM_LIAISON_INSTITUTION_NOT_ELIGIBLE",
                "L'institution n'est pas à l'état « en homologation »",
                Map.of("institutionId", institution.id().toString(), "statut",
institution.statut()));
        }
        if (porteur != null && !"BANQUE".equals(porteur.categorie())) {
            throw HomologationException.conflict("HOM_LIAISON_CARRIER_NOT_BANK",
                "Le porteur désigné n'est pas une banque",
Map.of("banquePartenaireId", porteur.id().toString()));
        }
        verifierChefDeProjet(chefDeProjet);
    }

    public void verifierChefDeProjet(CollaboratorView c) {
        if (c == null || !c.actif() || c.institutionId() != null ||
Collections.disjoint(c.roles(), ROLES_CHEF_DE_PROJET)) {
            throw
HomologationException.conflict("HOM_LIAISON_PROJECT_LEAD_NOT_HABILITATED",
```

```
                "Le chef de projet doit être un collaborateur habilité du DES",
Map.of());
        }
    }
}
```

```
// application/usecase/OpenLiaisonService.java
@Service
@Transactional
@RequiredArgsConstructor
public class OpenLiaisonService implements OpenLiaisonUseCase {

    private final ParticipantDirectoryPort participants;
    private final CollaboratorDirectoryPort collaborators;
    private final LiaisonOpeningPolicy policy;
    private final LiaisonRepository liaisons;
    private final ReferentialVersionRepository versions;
    private final OutboxPublisher outboxPublisher;
    private final Clock clock;

    @Override
    public Liaison ouvrir(OpenLiaisonCommand cmd) {
        InstitutionView institution =
participants.getInstitution(cmd.institutionId());
        InstitutionView porteur = cmd.banquePartenaireId() == null ? null :
participants.getInstitution(cmd.banquePartenaireId());
        policy.verifier(institution, porteur,
collaborators.findCollaborator(cmd.chefDeProjetId()).orElse(null));
        if
(liaisons.existsByInstitutionIdAndTypeProjetAndEtatNot(cmd.institutionId(),
cmd.typeProjet(), EtatLiaison.CLOSE)) {
            throw doublon(cmd);
        }
        ReferentialVersion enVigueur =
versions.findFirstByTypeProjetAndEtat(cmd.typeProjet(),
EtatVersionReferentiel.PUBLIEE)
            .orElseThrow(() ->
HomologationException.conflict("HOM_REFERENTIAL_NONE_PUBLISHED",
                "Aucune version publiée pour ce type de projet", Map.of("typeProjet",
cmd.typeProjet().name())));
        Liaison l = Liaison.ouvrir(institution.id(), institution.nom(),
cmd.typeProjet(), enVigueur,
                                   cmd.chefDeProjetId(), cmd.banquePartenaireId(),
clock.instant());
        try {
            liaisons.saveAndFlush(l);
        } catch (DataIntegrityViolationException e) {       // ux_liaison_active :
ouverture concurrente
            throw doublon(cmd);
        }
        outboxPublisher.publish("liaison", l.getId().toString(),
"homologation.liaison-opened.v1",
            new LiaisonOpenedPayload(l.getId(), l.getInstitutionId(),
l.getTypeProjet(), enVigueur.getId(),
                                     enVigueur.getNumero(), l.getChefDeProjetId(),
l.getBanquePartenaireId()));
        return l;
    }

    @Override
    public Liaison designerChefDeProjet(UUID liaisonId, UUID chefDeProjetId) {

policy.verifierChefDeProjet(collaborators.findCollaborator(chefDeProjetId).orElse(nul
l));
        Liaison l = liaisons.findByIdForUpdate(liaisonId)
            .orElseThrow(() ->
HomologationException.notFound("HOM_LIAISON_NOT_FOUND", liaisonId));
        l.designerChefDeProjet(chefDeProjetId);
        return l;
```

```
    }

    private static HomologationException doublon(OpenLiaisonCommand cmd) {
        return HomologationException.conflict("HOM_LIAISON_DUPLICATE",
            "Une liaison active existe déjà pour cette institution et ce type de
projet",
            Map.of("institutionId", cmd.institutionId().toString(), "typeProjet",
cmd.typeProjet().name()));
    }
}
```

A4. Ressource REST

Méthode et chemin	Permission	Réponse
`POST /liaisons`	`HOM_LIAISON_OPEN`	201, 409, 503
`GET /liaisons/{id}`	`HOM_LIAISON_READ`	200 : liaison, `intervenants`, `transitionsPermises`, `actionsPermises`
`PUT /liaisons/{id}/chef-de-projet`	`HOM_LIAISON_OPEN`	200, 409

```
// web/rest/LiaisonResource.java
@RestController
@RequestMapping("/api/v1/liaisons")
@RequiredArgsConstructor
public class LiaisonResource {

    private final OpenLiaisonUseCase useCase;
    private final LiaisonQueryService queries;   // assemble la fiche et la liste des
intervenants

    @PostMapping
    @PreAuthorize("hasAuthority('HOM_LIAISON_OPEN')")
    public ResponseEntity<LiaisonDTO> ouvrir(@Valid @RequestBody OpenLiaisonCommand
cmd) {
        Liaison l = useCase.ouvrir(cmd);
        return ResponseEntity.created(URI.create("/api/v1/liaisons/" +
l.getId())).body(queries.fiche(l.getId()));
    }

    @GetMapping("/{id}")
    @PreAuthorize("hasAuthority('HOM_LIAISON_READ')")
    public LiaisonDTO lire(@PathVariable UUID id) {
        return queries.fiche(id);
    }

    @PutMapping("/{id}/chef-de-projet")
    @PreAuthorize("hasAuthority('HOM_LIAISON_OPEN')")
    public LiaisonDTO designer(@PathVariable UUID id, @Valid @RequestBody
DesignationRequest req) {
        useCase.designerChefDeProjet(id, req.chefDeProjetId());
        return queries.fiche(id);
    }
}
```

`LiaisonDTO.intervenants` liste l'institution (`INSTITUTION`), la banque partenaire (`BANQUE_PARTENAIRE`) lorsqu'elle existe et le chef de projet (`CHEF_DE_PROJET`) ; c'est cette liste que sert aussi la route interne `/liaisons/{id}/intervenants`.

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
//
```

```
src/test/java/org/gimac/des/homologation/application/usecase/OpenLiaisonServiceTest.j
ava
@ExtendWith(MockitoExtension.class)
class OpenLiaisonServiceTest {

    static final UUID EP_A = UUID.randomUUID(), BANQUE_B = UUID.randomUUID(), CHEF =
UUID.randomUUID();

    @Mock ParticipantDirectoryPort participants;
    @Mock CollaboratorDirectoryPort collaborators;
    @Mock LiaisonRepository liaisons;
    @Mock ReferentialVersionRepository versions;
    @Mock OutboxPublisher outboxPublisher;
    OpenLiaisonService service;

    @BeforeEach
    void init() {
        service = new OpenLiaisonService(participants, collaborators, new
LiaisonOpeningPolicy(), liaisons, versions,
            outboxPublisher, Clock.fixed(Instant.parse("2026-10-06T09:00:00Z"),
ZoneOffset.UTC));
        when(participants.getInstitution(EP_A)).thenReturn(new InstitutionView(EP_A,
"Établissement de paiement A",
            "EN_HOMOLOGATION", "ETABLISSEMENT_PAIEMENT"));
        when(collaborators.findCollaborator(CHEF)).thenReturn(Optional.of(new
CollaboratorView(CHEF, "Ingénieur 1",
            Set.of("INGENIEUR_HOMOLOGATION"), null, Set.of(), true)));
    }

    @Test
    @DisplayName("HOM-04 : une seconde liaison active pour la même institution et le
même type est refusée")
    void uniciteInstitutionEtType() {
        when(liaisons.existsByInstitutionIdAndTypeProjetAndEtatNot(EP_A,
TypeProjet.PAIEMENT_MOBILE, EtatLiaison.CLOSE))
            .thenReturn(true);

        assertThatThrownBy(() -> service.ouvrir(new OpenLiaisonCommand(EP_A,
TypeProjet.PAIEMENT_MOBILE, CHEF, null)))
            .isInstanceOfSatisfying(HomologationException.class, ex ->
assertThat(ex.code()).isEqualTo("HOM_LIAISON_DUPLICATE"));
        verify(liaisons, never()).saveAndFlush(any());
        verifyNoInteractions(outboxPublisher);
    }

    @Test
    @DisplayName("HOM-04 : le portage par une banque partenaire ne change pas le type
de projet")
    void portageConserveLeTypeDeProjet() {
        when(participants.getInstitution(BANQUE_B)).thenReturn(new
InstitutionView(BANQUE_B, "Banque B", "ADHERENT", "BANQUE"));
        when(versions.findFirstByTypeProjetAndEtat(TypeProjet.PAIEMENT_MOBILE,
EtatVersionReferentiel.PUBLIEE))
            .thenReturn(Optional.of(ReferentialVersionSamples.publiee(TypeProjet.PAIE
MENT_MOBILE, 1)));

        Liaison l = service.ouvrir(new OpenLiaisonCommand(EP_A,
TypeProjet.PAIEMENT_MOBILE, CHEF, BANQUE_B));

        assertThat(l.getTypeProjet()).isEqualTo(TypeProjet.PAIEMENT_MOBILE);
        assertThat(l.getBanquePartenaireId()).isEqualTo(BANQUE_B);
        assertThat(l.getEtat()).isEqualTo(EtatLiaison.OUVERTE);
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
// src/test/java/org/gimac/des/homologation/web/rest/LiaisonOpeningIT.java
class LiaisonOpeningIT extends AbstractHomologationIT {

    private static final UUID INGENIEUR = UUID.fromString("00000000-0000-0000-0000-
00000000e001");

    @Autowired ParticipantStubAdapter participantStub;

    @Test
    @DisplayName("HOM-04 : l'ouverture est refusée pour une institution à l'état
candidat, sans liaison ni outbox")
    void refusInstitutionCandidate() throws Exception {
        UUID candidate = UUID.randomUUID();
        participantStub.enregistrer(new InstitutionView(candidate, "Établissement de
paiement D", "CANDIDAT", "ETABLISSEMENT_PAIEMENT"));
        fixtures.referentielPublie(TypeProjet.PAIEMENT_MOBILE);

        mvc.perform(post("/api/v1/liaisons").with(utilisateur(INGENIEUR,
"HOM_LIAISON_OPEN"))
                .contentType(MediaType.APPLICATION_JSON)
                .content("""

{"institutionId":"%s","typeProjet":"PAIEMENT_MOBILE","chefDeProjetId":"%s"}
                    """.formatted(candidate, fixtures.chefDeProjetHabilite())))
            .andExpect(status().isConflict())
            .andExpect(jsonPath("$.code").value("HOM_LIAISON_INSTITUTION_NOT_ELIGIBLE
"))
            .andExpect(jsonPath("$.details.statut").value("CANDIDAT"));

        assertThat(jdbc.queryForObject("select count(*) from liaison where
institution_id = ?", Integer.class, candidate)).isZero();
        assertThat(jdbc.queryForObject("select count(*) from outbox_event where
event_type = 'homologation.liaison-opened.v1' "
            + "and payload ->> 'institutionId' = ?", Integer.class,
candidate.toString())).isZero();
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: HOM-04 Ouverture d'une liaison

  Contexte:
    Étant donné un utilisateur authentifié avec le rôle "INGENIEUR_HOMOLOGATION"
    Et une version publiée du référentiel "PAIEMENT_MOBILE"

  Scénario: Nominal : ouverture d'une liaison
    Étant donné l'Établissement de paiement A à l'état "EN_HOMOLOGATION" dans MS-
PARTICIPANT
    Quand l'ingénieur d'homologation ouvre une liaison de type "PAIEMENT_MOBILE" en
désignant un chef de projet habilité
    Alors la liaison est créée à l'état "OUVERTE" et liée à la version en vigueur du
référentiel, qui reste celle de la liaison jusqu'à une éventuelle migration
    Et l'événement "homologation.liaison-opened.v1" est émis

  Scénario: Alternatif : participant porté par une banque partenaire
    Étant donné un établissement de paiement à l'état "EN_HOMOLOGATION" porté par la
Banque B
    Quand la liaison est ouverte avec l'attribut de portage
    Alors le type de projet de la liaison reste "PAIEMENT_MOBILE"
    Et la Banque B figure parmi les intervenants de la liaison avec le rôle
"BANQUE_PARTENAIRE"

  Plan du scénario: Erreur : institution non éligible ou liaison en double
```

```
    Étant donné <situation>
    Quand l'ouverture est demandée
    Alors l'ouverture est refusée avec le code "<code>"
    Et aucune liaison n'est créée

    Exemples:
      | situation
| code                                  |
      | une institution à l'état "CANDIDAT"
| HOM_LIAISON_INSTITUTION_NOT_ELIGIBLE |
      | une liaison déjà ouverte pour l'Établissement de paiement A et le type
"PAIEMENT_MOBILE"        | HOM_LIAISON_DUPLICATE                |
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-ingenieur-homologation
BASE="${GATEWAY_URL}/bff/des/homologation"

# 1. Ouverture d'une liaison portée par une banque partenaire
LIAISON_ID=$(curl -s -X POST "${BASE}/liaisons" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" \
  -d "{\"institutionId\":\"${EP_A_ID}\",\"typeProjet\":\"PAIEMENT_MOBILE\",
       \"chefDeProjetId\":\"${CHEF_PROJET_ID}\",\"banquePartenaireId\":\"$
{BANQUE_B_ID}\"}" | jq -r .id)

# 2. Fiche de la liaison : état, version, intervenants et actions permises
curl -s "${BASE}/liaisons/${LIAISON_ID}" -H "Authorization: Bearer ${TOKEN}" \
  | jq '{etat, typeProjet, versionNumero, intervenants: [.intervenants[].role],
actionsPermises}'

# 3. Refus : seconde liaison pour le même couple institution et type, attendu 409
HOM_LIAISON_DUPLICATE
curl -s -o /tmp/hom04.json -w "%{http_code}\n" -X POST "${BASE}/liaisons" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" \
  -d "{\"institutionId\":\"$
{EP_A_ID}\",\"typeProjet\":\"PAIEMENT_MOBILE\",\"chefDeProjetId\":\"$
{CHEF_PROJET_ID}\"}"
jq -r .code /tmp/hom04.json

# 4. Refus par permission : un correspondant n'ouvre pas de liaison, attendu 403
source ./scripts/token.sh test-correspondant-a
curl -s -o /dev/null -w "%{http_code}\n" -X POST "${BASE}/liaisons" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" \
  -d "{\"institutionId\":\"$
{EP_A_ID}\",\"typeProjet\":\"PAIEMENT_MOBILE\",\"chefDeProjetId\":\"$
{CHEF_PROJET_ID}\"}"
```

HOM-05 | Dériver le périmètre contractualisé des services souscrits

Élément	Valeur
Points	5
Priorité	Must Have
Sprint cible	Sprint 3
Écrans	DES-09
Dépendances	HOM-04, PAR-05, SOC-05

Objectif

Dériver sans saisie, à chaque `participant.subscription-changed.v1`, le périmètre contractualisé de toutes les liaisons non closes de l'institution : un élément de périmètre par service, à engager s'il est souscrit, hors périmètre sinon, avec propagation aux opérations et sans jamais supprimer une opération close ni une preuve. Résultat vérifiable : la redélivrance d'un même événement ne produit aucun effet supplémentaire, et un événement non conforme au schéma est placé au rebut.

A. Implémentation

A1. Tables `contractual_scope`, `scope_item` et `rejected_record`

Le périmètre est lié un à un à la liaison et garde l'identifiant de l'événement source, ce qui permet de retrouver la souscription à l'origine de chaque état. La table `rejected_record` (JDL) est introduite par ce récit, premier consommateur du service ; HOM-14 lui ajoute l'écran de rebut.

```
<!-- resources/config/liquibase/changelog/20261005000300_hom05_perimetre.xml (en-tête
JHipster conservé) -->
<databaseChangeLog>
  <changeSet id="hom-05-perimetre" author="mshomologation">
    <createTable tableName="contractual_scope">
      <column name="id" type="uuid"><constraints primaryKey="true"/></column>
      <column name="liaison_id" type="uuid">
        <constraints nullable="false" unique="true" foreignKeyName="fk_scope_liaison"
references="liaison(id)"/>
      </column>
      <column name="date_derivation" type="timestamptz"/>
      <column name="source_event_id" type="uuid"/>
    </createTable>
    <createTable tableName="scope_item">
      <column name="id" type="uuid"><constraints primaryKey="true"/></column>
      <column name="scope_id" type="uuid">
        <constraints nullable="false" foreignKeyName="fk_scope_item_scope"
references="contractual_scope(id)"/>
      </column>
      <column name="service_code" type="varchar(40)"><constraints
nullable="false"/></column>
      <column name="service_libelle" type="varchar(200)"/>
      <column name="etat" type="varchar(20)"><constraints nullable="false"/></column>
    </createTable>
    <addUniqueConstraint tableName="scope_item" columnNames="scope_id, service_code"
constraintName="ux_scope_item_service"/>
    <createTable tableName="rejected_record">
      <column name="id" type="uuid"><constraints primaryKey="true"/></column>
      <column name="source" type="varchar(120)"><constraints
nullable="false"/></column>
```

```
      <column name="charge_utile" type="text"><constraints
nullable="false"/></column>
      <column name="empreinte" type="char(64)"><constraints
nullable="false"/></column>
      <column name="motif" type="varchar(500)"><constraints
nullable="false"/></column>
      <column name="date_rejet" type="timestamptz"><constraints
nullable="false"/></column>
      <column name="relance" type="boolean" defaultValueBoolean="false"/>
    </createTable>
  </changeSet>
</databaseChangeLog>
```

A2. Dérivation du périmètre (service de domaine pur)

L'univers des services est la réunion des services portés par la version du référentiel de la liaison, des services souscrits et des éléments déjà connus. Règles retenues : le service réservé `TRANSVERSE` est toujours dans le périmètre ; un service souscrit qui entre au périmètre prend l'état `A_ENGAGER`, un service déjà engagé garde son état d'avancement ; un service non souscrit passe `HORS_PERIMETRE`. Un service souscrit sans aucune opération dans la version entre au périmètre : HOM-17 le déclarera non calculable (« référentiel incomplet ») au lieu de l'ignorer.

```
// domain/service/ScopeDerivation.java
public final class ScopeDerivation {

    public static final String TRANSVERSE = "TRANSVERSE";

    public record Resultat(Map<String, EtatScopeItem> etats, Set<String> entrants,
Set<String> sortants) {}

    public Resultat deriver(Set<String> servicesReferentiel, Set<String> souscrits,
Map<String, EtatScopeItem> actuels) {
        Set<String> univers = new TreeSet<>(servicesReferentiel);
        univers.addAll(souscrits);
        univers.addAll(actuels.keySet());
        Map<String, EtatScopeItem> etats = new TreeMap<>();
        Set<String> entrants = new TreeSet<>(), sortants = new TreeSet<>();
        for (String service : univers) {
            EtatScopeItem avant = actuels.get(service);
            boolean dansPerimetre = TRANSVERSE.equals(service) ||
souscrits.contains(service);
            if (dansPerimetre) {
                boolean entrant = avant == null || avant ==
EtatScopeItem.HORS_PERIMETRE;
                etats.put(service, entrant ? EtatScopeItem.A_ENGAGER : avant);
                if (entrant) entrants.add(service);
            } else {
                etats.put(service, EtatScopeItem.HORS_PERIMETRE);
                if (avant != null && avant != EtatScopeItem.HORS_PERIMETRE)
sortants.add(service);
            }
        }
        return new Resultat(Map.copyOf(etats), Set.copyOf(entrants),
Set.copyOf(sortants));
    }
}
```

A3. Consommateur idempotent, contrôle de schéma et rebut

Le sujet `participant.subscription` multiplexe plusieurs types : seuls les événements `participant.subscription-changed.v1` sont traités. La charge utile est validée contre le schéma versionné `contracts/participant.subscription-changed.v1.schema.json`

(celui du test de contrat, B) ; un événement non conforme est écrit dans `rejected_record` avec son empreinte SHA-256 puis publié sur `participant.subscription.dlq`. L'enregistrement dans `processed_event` et l'effet métier partagent la transaction du consommateur.

```
// infrastructure/messaging/consumer/SubscriptionChangedConsumer.java
@Component
@RequiredArgsConstructor
public class SubscriptionChangedConsumer {

    static final String GROUP = "mshomologation-subscription";
    static final String TYPE = "participant.subscription-changed.v1";

    private final IdempotentConsumer idempotentConsumer;
    private final EventSchemaValidator schemas;          // schémas JSON chargés
depuis classpath:contracts/
    private final RejectedEventRouter rejets;
    private final DeriveScopeUseCase useCase;
    private final ObjectMapper json;

    @KafkaListener(topics = "participant.subscription", groupId = GROUP)
    @Transactional
    public void onMessage(ConsumerRecord<String, String> record) {
        CloudEventEnvelope event;
        SubscriptionChangedPayload payload;
        try {
            event = json.readValue(record.value(), CloudEventEnvelope.class);
            if (!TYPE.equals(event.type())) return;                      // autre
type du même sujet
            schemas.valider(TYPE, event.data());                         // lève
InvalidEventException
            payload = json.treeToValue(event.data(),
SubscriptionChangedPayload.class);
        } catch (InvalidEventException | JsonProcessingException e) {
            rejets.rejeter(record, e.getMessage());
            return;
        }
        idempotentConsumer.handleOnce(event.id(), GROUP, () ->
useCase.appliquer(payload, event.id()));
    }
}
```

```
// infrastructure/messaging/consumer/RejectedEventRouter.java
@Component
@RequiredArgsConstructor
public class RejectedEventRouter {

    private final RejectedRecordRepository rejected;
    private final KafkaTemplate<String, String> kafka;
    private final Clock clock;

    public void rejeter(ConsumerRecord<String, String> record, String motif) {
        String empreinte = DigestUtils.sha256Hex(record.value());
        rejected.save(RejectedRecord.nouveau(record.topic(), record.value(),
empreinte, motif, clock.instant()));
        kafka.send(record.topic() + ".dlq", record.key(), record.value());
    }
}
```

A4. Cas d'utilisation : recalcul du périmètre et propagation

Pour chaque liaison non close de l'institution, le périmètre est recalculé puis propagé aux opérations par `OperationGraphService.applyScope` (HOM-07) : à la première dérivation, cet appel réalise l'instanciation initiale ; ensuite, il fait passer hors périmètre les opérations non closes des services sortants et instancie les opérations des services entrants selon leurs

prédécesseurs. Une opération close d'un service sortant garde son état et ses preuves. Sur une liaison gelée, seule la mise à jour du périmètre est faite ; la propagation aux opérations est différée au dégel (HOM-06). L'événement `homologation.coverage-changed.v1` porte en sprint 3 la liste des services impactés et la cause `PERIMETRE` ; HOM-17 l'enrichit des taux recalculés. MS-AUDIT consommant tous les sujets, l'écart d'un retrait partiellement prouvé est consigné par l'événement de notification qui porte la liste des opérations closes concernées.

```
// application/usecase/DeriveScopeService.java
@Service
@Transactional
@RequiredArgsConstructor
public class DeriveScopeService implements DeriveScopeUseCase {

    private final LiaisonRepository liaisons;
    private final ContractualScopeRepository scopes;
    private final OperationInstanceRepository operations;
    private final ScopeDerivation derivation;
    private final OperationGraphService operationGraph;       // HOM-07
    private final NotificationRequester notifications;
    private final OutboxPublisher outboxPublisher;
    private final Clock clock;

    @Override
    public void appliquer(SubscriptionChangedPayload p, UUID eventId) {
        for (Liaison l : liaisons.findByInstitutionIdAndEtatNot(p.institutionId(),
EtatLiaison.CLOSE)) {
            ContractualScope scope = scopes.findByLiaisonId(l.getId()).orElseGet(() -
> scopes.save(ContractualScope.pour(l)));
            boolean initial = scope.getItems().isEmpty();
            var r = derivation.deriver(l.getVersion().codesDeService(),
p.codesSouscrits(), scope.etatsParService());
            scope.appliquer(r.etats(), p.libellesParCode(), eventId,
clock.instant());
            if (l.getEtat() != EtatLiaison.GELEE) {
                operationGraph.applyScope(l);
            }
            Set<String> impactes = new TreeSet<>(r.entrants());
            impactes.addAll(r.sortants());
            if (impactes.isEmpty()) continue;
            outboxPublisher.publish("coverage", l.getId().toString(),
"homologation.coverage-changed.v1",
                new CoverageChangedPayload(l.getId(), List.copyOf(impactes),
"PERIMETRE"));
            if (!initial) {
                List<String> closesSortantes =
operations.codesClosParServices(l.getId(), r.sortants());
                notifications.demander(closesSortantes.isEmpty() ?
"HOM_PERIMETRE_MODIFIE" : "HOM_PERIMETRE_RETRAIT_PARTIEL",
                    Destinataires.utilisateurs(l.getChefDeProjetId()),
                    Map.of("liaisonId", l.getId(), "entrants", r.entrants(),
"sortants", r.sortants(),
                           "operationsClosesConservees", closesSortantes));
            }
        }
    }
}
```

```
// application/usecase/NotificationRequester.java
// Point unique d'émission des demandes de notification (consommées par MS-
NOTIFICATION, NOT-14).
@Component
@RequiredArgsConstructor
public class NotificationRequester {
```

```
    private final OutboxPublisher outboxPublisher;

    public void demander(String modele, List<Destinataires.Destinataire>
destinataires, Map<String, Object> parametres) {
        UUID demandeId = UUID.randomUUID();
        outboxPublisher.publish("notification", demandeId.toString(),
"homologation.notification-requested.v1",
            new NotificationRequestedPayload(demandeId, modele, destinataires,
parametres));
    }
}

// domain/event/Destinataires.java
public final class Destinataires {
    /** type : UTILISATEUR (identifiant de collaborateur) ou
CORRESPONDANTS_INSTITUTION (identifiant d'institution). */
    public record Destinataire(String type, UUID id) {}

    public static List<Destinataire> utilisateurs(UUID... ids) {
        return Arrays.stream(ids).filter(Objects::nonNull).distinct().map(id -> new
Destinataire("UTILISATEUR", id)).toList();
    }
    public static List<Destinataire> correspondants(UUID institutionId) {
        return List.of(new Destinataire("CORRESPONDANTS_INSTITUTION",
institutionId));
    }
    public static List<Destinataire>
responsablesEtChefDeProjet(Collection<OperationInstance> ops, UUID chefDeProjetId) {
        return
utilisateurs(Stream.concat(ops.stream().map(OperationInstance::responsableActifId),
Stream.of(chefDeProjetId))
            .toArray(UUID[]::new));
    }
}
```

A5. Vue du périmètre en lecture seule

`GET /liaisons/{id}/perimetre` (permission `HOM_LIAISON_READ`) renvoie la date de dérivation, l'événement source et, par service, le libellé et l'état ; `actionsPermises` est toujours vide, car toute correction passe par MS-PARTICIPANT. Aucune route d'écriture n'est exposée.

```
// web/rest/ScopeResource.java
@RestController
@RequestMapping("/api/v1/liaisons/{id}/perimetre")
@RequiredArgsConstructor
public class ScopeResource {

    private final ContractualScopeRepository scopes;

    @GetMapping
    @PreAuthorize("hasAuthority('HOM_LIAISON_READ')")
    public PerimetreDTO lire(@PathVariable UUID id) {
        return scopes.findByLiaisonId(id).map(PerimetreDTO::of)
            .orElseThrow(() ->
HomologationException.notFound("HOM_SCOPE_NOT_DERIVED", id));
    }
}
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
// src/test/java/org/gimac/des/homologation/domain/service/ScopeDerivationTest.java
class ScopeDerivationTest {

    private final ScopeDerivation derivation = new ScopeDerivation();
```

```
    private final Set<String> servicesVersion = Set.of("TRANSVERSE", "TRANSFERT",
"PAIEMENT_MARCHAND", "RETRAIT");

    @Test
    @DisplayName("HOM-05 : les services souscrits sont à engager, les autres hors
périmètre, le transverse toujours inclus")
    void derivationInitiale() {
        var r = derivation.deriver(servicesVersion, Set.of("TRANSFERT", "RETRAIT"),
Map.of());
        assertThat(r.etats()).containsExactlyInAnyOrderEntriesOf(Map.of(
            "TRANSVERSE", EtatScopeItem.A_ENGAGER, "TRANSFERT",
EtatScopeItem.A_ENGAGER,
            "RETRAIT", EtatScopeItem.A_ENGAGER, "PAIEMENT_MARCHAND",
EtatScopeItem.HORS_PERIMETRE));
        assertThat(r.sortants()).isEmpty();
    }

    @Test
    @DisplayName("HOM-05 : un retrait fait sortir le service et un service engagé
garde son avancement")
    void retraitEtConservation() {
        var actuels = Map.of("TRANSVERSE", EtatScopeItem.EN_COURS, "TRANSFERT",
EtatScopeItem.COUVERT,
                             "RETRAIT", EtatScopeItem.EN_COURS, "PAIEMENT_MARCHAND",
EtatScopeItem.HORS_PERIMETRE);
        var r = derivation.deriver(servicesVersion, Set.of("TRANSFERT",
"PAIEMENT_MARCHAND"), actuels);
        assertThat(r.etats()).containsEntry("TRANSFERT",
EtatScopeItem.COUVERT).containsEntry("RETRAIT", EtatScopeItem.HORS_PERIMETRE);
        assertThat(r.entrants()).containsExactly("PAIEMENT_MARCHAND");
        assertThat(r.sortants()).containsExactly("RETRAIT");
    }
}
```

```
// src/test/java/org/gimac/des/homologation/infrastructure/messaging/consumer/
SubscriptionChangedContractTest.java
class SubscriptionChangedContractTest {

    private final EventSchemaValidator schemas =
EventSchemaValidator.depuisClasspath("contracts/");
    private final ObjectMapper json = new ObjectMapper();

    @Test
    @DisplayName("HOM-05 : l'exemple publié par MS-PARTICIPANT respecte le schéma de
participant.subscription-changed.v1")
    void exempleConforme() throws Exception {
        JsonNode data =
json.readTree(getClass().getResource("/contracts/samples/participant.subscription-
changed.v1.json"));
        assertThatCode(() -> schemas.valider("participant.subscription-changed.v1",
data)).doesNotThrowAnyException();
    }

    @Test
    @DisplayName("HOM-05 : une charge utile sans identifiant d'institution est
rejetée par le schéma")
    void chargeUtileNonConforme() throws Exception {
        JsonNode data = json.readTree("{\"servicesSouscrits\":
[{\"code\":\"TRANSFERT\",\"libelle\":\"Transfert\"}]}");
        assertThatThrownBy(() -> schemas.valider("participant.subscription-
changed.v1", data))
            .isInstanceOf(InvalidEventException.class).hasMessageContaining("institut
ionId");
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
// src/test/java/org/gimac/des/homologation/infrastructure/messaging/consumer/
SubscriptionChangedConsumerIT.java
class SubscriptionChangedConsumerIT extends AbstractHomologationIT {

    @Autowired KafkaTemplate<String, String> kafka;

    @Test
    @DisplayName("HOM-05 : la redélivrance de l'événement de souscription ne produit
aucun effet supplémentaire")
    void redelivranceSansEffet() {
        UUID institution = UUID.randomUUID();
        UUID liaisonId = fixtures.liaisonOuverte(TypeProjet.PAIEMENT_MOBILE,
institution);   // version : 4 services
        var evenement = fixtures.cloudEvent("participant.subscription-changed.v1",
institution,   // identifiant et JSON
            Map.of("institutionId", institution, "servicesSouscrits",
                  List.of(Map.of("code", "TRANSFERT", "libelle", "Transfert"),
Map.of("code", "RETRAIT", "libelle", "Retrait"))));

        kafka.send("participant.subscription", institution.toString(),
evenement.json());
        await().atMost(Duration.ofSeconds(15)).untilAsserted(() ->
            assertThat(compter("select count(*) from scope_item si join
contractual_scope cs on cs.id = si.scope_id where cs.liaison_id = ?",
                liaisonId)).isEqualTo(4));

        kafka.send("participant.subscription", institution.toString(),
evenement.json());   // redélivrance à l'identique

await().during(Duration.ofSeconds(3)).atMost(Duration.ofSeconds(6)).untilAsserted(()
-> {
            assertThat(compter("select count(*) from processed_event where event_id =
?", evenement.id())).isEqualTo(1);
            assertThat(compter("select count(*) from outbox_event where aggregate_id
= ? and event_type = 'homologation.coverage-changed.v1'",
                liaisonId.toString())).isEqualTo(1);
        });
        assertThat(jdbc.queryForObject("select si.etat from scope_item si join
contractual_scope cs on cs.id = si.scope_id "
            + "where cs.liaison_id = ? and si.service_code = 'PAIEMENT_MARCHAND'",
String.class, liaisonId))
            .isEqualTo("HORS_PERIMETRE");
    }

    private int compter(String sql, Object arg) {
        return jdbc.queryForObject(sql, Integer.class, arg);
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: HOM-05 Dérivation du périmètre contractualisé

  Contexte:
    Étant donné une version du référentiel "PAIEMENT_MOBILE" portant les services
"TRANSFERT", "PAIEMENT_MARCHAND" et "RETRAIT"

  Scénario: Nominal : périmètre initial
    Étant donné une liaison ouverte de l'Établissement de paiement A et l'événement
"participant.subscription-changed.v1" de son institution souscrivant "TRANSFERT" et
"RETRAIT"
    Quand l'événement est consommé
    Alors les services "TRANSFERT" et "RETRAIT" entrent au périmètre à l'état
"A_ENGAGER" et le service "PAIEMENT_MARCHAND" est marqué "HORS_PERIMETRE"
```

```
    Et les opérations rattachées au service "PAIEMENT_MARCHAND" prennent l'état
"HORS_PERIMETRE"

  Scénario: Alternatif : souscription d'un service en cours de liaison
    Étant donné une liaison en homologation dont le service "PAIEMENT_MARCHAND" est
hors périmètre
    Quand un nouvel événement ajoute "PAIEMENT_MARCHAND" aux services souscrits
    Alors le service entre au périmètre et ses opérations sont instanciées selon
leurs prédécesseurs
    Et l'événement "homologation.coverage-changed.v1" est émis et le chef de projet
est notifié

  Scénario: Erreur : retrait d'un service déjà partiellement prouvé
    Étant donné le service "RETRAIT" dont des opérations sont closes
    Quand un événement retire "RETRAIT" des services souscrits
    Alors le service passe hors périmètre sans qu'aucune opération close ni aucune
preuve ne soit supprimée
    Et l'écart est signalé au chef de projet et consigné au registre d'audit
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-ingenieur-homologation
BASE="${GATEWAY_URL}/bff/des/homologation"

# 1. Périmètre dérivé : état par service, événement source et absence d'action
curl -s "${BASE}/liaisons/${LIAISON_ID}/perimetre" -H "Authorization: Bearer $
{TOKEN}" \
  | jq '{dateDerivation, sourceEventId, services: [.items[] | {serviceCode, etat}],
actionsPermises}'

# 2. Opérations devenues hors périmètre après dérivation
curl -s "${BASE}/liaisons/${LIAISON_ID}/operations?etat=HORS_PERIMETRE" -H
"Authorization: Bearer ${TOKEN}" \
  | jq '[.[] | {code, serviceCode}]'

# 3. Refus : le périmètre n'est pas modifiable par la plateforme ; aucune règle OPA
ni route d'écriture, attendu 403
curl -s -o /dev/null -w "%{http_code}\n" -X PUT
"${BASE}/liaisons/${LIAISON_ID}/perimetre" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" -d
'{"items":[]}'

# 4. Refus par permission : un correspondant ne lit pas la vue DES du périmètre,
attendu 403
source ./scripts/token.sh test-correspondant-b
curl -s -o /dev/null -w "%{http_code}\n" "${BASE}/liaisons/${LIAISON_ID}/perimetre" -
H "Authorization: Bearer ${TOKEN}"
```

HOM-06 | Geler une liaison sur changement de statut de l'institution

Élément	Valeur
Points	3
Priorité	Should Have
Sprint cible	Sprint 3
Écrans	DES-08, DES-09
Dépendances	HOM-04, PAR-02, NOT-14

Objectif

Geler automatiquement, sur `participant.status-changed.v1`, toute liaison active dont l'institution passe à l'état suspendu ou retiré, en mémorisant l'état de la liaison et de chacune de ses opérations ; refuser tout dépôt et toute clôture pendant le gel ; dégeler sur décision motivée du chef du DES lorsque l'institution est revenue « en homologation ». Résultat vérifiable : après le dégel, chaque opération retrouve l'état qu'elle avait avant le gel, et une liaison dont l'institution est retirée reste gelée.

A. Implémentation

A1. Gel, instantané et restauration (modèle et service de domaine pur)

Le récit n'introduit pas de table : il exploite `etat_avant_gel` (jsonb) et `motif_etat` de la liaison (HOM-04). Sont gelables les liaisons `OUVERTE`, `EN_HOMOLOGATION` et `APTE_CERTIFICATION` ; une liaison autorisée à la mise en service, certifiée ou close relève de MS-CERTIFICATION (suspension d'attestation) et n'est pas gelée ici. Pendant le gel, les états des opérations ne changent pas : aucun dépôt ni clôture n'est admis et la propagation d'un changement de périmètre est différée (HOM-05). Seule exception : l'horodatage d'une validation acquise avant le gel peut être obtenu pendant le gel (HOM-10) ; la restauration ne ramène donc jamais une opération close à l'état antérieur.

```
// domain/model/EtatAvantGel.java
public record EtatAvantGel(EtatLiaison etatLiaison, Map<UUID, EtatOperation>
etatsOperations, Instant dateGel) {}
```

```
// domain/model/Liaison.java (extrait propre à HOM-06)
private static final Set<EtatLiaison> GELABLES =
    EnumSet.of(EtatLiaison.OUVERTE, EtatLiaison.EN_HOMOLOGATION,
EtatLiaison.APTE_CERTIFICATION);

@JdbcTypeCode(SqlTypes.JSON) @Column(columnDefinition = "jsonb")
private EtatAvantGel etatAvantGel;

/** Gèle la liaison ; renvoie faux si elle n'est pas gelable ou déjà gelée (le motif
est alors mis à jour). */
public boolean geler(String motif, Map<UUID, EtatOperation> etatsOperations, Instant
instant) {
    if (etat == EtatLiaison.GELEE) { this.motifEtat = motif; return false; }
    if (!GELABLES.contains(etat)) return false;
    this.etatAvantGel = new EtatAvantGel(etat, Map.copyOf(etatsOperations), instant);
    this.etat = EtatLiaison.GELEE;
    this.motifEtat = motif;
    return true;
}
```

```
public EtatAvantGel degeler(String motif) {
    if (etat != EtatLiaison.GELEE) {
        throw HomologationException.conflict("HOM_LIAISON_NOT_FROZEN", "La liaison
n'est pas gelée", Map.of("etat", etat.name()));
    }
    EtatAvantGel instantane = etatAvantGel;
    this.etat = instantane.etatLiaison();
    this.motifEtat = motif;
    this.etatAvantGel = null;
    return instantane;
}
```

```
// domain/service/FreezeRestoration.java
public final class FreezeRestoration {

    /** Calcule les états à rétablir : ceux de l'instantané, sauf une opération close
pendant le gel par horodatage différé. */
    public Map<UUID, EtatOperation> aRetablir(EtatAvantGel instantane, Map<UUID,
EtatOperation> etatsCourants) {
        Map<UUID, EtatOperation> cibles = new HashMap<>();
        instantane.etatsOperations().forEach((id, avant) -> {
            EtatOperation courant = etatsCourants.get(id);
            boolean closeParHorodatage = courant == EtatOperation.CLOSE && avant ==
EtatOperation.VALIDEE_HORODATAGE_EN_ATTENTE;
            if (courant != null && courant != avant && !closeParHorodatage)
cibles.put(id, avant);
        });
        return cibles;
    }
}

// domain/service/LiaisonFreezeGuard.java
public final class LiaisonFreezeGuard {
    /** Première instruction des cas d'utilisation de dépôt (HOM-09) et de clôture
(HOM-10). */
    public void assertNotFrozen(Liaison liaison) {
        if (liaison.getEtat() == EtatLiaison.GELEE) {
            throw HomologationException.conflict("HOM_LIAISON_FROZEN", "Liaison gelée
: " + liaison.getMotifEtat(),
                Map.of("liaisonId", liaison.getId().toString(), "motif",
liaison.getMotifEtat()));
        }
    }
}
```

A2. Consommateur idempotent de `participant.status-changed.v1`

Seuls les statuts `SUSPENDU` et `RETIRE` gèlent ; le retour à `EN_HOMOLOGATION` ne dégèle pas automatiquement mais notifie le chef de projet que le dégel est possible. Chaque liaison gelée produit `homologation.liaison-frozen.v1` et une demande de notification au chef de projet et aux correspondants de l'institution. Un événement non conforme suit le chemin de rebut de HOM-05.

```
// infrastructure/messaging/consumer/ParticipantStatusConsumer.java
@Component
@RequiredArgsConstructor
public class ParticipantStatusConsumer {

    static final String GROUP = "mshomologation-status";
    static final String TYPE = "participant.status-changed.v1";

    private final IdempotentConsumer idempotentConsumer;
    private final EventSchemaValidator schemas;
    private final RejectedEventRouter rejets;
```

```
    private final FreezeLiaisonUseCase useCase;
    private final ObjectMapper json;

    @KafkaListener(topics = "participant.status", groupId = GROUP)
    @Transactional
    public void onMessage(ConsumerRecord<String, String> record) {
        CloudEventEnvelope event;
        StatusChangedPayload payload;
        try {
            event = json.readValue(record.value(), CloudEventEnvelope.class);
            if (!TYPE.equals(event.type())) return;
            schemas.valider(TYPE, event.data());
            payload = json.treeToValue(event.data(), StatusChangedPayload.class);
        } catch (InvalidEventException | JsonProcessingException e) {
            rejets.rejeter(record, e.getMessage());
            return;
        }
        idempotentConsumer.handleOnce(event.id(), GROUP, () ->
useCase.appliquerStatut(payload));
    }
}
```

```
// application/usecase/FreezeLiaisonService.java
@Service
@Transactional
@RequiredArgsConstructor
public class FreezeLiaisonService implements FreezeLiaisonUseCase {

    private static final Set<String> STATUTS_GELANTS = Set.of("SUSPENDU", "RETIRE");

    private final LiaisonRepository liaisons;
    private final OperationInstanceRepository operations;
    private final ParticipantDirectoryPort participants;
    private final FreezeRestoration restoration;
    private final OperationGraphService operationGraph;     // HOM-07
    private final NotificationRequester notifications;
    private final OutboxPublisher outboxPublisher;
    private final Clock clock;

    @Override
    public void appliquerStatut(StatusChangedPayload p) {
        for (Liaison l : liaisons.findByInstitutionIdAndEtatNot(p.institutionId(),
EtatLiaison.CLOSE)) {
            if (STATUTS_GELANTS.contains(p.statut())) {
                String motif = "Statut de l'institution : " + p.statut() + " (" +
p.motif() + ")";
                if (l.geler(motif, operations.etatsParId(l.getId()),
clock.instant())) {
                    outboxPublisher.publish("liaison", l.getId().toString(),
"homologation.liaison-frozen.v1",
                        new LiaisonFrozenPayload(l.getId(), l.getInstitutionId(),
p.statut(), motif));
                    notifications.demander("HOM_LIAISON_GELEE",

Stream.concat(Destinataires.utilisateurs(l.getChefDeProjetId()).stream(),

Destinataires.correspondants(l.getInstitutionId()).stream()).toList(),
                        Map.of("liaisonId", l.getId(), "motif", motif));
                }
            } else if ("EN_HOMOLOGATION".equals(p.statut()) && l.getEtat() ==
EtatLiaison.GELEE) {
                notifications.demander("HOM_LIAISON_DEGEL_POSSIBLE",
Destinataires.utilisateurs(l.getChefDeProjetId()),
                    Map.of("liaisonId", l.getId()));
            }
        }
    }

    @Override
```

```
    public Liaison degeler(UUID liaisonId, String motif, UUID decideurId) {
        Liaison l = liaisons.findByIdForUpdate(liaisonId)
            .orElseThrow(() ->
HomologationException.notFound("HOM_LIAISON_NOT_FOUND", liaisonId));
        String statut = participants.getInstitution(l.getInstitutionId()).statut();
// lecture à jour, 503 si indisponible
        if (!"EN_HOMOLOGATION".equals(statut)) {
            throw HomologationException.conflict("HOM_LIAISON_UNFREEZE_NOT_ELIGIBLE",
                "L'institution n'est pas revenue à l'état « en homologation »",
Map.of("statut", statut));
        }
        EtatAvantGel instantane = l.degeler(motif);
        restoration.aRetablir(instantane, operations.etatsParId(liaisonId))
            .forEach((opId, etat) ->
operations.getReferenceById(opId).retablir(etat));
        operationGraph.applyScope(l);                          // propagation différée
d'un changement de périmètre
        outboxPublisher.publish("liaison", liaisonId.toString(),
"homologation.liaison-unfrozen.v1",
            new LiaisonUnfrozenPayload(liaisonId, motif, decideurId,
instantane.dateGel()));
        return l;
    }
}
```

A3. Commande de dégel motivé et garde sur les commandes de dépôt et de clôture

Méthode et chemin	Permission	Réponse
`POST /liaisons/{id}/degel` (corps `motif` obligatoire)	`HOM_LIAISON_UNFREEZE` (rôle `CHEF_DES`)	200, 400 sans motif, 409 `HOM_LIAISON_NOT_FROZEN` ou `HOM_LIAISON_UNFREEZE_NOT_ELIGIBLE`

```
// web/rest/LiaisonFreezeResource.java
@RestController
@RequestMapping("/api/v1/liaisons/{id}/degel")
@RequiredArgsConstructor
public class LiaisonFreezeResource {

    private final FreezeLiaisonUseCase useCase;
    private final LiaisonQueryService queries;
    private final CurrentUser currentUser;

    public record DegelRequest(@NotBlank @Size(max = 500) String motif) {}

    @PostMapping
    @PreAuthorize("hasAuthority('HOM_LIAISON_UNFREEZE')")
    public LiaisonDTO degeler(@PathVariable UUID id, @Valid @RequestBody DegelRequest
req) {
        useCase.degeler(id, req.motif(), currentUser.id());
        return queries.fiche(id);
    }
}
```

```
// application/usecase/SubmitEvidenceService.java et ValidateOperationService.java
(HOM-09, HOM-10) : premières lignes
OperationInstance op = operations.findByIdForUpdate(operationId)
    .orElseThrow(() -> HomologationException.notFound("HOM_OPERATION_NOT_FOUND",
operationId));
freezeGuard.assertNotFrozen(op.getLiaison());   // HOM-06 : 409 HOM_LIAISON_FROZEN
operationGraph.assertActionable(op);            // HOM-07 : 409 HOM_OPERATION_LOCKED
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
// src/test/java/org/gimac/des/homologation/domain/service/FreezeRestorationTest.java
class FreezeRestorationTest {

    private final FreezeRestoration restoration = new FreezeRestoration();
    private final UUID op1 = UUID.randomUUID(), op2 = UUID.randomUUID(), op3 =
UUID.randomUUID();

    @Test
    @DisplayName("HOM-06 : après dégel, chaque opération retrouve l'état qu'elle
avait avant le gel")
    void restaurationDesEtats() {
        var instantane = new EtatAvantGel(EtatLiaison.EN_HOMOLOGATION,
            Map.of(op1, EtatOperation.ELIGIBLE, op2, EtatOperation.PREUVE_DEPOSEE,
op3, EtatOperation.VERROUILLEE),
            Instant.parse("2026-10-07T10:00:00Z"));
        var courants = Map.of(op1, EtatOperation.HORS_PERIMETRE, op2,
EtatOperation.PREUVE_DEPOSEE, op3, EtatOperation.VERROUILLEE);

        assertThat(restoration.aRetablir(instantane,
courants)).containsExactly(entry(op1, EtatOperation.ELIGIBLE));
    }

    @Test
    @DisplayName("HOM-06 : une opération close pendant le gel par horodatage différé
n'est pas rétrogradée")
    void clotureParHorodatageConservee() {
        var instantane = new EtatAvantGel(EtatLiaison.EN_HOMOLOGATION,
            Map.of(op1, EtatOperation.VALIDEE_HORODATAGE_EN_ATTENTE),
Instant.parse("2026-10-07T10:00:00Z"));
        assertThat(restoration.aRetablir(instantane, Map.of(op1,
EtatOperation.CLOSE))).isEmpty();
    }

    @Test
    @DisplayName("HOM-06 : la garde refuse toute action sur une liaison gelée en
restituant le motif")
    void gardeDeGel() {
        Liaison l = LiaisonSamples.enHomologation();
        l.geler("Statut de l'institution : SUSPENDU (décision du conseil)", Map.of(),
Instant.parse("2026-10-07T10:00:00Z"));
        assertThatThrownBy(() -> new LiaisonFreezeGuard().assertNotFrozen(l))
            .isInstanceOfSatisfying(HomologationException.class, ex -> {
                assertThat(ex.code()).isEqualTo("HOM_LIAISON_FROZEN");
                assertThat(ex.details()).containsEntry("motif", "Statut de
l'institution : SUSPENDU (décision du conseil)");
            });
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
// src/test/java/org/gimac/des/homologation/infrastructure/messaging/consumer/
ParticipantStatusConsumerIT.java
class ParticipantStatusConsumerIT extends AbstractHomologationIT {

    private static final UUID VALIDATEUR = UUID.fromString("00000000-0000-0000-0000-
00000000e002");

    @Autowired KafkaTemplate<String, String> kafka;

    @Test
    @DisplayName("HOM-06 : le gel sur suspension est émis puis la clôture d'une
opération est refusée")
    void gelSurSuspensionPuisRefusDeCloture() throws Exception {
        UUID institution = UUID.randomUUID();
```

```
        var liaison = fixtures.liaisonEnHomologationAvecOperationPrete(institution);
// opération avec preuve scellée
        var evenement = fixtures.cloudEvent("participant.status-changed.v1",
institution,
            Map.of("institutionId", institution, "statut", "SUSPENDU", "motif",
"décision du conseil"));

        kafka.send("participant.status", institution.toString(), evenement.json());
        await().atMost(Duration.ofSeconds(15)).untilAsserted(() ->
            assertThat(jdbc.queryForObject("select etat from liaison where id = ?",
String.class, liaison.id())).isEqualTo("GELEE"));
        assertThat(jdbc.queryForObject("select count(*) from outbox_event where
aggregate_id = ? and event_type = ?",
            Integer.class, liaison.id().toString(), "homologation.liaison-
frozen.v1")).isEqualTo(1);

        mvc.perform(post("/api/v1/operations/{id}/validation",
liaison.operationPreteId())
                .with(utilisateur(VALIDATEUR, "HOM_OPERATION_VALIDATE"))
                .header("X-Reauth-Assertion", "assertion-de-test"))           //
adaptateur de substitution en profil test
            .andExpect(status().isConflict())
            .andExpect(jsonPath("$.code").value("HOM_LIAISON_FROZEN"));
        assertThat(jdbc.queryForObject("select etat from operation_instance where id
= ?", String.class,
            liaison.operationPreteId())).isEqualTo("PREUVE_DEPOSEE");
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: HOM-06 Gel d'une liaison sur changement de statut de l'institution

  Plan du scénario: Nominal : gel sur suspension
    Étant donné une liaison en homologation de l'Établissement de paiement A
    Quand l'événement "participant.status-changed.v1" indique que l'institution est
"<statut>"
    Alors la liaison passe à l'état "GELEE" avec le motif reçu et aucune opération ne
peut plus recevoir de dépôt ni de clôture
    Et l'événement "homologation.liaison-frozen.v1" est émis et le chef de projet
ainsi que les correspondants sont notifiés

    Exemples:
      | statut   |
      | SUSPENDU |
      | RETIRE   |

  Scénario: Alternatif : dégel motivé
    Étant donné une liaison gelée dont l'institution est revenue à l'état
"EN_HOMOLOGATION"
    Et un utilisateur authentifié avec le rôle "CHEF_DES"
    Quand le chef du DES décide le dégel avec le motif "reprise d'activité constatée"
    Alors chaque opération retrouve l'état qu'elle avait avant le gel
    Et l'événement "homologation.liaison-unfrozen.v1" est émis

  Plan du scénario: Erreur : action sur une liaison gelée
    Étant donné une liaison gelée
    Et un utilisateur authentifié avec le rôle "INGENIEUR_HOMOLOGATION"
    Quand un utilisateur tente de <action>
    Alors l'action est refusée avec le code "HOM_LIAISON_FROZEN" et le motif du gel

    Exemples:
      | action                                  |
      | clore une opération                     |
      | déposer une preuve sur une opération    |
```

E. Tests cURL via la passerelle

```
BASE="${GATEWAY_URL}/bff/des/homologation"

# 1. Refus : dépôt de preuve sur une liaison gelée, attendu 409 HOM_LIAISON_FROZEN
source ./scripts/token.sh test-ingenieur-homologation
curl -s -o /tmp/hom06.json -w "%{http_code}\n" -X POST "${BASE}/operations/$
{OPERATION_ID}/preuves" \
  -H "Authorization: Bearer ${TOKEN}" -F "typePiece=RAPPORT_PARAMETRAGE" -F
"fichier=@./recette/rapport-parametrage.pdf"
jq '{code, motif: .details.motif}' /tmp/hom06.json

# 2. Refus par permission : l'ingénieur ne dégèle pas, attendu 403
curl -s -o /dev/null -w "%{http_code}\n" -X POST
"${BASE}/liaisons/${LIAISON_ID}/degel" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" -d
'{"motif":"reprise"}'

# 3. Nominal : dégel motivé par le chef du DES, institution revenue en homologation
source ./scripts/token.sh test-chef-des
curl -s -X POST "${BASE}/liaisons/${LIAISON_ID}/degel" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" \
  -d '{"motif":"reprise de l’activité constatée par MS-PARTICIPANT"}' | jq '{etat,
motifEtat, actionsPermises}'

# 4. Refus : dégel d'une liaison dont l'institution est retirée, attendu 409
HOM_LIAISON_UNFREEZE_NOT_ELIGIBLE
curl -s -o /tmp/hom06-retire.json -w "%{http_code}\n" -X POST "${BASE}/liaisons/$
{LIAISON_RETIREE_ID}/degel" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" -d
'{"motif":"demande de reprise"}'
jq -r .code /tmp/hom06-retire.json
```

HOM-07 | Instancier les opérations d'une liaison par tri topologique

Élément	Valeur
Points	8
Priorité	Must Have
Sprint cible	Sprint 3
Écrans	DES-09
Dépendances	HOM-02, HOM-04, HOM-05

Objectif

Instancier les opérations d'une liaison dans l'ordre topologique de sa version du référentiel dès que son périmètre est établi, les placer aux états éligible, verrouillée ou hors périmètre, puis les débloquer au fil des clôtures (et les reverrouiller aux réouvertures) avec un compteur de degré entrant persisté. Résultat vérifiable : aucune opération n'est éligible avant la clôture de tous ses prédécesseurs dans le périmètre, un prédécesseur hors périmètre est neutre, et chaque déblocage émet `homologation.operation-unlocked.v1`.

A. Implémentation

A1. Table `operation_instance`

L'unicité `(liaison_id, code)` interdit tout doublon d'opération dans une liaison, y compris lors d'une reprise de migration (HOM-03). Une opération archivée par une migration reste liée à l'opération de référence de l'ancienne version ; les requêtes des opérations actives filtrent sur la version de la liaison.

```
<!-- resources/config/liquibase/changelog/20261005000400_hom07_operation_instance.xml
(en-tête JHipster conservé) -->
<databaseChangeLog>
  <changeSet id="hom-07-operation-instance" author="mshomologation">
    <createTable tableName="operation_instance">
      <column name="id" type="uuid"><constraints primaryKey="true"/></column>
      <column name="liaison_id" type="uuid">
        <constraints nullable="false" foreignKeyName="fk_opinst_liaison"
references="liaison(id)"/>
      </column>
      <column name="reference_operation_id" type="uuid">
        <constraints nullable="false" foreignKeyName="fk_opinst_refop"
references="reference_operation(id)"/>
      </column>
      <column name="code" type="varchar(40)"><constraints nullable="false"/></column>
      <column name="libelle" type="varchar(250)"><constraints
nullable="false"/></column>
      <column name="segment" type="varchar(24)"><constraints
nullable="false"/></column>
      <column name="partie" type="varchar(16)"><constraints
nullable="false"/></column>
      <column name="service_code" type="varchar(40)"><constraints
nullable="false"/></column>
      <column name="etat" type="varchar(32)"><constraints nullable="false"/></column>
      <column name="degre_entrant" type="integer"
defaultValueNumeric="0"><constraints nullable="false"/></column>
      <column name="echeance" type="date"/>
      <column name="date_etat" type="timestamptz"><constraints
nullable="false"/></column>
      <column name="grille_reponses" type="jsonb"/>
```

```
    </createTable>
    <addUniqueConstraint tableName="operation_instance" columnNames="liaison_id,
code" constraintName="ux_opinst_liaison_code"/>
    <createIndex tableName="operation_instance" indexName="ix_opinst_liaison_etat">
      <column name="liaison_id"/><column name="etat"/>
    </createIndex>
  </changeSet>
</databaseChangeLog>
```

A2. Prédécesseurs effectifs et éligibilité (services de domaine purs)

Choix retenu pour le traitement transitif : un prédécesseur hors périmètre est traversé, et ses propres prédécesseurs effectifs le remplacent. Il est ainsi neutre (il ne conditionne jamais le déblocage) sans rompre l'ordre du parcours : si `C` dépend de `H` hors périmètre, qui dépend de `A` dans le périmètre, `C` attend la clôture de `A`. Le graphe étant acyclique (HOM-02), la récursion mémorisée termine en O(V + E). L'évaluateur calcule le degré entrant (prédécesseurs effectifs non clos) et l'état cible ; il ne touche jamais une opération close, et conserve l'avancement d'une opération dans le périmètre dont une preuve est déjà déposée, validée ou qui a été rouverte.

```
// domain/service/EffectivePredecessorResolver.java
public final class EffectivePredecessorResolver {

    public Map<String, Set<String>> resoudre(Map<String, List<String>> predecesseurs,
Set<String> perimetre) {
        Map<String, Set<String>> memo = new HashMap<>();
        Map<String, Set<String>> resultat = new TreeMap<>();
        for (String code : perimetre) resultat.put(code, effectifs(code,
predecesseurs, perimetre, memo));
        return resultat;
    }

    private Set<String> effectifs(String code, Map<String, List<String>> preds,
Set<String> perimetre,
                                  Map<String, Set<String>> memo) {
        Set<String> connu = memo.get(code);
        if (connu != null) return connu;
        Set<String> res = new TreeSet<>();
        for (String p : preds.getOrDefault(code, List.of())) {
            if (perimetre.contains(p)) res.add(p);
            else res.addAll(effectifs(p, preds, perimetre, memo));   // prédécesseur
neutre : traversé
        }
        Set<String> fige = Set.copyOf(res);
        memo.put(code, fige);
        return fige;
    }
}
```

```
// domain/service/EligibilityEvaluator.java
public final class EligibilityEvaluator {

    private static final Set<EtatOperation> RECALCULABLES =
        EnumSet.of(EtatOperation.VERROUILLEE, EtatOperation.ELIGIBLE,
EtatOperation.HORS_PERIMETRE);

    public record Decision(int degreEntrant, EtatOperation etat, List<String>
predecesseursNonClos) {}

    public Decision evaluer(String code, boolean dansPerimetre, Set<String>
effectifs, Map<String, EtatOperation> etats) {
        EtatOperation courant = etats.get(code);
        List<String> nonClos = effectifs.stream().filter(p -> etats.get(p) !=
EtatOperation.CLOSE).sorted().toList();
```

```
        if (courant == EtatOperation.CLOSE) return new Decision(0, courant,
List.of());          // jamais rétrogradée
        if (!dansPerimetre) return new Decision(0, EtatOperation.HORS_PERIMETRE,
List.of());     // preuves conservées
        if (courant != null && !RECALCULABLES.contains(courant)) return new
Decision(nonClos.size(), courant, nonClos);
        return new Decision(nonClos.size(), nonClos.isEmpty() ?
EtatOperation.ELIGIBLE : EtatOperation.VERROUILLEE, nonClos);
    }
}
```

A3. Service d'application : instanciation, recalcul incrémental et garde de verrouillage

`applyScope` est appelé par la dérivation du périmètre (HOM-05), la migration (HOM-03) et le dégel (HOM-06). À la première invocation avec un périmètre établi, il instancie toutes les opérations de la version dans l'ordre topologique de référence enregistré à la publication (HOM-02), mémorise cet ordre dans `liaison.ordre_instanciation` et fait passer la liaison de `OUVERTE` à `EN_HOMOLOGATION`. `onClosed` et `onReopened` sont appelés dans la transaction de la clôture (HOM-10) et de la réouverture (HOM-11) et ne réévaluent que les successeurs effectifs. Seule la transition `VERROUILLEE` → `ELIGIBLE` d'une opération existante constitue un déblocage publié par l'outbox ; l'état initial n'en est pas un.

```
// application/usecase/OperationGraphService.java
@Service
@Transactional
@RequiredArgsConstructor
public class OperationGraphService {

    private final OperationInstanceRepository operations;   // actives(liaisonId) :
Map code -> instance, version de la liaison
    private final ContractualScopeRepository scopes;
    private final AssignmentRepository assignments;         // HOM-08 : responsable
actif à notifier
    private final EffectivePredecessorResolver resolver;
    private final EligibilityEvaluator evaluator;
    private final NotificationRequester notifications;
    private final OutboxPublisher outboxPublisher;
    private final Clock clock;

    record GrapheEffectif(Map<String, Set<String>> predecesseurs, Map<String,
Set<String>> successeurs, Set<String> perimetre) {}

    public void applyScope(Liaison l) {
        Optional<ContractualScope> scope = scopes.findByLiaisonId(l.getId());
        if (scope.isEmpty() || scope.get().getItems().isEmpty()) return;          //
périmètre non encore établi
        ReferentialVersion v = l.getVersion();
        Map<String, OperationInstance> instances = operations.actives(l.getId());
        boolean initiale = instances.isEmpty();
        if (initiale) {
            for (String code : v.getOrdreTopologique()) {
                instances.put(code, operations.save(OperationInstance.instancier(l,
v.operation(code))));
            }
            l.demarrerHomologation(v.getOrdreTopologique(), clock.instant());
        }
        GrapheEffectif g = graphe(l, scope.get());
        Map<String, EtatOperation> etats = etats(instances);
        for (String code : v.getOrdreTopologique()) {
            OperationInstance op = instances.get(code);
            if (op == null) continue;
            appliquer(l, op, evaluator.evaluer(code, g.perimetre().contains(code),
                g.predecesseurs().getOrDefault(code, Set.of()), etats), !initiale);
        }
    }
```

```
    public void onClosed(OperationInstance close) { reevaluerSuccesseurs(close); }

    public void onReopened(OperationInstance rouverte)
{ reevaluerSuccesseurs(rouverte); }   // successeurs non clos reverrouillés

    /** Garde appelée en tête des commandes de dépôt (HOM-09) et de clôture (HOM-10).
*/
    public void assertActionable(OperationInstance op) {
        if (op.getEtat() == EtatOperation.HORS_PERIMETRE) {
            throw HomologationException.conflict("HOM_OPERATION_OUT_OF_SCOPE",
"Opération hors périmètre",
                Map.of("operationId", op.getId().toString()));
        }
        if (op.getEtat() == EtatOperation.VERROUILLEE) {
            Liaison l = op.getLiaison();
            GrapheEffectif g = graphe(l,
scopes.findByLiaisonId(l.getId()).orElseThrow());
            List<String> nonClos = evaluator.evaluer(op.getCode(), true,
g.predecesseurs().get(op.getCode()),
                etats(operations.actives(l.getId()))).predecesseursNonClos();
            throw HomologationException.conflict("HOM_OPERATION_LOCKED", "Opération
verrouillée : prédécesseurs non clos",
                Map.of("operationId", op.getId().toString(), "predecesseursNonClos",
nonClos));
        }
    }

    private void reevaluerSuccesseurs(OperationInstance source) {
        Liaison l = source.getLiaison();
        GrapheEffectif g = graphe(l,
scopes.findByLiaisonId(l.getId()).orElseThrow());
        Map<String, OperationInstance> instances = operations.actives(l.getId());
        Map<String, EtatOperation> etats = etats(instances);
        for (String succ : g.successeurs().getOrDefault(source.getCode(), Set.of()))
{
            appliquer(l, instances.get(succ), evaluator.evaluer(succ, true,
g.predecesseurs().get(succ), etats), true);
        }
    }

    private void appliquer(Liaison l, OperationInstance op,
EligibilityEvaluator.Decision d, boolean publierDeblocage) {
        EtatOperation avant = op.getEtat();
        op.majEligibilite(d.degreEntrant(), d.etat(), clock.instant());          //
degré entrant persisté
        if (publierDeblocage && avant == EtatOperation.VERROUILLEE && d.etat() ==
EtatOperation.ELIGIBLE) {
            UUID responsable = assignments.responsableActif(op.getId()).orElse(null);
            outboxPublisher.publish("operation", op.getId().toString(),
"homologation.operation-unlocked.v1",
                new OperationUnlockedPayload(op.getId(), l.getId(), op.getCode(),
responsable));
            notifications.demander("HOM_OPERATION_DEBLOQUEE",
                Destinataires.utilisateurs(responsable != null ? responsable :
l.getChefDeProjetId()),
                Map.of("operationId", op.getId(), "code", op.getCode(), "liaisonId",
l.getId()));
        }
    }

    private GrapheEffectif graphe(Liaison l, ContractualScope scope) {
        ReferentialVersion v = l.getVersion();
        Set<String> services = scope.servicesDansLePerimetre();                  //
TRANSVERSE toujours inclus
        Set<String> perimetre = v.getOperations().stream().filter(r ->
services.contains(r.getServiceCode()))
            .map(ReferenceOperation::getCode).collect(Collectors.toSet());
        Map<String, Set<String>> preds = resolver.resoudre(v.predecesseursParCode(),
```

```
perimetre);
        Map<String, Set<String>> succ = new HashMap<>();
        preds.forEach((code, ps) -> ps.forEach(p -> succ.computeIfAbsent(p, k -> new
TreeSet<>()).add(code)));
        return new GrapheEffectif(preds, succ, perimetre);
    }

    private static Map<String, EtatOperation> etats(Map<String, OperationInstance>
instances) {
        Map<String, EtatOperation> etats = new HashMap<>();
        instances.forEach((code, op) -> etats.put(code, op.getEtat()));
        return etats;
    }
}
```

A4. Ressources de lecture : opérations filtrables et graphe par couches

Méthode et chemin	Permission	Réponse
`GET /liaisons/{id}/operations?etat=&segment=&partie=`	`HOM_LIAISON_READ`	200 : opérations actives, `degreEntrant`, `actionsPermises`
`GET /liaisons/{id}/graphe`	`HOM_LIAISON_READ`	200 : `noeuds[{operationId, code, segment, partie, etat, couche, degreEntrant}]`, `arcs[{de, vers, neutre}]`

Les couches sont celles du tri de Kahn (`TopologicalSorter`, HOM-02) sur le graphe de la version ; un arc dont l'origine est hors périmètre est marqué `neutre` pour que `DagView` le représente en pointillé.

```
// web/rest/LiaisonOperationsResource.java
@RestController
@RequestMapping("/api/v1/liaisons/{id}")
@RequiredArgsConstructor
public class LiaisonOperationsResource {

    private final OperationInstanceRepository operations;     //
JpaSpecificationExecutor
    private final LiaisonGraphQueryService graphes;
    private final OperationMapper mapper;

    @GetMapping("/operations")
    @PreAuthorize("hasAuthority('HOM_LIAISON_READ')")
    public List<OperationDTO> lister(@PathVariable UUID id,
                                     @RequestParam(required = false) EtatOperation
etat,
                                     @RequestParam(required = false) Segment segment,
                                     @RequestParam(required = false) Partie partie) {
        Specification<OperationInstance> spec =
OperationSpecifications.activesDeLaLiaison(id)
            .and(OperationSpecifications.etat(etat))           // une spécification
nulle est ignorée
            .and(OperationSpecifications.segment(segment))
            .and(OperationSpecifications.partie(partie));
        return operations.findAll(spec,
Sort.by("dateEtat")).stream().map(mapper::toDto).toList();
    }

    @GetMapping("/graphe")
    @PreAuthorize("hasAuthority('HOM_LIAISON_READ')")
    public GrapheLiaisonDTO graphe(@PathVariable UUID id) {
        return graphes.graphe(id);
    }
}
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
// src/test/java/org/gimac/des/homologation/domain/service/EligibilityTest.java
class EligibilityTest {

    private final EffectivePredecessorResolver resolver = new
EffectivePredecessorResolver();
    private final EligibilityEvaluator evaluator = new EligibilityEvaluator();

    @Test
    @DisplayName("HOM-07 : aucune opération n'est éligible avant la clôture de ses
prédécesseurs dans le périmètre")
    void eligibleSeulementApresCloture() {
        Map<String, List<String>> graphe = Map.of("A", List.of(), "B", List.of(),
"C", List.of("A", "B"));
        Set<String> perimetre = Set.of("A", "B", "C");
        Set<String> effectifsC = resolver.resoudre(graphe, perimetre).get("C");

        var avant = evaluator.evaluer("C", true, effectifsC, Map.of("A", CLOSE, "B",
ELIGIBLE, "C", VERROUILLEE));
        assertThat(avant.etat()).isEqualTo(VERROUILLEE);
        assertThat(avant.degreEntrant()).isEqualTo(1);
        assertThat(avant.predecesseursNonClos()).containsExactly("B");

        var apres = evaluator.evaluer("C", true, effectifsC, Map.of("A", CLOSE, "B",
CLOSE, "C", VERROUILLEE));
        assertThat(apres.etat()).isEqualTo(ELIGIBLE);
        assertThat(apres.degreEntrant()).isZero();
    }

    @Test
    @DisplayName("HOM-07 : un prédécesseur hors périmètre est neutre et traversé
jusqu'aux prédécesseurs du périmètre")
    void neutraliteTransitive() {
        // C dépend de H (hors périmètre) ; H dépend de A ; D ne dépend que de H2
(hors périmètre, sans prédécesseur)
        Map<String, List<String>> graphe = Map.of("A", List.of(), "H", List.of("A"),
"C", List.of("H"),
                                                  "H2", List.of(), "D",
List.of("H2"));
        var effectifs = resolver.resoudre(graphe, Set.of("A", "C", "D"));

        assertThat(effectifs.get("C")).containsExactly("A");
        assertThat(effectifs.get("D")).isEmpty();
        assertThat(evaluator.evaluer("D", true, effectifs.get("D"),
Map.of()).etat()).isEqualTo(ELIGIBLE);
        assertThat(evaluator.evaluer("H", false, Set.of(),
Map.of()).etat()).isEqualTo(HORS_PERIMETRE);
    }

    @Test
    @DisplayName("HOM-07 : une opération close n'est jamais rétrogradée par un
recalcul")
    void clotureJamaisRetrogradee() {
        assertThat(evaluator.evaluer("C", false, Set.of("A"), Map.of("C",
CLOSE)).etat()).isEqualTo(CLOSE);
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
// src/test/java/org/gimac/des/homologation/application/usecase/OperationGraphIT.java
class OperationGraphIT extends AbstractHomologationIT {

    private static final UUID INGENIEUR = UUID.fromString("00000000-0000-0000-0000-
00000000e001");
```

```
    @Autowired OperationGraphService operationGraph;
    @Autowired TransactionTemplate tx;

    @Test
    @DisplayName("HOM-07 : instanciation initiale puis déblocage publié à la clôture
du dernier prédécesseur")
    void instanciationPuisDeblocage() throws Exception {
        // PM-AMONT-01 -> PM-BR-01 ; PM-BR-02 dépend de PM-BR-01 et de PM-QR-01
(service PAIEMENT_MARCHAND hors périmètre)
        UUID liaisonId = fixtures.liaisonAvecPerimetre(TypeProjet.PAIEMENT_MOBILE,
Set.of("TRANSFERT"));
        tx.executeWithoutResult(s ->
operationGraph.applyScope(fixtures.liaison(liaisonId)));

        mvc.perform(get("/api/v1/liaisons/{id}/operations", liaisonId).param("etat",
"ELIGIBLE")
                .with(utilisateur(INGENIEUR, "HOM_LIAISON_READ")))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$[*].code").value(containsInAnyOrder("PM-AMONT-
01")));
        assertThat(jdbc.queryForObject("select etat from operation_instance where
liaison_id = ? and code = 'PM-QR-01'",
            String.class, liaisonId)).isEqualTo("HORS_PERIMETRE");
        assertThat(jdbc.queryForObject("select ordre_instanciation::text from liaison
where id = ?", String.class, liaisonId))
            .startsWith("[\"PM-AMONT-01\"");

        fixtures.clore(liaisonId, "PM-AMONT-01");      // simule HOM-10 : état CLOSE
puis onClosed dans la même transaction
        fixtures.clore(liaisonId, "PM-BR-01");

        assertThat(jdbc.queryForObject("select etat from operation_instance where
liaison_id = ? and code = 'PM-BR-02'",
            String.class, liaisonId)).isEqualTo("ELIGIBLE");
        assertThat(jdbc.queryForObject("select count(*) from outbox_event where
event_type = 'homologation.operation-unlocked.v1' "
            + "and payload ->> 'liaisonId' = ? and payload ->> 'code' = 'PM-BR-02'",
Integer.class, liaisonId.toString())).isEqualTo(1);
    }

    @Test
    @DisplayName("HOM-07 : deux cents opérations instanciées sur cinquante liaisons
simultanées, sans doublon")
    void chargeCinquanteLiaisons() throws Exception {
        UUID version =
fixtures.versionPublieeSynthetique(TypeProjet.CARTE_INTERBANCAIRE, 200);
        List<UUID> liaisons = IntStream.range(0, 50).mapToObj(i ->
fixtures.liaisonAvecPerimetreComplet(version)).toList();
        ExecutorService pool = Executors.newFixedThreadPool(50);
        long debut = System.nanoTime();
        List<Future<?>> taches = liaisons.stream()
            .map(id -> pool.submit(() -> tx.executeWithoutResult(s ->
operationGraph.applyScope(fixtures.liaison(id))))).toList();
        for (Future<?> t : taches) t.get(60, TimeUnit.SECONDS);
        pool.shutdown();

        assertThat(Duration.ofNanos(System.nanoTime() -
debut)).isLessThan(Duration.ofSeconds(30));
        assertThat(jdbc.queryForObject("select count(*) from operation_instance where
liaison_id = any(?)", Integer.class,
            (Object) liaisons.toArray(UUID[]::new))).isEqualTo(10_000);
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
```

```
Fonctionnalité: HOM-07 Instanciation des opérations par tri topologique

  Contexte:
    Étant donné une version publiée du référentiel "PAIEMENT_MOBILE"
    Et un utilisateur authentifié avec le rôle "INGENIEUR_HOMOLOGATION"

  Scénario: Nominal : instanciation initiale
    Étant donné une liaison de l'Établissement de paiement A dont le périmètre
contractualisé est établi sans le service "PAIEMENT_MARCHAND"
    Quand les opérations sont instanciées
    Alors les opérations sans prédécesseur sont "ELIGIBLE", les autres "VERROUILLEE",
et celles du service "PAIEMENT_MARCHAND" sont "HORS_PERIMETRE"
    Et l'ordre topologique d'instanciation est enregistré avec la liaison

  Scénario: Alternatif : déblocage à la clôture du dernier prédécesseur
    Étant donné l'opération verrouillée "PM-BR-02" dont seul le prédécesseur "PM-BR-
01" reste ouvert
    Quand "PM-BR-01" est clos
    Alors "PM-BR-02" devient "ELIGIBLE" et son responsable est notifié
    Et l'événement "homologation.operation-unlocked.v1" est émis

  Scénario: Alternatif : prédécesseur hors périmètre
    Étant donné l'opération "PM-BR-02" dont le prédécesseur "PM-QR-01" est hors
périmètre
    Quand l'éligibilité est évaluée
    Alors "PM-QR-01" est tenu pour neutre et seuls les prédécesseurs dans le
périmètre conditionnent le déblocage

  Scénario: Erreur : action sur une opération verrouillée
    Étant donné l'opération verrouillée "PM-BR-02"
    Quand un utilisateur tente d'y déposer une preuve
    Alors l'action est refusée avec le code "HOM_OPERATION_LOCKED"
    Et la liste des prédécesseurs non clos "PM-BR-01" est restituée
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-ingenieur-homologation
BASE="${GATEWAY_URL}/bff/des/homologation"

# 1. Opérations éligibles du tronc commun amont incombant au participant
curl -s "${BASE}/liaisons/${LIAISON_ID}/operations?
etat=ELIGIBLE&segment=TRONC_COMMUN_AMONT&partie=PARTICIPANT" \
  -H "Authorization: Bearer ${TOKEN}" | jq '[.[] | {code, etat, degreEntrant}]'

# 2. Graphe de la liaison par couches : nombre d'opérations par état et arcs neutres
curl -s "${BASE}/liaisons/${LIAISON_ID}/graphe" -H "Authorization: Bearer ${TOKEN}" \
  | jq '{parEtat: (.noeuds | group_by(.etat) | map({(.[0].etat): length}) | add),
arcsNeutres: ([.arcs[] | select(.neutre)] | length)}'

# 3. Refus : dépôt sur une opération verrouillée, attendu 409 HOM_OPERATION_LOCKED
avec prédécesseurs non clos
curl -s -o /tmp/hom07.json -w "%{http_code}\n" -X POST "${BASE}/operations/$
{OPERATION_VERROUILLEE_ID}/preuves" \
  -H "Authorization: Bearer ${TOKEN}" -F "typePiece=RAPPORT_PARAMETRAGE" -F
"fichier=@./recette/rapport-parametrage.pdf"
jq '{code, predecesseursNonClos: .details.predecesseursNonClos}' /tmp/hom07.json

# 4. Refus sans jeton : attendu 401 à la passerelle
curl -s -o /dev/null -w "%{http_code}\n" "${BASE}/liaisons/${LIAISON_ID}/operations"
```

HOM-08 | Attribuer le responsable principal de chaque opération

Élément	Valeur
Points	3
Priorité	Must Have
Sprint cible	Sprint 3
Écrans	DES-09
Dépendances	HOM-07, IAM-04, PAR-07

Objectif

Attribuer à chaque opération un responsable principal unique, collaborateur du DES ou correspondant de l'institution selon la partie à qui l'opération incombe, avec proposition par défaut, réattribution motivée, historique conservé et attribution en masse par liaison. Résultat vérifiable : un responsable incompatible avec la partie ou appartenant à une autre institution est refusé (409 `HOM_ASSIGNMENT_PARTY_MISMATCH`), et chaque attribution confirmée émet `homologation.operation-assigned.v1`.

A. Implémentation

A1. Table `assignment` et unicité du responsable actif

L'historique est conservé en ne supprimant jamais une attribution : la précédente passe `actif = false`. L'index unique partiel garantit au plus un responsable actif par opération, y compris sous accès concurrent.

```
<!-- resources/config/liquibase/changelog/20261005000500_hom08_assignment.xml (en-
tête JHipster conservé) -->
<databaseChangeLog>
  <changeSet id="hom-08-assignment" author="mshomologation">
    <createTable tableName="assignment">
      <column name="id" type="uuid"><constraints primaryKey="true"/></column>
      <column name="operation_id" type="uuid">
        <constraints nullable="false" foreignKeyName="fk_assignment_operation"
references="operation_instance(id)"/>
      </column>
      <column name="responsable_id" type="uuid"><constraints
nullable="false"/></column>
      <column name="responsable_nom" type="varchar(200)"><constraints
nullable="false"/></column>
      <column name="partie" type="varchar(16)"><constraints
nullable="false"/></column>
      <column name="motif" type="varchar(500)"/>
      <column name="date_attribution" type="timestamptz"><constraints
nullable="false"/></column>
      <column name="actif" type="boolean"><constraints nullable="false"/></column>
    </createTable>
    <sql>CREATE UNIQUE INDEX ux_assignment_actif ON assignment (operation_id) WHERE
actif;</sql>
  </changeSet>
</databaseChangeLog>
```

A2. Compatibilité avec la partie et proposition par défaut

Règles retenues : une opération incombant au DES n'admet qu'un collaborateur actif du DES (sans institution de rattachement, rôle `INGENIEUR_HOMOLOGATION` ou `CHEF_DES`) ; une

opération incombant au participant n'admet qu'un correspondant actif de l'institution de la liaison (et non de la banque partenaire, qui intervient sans répondre des opérations). La proposition par défaut est calculée à la lecture et n'est pas persistée, afin de suivre les changements de correspondants dans MS-PARTICIPANT : pour la partie participant, le correspondant de l'institution qui exerce la fonction déclarée au référentiel (`fonctionResponsable`, par exemple `MONETIQUE`), lu par l'interface interne de MS-PARTICIPANT (PAR-07) ; pour la partie DES, le chef de projet de la liaison.

```
// domain/service/AssignmentPolicy.java
public final class AssignmentPolicy {

    private static final Set<String> ROLES_DES = Set.of("INGENIEUR_HOMOLOGATION",
"CHEF_DES");

    public void verifier(Partie partie, UUID institutionDeLaLiaison, CollaboratorView
responsable) {
        boolean compatible = responsable.actif() && switch (partie) {
            case DES -> responsable.institutionId() == null && !
Collections.disjoint(responsable.roles(), ROLES_DES);
            case PARTICIPANT ->
responsable.roles().contains("CORRESPONDANT_INSTITUTION")
                                    &&
institutionDeLaLiaison.equals(responsable.institutionId());
        };
        if (!compatible) {
            throw HomologationException.conflict("HOM_ASSIGNMENT_PARTY_MISMATCH",
                partie == Partie.DES
                    ? "Opération incombant au DES : le responsable doit être un
collaborateur du DES"
                    : "Opération incombant au participant : le responsable doit être
un correspondant de l'institution de la liaison",
                Map.of("partie", partie.name(), "responsableId",
responsable.id().toString()));
        }
    }
}
```

```
// application/port/out/ParticipantDirectoryPort.java (extension HOM-08)
record CorrespondentView(UUID userId, String nomAffiche, String fonction) {}
/** GET /api/v1/institutions/{id}/correspondants?fonction= de MS-PARTICIPANT ; le
stub lit ses jeux de données. */
Optional<CorrespondentView> findCorrespondent(UUID institutionId, String fonction);
```

A3. Cas d'utilisation : attribution, réattribution motivée, proposition et attribution en masse

La réattribution exige un motif ; l'ancienne attribution est désactivée et écrite avant l'insertion de la nouvelle pour respecter l'index partiel. Le responsable actif est la source du premier des quatre éléments de clôture (HOM-10) et le destinataire des notifications de déblocage (HOM-07). L'attribution en masse applique la proposition par défaut aux opérations actives sans responsable, hors opérations closes ou hors périmètre, et restitue les opérations sans proposition possible.

```
// application/usecase/AssignOperationService.java
@Service
@Transactional
@RequiredArgsConstructor
public class AssignOperationService implements AssignOperationUseCase {

    private final OperationInstanceRepository operations;
    private final AssignmentRepository assignments;
```

```
    private final CollaboratorDirectoryPort collaborators;   // HOM-04
    private final ParticipantDirectoryPort participants;
    private final AssignmentPolicy policy;
    private final NotificationRequester notifications;
    private final OutboxPublisher outboxPublisher;
    private final Clock clock;

    @Override
    public Assignment attribuer(UUID operationId, UUID responsableId, String motif) {
        OperationInstance op = operations.findByIdForUpdate(operationId)
            .orElseThrow(() ->
HomologationException.notFound("HOM_OPERATION_NOT_FOUND", operationId));
        CollaboratorView responsable = collaborators.findCollaborator(responsableId)
            .orElseThrow(() ->
HomologationException.conflict("HOM_ASSIGNMENT_UNKNOWN_USER", "Utilisateur inconnu",
                Map.of("responsableId", responsableId.toString())));
        policy.verifier(op.getPartie(), op.getLiaison().getInstitutionId(),
responsable);

        Optional<Assignment> actuel =
assignments.findByOperationIdAndActifTrue(operationId);
        if (actuel.isPresent() &&
actuel.get().getResponsableId().equals(responsableId)) return actuel.get();   // sans
effet
        if (actuel.isPresent()) {
            if (motif == null || motif.isBlank()) {
                throw
HomologationException.conflict("HOM_ASSIGNMENT_REASON_REQUIRED", "Motif de
réattribution obligatoire", Map.of());
            }
            actuel.get().desactiver();
            assignments.saveAndFlush(actuel.get());
        }
        Assignment nouvelle = assignments.save(Assignment.nouvelle(op,
responsable.id(), responsable.nomAffiche(),
            op.getPartie(), motif, clock.instant()));
        UUID ancien = actuel.map(Assignment::getResponsableId).orElse(null);
        outboxPublisher.publish("operation", operationId.toString(),
"homologation.operation-assigned.v1",
            new OperationAssignedPayload(operationId, op.getLiaison().getId(),
responsable.id(), op.getPartie(), ancien, motif));
        if (ancien != null) {
            notifications.demander("HOM_OPERATION_REATTRIBUEE",
Destinataires.utilisateurs(ancien, responsable.id()),
                Map.of("operationId", operationId, "code", op.getCode(), "motif",
motif));
        }
        return nouvelle;
    }

    @Override
    @Transactional(readOnly = true)
    public Optional<UUID> proposer(OperationInstance op) {
        String fonction = op.getReferenceOperation().getFonctionResponsable();
        return switch (op.getPartie()) {
            case PARTICIPANT ->
participants.findCorrespondent(op.getLiaison().getInstitutionId(), fonction)
                                            .map(ParticipantDirectoryPort.Corresponde
ntView::userId);
            case DES -> Optional.of(op.getLiaison().getChefDeProjetId());
        };
    }

    @Override
    public MassAssignmentReport attribuerParDefaut(UUID liaisonId) {
        List<String> attribuees = new ArrayList<>(), sansProposition = new
ArrayList<>();
        for (OperationInstance op : operations.actives(liaisonId).values()) {
            if (op.getEtat() == EtatOperation.CLOSE || op.getEtat() ==
```

```
EtatOperation.HORS_PERIMETRE
                    || assignments.existsByOperationIdAndActifTrue(op.getId()))
continue;
            proposer(op).ifPresentOrElse(id -> { attribuer(op.getId(), id, null);
attribuees.add(op.getCode()); },
                                         () -> sansProposition.add(op.getCode()));
        }
        return new MassAssignmentReport(attribuees, sansProposition);
    }
}
```

A4. Ressource REST

Méthode et chemin	Permission	Réponse
`PUT /operations/{id}/responsable` (corps `responsableId`, `motif` exigé en réattribution)	`HOM_OPERATION_ASSIGN`	200, 409
`GET /operations/{id}/attributions`	`HOM_LIAISON_READ`	200 : `proposition`, `historique` (plus récente en tête)
`PUT /liaisons/{id}/operations/responsables` (attribution en masse selon les propositions)	`HOM_OPERATION_ASSIGN`	200 : opérations attribuées et sans proposition

La route d'attribution en masse est une sous-ressource de `/liaisons/{id}/operations` ajoutée par ce récit pour satisfaire la tâche « attribution en masse par liaison ».

```
// web/rest/AssignmentResource.java
@RestController
@RequestMapping("/api/v1")
@RequiredArgsConstructor
public class AssignmentResource {

    private final AssignOperationUseCase useCase;
    private final AssignmentQueryService queries;

    public record AssignRequest(@NotNull UUID responsableId, @Size(max = 500) String
motif) {}

    @PutMapping("/operations/{id}/responsable")
    @PreAuthorize("hasAuthority('HOM_OPERATION_ASSIGN')")
    public AssignmentDTO attribuer(@PathVariable UUID id, @Valid @RequestBody
AssignRequest req) {
        return AssignmentDTO.of(useCase.attribuer(id, req.responsableId(),
req.motif()));
    }

    @GetMapping("/operations/{id}/attributions")
    @PreAuthorize("hasAuthority('HOM_LIAISON_READ')")
    public AttributionsDTO attributions(@PathVariable UUID id) {
        return queries.attributions(id);          // proposition calculée par
useCase.proposer + historique trié
    }

    @PutMapping("/liaisons/{id}/operations/responsables")
    @PreAuthorize("hasAuthority('HOM_OPERATION_ASSIGN')")
    public MassAssignmentReport attribuerEnMasse(@PathVariable UUID id) {
        return useCase.attribuerParDefaut(id);
    }
}
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
// src/test/java/org/gimac/des/homologation/domain/service/AssignmentPolicyTest.java
class AssignmentPolicyTest {

    private final AssignmentPolicy policy = new AssignmentPolicy();
    private final UUID institutionA = UUID.randomUUID(), institutionB =
UUID.randomUUID();

    @Test
    @DisplayName("HOM-08 : un correspondant d'une autre institution est refusé comme
responsable")
    void refusCorrespondantAutreInstitution() {
        var correspondantB = new CollaboratorView(UUID.randomUUID(), "Correspondant
monétique B",
            Set.of("CORRESPONDANT_INSTITUTION"), institutionB, Set.of("MONETIQUE"),
true);
        assertThatThrownBy(() -> policy.verifier(Partie.PARTICIPANT, institutionA,
correspondantB))
            .isInstanceOfSatisfying(HomologationException.class,
                ex ->
assertThat(ex.code()).isEqualTo("HOM_ASSIGNMENT_PARTY_MISMATCH"));
    }

    @Test
    @DisplayName("HOM-08 : un correspondant d'institution est refusé pour une
opération incombant au DES")
    void refusCorrespondantPourOperationDes() {
        var correspondantA = new CollaboratorView(UUID.randomUUID(), "Correspondant
monétique A",
            Set.of("CORRESPONDANT_INSTITUTION"), institutionA, Set.of("MONETIQUE"),
true);
        assertThatThrownBy(() -> policy.verifier(Partie.DES, institutionA,
correspondantA))
            .hasMessageContaining("collaborateur du DES");
    }
}
```

```
//
src/test/java/org/gimac/des/homologation/application/usecase/AssignOperationServiceTe
st.java
@ExtendWith(MockitoExtension.class)
class AssignOperationServiceTest {

    @Mock OperationInstanceRepository operations;
    @Mock AssignmentRepository assignments;
    @Mock CollaboratorDirectoryPort collaborators;
    @Mock ParticipantDirectoryPort participants;
    @Mock NotificationRequester notifications;
    @Mock OutboxPublisher outboxPublisher;
    AssignOperationService service;

    @BeforeEach
    void init() {
        service = new AssignOperationService(operations, assignments, collaborators,
participants, new AssignmentPolicy(),
            notifications, outboxPublisher, Clock.fixed(Instant.parse("2026-10-
08T08:00:00Z"), ZoneOffset.UTC));
    }

    @Test
    @DisplayName("HOM-08 : le correspondant monétique de l'institution est proposé
pour une opération du participant")
    void propositionCorrespondantMonetique() {
        OperationInstance op = OperationInstanceSamples.participant("PM-BR-04",
"MONETIQUE");
        UUID correspondant = UUID.randomUUID();
        when(participants.findCorrespondent(op.getLiaison().getInstitutionId(),
```

```
"MONETIQUE"))
            .thenReturn(Optional.of(new
ParticipantDirectoryPort.CorrespondentView(correspondant, "Correspondant monétique
A", "MONETIQUE")));

        assertThat(service.proposer(op)).contains(correspondant);
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
// src/test/java/org/gimac/des/homologation/web/rest/AssignmentIT.java
class AssignmentIT extends AbstractHomologationIT {

    private static final UUID CHEF_DES = UUID.fromString("00000000-0000-0000-0000-
00000000c001");

    @Test
    @DisplayName("HOM-08 : la réattribution motivée notifie l'ancien et le nouveau
responsable et conserve l'historique")
    void reattributionNotifieLesDeuxResponsables() throws Exception {
        var op = fixtures.operationDesAttribuee(TypeProjet.CARTE_INTERBANCAIRE);   //
attribuée à un ingénieur du DES
        UUID nouveau = fixtures.collaborateurDes("INGENIEUR_HOMOLOGATION");

        mvc.perform(put("/api/v1/operations/{id}/responsable",
op.id()).with(utilisateur(CHEF_DES, "HOM_OPERATION_ASSIGN"))
                .contentType(MediaType.APPLICATION_JSON)
                .content("{\"responsableId\":\"%s\",\"motif\":\"rééquilibrage de
charge\"}".formatted(nouveau)))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.responsableId").value(nouveau.toString()));

        String destinataires = jdbc.queryForObject("select payload -> 'destinataires'
from outbox_event "
            + "where event_type = 'homologation.notification-requested.v1' and
payload -> 'parametres' ->> 'operationId' = ?",
            String.class, op.id().toString());
        assertThat(JsonPath.<List<String>>read(destinataires, "$[*].id"))
            .containsExactlyInAnyOrder(op.responsableId().toString(),
nouveau.toString());

        mvc.perform(get("/api/v1/operations/{id}/attributions",
op.id()).with(utilisateur(CHEF_DES, "HOM_LIAISON_READ")))
            .andExpect(jsonPath("$.historique.length()").value(2))
            .andExpect(jsonPath("$.historique[0].actif").value(true))
            .andExpect(jsonPath("$.historique[1].actif").value(false));
        assertThat(jdbc.queryForObject("select count(*) from outbox_event where
aggregate_id = ? "
            + "and event_type = 'homologation.operation-assigned.v1'", Integer.class,
op.id().toString())).isEqualTo(1);
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: HOM-08 Attribution du responsable principal d'une opération

  Contexte:
    Étant donné un utilisateur authentifié avec le rôle "INGENIEUR_HOMOLOGATION"
    Et une liaison en homologation de l'Établissement de paiement A

  Scénario: Nominal : attribution par défaut
    Étant donné une opération incombant au participant et relevant de la fonction
```

```
"MONETIQUE"
    Quand l'opération est instanciée
    Alors le correspondant monétique de l'Établissement de paiement A est proposé
comme responsable principal
    Et l'attribution confirmée émet l'événement "homologation.operation-assigned.v1"

  Scénario: Alternatif : réattribution motivée
    Étant donné une opération attribuée à un ingénieur d'homologation
    Quand elle est réattribuée à un autre collaborateur du DES avec le motif
"rééquilibrage de charge"
    Alors l'ancien et le nouveau responsable sont notifiés
    Et l'historique des attributions est conservé

  Plan du scénario: Erreur : responsable incompatible avec la partie
    Étant donné une opération incombant au DES
    Quand <responsable> est désigné comme responsable
    Alors l'attribution est refusée avec le code "HOM_ASSIGNMENT_PARTY_MISMATCH" et
le motif de l'incompatibilité

    Exemples:
      | responsable                                              |
      | le correspondant de l'Établissement de paiement A        |
      | le correspondant de la Banque B, autre institution       |

  Scénario: Recette : attribution en masse d'une liaison de type carte interbancaire
    Étant donné une liaison "CARTE_INTERBANCAIRE" de la Banque B dont les opérations
actives n'ont pas de responsable
    Quand l'attribution en masse selon les propositions par défaut est demandée
    Alors chaque opération active du périmètre non close reçoit un responsable
compatible avec sa partie
    Et les opérations sans proposition possible sont restituées
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-ingenieur-homologation
BASE="${GATEWAY_URL}/bff/des/homologation"

# 1. Proposition par défaut et historique d'une opération
curl -s "${BASE}/operations/${OPERATION_ID}/attributions" -H "Authorization: Bearer $
{TOKEN}" \
  | jq '{proposition, historique: [.historique[] | {responsableNom, actif, motif}]}'

# 2. Attribution confirmée au correspondant proposé
curl -s -X PUT "${BASE}/operations/${OPERATION_ID}/responsable" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" \
  -d "{\"responsableId\":\"${CORRESPONDANT_MONETIQUE_A_ID}\"}" | jq '{responsableId,
partie, actif}'

# 3. Attribution en masse d'une liaison carte interbancaire
curl -s -X PUT "${BASE}/liaisons/${LIAISON_CI_ID}/operations/responsables" \
  -H "Authorization: Bearer ${TOKEN}" | jq '{attribuees: (.attribuees | length),
sansProposition}'

# 4. Refus : correspondant désigné sur une opération incombant au DES, attendu 409
HOM_ASSIGNMENT_PARTY_MISMATCH
curl -s -o /tmp/hom08.json -w "%{http_code}\n" -X PUT "${BASE}/operations/$
{OPERATION_DES_ID}/responsable" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" \
  -d "{\"responsableId\":\"${CORRESPONDANT_MONETIQUE_A_ID}\",\"motif\":\"essai\"}"
jq -r .code /tmp/hom08.json

# 5. Refus par permission : un correspondant n'attribue pas, attendu 403
source ./scripts/token.sh test-correspondant-a
curl -s -o /dev/null -w "%{http_code}\n" -X PUT
"${BASE}/operations/${OPERATION_ID}/responsable" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" \
  -d "{\"responsableId\":\"${CORRESPONDANT_MONETIQUE_A_ID}\"}"
```

HOM-09 | Déposer la preuve attendue d'une opération

Élément	Valeur
Points	5
Priorité	Must Have
Sprint cible	Sprint 3
Écrans	DES-09, DES-10
Dépendances	HOM-07, DOC-01, DOC-09, DOC-04

Objectif

Rattacher à une opération la pièce attendue, contrôlée et scellée en WORM par MS-DOCUMENT, par sa référence, son empreinte SHA-256 et sa version, l'opération passant à `PREUVE_DEPOSEE`. Un rejet de la chaîne de contrôle laisse l'opération inchangée ; une nouvelle version annule toute validation en attente sur la version remplacée.

A. Implémentation

A1. Port `DocumentDepositPort` et ses adaptateurs

Le dépôt passe par l'interface interne de MS-DOCUMENT (antivirus et détection de numéro de carte, DOC-04). Un rejet devient `DocumentRejectedException` (422, motif relayé), une panne `DocumentUnavailableException` (503). Les profils `dev` et `test` fixent `gimac.adapters.document-deposit=stub`.

```
// application/port/out/DocumentDepositPort.java
public interface DocumentDepositPort {
    DepositReceipt deposit(DepositRequest request);
    record DepositRequest(UUID operationId, UUID institutionId, String typePiece,
                          String nomFichier, String contentType, byte[] contenu) {}
    record DepositReceipt(String documentRef, String empreinte, boolean scelle) {}
}
```

```
// infrastructure/client/MsDocumentDepositAdapter.java
@Component
@ConditionalOnProperty(name = "gimac.adapters.document-deposit", havingValue =
"real")
class MsDocumentDepositAdapter implements DocumentDepositPort {
    private final RestClient msDocument; // client du socle portant le jeton de
service
    MsDocumentDepositAdapter(@Qualifier("msDocumentRestClient") RestClient c)
{ this.msDocument = c; }

    @Override
    public DepositReceipt deposit(DepositRequest r) {
        var parts = new MultipartBodyBuilder();
        parts.part("fichier", new ByteArrayResource(r.contenu()) {
            @Override public String getFilename() { return r.nomFichier(); }
        }).contentType(MediaType.parseMediaType(r.contentType()));
        parts.part("typePiece", r.typePiece());
        parts.part("objetMetier", "HOM_OPERATION:" + r.operationId());
        try {
            return
msDocument.post().uri("/api/v1/internal/documents").body(parts.build())
                .contentType(MediaType.MULTIPART_FORM_DATA).retrieve().body(DepositRe
ceipt.class);
        } catch (HttpClientErrorException.UnprocessableEntity e) {
```

```
            throw new
DocumentRejectedException(e.getResponseBodyAs(ProblemDetail.class)); // rien n'est
rattaché
        } catch (ResourceAccessException | HttpServerErrorException e) {
            throw new DocumentUnavailableException(e);
        }
    }
}
```

```
// infrastructure/client/DocumentDepositStubAdapter.java
@Component
@ConditionalOnProperty(name = "gimac.adapters.document-deposit", havingValue =
"stub")
class DocumentDepositStubAdapter implements DocumentDepositPort {
    private static final Pattern PAN = Pattern.compile("(?<!\\d)\\d{16}(?!\\d)");

    @Override
    public DepositReceipt deposit(DepositRequest r) {
        String texte = new String(r.contenu(), StandardCharsets.ISO_8859_1);
        // reproduit les deux motifs de rejet de la chaîne de contrôle
        if (texte.contains("EICAR-STANDARD-ANTIVIRUS-TEST-FILE")) throw new
DocumentRejectedException("DOC_MALWARE_DETECTED", "Antivirus");
        if (PAN.matcher(texte).find()) throw new
DocumentRejectedException("DOC_PAN_DETECTED", "Numéro de carte détecté");
        return new DepositReceipt("DOC-STUB-" + UUID.randomUUID(),
Digests.sha256Hex(r.contenu()), true);
    }
}
```

A2. Politique de dépôt et cas d'utilisation

`EvidencePolicy` (domaine pur) refuse les états `VERROUILLEE`, `CLOSE` et `HORS_PERIMETRE`, et contrôle le type attendu. Choix retenu : `preuveAttendue` est indexée par catégorie de participant, portée par le type de projet de la liaison (clé `*` pour la valeur commune). L'appel à MS-DOCUMENT a lieu hors transaction ; le rattachement, l'invalidation de la version courante, l'annulation des validations en attente (clôtures actives sans jeton, HOM-10) et l'outbox forment une seule transaction.

```
// domain/service/EvidencePolicy.java
public final class EvidencePolicy {
    private static final Set<EtatOperation> ETATS_DEPOT = EnumSet.of(ELIGIBLE,
PREUVE_DEPOSEE,
        VALIDEE_HORODATAGE_EN_ATTENTE, ROUVERTE);

    public void checkState(EtatOperation etat) {
        if (!ETATS_DEPOT.contains(etat))
            throw new DomainRuleException("HOM_EVIDENCE_STATE_FORBIDDEN", "Dépôt
impossible à l'état " + etat, Map.of("etat", etat));
    }

    public void checkExpectedType(Map<String, List<String>> attendu, TypeProjet
categorie, String typePiece) {
        List<String> admis = attendu.getOrDefault(categorie.name(),
attendu.getOrDefault("*", List.of()));
        if (!admis.contains(typePiece))
            throw new DomainRuleException("HOM_EVIDENCE_TYPE_NOT_EXPECTED", "Type de
pièce non attendu",
                Map.of("typePiece", typePiece, "typesAttendus", admis));
    }
}
```

```
// application/usecase/SubmitEvidenceService.java
@Service
@RequiredArgsConstructor
```

```
public class SubmitEvidenceService implements SubmitEvidenceUseCase {
    private final OperationInstanceRepository operations;
    private final EvidenceReferenceRepository evidences;
    private final ClosureRecordRepository closures;
    private final DocumentDepositPort documentDeposit;
    private final OutboxPublisher outboxPublisher;
    private final TransactionTemplate tx;
    private final Clock clock;
    private final EvidencePolicy policy = new EvidencePolicy();

    @Override
    public EvidenceReference submit(UUID operationId, EvidenceUpload up, CurrentUser
deposant) {
        OperationInstance op =
operations.findDetailedById(operationId).orElseThrow(NotFoundException::operation);
        policy.checkState(op.getEtat());

policy.checkExpectedType(op.getReferenceOperation().preuveAttendueParCategorie(),
            op.getLiaison().getTypeProjet(), up.typePiece());
        DepositReceipt recu = documentDeposit.deposit(new DepositRequest(operationId,
            op.getLiaison().getInstitutionId(), up.typePiece(), up.nomFichier(),
up.contentType(), up.contenu()));

        return tx.execute(status -> {
            OperationInstance locked =
operations.findByIdForUpdate(operationId).orElseThrow();
            policy.checkState(locked.getEtat()); // revérifié sous verrou
            Instant now = clock.instant();
            int version = evidences.findMaxVersion(operationId).orElse(0) + 1;
            evidences.invalidateCurrent(operationId); // l'ancienne version reste
consultable (valide = false)
            int annulees = closures.cancelPendingValidations(operationId,
"VERSION_DE_PREUVE_REMPLACEE", now);
            EvidenceReference ref = evidences.save(new
EvidenceReference().operation(locked)
                .documentRef(recu.documentRef()).empreinte(recu.empreinte()).version(
version)
                .typePiece(up.typePiece()).deposantId(deposant.id()).dateDepot(now).s
celle(recu.scelle()).valide(true));
            locked.etat(PREUVE_DEPOSEE).dateEtat(now);
            // type d'agrégat "Operation" : sujet homologation.operation
            outboxPublisher.publish("Operation", operationId.toString(),
"homologation.evidence-submitted.v1",
                new EvidenceSubmittedV1(operationId, locked.getLiaison().getId(),
ref.getDocumentRef(),
                    ref.getEmpreinte(), version, up.typePiece(), deposant.id(), now,
annulees));
            return ref;
        });
    }
}
```

`ClosureRecordRepository.cancelPendingValidations` est une requête `@Modifying` qui passe à `actif = false` (motif et date renseignés) les clôtures actives dont `jetonHorodatage` est nul. `Digests.sha256Hex(byte[] | String)` (domaine) encapsule `MessageDigest` et `HexFormat` pour tout le lot.

A3. Confirmation du scellement et ressource REST

```
// infrastructure/messaging/consumer/DocumentSealedConsumer.java
@Component
@RequiredArgsConstructor
class DocumentSealedConsumer {
    private static final String GROUP = "mshomologation-document";
    private final IdempotentConsumer idempotentConsumer;
    private final EvidenceReferenceRepository evidences;
    private final RejectedRecordRepository rejected;
```

```
    private final KafkaTemplate<String, String> kafka;
    private final ObjectMapper mapper;
    private final Clock clock;

    @KafkaListener(topics = "document.document", groupId = GROUP)
    @Transactional
    public void onMessage(ConsumerRecord<String, String> rec) throws
JsonProcessingException {
        JsonNode ev = mapper.readTree(rec.value());
        if (!"document.document-sealed.v1".equals(ev.path("type").asText()))
return; // autres types du sujet
        String empreinte = ev.path("data").path("empreinte").asText();
        idempotentConsumer.handleOnce(ev.path("id").asText(), GROUP, () ->

evidences.findByDocumentRef(ev.path("data").path("documentRef").asText()).ifPresent(r
ef -> {
                if (ref.getEmpreinte().equalsIgnoreCase(empreinte)) {
                    ref.setScelle(true); // deuxième élément de clôture acquis
                } else { // empreinte divergente : rebut et sujet de rebut
                    rejected.save(new
RejectedRecord().source("document.document").chargeUtile(rec.value())
                        .empreinte(empreinte).motif("EMPREINTE_DIVERGENTE").dateRejet
(clock.instant()).relance(false));
                    kafka.send("document.document.dlq", rec.key(), rec.value());
                }
            }));
    }
}
```

```
// web/rest/OperationEvidenceResource.java
@RestController
@RequestMapping("/api/v1/operations/{id}/preuves")
@RequiredArgsConstructor
class OperationEvidenceResource {
    private final SubmitEvidenceUseCase submitEvidence;
    private final EvidenceReferenceRepository evidences;
    private final EvidenceMapper mapper;
    private final CurrentUserProvider currentUser; // socle : utilisateur extrait du
jeton

    @PostMapping(consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
    @PreAuthorize("hasAuthority('HOM_EVIDENCE_SUBMIT')")
    ResponseEntity<EvidenceDTO> submit(@PathVariable UUID id, @RequestParam String
typePiece,
                                       @RequestPart("fichier") MultipartFile f)
throws IOException {
        var ref = submitEvidence.submit(id, new EvidenceUpload(typePiece,
f.getOriginalFilename(),
            f.getContentType(), f.getBytes()), currentUser.get());
        return ResponseEntity.status(HttpStatus.CREATED).body(mapper.toDto(ref));
    }

    @GetMapping
    @PreAuthorize("hasAuthority('HOM_LIAISON_READ')")
    List<EvidenceDTO> list(@PathVariable UUID id) { // toutes les versions, la plus
récente en tête
        return
evidences.findByOperationIdOrderByVersionDesc(id).stream().map(mapper::toDto).toList(
);
    }
}
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
//
src/test/java/org/gimac/des/homologation/application/usecase/SubmitEvidenceServiceTes
t.java
```

```
@ExtendWith(MockitoExtension.class)
class SubmitEvidenceServiceTest {
    @Mock OperationInstanceRepository operations;
    @Mock EvidenceReferenceRepository evidences;
    @Mock ClosureRecordRepository closures;
    @Mock DocumentDepositPort documentDeposit;
    @Mock OutboxPublisher outbox;
    @Mock TransactionTemplate tx;
    SubmitEvidenceService service;

    @BeforeEach
    void setUp() {
        service = new SubmitEvidenceService(operations, evidences, closures,
documentDeposit, outbox, tx, Clock.systemUTC());
    }

    @Test
    @DisplayName("HOM-09 : un type de pièce non attendu est refusé sans appel à MS-
DOCUMENT")
    void refuseUnexpectedType() {
        OperationInstance op = OperationFixtures.operation(ELIGIBLE,
"RAPPORT_PARAMETRAGE");
        when(operations.findDetailedById(op.getId())).thenReturn(Optional.of(op));
        assertThatThrownBy(() -> service.submit(op.getId(),
OperationFixtures.upload("CAPTURE_ECRAN"), OperationFixtures.ingenieur()))
            .hasFieldOrPropertyWithValue("code", "HOM_EVIDENCE_TYPE_NOT_EXPECTED");
        verifyNoInteractions(documentDeposit, tx);
    }

    @Test
    @DisplayName("HOM-09 : une nouvelle version annule la validation en attente sur
la version remplacée")
    void newVersionCancelsPendingValidation() {
        OperationInstance op =
OperationFixtures.operation(VALIDEE_HORODATAGE_EN_ATTENTE, "RAPPORT_PARAMETRAGE");
        when(operations.findDetailedById(op.getId())).thenReturn(Optional.of(op));
        when(operations.findByIdForUpdate(op.getId())).thenReturn(Optional.of(op));
        when(documentDeposit.deposit(any())).thenReturn(new DepositReceipt("DOC-2",
"ab".repeat(32), true));
        when(evidences.findMaxVersion(op.getId())).thenReturn(Optional.of(1));
        when(evidences.save(any())).thenAnswer(inv -> inv.getArgument(0));
        when(tx.execute(any())).thenAnswer(inv -> inv.<TransactionCallback<?
>>getArgument(0).doInTransaction(null));

        EvidenceReference ref = service.submit(op.getId(),
OperationFixtures.upload("RAPPORT_PARAMETRAGE"), OperationFixtures.ingenieur());

        assertThat(ref.getVersion()).isEqualTo(2);
        assertThat(op.getEtat()).isEqualTo(PREUVE_DEPOSEE);
        verify(evidences).invalidateCurrent(op.getId());
        verify(closures).cancelPendingValidations(eq(op.getId()),
eq("VERSION_DE_PREUVE_REMPLACEE"), any());
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

`HomologationFixtures` (fabrique de données de test, enrichie à chaque récit) et `TestJwt.as(userId, permissions...)` (jeton de test Spring Security) servent à tous les tests d'intégration du lot.

```
// src/test/java/org/gimac/des/homologation/web/rest/OperationEvidenceIT.java
@SpringBootTest(properties = "gimac.adapters.document-deposit=real")
@AutoConfigureMockMvc
@Testcontainers
class OperationEvidenceIT {
```

```
    @Container @ServiceConnection static PostgreSQLContainer<?> postgres = new
PostgreSQLContainer<>("postgres:16-alpine");
    @Container @ServiceConnection static KafkaContainer kafka = new
KafkaContainer("apache/kafka:3.8.0");
    @RegisterExtension static WireMockExtension msDocument =
WireMockExtension.newInstance()
        .options(wireMockConfig().dynamicPort()).build();

    @DynamicPropertySource
    static void props(DynamicPropertyRegistry r)
{ r.add("gimac.clients.msdocument.base-url", msDocument::baseUrl); }

    @Autowired MockMvc mvc;
    @Autowired HomologationFixtures fixtures;
    @Autowired OperationInstanceRepository operations;
    @Autowired OutboxEventRepository outbox;

    @Test
    @DisplayName("HOM-09 : un rejet de la chaîne de contrôle est sans effet sur
l'opération")
    void rejectedByControlChain() throws Exception {
        UUID opId = fixtures.eligibleOperation(TypeProjet.PAIEMENT_MOBILE,
"RAPPORT_PARAMETRAGE");

msDocument.stubFor(post("/api/v1/internal/documents").willReturn(aResponse().withStat
us(422)
            .withHeader("Content-Type",
"application/problem+json").withBody("{\"code\":\"DOC_PAN_DETECTED\"}")));

        mvc.perform(multipart("/api/v1/operations/{id}/preuves", opId)
                .file(new MockMultipartFile("fichier", "rapport.pdf",
"application/pdf", "x".getBytes()))
                .param("typePiece", "RAPPORT_PARAMETRAGE")
                .with(TestJwt.as(fixtures.ingenieurId(), "HOM_EVIDENCE_SUBMIT")))
            .andExpect(status().isUnprocessableEntity())
            .andExpect(jsonPath("$.code").value("DOC_PAN_DETECTED"));


assertThat(operations.findById(opId)).get().extracting(OperationInstance::getEtat).is
EqualTo(ELIGIBLE);
        assertThat(outbox.countByAggregateId(opId.toString())).isZero();
    }
}
```

Un second cas du même fichier (contrat) répond 201 depuis WireMock avec `documentRef`, `empreinte` et `scelle`, puis vérifie que la réponse de MS-HOMOLOGATION reprend ces trois valeurs avec `version = 1`.

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: HOM-09 Dépôt de la preuve attendue d'une opération

  Contexte:
    Étant donné une liaison de l'"Établissement de paiement A" de type
"PAIEMENT_MOBILE" en homologation
    Et un utilisateur connecté ayant le rôle "INGENIEUR_HOMOLOGATION"

  Scénario: Nominal : dépôt accepté
    Étant donné une opération éligible dont la preuve attendue est de type
"RAPPORT_PARAMETRAGE"
    Quand l'ingénieur d'homologation dépose une pièce du type "RAPPORT_PARAMETRAGE"
    Alors la réponse porte la référence, l'empreinte et l'état scellé renvoyés par
MS-DOCUMENT
    Et l'opération passe à l'état "PREUVE_DEPOSEE"
    Et l'événement "homologation.evidence-submitted.v1" est émis
```

```
  Scénario: Alternatif : nouvelle version de la preuve
    Étant donné une opération dont la preuve a été déposée en version 1
    Et une validation en attente d'horodatage portant sur la version 1
    Quand une nouvelle version de la pièce est déposée
    Alors la version 2 est rattachée et la version 1 reste consultable
    Et la validation portant sur la version 1 est annulée

  Plan du scénario: Erreur : pièce rejetée par la chaîne de contrôle
    Étant donné une opération éligible dont la preuve attendue est de type
"RAPPORT_PARAMETRAGE"
    Quand le dépôt d'une pièce <nature> est tenté
    Alors la réponse a le statut 422 avec le code "<code>"
    Et l'opération reste à l'état "ELIGIBLE" sans référence de preuve rattachée

    Exemples:
      | nature                            | code                 |
      | contenant un numéro de carte      | DOC_PAN_DETECTED     |
      | signalée par l'analyse antivirale | DOC_MALWARE_DETECTED |
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-ingenieur-homologation
BASE="${GATEWAY_URL}/bff/des/homologation"
OP_ID=$(curl -s "${BASE}/liaisons/${LIAISON_ID}/operations?etat=ELIGIBLE" \
  -H "Authorization: Bearer ${TOKEN}" | jq -r '.[0].id')

# Nominal : dépôt accepté (201)
curl -s -X POST "${BASE}/operations/${OP_ID}/preuves" -H "Authorization: Bearer $
{TOKEN}" \
  -F "typePiece=RAPPORT_PARAMETRAGE" -F "fichier=@./fixtures/rapport-
parametrage.pdf;type=application/pdf" \
  | jq '{documentRef, empreinte, version, scelle}'

# Refus : pièce contenant un numéro de carte de test (attendu : 422)
curl -s -o /dev/null -w "%{http_code}\n" -X POST
"${BASE}/operations/${OP_ID}/preuves" \
  -H "Authorization: Bearer ${TOKEN}" \
  -F "typePiece=RAPPORT_PARAMETRAGE" -F "fichier=@./fixtures/pan-de-
test.txt;type=text/plain"
```

HOM-10 | Clore une opération selon la règle des quatre éléments

Élément	Valeur
Points	8
Priorité	Must Have
Sprint cible	Sprint 3
Écrans	DES-09, DES-10
Dépendances	HOM-08, HOM-09, SOC-12, IAM-07, AUD-01

Objectif

Clore une opération seulement lorsque sont réunis un responsable, une preuve scellée en WORM, une validation par un tiers distinct du responsable et du déposant (après réauthentification FIDO2) et un jeton RFC 3161 vérifié. Tout refus nomme l'élément manquant ; une autorité d'horodatage muette laisse l'opération à `VALIDEE_HORODATAGE_EN_ATTENTE` jusqu'à obtention du jeton, sans nouvelle validation.

A. Implémentation

A1. Garde de clôture `ClosureGuard` (service de domaine pur)

La garde reçoit des faits déjà assemblés (aucune dépendance JPA) et renvoie le premier élément manquant sous le code `HOM_CLOSURE_ELEMENT_MISSING` (409), avec `details.element`. Elle sert aussi à `GET /operations/{id}/elements-de-cloture` (écran DES-10).

```
// domain/service/ClosureGuard.java
public final class ClosureGuard {

    public enum Element { RESPONSABLE, PREUVE_SCELLEE, VALIDATION_TIERS_DISTINCT,
HORODATAGE_RFC3161 }

    public record ClosureFacts(UUID responsableId, UUID deposantId, String
empreintePreuve,
                               boolean preuveScellee, UUID validateurId, boolean
jetonVerifie) {}

    public record ElementStatus(Element element, boolean present, String libelle) {}

    public List<ElementStatus> status(ClosureFacts f) {
        boolean distinct = f.validateurId() != null && !
f.validateurId().equals(f.responsableId())
            && !f.validateurId().equals(f.deposantId());
        return List.of(
            new ElementStatus(Element.RESPONSABLE, f.responsableId() != null,
"responsable principal actif"),
            new ElementStatus(Element.PREUVE_SCELLEE, f.preuveScellee(), "preuve
courante scellée en WORM"),
            new ElementStatus(Element.VALIDATION_TIERS_DISTINCT, distinct,
"validateur distinct du responsable et du déposant"),
            new ElementStatus(Element.HORODATAGE_RFC3161, f.jetonVerifie(), "jeton
RFC 3161 vérifié"));
    }

    /** À la validation : les trois premiers éléments, avant toute demande de jeton.
*/
    public void checkValidation(ClosureFacts f) { refuseFirstMissing(f, false); }
```

```
    /** À la clôture effective : les quatre éléments. */
    public void checkClosure(ClosureFacts f) { refuseFirstMissing(f, true); }

    private void refuseFirstMissing(ClosureFacts f, boolean avecJeton) {
        status(f).stream()
            .filter(s -> avecJeton || s.element() != Element.HORODATAGE_RFC3161)
            .filter(s -> !s.present()).findFirst()
            .ifPresent(m -> {
                String message = m.element() == Element.VALIDATION_TIERS_DISTINCT
                    ? "validation par un tiers distinct exigée" : "Élément de clôture
manquant : " + m.libelle();
                throw new DomainRuleException("HOM_CLOSURE_ELEMENT_MISSING", message,
Map.of("element", m.element()));
            });
    }
}
```

A2. Ports `ReauthenticationPort` et `TimestampAuthorityPort`

La réauthentification est vérifiée par MS-IAM (IAM-07) pour l'objet `HOM_OPERATION_VALIDATE:<operationId>`. L'horodatage suit RFC 3161 : demande `SHA-256` avec nonce aléatoire de 64 bits, contrôle de la réponse contre la demande (statut, empreinte, nonce) puis de la signature de l'autorité ; toute anomalie lève `TimestampUnavailableException` et aucun jeton n'est conservé. L'adaptateur réel utilise `bcpkix-jdk18on`.

```
// application/port/out/ReauthenticationPort.java
public interface ReauthenticationPort {
    boolean verify(String assertion, UUID userId, String purpose);
}

// application/port/out/TimestampAuthorityPort.java
public interface TimestampAuthorityPort {
    StampedToken stamp(byte[] sha256, BigInteger nonce);
    record StampedToken(byte[] jetonDer, Instant dateHorodatage) {}
}
```

```
// infrastructure/client/Rfc3161TimestampAdapter.java
@Component
@ConditionalOnProperty(name = "gimac.adapters.timestamp-authority", havingValue =
"real")
class Rfc3161TimestampAdapter implements TimestampAuthorityPort {
    private final RestClient tsa;                // ${gimac.clients.tsa.base-url},
délai de 5 s
    private final X509Certificate certificatTsa; // lu depuis $
{gimac.clients.tsa.certificate}

    Rfc3161TimestampAdapter(@Qualifier("tsaRestClient") RestClient tsa,
                            @Value("${gimac.clients.tsa.certificate}") Resource cert)
throws Exception {
        this.tsa = tsa;
        try (InputStream in = cert.getInputStream()) {
            this.certificatTsa = (X509Certificate)
CertificateFactory.getInstance("X.509").generateCertificate(in);
        }
    }

    @Override
    public StampedToken stamp(byte[] sha256, BigInteger nonce) {
        TimeStampRequestGenerator generator = new TimeStampRequestGenerator();
        generator.setCertReq(true);
        TimeStampRequest demande = generator.generate(TSPAlgorithms.SHA256, sha256,
nonce);
        try {
```

```
            byte[] brut =
tsa.post().contentType(MediaType.parseMediaType("application/timestamp-query"))
                .body(demande.getEncoded()).retrieve().body(byte[].class);
            TimeStampResponse reponse = new TimeStampResponse(brut);
            reponse.validate(demande); // statut, empreinte et nonce identiques à la
demande
            TimeStampToken jeton = reponse.getTimeStampToken();
            if (jeton == null) throw new TimestampUnavailableException("Refus de
l'autorité : " + reponse.getStatusString());
            jeton.validate(new
JcaSimpleSignerInfoVerifierBuilder().build(certificatTsa)); // signature de
l'autorité
            return new StampedToken(jeton.getEncoded(),
jeton.getTimeStampInfo().getGenTime().toInstant());
        } catch (RestClientException | IOException | TSPException |
OperatorCreationException e) {
            throw new TimestampUnavailableException(e);
        }
    }
}
```

```
// infrastructure/client/TimestampAuthorityStubAdapter.java  (profils dev et test)
@Component
@ConditionalOnProperty(name = "gimac.adapters.timestamp-authority", havingValue =
"stub")
public class TimestampAuthorityStubAdapter implements TimestampAuthorityPort {
    private final AtomicBoolean disponible = new AtomicBoolean(true);
    public void setDisponible(boolean valeur) { disponible.set(valeur); } // piloté
par les tests

    @Override
    public StampedToken stamp(byte[] sha256, BigInteger nonce) {
        if (!disponible.get()) throw new TimestampUnavailableException("autorité de
substitution indisponible");
        return new StampedToken(("STUB-TSR:" + HexFormat.of().formatHex(sha256) + ":"
+ nonce).getBytes(), Instant.now());
    }
}
```

`MsIamReauthenticationAdapter` (réel) poste `{assertion, userId, purpose}` sur `/api/v1/internal/reauthentications/verification` et lit `{valide}` ; `ReauthenticationStubAdapter` n'accepte que l'assertion `stub-ok:<purpose>`. Sélection par `gimac.adapters.reauthentication`.

A3. Validation : distinction des rôles et empreinte de clôture

La validation est enregistrée dans une première transaction (état `VALIDEE_HORODATAGE_EN_ATTENTE`, `ClosureRecord` actif sans jeton). L'empreinte de clôture est le SHA-256 de (opération, empreinte de la preuve, validateur, instant). La demande de jeton suit, hors transaction.

```
// application/usecase/ValidateOperationService.java
@Service
@RequiredArgsConstructor
public class ValidateOperationService implements ValidateOperationUseCase {
    private final ReauthenticationPort reauthentication;
    private final OperationInstanceRepository operations;
    private final ClosureRecordRepository closures;
    private final ClosureFactsAssembler facts;   // responsable actif (HOM-08),
preuve valide (HOM-09)
    private final ClosureCompletionService completion;
    private final TransactionTemplate tx;
    private final Clock clock;
    private final ClosureGuard guard = new ClosureGuard();
```

```
    @Override
    public OperationInstance validate(UUID operationId, String assertion, CurrentUser
validateur) {
        if (assertion == null || !reauthentication.verify(assertion, validateur.id(),
"HOM_OPERATION_VALIDATE:" + operationId)) {
            throw new ReauthenticationRequiredException(); // 403, code
HOM_REAUTH_REQUIRED
        }
        UUID closureId = tx.execute(status -> {
            OperationInstance op =
operations.findByIdForUpdate(operationId).orElseThrow(NotFoundException::operation);
            if (op.getEtat() != EtatOperation.PREUVE_DEPOSEE)
                throw new DomainRuleException("HOM_OPERATION_TRANSITION_FORBIDDEN",
"Validation impossible", Map.of("etat", op.getEtat()));
            ClosureGuard.ClosureFacts f = facts.of(op, validateur.id(), false);
            guard.checkValidation(f); // responsable, preuve scellée, tiers distinct
            Instant instant = clock.instant();
            String empreinte = Digests.sha256Hex(String.join("|",
op.getId().toString(), f.empreintePreuve(),
                validateur.id().toString(), instant.toString()));
            op.etat(EtatOperation.VALIDEE_HORODATAGE_EN_ATTENTE).dateEtat(instant);
            return closures.save(new
ClosureRecord().operation(op).validateurId(validateur.id())
                .dateValidation(instant).empreinteCloture(empreinte).actif(true)).get
Id();
        });
        return completion.complete(closureId); // un échec d'horodatage laisse la
validation acquise
    }
}
```

A4. Horodatage, clôture, outbox et réévaluation des successeurs

`complete` est idempotente : sans effet si la clôture est déjà horodatée ou annulée (nouvelle version de preuve, HOM-09). Le jeton, l'état `CLOSE`, le décrément des degrés entrants des successeurs dans le périmètre et les événements sont écrits dans une seule transaction. Le travail planifié rappelle `complete` pour chaque clôture en attente, sans nouvelle validation ; plusieurs réplicas peuvent l'exécuter puisque la seconde vérification se fait sous verrou.

```
// application/usecase/ClosureCompletionService.java
@Service
@RequiredArgsConstructor
public class ClosureCompletionService {
    private static final SecureRandom RANDOM = new SecureRandom();
    private final ClosureRecordRepository closures;
    private final OperationInstanceRepository operations;
    private final TimestampAuthorityPort timestampAuthority;
    private final ClosureFactsAssembler facts;
    private final OutboxPublisher outboxPublisher;
    private final TransactionTemplate tx;
    private final ClosureGuard guard = new ClosureGuard();

    public OperationInstance complete(UUID closureId) {
        ClosureRecord attente = closures.findById(closureId).orElseThrow();
        UUID operationId = attente.getOperation().getId();
        if (!attente.getActif() || attente.getJetonHorodatage() != null) return
operations.findById(operationId).orElseThrow();
        StampedToken jeton;
        try {
            jeton =
timestampAuthority.stamp(HexFormat.of().parseHex(attente.getEmpreinteCloture()), new
BigInteger(64, RANDOM));
        } catch (TimestampUnavailableException e) {
            return operations.findById(operationId).orElseThrow(); // reste
VALIDEE_HORODATAGE_EN_ATTENTE
        }
```

```
        return tx.execute(status -> {
            OperationInstance op =
operations.findByIdForUpdate(operationId).orElseThrow();
            ClosureRecord rec = closures.findById(closureId).orElseThrow();
            if (!rec.getActif() || rec.getJetonHorodatage() != null) return op; //
traité entre-temps

rec.jetonHorodatage(Base64.getEncoder().encodeToString(jeton.jetonDer())).dateHorodat
age(jeton.dateHorodatage());
            guard.checkClosure(facts.of(op, rec.getValidateurId(), true));
            op.etat(EtatOperation.CLOSE).dateEtat(jeton.dateHorodatage());
            outboxPublisher.publish("Operation", op.getId().toString(),
"homologation.operation-closed.v1",
                new OperationClosedV1(op.getId(), op.getLiaison().getId(),
op.getCode(), op.getServiceCode(),
                    rec.getValidateurId(), rec.getEmpreinteCloture(),
rec.getDateHorodatage()));
            for (OperationInstance s :
operations.findSuccessorsForUpdate(op.getLiaison().getId(), op.getCode())) {
                s.degreEntrant(s.getDegreEntrant() - 1);
                if (s.getDegreEntrant() == 0 && s.getEtat() ==
EtatOperation.VERROUILLEE) {
                    s.etat(EtatOperation.ELIGIBLE).dateEtat(jeton.dateHorodatage());
                    outboxPublisher.publish("Operation", s.getId().toString(),
"homologation.operation-unlocked.v1",
                        new OperationUnlockedV1(s.getId(), op.getLiaison().getId(),
s.getCode(), op.getCode()));
                }
            }
            return op;
        });
    }
}
```

```
// infrastructure/persistence/OperationInstanceRepository.java (extrait)
@Query(value = """
    select oi.* from operation_instance oi join reference_operation ro on ro.id =
oi.reference_operation_id
    where oi.liaison_id = :liaisonId and oi.etat <> 'HORS_PERIMETRE'
      and cast(ro.predecesseurs as jsonb) @> jsonb_build_array(cast(:code as text))
    for update of oi""", nativeQuery = true)
List<OperationInstance> findSuccessorsForUpdate(UUID liaisonId, String code);

// infrastructure/scheduling/PendingTimestampRetryJob.java
@Component
@RequiredArgsConstructor
public class PendingTimestampRetryJob {
    private final ClosureRecordRepository closures;
    private final ClosureCompletionService completion;

    @Scheduled(fixedDelayString = "${gimac.timestamp.retry-delay-ms:60000}")
    public void retryPending() { // clôtures actives sans jeton, les plus anciennes
d'abord
        closures.findPendingTimestampIds(PageRequest.of(0,
50)).forEach(completion::complete);
    }
}
```

A5. Ressource REST

```
// web/rest/OperationClosureResource.java
@RestController
@RequestMapping("/api/v1/operations/{id}")
@RequiredArgsConstructor
class OperationClosureResource {
    private final ValidateOperationUseCase validateOperation;
    private final ClosureElementsQuery closureElements; // faits de la clôture active
+ guard.status(...)
```

```
    private final OperationMapper mapper;
    private final CurrentUserProvider currentUser;

    @PostMapping("/validation")
    @PreAuthorize("hasAuthority('HOM_OPERATION_VALIDATE')")
    ResponseEntity<OperationDTO> validate(@PathVariable UUID id,
            @RequestHeader(name = "X-Reauth-Assertion", required = false) String
assertion) {
        OperationInstance op = validateOperation.validate(id, assertion,
currentUser.get());
        // 200 si close ; 202 si le jeton reste à obtenir (traitement différé)
        return ResponseEntity.status(op.getEtat() == EtatOperation.CLOSE ?
HttpStatus.OK : HttpStatus.ACCEPTED)
            .body(mapper.toDto(op));
    }

    @GetMapping("/elements-de-cloture")
    @PreAuthorize("hasAuthority('HOM_LIAISON_READ')")
    List<ClosureGuard.ElementStatus> elements(@PathVariable UUID id) { return
closureElements.status(id); }
}
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
// src/test/java/org/gimac/des/homologation/domain/service/ClosureGuardTest.java
class ClosureGuardTest {
    static final UUID RESP = UUID.randomUUID(), DEPOSANT = UUID.randomUUID(), TIERS =
UUID.randomUUID();
    final ClosureGuard guard = new ClosureGuard();

    static Stream<Arguments> absences() {
        return Stream.of(
            arguments(new ClosureFacts(null, DEPOSANT, "ab", true, TIERS, true),
Element.RESPONSABLE),
            arguments(new ClosureFacts(RESP, DEPOSANT, "ab", false, TIERS, true),
Element.PREUVE_SCELLEE),
            arguments(new ClosureFacts(RESP, DEPOSANT, "ab", true, RESP, true),
Element.VALIDATION_TIERS_DISTINCT),
            arguments(new ClosureFacts(RESP, DEPOSANT, "ab", true, TIERS, false),
Element.HORODATAGE_RFC3161));
    }

    @ParameterizedTest(name = "{1}")
    @MethodSource("absences")
    @DisplayName("HOM-10 : chacune des quatre absences possibles produit un refus
nommé")
    void eachMissingElementIsNamed(ClosureFacts facts, Element attendu) {
        assertThatThrownBy(() -> guard.checkClosure(facts))
            .isInstanceOf(DomainRuleException.class)
            .extracting("details").asInstanceOf(InstanceOfAssertFactories.MAP).contai
nsEntry("element", attendu);
    }

    @Test
    @DisplayName("HOM-10 : la validation par le déposant est refusée comme non
distincte")
    void depositorCannotValidate() {
        assertThatThrownBy(() -> guard.checkValidation(new ClosureFacts(RESP,
DEPOSANT, "ab", true, DEPOSANT, false)))
            .hasMessage("validation par un tiers distinct exigée");
    }
}
```

```
//
src/test/java/org/gimac/des/homologation/application/usecase/ClosureCompletionService
Test.java
@ExtendWith(MockitoExtension.class)
```

```
class ClosureCompletionServiceTest {
    @Mock ClosureRecordRepository closures;
    @Mock OperationInstanceRepository operations;
    @Mock TimestampAuthorityPort tsa;
    @Mock ClosureFactsAssembler facts;
    @Mock OutboxPublisher outbox;
    @Mock TransactionTemplate tx;
    @InjectMocks ClosureCompletionService service;

    @Test
    @DisplayName("HOM-10 : autorité muette, l'opération reste validée en attente et
rien n'est publié")
    void timestampUnavailableKeepsPending() {
        OperationInstance op =
OperationFixtures.operation(EtatOperation.VALIDEE_HORODATAGE_EN_ATTENTE,
"RAPPORT_PARAMETRAGE");
        ClosureRecord rec = OperationFixtures.pendingClosure(op);
        when(closures.findById(rec.getId())).thenReturn(Optional.of(rec));
        when(operations.findById(op.getId())).thenReturn(Optional.of(op));
        when(tsa.stamp(any(), any())).thenThrow(new
TimestampUnavailableException("délai dépassé"));


assertThat(service.complete(rec.getId()).getEtat()).isEqualTo(EtatOperation.VALIDEE_H
ORODATAGE_EN_ATTENTE);
        assertThat(rec.getJetonHorodatage()).isNull();
        verifyNoInteractions(tx, outbox);
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
// src/test/java/org/gimac/des/homologation/web/rest/OperationClosureIT.java
@SpringBootTest(properties = {"gimac.adapters.reauthentication=real",
"gimac.adapters.timestamp-authority=stub"})
@AutoConfigureMockMvc
@Testcontainers
class OperationClosureIT {
    @Container @ServiceConnection static PostgreSQLContainer<?> postgres = new
PostgreSQLContainer<>("postgres:16-alpine");
    @Container @ServiceConnection static KafkaContainer kafka = new
KafkaContainer("apache/kafka:3.8.0");
    @RegisterExtension static WireMockExtension msIam =
WireMockExtension.newInstance()
        .options(wireMockConfig().dynamicPort()).build();

    @DynamicPropertySource
    static void props(DynamicPropertyRegistry r) { r.add("gimac.clients.msiam.base-
url", msIam::baseUrl); }

    @Autowired MockMvc mvc;
    @Autowired HomologationFixtures fixtures;
    @Autowired TimestampAuthorityStubAdapter tsa;
    @Autowired PendingTimestampRetryJob retryJob;
    @Autowired OperationInstanceRepository operations;
    @Autowired ClosureRecordRepository closures;
    @Autowired OutboxEventRepository outbox;

    @Test
    @DisplayName("HOM-10 : autorité d'horodatage indisponible puis reprise, sans
nouvelle validation")
    void timestampOutageThenRecovery() throws Exception {
        UUID opId = fixtures.operationWithSealedEvidence(); // responsable et
déposant distincts du validateur

msIam.stubFor(post("/api/v1/internal/reauthentications/verification").willReturn(okJs
on("{\"valide\":true}")));
```

```
        tsa.setDisponible(false);

        mvc.perform(post("/api/v1/operations/{id}/validation", opId).header("X-
Reauth-Assertion", "assertion-fido2")
                .with(TestJwt.as(fixtures.chefDesId(), "HOM_OPERATION_VALIDATE")))
            .andExpect(status().isAccepted())
            .andExpect(jsonPath("$.etat").value("VALIDEE_HORODATAGE_EN_ATTENTE"));
        assertThat(outbox.countByEventType("homologation.operation-
closed.v1")).isZero();

        tsa.setDisponible(true);
        retryJob.retryPending();


assertThat(operations.findById(opId)).get().extracting(OperationInstance::getEtat).is
EqualTo(EtatOperation.CLOSE);
        assertThat(closures.findByOperationId(opId)).singleElement()
            .satisfies(c -> assertThat(c.getJetonHorodatage()).isNotBlank());
        assertThat(outbox.countByEventType("homologation.operation-
closed.v1")).isEqualTo(1);
        msIam.verify(1,
postRequestedFor(urlEqualTo("/api/v1/internal/reauthentications/verification")));
    }

    @Test
    @DisplayName("HOM-10 : validation refusée sans réauthentification FIDO2")
    void refusedWithoutReauthentication() throws Exception {
        UUID opId = fixtures.operationWithSealedEvidence();
        mvc.perform(post("/api/v1/operations/{id}/validation", opId)
                .with(TestJwt.as(fixtures.chefDesId(), "HOM_OPERATION_VALIDATE")))
            .andExpect(status().isForbidden())
            .andExpect(jsonPath("$.code").value("HOM_REAUTH_REQUIRED"));
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: HOM-10 Clôture selon la règle des quatre éléments

  Contexte:
    Étant donné une liaison de la "Banque B" de type "CARTE_INTERBANCAIRE" en
homologation
    Et une opération à l'état "PREUVE_DEPOSEE" dont le responsable et le déposant
sont "test-ingenieur-homologation"

  Scénario: Nominal : clôture avec les quatre éléments
    Étant donné une opération dotée d'un responsable et d'une preuve scellée
    Quand "test-chef-des" valide l'opération après réauthentification par clé FIDO2
    Alors un jeton RFC 3161 est obtenu sur l'empreinte de l'opération, de la preuve,
du validateur et de l'instant, puis vérifié
    Et l'opération passe à l'état "CLOSE"
    Et l'événement "homologation.operation-closed.v1" est émis
    Et les successeurs dont le degré entrant atteint zéro deviennent éligibles

  Scénario: Alternatif : validateur non distinct
    Quand "test-ingenieur-homologation" demande lui-même la clôture après
réauthentification
    Alors la réponse a le statut 409 avec le code "HOM_CLOSURE_ELEMENT_MISSING"
    Et le motif est "validation par un tiers distinct exigée"

  Scénario: Erreur : élément manquant
    Étant donné une opération dont la preuve n'est pas scellée
    Quand "test-chef-des" demande la clôture après réauthentification
    Alors la clôture est refusée et l'élément manquant "PREUVE_SCELLEE" est nommé

  Scénario: Erreur : autorité d'horodatage indisponible
```

```
    Étant donné que l'autorité d'horodatage ne répond pas
    Quand "test-chef-des" valide l'opération après réauthentification
    Alors la réponse a le statut 202 et l'opération reste à l'état
"VALIDEE_HORODATAGE_EN_ATTENTE"
    Et l'opération n'est pas close
    Quand l'autorité d'horodatage redevient disponible et que le réessai planifié
s'exécute
    Alors l'opération passe à l'état "CLOSE" sans nouvelle validation
```

E. Tests cURL via la passerelle

En recette, l'adaptateur de substitution de `ReauthenticationPort` accepte l'assertion `stub-ok:<objet>` ; en production, l'assertion FIDO2 est produite par `SignDialog` (WebAuthn).

```
source ./scripts/token.sh test-chef-des
BASE="${GATEWAY_URL}/bff/des/homologation"

# État des quatre éléments
curl -s "${BASE}/operations/${OP_ID}/elements-de-cloture" -H "Authorization: Bearer $
{TOKEN}" \
  | jq 'map({element, present})'

# Nominal : validation par un tiers distinct, réauthentification forte
curl -s -X POST "${BASE}/operations/${OP_ID}/validation" \
  -H "Authorization: Bearer ${TOKEN}" \
  -H "X-Reauth-Assertion: stub-ok:HOM_OPERATION_VALIDATE:${OP_ID}" \
  | jq '{etat, transitionsPermises}'
# Attendu : "CLOSE" (200), ou "VALIDEE_HORODATAGE_EN_ATTENTE" (202) si l'autorité ne
répond pas

# Refus : sans réauthentification (attendu : 403)
curl -s -o /dev/null -w "%{http_code}\n" -X POST
"${BASE}/operations/${OP_ID_2}/validation" \
  -H "Authorization: Bearer ${TOKEN}"

# Refus : le responsable valide lui-même (attendu : 409)
source ./scripts/token.sh test-ingenieur-homologation
curl -s -o /dev/null -w "%{http_code}\n" -X POST
"${BASE}/operations/${OP_ID_2}/validation" \
  -H "Authorization: Bearer ${TOKEN}" \
  -H "X-Reauth-Assertion: stub-ok:HOM_OPERATION_VALIDATE:${OP_ID_2}"
```

HOM-11 | Rouvrir une opération close sur décision motivée

Élément	Valeur
Points	3
Priorité	Must Have
Sprint cible	Sprint 3
Écrans	DES-09
Dépendances	HOM-10, HOM-07

Objectif

Rouvrir une opération close avec un motif obligatoire, en conservant la clôture antérieure dans l'historique, en reverrouillant les successeurs non clos, en signalant « à revoir » les successeurs déjà clos, puis en émettant `homologation.operation-reopened.v1` et `homologation.coverage-changed.v1`. Une demande sans motif est refusée (400).

A. Implémentation

A1. Recalcul des couvertures : interface et version livrée par ce récit

Le recalcul complet des couvertures est livré par HOM-17 (Sprint 4). Ce récit fixe l'interface appelée à la clôture, à la réouverture et au rapprochement, et une première implémentation qui marque les couvertures du service comme non calculables (`RECALCUL_EN_ATTENTE`) et publie l'événement dans la transaction appelante ; HOM-17 en remplace le corps sans changer l'interface.

```
// application/port/in/CoverageRecalculationUseCase.java
public interface CoverageRecalculationUseCase {
    /** Recalcule les couvertures d'un service de la liaison et publie
homologation.coverage-changed.v1. */
    void recalculate(UUID liaisonId, String serviceCode, String cause);
}

// application/usecase/CoverageRecalculationService.java
@Service
@RequiredArgsConstructor
public class CoverageRecalculationService implements CoverageRecalculationUseCase {
    private final CoverageRepository coverages;
    private final OutboxPublisher outboxPublisher;
    private final Clock clock;

    @Override
    @Transactional(propagation = Propagation.MANDATORY)
    public void recalculate(UUID liaisonId, String serviceCode, String cause) {
        Instant now = clock.instant();
        coverages.findByLiaisonIdAndServiceCode(liaisonId, serviceCode)
            .forEach(c ->
c.calculable(false).motifNonCalculable("RECALCUL_EN_ATTENTE").dateCalcul(now));
        outboxPublisher.publish("Coverage", liaisonId.toString(),
"homologation.coverage-changed.v1",
            new CoverageChangedV1(liaisonId, serviceCode, cause, now));
    }
}
```

A2. Cas d'utilisation de réouverture

La clôture active est désactivée (`actif = false`, motif, date), jamais supprimée. Chaque successeur dans le périmètre reçoit l'incrément de degré entrant symétrique du décrément de HOM-10 : un successeur non clos repasse à `VERROUILLEE` (sa validation éventuellement en attente est annulée), un successeur clos garde sa clôture et figure dans la liste « à revoir » de l'événement et de la réponse.

```
// application/usecase/ReopenOperationService.java
@Service
@RequiredArgsConstructor
public class ReopenOperationService implements ReopenOperationUseCase {
    private final OperationInstanceRepository operations;
    private final ClosureRecordRepository closures;
    private final CoverageRecalculationUseCase coverages;
    private final OutboxPublisher outboxPublisher;
    private final Clock clock;

    @Override
    @Transactional
    public ReopenOutcome reopen(UUID operationId, String motif, CurrentUser
demandeur) {
        if (motif == null || motif.isBlank()) throw new
BadRequestProblem("HOM_REOPEN_REASON_REQUIRED");
        OperationInstance op =
operations.findByIdForUpdate(operationId).orElseThrow(NotFoundException::operation);
        if (op.getEtat() != EtatOperation.CLOSE)
            throw new DomainRuleException("HOM_OPERATION_TRANSITION_FORBIDDEN",
"Seule une opération close se rouvre", Map.of("etat", op.getEtat()));
        Instant now = clock.instant();
        closures.findByOperationIdAndActifTrue(operationId).orElseThrow()
            .actif(false).motifReouverture(motif).dateReouverture(now); // la clôture
demeure dans l'historique
        op.etat(EtatOperation.ROUVERTE).dateEtat(now);

        List<String> reverrouilles = new ArrayList<>(), aRevoir = new ArrayList<>();
        for (OperationInstance s :
operations.findSuccessorsForUpdate(op.getLiaison().getId(), op.getCode())) {
            s.degreEntrant(s.getDegreEntrant() + 1);
            if (s.getEtat() == EtatOperation.CLOSE) {
                aRevoir.add(s.getCode());
            } else if (s.getEtat() != EtatOperation.VERROUILLEE) {
                closures.cancelPendingValidations(s.getId(), "PREDECESSEUR_ROUVERT",
now);
                s.etat(EtatOperation.VERROUILLEE).dateEtat(now);
                reverrouilles.add(s.getCode());
            }
        }
        outboxPublisher.publish("Operation", op.getId().toString(),
"homologation.operation-reopened.v1",
            new OperationReopenedV1(op.getId(), op.getLiaison().getId(),
op.getLiaison().getInstitutionId(),
                op.getCode(), op.getServiceCode(), motif, demandeur.id(), now,
reverrouilles, aRevoir));
        coverages.recalculate(op.getLiaison().getId(), op.getServiceCode(),
"OPERATION_ROUVERTE");
        return new ReopenOutcome(op, reverrouilles, aRevoir);
    }
}
```

A3. Ressource REST

```
// web/rest/OperationReopenResource.java
@RestController
@RequiredArgsConstructor
class OperationReopenResource {
```

```
    private final ReopenOperationUseCase reopenOperation;
    private final OperationMapper mapper;
    private final CurrentUserProvider currentUser;

    record ReopenRequest(@NotBlank(message = "HOM_REOPEN_REASON_REQUIRED") @Size(max
= 1000) String motif) {}

    @PostMapping("/api/v1/operations/{id}/reouverture")
    @PreAuthorize("hasAuthority('HOM_OPERATION_REOPEN')")
    ReopenDTO reopen(@PathVariable UUID id, @Valid @RequestBody ReopenRequest
request) {
        return mapper.toDto(reopenOperation.reopen(id, request.motif(),
currentUser.get()));
    }
}
```

MS-CERTIFICATION consomme `homologation.operation-reopened.v1` sur le sujet `homologation.operation` (récits CER) : il signale au chef du DES que le socle d'évaluation d'un circuit ouvert a changé. Le présent récit garantit le contrat de l'événement (attributs CloudEvents, `liaisonId`, `institutionId`, `motif`, listes `reverrouilles` et `aRevoir`).

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
//
src/test/java/org/gimac/des/homologation/application/usecase/ReopenOperationServiceTe
st.java
@ExtendWith(MockitoExtension.class)
class ReopenOperationServiceTest {
    @Mock OperationInstanceRepository operations;
    @Mock ClosureRecordRepository closures;
    @Mock CoverageRecalculationUseCase coverages;
    @Mock OutboxPublisher outbox;
    ReopenOperationService service;

    @BeforeEach
    void setUp() { service = new ReopenOperationService(operations, closures,
coverages, outbox, Clock.systemUTC()); }

    @Test
    @DisplayName("HOM-11 : la clôture antérieure est conservée dans l'historique et
les successeurs traités")
    void priorClosureKeptAndSuccessorsHandled() {
        OperationInstance op = OperationFixtures.operation(EtatOperation.CLOSE,
"RAPPORT_PARAMETRAGE");
        ClosureRecord cloture = OperationFixtures.closedRecord(op);
        OperationInstance eligible = OperationFixtures.successor(op,
EtatOperation.ELIGIBLE, 0);
        OperationInstance close = OperationFixtures.successor(op,
EtatOperation.CLOSE, 0);
        when(operations.findByIdForUpdate(op.getId())).thenReturn(Optional.of(op));

when(closures.findByOperationIdAndActifTrue(op.getId())).thenReturn(Optional.of(clotu
re));
        when(operations.findSuccessorsForUpdate(any(),
eq(op.getCode()))).thenReturn(List.of(eligible, close));

        ReopenOutcome out = service.reopen(op.getId(), "Preuve de paramétrage
incomplète", OperationFixtures.ingenieur());

        assertThat(cloture.getActif()).isFalse();
        assertThat(cloture.getMotifReouverture()).isEqualTo("Preuve de paramétrage
incomplète");
        assertThat(cloture.getJetonHorodatage()).isNotBlank(); // rien n'est effacé
        verify(closures, never()).delete(any());
        assertThat(eligible.getEtat()).isEqualTo(EtatOperation.VERROUILLEE);
        assertThat(eligible.getDegreEntrant()).isEqualTo(1);
```

```
        assertThat(close.getEtat()).isEqualTo(EtatOperation.CLOSE);
        assertThat(out.aRevoir()).containsExactly(close.getCode());
    }

    @Test
    @DisplayName("HOM-11 : une réouverture sans motif est refusée")
    void refusedWithoutReason() {
        assertThatThrownBy(() -> service.reopen(UUID.randomUUID(), "  ",
OperationFixtures.ingenieur()))
            .hasFieldOrPropertyWithValue("code", "HOM_REOPEN_REASON_REQUIRED");
        verifyNoInteractions(operations, outbox, coverages);
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

Le test lit le sujet `homologation.operation` comme le fera MS-CERTIFICATION et vérifie le contrat de l'événement publié par le relais de l'outbox.

```
// src/test/java/org/gimac/des/homologation/web/rest/OperationReopenIT.java
@SpringBootTest
@AutoConfigureMockMvc
@Testcontainers
class OperationReopenIT {
    @Container @ServiceConnection static PostgreSQLContainer<?> postgres = new
PostgreSQLContainer<>("postgres:16-alpine");
    @Container @ServiceConnection static KafkaContainer kafka = new
KafkaContainer("apache/kafka:3.8.0");

    @Autowired MockMvc mvc;
    @Autowired HomologationFixtures fixtures;
    @Autowired ObjectMapper json;

    @Test
    @DisplayName("HOM-11 : l'événement de réouverture est publié au format attendu
par MS-CERTIFICATION")
    void reopenedEventContract() throws Exception {
        UUID opId = fixtures.closedOperation();
        mvc.perform(post("/api/v1/operations/{id}/reouverture",
opId).contentType(MediaType.APPLICATION_JSON)
                .content("{\"motif\":\"Preuve insuffisante\"}")
                .with(TestJwt.as(fixtures.ingenieurId(), "HOM_OPERATION_REOPEN")))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.operation.etat").value("ROUVERTE"));

        try (var consumer = new KafkaConsumer<>(Map.<String, Object>of(
                BOOTSTRAP_SERVERS_CONFIG, kafka.getBootstrapServers(),
GROUP_ID_CONFIG, "it-hom-11",
                AUTO_OFFSET_RESET_CONFIG, "earliest"), new StringDeserializer(), new
StringDeserializer())) {
            consumer.subscribe(List.of("homologation.operation"));
            List<JsonNode> recus = new ArrayList<>();
            await().atMost(Duration.ofSeconds(15)).untilAsserted(() -> {
                for (var r : consumer.poll(Duration.ofMillis(200)))
recus.add(json.readTree(r.value()));
                assertThat(recus).anySatisfy(e -> {

assertThat(e.path("type").asText()).isEqualTo("homologation.operation-reopened.v1");

assertThat(e.path("subject").asText()).isEqualTo(opId.toString());

assertThat(e.path("data").path("motif").asText()).isEqualTo("Preuve insuffisante");

assertThat(e.path("data").path("institutionId").asText()).isNotBlank();
                });
            });
        }
```

```
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: HOM-11 Réouverture motivée d'une opération close

  Contexte:
    Étant donné une liaison de la "Banque B" de type "CARTE_INTERBANCAIRE" en
homologation
    Et un utilisateur connecté ayant le rôle "INGENIEUR_HOMOLOGATION"

  Scénario: Nominal : réouverture motivée
    Étant donné une opération close ayant un successeur éligible et un successeur
clos
    Quand l'ingénieur d'homologation la rouvre avec le motif "Preuve insuffisante"
    Alors l'opération passe à l'état "ROUVERTE" et sa clôture antérieure demeure dans
l'historique
    Et le successeur éligible est reverrouillé et le successeur clos est signalé « à
revoir »
    Et les couvertures du service sont recalculées
    Et les événements "homologation.operation-reopened.v1" et "homologation.coverage-
changed.v1" sont émis

  Scénario: Alternatif : réouverture pendant le circuit de quitus
    Étant donné une liaison dont le circuit de quitus est ouvert dans MS-
CERTIFICATION
    Quand une de ses opérations est rouverte
    Alors l'événement de réouverture est consommé par MS-CERTIFICATION
    Et le chef du DES est notifié que le socle d'évaluation a changé

  Scénario: Erreur : motif absent
    Étant donné une opération close
    Quand la réouverture est demandée sans motif
    Alors la réponse a le statut 400 avec le code "HOM_REOPEN_REASON_REQUIRED"
    Et l'opération reste à l'état "CLOSE"
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-ingenieur-homologation
BASE="${GATEWAY_URL}/bff/des/homologation"

# Nominal : réouverture motivée
curl -s -X POST "${BASE}/operations/${OP_ID}/reouverture" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" \
  -d '{"motif":"Preuve de paramétrage incomplète"}' \
  | jq '{etat: .operation.etat, reverrouilles, aRevoir}'

# Refus : motif absent (attendu : 400)
curl -s -o /dev/null -w "%{http_code}\n" -X POST
"${BASE}/operations/${OP_ID_CLOS}/reouverture" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" -d
'{"motif":""}'
```

HOM-12 | Suivre la vérification du code QR marchand statique et dynamique

Élément	Valeur
Points	3
Priorité	Should Have
Sprint cible	Sprint 4
Écrans	DES-09, DES-10
Dépendances	HOM-01, HOM-10

Objectif

Déclarer dans la branche paiement mobile deux opérations de vérification du code QR marchand (statique et dynamique), chacune dotée d'une grille de contrôle, et étendre la garde de clôture pour exiger une grille entièrement conforme. Un point non conforme bloque la clôture et est restitué ; la clôture est comptée dans la couverture du service de paiement marchand.

A. Implémentation

A1. Modèle de grille et déclaration des deux opérations

La grille est un objet-valeur sérialisé dans `ReferenceOperation.grille` ; les réponses sont sérialisées dans `OperationInstance.grilleReponses`. Les deux opérations sont déclarées dans une version brouillon du référentiel par `PUT /referentiels/{id}/operations/{code}` (HOM-01), avec le service `PAIEMENT_MARCHAND` : si l'institution ne l'a pas souscrit, l'instanciation (HOM-07) les place hors périmètre.

```
// domain/model/ControlGrid.java
public record ControlGrid(List<ControlPoint> points) {
    public record ControlPoint(String code, String libelle) {}
}

// domain/model/GridAnswer.java
public record GridAnswer(Conformite reponse, String commentaire) {
    public enum Conformite { CONFORME, NON_CONFORME, NON_RENSEIGNE }
}
```

```
{
  "PM-QR-STATIQUE": {
    "segment": "BRANCHE", "partie": "DES", "serviceCode": "PAIEMENT_MARCHAND",
"natureCouverture": "CERTIFICATION",
    "preuveAttendue": { "PAIEMENT_MOBILE": ["CAPTURE_VERIFICATION_QR"] },
    "grille": { "points": [
      { "code": "ID_MARCHAND", "libelle": "Identifiant du marchand" },
      { "code": "CATEGORIE", "libelle": "Catégorie du marchand" },
      { "code": "DEVISE", "libelle": "Devise" },
      { "code": "PAYS", "libelle": "Pays" },
      { "code": "LISIBILITE", "libelle": "Lisibilité du code" } ] }
  },
  "PM-QR-DYNAMIQUE": {
    "segment": "BRANCHE", "partie": "DES", "serviceCode": "PAIEMENT_MARCHAND",
"natureCouverture": "CERTIFICATION",
    "preuveAttendue": { "PAIEMENT_MOBILE": ["CAPTURE_VERIFICATION_QR"] },
    "grille": { "points": [
      { "code": "ID_MARCHAND", "libelle": "Identifiant du marchand" },
      { "code": "CATEGORIE", "libelle": "Catégorie du marchand" },
```

```
      { "code": "DEVISE", "libelle": "Devise" },
      { "code": "PAYS", "libelle": "Pays" },
      { "code": "LISIBILITE", "libelle": "Lisibilité du code" },
      { "code": "MONTANT_REFERENCE_UNIQUE", "libelle": "Montant et référence de
transaction à usage unique" } ] }
  }
}
```

A2. Évaluation de la grille et extension de la garde de clôture

Un point absent des réponses vaut `NON_RENSEIGNE` et bloque la clôture comme un point non conforme. La garde est appelée par `ValidateOperationService` (HOM-10) juste après `checkValidation`, avant toute demande de jeton ; la grille ne peut plus changer ensuite, car sa modification n'est admise qu'aux états `ELIGIBLE`, `PREUVE_DEPOSEE` et `ROUVERTE`.

```
// domain/service/GridEvaluator.java
public final class GridEvaluator {

    public List<String> nonConformes(ControlGrid grille, Map<String, GridAnswer>
reponses) {
        if (grille == null) return List.of(); // opération sans grille
        return grille.points().stream()
            .filter(p ->
Optional.ofNullable(reponses.get(p.code())).map(GridAnswer::reponse)
                .orElse(Conformite.NON_RENSEIGNE) != Conformite.CONFORME)
            .map(ControlPoint::code).toList();
    }

    public void checkKnownCodes(ControlGrid grille, Map<String, GridAnswer> reponses)
{
        Set<String> connus =
grille.points().stream().map(ControlPoint::code).collect(Collectors.toSet());
        List<String> inconnus = reponses.keySet().stream().filter(c -> !
connus.contains(c)).sorted().toList();
        if (!inconnus.isEmpty()) throw new
BadRequestProblem("HOM_GRID_UNKNOWN_POINT", Map.of("points", inconnus));
    }
}

// domain/service/ClosureGuard.java (ajout HOM-12)
public void checkGrid(List<String> pointsNonConformes) {
    if (!pointsNonConformes.isEmpty()) {
        throw new DomainRuleException("HOM_CLOSURE_GRID_NOT_CONFORMING", "Grille de
contrôle non entièrement conforme",
            Map.of("element", "GRILLE_CONFORME", "pointsNonConformes",
pointsNonConformes));
    }
}

// application/usecase/ValidateOperationService.java (ajout HOM-12, dans la
transaction de validation)
guard.checkValidation(f);
guard.checkGrid(gridEvaluator.nonConformes(gridCodec.grid(op.getReferenceOperation())
, gridCodec.answers(op)));
```

A3. Cas d'utilisation et ressource REST de la grille

`GridCodec` (composant applicatif fondé sur `ObjectMapper`) convertit les colonnes JSON en objets-valeurs.

```
// application/usecase/GridService.java
@Service
@RequiredArgsConstructor
public class GridService implements GridUseCase {
    private static final Set<EtatOperation> MODIFIABLE = EnumSet.of(ELIGIBLE,
```

```
PREUVE_DEPOSEE, ROUVERTE);
    private final OperationInstanceRepository operations;
    private final GridCodec codec;
    private final GridEvaluator evaluator = new GridEvaluator();

    @Override
    @Transactional
    public GridView answer(UUID operationId, Map<String, GridAnswer> reponses) {
        OperationInstance op =
operations.findByIdForUpdate(operationId).orElseThrow(NotFoundException::operation);
        if (!MODIFIABLE.contains(op.getEtat()))
            throw new DomainRuleException("HOM_GRID_STATE_FORBIDDEN", "Grille non
modifiable", Map.of("etat", op.getEtat()));
        ControlGrid grille =
Optional.ofNullable(codec.grid(op.getReferenceOperation()))
            .orElseThrow(() -> new DomainRuleException("HOM_GRID_NOT_DEFINED",
"Opération sans grille", Map.of()));
        evaluator.checkKnownCodes(grille, reponses);
        op.grilleReponses(codec.write(reponses));
        return new GridView(grille, reponses, evaluator.nonConformes(grille,
reponses));
    }

    @Override
    @Transactional(readOnly = true)
    public GridView read(UUID operationId) {
        OperationInstance op =
operations.findDetailedById(operationId).orElseThrow(NotFoundException::operation);
        ControlGrid grille = codec.grid(op.getReferenceOperation());
        return new GridView(grille, codec.answers(op), evaluator.nonConformes(grille,
codec.answers(op)));
    }
}

// web/rest/OperationGridResource.java
@RestController
@RequestMapping("/api/v1/operations/{id}/grille")
@RequiredArgsConstructor
class OperationGridResource {
    private final GridUseCase grids;

    @PutMapping
    @PreAuthorize("hasAuthority('HOM_GRID_EDIT')")
    GridView answer(@PathVariable UUID id, @RequestBody @NotEmpty Map<String, @Valid
GridAnswer> reponses) {
        return grids.answer(id, reponses);
    }

    @GetMapping
    @PreAuthorize("hasAuthority('HOM_LIAISON_READ')")
    GridView read(@PathVariable UUID id) { return grids.read(id); }
}
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
// src/test/java/org/gimac/des/homologation/domain/service/GridClosureTest.java
class GridClosureTest {
    final GridEvaluator evaluator = new GridEvaluator();
    final ClosureGuard guard = new ClosureGuard();
    final ControlGrid dynamique = QrGrids.dynamique(); // grille de A1

    @Test
    @DisplayName("HOM-12 : la clôture est refusée avec un point non conforme,
restitué")
    void refusedWithNonConformingPoint() {
        Map<String, GridAnswer> reponses = QrGrids.toutConforme(dynamique);
        reponses.put("MONTANT_REFERENCE_UNIQUE", new
```

```
GridAnswer(Conformite.NON_CONFORME, "référence réutilisable"));

        assertThatThrownBy(() -> guard.checkGrid(evaluator.nonConformes(dynamique,
reponses)))
            .hasFieldOrPropertyWithValue("code", "HOM_CLOSURE_GRID_NOT_CONFORMING")
            .extracting("details").asInstanceOf(InstanceOfAssertFactories.MAP)
            .containsEntry("pointsNonConformes",
List.of("MONTANT_REFERENCE_UNIQUE"));
    }

    @Test
    @DisplayName("HOM-12 : un point non renseigné bloque la clôture ; une grille
entièrement conforme la permet")
    void unansweredBlocksAndFullConformityPasses() {
        Map<String, GridAnswer> reponses = QrGrids.toutConforme(dynamique);
        assertThatCode(() -> guard.checkGrid(evaluator.nonConformes(dynamique,
reponses))).doesNotThrowAnyException();
        reponses.remove("LISIBILITE");
        assertThat(evaluator.nonConformes(dynamique,
reponses)).containsExactly("LISIBILITE");
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

Les adaptateurs de substitution (profil `test`) servent pour la réauthentification et l'horodatage.

```
// src/test/java/org/gimac/des/homologation/web/rest/OperationGridIT.java
@SpringBootTest
@AutoConfigureMockMvc
@Testcontainers
class OperationGridIT {
    @Container @ServiceConnection static PostgreSQLContainer<?> postgres = new
PostgreSQLContainer<>("postgres:16-alpine");
    @Container @ServiceConnection static KafkaContainer kafka = new
KafkaContainer("apache/kafka:3.8.0");

    @Autowired MockMvc mvc;
    @Autowired HomologationFixtures fixtures;

    @Test
    @DisplayName("HOM-12 : grille dynamique non conforme, validation refusée avec le
point restitué")
    void dynamicGridBlocksClosure() throws Exception {
        UUID opId = fixtures.qrOperationWithSealedEvidence("PM-QR-DYNAMIQUE");
        mvc.perform(put("/api/v1/operations/{id}/grille",
opId).contentType(MediaType.APPLICATION_JSON)
                .content(QrGrids.json("PM-QR-DYNAMIQUE",
Map.of("MONTANT_REFERENCE_UNIQUE", "NON_CONFORME")))
                .with(TestJwt.as(fixtures.ingenieurId(), "HOM_GRID_EDIT")))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.nonConformes[0]").value("MONTANT_REFERENCE_UNIQUE"
));

        mvc.perform(post("/api/v1/operations/{id}/validation", opId)
                .header("X-Reauth-Assertion", "stub-ok:HOM_OPERATION_VALIDATE:" +
opId)
                .with(TestJwt.as(fixtures.chefDesId(), "HOM_OPERATION_VALIDATE")))
            .andExpect(status().isConflict())
            .andExpect(jsonPath("$.code").value("HOM_CLOSURE_GRID_NOT_CONFORMING"))
            .andExpect(jsonPath("$.details.pointsNonConformes[0]").value("MONTANT_REF
ERENCE_UNIQUE"));
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: HOM-12 Vérification du code QR marchand statique et dynamique

  Contexte:
    Étant donné une liaison de l'"Établissement de paiement A" de type
"PAIEMENT_MOBILE" en homologation

  Scénario: Nominal : vérification du code QR statique
    Étant donné l'opération "PM-QR-STATIQUE" éligible avec sa grille (identifiant du
marchand, catégorie, devise, pays, lisibilité)
    Quand l'ingénieur renseigne tous les points de la grille comme conformes et
dépose la capture de preuve
    Et un validateur distinct valide l'opération après réauthentification
    Alors l'opération passe à l'état "CLOSE" selon la règle des quatre éléments
    Et sa clôture est comptée dans la couverture du service "PAIEMENT_MARCHAND"

  Scénario: Alternatif : service hors périmètre
    Étant donné une liaison dont l'institution n'a pas souscrit le service
"PAIEMENT_MARCHAND"
    Quand les opérations sont instanciées
    Alors les opérations "PM-QR-STATIQUE" et "PM-QR-DYNAMIQUE" sont à l'état
"HORS_PERIMETRE"

  Scénario: Erreur : point non conforme sur le code QR dynamique
    Étant donné l'opération "PM-QR-DYNAMIQUE" dont le point
"MONTANT_REFERENCE_UNIQUE" est non conforme
    Quand la clôture est demandée
    Alors la réponse a le statut 409 avec le code "HOM_CLOSURE_GRID_NOT_CONFORMING"
    Et le point non conforme "MONTANT_REFERENCE_UNIQUE" est restitué
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-ingenieur-homologation
BASE="${GATEWAY_URL}/bff/des/homologation"

# Nominal : grille du code QR statique entièrement conforme
curl -s -X PUT "${BASE}/operations/${OP_QR_STATIQUE}/grille" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" \
  -d '{"ID_MARCHAND":{"reponse":"CONFORME"},"CATEGORIE":
{"reponse":"CONFORME"},"DEVISE":{"reponse":"CONFORME"},
       "PAYS":{"reponse":"CONFORME"},"LISIBILITE":{"reponse":"CONFORME"}}' \
  | jq '{nonConformes}'
# Attendu : {"nonConformes": []}

# Refus : point inconnu de la grille (attendu : 400)
curl -s -o /dev/null -w "%{http_code}\n" -X PUT "${BASE}/operations/$
{OP_QR_STATIQUE}/grille" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" \
  -d '{"POINT_INEXISTANT":{"reponse":"CONFORME"}}'
```

HOM-13 | Reprendre en lecture seule les résultats des campagnes du simulateur

Élément	Valeur
Points	8
Priorité	Must Have
Sprint cible	Sprint 4
Écrans	DES-09
Dépendances	HOM-07, SOC-05, SOC-07, HOM-16

Objectif

Reprendre périodiquement, et sur notification, les résultats des campagnes du simulateur rattachées à chaque liaison, à travers une couche anticorruption en lecture seule, et les enregistrer tels que fournis (verdict, date d'exécution, version du scénario, référence de trace) avec l'empreinte de la réponse lue. La clé (liaison, campagneId, executionId) garantit l'absence de doublon ; aucun verdict repris n'est modifiable ni recalculé.

A. Implémentation

A1. Port `SimulatorReaderPort` (couche anticorruption)

Le port ne comporte que des lectures ; le test d'architecture de HOM-16 interdit d'y ajouter une méthode d'écriture. Choix retenu : le simulateur identifie le participant par l'identifiant GIMAC de l'institution de la liaison ; les résultats sont lus par page à partir d'une position (curseur `ReplayCursor`).

```
// application/port/out/SimulatorReaderPort.java
public interface SimulatorReaderPort {

    List<String> listCampaignIds(UUID institutionId);

    ResultPage readResults(String campagneId, long depuisPosition);

    record SimulatorResult(String campagneId, String executionId, String idScenario,
Verdict verdict,
                           Instant dateExecution, String versionScenario, String
referenceTrace) {}

    record ResultPage(List<SimulatorResult> resultats, long positionSuivante, String
empreinteReponse) {}
}
```

L'adaptateur réel n'émet que des `GET` avec le client `simulatorRestClient` configuré en lecture seule par HOM-16 ; il calcule l'empreinte SHA-256 des octets reçus avant toute interprétation. Le traducteur est le seul à connaître le format du contrat de lecture du simulateur (SOC-07) : il traduit les libellés de verdict sans jamais les recalculer et valide la page entière, sinon il lève `MalformedSimulatorResponseException` (traitée par HOM-14).

```
// infrastructure/client/simulator/SimulatorHttpReaderAdapter.java
@Component
@ConditionalOnProperty(name = "gimac.adapters.simulator-reader", havingValue =
"real")
class SimulatorHttpReaderAdapter implements SimulatorReaderPort {
    private final RestClient simulator;
```

```
    private final SimulatorResponseTranslator translator;

    SimulatorHttpReaderAdapter(@Qualifier("simulatorRestClient") RestClient
simulator, SimulatorResponseTranslator t) {
        this.simulator = simulator;
        this.translator = t;
    }

    @Override
    public List<String> listCampaignIds(UUID institutionId) {
        return translator.campaignIds(get("/lecture/v1/participants/{id}/campagnes",
institutionId));
    }

    @Override
    public ResultPage readResults(String campagneId, long depuis) {
        byte[] brut = get("/lecture/v1/campagnes/{id}/resultats?depuis={p}",
campagneId, depuis);
        return translator.results(campagneId, brut, Digests.sha256Hex(brut)); //
empreinte de la réponse telle que lue
    }

    private byte[] get(String uri, Object... variables) {
        try {
            return simulator.get().uri(uri,
variables).accept(MediaType.APPLICATION_JSON).retrieve().body(byte[].class);
        } catch (ResourceAccessException | HttpServerErrorException e) {
            throw new SimulatorUnavailableException(e);
        }
    }
}

// infrastructure/client/simulator/SimulatorResponseTranslator.java
@Component
@RequiredArgsConstructor
class SimulatorResponseTranslator {
    // traduction de vocabulaire, jamais de recalcul : le verdict du simulateur fait
foi
    private static final Map<String, Verdict> VERDICTS = Map.of(
        "PASSED", Verdict.FAVORABLE, "FAILED", Verdict.DEFAVORABLE, "NOT_EXECUTED",
Verdict.NON_EXECUTE);
    private final ObjectMapper mapper;

    ResultPage results(String campagneId, byte[] brut, String empreinte) {
        try {
            JsonNode racine = mapper.readTree(brut);
            List<SimulatorResult> resultats = new ArrayList<>();
            for (JsonNode n : racine.path("resultats")) {
                Verdict v = Optional.ofNullable(VERDICTS.get(required(n, "verdict")))
                    .orElseThrow(() -> new IllegalArgumentException("verdict
inconnu"));
                resultats.add(new SimulatorResult(campagneId, required(n,
"executionId"), required(n, "scenarioId"), v,
                    Instant.parse(required(n, "executedAt")), required(n,
"scenarioVersion"), required(n, "traceRef")));
            }
            return new ResultPage(List.copyOf(resultats),
Long.parseLong(required(racine, "nextPosition")), empreinte);
        } catch (IOException | RuntimeException e) { // page entière rejetée, aucun
résultat partiel
            throw new MalformedSimulatorResponseException(campagneId, brut,
empreinte, e.getMessage());
        }
    }

    private static String required(JsonNode n, String champ) {
        if (!n.hasNonNull(champ)) throw new IllegalArgumentException("champ absent :
" + champ);
        return n.get(champ).asText();
```

```
    }
    // campaignIds(byte[]) : même principe sur le tableau "campagnes"
}
```

`SimulatorReaderStubAdapter` (profils `dev` et `test`) sert des campagnes tenues en mémoire, alimentées par la recette et les tests (`alimenter(institutionId, SimulatorResult)`), et expose `setDisponible(boolean)` pour HOM-14.

A2. Reprise idempotente et publication

L'insertion utilise `ON CONFLICT DO NOTHING` sur la contrainte d'unicité (liaison, campagneId, executionId) : un cycle périodique et une lecture ciblée concurrents ne peuvent pas créer de doublon, et seul un résultat réellement inséré émet un événement.

```
// infrastructure/persistence/CampaignResultRepository.java (extrait)
@Modifying
@Query(value = """
    insert into campaign_result (id, liaison_id, campagne_id, execution_id,
id_scenario, verdict, date_execution,
                                 version_scenario, reference_trace,
empreinte_reponse, date_reprise)
    values
(:#{#r.id()}, :#{#r.liaisonId()}, :#{#r.campagneId()}, :#{#r.executionId()}, :#{#r.id
Scenario()},
            :#{#r.verdict()}, :#{#r.dateExecution()}, :#{#r.versionScenario()}, :#{#r
.referenceTrace()},
            :#{#r.empreinteReponse()}, :#{#r.dateReprise()})
    on conflict (liaison_id, campagne_id, execution_id) do nothing""", nativeQuery =
true)
int insertIfAbsent(@Param("r") CampaignResultRow r);
```

```
// application/usecase/CampaignImportService.java
@Service
@RequiredArgsConstructor
public class CampaignImportService implements ReplayUseCase {
    private static final Set<EtatLiaison> SUIVIES =
EnumSet.of(EtatLiaison.EN_HOMOLOGATION, EtatLiaison.GELEE,
        EtatLiaison.APTE_CERTIFICATION, EtatLiaison.MISE_EN_SERVICE_AUTORISEE);
    private final SimulatorReaderPort simulator;
    private final LiaisonRepository liaisons;
    private final CampaignResultRepository results;
    private final ReplayCursorRepository cursors;
    private final OutboxPublisher outboxPublisher;
    private final TransactionTemplate tx;
    private final Clock clock;

    public List<Liaison> liaisonsSuivies() { return liaisons.findByEtatIn(SUIVIES); }

    public List<String> campaignsOf(Liaison l) { return
simulator.listCampaignIds(l.getInstitutionId()); }

    @Override
    public void runAll() { liaisonsSuivies().forEach(l -> campaignsOf(l).forEach(c ->
importCampaign(l, c))); }

    @Override
    public void runCampaign(String campagneId) { // lecture ciblée ; campagne encore
inconnue : cycle complet anticipé
        cursors.findByCampagneId(campagneId).ifPresentOrElse(c ->
importCampaign(liaison(c.getLiaisonId()), campagneId), this::runAll);
    }

    public Liaison liaison(UUID liaisonId) { return
liaisons.getReferenceById(liaisonId); }

    public int importCampaign(Liaison liaison, String campagneId) {
```

```
        long depuis = cursors.findOrCreate(campagneId,
liaison.getId()).getPosition();
        ResultPage page = simulator.readResults(campagneId, depuis); // hors
transaction
        return tx.execute(status -> {
            Instant now = clock.instant();
            int nouveaux = 0;
            for (SimulatorResult r : page.resultats()) {
                CampaignResultRow row = CampaignResultRow.of(UUID.randomUUID(),
liaison.getId(), r, page.empreinteReponse(), now);
                if (results.insertIfAbsent(row) == 1) {
                    nouveaux++;
                    outboxPublisher.publish("Campaign", row.id().toString(),
"homologation.campaign-result-imported.v1",
                        new CampaignResultImportedV1(row.id(), liaison.getId(),
r.campagneId(), r.executionId(),
                            r.idScenario(), r.verdict(), r.dateExecution(),
r.referenceTrace(), page.empreinteReponse()));
                }
            }
            cursors.advance(campagneId, liaison.getId(), page.positionSuivante(),
now);
            return nouveaux;
        });
    }
}
```

A3. Planification et réception des notifications

Le cycle périodique passe par `ReplayUseCase`, que HOM-14 décore par le disjoncteur. Le récepteur de notifications ne fait que recevoir : il ne retient que `campagneId` (tout autre champ est ignoré), ne rappelle jamais le simulateur en retour et place la campagne dans une file de lectures ciblées vidée toutes les cinq secondes, donc avant le cycle suivant. Choix retenu : l'appel porte le jeton de service du simulateur avec la permission `INTERNAL_SIMULATEUR_NOTIFY`, à ajouter au catalogue de MS-IAM selon le motif `INTERNAL_<SERVICE>_*`.

```
// infrastructure/scheduling/CampaignReplayScheduler.java
@Component
@RequiredArgsConstructor
public class CampaignReplayScheduler {
    private final ReplayUseCase replay;
    private final TargetedReplayQueue targeted; // ensemble concurrent ; drain()
renvoie et vide

    @Scheduled(initialDelayString = "${gimac.replay.initial-delay-ms:60000}",
fixedDelayString = "${gimac.replay.period-ms:300000}")
    public void periodicCycle() { replay.runAll(); }

    @Scheduled(fixedDelayString = "${gimac.replay.targeted-period-ms:5000}")
    public void targetedReads() { targeted.drain().forEach(replay::runCampaign); }
}

// web/rest/SimulatorNotificationResource.java
@RestController
@RequiredArgsConstructor
class SimulatorNotificationResource {
    private final TargetedReplayQueue targeted;

    @JsonIgnoreProperties(ignoreUnknown = true)
    record SimulatorNotification(@NotNull @Pattern(regexp = "[A-Za-z0-9._-]{1,64}")
String campagneId) {}

    @PostMapping("/api/v1/simulateur/notifications")
    @PreAuthorize("hasAuthority('INTERNAL_SIMULATEUR_NOTIFY')")
```

```
    ResponseEntity<Void> receive(@Valid @RequestBody SimulatorNotification
notification) {
        targeted.enqueue(notification.campagneId());
        return ResponseEntity.accepted().build();
    }
}
```

A4. Consultation en lecture seule et refus journalisé de toute modification

Aucune route d'écriture n'existe sur les résultats repris. Un filtre intercepte toute méthode autre que `GET`, `HEAD` ou `OPTIONS` sous `/liaisons/{id}/campagnes/**` (y compris vers un chemin inexistant) : il répond 409 `HOM_VERDICT_READ_ONLY` et publie `homologation.verdict-modification-refused.v1`, consommé par MS-AUDIT, dans une transaction indépendante.

```
// web/rest/CampaignResource.java
@GetMapping("/api/v1/liaisons/{id}/campagnes")
@PreAuthorize("hasAuthority('HOM_LIAISON_READ')")
List<CampaignResultDTO> list(@PathVariable UUID id) { // tous les résultats,
exécution la plus récente en tête
    return
results.findByLiaisonIdOrderByDateExecutionDesc(id).stream().map(mapper::toDto).toLis
t();
}

// web/rest/CampaignReadOnlyFilter.java
@Component
@RequiredArgsConstructor
class CampaignReadOnlyFilter extends OncePerRequestFilter {
    private static final PathPattern CAMPAGNES =
PathPatternParser.defaultInstance.parse("/api/v1/liaisons/{id}/campagnes/**");
    private static final Set<String> LECTURE = Set.of("GET", "HEAD", "OPTIONS");
    private final VerdictTamperingAudit audit; // @Transactional(REQUIRES_NEW) +
outboxPublisher.publish(...)

    @Override
    protected boolean shouldNotFilter(HttpServletRequest req) {
        return LECTURE.contains(req.getMethod()) || !
CAMPAGNES.matches(PathContainer.parsePath(req.getRequestURI()));
    }

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
FilterChain chain) throws IOException {
        String liaisonId =
CAMPAGNES.matchAndExtract(PathContainer.parsePath(req.getRequestURI())).getUriVariabl
es().get("id");
        audit.record(liaisonId, req.getMethod(), req.getRequestURI(),
req.getUserPrincipal()); // tentative journalisée
        res.setStatus(HttpStatus.CONFLICT.value());
        res.setContentType(MediaType.APPLICATION_PROBLEM_JSON_VALUE);
        res.getWriter().write("{\"code\":\"HOM_VERDICT_READ_ONLY\",\"message\":\"Un
verdict repris n'est ni modifiable ni recalculé\"}");
    }
}
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
//
src/test/java/org/gimac/des/homologation/application/usecase/CampaignImportServiceTes
t.java
@ExtendWith(MockitoExtension.class)
class CampaignImportServiceTest {
    @Mock SimulatorReaderPort simulator;
```

```
    @Mock LiaisonRepository liaisons;
    @Mock CampaignResultRepository results;
    @Mock ReplayCursorRepository cursors;
    @Mock OutboxPublisher outbox;
    @Mock TransactionTemplate tx;
    CampaignImportService service;
    final Liaison liaison =
OperationFixtures.liaison(TypeProjet.CARTE_INTERBANCAIRE);
    final SimulatorResult r = new SimulatorResult("CMP-1", "EXE-1", "SC-ATM-001",
Verdict.DEFAVORABLE,
        Instant.parse("2026-10-20T09:00:00Z"), "3", "TRC-42");

    @BeforeEach
    void setUp() {
        service = new CampaignImportService(simulator, liaisons, results, cursors,
outbox, tx, Clock.systemUTC());
        when(cursors.findOrCreate("CMP-1", liaison.getId())).thenReturn(new
ReplayCursor().position(0L));
        when(simulator.readResults("CMP-1", 0L)).thenReturn(new
ResultPage(List.of(r), 1L, "cd".repeat(32)));
        when(tx.execute(any())).thenAnswer(inv -> inv.<TransactionCallback<?
>>getArgument(0).doInTransaction(null));
    }

    @Test
    @DisplayName("HOM-13 : idempotence sur la clé campagne et exécution, aucun
événement pour un résultat déjà repris")
    void alreadyImportedIsIgnored() {
        when(results.insertIfAbsent(any())).thenReturn(0); // conflit sur (liaison,
campagneId, executionId)
        assertThat(service.importCampaign(liaison, "CMP-1")).isZero();
        verifyNoInteractions(outbox);
    }

    @Test
    @DisplayName("HOM-13 : le résultat est enregistré tel que fourni, avec
l'empreinte de la réponse")
    void storedAsProvided() {
        when(results.insertIfAbsent(any())).thenReturn(1);
        service.importCampaign(liaison, "CMP-1");
        ArgumentCaptor<CampaignResultRow> row =
ArgumentCaptor.forClass(CampaignResultRow.class);
        verify(results).insertIfAbsent(row.capture());
        assertThat(row.getValue()).extracting(CampaignResultRow::verdict,
CampaignResultRow::referenceTrace,
            CampaignResultRow::empreinteReponse).containsExactly("DEFAVORABLE", "TRC-
42", "cd".repeat(32));
        verify(outbox).publish(eq("Campaign"), anyString(),
eq("homologation.campaign-result-imported.v1"), any());
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

Le simulateur est remplacé par WireMock derrière l'adaptateur réel ; le test vérifie qu'aucune requête autre que `GET` ne l'a atteint.

```
// src/test/java/org/gimac/des/homologation/web/rest/CampaignReplayIT.java
@SpringBootTest(properties = {"gimac.adapters.simulator-reader=real",
"gimac.replay.period-ms=3600000"})
@AutoConfigureMockMvc
@Testcontainers
class CampaignReplayIT {
    @Container @ServiceConnection static PostgreSQLContainer<?> postgres = new
PostgreSQLContainer<>("postgres:16-alpine");
    @Container @ServiceConnection static KafkaContainer kafka = new
KafkaContainer("apache/kafka:3.8.0");
```

```
    @RegisterExtension static WireMockExtension simulateur =
WireMockExtension.newInstance()
        .options(wireMockConfig().dynamicPort()).build();

    @DynamicPropertySource
    static void props(DynamicPropertyRegistry r)
{ r.add("gimac.clients.simulator.base-url", simulateur::baseUrl); }

    @Autowired MockMvc mvc;
    @Autowired HomologationFixtures fixtures;
    @Autowired CampaignImportService replay;
    @Autowired CampaignResultRepository results;
    @Autowired OutboxEventRepository outbox;

    @Test
    @DisplayName("HOM-13 : reprise contre un simulateur en lecture seule, sans
doublon ni modification possible")
    void readOnlyReplay() throws Exception {
        Liaison liaison = fixtures.liaisonEnHomologation("Banque B");
        simulateur.stubFor(get(urlPathMatching("/lecture/v1/participants/.*/
campagnes"))
            .willReturn(okJson("{\"campagnes\":[\"CMP-1\"]}")));
        simulateur.stubFor(get(urlPathEqualTo("/lecture/v1/campagnes/CMP-1/
resultats"))
            .willReturn(okJson(SimulatorPayloads.twoResults("CMP-1")))); // contrat
de lecture (SOC-07)

        replay.runAll();
        replay.runAll(); // seconde lecture des mêmes exécutions


assertThat(results.findByLiaisonIdOrderByDateExecutionDesc(liaison.getId())).hasSize(
2)
            .allSatisfy(r -> assertThat(r.getEmpreinteReponse()).hasSize(64));
        assertThat(outbox.countByEventType("homologation.campaign-result-
imported.v1")).isEqualTo(2);

        mvc.perform(put("/api/v1/liaisons/{id}/campagnes/CMP-1",
liaison.getId()).contentType(MediaType.APPLICATION_JSON)
                .content("{\"verdict\":\"FAVORABLE\"}").with(TestJwt.as(fixtures.inge
nieurId(), "HOM_LIAISON_READ")))
            .andExpect(status().isConflict())
            .andExpect(jsonPath("$.code").value("HOM_VERDICT_READ_ONLY"));
        assertThat(outbox.countByEventType("homologation.verdict-modification-
refused.v1")).isEqualTo(1);

        simulateur.verify(0, postRequestedFor(anyUrl()));   // aucune écriture n'a
atteint le simulateur
        simulateur.verify(0, putRequestedFor(anyUrl()));
        simulateur.verify(0, patchRequestedFor(anyUrl()));
        simulateur.verify(0, deleteRequestedFor(anyUrl()));
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: HOM-13 Reprise en lecture seule des résultats des campagnes du
simulateur

  Contexte:
    Étant donné une liaison de la "Banque B" en homologation
    Et un simulateur de substitution en lecture seule

  Scénario: Nominal : reprise périodique
    Étant donné une campagne "CMP-1" du simulateur rattachée à la liaison avec deux
exécutions
```

```
    Quand le cycle périodique de reprise s'exécute
    Alors chaque résultat est enregistré tel que fourni avec son verdict, sa date
d'exécution, sa version de scénario et sa référence de trace
    Et chaque résultat porte l'empreinte de la réponse lue
    Et l'événement "homologation.campaign-result-imported.v1" est émis pour chaque
résultat nouveau

  Scénario: Alternatif : reprise sur notification
    Étant donné une notification du simulateur signalant la fin de la campagne "CMP-
2" avec des champs supplémentaires
    Quand la notification est reçue
    Alors seul l'identifiant de campagne "CMP-2" est retenu
    Et une lecture ciblée de "CMP-2" est effectuée avant le cycle périodique suivant

  Scénario: Alternatif : résultat déjà repris
    Étant donné un résultat déjà enregistré pour la campagne "CMP-1" et l'exécution
"EXE-1"
    Quand il est lu à nouveau
    Alors aucun doublon n'est créé et aucun événement n'est émis

  Scénario: Erreur : tentative de modification d'un verdict
    Étant donné un résultat repris affiché à l'ingénieur
    Quand une modification du verdict est tentée par appel direct
    Alors la réponse a le statut 409 avec le code "HOM_VERDICT_READ_ONLY"
    Et l'événement "homologation.verdict-modification-refused.v1" est émis vers MS-
AUDIT
    Et l'écran des campagnes ne propose aucune action de modification
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-ingenieur-homologation
BASE="${GATEWAY_URL}/bff/des/homologation"

# Nominal : résultats repris, référence de trace et empreinte
curl -s "${BASE}/liaisons/${LIAISON_ID}/campagnes" -H "Authorization: Bearer $
{TOKEN}" \
  | jq 'map({campagneId, executionId, idScenario, verdict, referenceTrace,
empreinteReponse})'

# Refus : modification d'un verdict (attendu : 409, tentative journalisée)
curl -s -o /dev/null -w "%{http_code}\n" -X PUT
"${BASE}/liaisons/${LIAISON_ID}/campagnes/CMP-1" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" -d
'{"verdict":"FAVORABLE"}'

# Refus : un utilisateur ne peut pas forger une notification du simulateur (attendu :
403)
curl -s -o /dev/null -w "%{http_code}\n" -X POST "${BASE}/simulateur/notifications" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" -d
'{"campagneId":"CMP-1"}'
```

HOM-14 | Tolérer l'indisponibilité du simulateur de certification

Élément	Valeur
Points	5
Priorité	Must Have
Sprint cible	Sprint 4
Écrans	DES-08, DES-09
Dépendances	HOM-13, SOC-08

Objectif

Protéger la reprise par un disjoncteur à trois états (fermé, ouvert, demi-ouvert) à seuils paramétrables : son ouverture suspend la seule reprise, affiche le bandeau « reprise suspendue depuis » et déclenche une alerte vers MS-MONITORING ; sa fermeture reprend chaque campagne à la dernière position enregistrée, sans doublon. Une réponse mal formée est placée au rebut avec son empreinte et son motif, sans aucun résultat partiel.

A. Implémentation

A1. Disjoncteur `ReplayCircuitBreaker` (service de domaine pur)

Le simulateur étant un tiers unique, le disjoncteur est global au service. Au-delà du seuil d'échecs consécutifs, il s'ouvre ; une fois le délai d'ouverture écoulé, chaque cycle tente au plus une lecture d'essai (demi-ouverture) : un succès le ferme, un échec le rouvre en conservant l'instant d'ouverture affiché par le bandeau.

```
// domain/service/ReplayCircuitBreaker.java
public final class ReplayCircuitBreaker {

    public record State(EtatDisjoncteur etat, int echecs, Instant depuis) {}
    public enum Decision { LIRE, LIRE_ESSAI, SUSPENDRE }

    private final int seuil;
    private final Duration delaiOuverture;

    public ReplayCircuitBreaker(int seuil, Duration delaiOuverture) {
        this.seuil = seuil;
        this.delaiOuverture = delaiOuverture;
    }

    public Decision decide(State s, Instant now) {
        return switch (s.etat()) {
            case FERME -> Decision.LIRE;
            case DEMI_OUVERT -> Decision.LIRE_ESSAI;
            case OUVERT -> now.isBefore(s.depuis().plus(delaiOuverture)) ?
Decision.SUSPENDRE : Decision.LIRE_ESSAI;
        };
    }

    public State halfOpen(State s) { return new State(EtatDisjoncteur.DEMI_OUVERT,
s.echecs(), s.depuis()); }

    public State onSuccess(State s, Instant now) { return new
State(EtatDisjoncteur.FERME, 0, now); }

    public State onFailure(State s, Instant now) {
        int n = s.echecs() + 1;
```

```
        if (s.etat() != EtatDisjoncteur.FERME) return new
State(EtatDisjoncteur.OUVERT, n, s.depuis()); // essai échoué
        return n >= seuil ? new State(EtatDisjoncteur.OUVERT, n, now) : new
State(EtatDisjoncteur.FERME, n, s.depuis());
    }
}
```

A2. Curseur persistant et ligne de pilotage

Complément du JDL : `ReplayCursor` reçoit `dateEtatDisjoncteur` (instant du dernier changement d'état, nécessaire au délai de demi-ouverture et au bandeau). L'état global est porté par une ligne de pilotage (`campagneId = '*'`, `liaisonId` nul) ; les autres lignes portent la position de chaque campagne.

```
<!-- config/liquibase/changelog/20261012000000_hom14_replay_cursor.xml -->
<databaseChangeLog xmlns="http://www.liquibase.org/xml/ns/dbchangelog">
  <changeSet id="hom14-replay-cursor" author="pi-gimac-01">
    <addColumn tableName="replay_cursor">
      <column name="date_etat_disjoncteur" type="timestamp with time zone"/>
    </addColumn>
    <sql>alter table replay_cursor add constraint ux_replay_cursor unique nulls not
distinct (campagne_id, liaison_id)</sql>
    <insert tableName="replay_cursor">
      <column name="id" valueComputed="gen_random_uuid()"/>
      <column name="campagne_id" value="*"/>
      <column name="position" valueNumeric="0"/>
      <column name="echecs_consecutifs" valueNumeric="0"/>
      <column name="etat_disjoncteur" value="FERME"/>
    </insert>
  </changeSet>
</databaseChangeLog>
```

A3. Reprise protégée, rebut et métriques

`ResilientReplayService` décore `ReplayUseCase` (HOM-13) et devient l'implémentation principale. Une réponse mal formée ne compte pas comme une indisponibilité (le simulateur a répondu) : la page est mise au rebut une seule fois par empreinte, le curseur de la campagne n'avance pas et les autres campagnes poursuivent. Les lectures sont idempotentes (HOM-13), ce qui rend inoffensif un cycle concurrent sur un autre réplica. Le disjoncteur est déclaré comme bean dans `config/ReplayConfiguration` à partir de `gimac.replay.breaker.threshold` (5 par défaut) et `gimac.replay.breaker.open-duration` (`PT5M` par défaut).

```
// application/usecase/ResilientReplayService.java
@Service
@Primary
@RequiredArgsConstructor
public class ResilientReplayService implements ReplayUseCase {
    private static final String PILOTE = "*";
    private final CampaignImportService imports;
    private final ReplayCursorRepository cursors;
    private final RejectedRecordRepository rejected;
    private final ReplayCircuitBreaker breaker; // gimac.replay.breaker.threshold,
gimac.replay.breaker.open-duration
    private final ReplayMetrics metrics;
    private final TransactionTemplate tx;
    private final Clock clock;

    @Override
    public void runAll() { guarded(this::readAll); }

    @Override
```

```
    public void runCampaign(String campagneId) { // campagne encore inconnue :
lecture complète anticipée
        guarded(() -> cursors.findByCampagneId(campagneId).ifPresentOrElse(
            c -> importOrQuarantine(imports.liaison(c.getLiaisonId()), campagneId),
this::readAll));
    }

    private void readAll() {
        imports.liaisonsSuivies().forEach(l -> imports.campaignsOf(l).forEach(c ->
importOrQuarantine(l, c)));
    }

    private void guarded(Runnable lecture) {
        State avant = pilot();
        Decision decision = breaker.decide(avant, clock.instant());
        if (decision == Decision.SUSPENDRE) { metrics.cycleSuspendu(); return; }
        if (decision == Decision.LIRE_ESSAI) save(breaker.halfOpen(avant));
        try {
            lecture.run();
            save(breaker.onSuccess(pilot(), clock.instant()));
        } catch (SimulatorUnavailableException e) {
            metrics.echecLecture();
            save(breaker.onFailure(pilot(), clock.instant()));
        }
    }

    private void importOrQuarantine(Liaison liaison, String campagneId) {
        try {
            imports.importCampaign(liaison, campagneId);
        } catch (MalformedSimulatorResponseException e) {
            String source = "simulateur:campagne:" + campagneId;
            tx.executeWithoutResult(s -> {
                if (!rejected.existsBySourceAndEmpreinteAndRelanceFalse(source,
e.empreinte())) {
                    rejected.save(new
RejectedRecord().source(source).chargeUtile(e.brutTexte()).empreinte(e.empreinte())
                        .motif(e.getMessage()).dateRejet(clock.instant()).relance(fal
se));
                    metrics.rebut();
                }
            });
        }
    }

    private State pilot() {
        ReplayCursor p =
cursors.findByCampagneIdAndLiaisonIdIsNull(PILOTE).orElseThrow();
        return new State(p.getEtatDisjoncteur(), p.getEchecsConsecutifs(),
p.getDateEtatDisjoncteur());
    }

    private void save(State s) {
        tx.executeWithoutResult(t -> cursors.updatePilot(PILOTE, s.etat(),
s.echecs(), s.depuis()));
        metrics.etatDisjoncteur(s.etat());
    }
}
```

```
// infrastructure/monitoring/ReplayMetrics.java
@Component
public class ReplayMetrics {
    private final AtomicInteger etat = new AtomicInteger();
    private final Counter echecs, rebut, suspendus;

    public ReplayMetrics(MeterRegistry registry) {
        Gauge.builder("gimac.replay.breaker.state", etat, AtomicInteger::get)
            .description("0 fermé, 1 demi-ouvert, 2 ouvert").register(registry);
        echecs = registry.counter("gimac.replay.read.failures");
        rebut = registry.counter("gimac.replay.rejected");
```

```
        suspendus = registry.counter("gimac.replay.cycles.suspended");
    }

    public void etatDisjoncteur(EtatDisjoncteur e) {
        etat.set(switch (e) { case FERME -> 0; case DEMI_OUVERT -> 1; case OUVERT ->
2; });
    }
    public void echecLecture() { echecs.increment(); }
    public void rebut() { rebut.increment(); }
    public void cycleSuspendu() { suspendus.increment(); }
}
```

```
# deploy/k8s/mshomologation/prometheus-rule-replay.yaml
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: mshomologation-replay
  labels: { app: mshomologation }
spec:
  groups:
    - name: reprise-simulateur
      rules:
        - alert: GimacRepriseSimulateurSuspendue
          expr: max(gimac_replay_breaker_state{application="mshomologation"}) == 2
          for: 1m
          labels: { severity: warning, route: ms-monitoring }   # routage
Alertmanager vers MS-MONITORING (SOC-08)
          annotations:
            summary: "Reprise du simulateur suspendue : disjoncteur ouvert"
```

A4. Ressources REST

`GET /reprise/etat` alimente le bandeau des écrans DES-08 et DES-09 (lisible avec `HOM_LIAISON_READ`) ; le rebut et sa relance exigent `HOM_REPLAY_ADMIN`. La relance marque l'enregistrement et place la campagne en file de lecture ciblée (HOM-13).

```
// web/rest/ReplayResource.java
@RestController
@RequestMapping("/api/v1/reprise")
@RequiredArgsConstructor
class ReplayResource {
    private final ReplayQueries queries;
    private final RelaunchRejectedUseCase relaunch;

    @GetMapping("/etat")
    @PreAuthorize("hasAuthority('HOM_LIAISON_READ')")
    ReplayStateDTO etat() { // {etatDisjoncteur, suspenduDepuis, echecsConsecutifs,
campagnes[{campagneId, position, dateDernierSucces}]}
        return queries.state();
    }

    @GetMapping("/rebut")
    @PreAuthorize("hasAuthority('HOM_REPLAY_ADMIN')")
    List<RejectedRecordDTO> rebut() { return queries.rejected(); }

    @PostMapping("/rebut/{id}/relance")
    @PreAuthorize("hasAuthority('HOM_REPLAY_ADMIN')")
    ResponseEntity<Void> relance(@PathVariable UUID id) {
        relaunch.relaunch(id); // 409 HOM_REJECTED_ALREADY_RELAUNCHED si déjà relancé
        return ResponseEntity.accepted().build();
    }
}
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
//
src/test/java/org/gimac/des/homologation/domain/service/ReplayCircuitBreakerTest.java
class ReplayCircuitBreakerTest {
    final ReplayCircuitBreaker breaker = new ReplayCircuitBreaker(3,
Duration.ofMinutes(5));
    final Instant t0 = Instant.parse("2026-10-20T08:00:00Z");

    @Test
    @DisplayName("HOM-14 : transitions fermé, ouvert, demi-ouvert puis fermé")
    void fullCycle() {
        State s = new State(EtatDisjoncteur.FERME, 0, t0);
        s = breaker.onFailure(breaker.onFailure(s, t0), t0);
        assertThat(s.etat()).isEqualTo(EtatDisjoncteur.FERME);
        s = breaker.onFailure(s, t0);
        assertThat(s.etat()).isEqualTo(EtatDisjoncteur.OUVERT);
        assertThat(breaker.decide(s,
t0.plusSeconds(60))).isEqualTo(Decision.SUSPENDRE);
        assertThat(breaker.decide(s,
t0.plus(Duration.ofMinutes(5)))).isEqualTo(Decision.LIRE_ESSAI);
        s = breaker.onSuccess(breaker.halfOpen(s), t0.plus(Duration.ofMinutes(5)));
        assertThat(s).extracting(State::etat,
State::echecs).containsExactly(EtatDisjoncteur.FERME, 0);
    }

    @Test
    @DisplayName("HOM-14 : un essai en demi-ouverture qui échoue rouvre le
disjoncteur sans changer l'instant d'ouverture")
    void failedTrialReopens() {
        State ouvert = new State(EtatDisjoncteur.OUVERT, 3, t0);
        State apres = breaker.onFailure(breaker.halfOpen(ouvert),
t0.plus(Duration.ofMinutes(6)));
        assertThat(apres).extracting(State::etat,
State::depuis).containsExactly(EtatDisjoncteur.OUVERT, t0);
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
//
src/test/java/org/gimac/des/homologation/application/usecase/ResilientReplayIT.java
@SpringBootTest(properties = {"gimac.adapters.simulator-reader=real",
"gimac.replay.period-ms=3600000",
    "gimac.replay.breaker.threshold=2", "gimac.replay.breaker.open-duration=PT0S"})
@AutoConfigureMockMvc
@Testcontainers
class ResilientReplayIT {
    @Container @ServiceConnection static PostgreSQLContainer<?> postgres = new
PostgreSQLContainer<>("postgres:16-alpine");
    @Container @ServiceConnection static KafkaContainer kafka = new
KafkaContainer("apache/kafka:3.8.0");
    @RegisterExtension static WireMockExtension simulateur =
WireMockExtension.newInstance()
        .options(wireMockConfig().dynamicPort()).build();

    @DynamicPropertySource
    static void props(DynamicPropertyRegistry r)
{ r.add("gimac.clients.simulator.base-url", simulateur::baseUrl); }

    @Autowired MockMvc mvc;
    @Autowired HomologationFixtures fixtures;
    @Autowired ReplayUseCase replay;
    @Autowired CampaignResultRepository results;
    @Autowired RejectedRecordRepository rejected;

    @Test
```

```
    @DisplayName("HOM-14 : arrêt du simulateur puis rétablissement sans doublon")
    void outageThenRecoveryWithoutDuplicates() throws Exception {
        Liaison liaison = fixtures.liaisonEnHomologation("Banque B");
        simulateur.stubFor(get(urlPathMatching("/lecture/v1/participants/.*/
campagnes")).willReturn(okJson("{\"campagnes\":[\"CMP-1\"]}")));
        simulateur.stubFor(get(urlPathEqualTo("/lecture/v1/campagnes/CMP-1/
resultats")).willReturn(okJson(SimulatorPayloads.twoResults("CMP-1"))));
        replay.runAll();

        simulateur.resetAll();
        simulateur.stubFor(any(anyUrl()).willReturn(serviceUnavailable()));
        replay.runAll();
        replay.runAll();

mvc.perform(get("/api/v1/reprise/etat").with(TestJwt.as(fixtures.ingenieurId(),
"HOM_LIAISON_READ")))
            .andExpect(jsonPath("$.etatDisjoncteur").value("OUVERT"))
            .andExpect(jsonPath("$.suspenduDepuis").isNotEmpty());

        simulateur.resetAll();
        simulateur.stubFor(get(urlPathMatching("/lecture/v1/participants/.*/
campagnes")).willReturn(okJson("{\"campagnes\":[\"CMP-1\"]}")));
        simulateur.stubFor(get(urlPathEqualTo("/lecture/v1/campagnes/CMP-1/
resultats")).willReturn(okJson(SimulatorPayloads.twoResults("CMP-1"))));
        replay.runAll(); // lecture d'essai réussie : fermeture


mvc.perform(get("/api/v1/reprise/etat").with(TestJwt.as(fixtures.ingenieurId(),
"HOM_LIAISON_READ")))
            .andExpect(jsonPath("$.etatDisjoncteur").value("FERME"));

assertThat(results.findByLiaisonIdOrderByDateExecutionDesc(liaison.getId())).hasSize(
2);
        simulateur.verify(getRequestedFor(urlPathEqualTo("/lecture/v1/campagnes/CMP-
1/resultats"))
            .withQueryParam("depuis", equalTo("2"))); // reprise à la dernière
position enregistrée
    }

    @Test
    @DisplayName("HOM-14 : une réponse mal formée est dirigée vers le rebut sans
résultat partiel")
    void malformedResponseGoesToRejected() {
        Liaison liaison = fixtures.liaisonEnHomologation("Établissement de paiement
A");
        simulateur.stubFor(get(urlPathMatching("/lecture/v1/participants/.*/
campagnes")).willReturn(okJson("{\"campagnes\":[\"CMP-9\"]}")));
        simulateur.stubFor(get(urlPathEqualTo("/lecture/v1/campagnes/CMP-9/
resultats"))
            .willReturn(okJson(SimulatorPayloads.oneValidOneWithoutVerdict("CMP-
9"))));

        replay.runAll();


assertThat(results.findByLiaisonIdOrderByDateExecutionDesc(liaison.getId())).isEmpty(
);
        assertThat(rejected.findAll()).singleElement().satisfies(r -> {
            assertThat(r.getSource()).isEqualTo("simulateur:campagne:CMP-9");
            assertThat(r.getEmpreinte()).hasSize(64);
            assertThat(r.getMotif()).contains("verdict");
        });
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: HOM-14 Tolérance à l'indisponibilité du simulateur

  Contexte:
    Étant donné une liaison de la "Banque B" en homologation dont la campagne "CMP-1"
est suivie
    Et un seuil d'ouverture du disjoncteur fixé à 2 échecs consécutifs

  Scénario: Nominal : ouverture du disjoncteur
    Étant donné que le simulateur de substitution est arrêté
    Quand deux cycles de reprise échouent
    Alors le disjoncteur est "OUVERT" et la reprise est suspendue
    Et le bandeau « reprise suspendue depuis » s'affiche sur les écrans DES-08 et
DES-09
    Et l'alerte "GimacRepriseSimulateurSuspendue" est émise vers MS-MONITORING
    Et la consultation des liaisons et la clôture des opérations restent disponibles

  Scénario: Alternatif : rattrapage après rétablissement
    Étant donné un disjoncteur ouvert dont le délai d'ouverture est écoulé
    Quand le simulateur est rétabli et que la lecture d'essai en demi-ouverture
réussit
    Alors le disjoncteur est "FERME"
    Et la reprise repart de la dernière position enregistrée de "CMP-1"
    Et aucun résultat n'est enregistré deux fois

  Scénario: Erreur : réponse mal formée
    Étant donné une réponse du simulateur dont un résultat n'a pas de verdict
    Quand elle est lue
    Alors elle est placée au rebut avec son empreinte et le motif du rejet
    Et aucun résultat de cette réponse n'est enregistré
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-ingenieur-homologation
BASE="${GATEWAY_URL}/bff/des/homologation"

# Nominal : état du disjoncteur et positions (bandeau)
curl -s "${BASE}/reprise/etat" -H "Authorization: Bearer ${TOKEN}" \
  | jq '{etatDisjoncteur, suspenduDepuis, echecsConsecutifs, campagnes: [.campagnes[]
| {campagneId, position}]}'

# Rebut et relance (administration de la reprise)
REBUT_ID=$(curl -s "${BASE}/reprise/rebut" -H "Authorization: Bearer ${TOKEN}" | jq -
r '.[0].id')
curl -s -o /dev/null -w "%{http_code}\n" -X POST
"${BASE}/reprise/rebut/${REBUT_ID}/relance" \
  -H "Authorization: Bearer ${TOKEN}"
# Attendu : 202 si le jeton porte HOM_REPLAY_ADMIN

# Refus : un correspondant d'institution ne consulte pas le rebut (attendu : 403)
source ./scripts/token.sh test-correspondant-a
curl -s -o /dev/null -w "%{http_code}\n" "${BASE}/reprise/rebut" -H "Authorization:
Bearer ${TOKEN}"
```

HOM-15 | Rapprocher les scénarios repris des opérations et isoler les scénarios inconnus

Élément	Valeur
Points	5
Priorité	Must Have
Sprint cible	Sprint 4
Écrans	DES-09
Dépendances	HOM-13, HOM-01

Objectif

Rattacher chaque résultat repris à l'opération dont l'`idScenario`, déclaré dans la version du référentiel de la liaison, est strictement identique, recalculer alors la couverture du service, et retenir l'exécution la plus récente d'un même scénario. Un scénario sans correspondance est placé en file de qualification, sans effet sur les couvertures, et l'ingénieur d'homologation en est notifié.

A. Implémentation

A1. Index des scénarios et règle de l'exécution la plus récente (domaine pur)

Le rapprochement se fait par égalité exacte : aucune normalisation de casse ni d'espaces, aucune approximation. L'index d'une version publiée est immuable, donc mis en cache par identifiant de version (`ScenarioIndexCache`, `ConcurrentHashMap`). À date d'exécution égale, l'identifiant d'exécution le plus grand départage (choix explicite, stable et reproductible).

```
// domain/service/ScenarioIndex.java
public final class ScenarioIndex {
    private final Map<String, String> codeParScenario; // idScenario vers code
d'opération de référence

    private ScenarioIndex(Map<String, String> declarations) { this.codeParScenario =
Map.copyOf(declarations); }

    /** Les doublons d'idScenario sont refusés à la publication de la version (HOM-
02). */
    public static ScenarioIndex of(Map<String, String> declarations) { return new
ScenarioIndex(declarations); }

    public Optional<String> operationCodeFor(String idScenario) {
        return Optional.ofNullable(idScenario).map(codeParScenario::get);
    }
}

// domain/service/LatestExecutionRule.java
public final class LatestExecutionRule {
    public record Execution(String executionId, Instant dateExecution, Verdict
verdict) {}

    private static final Comparator<Execution> ORDRE =

Comparator.comparing(Execution::dateExecution).thenComparing(Execution::executionId);

    public Optional<Execution> retained(Collection<Execution> executions) { return
executions.stream().max(ORDRE); }
```

```
}
```

A2. Rapprochement, file de qualification et nouveau rapprochement après migration

Le consommateur interne de `homologation.campaign-result-imported.v1` (sujet `homologation.campaign`, groupe `mshomologation-reconciliation`) appelle `reconcile` à travers `idempotentConsumer.handleOnce(...)`, le `subject` de l'événement portant l'identifiant du `CampaignResult`. Choix retenu pour la notification : un événement `homologation.notification-requested.v1` (sujet `homologation.notification`, consommé par MS-NOTIFICATION) adressé à l'ingénieur d'homologation en charge de la liaison. `rematchAfterMigration` est appelée par le cas d'utilisation de migration lorsqu'il bascule une liaison sur la version cible ; une entrée de la file dont tous les résultats sont rattachés en est retirée, la trace restant dans les événements consommés par MS-AUDIT.

```
// application/usecase/ReconciliationService.java
@Service
@RequiredArgsConstructor
public class ReconciliationService implements ReconciliationUseCase {
    private final CampaignResultRepository results;
    private final OperationInstanceRepository operations;
    private final UnmatchedScenarioRepository unmatched;
    private final ScenarioIndexCache indexes;
    private final CoverageRecalculationUseCase coverages; // HOM-11, calcul complet
HOM-17
    private final OutboxPublisher outboxPublisher;
    private final Clock clock;
    private final LatestExecutionRule latest = new LatestExecutionRule();

    @Override
    @Transactional
    public void reconcile(UUID campaignResultId) {
        CampaignResult r = results.findByIdForUpdate(campaignResultId).orElseThrow();
        if (r.getOperation() != null) return; // déjà rattaché
        Liaison liaison = r.getLiaison();

indexes.forVersion(liaison.getVersion().getId()).operationCodeFor(r.getIdScenario())
            .flatMap(code -> operations.findByLiaisonIdAndCode(liaison.getId(),
code))
            .ifPresentOrElse(op -> {
                r.operation(op);
                coverages.recalculate(liaison.getId(), op.getServiceCode(),
"RESULTAT_RAPPROCHE");
            }, () -> enqueue(liaison, r));
    }

    private void enqueue(Liaison liaison, CampaignResult r) {
        if (unmatched.existsByLiaisonIdAndIdScenarioAndCampagneId(liaison.getId(),
r.getIdScenario(), r.getCampagneId())) return;
        unmatched.save(new
UnmatchedScenario().liaison(liaison).idScenario(r.getIdScenario()).campagneId(r.getCa
mpagneId())
            .qualification(QualificationScenario.A_QUALIFIER).dateDetection(clock.ins
tant()));
        outboxPublisher.publish("Notification", liaison.getId().toString(),
"homologation.notification-requested.v1",
            new NotificationRequestedV1("UTILISATEUR",
liaison.getChefDeProjetId().toString(), "SCENARIO_INCONNU",
                Map.of("liaisonId", liaison.getId(), "idScenario", r.getIdScenario(),
"campagneId", r.getCampagneId())));
    }

    @Override
    @Transactional
    public int rematchAfterMigration(UUID liaisonId) {
```

```
        int rapproches = 0;
        for (UnmatchedScenario u : unmatched.findByLiaisonId(liaisonId)) {
            List<CampaignResult> enAttente =
results.findByLiaisonIdAndIdScenarioAndOperationIsNull(liaisonId, u.getIdScenario());
            enAttente.forEach(r -> reconcile(r.getId()));
            if (enAttente.stream().allMatch(r -> r.getOperation() != null)) {
                unmatched.delete(u);
                rapproches++;
            }
        }
        return rapproches;
    }

    /** Verdict qui fait foi pour une opération : exécution la plus récente (utilisé
par HOM-17 et la vue DES-09). */
    @Override
    @Transactional(readOnly = true)
    public Optional<Execution> retainedExecution(UUID operationId) {
        return latest.retained(results.findByOperationId(operationId).stream()
            .map(c -> new Execution(c.getExecutionId(), c.getDateExecution(),
c.getVerdict())).toList());
    }
}
```

A3. Ressources REST de la file de qualification

La qualification n'admet que `A_DECLARER_AU_REFERENTIEL` ou `HORS_PARCOURS`, avec un motif obligatoire ; elle n'a aucun effet sur les couvertures.

```
// web/rest/UnmatchedScenarioResource.java
@RestController
@RequiredArgsConstructor
class UnmatchedScenarioResource {
    private final UnmatchedScenarioRepository unmatched;
    private final QualifyScenarioUseCase qualify;
    private final UnmatchedScenarioMapper mapper;

    record QualificationRequest(@NotNull QualificationScenario qualification,
@NotBlank @Size(max = 1000) String motif) {
        @AssertTrue(message = "HOM_SCENARIO_QUALIFICATION_INVALID")
        boolean isFinale() { return qualification !=
QualificationScenario.A_QUALIFIER; }
    }

    @GetMapping("/api/v1/liaisons/{id}/scenarios-inconnus")
    @PreAuthorize("hasAuthority('HOM_LIAISON_READ')")
    List<UnmatchedScenarioDTO> list(@PathVariable UUID id) {
        return
unmatched.findByLiaisonIdOrderByDateDetectionDesc(id).stream().map(mapper::toDto).toL
ist();
    }

    @PostMapping("/api/v1/scenarios-inconnus/{id}/qualification")
    @PreAuthorize("hasAuthority('HOM_SCENARIO_QUALIFY')")
    UnmatchedScenarioDTO qualify(@PathVariable UUID id, @Valid @RequestBody
QualificationRequest req) {
        return mapper.toDto(qualify.qualify(id, req.qualification(),
req.motif())); // renseigne qualification et motif
    }
}
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
//
src/test/java/org/gimac/des/homologation/domain/service/ReconciliationRulesTest.java
class ReconciliationRulesTest {
```

```
    @Test
    @DisplayName("HOM-15 : rattachement par identifiant exact uniquement")
    void exactIdentifierOnly() {
        ScenarioIndex index = ScenarioIndex.of(Map.of("SC-ATM-001", "CI-RETRAIT-
DAB"));
        assertThat(index.operationCodeFor("SC-ATM-001")).contains("CI-RETRAIT-DAB");
        assertThat(index.operationCodeFor("sc-atm-001")).isEmpty();
        assertThat(index.operationCodeFor("SC-ATM-001 ")).isEmpty();
        assertThat(index.operationCodeFor("SC-ATM-0010")).isEmpty();
        assertThat(index.operationCodeFor(null)).isEmpty();
    }

    @Test
    @DisplayName("HOM-15 : l'exécution la plus récente est retenue")
    void latestExecutionRetained() {
        var ancienne = new Execution("EXE-1", Instant.parse("2026-10-20T08:00:00Z"),
Verdict.DEFAVORABLE);
        var recente = new Execution("EXE-2", Instant.parse("2026-10-21T08:00:00Z"),
Verdict.FAVORABLE);
        var memeInstant = new Execution("EXE-3", Instant.parse("2026-10-
21T08:00:00Z"), Verdict.DEFAVORABLE);

        assertThat(new LatestExecutionRule().retained(List.of(recente,
ancienne))).get()
            .extracting(Execution::verdict).isEqualTo(Verdict.FAVORABLE);
        assertThat(new LatestExecutionRule().retained(List.of(recente, memeInstant,
ancienne))).get()
            .extracting(Execution::executionId).isEqualTo("EXE-3"); // départage par
identifiant
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

Le simulateur de substitution (profil `test`) fournit un résultat dont le scénario n'est pas déclaré dans la version 1 ; la chaîne complète passe par l'outbox, Kafka et le consommateur.

```
// src/test/java/org/gimac/des/homologation/application/usecase/ReconciliationIT.java
@SpringBootTest(properties = "gimac.replay.period-ms=3600000")
@Testcontainers
class ReconciliationIT {
    @Container @ServiceConnection static PostgreSQLContainer<?> postgres = new
PostgreSQLContainer<>("postgres:16-alpine");
    @Container @ServiceConnection static KafkaContainer kafka = new
KafkaContainer("apache/kafka:3.8.0");

    @Autowired HomologationFixtures fixtures;
    @Autowired SimulatorReaderStubAdapter simulateur;
    @Autowired CampaignImportService replay;
    @Autowired ReconciliationUseCase reconciliation;
    @Autowired UnmatchedScenarioRepository unmatched;
    @Autowired CampaignResultRepository results;
    @Autowired OutboxEventRepository outbox;

    @Test
    @DisplayName("HOM-15 : scénario inconnu mis en qualification puis rapproché après
migration")
    void unknownThenMatchedAfterMigration() {
        Liaison liaison = fixtures.liaisonEnHomologation("Banque B"); // version 1
sans "SC-QR-DYN-001"
        simulateur.alimenter(liaison.getInstitutionId(),
SimulatorPayloads.result("CMP-3", "EXE-1", "SC-QR-DYN-001"));

        replay.runAll();
        await().atMost(Duration.ofSeconds(15)).untilAsserted(() ->
            assertThat(unmatched.findByLiaisonId(liaison.getId())).singleElement()
```

```
                .extracting(UnmatchedScenario::getQualification).isEqualTo(Qualificat
ionScenario.A_QUALIFIER));
        assertThat(outbox.countByEventType("homologation.coverage-
changed.v1")).isZero();
        assertThat(outbox.countByEventType("homologation.notification-
requested.v1")).isEqualTo(1);

        fixtures.switchLiaisonToVersionDeclaring(liaison.getId(), "SC-QR-DYN-001",
"PM-QR-DYNAMIQUE");

assertThat(reconciliation.rematchAfterMigration(liaison.getId())).isEqualTo(1);

        assertThat(unmatched.findByLiaisonId(liaison.getId())).isEmpty();

assertThat(results.findByLiaisonIdOrderByDateExecutionDesc(liaison.getId())).singleEl
ement()
            .satisfies(r -> assertThat(r.getOperation().getCode()).isEqualTo("PM-QR-
DYNAMIQUE"));
        assertThat(outbox.countByEventType("homologation.coverage-
changed.v1")).isEqualTo(1);
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: HOM-15 Rapprochement des scénarios repris et file de qualification

  Contexte:
    Étant donné une liaison de la "Banque B" en homologation sur la version 1 du
référentiel
    Et l'opération "CI-RETRAIT-DAB" déclarant le scénario "SC-ATM-001" dans cette
version

  Scénario: Nominal : scénario reconnu
    Étant donné un résultat repris pour le scénario "SC-ATM-001"
    Quand le rapprochement s'exécute
    Alors le résultat est rattaché à l'opération "CI-RETRAIT-DAB"
    Et la couverture de certification du service concerné est recalculée
    Et l'événement "homologation.coverage-changed.v1" est émis

  Scénario: Alternatif : exécutions multiples d'un même scénario
    Étant donné deux exécutions reprises du scénario "SC-ATM-001", la plus récente
étant favorable
    Quand la couverture est évaluée
    Alors le résultat de l'exécution la plus récente fait foi
    Et les deux exécutions restent consultables dans l'historique des campagnes

  Scénario: Erreur : scénario inconnu
    Étant donné un résultat repris pour le scénario "SC-INCONNU-042" déclaré dans
aucune opération de la version 1
    Quand le rapprochement s'exécute
    Alors le résultat est placé en file de qualification sans effet sur les
couvertures
    Et l'ingénieur d'homologation est notifié
    Quand il qualifie le scénario « hors parcours » avec le motif "Scénario d'essai
interne du simulateur"
    Alors la qualification et le motif sont enregistrés
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-ingenieur-homologation
BASE="${GATEWAY_URL}/bff/des/homologation"

# Nominal : file de qualification de la liaison
```

```
curl -s "${BASE}/liaisons/${LIAISON_ID}/scenarios-inconnus" -H "Authorization: Bearer
${TOKEN}" \
  | jq 'map({id, idScenario, campagneId, qualification})'
SCN_ID=$(curl -s "${BASE}/liaisons/${LIAISON_ID}/scenarios-inconnus" -H
"Authorization: Bearer ${TOKEN}" | jq -r '.[0].id')

# Nominal : qualification motivée
curl -s -X POST "${BASE}/scenarios-inconnus/${SCN_ID}/qualification" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" \
  -d '{"qualification":"A_DECLARER_AU_REFERENTIEL","motif":"Scénario prévu dans la
prochaine version"}' \
  | jq '{idScenario, qualification, motif}'

# Refus : qualification sans motif (attendu : 400)
curl -s -o /dev/null -w "%{http_code}\n" -X POST "${BASE}/scenarios-inconnus/$
{SCN_ID}/qualification" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" -d
'{"qualification":"HORS_PARCOURS"}'
```

HOM-16 | Garantir la non-intrusion vis-à-vis du simulateur de certification

Élément	Valeur
Points	3
Priorité	Must Have
Sprint cible	Sprint 4
Écrans	Aucun (exigence transverse de sécurité)
Dépendances	SOC-07, DOC-01

Objectif

Garantir par deux barrières indépendantes que MS-HOMOLOGATION n'émet que des lectures vers le simulateur : une politique de flux sortants Kubernetes limitée au port de lecture, et une garde applicative qui bloque et journalise vers MS-AUDIT toute autre méthode HTTP, signalée au RSSI. Les pièces des opérations d'échange de clés se limitent au procès-verbal de cérémonie et aux valeurs de contrôle ; tout composant de clé est refusé avant d'atteindre la GED.

A. Implémentation

A1. Politique de flux sortants (NetworkPolicy)

Le simulateur expose son interface de lecture sur un port dédié (SOC-07) ; aucune autre destination de sa zone n'est atteignable. Le socle applique un refus par défaut des flux sortants dans l'espace de noms (SOC-05) ; les politiques étant additives, aucune autre politique de l'espace de noms ne cible la zone du simulateur (vérifié par le recensement du test C).

```
# deploy/k8s/mshomologation/networkpolicy-egress-simulateur.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: mshomologation-egress-simulateur
  namespace: gimac-des
spec:
  podSelector:
    matchLabels:
      app: mshomologation
  policyTypes: [Egress]
  egress:
    # seul flux admis vers la zone du simulateur : le port de l'interface de lecture
    - to:
        - namespaceSelector:
            matchLabels:
              gimac.org/zone: simulateur-certification
          podSelector:
            matchLabels:
              app: simulateur-lecture
      ports:
        - protocol: TCP
          port: 8443
```

A2. Garde applicative en lecture seule et journalisation des tentatives

Le client `simulatorRestClient` utilisé par HOM-13 porte un intercepteur qui laisse passer le seul `GET`. Toute autre méthode est bloquée avant l'émission ; la tentative (méthode et chemin, jamais le corps) est publiée dans une transaction indépendante, pour survivre à l'annulation de

la transaction appelante : `homologation.simulator-write-blocked.v1` pour MS-AUDIT et une demande de notification adressée au rôle `RSSI`.

```
// infrastructure/client/simulator/SimulatorClientConfiguration.java
@Configuration
class SimulatorClientConfiguration {
    @Bean("simulatorRestClient")
    RestClient simulatorRestClient(RestClient.Builder builder,
ReadOnlyGuardInterceptor guard,
                                   @Value("${gimac.clients.simulator.base-url}")
String baseUrl) {
        return builder.baseUrl(baseUrl).requestInterceptor(guard).build(); // mTLS
fourni par le socle
    }
}

// infrastructure/client/simulator/ReadOnlyGuardInterceptor.java
@Component
@RequiredArgsConstructor
class ReadOnlyGuardInterceptor implements ClientHttpRequestInterceptor {
    private final BlockedWriteAudit audit;

    @Override
    public ClientHttpResponse intercept(HttpRequest request, byte[] body,
ClientHttpRequestExecution execution)
            throws IOException {
        if (!HttpMethod.GET.equals(request.getMethod())) {
            audit.record(request.getMethod().name(), request.getURI().getPath());
            throw new SimulatorWriteBlockedException(request.getMethod().name());
        }
        return execution.execute(request, body);
    }
}

// application/usecase/BlockedWriteAudit.java
@Service
@RequiredArgsConstructor
public class BlockedWriteAudit {
    private final OutboxPublisher outboxPublisher;
    private final Clock clock;

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void record(String methode, String chemin) {
        Instant now = clock.instant();
        outboxPublisher.publish("Campaign", "simulateur", "homologation.simulator-
write-blocked.v1",
            new SimulatorWriteBlockedV1(methode, chemin, now));
        outboxPublisher.publish("Notification", "simulateur",
"homologation.notification-requested.v1",
            new NotificationRequestedV1("ROLE", "RSSI",
"ECRITURE_SIMULATEUR_BLOQUEE", Map.of("methode", methode, "chemin", chemin)));
    }
}
```

A3. Pièces de sécurité réduites à des condensés

`KeyMaterialPolicy` est appelée par `SubmitEvidenceService` (HOM-09) avant l'appel à MS-DOCUMENT, pour toutes les opérations : un composant de clé n'atteint jamais la GED. Les types interdits sont refusés quel que soit le référentiel ; les valeurs de contrôle (`VALEURS_DE_CONTROLE`, format texte ou CSV) sont analysées et toute suite de 32 à 48 chiffres hexadécimaux, éventuellement groupés par quatre (forme d'un composant de clé double ou triple longueur), est rejetée en 422. Une valeur de contrôle de 6 caractères ou un condensé SHA-256 de 64 caractères ne sont pas concernés.

```
// domain/service/KeyMaterialPolicy.java
```

```
public final class KeyMaterialPolicy {
    public static final Set<String> TYPES_INTERDITS = Set.of("COMPOSANT_DE_CLE",
"CLE_CHIFFREE", "BLOC_CLE");
    private static final Pattern COMPOSANT =
        Pattern.compile("(?i)(?<![0-9a-f])(?:[0-9a-f]{4}[ -]?){7,11}[0-9a-f]{4}(?![0-
9a-f])");

    public void check(String typePiece, String contentType, byte[] contenu) {
        if (TYPES_INTERDITS.contains(typePiece)) {
            throw new DocumentRejectedException("HOM_KEY_COMPONENT_FORBIDDEN", "Type
de pièce interdit : " + typePiece);
        }
        boolean texte = contentType != null && (contentType.startsWith("text/") ||
contentType.equals("application/csv"));
        if (texte && COMPOSANT.matcher(new String(contenu,
StandardCharsets.ISO_8859_1)).find()) {
            throw new DocumentRejectedException("HOM_KEY_COMPONENT_FORBIDDEN",
                "La pièce contient une valeur ayant la forme d'un composant de clé ;
seules les valeurs de contrôle sont admises");
        }
    }
}

// application/usecase/SubmitEvidenceService.java (ajout HOM-16, avant
documentDeposit.deposit)
keyMaterial.check(up.typePiece(), up.contentType(), up.contenu());
```

Les opérations d'échange de clés de la branche carte interbancaire déclarent `preuveAttendue = {"CARTE_INTERBANCAIRE": ["PV_CEREMONIE_CLES", "VALEURS_DE_CONTROLE"]}`.

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
//
src/test/java/org/gimac/des/homologation/architecture/SimulatorReadOnlyArchTest.java
class SimulatorReadOnlyArchTest {
    static final JavaClasses CLASSES = new ClassFileImporter()
        .withImportOption(ImportOption.Predefined.DO_NOT_INCLUDE_TESTS).importPackage
s("org.gimac.des.homologation");

    @Test
    @DisplayName("HOM-16 : l'adaptateur du simulateur n'utilise aucune méthode HTTP
autre que GET")
    void onlyGetTowardsSimulator() {
        noClasses().that().resideInAPackage("..infrastructure.client.simulator..")
            .should().callMethod(RestClient.class,
"post").orShould().callMethod(RestClient.class, "put")
            .orShould().callMethod(RestClient.class,
"patch").orShould().callMethod(RestClient.class, "delete")
            .orShould().callMethod(RestClient.class, "method", HttpMethod.class)
            .orShould().accessField(HttpMethod.class,
"POST").orShould().accessField(HttpMethod.class, "PUT")
            .orShould().accessField(HttpMethod.class,
"PATCH").orShould().accessField(HttpMethod.class, "DELETE")
            .orShould().dependOnClassesThat().belongToAnyOf(RestTemplate.class,
WebClient.class)
            .check(CLASSES);
    }

    @Test
    @DisplayName("HOM-16 : le port SimulatorReaderPort ne déclare que des lectures")
    void portIsReadOnly() {
        methods().that().areDeclaredIn(SimulatorReaderPort.class)
            .should().haveNameMatching("(list|read)[A-Z].*")
            .check(CLASSES);
    }
}
```

```
// src/test/java/org/gimac/des/homologation/domain/service/KeyMaterialPolicyTest.java
class KeyMaterialPolicyTest {
    final KeyMaterialPolicy policy = new KeyMaterialPolicy();

    @Test
    @DisplayName("HOM-16 : un composant de clé est refusé, les valeurs de contrôle
sont acceptées")
    void keyComponentRefusedCheckValuesAccepted() {
        byte[] kcv = "ZMK;KCV\nZMK-A;3F9A21\n".getBytes();
        byte[] composant = "ZMK composant 1;0123 4567 89AB CDEF FEDC BA98 7654 3210\
n".getBytes();

        assertThatCode(() -> policy.check("VALEURS_DE_CONTROLE", "text/csv",
kcv)).doesNotThrowAnyException();
        assertThatThrownBy(() -> policy.check("VALEURS_DE_CONTROLE", "text/csv",
composant))
            .hasFieldOrPropertyWithValue("code", "HOM_KEY_COMPONENT_FORBIDDEN");
        assertThatThrownBy(() -> policy.check("COMPOSANT_DE_CLE", "application/pdf",
new byte[0]))
            .hasFieldOrPropertyWithValue("code", "HOM_KEY_COMPONENT_FORBIDDEN");
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
// src/test/java/org/gimac/des/homologation/security/NonIntrusionIT.java
@SpringBootTest(properties = {"gimac.adapters.simulator-reader=real",
"gimac.adapters.document-deposit=real"})
@AutoConfigureMockMvc
@Testcontainers
class NonIntrusionIT {
    @Container @ServiceConnection static PostgreSQLContainer<?> postgres = new
PostgreSQLContainer<>("postgres:16-alpine");
    @Container @ServiceConnection static KafkaContainer kafka = new
KafkaContainer("apache/kafka:3.8.0");
    @RegisterExtension static WireMockExtension simulateur =
WireMockExtension.newInstance().options(wireMockConfig().dynamicPort()).build();
    @RegisterExtension static WireMockExtension msDocument =
WireMockExtension.newInstance().options(wireMockConfig().dynamicPort()).build();

    @DynamicPropertySource
    static void props(DynamicPropertyRegistry r) {
        r.add("gimac.clients.simulator.base-url", simulateur::baseUrl);
        r.add("gimac.clients.msdocument.base-url", msDocument::baseUrl);
    }

    @Autowired MockMvc mvc;
    @Autowired HomologationFixtures fixtures;
    @Autowired @Qualifier("simulatorRestClient") RestClient simulatorClient;
    @Autowired RequestMappingHandlerMapping mappings;
    @Autowired OutboxEventRepository outbox;

    @Test
    @DisplayName("HOM-16 : recensement des routes, aucune route d'écriture sur les
campagnes ou les verdicts")
    void noWriteRouteOnCampaigns() {
        assertThat(mappings.getHandlerMethods().keySet())
            .filteredOn(info -> info.getPatternValues().stream().anyMatch(p ->
p.contains("/campagnes")))
            .allSatisfy(info ->
assertThat(info.getMethodsCondition().getMethods()).containsOnly(RequestMethod.GET));
    }

    @Test
    @DisplayName("HOM-16 : une tentative d'écriture vers le simulateur est bloquée et
journalisée")
    void writeAttemptBlockedAndAudited() {
```

```
        assertThatThrownBy(() ->
simulatorClient.post().uri("/lecture/v1/campagnes").retrieve().toBodilessEntity())
            .isInstanceOf(SimulatorWriteBlockedException.class);
        simulateur.verify(0, anyRequestedFor(anyUrl()));
        assertThat(outbox.countByEventType("homologation.simulator-write-
blocked.v1")).isEqualTo(1);
    }

    @Test
    @DisplayName("HOM-16 : une pièce contenant un composant de clé est rejetée sans
atteindre MS-DOCUMENT")
    void keyComponentRejected() throws Exception {
        UUID opId = fixtures.eligibleOperation(TypeProjet.CARTE_INTERBANCAIRE,
"VALEURS_DE_CONTROLE");
        mvc.perform(multipart("/api/v1/operations/{id}/preuves", opId)
                .file(new MockMultipartFile("fichier", "kcv.csv", "text/csv",
                    "ZMK;0123456789ABCDEFFEDCBA9876543210\n".getBytes()))
                .param("typePiece", "VALEURS_DE_CONTROLE")
                .with(TestJwt.as(fixtures.ingenieurId(), "HOM_EVIDENCE_SUBMIT")))
            .andExpect(status().isUnprocessableEntity())
            .andExpect(jsonPath("$.code").value("HOM_KEY_COMPONENT_FORBIDDEN"));
        msDocument.verify(0, anyRequestedFor(anyUrl()));
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: HOM-16 Non-intrusion vis-à-vis du simulateur de certification

  Scénario: Nominal : flux sortants limités à la lecture
    Étant donné la politique de flux sortants du service "mshomologation"
    Quand le service s'adresse au simulateur pour reprendre les campagnes
    Alors seules des requêtes GET vers le port de lecture sont émises
    Et le recensement des routes du service ne comporte aucune route d'écriture sur
les campagnes

  Scénario: Alternatif : pièce de sécurité réduite à un condensé
    Étant donné une opération de la branche carte interbancaire relative à un échange
de clés
    Quand le procès-verbal de cérémonie puis le fichier des valeurs de contrôle sont
déposés
    Alors les deux pièces sont acceptées
    Mais un fichier contenant un composant de clé est refusé avec le code
"HOM_KEY_COMPONENT_FORBIDDEN"

  Scénario: Erreur : tentative d'écriture vers le simulateur
    Étant donné une requête POST émise par erreur vers le simulateur
    Quand elle atteint la politique de flux sortants
    Alors elle est bloquée avant toute émission réseau
    Et l'événement "homologation.simulator-write-blocked.v1" est journalisé par MS-
AUDIT
    Et le RSSI est notifié
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-ingenieur-homologation
BASE="${GATEWAY_URL}/bff/des/homologation"

# Nominal : dépôt des valeurs de contrôle d'une opération d'échange de clés (201)
curl -s -X POST "${BASE}/operations/${OP_ECHANGE_CLES}/preuves" -H "Authorization:
Bearer ${TOKEN}" \
  -F "typePiece=VALEURS_DE_CONTROLE" -F "fichier=@./fixtures/valeurs-de-
controle.csv;type=text/csv" \
  | jq '{typePiece, empreinte, scelle}'
```

```
# Refus : fichier contenant un composant de clé (attendu : 422)
curl -s -o /dev/null -w "%{http_code}\n" -X POST "${BASE}/operations/$
{OP_ECHANGE_CLES}/preuves" \
  -H "Authorization: Bearer ${TOKEN}" \
  -F "typePiece=VALEURS_DE_CONTROLE" -F "fichier=@./fixtures/composant-de-cle-
factice.csv;type=text/csv"

# Refus : aucune route d'écriture exposée sur les résultats repris (attendu : 409,
tentative journalisée)
curl -s -o /dev/null -w "%{http_code}\n" -X DELETE
"${BASE}/liaisons/${LIAISON_ID}/campagnes/CMP-1" \
  -H "Authorization: Bearer ${TOKEN}"

# Contrôle réseau depuis le pod (recette) : le port d'administration du simulateur
est injoignable (attendu : 000)
kubectl -n gimac-des exec deploy/mshomologation -- \
  curl -s -o /dev/null -w "%{http_code}\n" --max-time 5 -X POST "$
{SIMULATOR_ADMIN_URL}/campagnes"
```

HOM-17 | Calculer les couvertures de certification et de mise en service par service

Élément	Valeur
Points	5
Priorité	Must Have
Sprint cible	Sprint 4
Écrans	DES-09
Dépendances	HOM-05, HOM-10, HOM-15

Objectif

Conserver pour chaque liaison et chaque service du périmètre deux couvertures distinctes (certification ; mise en service), recalculées à chaque clôture, réouverture ou rapprochement, avec émission de `homologation.coverage-changed.v1` quand une valeur change. Un service sans opération de référence est non calculable, jamais couvert à cent pour cent.

A. Implémentation

A1. Service de domaine `CoverageCalculator`

Service pur. Les services `HORS_PERIMETRE` sont écartés par l'appelant et n'entrent dans aucun dénominateur. Choix retenus : le dernier résultat d'un scénario est celui de plus grande `dateExecution` (départage par `executionId`) parmi les verdicts `FAVORABLE` et `DEFAVORABLE` (un `NON_EXECUTE` ne remplace pas un verdict) ; le taux est tronqué (`RoundingMode.DOWN`) pour qu'aucun 99,995 ne s'affiche comme 100.

```
// domain/service/CoverageCalculator.java
public final class CoverageCalculator {
    public static final String REFERENTIEL_INCOMPLET = "REFERENTIEL_INCOMPLET";
    public record RequiredScenario(String serviceCode, String idScenario) {}
    public record ResultView(String idScenario, Verdict verdict, Instant
dateExecution, String executionId) {}
    public record GoLiveOperationView(String serviceCode, String code, EtatOperation
etat) {}
    public record CoverageSnapshot(String serviceCode, NatureCouverture nature, int
numerateur, int denominateur,
                                   BigDecimal taux, boolean calculable, String
motifNonCalculable, List<String> manquants) {}

    public List<CoverageSnapshot> compute(Set<String> services,
List<RequiredScenario> required,
                                          List<ResultView> results,
List<GoLiveOperationView> goLiveOps) {
        Map<String, Verdict> last = results.stream().filter(r -> r.verdict() !=
Verdict.NON_EXECUTE)
            .collect(Collectors.groupingBy(ResultView::idScenario,
Collectors.collectingAndThen(

Collectors.maxBy(Comparator.comparing(ResultView::dateExecution).thenComparing(Result
View::executionId)),
                r -> r.orElseThrow().verdict())));
        List<CoverageSnapshot> out = new ArrayList<>();
        for (String s : new TreeSet<>(services)) {                      // ordre
stable
            List<String> sc = required.stream().filter(r ->
```

```
r.serviceCode().equals(s))
                .map(RequiredScenario::idScenario).distinct().sorted().toList();
            out.add(snapshot(s, NatureCouverture.CERTIFICATION, sc.size(),
                sc.stream().filter(id -> last.get(id) !=
Verdict.FAVORABLE).toList()));
            List<GoLiveOperationView> ops = goLiveOps.stream().filter(o ->
o.serviceCode().equals(s)).toList();
            out.add(snapshot(s, NatureCouverture.MISE_EN_SERVICE, ops.size(),
                ops.stream().filter(o -> o.etat() !=
EtatOperation.CLOSE).map(GoLiveOperationView::code).sorted().toList()));
        }
        return out;
    }

    private CoverageSnapshot snapshot(String s, NatureCouverture n, int den,
List<String> manquants) {
        if (den == 0) return new CoverageSnapshot(s, n, 0, 0, null, false,
REFERENTIEL_INCOMPLET, List.of());
        int num = den - manquants.size();
        return new CoverageSnapshot(s, n, num, den,
            BigDecimal.valueOf(num * 100L).divide(BigDecimal.valueOf(den), 2,
RoundingMode.DOWN), true, null, manquants);
    }
}
```

A2. Recalcul incrémental, émission et lecture

HOM-10 (clôture), HOM-11 (réouverture) et HOM-15 (rapprochement) publient l'événement applicatif `CoverageTrigger` ; l'écouteur synchrone recalcule le seul service concerné dans la même transaction (couverture et outbox validées ensemble), puis signale `CoverageRecalculated` à HOM-18. `GET /liaisons/{id}/couvertures` (permission `HOM_LIAISON_READ`) renvoie par service les deux couvertures et les éléments manquants, recalculés à la lecture par le même service de domaine.

```
// application/usecase/RecalculateCoverageService.java
@Service
@RequiredArgsConstructor
public class RecalculateCoverageService {
    public record CoverageTrigger(UUID liaisonId, String serviceCode, String cause)
{}
    public record CoverageRecalculated(UUID liaisonId) {}

    private final CoverageCalculator calculator = new CoverageCalculator();
    private final LiaisonRepository liaisons;
    private final ScopeItemRepository scopeItems;
    private final ReferenceOperationRepository references;
    private final CampaignResultRepository results;
    private final OperationInstanceRepository operations;
    private final CoverageRepository coverages;
    private final OutboxPublisher outboxPublisher;
    private final ApplicationEventPublisher events;

    @EventListener
    @Transactional(propagation = Propagation.MANDATORY)
    public void on(CoverageTrigger t) {
        Liaison l = liaisons.getReferenceById(t.liaisonId());
        if (scopeItems.findServiceCodesInScope(l.getId()).contains(t.serviceCode()))
{ // exclut HORS_PERIMETRE
            calculator.compute(Set.of(t.serviceCode()),
                    references.findRequiredScenarios(l.getVersion().getId(),
t.serviceCode()),
                    results.findViewsByLiaisonAndService(l.getId(), t.serviceCode()),
                    operations.findGoLiveViews(l.getId(), t.serviceCode()))
                .forEach(s -> upsert(l, s, t.cause()));
        }
        events.publishEvent(new CoverageRecalculated(l.getId()));
```

```
    }

    private void upsert(Liaison l, CoverageCalculator.CoverageSnapshot s, String
cause) {
        Coverage c = coverages.findByLiaisonIdAndServiceCodeAndNature(l.getId(),
s.serviceCode(), s.nature())
            .orElseGet(() -> Coverage.of(l, s.serviceCode(), s.nature()));
        // apply(...) renvoie vrai si une valeur a changé : sinon, aucune émission
        if (c.apply(s.numerateur(), s.denominateur(), s.taux(), s.calculable(),
s.motifNonCalculable(), Instant.now())) {
            outboxPublisher.publish("coverage", l.getId().toString(),
"homologation.coverage-changed.v1",
                new CoverageChangedV1(l.getId(), l.getInstitutionId(),
s.serviceCode(), s.nature(),
                    s.numerateur(), s.denominateur(), s.taux(), s.calculable(),
s.motifNonCalculable(), cause));
        }
        coverages.save(c);
    }
}
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
//
src/test/java/org/gimac/des/homologation/domain/service/CoverageCalculatorTest.java
class CoverageCalculatorTest {
    private final CoverageCalculator calc = new CoverageCalculator();
    private static final Instant T0 = Instant.parse("2026-10-05T08:00:00Z");

    @Test
    @DisplayName("HOM-17 : les deux couvertures sont calculées séparément sur un jeu
de référence")
    void deuxCouvertures() {
        var out = calc.compute(Set.of("RETRAIT_GAB"),
            List.of(new RequiredScenario("RETRAIT_GAB", "SC-RET-001"), new
RequiredScenario("RETRAIT_GAB", "SC-RET-002")),
            List.of(new ResultView("SC-RET-001", FAVORABLE, T0, "EX-1"), new
ResultView("SC-RET-002", NON_EXECUTE, T0, "EX-2")),
            List.of(new GoLiveOperationView("RETRAIT_GAB", "OP-MES-01", CLOSE), new
GoLiveOperationView("RETRAIT_GAB", "OP-MES-02", ELIGIBLE)));
        assertThat(out).extracting(CoverageSnapshot::nature, CoverageSnapshot::taux)
            .containsExactly(tuple(CERTIFICATION, new BigDecimal("50.00")),
tuple(MISE_EN_SERVICE, new BigDecimal("50.00")));
        assertThat(out.get(0).manquants()).containsExactly("SC-RET-002");
    }

    @Test
    @DisplayName("HOM-17 : un service sans opération de référence est non calculable
(référentiel incomplet)")
    void nonCalculable() {
        assertThat(calc.compute(Set.of("TRANSFERT_P2P"), List.of(), List.of(),
List.of())).allSatisfy(s -> {
            assertThat(s.calculable()).isFalse();
            assertThat(s.motifNonCalculable()).isEqualTo("REFERENTIEL_INCOMPLET");
            assertThat(s.taux()).isNull();
        });
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

La classe de base `AbstractHomologationIT` est posée ici et réutilisée par HOM-18 à HOM-24.

```
// src/test/java/org/gimac/des/homologation/support/AbstractHomologationIT.java
```

```
@SpringBootTest @AutoConfigureMockMvc @Testcontainers @ActiveProfiles("test")
public abstract class AbstractHomologationIT {
    @Container @ServiceConnection static PostgreSQLContainer<?> postgres = new
PostgreSQLContainer<>("postgres:16-alpine");
    @Container @ServiceConnection static KafkaContainer kafka = new
KafkaContainer("apache/kafka:3.8.1");
    @Autowired protected MockMvc mockMvc;
    @Autowired protected JdbcTemplate jdbc;
    @Autowired protected TransactionTemplate tx;
    @Autowired protected HomologationFixtures fixtures;   // jeux de données du
module
    @Autowired protected ApplicationEventPublisher events;

    protected long outbox(String type, UUID aggregateId) {
        return jdbc.queryForObject("select count(*) from outbox_event where
event_type = ? and aggregate_id::text = ?",
            Long.class, type, aggregateId.toString());
    }
}

// src/test/java/org/gimac/des/homologation/web/rest/CoverageResourceIT.java
class CoverageResourceIT extends AbstractHomologationIT {
    @Test
    @DisplayName("HOM-17 : recalcul après un résultat défavorable et émission de
coverage-changed")
    void recalculApresDefavorable() throws Exception {
        UUID id = fixtures.liaisonAvecResultats("RETRAIT_GAB", Map.of("SC-RET-001",
FAVORABLE, "SC-RET-002", FAVORABLE));
        tx.executeWithoutResult(s -> {
            fixtures.ajouterResultat(id, "SC-RET-002", DEFAVORABLE, "EX-9");
            events.publishEvent(new CoverageTrigger(id, "RETRAIT_GAB",
"RAPPROCHEMENT"));
        });
        mockMvc.perform(get("/api/v1/liaisons/{id}/couvertures", id)
                .with(jwt().authorities(new
SimpleGrantedAuthority("HOM_LIAISON_READ"))))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$[0].certification.taux").value(50.00))
            .andExpect(jsonPath("$[0].certification.manquants[0]").value("SC-RET-
002"));
        assertThat(outbox("homologation.coverage-changed.v1", id)).isEqualTo(1);
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: HOM-17 Couvertures de certification et de mise en service par service

  Scénario: Nominal : recalcul à la clôture et à la reprise
    Étant donné une liaison dont le périmètre comprend trois services
    Quand une opération de mise en service du service "RETRAIT_GAB" est close ou un
résultat de scénario de ce service est rapproché
    Alors les couvertures de certification et de mise en service du service
"RETRAIT_GAB" sont recalculées
    Et l'événement "homologation.coverage-changed.v1" est émis

  Scénario: Alternatif : résultat défavorable plus récent
    Étant donné un scénario "SC-RET-002" dont le dernier résultat favorable est suivi
d'un résultat défavorable
    Quand la couverture est recalculée
    Alors la couverture de certification du service "RETRAIT_GAB" vaut "50.00"

  Scénario: Erreur : service sans opération de référence
    Étant donné un service "TRANSFERT_P2P" du périmètre sans opération rattachée dans
la version
    Quand la couverture est calculée
```

```
    Alors la couverture est non calculable avec le motif "REFERENTIEL_INCOMPLET"
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-chef-des
# Nominal : deux couvertures par service et éléments manquants
curl -s "${GATEWAY_URL}/bff/des/homologation/liaisons/${LIAISON_ID}/couvertures" -H
"Authorization: Bearer ${TOKEN}" \
  | jq '.[] | {serviceCode, cert: .certification.taux, mes: .miseEnService.taux,
manquants: .certification.manquants}'

# Refus : un correspondant n'accède pas aux routes du DES (OPA), attendu 403
source ./scripts/token.sh test-correspondant-a
curl -s -o /dev/null -w "%{http_code}\n" \
  "${GATEWAY_URL}/bff/des/homologation/liaisons/${LIAISON_ID}/couvertures" -H
"Authorization: Bearer ${TOKEN}"
```

HOM-18 | Détecter l'aptitude à la certification et saisir MS-CERTIFICATION

Élément	Valeur
Points	5
Priorité	Must Have
Sprint cible	Sprint 4
Écrans	DES-09
Dépendances	HOM-17, SOC-03

Objectif

Déclarer une liaison apte à la certification dès que la couverture de certification vaut cent pour cent sur tout le périmètre et que les opérations préalables au circuit sont closes, calculer l'empreinte du socle d'évaluation et émettre une seule fois par cycle `homologation.liaison-ready-for-certification.v1` ; exposer à MS-CERTIFICATION le socle et les intervenants par deux interfaces internes.

A. Implémentation

A1. Règle d'aptitude

Choix retenus : seule une liaison `EN_HOMOLOGATION` peut devenir apte (une liaison `GELEE` ou déjà `APTE_CERTIFICATION` ne l'est jamais, ce qui porte aussi la déduplication) ; une opération est « préalable au circuit » si elle n'est ni de nature `MISE_EN_SERVICE` ni du segment `TRONC_COMMUN_AVAL`, et n'est pas `HORS_PERIMETRE`.

```
// domain/service/AptitudeRule.java
public final class AptitudeRule {
    private static final BigDecimal CENT = new BigDecimal("100.00");
    public record CoverageView(String serviceCode, NatureCouverture nature, boolean
calculable, BigDecimal taux) {}
    public record OperationView(String code, Segment segment, NatureCouverture
nature, EtatOperation etat) {}

    static boolean isPrerequisite(OperationView o) {
        return o.nature() != NatureCouverture.MISE_EN_SERVICE && o.segment() !=
Segment.TRONC_COMMUN_AVAL
            && o.etat() != EtatOperation.HORS_PERIMETRE;
    }

    public boolean isReady(EtatLiaison etat, Set<String> services, List<CoverageView>
cov, List<OperationView> ops) {
        if (etat != EtatLiaison.EN_HOMOLOGATION || services.isEmpty()) return false;
        boolean certComplete = services.stream().allMatch(s ->
cov.stream().anyMatch(c -> c.serviceCode().equals(s)
            && c.nature() == NatureCouverture.CERTIFICATION && c.calculable() &&
CENT.compareTo(c.taux()) == 0));
        return certComplete &&
ops.stream().filter(AptitudeRule::isPrerequisite).allMatch(o -> o.etat() ==
EtatOperation.CLOSE);
    }
}
```

A2. Empreinte canonique du socle (`EvaluationBase`)

Le socle est sérialisé en texte canonique : une ligne par élément, sections dans un ordre fixe (référentiel, services, résultats retenus, preuves, horodatages), éléments triés par clé, champs séparés par `|`, puis SHA-256 en hexadécimal minuscule. Le jeton RFC 3161 entre par son propre SHA-256 (octets décodés du base64). Le préfixe `gimac-socle-v1` versionne le format.

```
// domain/service/EvaluationBaseHasher.java
public final class EvaluationBaseHasher {
    public record RetainedResult(String idScenario, String campagneId, String
executionId, Verdict verdict, String empreinteReponse) {}
    public record EvidenceView(String operationCode, String documentRef, int version,
String empreinte) {}
    public record TimestampView(String operationCode, String empreinteCloture, String
jetonBase64) {}
    public record Composition(UUID versionId, int versionNumero, List<String>
services, List<RetainedResult> resultats,
                              List<EvidenceView> preuves, List<TimestampView>
horodatages) {}

    public String fingerprint(Composition c) {
        StringBuilder sb = new StringBuilder("gimac-socle-v1\nreferentiel|" +
c.versionId() + "|" + c.versionNumero() + "\n");
        c.services().stream().sorted().forEach(s -> line(sb, "service", s));

c.resultats().stream().sorted(Comparator.comparing(RetainedResult::idScenario)).forEa
ch(r ->
            line(sb, "resultat", r.idScenario(), r.campagneId(), r.executionId(),
r.verdict().name(), r.empreinteReponse()));
        c.preuves().stream().sorted(Comparator.comparing(EvidenceView::operationCode)
                .thenComparing(EvidenceView::documentRef).thenComparingInt(EvidenceVi
ew::version))
            .forEach(p -> line(sb, "preuve", p.operationCode(), p.documentRef(),
String.valueOf(p.version()), p.empreinte()));

c.horodatages().stream().sorted(Comparator.comparing(TimestampView::operationCode)).f
orEach(t ->
            line(sb, "horodatage", t.operationCode(), t.empreinteCloture(),
sha256(Base64.getDecoder().decode(t.jetonBase64()))));
        return sha256(sb.toString().getBytes(StandardCharsets.UTF_8));
    }

    private static void line(StringBuilder sb, String kind, String... f) {
        sb.append(kind);
        for (String v : f) sb.append('|').append(Objects.toString(v, ""));
        sb.append('\n');
    }

    public static String sha256(byte[] data) {
        try { return HexFormat.of().formatHex(MessageDigest.getInstance("SHA-
256").digest(data)); }
        catch (NoSuchAlgorithmException e) { throw new IllegalStateException(e); }
    }
}
```

A3. Détection, numérotation des cycles et émission

L'écouteur de `CoverageRecalculated` (HOM-17) verrouille la liaison (`SELECT ... FOR UPDATE`) pour sérialiser les recalculs concurrents. Le cycle vaut `cycleAptitude + 1` et n'est jamais remis à zéro : un retour en homologation (HOM-19) suivi d'une nouvelle aptitude produit donc un cycle incrémenté, que l'empreinte ait changé ou non. La contrainte `ux_evaluation_base_liaison_cycle (liaison_id, cycle)` complète la déduplication.

```
// application/usecase/DetectAptitudeService.java
@Service
@RequiredArgsConstructor
public class DetectAptitudeService {
    private final AptitudeRule rule = new AptitudeRule();
    private final EvaluationBaseHasher hasher = new EvaluationBaseHasher();
    private final LiaisonRepository liaisons;
    private final ScopeItemRepository scopeItems;
    private final CoverageRepository coverages;
    private final OperationInstanceRepository operations;
    private final EvaluationCompositionReader compositionReader; // résultats
retenus, preuves scellées valides, clôtures actives
    private final EvaluationBaseRepository bases;
    private final OutboxPublisher outboxPublisher;
    private final ObjectMapper mapper;

    @EventListener
    @Transactional(propagation = Propagation.MANDATORY)
    public void on(RecalculateCoverageService.CoverageRecalculated signal) {
        Liaison l = liaisons.findByIdForUpdate(signal.liaisonId()).orElseThrow();
        Set<String> services = scopeItems.findServiceCodesInScope(l.getId());
        if (!rule.isReady(l.getEtat(), services, coverages.findViews(l.getId()),
operations.findAptitudeViews(l.getId()))) return;
        int cycle = l.getCycleAptitude() + 1;
        if (bases.existsByLiaisonIdAndCycle(l.getId(), cycle)) return;          //
déjà émis pour ce cycle
        EvaluationBaseHasher.Composition composition = compositionReader.read(l,
services);
        String empreinte = hasher.fingerprint(composition);
        bases.save(EvaluationBase.of(l, empreinte, cycle, Instant.now(),
mapper.valueToTree(composition).toString()));
        l.markReadyForCertification(cycle);   // EN_HOMOLOGATION ->
APTE_CERTIFICATION (automate de HOM-19)
        outboxPublisher.publish("liaison", l.getId().toString(),
"homologation.liaison-ready-for-certification.v1",
            new LiaisonReadyForCertificationV1(l.getId(), l.getInstitutionId(),
l.getTypeProjet(), cycle, empreinte,
                l.getVersion().getId(), List.copyOf(new TreeSet<>(services))));
    }
}
```

A4. Interfaces internes pour MS-CERTIFICATION

Les deux routes ne sont appelables qu'avec le jeton de service `mscertification-service` portant `INTERNAL_HOMOLOGATION_READ` ; OPA les refuse à tout jeton d'utilisateur. Les intervenants servent à MS-CERTIFICATION pour exclure des signataires ceux qui ont œuvré sur la liaison.

```
// web/rest/InternalLiaisonResource.java
@RestController
@RequestMapping("/api/v1/liaisons/{id}")
@PreAuthorize("hasAuthority('INTERNAL_HOMOLOGATION_READ')")
@RequiredArgsConstructor
public class InternalLiaisonResource {
    private final EvaluationBaseQueryService query;

    @GetMapping("/socle-evaluation")   // dernier cycle par défaut ; 404
HOM_EVALUATION_BASE_NOT_FOUND
    public EvaluationBaseDTO socle(@PathVariable UUID id, @RequestParam(required =
false) Integer cycle) {
        return query.socle(id, cycle);   // (liaisonId, cycle, empreinte, dateCalcul,
composition)
    }

    @GetMapping("/intervenants")
    public IntervenantsDTO intervenants(@PathVariable UUID id) {
        return query.intervenants(id);   // (chefDeProjetId, responsables, deposants,
```

```
validateurs)
    }
}
```

Le schéma JSON de l'événement est versionné dans `src/main/resources/contracts/events/homologation.liaison-ready-for-certification.v1.json` et publié dans l'artefact de contrats partagé avec MS-CERTIFICATION.

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
//
src/test/java/org/gimac/des/homologation/domain/service/EvaluationBaseHasherTest.java
class EvaluationBaseHasherTest {
    private final EvaluationBaseHasher hasher = new EvaluationBaseHasher();
    private final UUID v = UUID.fromString("7d1c5a8e-0000-4000-8000-000000000001");
    private final RetainedResult r1 = new RetainedResult("SC-RET-001", "CMP-1", "EX-
1", FAVORABLE, "ab12");
    private final RetainedResult r2 = new RetainedResult("SC-RET-002", "CMP-1", "EX-
2", FAVORABLE, "cd34");

    @Test
    @DisplayName("HOM-18 : l'empreinte est stable pour un socle inchangé, quel que
soit l'ordre de lecture")
    void empreinteStable() {
        var a = new Composition(v, 3, List.of("RETRAIT_GAB", "PAIEMENT_TPE"),
List.of(r1, r2), List.of(), List.of());
        var b = new Composition(v, 3, List.of("PAIEMENT_TPE", "RETRAIT_GAB"),
List.of(r2, r1), List.of(), List.of());

assertThat(hasher.fingerprint(a)).hasSize(64).isEqualTo(hasher.fingerprint(b));
        var c = new Composition(v, 3, List.of("RETRAIT_GAB", "PAIEMENT_TPE"),
            List.of(r1, new RetainedResult("SC-RET-002", "CMP-1", "EX-3", FAVORABLE,
"ef56")), List.of(), List.of());
        assertThat(hasher.fingerprint(c)).isNotEqualTo(hasher.fingerprint(a));
    }

    @Test
    @DisplayName("HOM-18 : une liaison gelée n'est jamais déclarée apte")
    void liaisonGelee() {
        var cov = List.of(new AptitudeRule.CoverageView("RETRAIT_GAB", CERTIFICATION,
true, new BigDecimal("100.00")));
        assertThat(new AptitudeRule().isReady(EtatLiaison.GELEE,
Set.of("RETRAIT_GAB"), cov, List.of())).isFalse();
        assertThat(new AptitudeRule().isReady(EtatLiaison.EN_HOMOLOGATION,
Set.of("RETRAIT_GAB"), cov, List.of())).isTrue();
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
// src/test/java/org/gimac/des/homologation/application/usecase/DetectAptitudeIT.java
class DetectAptitudeIT extends AbstractHomologationIT {
    @Autowired ObjectMapper mapper;

    @Test
    @DisplayName("HOM-18 : émission unique par cycle et conformité au schéma
partagé")
    void emissionUniqueParCycle() throws Exception {
        UUID id = fixtures.liaisonPretePourAptitude("RETRAIT_GAB");  // couverture
100 %, préalables clos
        for (int i = 0; i < 2; i++) {                                  // deux
recalculs successifs sans changement
            tx.executeWithoutResult(s -> events.publishEvent(new CoverageTrigger(id,
```

```
"RETRAIT_GAB", "CLOTURE")));
        }
        assertThat(outbox("homologation.liaison-ready-for-certification.v1",
id)).isEqualTo(1);
        String payload = jdbc.queryForObject("select payload::text from outbox_event
where event_type = ? and aggregate_id::text = ?",
            String.class, "homologation.liaison-ready-for-certification.v1",
id.toString());
        JsonSchema schema =
JsonSchemaFactory.getInstance(SpecVersion.VersionFlag.V202012).getSchema(
            getClass().getResourceAsStream("/contracts/events/homologation.liaison-
ready-for-certification.v1.json"));
        assertThat(schema.validate(mapper.readTree(payload))).isEmpty();
        assertThat(mapper.readTree(payload).get("cycle").asInt()).isEqualTo(1);
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: HOM-18 Aptitude à la certification et saisine de MS-CERTIFICATION

  Scénario: Nominal : liaison déclarée apte
    Étant donné une liaison dont la couverture de certification atteint cent pour
cent sur chaque service et dont les opérations préalables sont closes
    Quand le dernier recalcul des couvertures s'achève
    Alors la liaison passe à l'état "APTE_CERTIFICATION" et l'empreinte du socle
d'évaluation est calculée
    Et l'événement "homologation.liaison-ready-for-certification.v1" portant
l'empreinte et le cycle 1 est émis

  Scénario: Alternatif : recalcul sans changement
    Étant donné une aptitude déjà signalée pour le cycle 1
    Quand un recalcul intervient sans modification du socle
    Alors aucun nouvel événement d'aptitude n'est émis

  Scénario: Alternatif : nouveau cycle après retour en homologation
    Étant donné une liaison replacée en homologation après un refus de quitus au
cycle 1
    Quand les conditions d'aptitude sont de nouveau réunies
    Alors un événement d'aptitude est émis avec le cycle 2 et l'empreinte du socle

  Scénario: Erreur : liaison gelée
    Étant donné une liaison gelée dont les couvertures atteignent cent pour cent
    Quand le recalcul s'achève
    Alors la liaison n'est pas déclarée apte
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-chef-des
# Nominal : état et cycle d'aptitude de la liaison
curl -s "${GATEWAY_URL}/bff/des/homologation/liaisons/${LIAISON_ID}" -H
"Authorization: Bearer ${TOKEN}" \
  | jq '{etat, cycleAptitude, transitionsPermises}'

# Refus : l'interface interne du socle n'accepte pas un jeton d'utilisateur, attendu
403
curl -s -o /dev/null -w "%{http_code}\n" \
  "${GATEWAY_URL}/bff/des/homologation/liaisons/${LIAISON_ID}/socle-evaluation" -H
"Authorization: Bearer ${TOKEN}"
```

HOM-19 | Refléter dans la liaison les décisions du circuit de certification

Élément	Valeur
Points	5
Priorité	Must Have
Sprint cible	Sprint 4
Écrans	DES-09
Dépendances	HOM-11, HOM-18, CER-04, CER-12

Objectif

Consommer de façon idempotente les décisions de MS-CERTIFICATION pour faire évoluer la liaison selon une table de transitions explicite : un quitus défavorable la replace en homologation et rouvre les opérations désignées ; l'autorisation de mise en service puis l'attestation la mènent à « certifiée » puis « close ». Un code désigné inconnu est signalé sans bloquer le traitement.

A. Implémentation

A1. Automate d'états de la liaison

État de départ	Déclencheur	État d'arrivée
`EN_HOMOLOGATION`	`APTITUDE_DETECTEE` (HOM-18)	`APTE_CERTIFICATION`
`APTE_CERTIFICATION`	`QUITUS_DEFAVORABLE`	`EN_HOMOLOGATION`
`APTE_CERTIFICATION`	`AUTORISATION_MISE_EN_SERVICE`	`MISE_EN_SERVICE_AUTORISEE`
`MISE_EN_SERVICE_AUTORISEE`	`ATTESTATION_EMISE`	`CERTIFIEE`
`CERTIFIEE`	`CLOTURE`	`CLOSE`

Les transitions d'ouverture et de gel (HOM-05, HOM-07) s'ajoutent à la même table. `allowed(...)` alimente `transitionsPermises` des réponses.

```
// domain/service/LiaisonStateMachine.java
public final class LiaisonStateMachine {
    public enum Trigger { APTITUDE_DETECTEE, QUITUS_DEFAVORABLE,
AUTORISATION_MISE_EN_SERVICE, ATTESTATION_EMISE, CLOTURE }

    private static final Map<EtatLiaison, Map<Trigger, EtatLiaison>> TABLE = new
EnumMap<>(Map.of(
        EtatLiaison.EN_HOMOLOGATION, Map.of(Trigger.APTITUDE_DETECTEE,
EtatLiaison.APTE_CERTIFICATION),
        EtatLiaison.APTE_CERTIFICATION, Map.of(Trigger.QUITUS_DEFAVORABLE,
EtatLiaison.EN_HOMOLOGATION,
                                               Trigger.AUTORISATION_MISE_EN_SERVICE,
EtatLiaison.MISE_EN_SERVICE_AUTORISEE),
        EtatLiaison.MISE_EN_SERVICE_AUTORISEE, Map.of(Trigger.ATTESTATION_EMISE,
EtatLiaison.CERTIFIEE),
        EtatLiaison.CERTIFIEE, Map.of(Trigger.CLOTURE, EtatLiaison.CLOSE)));

    public EtatLiaison next(EtatLiaison from, Trigger t) {
        EtatLiaison to = TABLE.getOrDefault(from, Map.of()).get(t);
        if (to == null) throw new IllegalLiaisonTransitionException(from, t); // non
rejouable : rejected_record puis DLQ
        return to;
    }

    public Set<Trigger> allowed(EtatLiaison from) { return TABLE.getOrDefault(from,
```

```
Map.of()).keySet(); }
}
```

A2. Consommateur idempotent des trois événements

Un seul groupe écoute les sujets `certification.quitus`, `certification.dossier` et `certification.attestation` ; les autres types multiplexés sont ignorés. L'appel à `handleOnce` inscrit `processed_event` dans la transaction du traitement : une redélivrance est acquittée sans effet.

```
// infrastructure/messaging/consumer/CertificationDecisionConsumer.java
@Component
@RequiredArgsConstructor
public class CertificationDecisionConsumer {
    static final String GROUP = "mshomologation-certification-decisions";
    private final IdempotentConsumer idempotentConsumer;
    private final ApplyCertificationDecisionUseCase useCase;
    private final ObjectMapper mapper;

    @KafkaListener(topics = {"certification.quitus", "certification.dossier",
"certification.attestation"}, groupId = GROUP)
    public void onMessage(CloudEventEnvelope e) {
        switch (e.type()) {
            case "certification.quitus-signed.v1" ->
idempotentConsumer.handleOnce(e.id(), GROUP,
                () -> useCase.onQuitusSigned(mapper.convertValue(e.data(),
QuitusSignedV1.class)));
            case "certification.go-live-authorized.v1" ->
idempotentConsumer.handleOnce(e.id(), GROUP,
                () -> useCase.onGoLiveAuthorized(mapper.convertValue(e.data(),
GoLiveAuthorizedV1.class)));
            case "certification.attestation-issued.v1" ->
idempotentConsumer.handleOnce(e.id(), GROUP,
                () -> useCase.onAttestationIssued(mapper.convertValue(e.data(),
AttestationIssuedV1.class)));
            default -> { }
        }
    }
}
// QuitusSignedV1(circuitId, liaisonId, cycle, nature, decisionFavorable, motif,
List<String> operationsDesignees, socleEmpreinte)
// GoLiveAuthorizedV1(dossierId, liaisonId, dateAutorisation) ;
AttestationIssuedV1(numero, liaisonId, dateEmission)
```

A3. Application des décisions, réouverture groupée et anomalies

La réouverture délègue à `ReopenOperationUseCase` (HOM-11), qui publie `homologation.operation-reopened.v1`. Garde anti-doublon supplémentaire : seule une opération `CLOSE` est rouverte. Les demandes de notification passent par `NotificationRequestPublisher`, posé ici et réutilisé par HOM-21 à HOM-24 (événement `homologation.notification-requested.v1` sur le sujet `homologation.notification`).

```
// application/usecase/support/NotificationRequestPublisher.java
@Component
@RequiredArgsConstructor
public class NotificationRequestPublisher {
    /** Destinataires : utilisateurs nommés, rôles, et institution quand le message
part vers le portail. */
    public record Audience(List<UUID> utilisateurs, List<String> roles, UUID
institutionId) {}
    private final OutboxPublisher outboxPublisher;

    public void request(UUID liaisonId, String modele, Audience audience, Map<String,
```

```
Object> variables) {
        outboxPublisher.publish("notification", liaisonId.toString(),
"homologation.notification-requested.v1",
            new NotificationRequestedV1(UUID.randomUUID(), liaisonId, modele,
audience, variables));
    }
}
```

```
// application/usecase/ApplyCertificationDecisionService.java
@Service
@Transactional
@RequiredArgsConstructor
public class ApplyCertificationDecisionService implements
ApplyCertificationDecisionUseCase {
    private final LiaisonStateMachine machine = new LiaisonStateMachine();
    private final LiaisonRepository liaisons;
    private final OperationInstanceRepository operations;
    private final ReopenOperationUseCase reopen;
    private final OutboxPublisher outboxPublisher;
    private final NotificationRequestPublisher notifications;

    public void onQuitusSigned(QuitusSignedV1 e) {
        if (e.decisionFavorable()) return;                        // un quitus
favorable ne modifie pas la liaison
        Liaison l = liaisons.findByIdForUpdate(e.liaisonId()).orElseThrow();
        l.setEtat(machine.next(l.getEtat(), Trigger.QUITUS_DEFAVORABLE));
        l.setMotifEtat(e.motif());
        Map<String, OperationInstance> byCode =
operations.findByLiaisonId(l.getId()).stream()
            .collect(Collectors.toMap(OperationInstance::getCode,
Function.identity()));
        List<String> rouvertes = new ArrayList<>(), inconnues = new ArrayList<>();
        for (String code : new TreeSet<>(e.operationsDesignees())) {
            OperationInstance op = byCode.get(code);
            if (op == null) { inconnues.add(code); continue; }
            if (op.getEtat() == EtatOperation.CLOSE) { reopen.reopen(op.getId(),
e.motif()); rouvertes.add(code); }
        }
        outboxPublisher.publish("liaison", l.getId().toString(),
"homologation.liaison-returned.v1",
            new LiaisonReturnedV1(l.getId(), l.getInstitutionId(), e.cycle(),
e.nature(), e.motif(), rouvertes, inconnues));
        notifications.request(l.getId(), "HOM_LIAISON_RETOURNEE",
            new Audience(List.of(l.getChefDeProjetId()),
List.of("CORRESPONDANT_INSTITUTION"), l.getInstitutionId()),
            Map.of("motif", e.motif(), "operations", rouvertes));
        if (!inconnues.isEmpty()) {                                // anomalie
signalée, traitement non bloqué
            notifications.request(l.getId(), "HOM_DESIGNATION_INCONNUE",
                new Audience(List.of(), List.of("INGENIEUR_HOMOLOGATION"), null),
Map.of("codes", inconnues, "circuitId", e.circuitId()));
        }
    }

    public void onGoLiveAuthorized(GoLiveAuthorizedV1 e) {
        Liaison l = liaisons.findByIdForUpdate(e.liaisonId()).orElseThrow();
        l.setEtat(machine.next(l.getEtat(), Trigger.AUTORISATION_MISE_EN_SERVICE));
    }

    public void onAttestationIssued(AttestationIssuedV1 e) {
        Liaison l = liaisons.findByIdForUpdate(e.liaisonId()).orElseThrow();
        l.setEtat(machine.next(l.getEtat(), Trigger.ATTESTATION_EMISE));   //
CERTIFIEE
        l.setEtat(machine.next(l.getEtat(), Trigger.CLOTURE));             // puis
CLOSE, même transaction
        l.setMotifEtat("Attestation " + e.numero());
    }
}
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
//
src/test/java/org/gimac/des/homologation/application/usecase/ApplyCertificationDecisi
onServiceTest.java
@ExtendWith(MockitoExtension.class)
class ApplyCertificationDecisionServiceTest {
    @Mock LiaisonRepository liaisons; @Mock OperationInstanceRepository operations;
    @Mock ReopenOperationUseCase reopen; @Mock OutboxPublisher outbox; @Mock
NotificationRequestPublisher notifications;
    @InjectMocks ApplyCertificationDecisionService service;

    @Test
    @DisplayName("HOM-19 : transitions de la liaison selon la décision reçue")
    void transitions() {
        var m = new LiaisonStateMachine();
        assertThat(m.next(APTE_CERTIFICATION,
QUITUS_DEFAVORABLE)).isEqualTo(EN_HOMOLOGATION);
        assertThat(m.next(m.next(APTE_CERTIFICATION, AUTORISATION_MISE_EN_SERVICE),
ATTESTATION_EMISE)).isEqualTo(CERTIFIEE);
        assertThatThrownBy(() -> m.next(EN_HOMOLOGATION,
ATTESTATION_EMISE)).isInstanceOf(IllegalLiaisonTransitionException.class);
    }

    @Test
    @DisplayName("HOM-19 : un code désigné inconnu est signalé sans bloquer la
réouverture des opérations connues")
    void codeInconnu() {
        Liaison l = LiaisonTestData.liaison(APTE_CERTIFICATION);
        OperationInstance op = LiaisonTestData.operation(l, "OP-BR-12",
EtatOperation.CLOSE);
        when(liaisons.findByIdForUpdate(l.getId())).thenReturn(Optional.of(l));
        when(operations.findByLiaisonId(l.getId())).thenReturn(List.of(op));
        service.onQuitusSigned(new QuitusSignedV1(UUID.randomUUID(), l.getId(), 1,
"SECURITE", false,
            "Journalisation incomplète", List.of("OP-BR-12", "OP-XX-99"), "ab12"));
        assertThat(l.getEtat()).isEqualTo(EN_HOMOLOGATION);
        verify(reopen).reopen(op.getId(), "Journalisation incomplète");
        verify(notifications).request(eq(l.getId()), eq("HOM_DESIGNATION_INCONNUE"),
any(), argThat(v -> v.get("codes").equals(List.of("OP-XX-99"))));
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
// src/test/java/org/gimac/des/homologation/infrastructure/messaging/consumer/
CertificationDecisionConsumerIT.java
class CertificationDecisionConsumerIT extends AbstractHomologationIT {
    @Autowired KafkaTemplate<String, String> kafkaTemplate;

    @Test
    @DisplayName("HOM-19 : la redélivrance d'une décision défavorable ne rouvre pas
deux fois")
    void redelivranceSansDoubleReouverture() {
        UUID id = fixtures.liaisonApteAvecOperationsCloses("OP-BR-12", "OP-BR-14");
        String event = fixtures.cloudEvent("certification.quitus-signed.v1",
UUID.randomUUID(), Map.of(
            "circuitId", UUID.randomUUID(), "liaisonId", id, "cycle", 1, "nature",
"TECHNIQUE", "decisionFavorable", false,
            "motif", "Trace de compensation absente", "operationsDesignees",
List.of("OP-BR-12", "OP-BR-14"), "socleEmpreinte", "ab12"));
        fixtures.assertConformeAuSchema("certification.quitus-signed.v1",
event);   // contrat MS-CERTIFICATION
        kafkaTemplate.send("certification.quitus", id.toString(), event);
        kafkaTemplate.send("certification.quitus", id.toString(),
event);            // même identifiant CloudEvents
        await().atMost(Duration.ofSeconds(15)).untilAsserted(() -> {
```

```
            assertThat(outbox("homologation.liaison-returned.v1", id)).isEqualTo(1);
            assertThat(jdbc.queryForObject("select count(*) from outbox_event where
event_type = 'homologation.operation-reopened.v1' "
                + "and payload->>'liaisonId' = ?", Long.class,
id.toString())).isEqualTo(2);
        });
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: HOM-19 Décisions du circuit de certification reportées sur la liaison

  Scénario: Nominal : retour en homologation après refus
    Étant donné une liaison apte à la certification
    Quand l'événement "certification.quitus-signed.v1" portant une décision
défavorable, un motif et deux opérations désignées est consommé
    Alors la liaison revient à l'état "EN_HOMOLOGATION" avec le motif du refus
    Et les deux opérations sont rouvertes avec ce motif, le chef de projet et les
correspondants sont notifiés et l'événement "homologation.liaison-returned.v1" est
émis

  Scénario: Alternatif : autorisation puis attestation
    Étant donné une liaison apte à la certification
    Quand les événements "certification.go-live-authorized.v1" puis
"certification.attestation-issued.v1" sont consommés
    Alors la liaison passe successivement aux états "MISE_EN_SERVICE_AUTORISEE" et
"CERTIFIEE", puis "CLOSE"

  Scénario: Erreur : opération désignée inconnue
    Étant donné une décision défavorable désignant le code "OP-XX-99" absent de la
liaison
    Quand l'événement est consommé
    Alors la liaison est replacée en homologation et les opérations connues sont
rouvertes
    Et le code "OP-XX-99" est signalé en anomalie à l'ingénieur d'homologation
```

E. Tests cURL via la passerelle

```
# Précondition : refus du quitus technique signé en recette (adaptateur de
réauthentification de substitution)
source ./scripts/token.sh test-chef-des
curl -s -X POST "${GATEWAY_URL}/bff/des/certification/circuits/${CIRCUIT_ID}/quitus/
technique" \
  -H "Authorization: Bearer ${TOKEN}" -H "X-Reauth-Assertion: ${REAUTH_ASSERTION}" -H
"Content-Type: application/json" \
  -d '{"decisionFavorable":false,"motif":"Trace de compensation
absente","operationsDesignees":["OP-BR-12","OP-BR-14"]}' | jq .etat

# Nominal : liaison revenue en homologation, opérations rouvertes
source ./scripts/token.sh test-ingenieur-homologation
curl -s "${GATEWAY_URL}/bff/des/homologation/liaisons/${LIAISON_ID}" -H
"Authorization: Bearer ${TOKEN}" | jq '{etat, motifEtat}'
curl -s "${GATEWAY_URL}/bff/des/homologation/liaisons/${LIAISON_ID}/operations?
etat=ROUVERTE" \
  -H "Authorization: Bearer ${TOKEN}" | jq '[.[].code]'

# Refus : un correspondant ne lit pas la liaison par la route du DES, attendu 403
source ./scripts/token.sh test-correspondant-a
curl -s -o /dev/null -w "%{http_code}\n"
"${GATEWAY_URL}/bff/des/homologation/liaisons/${LIAISON_ID}" -H "Authorization:
Bearer ${TOKEN}"
```

HOM-20 | Piloter les liaisons dans un tableau de bord

Élément	Valeur
Points	5
Priorité	Should Have
Sprint cible	Sprint 4
Écrans	DES-08, DES-09
Dépendances	HOM-07, HOM-17, IHM-06, IHM-05, GW-11

Objectif

Servir `GET /tableau-de-bord/liaisons` (filtré, paginé) à partir d'une projection de lecture donnant pour chaque liaison sa phase courante, le délai consommé et le délai standard de chaque phase, ses couvertures et ses opérations en retard avec la partie qui en répond, et `GET /liaisons/{id}/historique` pour le tiroir de détail ; chaque collaborateur ne voit que son périmètre.

A. Implémentation

A1. Calcul des délais par phase et des retards

Choix retenus : les phases sont ordonnées selon le rang topologique de leur première opération ; la phase 1 commence à l'ouverture de la liaison, chaque phase suivante à la date de clôture la plus tardive de la précédente ; la phase courante est la première non achevée. Les délais standards (en jours par phase) sont lus de `gimac.homologation.delais-standards`. La date de clôture est la `dateEtat` d'une opération `CLOSE`, convertie dans le fuseau `Africa/Douala`.

```
// domain/service/PhaseDelayCalculator.java
public final class PhaseDelayCalculator {
    public record OperationTimeline(String code, String phase, int rang,
EtatOperation etat, Partie partie,
                                    LocalDate dateCloture, LocalDate echeance) {}
    public record PhaseDelay(String phase, long joursConsommes, Integer
joursStandard, boolean depasse, boolean achevee) {}
    public record Late(String code, Partie partie, long joursDepassement) {}

    public List<PhaseDelay> delays(LocalDate ouverture, List<OperationTimeline> ops,
Map<String, Integer> std, LocalDate today) {
        Map<String, List<OperationTimeline>> byPhase = ops.stream().filter(o ->
o.etat() != EtatOperation.HORS_PERIMETRE)
            .sorted(Comparator.comparingInt(OperationTimeline::rang))
            .collect(Collectors.groupingBy(OperationTimeline::phase,
LinkedHashMap::new, Collectors.toList()));
        List<PhaseDelay> out = new ArrayList<>();
        LocalDate debut = ouverture;
        for (var e : byPhase.entrySet()) {
            boolean achevee = e.getValue().stream().allMatch(o -> o.etat() ==
EtatOperation.CLOSE);
            LocalDate fin = achevee ?
e.getValue().stream().map(OperationTimeline::dateCloture).max(Comparator.naturalOrder
()).orElse(debut) : today;
            long jours = Math.max(0, ChronoUnit.DAYS.between(debut, fin));
            Integer s = std.get(e.getKey());
            out.add(new PhaseDelay(e.getKey(), jours, s, s != null && jours > s,
achevee));
```

```
            if (!achevee) break;                                    // phases
suivantes non commencées
            debut = fin;
        }
        return out;
    }

    public List<Late> late(List<OperationTimeline> ops, LocalDate today) {
        return ops.stream()
            .filter(o -> o.etat() != EtatOperation.CLOSE && o.etat() !=
EtatOperation.HORS_PERIMETRE
                    && o.echeance() != null && o.echeance().isBefore(today))
            .map(o -> new Late(o.code(), o.partie(),
ChronoUnit.DAYS.between(o.echeance(), today)))
            .sorted(Comparator.comparingLong(Late::joursDepassement).reversed()).toLi
st();
    }
}
```

A2. Projection de lecture (vue SQL)

La vue agrège une ligne par liaison ; les sous-requêtes évitent la multiplication des lignes entre couvertures et opérations. Elle est mappée par une entité `@Immutable` et interrogée par `JpaSpecificationExecutor`.

```
-- config/liquibase/changelog/20261012000000_vue_tableau_de_bord.sql
--liquibase formatted sql
--changeset gimac-des:hom-20-vue-tableau-de-bord runOnChange:true
create or replace view v_tableau_bord_liaison as
select l.id, l.institution_id, l.institution_nom, l.type_projet, l.etat,
l.chef_de_projet_id, l.date_ouverture,
  (select round(avg(c.taux), 2) from coverage c where c.liaison_id = l.id and
c.nature = 'CERTIFICATION' and c.calculable) as couverture_certification,
  (select round(avg(c.taux), 2) from coverage c where c.liaison_id = l.id and
c.nature = 'MISE_EN_SERVICE' and c.calculable) as couverture_mise_en_service,
  (select count(*) from coverage c where c.liaison_id = l.id and not c.calculable) as
couvertures_non_calculables,
  (select count(*) from operation_instance o where o.liaison_id = l.id and o.etat not
in ('CLOSE', 'HORS_PERIMETRE')
      and o.echeance < (now() at time zone 'Africa/Douala')::date) as nb_retards
from liaison l;
```

A3. Cas d'utilisation, contrôle de périmètre et ressources

Choix de périmètre : `CHEF_DES` et `AUDITEUR` voient toutes les liaisons ; tout autre détenteur de `HOM_DASHBOARD_READ` ne voit que celles dont il est chef de projet. Les délais par phase ne sont calculés que pour la page servie (une requête groupée des opérations). Taille de page bornée à 100 ; total dans l'en-tête `X-Total-Count`. Le tiroir combine `/liaisons/{id}/graphe`, `/liaisons/{id}/perimetre` (existants) et `/liaisons/{id}/historique`, lu dans `outbox_event` (événements dont la charge porte `liaisonId`), du plus récent au plus ancien.

```
// application/usecase/DashboardQueryService.java
@Service
@Transactional(readOnly = true)
@RequiredArgsConstructor
public class DashboardQueryService {
    private final PhaseDelayCalculator calculator = new PhaseDelayCalculator();
    private final LiaisonDashboardRepository rows;
    private final OperationInstanceRepository operations;
    private final DelaisStandardsProperties standards;
    private final Clock clock;
```

```
    public Page<DashboardLiaisonDTO> search(DashboardFilter f, Pageable p,
CurrentUser user) {
        Specification<LiaisonDashboardRow> spec =
Specification.where(eq("institutionId", f.institutionId()))
            .and(eq("typeProjet", f.typeProjet())).and(eq("chefDeProjetId",
f.chefDeProjetId()))
            .and(eq("etat", f.etat())).and(scope(user));
        Page<LiaisonDashboardRow> page = rows.findAll(spec, p);
        Map<UUID, List<OperationTimeline>> t =
operations.findTimelines(page.map(LiaisonDashboardRow::getId).toList());
        LocalDate today = LocalDate.now(clock);
        return page.map(r -> {
            var ops = t.getOrDefault(r.getId(), List.of());
            var d = calculator.delays(r.getDateOuverture(), ops,
standards.parPhase(), today);
            String phase = d.stream().filter(x -> !x.achevee()).findFirst().or(() ->
d.stream().reduce((a, b) -> b))
                .map(PhaseDelay::phase).orElse(null);
            return DashboardLiaisonDTO.of(r, phase, d, calculator.late(ops, today),
List.of("VOIR_DETAIL"));
        });
    }

    private Specification<LiaisonDashboardRow> scope(CurrentUser u) {
        if (u.hasRole("CHEF_DES") || u.hasRole("AUDITEUR")) return null;          //
vue complète
        return (root, q, cb) -> cb.equal(root.get("chefDeProjetId"), u.id());
    }

    private static <T> Specification<LiaisonDashboardRow> eq(String attr, T value) {
        return value == null ? null : (root, q, cb) -> cb.equal(root.get(attr),
value);
    }
}

// web/rest/DashboardResource.java
@GetMapping("/api/v1/tableau-de-bord/liaisons")
@PreAuthorize("hasAuthority('HOM_DASHBOARD_READ')")
public ResponseEntity<List<DashboardLiaisonDTO>> list(DashboardFilter filter,
        @PageableDefault(size = 20, sort = "dateOuverture", direction =
Sort.Direction.DESC) Pageable pageable) {
    Page<DashboardLiaisonDTO> page = query.search(filter,
PageRequest.of(pageable.getPageNumber(),
        Math.min(pageable.getPageSize(), 100), pageable.getSort()),
currentUser.get());
    return ResponseEntity.ok().headers(PaginationUtil.generatePaginationHttpHeaders(
        ServletUriComponentsBuilder.fromCurrentRequest(),
page)).body(page.getContent());
}
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
//
src/test/java/org/gimac/des/homologation/domain/service/PhaseDelayCalculatorTest.java
class PhaseDelayCalculatorTest {
    private final PhaseDelayCalculator calc = new PhaseDelayCalculator();
    private static final LocalDate OUV = LocalDate.of(2026, 9, 1), AUJ =
LocalDate.of(2026, 10, 1);

    @Test
    @DisplayName("HOM-20 : délais par phase chaînés et dépassement du délai
standard")
    void delaisParPhase() {
        var ops = List.of(
            new OperationTimeline("OP-AMT-01", "CADRAGE", 1, CLOSE, Partie.DES,
LocalDate.of(2026, 9, 11), null),
            new OperationTimeline("OP-BR-02", "ESSAIS", 2, ELIGIBLE,
```

```
Partie.PARTICIPANT, null, LocalDate.of(2026, 9, 25)));
        var d = calc.delays(OUV, ops, Map.of("CADRAGE", 15, "ESSAIS", 15), AUJ);
        assertThat(d).extracting(PhaseDelay::phase, PhaseDelay::joursConsommes,
PhaseDelay::depasse, PhaseDelay::achevee)
            .containsExactly(tuple("CADRAGE", 10L, false, true), tuple("ESSAIS", 20L,
true, false));
    }

    @Test
    @DisplayName("HOM-20 : une opération non close échue est en retard avec sa partie
et ses jours de dépassement")
    void retards() {
        var ops = List.of(new OperationTimeline("OP-BR-02", "ESSAIS", 2, ELIGIBLE,
Partie.PARTICIPANT, null, LocalDate.of(2026, 9, 25)),
                          new OperationTimeline("OP-BR-03", "ESSAIS", 3, CLOSE,
Partie.DES, AUJ, LocalDate.of(2026, 9, 20)));
        assertThat(calc.late(ops, AUJ)).containsExactly(new Late("OP-BR-02",
Partie.PARTICIPANT, 6));
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
// src/test/java/org/gimac/des/homologation/web/rest/DashboardResourceIT.java
class DashboardResourceIT extends AbstractHomologationIT {
    @Test
    @DisplayName("HOM-20 : filtres, pagination et restriction au périmètre du
collaborateur")
    void filtresEtPerimetre() throws Exception {
        UUID ingenieur = UUID.randomUUID();
        fixtures.liaison("Établissement de paiement A", TypeProjet.PAIEMENT_MOBILE,
ingenieur);
        fixtures.liaison("Banque B", TypeProjet.CARTE_INTERBANCAIRE,
UUID.randomUUID());
        mockMvc.perform(get("/api/v1/tableau-de-bord/liaisons").param("typeProjet",
"PAIEMENT_MOBILE").param("size", "10")
                .with(jwt().jwt(j ->
j.subject(UUID.randomUUID().toString()).claim("roles", List.of("CHEF_DES")))
                    .authorities(new SimpleGrantedAuthority("HOM_DASHBOARD_READ"))))
            .andExpect(status().isOk()).andExpect(header().string("X-Total-Count",
"1"))
            .andExpect(jsonPath("$[0].institutionNom").value("Établissement de
paiement A"))
            .andExpect(jsonPath("$[0].phasesDelais").isArray());
        mockMvc.perform(get("/api/v1/tableau-de-bord/liaisons")
                .with(jwt().jwt(j -> j.subject(ingenieur.toString()).claim("roles",
List.of("INGENIEUR_HOMOLOGATION")))
                    .authorities(new SimpleGrantedAuthority("HOM_DASHBOARD_READ"))))
            .andExpect(header().string("X-Total-Count", "1"))
            .andExpect(jsonPath("$[0].chefDeProjetId").value(ingenieur.toString()));
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: HOM-20 Tableau de bord des liaisons

  Scénario: Nominal : vue consolidée
    Étant donné des liaisons ouvertes pour "Établissement de paiement A" et "Banque
B"
    Quand le chef du DES ouvre le tableau de bord
    Alors chaque liaison affiche sa phase courante, le délai consommé et le délai
standard de chaque phase, ses couvertures et le nombre d'opérations en retard
    Et le tableau est filtrable par institution, type de projet, chef de projet et
```

```
état

  Scénario: Alternatif : opérations en retard
    Étant donné une opération non close dont l'échéance est dépassée de 6 jours et
qui incombe au participant
    Quand le tableau est affiché
    Alors l'opération apparaît en retard de 6 jours avec la partie "PARTICIPANT"

  Scénario: Alternatif : détail d'une liaison
    Étant donné le tableau de bord affiché
    Quand l'utilisateur sélectionne une liaison
    Alors le graphe des opérations, le périmètre et l'historique des événements de la
liaison sont servis
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-chef-des
# Nominal : liaisons de paiement mobile en homologation, page 0
curl -s -D /tmp/entetes.txt
"${GATEWAY_URL}/bff/des/homologation/tableau-de-bord/liaisons?
typeProjet=PAIEMENT_MOBILE&etat=EN_HOMOLOGATION&page=0&size=20" \
  -H "Authorization: Bearer ${TOKEN}" | jq '.[] | {institutionNom, phaseCourante,
couvertureCertification, retards: [.retards[] | {code, partie, joursDepassement}]}'
grep -i x-total-count /tmp/entetes.txt
# Tiroir : historique de la liaison
curl -s "${GATEWAY_URL}/bff/des/homologation/liaisons/${LIAISON_ID}/historique?
size=10" -H "Authorization: Bearer ${TOKEN}" | jq '.[].type'

# Refus : un correspondant n'a pas accès au tableau de bord, attendu 403
source ./scripts/token.sh test-correspondant-a
curl -s -o /dev/null -w "%{http_code}\n"
"${GATEWAY_URL}/bff/des/homologation/tableau-de-bord/liaisons" -H "Authorization:
Bearer ${TOKEN}"
```

HOM-21 | Notifier les échéances des opérations

Élément	Valeur
Points	3
Priorité	Should Have
Sprint cible	Sprint 5
Écrans	Aucun (traitement planifié ; paramètres par `PUT /parametres/echeances`)
Dépendances	HOM-08, NOT-14

Objectif

Chaque jour, détecter les opérations non closes dont l'échéance approche, est atteinte ou est dépassée et émettre par l'outbox `homologation.notification-requested.v1` vers le responsable et le chef de projet, une seule fois par couple opération et seuil, même si le passage est relancé.

A. Implémentation

A1. Seuils paramétrables et sélection

Les seuils sont conservés dans une table à ligne unique `parametre_echeance` (entité `DeadlineSettings` : `joursAvantEcheance`, défaut 5 ; `periodeRetardJours`, défaut 7), modifiée par `PUT /parametres/echeances` (permission `HOM_DEADLINE_CONFIG`, bornes 1 à 30, sinon 400). Chaque seuil a un libellé stable qui sert de clé d'unicité dans `DeadlineNotification.seuil` : `J-5`, `J0`, puis `J+7`, `J+14`, ... pour le retard récurrent. La sélection rattrape un passage manqué : une opération à J-3 non encore notifiée reçoit le seuil `J-5`.

```
// domain/service/DeadlineSelector.java
public final class DeadlineSelector {
    public record Settings(int joursAvantEcheance, int periodeRetardJours) {}

    /** Seuil atteint à la date du jour, ou vide si l'échéance est encore lointaine.
*/
    public Optional<String> select(LocalDate echeance, LocalDate today, Settings s) {
        long jours = ChronoUnit.DAYS.between(today, echeance);        // positif :
avant l'échéance
        if (jours > s.joursAvantEcheance()) return Optional.empty();
        if (jours > 0) return Optional.of("J-" + s.joursAvantEcheance());
        long retard = -jours;
        long rang = retard / s.periodeRetardJours();
        return Optional.of(rang == 0 ? "J0" : "J+" + rang * s.periodeRetardJours());
    }
}
```

A2. Passage quotidien idempotent

Les candidates sont les opérations `ELIGIBLE`, `PREUVE_DEPOSEE`, `VALIDEE_HORODATAGE_EN_ATTENTE` ou `ROUVERTE`, d'une liaison non `GELEE`, dont l'échéance est antérieure à l'horizon ; une opération close ou d'une liaison gelée n'est donc jamais sélectionnée. Chaque opération est traitée dans sa propre transaction : l'inscription de `DeadlineNotification` et l'écriture de l'outbox sont validées ensemble, et la contrainte d'unicité (operation_id, seuil) protège aussi contre deux réplicas simultanés.

```
// infrastructure/scheduling/DeadlineNotificationJob.java
@Component
@RequiredArgsConstructor
public class DeadlineNotificationJob {
    private static final Set<EtatOperation> OUVERTES = EnumSet.of(ELIGIBLE,
PREUVE_DEPOSEE, VALIDEE_HORODATAGE_EN_ATTENTE, ROUVERTE);
    private final DeadlineSelector selector = new DeadlineSelector();
    private final OperationInstanceRepository operations;
    private final DeadlineSettingsRepository settings;
    private final DeadlineNotificationRepository sent;
    private final RecipientResolver recipients;
    private final NotificationRequestPublisher notifications;
    private final TransactionTemplate tx;
    private final Clock clock;

    @Scheduled(cron = "${gimac.homologation.echeances.cron:0 10 6 * * *}", zone =
"Africa/Douala")
    public void run() { runFor(LocalDate.now(clock)); }

    public int runFor(LocalDate today) {
        DeadlineSelector.Settings s = settings.current().toSettings();
        int emitted = 0;
        for (DeadlineCandidate c : operations.findDeadlineCandidates(OUVERTES,
EtatLiaison.GELEE, today.plusDays(s.joursAvantEcheance()))) {
            Optional<String> seuil = selector.select(c.echeance(), today, s);
            if (seuil.isEmpty()) continue;
            try {
                if (Boolean.TRUE.equals(tx.execute(st -> emitOnce(c, seuil.get()))))
emitted++;
            } catch (DataIntegrityViolationException doublon) { /* couple déjà traité
par un autre réplica */ }
        }
        return emitted;
    }

    private boolean emitOnce(DeadlineCandidate c, String seuil) {
        if (sent.existsByOperationIdAndSeuil(c.operationId(), seuil)) return false;
// relance : sans effet
        sent.saveAndFlush(new DeadlineNotification(c.operationId(), seuil,
Instant.now(clock)));
        notifications.request(c.liaisonId(), "HOM_ECHEANCE_OPERATION",
recipients.resolve(c),
            Map.of("operation", c.code(), "echeance", c.echeance().toString(),
"seuil", seuil));
        return true;
    }
}
```

A3. Résolution du destinataire selon la partie

Le responsable actif (`Assignment.actif`) et le chef de projet sont toujours destinataires. Pour une opération de la partie `PARTICIPANT`, l'institution est jointe pour que MS-NOTIFICATION emprunte le canal du portail ; en l'absence de responsable désigné, le message va aux correspondants de l'institution.

```
// application/usecase/RecipientResolver.java
@Component
public class RecipientResolver {
    public NotificationRequestPublisher.Audience resolve(DeadlineCandidate c) {
        List<UUID> users = Stream.of(c.responsableId(),
c.chefDeProjetId()).filter(Objects::nonNull).distinct().toList();
        boolean participant = c.partie() == Partie.PARTICIPANT;
        List<String> roles = participant && c.responsableId() == null ?
List.of("CORRESPONDANT_INSTITUTION") : List.of();
        return new NotificationRequestPublisher.Audience(users, roles, participant ?
c.institutionId() : null);
    }
```

```
}
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
// src/test/java/org/gimac/des/homologation/domain/service/DeadlineSelectorTest.java
class DeadlineSelectorTest {
    private final DeadlineSelector selector = new DeadlineSelector();
    private final DeadlineSelector.Settings s = new DeadlineSelector.Settings(5, 7);
    private static final LocalDate AUJ = LocalDate.of(2026, 10, 12);

    @ParameterizedTest(name = "échéance à J{0} : seuil {1}")
    @CsvSource({"6,", "5,J-5", "3,J-5", "0,J0", "-6,J0", "-7,J+7", "-15,J+14"})
    @DisplayName("HOM-21 : sélection des opérations par seuil")
    void selection(int decalage, String attendu) {
        assertThat(selector.select(AUJ.plusDays(decalage), AUJ,
s)).isEqualTo(Optional.ofNullable(attendu));
    }
}

// src/test/java/org/gimac/des/homologation/infrastructure/scheduling/
DeadlineNotificationJobTest.java
@ExtendWith(MockitoExtension.class)
class DeadlineNotificationJobTest {
    @Mock OperationInstanceRepository operations; @Mock DeadlineSettingsRepository
settings;
    @Mock DeadlineNotificationRepository sent; @Mock RecipientResolver recipients;
    @Mock NotificationRequestPublisher notifications;
    @Spy TransactionTemplate tx = new TransactionTemplate(new
PseudoTransactionManager());
    @Spy Clock clock = Clock.fixed(Instant.parse("2026-10-12T05:10:00Z"),
ZoneId.of("Africa/Douala"));
    @InjectMocks DeadlineNotificationJob job;

    @Test
    @DisplayName("HOM-21 : un couple opération et seuil déjà notifié n'est pas
réémis")
    void pasDeDoublon() {
        var c = DeadlineTestData.candidate(LocalDate.of(2026, 10, 17), Partie.DES);
        when(settings.current()).thenReturn(DeadlineSettings.defaults());
        when(operations.findDeadlineCandidates(any(), any(),
any())).thenReturn(List.of(c));
        when(sent.existsByOperationIdAndSeuil(c.operationId(), "J-
5")).thenReturn(true);
        assertThat(job.runFor(LocalDate.of(2026, 10, 12))).isZero();
        verifyNoInteractions(notifications);
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
// src/test/java/org/gimac/des/homologation/infrastructure/scheduling/
DeadlineNotificationJobIT.java
@TestPropertySource(properties = "gimac.homologation.echeances.cron=-")   //
planification désactivée en test
class DeadlineNotificationJobIT extends AbstractHomologationIT {
    @Autowired DeadlineNotificationJob job;

    @Test
    @DisplayName("HOM-21 : la relance du passage ne produit aucun doublon et ignore
les liaisons gelées")
    void relanceSansDoublon() {
        LocalDate auj = LocalDate.of(2026, 10, 12);
        UUID active =
fixtures.liaisonAvecOperationEcheance(EtatLiaison.EN_HOMOLOGATION, auj.plusDays(5));
        UUID gelee = fixtures.liaisonAvecOperationEcheance(EtatLiaison.GELEE,
```

```
auj.plusDays(5));
        assertThat(job.runFor(auj)).isEqualTo(1);
        assertThat(job.runFor(auj)).isZero();                                    //
relance après incident
        assertThat(outbox("homologation.notification-requested.v1",
active)).isEqualTo(1);
        assertThat(outbox("homologation.notification-requested.v1", gelee)).isZero();
        assertThat(jdbc.queryForObject("select count(*) from deadline_notification
where seuil = 'J-5'", Long.class)).isEqualTo(1);
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: HOM-21 Notification des échéances des opérations

  Scénario: Nominal : échéance proche
    Étant donné le seuil avant échéance paramétré à 5 jours
    Et une opération éligible dont l'échéance tombe dans 5 jours
    Quand le passage quotidien s'exécute
    Alors l'événement "homologation.notification-requested.v1" est émis vers le
responsable et le chef de projet avec le seuil "J-5"

  Scénario: Alternatif : opération close ou liaison gelée
    Étant donné une opération close avant son échéance et une opération d'une liaison
gelée
    Quand le passage quotidien s'exécute
    Alors aucune notification n'est émise

  Scénario: Erreur : double exécution du passage
    Étant donné un passage quotidien déjà exécuté pour la date
    Quand il est relancé après un incident
    Alors aucune notification n'est émise en double pour un même couple opération et
seuil
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-chef-des
# Nominal : paramétrage des seuils
curl -s -X PUT "${GATEWAY_URL}/bff/des/homologation/parametres/echeances" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" \
  -d '{"joursAvantEcheance":5,"periodeRetardJours":7}' | jq '{joursAvantEcheance,
periodeRetardJours}'

# Refus : valeur hors bornes, attendu 400
curl -s -o /dev/null -w "%{http_code}\n" -X PUT
"${GATEWAY_URL}/bff/des/homologation/parametres/echeances" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" -d
'{"joursAvantEcheance":0,"periodeRetardJours":7}'

# Refus : l'ingénieur d'homologation ne détient pas HOM_DEADLINE_CONFIG, attendu 403
source ./scripts/token.sh test-ingenieur-homologation
curl -s -o /dev/null -w "%{http_code}\n" -X PUT
"${GATEWAY_URL}/bff/des/homologation/parametres/echeances" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" -d
'{"joursAvantEcheance":3,"periodeRetardJours":7}'
```

HOM-22 | Exporter le dossier de preuves d'une liaison

Élément	Valeur
Points	5
Priorité	Must Have
Sprint cible	Sprint 4
Écrans	DES-09
Dépendances	HOM-10, HOM-13, DOC-05, DOC-09, AUD-01

Objectif

Produire pour une liaison apte un dossier de preuves vérifiable hors plateforme (archive contenant le manifeste, les pièces et l'empreinte du manifeste), après revérification de chaque empreinte, le déposer via MS-DOCUMENT et émettre `homologation.evidence-file-exported.v1` ; servir la même production à MS-CERTIFICATION pour un socle identifié par son empreinte ; interrompre l'export et alerter le RSSI sur toute empreinte divergente.

A. Implémentation

A1. Manifeste du dossier de preuves

Le manifeste est un JSON canonique (clés triées, opérations triées par rang topologique, pièces par `documentRef` puis `version`) ; son empreinte SHA-256 est calculée sur ces octets et écrite aussi dans `manifeste.sha256` à la racine de l'archive, ce qui permet à un auditeur de tout revérifier avec un outil standard. Le composeur `EvidenceManifestComposer` (service de domaine pur) assemble cette structure à partir des opérations, preuves, clôtures actives et résultats repris.

```
{
  "format": "gimac-dossier-preuves/1.0",
  "algorithme": "SHA-256",
  "liaison": { "id": "…", "institutionId": "…", "institutionNom": "Établissement de
paiement A", "typeProjet": "PAIEMENT_MOBILE" },
  "referentiel": { "id": "…", "numero": 3 },
  "socle": { "empreinte": "9f2c…", "cycle": 1 },
  "dateGeneration": "2026-10-14T09:12:00Z",
  "operations": [ {
    "code": "OP-BR-12", "libelle": "…", "segment": "BRANCHE", "partie":
"PARTICIPANT", "serviceCode": "RETRAIT_GAB",
    "pieces": [ { "documentRef": "…", "version": 2, "typePiece": "RAPPORT_ESSAI",
"empreinte": "3ab1…",
                  "deposantId": "…", "dateDepot": "…", "chemin": "pieces/OP-BR-12/v2-
rapport.pdf" } ],
    "cloture": { "validateurId": "…", "dateValidation": "…", "empreinteCloture":
"77c0…",
                 "jetonHorodatage": "MIIG…", "dateHorodatage": "…" },
    "resultats": [ { "campagneId": "…", "executionId": "…", "idScenario": "SC-RET-
001", "verdict": "FAVORABLE",
                     "dateExecution": "…", "referenceTrace": "…", "empreinteReponse":
"…" } ]
  } ]
}
```

A2. Vérification des empreintes et port de lecture des pièces

On ajoute au port `DocumentDepositPort` (HOM-09) la lecture en flux `openContent(documentRef, version)`. L'adaptateur réel appelle MS-DOCUMENT (`GET /api/v1/documents/{ref}/versions/{v}/contenu`, jeton de service) ; l'adaptateur de substitution (`DocumentDepositStubAdapter`, profils `dev` et `test`, `gimac.adapters.document-deposit=stub`) lit un répertoire local. Chaque pièce est hachée en flux pendant sa copie dans l'archive : aucune pièce n'est chargée entièrement en mémoire.

```
// application/usecase/EvidenceArchiveWriter.java
@Component
@RequiredArgsConstructor
public class EvidenceArchiveWriter {
    private final DocumentDepositPort documents;

    /** Écrit l'archive ; lève EvidenceHashMismatchException dès qu'une empreinte
recalculée diffère. */
    public void write(EvidenceManifest manifest, byte[] manifestBytes, String
manifestHash, OutputStream target) throws IOException {
        try (ZipOutputStream zip = new ZipOutputStream(target)) {
            for (var op : manifest.operations()) for (var p : op.pieces()) {
                zip.putNextEntry(new ZipEntry(p.chemin()));
                MessageDigest md = sha256();
                try (InputStream in = new
DigestInputStream(documents.openContent(p.documentRef(), p.version()), md)) {
                    in.transferTo(zip);
                }
                String recalculee = HexFormat.of().formatHex(md.digest());
                if (!recalculee.equals(p.empreinte())) throw new
EvidenceHashMismatchException(op.code(), p.documentRef(), p.empreinte(), recalculee);
                zip.closeEntry();
            }
            zip.putNextEntry(new ZipEntry("manifeste.json"));
zip.write(manifestBytes); zip.closeEntry();
            zip.putNextEntry(new ZipEntry("manifeste.sha256"));
zip.write((manifestHash + "  manifeste.json\n").getBytes(UTF_8)); zip.closeEntry();
        }
    }
}
```

A3. Cas d'utilisation d'export

États admis : `APTE_CERTIFICATION` et suivants (sinon 409 `HOM_LIAISON_NOT_READY`). Si un socle est demandé (appel interne de MS-CERTIFICATION), l'empreinte du socle recalculée par `EvaluationBaseHasher` (HOM-18) doit lui être égale, sinon 409 `HOM_SOCLE_DIVERGENT` ; un export déjà `PRODUIT` pour ce socle est renvoyé tel quel. L'inscription au registre d'audit se fait par l'événement `homologation.evidence-file-exported.v1`, porteur de l'empreinte du manifeste, que MS-AUDIT (AUD-01) chaîne dans son registre. En cas d'empreinte divergente, l'archive temporaire est supprimée, rien n'est déposé, l'échec est journalisé (`EvidenceFileExport.etat = INTERROMPU`) et le RSSI est alerté dans la même transaction validée ; la ressource répond 409 `HOM_EVIDENCE_HASH_MISMATCH`.

```
// application/usecase/ExportEvidenceFileService.java
@Service
@RequiredArgsConstructor
public class ExportEvidenceFileService implements ExportEvidenceFileUseCase {
    private static final Set<EtatLiaison> ADMIS = EnumSet.of(APTE_CERTIFICATION,
MISE_EN_SERVICE_AUTORISEE, CERTIFIEE, CLOSE);
    private final LiaisonRepository liaisons;
    private final EvidenceManifestComposer composer = new EvidenceManifestComposer();
    private final EvidenceSourceReader sources;          // lectures JPA :
opérations, preuves, clôtures actives, résultats
```

```
    private final EvaluationSocleReader socle;           // recalcule empreinte et
cycle courants (HOM-18)
    private final EvidenceArchiveWriter archiveWriter;
    private final DocumentDepositPort documents;
    private final EvidenceFileExportRepository exports;
    private final OutboxPublisher outboxPublisher;
    private final NotificationRequestPublisher notifications;

    @Transactional(noRollbackFor = EvidenceHashMismatchException.class)
    public ExportResult export(UUID liaisonId, String socleDemande, UUID demandeurId)
throws IOException {
        Liaison l = liaisons.findByIdForUpdate(liaisonId).orElseThrow(() -> new
DomainProblemException(NOT_FOUND, "HOM_LIAISON_NOT_FOUND"));
        if (!ADMIS.contains(l.getEtat())) throw new DomainProblemException(CONFLICT,
"HOM_LIAISON_NOT_READY");
        SocleCourant courant = socle.current(l);
        if (socleDemande != null) {
            if (!socleDemande.equals(courant.empreinte())) throw new
DomainProblemException(CONFLICT, "HOM_SOCLE_DIVERGENT");
            Optional<EvidenceFileExport> deja =
exports.findFirstByLiaisonIdAndSocleEmpreinteAndEtat(liaisonId, socleDemande,
"PRODUIT");
            if (deja.isPresent()) return ExportResult.existing(deja.get());
        }
        EvidenceManifest manifest = composer.compose(l, courant,
sources.operations(l), sources.preuves(l),
            sources.clotures(l), sources.resultats(l));
        byte[] bytes = CanonicalJson.write(manifest);
        String hash = EvaluationBaseHasher.sha256(bytes);
        Path tmp = Files.createTempFile("dossier-preuves-", ".zip");
        try {
            try (OutputStream out = Files.newOutputStream(tmp))
{ archiveWriter.write(manifest, bytes, hash, out); }
            String ref = documents.deposit(tmp, "DOSSIER_PREUVES", "application/zip",
l.getId());
            EvidenceFileExport e = exports.save(EvidenceFileExport.produit(l, new
String(bytes, UTF_8), hash, ref, demandeurId, courant.empreinte()));
            outboxPublisher.publish("liaison", l.getId().toString(),
"homologation.evidence-file-exported.v1",
                new EvidenceFileExportedV1(e.getId(), l.getId(),
l.getInstitutionId(), hash, ref, courant.empreinte(), courant.cycle(), demandeurId));
            return ExportResult.created(e);
        } catch (EvidenceHashMismatchException mismatch) {
            exports.save(EvidenceFileExport.interrompu(l, hash, demandeurId,
courant.empreinte()));
            notifications.request(l.getId(), "HOM_ALERTE_EMPREINTE_DIVERGENTE", new
Audience(List.of(), List.of("RSSI"), null),
                Map.of("operation", mismatch.operationCode(), "documentRef",
mismatch.documentRef()));
            throw mismatch;                                  // traduit en 409
HOM_EVIDENCE_HASH_MISMATCH
        } finally {
            Files.deleteIfExists(tmp);
        }
    }
}
```

A4. Ressources

`POST /liaisons/{id}/dossier-de-preuves` accepte `HOM_EVIDENCE_FILE_EXPORT` (utilisateur, corps vide) ou `INTERNAL_HOMOLOGATION_READ` (jeton de service de MS-CERTIFICATION, `socleEmpreinte` obligatoire) ; il répond 201 pour un nouvel export, 200 pour un export existant du même socle, avec `Location: /dossiers-de-preuves/{id}`. `GET /dossiers-de-preuves/{id}` renvoie référence documentaire, empreinte du manifeste, socle et état.

```
// web/rest/EvidenceFileResource.java
@PostMapping("/api/v1/liaisons/{id}/dossier-de-preuves")
@PreAuthorize("hasAnyAuthority('HOM_EVIDENCE_FILE_EXPORT',
'INTERNAL_HOMOLOGATION_READ')")
public ResponseEntity<EvidenceFileDTO> export(@PathVariable UUID id,
@RequestBody(required = false) ExportRequest body,
                                              Authentication auth) throws IOException
{
    boolean interne = auth.getAuthorities().stream().anyMatch(a ->
a.getAuthority().equals("INTERNAL_HOMOLOGATION_READ"));
    String socle = body == null ? null : body.socleEmpreinte();
    if (interne && socle == null) throw new DomainProblemException(BAD_REQUEST,
"HOM_SOCLE_REQUIRED");
    ExportResult r = useCase.export(id, socle, currentUser.get().id());
    return ResponseEntity.status(r.created() ? CREATED : OK)
        .location(URI.create("/api/v1/dossiers-de-preuves/" +
r.export().getId())).body(mapper.toDto(r.export()));
}
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
//
src/test/java/org/gimac/des/homologation/domain/service/EvidenceManifestComposerTest.
java
class EvidenceManifestComposerTest {
    @Test
    @DisplayName("HOM-22 : le manifeste est complet pour une liaison de référence")
    void manifesteComplet() {
        var ref = EvidenceTestData.liaisonDeReference();    // 2 opérations closes, 3
pièces, 2 résultats repris
        EvidenceManifest m = new EvidenceManifestComposer().compose(ref.liaison(),
ref.socle(), ref.operations(),
            ref.preuves(), ref.clotures(), ref.resultats());

assertThat(m.operations()).extracting(EvidenceManifest.Operation::code).containsExact
ly("OP-AMT-01", "OP-BR-12");
        assertThat(m.operations()).allSatisfy(o -> {
            assertThat(o.pieces()).allSatisfy(p ->
assertThat(p.empreinte()).hasSize(64));
            assertThat(o.cloture().jetonHorodatage()).isNotBlank();
            assertThat(o.cloture().validateurId()).isNotNull();
        });

assertThat(m.operations().get(1).resultats()).extracting("referenceTrace").doesNotCon
tainNull();
    }
}

//
src/test/java/org/gimac/des/homologation/application/usecase/EvidenceArchiveWriterTes
t.java
@ExtendWith(MockitoExtension.class)
class EvidenceArchiveWriterTest {
    @Mock DocumentDepositPort documents;
    @InjectMocks EvidenceArchiveWriter writer;

    @Test
    @DisplayName("HOM-22 : une pièce dont l'empreinte recalculée diffère interrompt
l'écriture")
    void empreinteDivergente() {
        var manifest = EvidenceTestData.manifestAvecUnePiece("OP-BR-12", "DOC-1",
"0".repeat(64));
        when(documents.openContent("DOC-1", 1)).thenReturn(new
ByteArrayInputStream("contenu altéré".getBytes(UTF_8)));
        assertThatThrownBy(() -> writer.write(manifest, new byte[0], "h",
OutputStream.nullOutputStream()))
            .isInstanceOf(EvidenceHashMismatchException.class).hasMessageContaining("
```

```
DOC-1");
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
// src/test/java/org/gimac/des/homologation/web/rest/EvidenceFileResourceIT.java
@TestPropertySource(properties = "gimac.adapters.document-deposit=real")
class EvidenceFileResourceIT extends AbstractHomologationIT {
    @RegisterExtension static WireMockExtension msDocument =
WireMockExtension.newInstance().options(wireMockConfig().dynamicPort()).build();
    @DynamicPropertySource static void props(DynamicPropertyRegistry r)
{ r.add("gimac.clients.ms-document.base-url", msDocument::baseUrl); }

    @Test
    @DisplayName("HOM-22 : l'export est interrompu sur empreinte divergente et le
RSSI est alerté")
    void interruptionSurEmpreinteDivergente() throws Exception {
        UUID id = fixtures.liaisonApteAvecPiece("OP-BR-12", "DOC-1",
EvaluationBaseHasher.sha256("original".getBytes(UTF_8)));
        msDocument.stubFor(WireMock.get(urlPathMatching("/api/v1/documents/DOC-1/
versions/1/contenu")).willReturn(ok("altere")));
        mockMvc.perform(MockMvcRequestBuilders.post("/api/v1/liaisons/{id}/dossier-
de-preuves", id)
                .with(jwt().authorities(new
SimpleGrantedAuthority("HOM_EVIDENCE_FILE_EXPORT"))))
            .andExpect(status().isConflict()).andExpect(jsonPath("$.code").value("HOM
_EVIDENCE_HASH_MISMATCH"));
        msDocument.verify(0,
postRequestedFor(urlPathMatching("/api/v1/documents.*")));      // aucun dépôt
        assertThat(outbox("homologation.evidence-file-exported.v1", id)).isZero();
        assertThat(jdbc.queryForObject("select count(*) from outbox_event where
event_type = 'homologation.notification-requested.v1' "
            + "and payload->'audience'->'roles' @> '[\"RSSI\"]'::jsonb",
Long.class)).isEqualTo(1);
        assertThat(jdbc.queryForObject("select etat from evidence_file_export where
liaison_id = ?", String.class, id)).isEqualTo("INTERROMPU");
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: HOM-22 Export du dossier de preuves d'une liaison

  Scénario: Nominal : export du dossier
    Étant donné une liaison apte à la certification
    Quand l'ingénieur d'homologation demande l'export du dossier de preuves
    Alors un manifeste listant chaque opération avec sa preuve, son empreinte, son
jeton d'horodatage, son validateur et les résultats repris est produit avec les
pièces
    Et le dossier est déposé via MS-DOCUMENT, l'empreinte du manifeste est inscrite
au registre d'audit et l'événement "homologation.evidence-file-exported.v1" est émis

  Scénario: Alternatif : demande de MS-CERTIFICATION
    Étant donné une demande interne de MS-CERTIFICATION pour le socle d'évaluation
courant identifié par son empreinte
    Quand l'export est produit
    Alors le dossier porte cette empreinte de socle et sa référence est renvoyée à
l'appelant

  Scénario: Erreur : empreinte divergente
    Étant donné une pièce dont l'empreinte recalculée à l'export diffère de
l'empreinte enregistrée
    Quand l'export s'exécute
```

```
    Alors l'export est interrompu sans produire de dossier
    Et une alerte est émise vers le RSSI
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-ingenieur-homologation
# Nominal : export puis consultation
EXPORT_ID=$(curl -s -X POST "${GATEWAY_URL}/bff/des/homologation/liaisons/$
{LIAISON_ID}/dossier-de-preuves" \
  -H "Authorization: Bearer ${TOKEN}" | jq -r .id)
curl -s "${GATEWAY_URL}/bff/des/homologation/dossiers-de-preuves/${EXPORT_ID}" -H
"Authorization: Bearer ${TOKEN}" \
  | jq '{etat, documentRef, empreinteManifeste, socleEmpreinte}'

# Refus : un correspondant ne peut pas exporter, attendu 403
source ./scripts/token.sh test-correspondant-a
curl -s -o /dev/null -w "%{http_code}\n" -X POST \
  "${GATEWAY_URL}/bff/des/homologation/liaisons/${LIAISON_ID}/dossier-de-preuves" -H
"Authorization: Bearer ${TOKEN}"
```

HOM-23 | Déposer une preuve depuis le portail des participants

Élément	Valeur
Points	5
Priorité	Must Have
Sprint cible	Sprint 4
Écrans	POR-03
Dépendances	HOM-09, DOC-10, GW-13, IHM-07

Objectif

Permettre au correspondant responsable d'une opération de son institution, incombant au participant, de déposer la pièce attendue par `POST /portail/operations/{id}/preuves` ; renvoyer un accusé portant l'empreinte, faire passer l'opération à `PREUVE_DEPOSEE` et notifier l'ingénieur d'homologation. Toute requête hors de l'institution du jeton reçoit 404 et est journalisée vers MS-AUDIT.

A. Implémentation

A1. Contexte du portail issu du jeton

L'institution provient exclusivement du claim `institution_id` du jeton ; aucun paramètre de requête ni champ du corps n'est lu à cette fin. Un jeton de portail sans ce claim est refusé (403 `HOM_PORTAL_NO_INSTITUTION`).

```
// infrastructure/security/PortalContextResolver.java
@Component
public class PortalContextResolver {
    public record PortalContext(UUID userId, UUID institutionId) {}

    public PortalContext current() {
        Jwt jwt = (Jwt)
SecurityContextHolder.getContext().getAuthentication().getPrincipal();
        String institution = jwt.getClaimAsString("institution_id");
        if (institution == null) throw new
DomainProblemException(HttpStatus.FORBIDDEN, "HOM_PORTAL_NO_INSTITUTION");
        return new PortalContext(UUID.fromString(jwt.getSubject()),
UUID.fromString(institution));
    }
}

// infrastructure/security/PortalAccessAuditor.java
@Component
@RequiredArgsConstructor
public class PortalAccessAuditor {
    private final OutboxPublisher outboxPublisher;

    /** Transaction propre : la trace survit à l'échec de la requête ; MS-AUDIT
consomme l'événement. */
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void denied(PortalContext ctx, String ressource, UUID ressourceId, String
action) {
        outboxPublisher.publish("liaison", ressourceId.toString(),
"homologation.portal-access-denied.v1",
            new PortalAccessDeniedV1(ctx.userId(), ctx.institutionId(), ressource,
ressourceId, action, Instant.now()));
    }
```

```
}
```

A2. Cas d'utilisation : cloisonnement, responsabilité, relais du dépôt

Ordre des contrôles : existence et cloisonnement (404 `HOM_PORTAL_RESOURCE_NOT_FOUND`, sans distinguer une ressource inexistante d'une ressource d'une autre institution), partie `PARTICIPANT` (sinon 409 `HOM_PORTAL_OPERATION_DES`), responsable actif égal au demandeur (sinon 409 `HOM_PORTAL_NOT_RESPONSIBLE`). Le dépôt proprement dit (type attendu, chaîne de contrôle de MS-DOCUMENT, `EvidenceReference`, passage à `PREUVE_DEPOSEE`, émission de `homologation.evidence-submitted.v1`) est délégué à `SubmitEvidenceUseCase` (HOM-09) ; un rejet de MS-DOCUMENT remonte en 422 avec le motif explicite renvoyé par la chaîne de contrôle.

```
// application/usecase/PortalEvidenceService.java
@Service
@RequiredArgsConstructor
public class PortalEvidenceService implements PortalEvidenceUseCase {
    private final OperationInstanceRepository operations;
    private final AssignmentRepository assignments;
    private final SubmitEvidenceUseCase submitEvidence;
    private final PortalAccessAuditor auditor;
    private final NotificationRequestPublisher notifications;

    @Transactional
    public EvidenceReceipt submit(PortalContext ctx, UUID operationId, String
typePiece, MultipartFile fichier) {
        OperationInstance op =
operations.findByIdAndLiaisonInstitutionId(operationId, ctx.institutionId())
            .orElseThrow(() -> {
                auditor.denied(ctx, "OPERATION", operationId, "DEPOT_PREUVE");
                return new DomainProblemException(HttpStatus.NOT_FOUND,
"HOM_PORTAL_RESOURCE_NOT_FOUND");
            });
        if (op.getPartie() != Partie.PARTICIPANT) {
            auditor.denied(ctx, "OPERATION", operationId,
"DEPOT_PREUVE_OPERATION_DES");
            throw new DomainProblemException(HttpStatus.CONFLICT,
"HOM_PORTAL_OPERATION_DES");
        }
        boolean responsable = assignments.findByOperationIdAndActifTrue(operationId)
            .map(a -> a.getResponsableId().equals(ctx.userId())).orElse(false);
        if (!responsable) throw new DomainProblemException(HttpStatus.CONFLICT,
"HOM_PORTAL_NOT_RESPONSIBLE");

        EvidenceReceipt receipt = submitEvidence.submit(new
SubmitEvidenceCommand(operationId, typePiece, fichier, ctx.userId()));
        notifications.request(op.getLiaison().getId(), "HOM_PREUVE_DEPOSEE_PORTAIL",
            new NotificationRequestPublisher.Audience(List.of(),
List.of("INGENIEUR_HOMOLOGATION"), null),
            Map.of("operation", op.getCode(), "empreinte", receipt.empreinte()));
        return receipt;                                      // (evidenceId,
documentRef, version, empreinte, dateDepot, etatOperation)
    }
}
```

A3. Ressource du portail

La route interne est `/api/v1/portail/operations/{id}/preuves`, exposée par la passerelle sous `/bff/portail/homologation/operations/{id}/preuves`. Taille maximale et types MIME sont ceux de la chaîne de contrôle de MS-DOCUMENT (DOC-10) ; la ressource ne fait que transmettre.

```
// web/rest/PortalEvidenceResource.java
@RestController
@RequestMapping("/api/v1/portail/operations/{id}/preuves")
@RequiredArgsConstructor
public class PortalEvidenceResource {
    private final PortalEvidenceUseCase useCase;
    private final PortalContextResolver portal;

    @PostMapping(consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
    @PreAuthorize("hasAuthority('HOM_PORTAL_SUBMIT')")
    public ResponseEntity<EvidenceReceiptDTO> submit(@PathVariable UUID id,
@RequestParam String typePiece,
                                                     @RequestPart("fichier")
MultipartFile fichier) {
        EvidenceReceipt r = useCase.submit(portal.current(), id, typePiece, fichier);
        return
ResponseEntity.status(HttpStatus.CREATED).body(EvidenceReceiptDTO.of(r,
List.of("CONSULTER_ACCUSE")));
    }
}
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
//
src/test/java/org/gimac/des/homologation/application/usecase/PortalEvidenceServiceTes
t.java
@ExtendWith(MockitoExtension.class)
class PortalEvidenceServiceTest {
    @Mock OperationInstanceRepository operations; @Mock AssignmentRepository
assignments;
    @Mock SubmitEvidenceUseCase submitEvidence; @Mock PortalAccessAuditor auditor;
@Mock NotificationRequestPublisher notifications;
    @InjectMocks PortalEvidenceService service;
    private final PortalContext banqueB = new PortalContext(UUID.randomUUID(),
UUID.randomUUID());
    private final MockMultipartFile pdf = new MockMultipartFile("fichier",
"rapport.pdf", "application/pdf", new byte[]{1});

    @Test
    @DisplayName("HOM-23 : un dépôt sur une opération d'une autre institution est
refusé en 404 et journalisé")
    void autreInstitution() {
        UUID opId = UUID.randomUUID();
        when(operations.findByIdAndLiaisonInstitutionId(opId,
banqueB.institutionId())).thenReturn(Optional.empty());
        assertThatThrownBy(() -> service.submit(banqueB, opId, "RAPPORT_ESSAI", pdf))
            .isInstanceOf(DomainProblemException.class).extracting("status").isEqualT
o(HttpStatus.NOT_FOUND);
        verify(auditor).denied(banqueB, "OPERATION", opId, "DEPOT_PREUVE");
        verifyNoInteractions(submitEvidence);
    }

    @Test
    @DisplayName("HOM-23 : un dépôt sur une opération incombant au DES est refusé")
    void operationDuDes() {
        OperationInstance op = LiaisonTestData.operation(Partie.DES,
banqueB.institutionId());
        when(operations.findByIdAndLiaisonInstitutionId(op.getId(),
banqueB.institutionId())).thenReturn(Optional.of(op));
        assertThatThrownBy(() -> service.submit(banqueB, op.getId(), "RAPPORT_ESSAI",
pdf))
            .isInstanceOf(DomainProblemException.class).hasMessageContaining("HOM_POR
TAL_OPERATION_DES");
        verifyNoInteractions(submitEvidence);
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
// src/test/java/org/gimac/des/homologation/web/rest/PortalEvidenceResourceIT.java
@TestPropertySource(properties = "gimac.adapters.document-deposit=real")
class PortalEvidenceResourceIT extends AbstractHomologationIT {
    @RegisterExtension static WireMockExtension msDocument =
WireMockExtension.newInstance().options(wireMockConfig().dynamicPort()).build();
    @DynamicPropertySource static void props(DynamicPropertyRegistry r)
{ r.add("gimac.clients.ms-document.base-url", msDocument::baseUrl); }

    @Test
    @DisplayName("HOM-23 : rejet par le contrôle de masquage avec message explicite")
    void rejetMasquage() throws Exception {
        var f = fixtures.operationParticipantEligible("Établissement de paiement A");
// (operationId, institutionId, responsableId)

msDocument.stubFor(WireMock.post(urlPathEqualTo("/api/v1/documents")).willReturn(aRes
ponse().withStatus(422)
            .withHeader("Content-Type", "application/problem+json")
            .withBody("{\"code\":\"DOC_MASKING_REJECTED\",\"message\":\"La pièce
contient un numéro de carte non masqué\"}")));
        mockMvc.perform(multipart("/api/v1/portail/operations/{id}/preuves",
f.operationId())
                .file(new MockMultipartFile("fichier", "journal.txt", "text/plain",
"journal".getBytes(UTF_8)))
                .param("typePiece", "JOURNAL_ESSAI")
                .with(jwt().jwt(j ->
j.subject(f.responsableId().toString()).claim("institution_id",
f.institutionId().toString()))
                    .authorities(new SimpleGrantedAuthority("HOM_PORTAL_SUBMIT"))))
            .andExpect(status().isUnprocessableEntity())
            .andExpect(jsonPath("$.code").value("DOC_MASKING_REJECTED"))
            .andExpect(jsonPath("$.message").value("La pièce contient un numéro de
carte non masqué"));
        assertThat(jdbc.queryForObject("select etat from operation_instance where id
= ?", String.class, f.operationId()))
            .isEqualTo("ELIGIBLE");
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: HOM-23 Dépôt d'une preuve depuis le portail des participants

  Scénario: Nominal : dépôt par le correspondant
    Étant donné une opération éligible incombant à "Établissement de paiement A" dont
le correspondant connecté est responsable
    Quand il dépose la pièce du type attendu depuis le portail
    Alors un accusé de dépôt avec l'empreinte de la pièce lui est présenté
    Et l'opération passe à l'état "PREUVE_DEPOSEE" et l'ingénieur d'homologation est
notifié

  Scénario: Alternatif : opération incombant au DES
    Étant donné une opération de sa liaison incombant au DES
    Quand le correspondant la consulte
    Alors aucune action de dépôt ne lui est proposée et toute requête directe de
dépôt est refusée

  Scénario: Erreur : opération d'une autre institution
    Étant donné un correspondant de la "Banque B"
    Quand il tente de déposer une pièce sur une opération de "Établissement de
paiement A"
    Alors la requête est refusée par le cloisonnement institutionnel avec le statut
404
    Et la tentative est journalisée vers MS-AUDIT
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-correspondant-a
# Nominal : dépôt et accusé avec empreinte
curl -s -X POST "${GATEWAY_URL}/bff/portail/homologation/operations/${OPERATION_ID}/
preuves" \
  -H "Authorization: Bearer ${TOKEN}" -F "typePiece=RAPPORT_ESSAI" -F
"fichier=@./fixtures/rapport-essai.pdf;type=application/pdf" \
  | jq '{empreinte, version, etatOperation}'

# Refus : correspondant de la Banque B sur une opération de l'Établissement de
paiement A, attendu 404
source ./scripts/token.sh test-correspondant-b
curl -s -o /dev/null -w "%{http_code}\n" -X POST
"${GATEWAY_URL}/bff/portail/homologation/operations/${OPERATION_ID}/preuves" \
  -H "Authorization: Bearer ${TOKEN}" -F "typePiece=RAPPORT_ESSAI" -F
"fichier=@./fixtures/rapport-essai.pdf;type=application/pdf"
```

HOM-24 | Suivre sa liaison depuis le portail des participants

Élément	Valeur
Points	5
Priorité	Must Have
Sprint cible	Sprint 4
Écrans	POR-03
Dépendances	HOM-17, GW-13, GW-12, IHM-07

Objectif

Servir au correspondant, sous `/portail/liaisons/**`, une projection de la liaison de son institution sans annotation interne du DES (ce qui lui incombe en tête avec les échéances, opérations, résultats du simulateur tels que repris, couvertures), la rafraîchir par le canal poussé de MS-GATEWAY filtré par institution, et présenter comme introuvable, avec journalisation, toute liaison d'une autre institution.

A. Implémentation

A1. Projection du portail construite par liste blanche

La projection est un ensemble de records dédiés : seuls les champs listés sont copiés, si bien qu'un champ ajouté plus tard à une entité ne peut pas fuiter. Annotations internes exclues par construction : `grilleReponses`, `Assignment.motif`, `motifEtat`, scénarios inconnus et leur qualification, identité des collaborateurs du DES (remplacée par la partie « DES »). Les résultats du simulateur sont copiés sans transformation (non-intrusion). La phase courante réutilise `PhaseDelayCalculator` (HOM-20).

```
// application/usecase/portal/PortalLiaisonMapper.java
public final class PortalLiaisonMapper {
    public record PortalLiaisonView(UUID id, TypeProjet typeProjet, EtatLiaison etat,
String phaseCourante,
        List<PortalTodo> aFaire, List<PortalOperation> operations, List<PortalResult>
resultats, List<PortalCoverage> couvertures) {}
    public record PortalTodo(UUID operationId, String code, String libelle, LocalDate
echeance, List<String> typesPieceAutorises,
                             List<String> actionsPermises) {}
    public record PortalOperation(UUID id, String code, String libelle, String
serviceCode, Partie partie, EtatOperation etat,
                                  LocalDate echeance, String responsable) {}
    public record PortalResult(String idScenario, String campagneId, Verdict verdict,
Instant dateExecution, String referenceTrace) {}
    public record PortalCoverage(String serviceCode, NatureCouverture nature,
BigDecimal taux, boolean calculable, String motifNonCalculable) {}

    private static final Set<EtatOperation> A_FAIRE =
EnumSet.of(EtatOperation.ELIGIBLE, EtatOperation.ROUVERTE);

    public PortalLiaisonView toView(Liaison l, String phase, List<OperationInstance>
ops, Map<UUID, UUID> responsables,
                                    UUID userId, List<CampaignResult> res,
List<Coverage> cov) {
        List<PortalTodo> todo = ops.stream()
            .filter(o -> o.getPartie() == Partie.PARTICIPANT &&
A_FAIRE.contains(o.getEtat()))
            .sorted(Comparator.comparing(OperationInstance::getEcheance,
```

```
Comparator.nullsLast(Comparator.naturalOrder())))
            .map(o -> new PortalTodo(o.getId(), o.getCode(), o.getLibelle(),
o.getEcheance(),
                o.getReferenceOperation().typesPieceAttendus(l.getTypeProjet()),
                userId.equals(responsables.get(o.getId())) ?
List.of("DEPOSER_PREUVE") : List.of()))
            .toList();
        return new PortalLiaisonView(l.getId(), l.getTypeProjet(), l.getEtat(),
phase, todo,
            ops.stream().filter(o -> o.getEtat() != EtatOperation.HORS_PERIMETRE)
               .map(o -> new PortalOperation(o.getId(), o.getCode(), o.getLibelle(),
o.getServiceCode(), o.getPartie(),
                  o.getEtat(), o.getEcheance(), o.getPartie() == Partie.DES ?
"DES" : "PARTICIPANT")).toList(),
            res.stream().map(r -> new PortalResult(r.getIdScenario(),
r.getCampagneId(), r.getVerdict(),
                r.getDateExecution(), r.getReferenceTrace())).toList(),
            cov.stream().map(c -> new PortalCoverage(c.getServiceCode(),
c.getNature(), c.getTaux(),
                c.getCalculable(), c.getMotifNonCalculable())).toList());
    }
}
```

A2. Lecture cloisonnée et ressources

Toutes les requêtes filtrent par `institutionId` issu de `PortalContextResolver` (HOM-23) ; une liaison absente pour cette institution donne 404 `HOM_PORTAL_RESOURCE_NOT_FOUND` et un appel à `PortalAccessAuditor.denied(...)`. Routes : `GET /api/v1/portail/liaisons` (liste de l'institution), `GET /api/v1/portail/liaisons/{id}` (projection complète), `GET /api/v1/portail/liaisons/{id}/a-faire` (section « ce qui incombe à mon institution » seule), toutes sous `HOM_PORTAL_READ`.

```
// web/rest/PortalLiaisonResource.java
@RestController
@RequestMapping("/api/v1/portail/liaisons")
@PreAuthorize("hasAuthority('HOM_PORTAL_READ')")
@RequiredArgsConstructor
public class PortalLiaisonResource {
    private final PortalLiaisonQueryService query;
    private final PortalContextResolver portal;

    @GetMapping
    public List<PortalLiaisonSummary> list() { return query.list(portal.current()); }

    @GetMapping("/{id}")
    public PortalLiaisonView get(@PathVariable UUID id) { return
query.view(portal.current(), id); }

    @GetMapping("/{id}/a-faire")
    public List<PortalTodo> todo(@PathVariable UUID id) { return
query.view(portal.current(), id).aFaire(); }
}

// application/usecase/portal/PortalLiaisonQueryService.java (extrait)
@Transactional(readOnly = true)
public PortalLiaisonView view(PortalContext ctx, UUID id) {
    Liaison l = liaisons.findByIdAndInstitutionId(id,
ctx.institutionId()).orElseThrow(() -> {
        auditor.denied(ctx, "LIAISON", id, "CONSULTATION");
        return new DomainProblemException(HttpStatus.NOT_FOUND,
"HOM_PORTAL_RESOURCE_NOT_FOUND");
    });
    List<OperationInstance> ops = operations.findByLiaisonId(id);
    return mapper.toView(l, phases.currentPhase(l, ops), ops,
assignments.activeResponsables(id), ctx.userId(),
        results.findByLiaisonIdOrderByDateExecutionDesc(id),
```

```
coverages.findByLiaisonId(id));
}
```

A3. Canal poussé filtré par institution

Les cas d'utilisation qui changent l'état d'une opération (HOM-08, HOM-10, HOM-11) publient l'événement applicatif `OperationStateChanged`. Pour une opération de la partie `PARTICIPANT` devenue `ELIGIBLE` ou `CLOSE`, l'écouteur émet par l'outbox `homologation.portal-refresh.v1` (sujet `homologation.liaison`, extension `institutionid` renseignée depuis `institutionId`) et une demande de notification aux correspondants. MS-GATEWAY (GW-12) consomme ce type et ne le pousse qu'aux sessions du portail dont le jeton porte le même `institution_id` ; la charge ne contient que des identifiants et le nouvel état, l'interface relit la projection.

```
// application/usecase/portal/PortalRefreshPublisher.java
@Component
@RequiredArgsConstructor
public class PortalRefreshPublisher {
    private static final Set<EtatOperation> SIGNIFICATIFS =
EnumSet.of(EtatOperation.ELIGIBLE, EtatOperation.CLOSE);
    private final OutboxPublisher outboxPublisher;
    private final NotificationRequestPublisher notifications;

    public record OperationStateChanged(UUID operationId, String code, UUID
liaisonId, UUID institutionId,
                                        Partie partie, EtatOperation nouvelEtat) {}

    @EventListener
    @Transactional(propagation = Propagation.MANDATORY)
    public void on(OperationStateChanged e) {
        if (e.partie() != Partie.PARTICIPANT || !
SIGNIFICATIFS.contains(e.nouvelEtat())) return;
        outboxPublisher.publish("liaison", e.liaisonId().toString(),
"homologation.portal-refresh.v1",
            new PortalRefreshV1(e.liaisonId(), e.institutionId(), e.operationId(),
e.nouvelEtat()));
        notifications.request(e.liaisonId(), "HOM_OPERATION_" + e.nouvelEtat(),
            new NotificationRequestPublisher.Audience(List.of(),
List.of("CORRESPONDANT_INSTITUTION"), e.institutionId()),
            Map.of("operation", e.code()));
    }
}
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
// src/test/java/org/gimac/des/homologation/application/usecase/portal/
PortalLiaisonMapperTest.java
class PortalLiaisonMapperTest {
    private final ObjectMapper json = new ObjectMapper().findAndRegisterModules();

    @Test
    @DisplayName("HOM-24 : la projection du portail n'expose aucune annotation
interne")
    void aucuneAnnotationInterne() throws Exception {
        var ref = PortalTestData.liaisonAnnotee();   // grille renseignée, motif
d'attribution, motifEtat, scénario inconnu
        String out = json.writeValueAsString(new
PortalLiaisonMapper().toView(ref.liaison(), "ESSAIS", ref.operations(),
            ref.responsables(), ref.correspondantId(), ref.resultats(),
ref.couvertures()));
        assertThat(out).doesNotContain("grille", "motifEtat", "\"motif\"",
"qualification", ref.nomCollaborateurDes());
        assertThat(out).contains("referenceTrace", "\"responsable\":\"DES\"");
```

```
    }

    @Test
    @DisplayName("HOM-24 : ce qui incombe à l'institution est trié par échéance et
seul le responsable peut déposer")
    void aFaireTrie() {
        var ref = PortalTestData.liaisonAvecDeuxOperationsParticipant();
        var view = new PortalLiaisonMapper().toView(ref.liaison(), "ESSAIS",
ref.operations(), ref.responsables(),
            ref.correspondantId(), List.of(), List.of());
        assertThat(view.aFaire()).extracting(PortalTodo::code).containsExactly("OP-
BR-02", "OP-BR-05");

assertThat(view.aFaire().get(0).actionsPermises()).containsExactly("DEPOSER_PREUVE");
        assertThat(view.aFaire().get(1).actionsPermises()).isEmpty();
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
// src/test/java/org/gimac/des/homologation/web/rest/PortalLiaisonResourceIT.java
class PortalLiaisonResourceIT extends AbstractHomologationIT {
    @Test
    @DisplayName("HOM-24 : cloisonnement par institution, 404 et journalisation de la
tentative")
    void cloisonnement() throws Exception {
        UUID banqueB = UUID.randomUUID();
        var a = fixtures.liaisonPortail("Établissement de paiement A");     //
(liaisonId, institutionId)
        mockMvc.perform(get("/api/v1/portail/liaisons/{id}",
a.liaisonId()).with(correspondant(banqueB)))
            .andExpect(status().isNotFound()).andExpect(jsonPath("$.code").value("HOM
_PORTAL_RESOURCE_NOT_FOUND"));
        assertThat(outbox("homologation.portal-access-denied.v1",
a.liaisonId())).isEqualTo(1);
        mockMvc.perform(get("/api/v1/portail/liaisons/{id}",
a.liaisonId()).param("institutionId", banqueB.toString())
                .with(correspondant(a.institutionId())))                    // le
paramètre de requête est ignoré
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.aFaire").isArray())
            .andExpect(jsonPath("$.operations[0].grilleReponses").doesNotExist());
    }

    private RequestPostProcessor correspondant(UUID institution) {
        return jwt().jwt(j ->
j.subject(UUID.randomUUID().toString()).claim("institution_id",
institution.toString()))
            .authorities(new SimpleGrantedAuthority("HOM_PORTAL_READ"));
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: HOM-24 Suivi de sa liaison depuis le portail des participants

  Scénario: Nominal : vue de la liaison
    Étant donné un correspondant de "Établissement de paiement A" authentifié sur le
portail
    Quand il ouvre la liaison de son institution
    Alors la section "ce qui incombe à mon institution" est présentée en tête avec
les échéances
    Et les résultats du simulateur sont affichés tels que repris, avec leur référence
de trace, ainsi que les couvertures par service
```

```
  Scénario: Alternatif : changement d'état
    Étant donné un correspondant abonné aux événements de sa liaison
    Quand une opération qui lui incombe devient éligible ou est close
    Alors la vue est mise à jour par le canal poussé (événement "homologation.portal-
refresh.v1" filtré par son institution)
    Et une notification lui est adressée

  Scénario: Erreur : liaison d'une autre institution
    Étant donné un correspondant de la "Banque B"
    Quand il demande la liaison de "Établissement de paiement A"
    Alors la ressource lui est présentée comme introuvable
    Et la tentative est journalisée et aucune annotation interne du DES n'est exposée
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-correspondant-a
# Nominal : liste, vue et section à faire de la liaison de l'institution
curl -s "${GATEWAY_URL}/bff/portail/homologation/liaisons" -H "Authorization: Bearer
${TOKEN}" | jq '.[] | {id, etat}'
curl -s "${GATEWAY_URL}/bff/portail/homologation/liaisons/${LIAISON_ID}" -H
"Authorization: Bearer ${TOKEN}" \
  | jq '{phaseCourante, aFaire: [.aFaire[] | {code, echeance, actionsPermises}],
couvertures, resultats: [.resultats[] | {idScenario, verdict, referenceTrace}]}'
curl -s "${GATEWAY_URL}/bff/portail/homologation/liaisons/${LIAISON_ID}/a-faire" -H
"Authorization: Bearer ${TOKEN}" | jq length

# Refus : correspondant de la Banque B sur la liaison de l'Établissement de paiement
A, attendu 404
source ./scripts/token.sh test-correspondant-b
curl -s -o /dev/null -w "%{http_code}\n"
"${GATEWAY_URL}/bff/portail/homologation/liaisons/${LIAISON_ID}" -H "Authorization:
Bearer ${TOKEN}"
```

MS-CERTIFICATION — Récits CER-01 à CER-15 : certification, quitus et mise en service

Le microservice conduit le circuit ordonné des quatre quitus sur un socle d'évaluation identifié par son empreinte, contrôle la complétude du dossier de mise en service, lit sans jamais l'écrire la fiche de changement, consigne le procès-verbal de mise en service, scelle et horodate les décisions, émet l'attestation de certification signée et tient le registre des certifications délivrées. Les quinze récits sont traités ci-dessous dans l'ordre des features du carnet.

CER-01 | Ouvrir le circuit de certification d'une liaison apte

Élément	Valeur
Points	5
Priorité	Must Have
Sprint cible	Sprint 4
Écrans	DES-11
Dépendances	HOM-18, SOC-05, NOT-14

Objectif

Consommer `homologation.liaison-ready-for-certification.v1` pour ouvrir, une seule fois par triplet (liaison, cycle, empreinte), un circuit de certification à quatre quitus et le dossier de mise en service, après relecture et vérification du socle d'évaluation auprès de MS-HOMOLOGATION. Un socle introuvable ou divergent n'ouvre rien : l'événement part au rebut et une alerte est émise vers MS-MONITORING.

A. Implémentation

A1. Entités du circuit, des quitus et du dossier

Le circuit porte la clé métier d'idempotence, doublée d'une contrainte d'unicité dans le changelog `20261005_cer01_circuit.xml`. À l'ouverture, le quitus technique est `ATTENDU` et les trois autres sont `NON_REQUIS`, état qui signifie ici « non encore requis à l'étape courante » ; l'automate du circuit (CER-03) les rend `ATTENDU` le moment venu. La table `QuitusRoles` associe à chaque nature son rôle signataire et sa permission ; elle est réutilisée par tous les récits du circuit.

```
// domain/model/CertificationCircuit.java
@Entity
@Table(name = "certification_circuit", uniqueConstraints = @UniqueConstraint(
    name = "ux_circuit_liaison_cycle_socle", columnNames = {"liaison_id", "cycle",
"socle_empreinte"}))
public class CertificationCircuit {
    @Id @GeneratedValue(strategy = GenerationType.UUID) private UUID id;
    @Column(name = "liaison_id", nullable = false) private UUID liaisonId;
    @Column(name = "institution_id", nullable = false) private UUID institutionId;
    @Column(nullable = false) private Integer cycle;
    @Column(name = "socle_empreinte", nullable = false, length = 64) private String
socleEmpreinte;
    @Enumerated(EnumType.STRING) @Column(nullable = false) private EtatCircuit etat;
    private Instant dateOuverture;
    private Instant dateFin;
    @OneToMany(mappedBy = "circuit", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Quitus> quitus = new ArrayList<>();

    protected CertificationCircuit() { }   // exigé par JPA

    /** Ouvre un circuit : quitus technique attendu, les trois autres non encore
requis. */
    public static CertificationCircuit open(UUID liaisonId, UUID institutionId, int
cycle, String socle, Instant now) {
        CertificationCircuit c = new CertificationCircuit();
        c.liaisonId = liaisonId; c.institutionId = institutionId; c.cycle = cycle;
        c.socleEmpreinte = socle; c.etat = EtatCircuit.OUVERT; c.dateOuverture = now;
        for (NatureQuitus n : NatureQuitus.values()) {
```

```
            c.quitus.add(Quitus.create(c, n, n == NatureQuitus.TECHNIQUE ?
EtatQuitus.ATTENDU : EtatQuitus.NON_REQUIS, now));
        }
        return c;
    }

    public Quitus quitus(NatureQuitus nature) {
        return quitus.stream().filter(q -> q.getNature() ==
nature).findFirst().orElseThrow();
    }

    public void interrupt(Instant now) {
        if (etat == EtatCircuit.OUVERT) { etat = EtatCircuit.INTERROMPU; dateFin =
now; }
    }
    // quitusStates() et applyStates(...) : CER-03 ; accesseurs omis
}
```

`Quitus` (JDL : nature, etat, dateAttendu, motifCaducite) n'expose que des méthodes métier : `create(...)`, `changeState(cible, now)` (renseigne `dateAttendu` au passage à `ATTENDU`) et `lapse(motif)` (état `CADUC` et motif conservé).

```
// domain/model/QuitusRoles.java
public final class QuitusRoles {
    private static final Map<NatureQuitus, String> ROLE = Map.of(
        TECHNIQUE, "CHEF_DES", SECURITE, "RSSI", CONFORMITE,
"RESPONSABLE_CONFORMITE", DIRECTION_GENERALE, "DIRECTION_GENERALE");
    private static final Map<NatureQuitus, String> PERMISSION = Map.of(
        TECHNIQUE, "CER_QUITUS_TECHNIQUE_SIGN", SECURITE, "CER_QUITUS_SECURITE_SIGN",
        CONFORMITE, "CER_QUITUS_CONFORMITE_SIGN", DIRECTION_GENERALE,
"CER_QUITUS_DG_SIGN");
    public static String roleOf(NatureQuitus n) { return ROLE.get(n); }
    public static String permissionOf(NatureQuitus n) { return PERMISSION.get(n); }
    private QuitusRoles() { }
}
```

`GoLiveDossier` (champs du JDL) reçoit deux méthodes : `open(liaisonId, institutionId, now)` crée le dossier à l'état `OUVERT` avec ses six conditions non satisfaites (CER-08), et `attachCircuit(circuit)` renseigne le circuit courant ; un nouveau cycle reprend ainsi le même dossier.

A2. Port de lecture de MS-HOMOLOGATION et ses deux adaptateurs

Le socle et les intervenants sont des objets-valeurs du domaine (`domain/model/EvaluationBaseSnapshot`, `CoverageLine`, `LiaisonParticipants`), afin que les services de domaine purs (CER-07, CER-08) ne dépendent pas de la couche application. Un socle absent (404) est rendu par `Optional.empty()` ; une indisponibilité lève `HomologationUnavailableException` (503, aucune valeur par défaut).

```
// application/port/out/HomologationReaderPort.java
public interface HomologationReaderPort {
    /** GET /liaisons/{id}/socle-evaluation (interne, jeton de service). */
    Optional<EvaluationBaseSnapshot> readEvaluationBase(UUID liaisonId);
    /** GET /liaisons/{id}/intervenants (interne) : chef de projet, responsables,
validateurs. */
    LiaisonParticipants readParticipants(UUID liaisonId);
}

// domain/model/EvaluationBaseSnapshot.java
public record EvaluationBaseSnapshot(UUID liaisonId, int cycle, String empreinte,
Instant dateCalcul,
        List<String> operationCodes, List<String> serviceCodes, List<CoverageLine>
couvertures) { }
```

```
public record CoverageLine(String serviceCode, String nature, BigDecimal taux,
boolean calculable) { }
public record LiaisonParticipants(UUID chefDeProjetId, Set<UUID> responsableIds,
Set<UUID> validateurIds) { }
```

```
// infrastructure/client/HomologationReaderRestAdapter.java
@Component
@ConditionalOnProperty(name = "gimac.adapters.homologation-reader", havingValue =
"real", matchIfMissing = true)
class HomologationReaderRestAdapter implements HomologationReaderPort {
    private final RestClient client;

    HomologationReaderRestAdapter(RestClient.Builder builder, ServiceTokenProvider
tokens,
                                  @Value("${gimac.clients.mshomologation.base-url}")
String baseUrl) {
        this.client = builder.baseUrl(baseUrl + "/api/v1")
            .requestInterceptor((req, body, exec) -> {            // jeton du client
mscertification-service
                req.getHeaders().setBearerAuth(tokens.token());
                return exec.execute(req, body);
            }).build();
    }

    @Override
    @CircuitBreaker(name = "mshomologation")
    public Optional<EvaluationBaseSnapshot> readEvaluationBase(UUID liaisonId) {
        try {
            return Optional.ofNullable(client.get().uri("/liaisons/{id}/socle-
evaluation", liaisonId)
                .retrieve().body(EvaluationBaseSnapshot.class));
        } catch (HttpClientErrorException.NotFound e) {
            return Optional.empty();
        } catch (RestClientException e) {
            throw new HomologationUnavailableException(liaisonId, e);
        }
    }
    // readParticipants : même motif sur /liaisons/{id}/intervenants
}
```

L'adaptateur de substitution `HomologationReaderStubAdapter` (`@Profile({"dev", "test"})`, `gimac.adapters.homologation-reader=stub`) sert des socles et intervenants enregistrés en mémoire par `register(...)`, ce qui permet aux étapes Cucumber de préparer un socle conforme ou divergent.

A3. Cas d'utilisation d'ouverture

Toutes les vérifications précèdent la première écriture : un rejet ne laisse ni circuit ni dossier. L'indisponibilité de MS-HOMOLOGATION lève une exception, annule la transaction et laisse le gestionnaire d'erreurs Kafka redélivrer l'événement avec retard exponentiel.

```
// application/usecase/OpenCertificationCircuitService.java
@Service
@RequiredArgsConstructor
public class OpenCertificationCircuitService {
    public enum Outcome { OPENED, ALREADY_OPEN, REJECTED }

    private final CertificationCircuitRepository circuits;
    private final GoLiveDossierRepository dossiers;
    private final HomologationReaderPort homologation;
    private final QuitusLapseService quitusLapse;       // contrôle de caducité des
quitus antérieurs (CER-06)
    private final RejectedEventWriter rejectedEvents;   // socle de messagerie :
rejected_record puis <sujet>.dlq
    private final MonitoringAlertPort alerts;
```

```
    private final OutboxPublisher outboxPublisher;
    private final Clock clock;

    @Transactional
    public Outcome open(LiaisonReadyForCertificationV1 evt, CloudEventEnvelope
source) {
        // Clé métier : un événement republié sous un autre identifiant CloudEvent
reste sans effet
        if (circuits.existsByLiaisonIdAndCycleAndSocleEmpreinte(evt.liaisonId(),
evt.cycle(), evt.socleEmpreinte())) {
            return Outcome.ALREADY_OPEN;
        }
        String motif = homologation.readEvaluationBase(evt.liaisonId())
            .map(s -> s.cycle() == evt.cycle() &&
s.empreinte().equals(evt.socleEmpreinte()) ? null
                : "Socle divergent : attendu " + evt.socleEmpreinte() + ", restitué "
+ s.empreinte())
            .orElse("Socle introuvable pour la liaison " + evt.liaisonId());
        if (motif != null) {                              // ouverture suspendue,
aucune écriture métier
            rejectedEvents.reject("homologation.liaison", source, motif);
            alerts.raise("CER_SOCLE_DIVERGENT", evt.liaisonId().toString(),
Map.of("cycle", evt.cycle(), "motif", motif));
            return Outcome.REJECTED;
        }
        Instant now = clock.instant();
        Optional<CertificationCircuit> previous =
circuits.findFirstByLiaisonIdOrderByCycleDesc(evt.liaisonId());
        CertificationCircuit circuit = circuits.save(CertificationCircuit.open(
            evt.liaisonId(), evt.institutionId(), evt.cycle(), evt.socleEmpreinte(),
now));
        GoLiveDossier dossier = dossiers.findFirstByLiaisonIdAndEtat(evt.liaisonId(),
EtatDossier.OUVERT)
            .orElseGet(() -> dossiers.save(GoLiveDossier.open(evt.liaisonId(),
evt.institutionId(), now)));
        dossier.attachCircuit(circuit);
        previous.ifPresent(p -> { p.interrupt(now); quitusLapse.onNewCycle(p,
circuit); });

        outboxPublisher.publish("CertificationCircuit", circuit.getId().toString(),
"certification.circuit-opened.v1",
            new CircuitOpenedV1(circuit.getId(), dossier.getId(), evt.liaisonId(),
evt.institutionId(),
                evt.cycle(), evt.socleEmpreinte(), now));
        circuit.getQuitus().stream().filter(q -> q.getEtat() ==
EtatQuitus.ATTENDU).forEach(q ->
            outboxPublisher.publish("Notification", circuit.getId().toString(),
"certification.notification-requested.v1",
                NotificationRequestedV1.toRole(QuitusRoles.roleOf(q.getNature()),
"CER_QUITUS_ATTENDU",
                    Map.of("circuitId", circuit.getId(), "nature", q.getNature()))));
// NOT-14
        return Outcome.OPENED;
    }
}
```

En Sprint 4, `QuitusLapseService` est livré dans une version conservatrice, remplacée en CER-06 par la comparaison d'empreintes : tout quitus favorable du cycle précédent devient caduc et rien n'est reconduit.

```
// application/usecase/QuitusLapseService.java (version Sprint 4, remplacée en CER-
06)
@Service
public class QuitusLapseService {
    public void onNewCycle(CertificationCircuit previous, CertificationCircuit next)
{
        previous.getQuitus().stream().filter(q -> q.getEtat() ==
```

```
EtatQuitus.SIGNE_FAVORABLE)
            .forEach(q -> q.lapse("Nouveau cycle " + next.getCycle() + " : contrôle
de caducité conservateur"));
    }
}
```

A4. Consommateur idempotent, alerte de supervision et lecture du circuit

Le sujet `homologation.liaison` multiplexe les types de l'agrégat ; le consommateur ne retient que l'aptitude. L'alerte vers MS-MONITORING est un événement `certification.alert-raised.v1` publié sur `certification.notification` par `OutboxMonitoringAlertAdapter` dans une transaction propre (`REQUIRES_NEW`), afin qu'elle survive à l'annulation de la transaction appelante (cas du coffre de clés en CER-02) ; l'adaptateur de substitution se contente de journaliser.

```
// infrastructure/messaging/consumer/LiaisonReadyForCertificationConsumer.java
@Component
@RequiredArgsConstructor
class LiaisonReadyForCertificationConsumer {
    static final String GROUP = "mscertification-aptitude";
    private final IdempotentConsumer idempotentConsumer;
    private final OpenCertificationCircuitService openCircuit;

    @KafkaListener(topics = "homologation.liaison", groupId = GROUP)
    void onEvent(CloudEventEnvelope envelope) {
        if (!"homologation.liaison-ready-for-
certification.v1".equals(envelope.type())) {
            return;
        }
        idempotentConsumer.handleOnce(envelope.id(), GROUP,
            () ->
openCircuit.open(envelope.dataAs(LiaisonReadyForCertificationV1.class), envelope));
    }
}


// web/rest/CircuitResource.java (extrait)
@GetMapping("/circuits/{id}")
@PreAuthorize("hasAuthority('CER_CIRCUIT_READ')")
public CircuitDTO get(@PathVariable UUID id, Authentication auth) {
    return circuitQueries.circuit(id, auth);   // quatre quitus, transitionsPermises
et actionsPermises
}
```

L'écran du circuit (tâche frontend) est traité dans le fragment DES-11 ; il affiche `CircuitDTO` (cycle, empreinte du socle, quatre quitus et leurs états) sans rien recalculer.

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
// test : application/usecase/OpenCertificationCircuitServiceTest.java
@ExtendWith(MockitoExtension.class)
class OpenCertificationCircuitServiceTest {
    @Mock CertificationCircuitRepository circuits; @Mock GoLiveDossierRepository
dossiers;
    @Mock HomologationReaderPort homologation; @Mock QuitusLapseService quitusLapse;
    @Mock RejectedEventWriter rejectedEvents; @Mock MonitoringAlertPort alerts; @Mock
OutboxPublisher outbox;
    final Clock clock = Clock.fixed(Instant.parse("2026-10-05T08:00:00Z"),
ZoneOffset.UTC);
    static final UUID LIAISON = UUID.randomUUID();
    static final String SOCLE = "a".repeat(64);
    OpenCertificationCircuitService service;
```

```
    @BeforeEach
    void setUp() {
        service = new OpenCertificationCircuitService(circuits, dossiers,
homologation, quitusLapse, rejectedEvents, alerts, outbox, clock);
    }

    @Test
    @DisplayName("CER-01 : un seul circuit est ouvert par liaison, cycle et
empreinte")
    void ouvertureUniqueParCycle() {
        when(circuits.existsByLiaisonIdAndCycleAndSocleEmpreinte(LIAISON, 1,
SOCLE)).thenReturn(false, true);

when(homologation.readEvaluationBase(LIAISON)).thenReturn(Optional.of(TestSocles.socl
e(LIAISON, 1, SOCLE)));
        when(circuits.save(any())).thenAnswer(TestSocles::persisted);          //
affecte un identifiant
        when(dossiers.save(any())).thenAnswer(TestSocles::persisted);

        assertThat(service.open(TestSocles.aptitude(LIAISON, 1, SOCLE),
null)).isEqualTo(Outcome.OPENED);
        assertThat(service.open(TestSocles.aptitude(LIAISON, 1, SOCLE),
null)).isEqualTo(Outcome.ALREADY_OPEN);

        verify(circuits, times(1)).save(argThat(c -> c.quitus(TECHNIQUE).getEtat() ==
EtatQuitus.ATTENDU
            && c.quitus(DIRECTION_GENERALE).getEtat() == EtatQuitus.NON_REQUIS));
        verify(outbox, times(1)).publish(eq("CertificationCircuit"), any(),
eq("certification.circuit-opened.v1"), any());
    }

    @Test
    @DisplayName("CER-01 : une empreinte divergente n'ouvre rien, part au rebut et
alerte MS-MONITORING")
    void empreinteDivergente() {

when(homologation.readEvaluationBase(LIAISON)).thenReturn(Optional.of(TestSocles.socl
e(LIAISON, 1, "b".repeat(64))));

        assertThat(service.open(TestSocles.aptitude(LIAISON, 1, SOCLE),
null)).isEqualTo(Outcome.REJECTED);

        verify(circuits, never()).save(any());
        verify(dossiers, never()).save(any());
        verify(rejectedEvents).reject(eq("homologation.liaison"), any(),
startsWith("Socle divergent"));
        verify(alerts).raise(eq("CER_SOCLE_DIVERGENT"), eq(LIAISON.toString()),
anyMap());
    }
}

// test : contract/LiaisonReadyContractTest.java
class LiaisonReadyContractTest {
    @Test
    @DisplayName("CER-01 : l'exemple d'aptitude respecte le schéma partagé avec MS-
HOMOLOGATION")
    void schemaPartage() throws Exception {
        JsonSchema schema =
JsonSchemaFactory.getInstance(SpecVersion.VersionFlag.V202012).getSchema(
            getClass().getResourceAsStream("/contracts/homologation/liaison-ready-
for-certification.v1.schema.json"));
        ObjectMapper mapper = new ObjectMapper().findAndRegisterModules();
        JsonNode exemple = mapper.readTree(
            getClass().getResourceAsStream("/contracts/homologation/liaison-ready-
for-certification.v1.example.json"));

        assertThat(schema.validate(exemple)).isEmpty();
        assertThat(mapper.treeToValue(exemple.get("data"),
LiaisonReadyForCertificationV1.class).socleEmpreinte()).hasSize(64);
```

```
    }
}
```

Le schéma et l'exemple sont versionnés dans le répertoire de contrats partagé avec MS-HOMOLOGATION (HOM-18) et copiés en ressources de test à la construction.

C. Tests d'intégration (@SpringBootTest + Testcontainers)

La classe de base ci-dessous sert à tous les tests d'intégration du service : conteneurs PostgreSQL et Kafka, un serveur WireMock par tiers, adaptateurs réels pointés sur WireMock, jeton de service figé, et l'aide `CertificationFixtures` (circuits, dossiers et quitus préparés par les cas d'utilisation eux-mêmes).

```
// test : AbstractCertificationIT.java
@SpringBootTest
@AutoConfigureMockMvc
@Testcontainers
@ActiveProfiles("test")
public abstract class AbstractCertificationIT {
    @Container static PostgreSQLContainer<?> postgres = new
PostgreSQLContainer<>("postgres:16-alpine");
    @Container static KafkaContainer kafka = new
KafkaContainer("apache/kafka:3.8.0");
    @RegisterExtension static WireMockExtension homologation = wiremock();
    @RegisterExtension static WireMockExtension iam = wiremock();
    @RegisterExtension static WireMockExtension keyVault = wiremock();
    @RegisterExtension static WireMockExtension documents = wiremock();

    @DynamicPropertySource
    static void properties(DynamicPropertyRegistry r) {
        r.add("spring.datasource.url", postgres::getJdbcUrl);
        r.add("spring.datasource.username", postgres::getUsername);
        r.add("spring.datasource.password", postgres::getPassword);
        r.add("spring.kafka.bootstrap-servers", kafka::getBootstrapServers);
        r.add("gimac.clients.mshomologation.base-url", homologation::baseUrl);
        r.add("gimac.clients.msiam.base-url", iam::baseUrl);
        r.add("gimac.clients.keyvault.base-url", keyVault::baseUrl);
        r.add("gimac.clients.msdocument.base-url", documents::baseUrl);
        Stream.of("homologation-reader", "reauthentication", "key-vault",
"delegation-reader", "document-verification")
            .forEach(p -> r.add("gimac.adapters." + p, () -> "real"));
    }

    @MockitoBean protected ServiceTokenProvider serviceTokens;
    @Autowired protected MockMvc mockMvc;
    @Autowired protected JdbcTemplate jdbc;
    @Autowired protected KafkaTemplate<String, String> kafkaTemplate;
    @Autowired protected CertificationFixtures fixtures;

    static WireMockExtension wiremock() {
        return
WireMockExtension.newInstance().options(wireMockConfig().dynamicPort()).build();
    }
    protected static RequestPostProcessor asUser(UUID userId, String role, String...
permissions) {
        return jwt().jwt(j -> j.subject(userId.toString()).claim("roles",
List.of(role)))
            .authorities(Arrays.stream(permissions).map(SimpleGrantedAuthority::new).
toList());
    }
    protected long outboxCount(String eventType, Object aggregateId) {
        return jdbc.queryForObject("select count(*) from outbox_event where
event_type = ? and aggregate_id = ?",
            Long.class, eventType, aggregateId.toString());
    }
```

```
}

// test : infrastructure/messaging/consumer/LiaisonReadyForCertificationIT.java
class LiaisonReadyForCertificationIT extends AbstractCertificationIT {
    @Test
    @DisplayName("CER-01 : une empreinte divergente dirige l'événement vers le rebut
sans ouvrir de circuit")
    void empreinteDivergenteAuRebut() {
        UUID liaison = UUID.randomUUID();
        homologation.stubFor(get(urlPathEqualTo("/api/v1/liaisons/" + liaison +
"/socle-evaluation"))
            .willReturn(okJson(fixtures.socleJson(liaison, 1, "b".repeat(64)))));

        kafkaTemplate.send("homologation.liaison", liaison.toString(),
fixtures.aptitudeEvent(liaison, 1, "a".repeat(64)));

        await().atMost(Duration.ofSeconds(20)).untilAsserted(() -> {
            assertThat(jdbc.queryForObject("select count(*) from rejected_record
where charge_utile like ?",
                Long.class, "%" + liaison + "%")).isEqualTo(1);
            assertThat(outboxCount("certification.alert-raised.v1",
liaison)).isEqualTo(1);
        });
        assertThat(jdbc.queryForObject("select count(*) from certification_circuit
where liaison_id = ?", Long.class, liaison)).isZero();
        assertThat(jdbc.queryForObject("select count(*) from go_live_dossier where
liaison_id = ?", Long.class, liaison)).isZero();
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: CER-01 Ouverture du circuit de certification d'une liaison apte

  Contexte:
    Étant donné la liaison "L-EPA-01" de l'Établissement de paiement A dont le socle
d'évaluation du cycle 1 a pour empreinte "a3f1"

  Scénario: Nominal : ouverture du circuit
    Étant donné l'événement homologation.liaison-ready-for-certification.v1 de la
liaison "L-EPA-01" portant l'empreinte "a3f1" et le cycle 1
    Quand l'événement est consommé
    Alors un circuit de certification à l'état "OUVERT" et un dossier de mise en
service à l'état "OUVERT" existent pour la liaison
    Et le socle a été lu auprès de MS-HOMOLOGATION et son empreinte vérifiée
    Et le quitus "TECHNIQUE" est "ATTENDU" et les trois autres sont "NON_REQUIS"
    Et une notification est demandée pour le rôle "CHEF_DES" et l'événement
certification.circuit-opened.v1 est émis

  Scénario: Alternatif : événement redélivré
    Étant donné un circuit déjà ouvert pour la liaison "L-EPA-01", le cycle 1 et
l'empreinte "a3f1"
    Quand l'événement d'aptitude est présenté une seconde fois
    Alors la liaison compte toujours un seul circuit et un seul dossier

  Scénario: Alternatif : nouveau cycle après un refus
    Étant donné un dossier dont le circuit du cycle 1 a été interrompu par un refus
    Et un quitus "TECHNIQUE" favorable dans ce circuit
    Quand un événement d'aptitude du cycle 2 est consommé
    Alors le même dossier porte un nouveau circuit du cycle 2
    Et le quitus "TECHNIQUE" du cycle 1 a été soumis au contrôle de caducité

  Scénario: Erreur : socle introuvable ou divergent
    Étant donné un événement d'aptitude portant l'empreinte "ffff" alors que MS-
HOMOLOGATION restitue "a3f1"
    Quand le circuit est ouvert
```

```
    Alors aucun circuit ni dossier n'est créé
    Et l'événement est placé au rebut sur "homologation.liaison.dlq"
    Et l'alerte "CER_SOCLE_DIVERGENT" est émise vers MS-MONITORING
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-chef-des
# Le circuit est retrouvé par l'identifiant publié dans certification.circuit-
opened.v1 (environnement de recette)
CIRCUIT_ID=$(kcat -b "${KAFKA_BOOTSTRAP}" -t certification.circuit -C -e -q \
  | jq -r --arg l "${LIAISON_ID}" 'select(.type=="certification.circuit-opened.v1"
and .data.liaisonId==$l) | .subject' | tail -1)

# Nominal : circuit ouvert, quitus technique attendu
curl -s "${GATEWAY_URL}/bff/des/certification/circuits/${CIRCUIT_ID}" \
  -H "Authorization: Bearer ${TOKEN}" \
  | jq '{etat, cycle, socleEmpreinte, quitus: [.quitus[] | {nature, etat}],
actionsPermises}'
# Attendu : etat "OUVERT" ; TECHNIQUE "ATTENDU" ; les trois autres "NON_REQUIS"

# Refus : sans jeton, la passerelle répond 401
curl -s -o /dev/null -w "%{http_code}\n"
"${GATEWAY_URL}/bff/des/certification/circuits/${CIRCUIT_ID}"
```

CER-02 | Signer le quitus technique après réauthentification FIDO2

Élément	Valeur
Points	8
Priorité	Must Have
Sprint cible	Sprint 4
Écrans	DES-11
Dépendances	CER-01, IAM-07, SOC-06, CER-07

Objectif

Livrer le mécanisme de signature réutilisé par les quatre quitus (`QuitusSigningService`) et l'appliquer au quitus technique : motif obligatoire, réauthentification FIDO2 vérifiée par MS-IAM, empreinte canonique SHA-256 de la décision signée par le coffre de clés, `SignedDecision` immuable écrite avec `certification.quitus-signed.v1` dans une seule transaction, puis quitus sécurité et conformité rendus attendus.

A. Implémentation

A1. Décision signée immuable

L'immuabilité est tenue à trois niveaux : aucun mutateur dans la classe, `@Immutable` pour Hibernate (aucune requête de mise à jour n'est générée), et un déclencheur PostgreSQL qui rejette toute mise à jour ou suppression, y compris par une requête SQL directe.

```
// domain/model/SignedDecision.java
@Entity
@Immutable
@Table(name = "signed_decision")
public class SignedDecision {
    @Id @GeneratedValue(strategy = GenerationType.UUID) private UUID id;
    @OneToOne(optional = false) @JoinColumn(name = "quitus_id", unique = true,
updatable = false) private Quitus quitus;
    @Column(nullable = false, updatable = false) private Boolean decisionFavorable;
    @Column(nullable = false, updatable = false, length = 2000) private String motif;
    @Column(nullable = false, updatable = false) private UUID signataireId;
    @Column(nullable = false, updatable = false) private String signataireRole;
    @Column(nullable = false, updatable = false) private Instant dateSignature;
    @Column(nullable = false, updatable = false, length = 64) private String
socleEmpreinte;
    @Column(nullable = false, updatable = false, length = 64) private String
empreinteDecision;
    @Column(nullable = false, updatable = false, columnDefinition = "text") private
String signature;
    @JdbcTypeCode(SqlTypes.JSON) @Column(updatable = false) private List<String>
operationsDesignees;
    @JdbcTypeCode(SqlTypes.JSON) @Column(updatable = false) private List<String>
servicesDesignes;
    @Column(nullable = false, updatable = false) private UUID referenceAudit;

    protected SignedDecision() { }

    public static SignedDecision of(Quitus quitus, DecisionContent c, String
empreinte, SignatureResult s, UUID auditRef) {
        SignedDecision d = new SignedDecision();
        d.quitus = quitus; d.decisionFavorable = c.favorable(); d.motif = c.motif();
        d.signataireId = c.signataireId(); d.signataireRole =
```

```
QuitusRoles.roleOf(c.nature());
        d.dateSignature = c.instant(); d.socleEmpreinte = c.socleEmpreinte();
d.empreinteDecision = empreinte;
        d.signature = s.signatureBase64(); d.operationsDesignees =
List.copyOf(c.operationsDesignees());
        d.servicesDesignes = List.copyOf(c.servicesDesignes()); d.referenceAudit =
auditRef;
        return d;
    }
    // accesseurs uniquement : aucun setter
}
```

```
-- config/liquibase/changelog/20261005_cer02_signed_decision_immutable.sql
--liquibase formatted sql
--changeset cer:cer02-signed-decision-immutable splitStatements:false
CREATE OR REPLACE FUNCTION forbid_signed_decision_mutation() RETURNS trigger AS $$
BEGIN
  RAISE EXCEPTION 'signed_decision est immuable : % refusé', TG_OP USING ERRCODE =
'23514';
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_signed_decision_immutable BEFORE UPDATE OR DELETE ON
signed_decision
  FOR EACH ROW EXECUTE FUNCTION forbid_signed_decision_mutation();
```

A2. Empreinte canonique de la décision

Le contenu signé couvre la liaison, l'empreinte du socle, la nature et le sens de la décision, le motif, le signataire, l'instant et les désignations (vides pour une décision favorable). La forme canonique trie les clés, normalise le texte en NFC, tronque l'instant à la milliseconde et préfixe chaque valeur par sa longueur, ce qui interdit toute ambiguïté (un motif contenant un saut de ligne ne peut pas simuler un autre champ).

```
// domain/service/DecisionCanonicalizer.java
public final class DecisionCanonicalizer {
    public static byte[] sha256(DecisionContent d) {
        SortedMap<String, String> f = new TreeMap<>();
        f.put("decision", d.favorable() ? "FAVORABLE" : "DEFAVORABLE");
        f.put("instant",
DateTimeFormatter.ISO_INSTANT.format(d.instant().truncatedTo(ChronoUnit.MILLIS)));
        f.put("liaison", d.liaisonId().toString());
        f.put("motif", Normalizer.normalize(d.motif().strip(), Normalizer.Form.NFC));
        f.put("nature", d.nature().name());
        f.put("operations", String.join(",", new
TreeSet<>(d.operationsDesignees())));
        f.put("services", String.join(",", new TreeSet<>(d.servicesDesignes())));
        f.put("signataire", d.signataireId().toString());
        f.put("socle", d.socleEmpreinte());
        StringBuilder sb = new StringBuilder();
        f.forEach((k, v) ->
sb.append(k).append('=').append(v.length()).append(':').append(v).append('\n'));
        try {
            return MessageDigest.getInstance("SHA-
256").digest(sb.toString().getBytes(StandardCharsets.UTF_8));
        } catch (NoSuchAlgorithmException e) {
            throw new IllegalStateException(e);
        }
    }
    private DecisionCanonicalizer() { }
}
```

A3. Ports de réauthentification et de signature, adaptateurs réels et de substitution

```
// application/port/out/ReauthenticationPort.java
```

```
public interface ReauthenticationPort {
    /** Assertion FIDO2 vérifiée par MS-IAM (IAM-07) : faux si échouée, expirée ou
rejouée. */
    boolean verify(String assertion, UUID userId, String purpose);
}

// application/port/out/KeyVaultSigningPort.java
public interface KeyVaultSigningPort {
    /** Signe l'empreinte avec la clé du signataire ; lève
KeyVaultUnavailableException. */
    SignatureResult sign(String keyAlias, byte[] sha256Digest);
    record SignatureResult(String signatureBase64, String keyVersion, String
algorithm) { }
}

// infrastructure/client/KeyVaultSigningRestAdapter.java
@Component
@ConditionalOnProperty(name = "gimac.adapters.key-vault", havingValue = "real",
matchIfMissing = true)
class KeyVaultSigningRestAdapter implements KeyVaultSigningPort {
    private final RestClient client;   // authentification mutuelle : SslBundle
"keyvault", délai de lecture 3 s

    KeyVaultSigningRestAdapter(RestClient.Builder builder, @Value("$
{gimac.clients.keyvault.base-url}") String baseUrl) {
        this.client = builder.baseUrl(baseUrl).build();
    }

    @Override
    @CircuitBreaker(name = "keyvault")
    public SignatureResult sign(String keyAlias, byte[] digest) {
        try {
            return client.post().uri("/v1/keys/{alias}/sign", keyAlias)
                .body(Map.of("digest", Base64.getEncoder().encodeToString(digest),
"algorithm", "RSASSA_PSS_SHA_256"))
                .retrieve().body(SignatureResult.class);
        } catch (RestClientException e) {
            throw new KeyVaultUnavailableException(keyAlias, e);
        }
    }
}
```

`KeyVaultSigningStubAdapter` (profils `dev` et `test`) signe en `SHA256withRSA` avec une clé éphémère générée en mémoire au démarrage, jamais persistée. `IamReauthenticationAdapter` appelle `POST /api/v1/reauthentifications/verification` de MS-IAM avec le jeton de service et retourne `valide` ; `ReauthenticationStubAdapter` (profil `dev`) n'accepte que l'assertion `stub-ok`.

A4. Service de signature réutilisable

Les règles propres aux autres récits s'insèrent comme gardes (`SignatureGuard`, ordonnées par `@Order`) : impartialité (CER-07), fraîcheur du socle (CER-06), désignations (CER-05), préalables du quitus final (CER-04). L'automate du circuit (`CircuitStateMachine`, CER-03) est livré dans le même sprint ; pour ce récit, il rend attendus les quitus sécurité et conformité après un quitus technique favorable.

```
// application/usecase/QuitusSigningService.java
@Service
@RequiredArgsConstructor
public class QuitusSigningService {
    private final CertificationCircuitRepository circuits;
    private final SignedDecisionRepository decisions;
```

```
    private final HomologationReaderPort homologation;
    private final CircuitStateMachine stateMachine;
    private final List<SignatureGuard> guards;
    private final ReauthenticationPort reauthentication;
    private final KeyVaultSigningPort keyVault;
    private final MonitoringAlertPort alerts;
    private final OutboxPublisher outboxPublisher;
    private final ApplicationEventPublisher domainEvents;
    private final Clock clock;

    @Transactional
    public SignedDecision sign(SignQuitusCommand cmd) {
        CertificationCircuit circuit =
circuits.findByIdForUpdate(cmd.circuitId())   // verrou : signatures parallèles
sérialisées
            .orElseThrow(() -> CertificationRuleException.notFound("circuit",
cmd.circuitId()));
        if (circuit.getEtat() != EtatCircuit.OUVERT) {
            throw CertificationRuleException.conflict("CER_CIRCUIT_NOT_OPEN", "Le
circuit n'est pas ouvert.", Map.of("etat", circuit.getEtat()));
        }
        EvaluationBaseSnapshot socle =
homologation.readEvaluationBase(circuit.getLiaisonId())
            .orElseThrow(() ->
CertificationRuleException.unavailable("CER_SOCLE_UNAVAILABLE", "Socle d'évaluation
introuvable."));
        if (cmd.motif() == null || cmd.motif().isBlank()) {
            throw CertificationRuleException.badRequest("CER_MOTIF_REQUIRED", "Le
motif de la décision est obligatoire.",
                cmd.favorable() ? Map.of() : Map.of("operationsValides",
socle.operationCodes(), "servicesValides", socle.serviceCodes()));
        }
        stateMachine.assertSignable(circuit.quitusStates(), cmd.nature());
        SigningContext ctx = new SigningContext(circuit, cmd, socle,
homologation.readParticipants(circuit.getLiaisonId()));
        guards.forEach(g -> g.check(ctx));
        if (cmd.reauthAssertion() == null
                || !reauthentication.verify(cmd.reauthAssertion(),
cmd.signataireId(), "QUITUS_" + cmd.nature())) {
            throw CertificationRuleException.forbidden("CER_REAUTH_FAILED",
"Réauthentification FIDO2 échouée ou expirée.");
        }
        DecisionContent content = DecisionContent.of(circuit, cmd,
clock.instant().truncatedTo(ChronoUnit.MILLIS));
        byte[] digest = DecisionCanonicalizer.sha256(content);
        SignatureResult signature;
        try {
            signature = keyVault.sign("signataire-" + cmd.signataireId(), digest);
        } catch (KeyVaultUnavailableException e) {
            alerts.raise("CER_KEYVAULT_UNAVAILABLE", circuit.getId().toString(),
Map.of("nature", cmd.nature().name()));
            throw CertificationRuleException.unavailable("CER_KEYVAULT_UNAVAILABLE",
"Coffre de clés indisponible : veuillez réessayer.");
        }
        // Aucune écriture avant ce point : un échec en amont ne laisse aucune
décision partielle.
        String empreinte = HexFormat.of().formatHex(digest);
        UUID auditRef = outboxPublisher.publish("Quitus", circuit.getId().toString(),
"certification.quitus-signed.v1",
            QuitusSignedV1.of(circuit, content, empreinte));                //
identifiant CloudEvent de l'entrée d'outbox
        SignedDecision decision =
decisions.save(SignedDecision.of(circuit.quitus(cmd.nature()), content, empreinte,
signature, auditRef));
        Map<NatureQuitus, EtatQuitus> next =
stateMachine.onDecision(circuit.quitusStates(), cmd.nature(), cmd.favorable());
        circuit.applyStates(next, stateMachine.circuitState(next),
content.instant()).forEach(n ->
            outboxPublisher.publish("Notification", circuit.getId().toString(),
```

```
"certification.notification-requested.v1",
                NotificationRequestedV1.toRole(QuitusRoles.roleOf(n),
"CER_QUITUS_ATTENDU", Map.of("circuitId", circuit.getId(), "nature", n))));
        domainEvents.publishEvent(new QuitusSignedDomainEvent(circuit.getId(),
cmd.nature(), cmd.favorable(),
            ctx.participants().chefDeProjetId(), content));
        return decision;
    }
}
```

`CertificationRuleException` porte `code`, statut HTTP et `details` ; le traducteur d'exceptions de Phase 0 la convertit en `ProblemDetail` (RFC 9457) avec `correlationId`. `QuitusSignedV1` reprend : circuitId, liaisonId, institutionId, cycle, nature, decision (`FAVORABLE` ou `DEFAVORABLE`), motif, signataireId, signataireRole, dateSignature, socleEmpreinte, empreinteDecision, operationsDesignees, servicesDesignes.

A5. Ressource REST du quitus technique

```
// web/rest/QuitusResource.java (première version, généralisée en CER-03)
@PostMapping("/circuits/{id}/quitus/technique")
@PreAuthorize("hasAuthority('CER_QUITUS_TECHNIQUE_SIGN')")
public ResponseEntity<SignedDecisionDTO> signTechnique(@PathVariable UUID id, @Valid
@RequestBody SignQuitusRequest body,
        @RequestHeader(name = "X-Reauth-Assertion", required = false) String
assertion, @AuthenticationPrincipal Jwt jwt) {
    SignedDecision d = signing.sign(body.toCommand(id, NatureQuitus.TECHNIQUE,
UUID.fromString(jwt.getSubject()), assertion));
    return ResponseEntity.status(HttpStatus.CREATED).body(decisionMapper.toDto(d));
}

@GetMapping("/circuits/{id}/quitus")
@PreAuthorize("hasAuthority('CER_CIRCUIT_READ')")
public List<QuitusDTO> list(@PathVariable UUID id) { return
quitusQueries.byCircuit(id); }

// web/rest/dto/SignQuitusRequest.java : le motif est contrôlé par le service (code
CER_MOTIF_REQUIRED)
public record SignQuitusRequest(@NotNull Boolean favorable, String motif,
        @NotBlank @Pattern(regexp = "[0-9a-f]{64}") String socleEmpreinte,
        List<String> operationsDesignees, List<String> servicesDesignes, UUID
delegationId) { }
```

L'écran de décision (DES-11, fragment frontend) présente le socle (couvertures, preuves, résultats repris), exige le motif et déclenche `SignDialog`, qui transmet l'assertion dans `X-Reauth-Assertion`.

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
// test : application/usecase/QuitusSigningServiceTest.java
@ExtendWith(MockitoExtension.class)
class QuitusSigningServiceTest {
    @Mock CertificationCircuitRepository circuits; @Mock SignedDecisionRepository
decisions;
    @Mock HomologationReaderPort homologation; @Mock ReauthenticationPort reauth;
@Mock KeyVaultSigningPort keyVault;
    @Mock MonitoringAlertPort alerts; @Mock OutboxPublisher outbox; @Mock
ApplicationEventPublisher domainEvents;
    CertificationCircuit circuit = TestCircuits.open("a".repeat(64));
    QuitusSigningService service;

    @BeforeEach
    void setUp() {
        service = new QuitusSigningService(circuits, decisions, homologation, new
```

```
CircuitStateMachine(), List.of(),
            reauth, keyVault, alerts, outbox, domainEvents, Clock.systemUTC());

lenient().when(circuits.findByIdForUpdate(circuit.getId())).thenReturn(Optional.of(ci
rcuit));

lenient().when(homologation.readEvaluationBase(any())).thenReturn(Optional.of(TestCir
cuits.socle("a".repeat(64))));

lenient().when(homologation.readParticipants(any())).thenReturn(TestCircuits.noPartic
ipants());
    }

    @Test
    @DisplayName("CER-02 : sans réauthentification réussie, ni signature ni
décision")
    void sansReauthentification() {
        when(reauth.verify(eq("expiree"), any(),
eq("QUITUS_TECHNIQUE"))).thenReturn(false);

        assertThatThrownBy(() -> service.sign(TestCircuits.technique(circuit, true,
"Socle conforme", "expiree")))
            .isInstanceOf(CertificationRuleException.class).hasFieldOrPropertyWithVal
ue("code", "CER_REAUTH_FAILED");
        verifyNoInteractions(keyVault, decisions, outbox);

assertThat(circuit.quitus(TECHNIQUE).getEtat()).isEqualTo(EtatQuitus.ATTENDU);
    }

    @Test
    @DisplayName("CER-02 : une demande sans motif est refusée avant toute
réauthentification")
    void motifAbsent() {
        assertThatThrownBy(() -> service.sign(TestCircuits.technique(circuit, true, "
", "stub-ok")))
            .hasFieldOrPropertyWithValue("code", "CER_MOTIF_REQUIRED");
        verifyNoInteractions(reauth, keyVault, decisions);
    }

    @Test
    @DisplayName("CER-02 : la décision est immuable après enregistrement (@Immutable,
aucun mutateur)")
    void decisionImmuable() {

assertThat(SignedDecision.class).hasAnnotation(org.hibernate.annotations.Immutable.cl
ass);

assertThat(Arrays.stream(SignedDecision.class.getMethods()).map(Method::getName)).non
eMatch(n -> n.startsWith("set"));
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
// test : web/rest/QuitusTechniqueIT.java
class QuitusTechniqueIT extends AbstractCertificationIT {
    static final UUID CHEF_DES = UUID.randomUUID();

    private ResultActions signer(CertificationCircuit c) throws Exception {
        return mockMvc.perform(post("/api/v1/circuits/{id}/quitus/technique",
c.getId())
            .with(asUser(CHEF_DES, "CHEF_DES", "CER_QUITUS_TECHNIQUE_SIGN"))
            .header("X-Reauth-Assertion", "assertion-valide")
            .contentType(MediaType.APPLICATION_JSON)
            .content(fixtures.decisionJson(true, "Couvertures à 100 %, preuves
scellées", c.getSocleEmpreinte())));
    }
```

```
    @Test
    @DisplayName("CER-02 : coffre de clés indisponible, aucune décision partielle et
incident signalé")
    void coffreIndisponible() throws Exception {
        CertificationCircuit c = fixtures.openCircuit();   // socle, intervenants et
réauthentification stubés dans WireMock

keyVault.stubFor(post(urlPathMatching("/v1/keys/.*/sign")).willReturn(serviceUnavaila
ble()));


signer(c).andExpect(status().isServiceUnavailable()).andExpect(jsonPath("$.code").val
ue("CER_KEYVAULT_UNAVAILABLE"));

        assertThat(fixtures.decisionCount(c.getId())).isZero();
        assertThat(outboxCount("certification.quitus-signed.v1",
c.getId())).isZero();
        assertThat(outboxCount("certification.alert-raised.v1",
c.getId())).isEqualTo(1);
        assertThat(fixtures.states(c.getId())).containsEntry(TECHNIQUE,
EtatQuitus.ATTENDU);
    }

    @Test
    @DisplayName("CER-02 : quitus technique signé, branches parallèles attendues,
décision non modifiable en base")
    void signatureNominale() throws Exception {
        CertificationCircuit c = fixtures.openCircuit();
        keyVault.stubFor(post(urlPathMatching("/v1/keys/.*/sign"))
            .willReturn(okJson("{\"signatureBase64\":\"c2ln\",\"keyVersion\":\"3\",\"
algorithm\":\"RSASSA_PSS_SHA_256\"}")));


signer(c).andExpect(status().isCreated()).andExpect(jsonPath("$.empreinteDecision").v
alue(matchesPattern("[0-9a-f]{64}")));

        assertThat(fixtures.states(c.getId())).containsEntry(SECURITE,
EtatQuitus.ATTENDU).containsEntry(CONFORMITE, EtatQuitus.ATTENDU);
        assertThat(outboxCount("certification.quitus-signed.v1",
c.getId())).isEqualTo(1);
        assertThatThrownBy(() -> jdbc.update("update signed_decision set motif =
'altéré'"))
            .isInstanceOf(DataAccessException.class).hasMessageContaining("immuable")
;
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: CER-02 Signature du quitus technique après réauthentification FIDO2

  Contexte:
    Étant donné un circuit ouvert pour la liaison "L-EPA-01" dont le quitus
"TECHNIQUE" est "ATTENDU"
    Et le socle d'évaluation présenté au chef du DES (couvertures, preuves, résultats
repris)

  Scénario: Nominal : quitus technique favorable
    Quand le chef du DES choisit une décision favorable avec le motif "Socle complet
et conforme" et réussit la réauthentification FIDO2
    Alors une décision portant la liaison, l'empreinte du socle, la décision, le
motif, le signataire et l'instant est signée par le coffre de clés
    Et la décision est enregistrée de façon immuable
    Et l'événement certification.quitus-signed.v1 est émis
    Et les quitus "SECURITE" et "CONFORMITE" sont "ATTENDU"
```

```
  Scénario: Alternatif : réauthentification échouée
    Étant donné une décision favorable préparée
    Quand la réauthentification FIDO2 échoue ou expire
    Alors la réponse porte le code "CER_REAUTH_FAILED"
    Et aucune signature n'est produite et aucune décision n'est enregistrée

  Scénario: Erreur : motif absent
    Étant donné une décision défavorable préparée
    Quand la signature est demandée sans motif
    Alors la demande est refusée avec le code "CER_MOTIF_REQUIRED"

  Scénario: Erreur : coffre de clés indisponible
    Étant donné une réauthentification réussie
    Quand le coffre de clés ne répond pas
    Alors la réponse a le statut 503 avec le code "CER_KEYVAULT_UNAVAILABLE" invitant
à réessayer
    Et aucune décision partielle n'est enregistrée
    Et l'alerte "CER_KEYVAULT_UNAVAILABLE" est émise vers MS-MONITORING
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-chef-des
# Assertion FIDO2 : produite par l'écran DES-11 ; en recette avec adaptateur de
substitution, valeur "stub-ok"
REAUTH_ASSERTION="${REAUTH_ASSERTION:-stub-ok}"
BASE="${GATEWAY_URL}/bff/des/certification/circuits/${CIRCUIT_ID}"
SOCLE=$(curl -s "${BASE}" -H "Authorization: Bearer ${TOKEN}" | jq -
r .socleEmpreinte)

# Refus : sans réauthentification -> 403 (CER_REAUTH_FAILED)
curl -s -o /dev/null -w "%{http_code}\n" -X POST "${BASE}/quitus/technique" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" \
  -d "{\"favorable\":true,\"motif\":\"Socle conforme\",\"socleEmpreinte\":\"$
{SOCLE}\"}"

# Nominal : quitus technique favorable
curl -s -X POST "${BASE}/quitus/technique" \
  -H "Authorization: Bearer ${TOKEN}" -H "X-Reauth-Assertion: ${REAUTH_ASSERTION}" -H
"Content-Type: application/json" \
  -d "{\"favorable\":true,\"motif\":\"Couvertures à 100 %, preuves
scellées\",\"socleEmpreinte\":\"${SOCLE}\"}" \
  | jq '{decisionFavorable, signataireRole, empreinteDecision, dateSignature}'
curl -s "${BASE}/quitus" -H "Authorization: Bearer ${TOKEN}" | jq '[.[] | {nature,
etat}]'
# Attendu : TECHNIQUE SIGNE_FAVORABLE ; SECURITE et CONFORMITE ATTENDU

# Recette : la décision figure au registre d'audit (consultation MS-AUDIT, AUD-01)
curl -s "${GATEWAY_URL}/bff/des/audit/evenements?subject=$
{CIRCUIT_ID}&type=certification.quitus-signed.v1" \
  -H "Authorization: Bearer ${TOKEN}" | jq '.[0] | {type, subject, time}'
```

CER-03 | Signer en parallèle les quitus sécurité et conformité

Élément	Valeur
Points	5
Priorité	Must Have
Sprint cible	Sprint 4
Écrans	DES-11
Dépendances	CER-02, CER-07

Objectif

Porter l'ordre du circuit dans un automate de domaine pur : technique d'abord, sécurité et conformité en parallèle dans un ordre indifférent, puis Direction Générale après la jonction des deux branches. Une seule ressource `POST /circuits/{id}/quitus/{nature}` dispatche vers le mécanisme de CER-02, avec une permission distincte par nature.

A. Implémentation

A1. Automate du circuit (service de domaine pur)

La table des préalables encode à la fois l'étape séquentielle, les deux branches parallèles et la condition de jonction : une nature devient attendue quand tous ses préalables sont favorables.

```
// domain/service/CircuitStateMachine.java
public final class CircuitStateMachine {
    private static final Map<NatureQuitus, Set<NatureQuitus>> PREALABLES = Map.of(
        TECHNIQUE, Set.of(),
        SECURITE, Set.of(TECHNIQUE),                    // branche parallèle 1
        CONFORMITE, Set.of(TECHNIQUE),                  // branche parallèle 2
        DIRECTION_GENERALE, Set.of(SECURITE, CONFORMITE));   // condition de jonction
    private static final Map<NatureQuitus, String> LIBELLE = Map.of(
        TECHNIQUE, "technique", SECURITE, "sécurité", CONFORMITE, "conformité",
DIRECTION_GENERALE, "Direction Générale");

    public void assertSignable(Map<NatureQuitus, EtatQuitus> states, NatureQuitus
nature) {
        List<NatureQuitus> manquants = PREALABLES.get(nature).stream()
            .filter(p -> states.get(p) != SIGNE_FAVORABLE).sorted().toList();
        if (!manquants.isEmpty()) {
            throw new CircuitRuleViolation("CER_PREREQUISITE_QUITUS",
                "quitus " +
manquants.stream().map(LIBELLE::get).collect(Collectors.joining(" et ")) + "
préalable requis",
                manquants.stream().map(p -> p + " (" + states.get(p) +
")").toList());
        }
        if (states.get(nature) != ATTENDU) {
            throw new CircuitRuleViolation("CER_QUITUS_NOT_EXPECTED",
                "Le quitus " + LIBELLE.get(nature) + " n'est pas attendu.",
List.of(nature + " (" + states.get(nature) + ")"));
        }
    }

    public Map<NatureQuitus, EtatQuitus> onDecision(Map<NatureQuitus, EtatQuitus>
states, NatureQuitus nature, boolean favourable) {
        EnumMap<NatureQuitus, EtatQuitus> next = new EnumMap<>(states);
        next.put(nature, favourable ? SIGNE_FAVORABLE : SIGNE_DEFAVORABLE);
        return favourable ? promote(next) : cancelPending(next);   // cancelPending :
```

```
CER-05
    }

    /** Rend attendue toute nature non encore requise dont les préalables sont tous
favorables. */
    public Map<NatureQuitus, EtatQuitus> promote(Map<NatureQuitus, EtatQuitus>
states) {
        EnumMap<NatureQuitus, EtatQuitus> next = new EnumMap<>(states);
        PREALABLES.forEach((n, pre) -> {
            if (next.get(n) == NON_REQUIS && pre.stream().allMatch(p -> next.get(p)
== SIGNE_FAVORABLE)) {
                next.put(n, ATTENDU);
            }
        });
        return next;
    }

    public EtatCircuit circuitState(Map<NatureQuitus, EtatQuitus> states) {
        if (states.containsValue(SIGNE_DEFAVORABLE)) return EtatCircuit.INTERROMPU;
        return states.get(DIRECTION_GENERALE) == SIGNE_FAVORABLE ? EtatCircuit.ACHEVE
: EtatCircuit.OUVERT;
    }
}
```

Le traducteur d'exceptions convertit `CircuitRuleViolation` en 409 avec `code`, `message` et `details.manquants`.

A2. Application des états au circuit

```
// domain/model/CertificationCircuit.java (ajout CER-03)
public Map<NatureQuitus, EtatQuitus> quitusStates() {
    EnumMap<NatureQuitus, EtatQuitus> m = new EnumMap<>(NatureQuitus.class);
    quitus.forEach(q -> m.put(q.getNature(), q.getEtat()));
    return m;
}

/** Applique les états calculés par l'automate ; renvoie les natures nouvellement
attendues (à notifier). */
public Set<NatureQuitus> applyStates(Map<NatureQuitus, EtatQuitus> target,
EtatCircuit etatCible, Instant now) {
    Set<NatureQuitus> nouvellementAttendues = EnumSet.noneOf(NatureQuitus.class);
    quitus.forEach(q -> {
        EtatQuitus cible = target.get(q.getNature());
        if (cible == EtatQuitus.ATTENDU && q.getEtat() != EtatQuitus.ATTENDU)
nouvellementAttendues.add(q.getNature());
        q.changeState(cible, now);
    });
    if (etatCible != EtatCircuit.OUVERT && etat == EtatCircuit.OUVERT) { etat =
etatCible; dateFin = now; }
    return nouvellementAttendues;
}
```

Le verrou pessimiste pris par `findByIdForUpdate` (CER-02) sérialise deux signatures simultanées sur le même circuit : la seconde lit l'état produit par la première, si bien que la jonction ne peut être ni manquée ni déclenchée deux fois. Chaque nature rendue attendue donne lieu à une demande de notification de son rôle signataire (code dans CER-02).

A3. Permissions par nature et ressource unique

```
// infrastructure/security/QuitusPermissions.java
@Component("quitusPermissions")
public class QuitusPermissions {
    /** Vrai si le jeton porte la permission propre à la nature demandée ; une nature
inconnue est refusée. */
    public boolean canSign(Authentication auth, String slug) {
```

```
        return NatureQuitusPath.parse(slug)
            .map(n -> auth.getAuthorities().stream().anyMatch(a ->
a.getAuthority().equals(QuitusRoles.permissionOf(n))))
            .orElse(false);
    }
}

// web/rest/NatureQuitusPath.java
final class NatureQuitusPath {
    static Optional<NatureQuitus> parse(String slug) {
        return switch (slug) {
            case "technique" -> Optional.of(TECHNIQUE);
            case "securite" -> Optional.of(SECURITE);
            case "conformite" -> Optional.of(CONFORMITE);
            case "direction-generale" -> Optional.of(DIRECTION_GENERALE);
            default -> Optional.empty();
        };
    }
}

// web/rest/QuitusResource.java (remplace la méthode dédiée au quitus technique de
CER-02)
@PostMapping("/circuits/{id}/quitus/{nature}")
@PreAuthorize("@quitusPermissions.canSign(authentication, #nature)")
public ResponseEntity<SignedDecisionDTO> sign(@PathVariable UUID id, @PathVariable
String nature,
        @Valid @RequestBody SignQuitusRequest body,
        @RequestHeader(name = "X-Reauth-Assertion", required = false) String
assertion,
        @AuthenticationPrincipal Jwt jwt) {
    NatureQuitus n = NatureQuitusPath.parse(nature).orElseThrow();   // garanti par
@PreAuthorize
    SignedDecision d = signing.sign(body.toCommand(id, n,
UUID.fromString(jwt.getSubject()), assertion));
    return ResponseEntity.status(HttpStatus.CREATED).body(decisionMapper.toDto(d));
}
```

La politique OPA de la passerelle applique la même correspondance (chemin `/bff/des/certification/circuits/*/quitus/securite` exige `CER_QUITUS_SECURITE_SIGN`, etc.) ; le service la revérifie par `@PreAuthorize`.

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
// test : domain/service/CircuitStateMachineTest.java
class CircuitStateMachineTest {
    final CircuitStateMachine machine = new CircuitStateMachine();

    static Map<NatureQuitus, EtatQuitus> etats(EtatQuitus t, EtatQuitus s, EtatQuitus
c, EtatQuitus dg) {
        return new EnumMap<>(Map.of(TECHNIQUE, t, SECURITE, s, CONFORMITE, c,
DIRECTION_GENERALE, dg));
    }

    @Test
    @DisplayName("CER-03 : la Direction Générale n'est attendue qu'à la jonction des
deux branches parallèles")
    void jonction() {
        var apresSecurite = machine.onDecision(etats(SIGNE_FAVORABLE, ATTENDU,
ATTENDU, NON_REQUIS), SECURITE, true);
        assertThat(apresSecurite).containsEntry(CONFORMITE,
ATTENDU).containsEntry(DIRECTION_GENERALE, NON_REQUIS);

        var apresConformite = machine.onDecision(apresSecurite, CONFORMITE, true);
        assertThat(apresConformite).containsEntry(DIRECTION_GENERALE, ATTENDU);

assertThat(machine.circuitState(apresConformite)).isEqualTo(EtatCircuit.OUVERT);
```

```
    }

    @Test
    @DisplayName("CER-03 : l'ordre entre sécurité et conformité est indifférent")
    void ordreIndifferent() {
        var conformiteDabord = machine.onDecision(etats(SIGNE_FAVORABLE, ATTENDU,
ATTENDU, NON_REQUIS), CONFORMITE, true);
        assertThatNoException().isThrownBy(() ->
machine.assertSignable(conformiteDabord, SECURITE));
        assertThat(machine.onDecision(conformiteDabord, SECURITE,
true)).containsEntry(DIRECTION_GENERALE, ATTENDU);
    }

    @Test
    @DisplayName("CER-03 : la signature sécurité est refusée avant le quitus
technique")
    void refusAvantTechnique() {
        assertThatThrownBy(() -> machine.assertSignable(etats(ATTENDU, NON_REQUIS,
NON_REQUIS, NON_REQUIS), SECURITE))
            .isInstanceOf(CircuitRuleViolation.class)
            .hasMessage("quitus technique préalable requis")
            .hasFieldOrPropertyWithValue("code", "CER_PREREQUISITE_QUITUS");
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
// test : web/rest/QuitusParallelesIT.java
class QuitusParallelesIT extends AbstractCertificationIT {
    static final UUID RSSI = UUID.randomUUID();
    static final UUID CONFORMITE_ID = UUID.randomUUID();

    @Test
    @DisplayName("CER-03 : cloisonnement des permissions par nature, puis jonction
vers la Direction Générale")
    void cloisonnementEtJonction() throws Exception {
        CertificationCircuit c = fixtures.circuitWithFavourable(TECHNIQUE);   //
coffre et réauthentification stubés
        String corps = fixtures.decisionJson(true, "Volet traité",
c.getSocleEmpreinte());

        mockMvc.perform(post("/api/v1/circuits/{id}/quitus/conformite", c.getId())
                .with(asUser(RSSI, "RSSI", "CER_QUITUS_SECURITE_SIGN")).header("X-
Reauth-Assertion", "ok")
                .contentType(MediaType.APPLICATION_JSON).content(corps))
            .andExpect(status().isForbidden());

        mockMvc.perform(post("/api/v1/circuits/{id}/quitus/securite", c.getId())
                .with(asUser(RSSI, "RSSI", "CER_QUITUS_SECURITE_SIGN")).header("X-
Reauth-Assertion", "ok")
                .contentType(MediaType.APPLICATION_JSON).content(corps))
            .andExpect(status().isCreated());
        assertThat(fixtures.states(c.getId())).containsEntry(CONFORMITE,
ATTENDU).containsEntry(DIRECTION_GENERALE, NON_REQUIS);

        mockMvc.perform(post("/api/v1/circuits/{id}/quitus/conformite", c.getId())
                .with(asUser(CONFORMITE_ID, "RESPONSABLE_CONFORMITE",
"CER_QUITUS_CONFORMITE_SIGN"))
                .header("X-Reauth-Assertion",
"ok").contentType(MediaType.APPLICATION_JSON).content(corps))
            .andExpect(status().isCreated());
        assertThat(fixtures.states(c.getId())).containsEntry(DIRECTION_GENERALE,
ATTENDU);
        assertThat(jdbc.queryForObject("select count(*) from outbox_event where
event_type = 'certification.notification-requested.v1'"
            + " and aggregate_id = ? and payload->>'role' = 'DIRECTION_GENERALE'",
Long.class, c.getId().toString())).isEqualTo(1);
```

```
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: CER-03 Signature en parallèle des quitus sécurité et conformité

  Scénario: Nominal : deux quitus en parallèle
    Étant donné un circuit dont le quitus "TECHNIQUE" est "SIGNE_FAVORABLE"
    Quand le RSSI signe le quitus "securite" favorable alors que le quitus
"CONFORMITE" est encore "ATTENDU"
    Alors le quitus "SECURITE" est "SIGNE_FAVORABLE" et le circuit attend le quitus
"CONFORMITE"
    Et le quitus "DIRECTION_GENERALE" est "NON_REQUIS"
    Quand le responsable conformité signe le quitus "conformite" favorable
    Alors le quitus "DIRECTION_GENERALE" est "ATTENDU"

  Scénario: Alternatif : conformité signée en premier
    Étant donné un circuit dont le quitus "TECHNIQUE" est "SIGNE_FAVORABLE"
    Quand le responsable conformité signe le quitus "conformite" favorable avant le
RSSI
    Alors le quitus "CONFORMITE" est "SIGNE_FAVORABLE"
    Et le RSSI peut ensuite signer le quitus "securite"

  Scénario: Erreur : quitus technique non acquis
    Étant donné un circuit dont le quitus "TECHNIQUE" est "ATTENDU"
    Quand le RSSI tente de signer le quitus "securite"
    Alors la signature est refusée avec le statut 409 et le message "quitus technique
préalable requis"

  Scénario: Erreur : quitus hors de la fonction du signataire
    Étant donné un RSSI authentifié portant la seule permission
"CER_QUITUS_SECURITE_SIGN"
    Quand il tente de signer le quitus "conformite"
    Alors la signature est refusée avec le statut 403
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-rssi
BASE="${GATEWAY_URL}/bff/des/certification/circuits/${CIRCUIT_ID}"
SOCLE=$(curl -s "${BASE}" -H "Authorization: Bearer ${TOKEN}" | jq -
r .socleEmpreinte)
BODY="{\"favorable\":true,\"motif\":\"Tests de sécurité
conformes\",\"socleEmpreinte\":\"${SOCLE}\"}"

# Refus : le RSSI ne peut pas signer le quitus conformité -> 403
curl -s -o /dev/null -w "%{http_code}\n" -X POST "${BASE}/quitus/conformite" \
  -H "Authorization: Bearer ${TOKEN}" -H "X-Reauth-Assertion: ${REAUTH_ASSERTION:-
stub-ok}" \
  -H "Content-Type: application/json" -d "${BODY}"

# Nominal : quitus sécurité, puis état du circuit
curl -s -X POST "${BASE}/quitus/securite" \
  -H "Authorization: Bearer ${TOKEN}" -H "X-Reauth-Assertion: ${REAUTH_ASSERTION:-
stub-ok}" \
  -H "Content-Type: application/json" -d "${BODY}" | jq '{signataireRole,
decisionFavorable}'

source ./scripts/token.sh test-conformite
curl -s -X POST "${BASE}/quitus/conformite" \
  -H "Authorization: Bearer ${TOKEN}" -H "X-Reauth-Assertion: ${REAUTH_ASSERTION:-
stub-ok}" \
  -H "Content-Type: application/json" -d "${BODY/Tests de sécurité conformes/Pièces
légales vérifiées}" | jq .signataireRole
```

```
curl -s "${BASE}/quitus" -H "Authorization: Bearer ${TOKEN}" | jq '[.[] | {nature,
etat}]'
# Attendu : DIRECTION_GENERALE ATTENDU
```

CER-04 | Signer le quitus de la Direction Générale et autoriser la mise en service

Élément	Valeur
Points	5
Priorité	Must Have
Sprint cible	Sprint 4
Écrans	DES-11
Dépendances	CER-03, CER-08, CER-09

Objectif

Garder le quitus final par ses trois préalables (favorables et non caducs), présenter à la Direction Générale le dossier de décision, puis émettre `certification.go-live-authorized.v1` et faire passer le dossier à l'état `AUTORISE` dès que le quitus final et une fiche de changement approuvée, relue depuis moins de quinze minutes, sont réunis : immédiatement après la signature, ou plus tard, à la relecture de la fiche, sans nouvelle signature.

A. Implémentation

A1. Garde des préalables du quitus final

La garde s'insère dans le mécanisme de signature de CER-02. Elle nomme chaque quitus manquant avec son état, ce qui couvre le cas d'un quitus devenu caduc (CER-06).

```
// application/usecase/guard/GoLivePrerequisitesGuard.java
@Component
@Order(40)
class GoLivePrerequisitesGuard implements SignatureGuard {
    private static final List<NatureQuitus> PREALABLES_DG = List.of(TECHNIQUE,
SECURITE, CONFORMITE);

    @Override
    public void check(SigningContext ctx) {
        if (ctx.command().nature() != DIRECTION_GENERALE) {
            return;
        }
        Map<NatureQuitus, EtatQuitus> states = ctx.circuit().quitusStates();
        List<Map<String, String>> manquants = PREALABLES_DG.stream()
            .filter(n -> states.get(n) != EtatQuitus.SIGNE_FAVORABLE)
            .map(n -> Map.of("nature", n.name(), "etat",
states.get(n).name())).toList();
        if (!manquants.isEmpty()) {
            throw CertificationRuleException.conflict("CER_PREREQUISITE_QUITUS",
                "Quitus préalable absent ou caduc : " + manquants.stream()
                    .map(m -> m.get("nature") + " " +
m.get("etat")).collect(Collectors.joining(", ")),
                Map.of("manquants", manquants));
        }
    }
}
```

A2. Émission de l'autorisation à la réunion du quitus final et de la fiche approuvée

La décision de la Direction Générale est acquise dès la validation de sa transaction, même si la fiche n'est pas lisible à cet instant. L'autorisation est donc tentée après validation (`AFTER_COMMIT`, nouvelle transaction), puis retentée par une tâche planifiée pour les dossiers

en attente. Choix explicite : seule une lecture **approuvée** de moins de quinze minutes est réutilisée ; une fiche non approuvée est relue à chaque tentative, afin que l'approbation soit constatée au plus tard au passage suivant de la tâche. Le port `ChangeRecordReaderPort` est celui de CER-09 ; il est seulement appelé ici.

```
// application/usecase/GoLiveAuthorizationService.java
@Service
@RequiredArgsConstructor
public class GoLiveAuthorizationService {
    public enum Outcome { AUTHORIZED, DEFERRED, NOT_APPLICABLE }
    static final Duration FRAICHEUR = Duration.ofMinutes(15);

    private final GoLiveDossierRepository dossiers;
    private final ChangeRecordSnapshotRepository snapshots;
    private final ChangeRecordReaderPort changeRecordReader;   // CER-09, lecture
seule
    private final DossierStateMachine dossierStateMachine;     // CER-08
    private final OutboxPublisher outboxPublisher;
    private final Clock clock;

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public Outcome tryAuthorize(UUID dossierId) {
        GoLiveDossier dossier = dossiers.findByIdForUpdate(dossierId).orElseThrow();
        CertificationCircuit circuit = dossier.getCircuitCourant();
        if (dossier.getEtat() != EtatDossier.OUVERT || circuit.getEtat() !=
EtatCircuit.ACHEVE) {
            return Outcome.NOT_APPLICABLE;
        }
        Optional<ChangeRecordSnapshot> fiche = freshApprovedRecord(dossier);
        if (fiche.isEmpty()) {
            return Outcome.DEFERRED;          // fiche absente, non approuvée ou
illisible : aucune valeur par défaut
        }
        Instant now = clock.instant();
        dossier.transitionTo(EtatDossier.AUTORISE, dossierStateMachine, now);
        outboxPublisher.publish("GoLiveDossier", dossierId.toString(),
"certification.go-live-authorized.v1",
            new GoLiveAuthorizedV1(dossierId, dossier.getLiaisonId(),
dossier.getInstitutionId(), circuit.getCycle(),
                circuit.getSocleEmpreinte(), fiche.get().getReference(),
fiche.get().getFenetreDebut(),
                fiche.get().getFenetreFin(), now));
        return Outcome.AUTHORIZED;
    }

    private Optional<ChangeRecordSnapshot> freshApprovedRecord(GoLiveDossier dossier)
{
        if (dossier.getReferenceFicheChangement() == null) {
            return Optional.empty();
        }
        Optional<ChangeRecordSnapshot> last =
snapshots.findFirstByDossierIdOrderByDateLectureDesc(dossier.getId())
            .filter(s -> s.isApproved() &&
s.getDateLecture().isAfter(clock.instant().minus(FRAICHEUR)));
        if (last.isPresent()) {
            return last;
        }
        try {
            ChangeRecordView view =
changeRecordReader.read(dossier.getReferenceFicheChangement());
            ChangeRecordSnapshot relue =
snapshots.save(ChangeRecordSnapshot.of(dossier, view, clock.instant()));
            return relue.isApproved() ? Optional.of(relue) : Optional.empty();
        } catch (ChangeRecordUnavailableException e) {
            return Optional.empty();
        }
    }
}
```

```
// application/usecase/GoLiveAuthorizationTrigger.java
@Component
@RequiredArgsConstructor
class GoLiveAuthorizationTrigger {
    private final GoLiveAuthorizationService authorization;
    private final GoLiveDossierRepository dossiers;

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    void onQuitusSigned(QuitusSignedDomainEvent e) {
        if (e.nature() == DIRECTION_GENERALE && e.favorable()) {

dossiers.findIdByCircuitCourantId(e.circuitId()).ifPresent(authorization::tryAuthoriz
e);
        }
    }

    /** Autorisations différées : dossiers ouverts dont le circuit est achevé. */
    @Scheduled(fixedDelayString = "${gimac.certification.pending-authorization-
delay:PT5M}")
    void retryPending() {
        dossiers.findIdsAwaitingAuthorization().forEach(authorization::tryAuthorize);
    }
}
```

`certification.go-live-authorized.v1` est consommé par MS-HOMOLOGATION (HOM-19), MS-PARTICIPANT et MS-NOTIFICATION, qui informe la production monétique du GIMAC ; la plateforme n'agit jamais elle-même sur la production. CER-09 appelle aussi `tryAuthorize` après chaque relecture réussie d'une fiche.

A3. Dossier de décision présenté à la Direction Générale

```
// web/rest/DossierResource.java (extrait CER-04)
@GetMapping("/dossiers/{id}/decision")
@PreAuthorize("hasAuthority('CER_DOSSIER_READ')")
public DecisionFileDTO decision(@PathVariable UUID id, Authentication auth) {
    return decisionFiles.build(id, auth);
}

// web/rest/dto/DecisionFileDTO.java
public record DecisionFileDTO(UUID dossierId, EtatDossier etat, String
socleEmpreinte,
        List<QuitusSummary> quitus,              // nature, état, rôle signataire,
date, empreinte de la décision
        ChangeRecordSummary fiche,               // référence, état d'approbation,
fenêtre, instant de lecture
        boolean ficheFraiche, List<String> actionsPermises) { }   //
SIGNER_DIRECTION_GENERALE si les préalables sont réunis
```

L'écran DES-11 affiche ce dossier avant d'ouvrir `SignDialog` ; la signature passe par `POST /circuits/{id}/quitus/direction-generale` (CER-03).

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
// test : application/usecase/guard/GoLivePrerequisitesGuardTest.java
class GoLivePrerequisitesGuardTest {
    @Test
    @DisplayName("CER-04 : le quitus final est refusé en présence d'un quitus
conformité caduc, nommé dans le refus")
    void refusSurQuitusCaduc() {
        SigningContext ctx = TestCircuits.context(DIRECTION_GENERALE,
            Map.of(TECHNIQUE, SIGNE_FAVORABLE, SECURITE, SIGNE_FAVORABLE, CONFORMITE,
CADUC, DIRECTION_GENERALE, ATTENDU));
```

```
        assertThatThrownBy(() -> new GoLivePrerequisitesGuard().check(ctx))
            .isInstanceOf(CertificationRuleException.class)
            .hasMessageContaining("CONFORMITE CADUC")
            .hasFieldOrPropertyWithValue("code", "CER_PREREQUISITE_QUITUS");
    }
}

// test : application/usecase/GoLiveAuthorizationServiceTest.java
@ExtendWith(MockitoExtension.class)
class GoLiveAuthorizationServiceTest {
    @Mock GoLiveDossierRepository dossiers; @Mock ChangeRecordSnapshotRepository
snapshots;
    @Mock ChangeRecordReaderPort reader; @Mock OutboxPublisher outbox;
    GoLiveAuthorizationService service;
    GoLiveDossier dossier = TestCircuits.dossierWithAchievedCircuit("CHG-2026-0042");

    @BeforeEach
    void setUp() {
        service = new GoLiveAuthorizationService(dossiers, snapshots, reader, new
DossierStateMachine(), outbox, Clock.systemUTC());

when(dossiers.findByIdForUpdate(dossier.getId())).thenReturn(Optional.of(dossier));
        when(snapshots.save(any())).thenAnswer(i -> i.getArgument(0));
    }

    @Test
    @DisplayName("CER-04 : fiche en attente d'approbation, autorisation différée sans
événement")
    void autorisationDifferee() {
        when(reader.read("CHG-2026-
0042")).thenReturn(TestCircuits.fiche("EN_ATTENTE"));

assertThat(service.tryAuthorize(dossier.getId())).isEqualTo(Outcome.DEFERRED);
        assertThat(dossier.getEtat()).isEqualTo(EtatDossier.OUVERT);
        verifyNoInteractions(outbox);
    }

    @Test
    @DisplayName("CER-04 : fiche relue approuvée, dossier autorisé et événement émis
sans nouvelle signature")
    void autorisationEmise() {
        when(reader.read("CHG-2026-
0042")).thenReturn(TestCircuits.fiche("APPROUVEE"));

assertThat(service.tryAuthorize(dossier.getId())).isEqualTo(Outcome.AUTHORIZED);
        assertThat(dossier.getEtat()).isEqualTo(EtatDossier.AUTORISE);
        verify(outbox).publish(eq("GoLiveDossier"), eq(dossier.getId().toString()),
eq("certification.go-live-authorized.v1"), any());
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

Le port de CER-09 est simulé par Mockito dans ce test, afin de piloter l'état d'approbation de la fiche entre deux lectures.

```
// test : application/usecase/GoLiveAuthorizationIT.java
class GoLiveAuthorizationIT extends AbstractCertificationIT {
    @MockitoBean ChangeRecordReaderPort changeRecords;
    @Autowired GoLiveAuthorizationTrigger trigger;
    static final UUID DG = UUID.randomUUID();

    @Test
    @DisplayName("CER-04 : l'autorisation est différée jusqu'à l'approbation de la
fiche, puis émise sans nouvelle signature")
    void autorisationDifferee JusquaApprobation() throws Exception {
        CertificationCircuit c = fixtures.circuitWithFavourable(TECHNIQUE, SECURITE,
```

```
CONFORMITE);
        UUID dossierId = fixtures.dossierOf(c, "CHG-2026-0042");
        when(changeRecords.read("CHG-2026-
0042")).thenReturn(fixtures.fiche("EN_ATTENTE"), fixtures.fiche("APPROUVEE"));

        mockMvc.perform(post("/api/v1/circuits/{id}/quitus/direction-generale",
c.getId())
                .with(asUser(DG, "DIRECTION_GENERALE",
"CER_QUITUS_DG_SIGN")).header("X-Reauth-Assertion", "ok")
                .contentType(MediaType.APPLICATION_JSON)
                .content(fixtures.decisionJson(true, "Mise en service autorisée",
c.getSocleEmpreinte())))
            .andExpect(status().isCreated());
        assertThat(fixtures.dossierState(dossierId)).isEqualTo(EtatDossier.OUVERT);
        assertThat(outboxCount("certification.go-live-authorized.v1",
dossierId)).isZero();

        trigger.retryPending();                          // passage de la tâche
planifiée : relecture approuvée

        assertThat(fixtures.dossierState(dossierId)).isEqualTo(EtatDossier.AUTORISE);
        assertThat(outboxCount("certification.go-live-authorized.v1",
dossierId)).isEqualTo(1);
        assertThat(fixtures.decisionCount(c.getId())).isEqualTo(4);   // aucune
signature supplémentaire
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: CER-04 Quitus de la Direction Générale et autorisation de mise en
service

  Scénario: Nominal : autorisation de mise en service
    Étant donné un dossier présentant les quitus "TECHNIQUE", "SECURITE" et
"CONFORMITE" favorables
    Et une fiche de changement "CHG-2026-0042" approuvée relue depuis moins de quinze
minutes
    Quand la Direction Générale signe un quitus favorable avec le motif "Mise en
service autorisée" après réauthentification FIDO2
    Alors le dossier passe à l'état "AUTORISE"
    Et l'événement certification.go-live-authorized.v1 est émis à destination de MS-
HOMOLOGATION, MS-PARTICIPANT et MS-NOTIFICATION

  Scénario: Alternatif : fiche de changement non encore approuvée
    Étant donné un quitus "DIRECTION_GENERALE" favorable et une fiche de changement
encore en attente d'approbation
    Et le dossier est à l'état "OUVERT"
    Quand la fiche est relue à l'état approuvée
    Alors l'autorisation de mise en service est émise sans nouvelle signature
    Et le dossier passe à l'état "AUTORISE"

  Scénario: Erreur : quitus préalable absent ou caduc
    Étant donné un dossier dont le quitus "CONFORMITE" est "CADUC"
    Quand la Direction Générale tente de signer
    Alors la signature est refusée avec le code "CER_PREREQUISITE_QUITUS"
    Et le refus nomme le quitus "CONFORMITE"
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-dg
BASE="${GATEWAY_URL}/bff/des/certification"
curl -s "${BASE}/dossiers/${DOSSIER_ID}/decision" -H "Authorization: Bearer ${TOKEN}"
\
```

```
  | jq '{etat, quitus: [.quitus[] | {nature, etat}], fiche: .fiche.etatApprobation,
ficheFraiche, actionsPermises}'
SOCLE=$(curl -s "${BASE}/dossiers/${DOSSIER_ID}/decision" -H "Authorization: Bearer $
{TOKEN}" | jq -r .socleEmpreinte)

# Nominal (recette complète des quatre quitus) : quitus final, puis état du dossier
curl -s -X POST "${BASE}/circuits/${CIRCUIT_ID}/quitus/direction-generale" \
  -H "Authorization: Bearer ${TOKEN}" -H "X-Reauth-Assertion: ${REAUTH_ASSERTION:-
stub-ok}" \
  -H "Content-Type: application/json" \
  -d "{\"favorable\":true,\"motif\":\"Mise en service
autorisée\",\"socleEmpreinte\":\"${SOCLE}\"}" | jq '{decisionFavorable,
dateSignature}'
curl -s "${BASE}/dossiers/${DOSSIER_ID}" -H "Authorization: Bearer ${TOKEN}" | jq
'{etat, dateAutorisation}'
# Attendu : "AUTORISE" si la fiche est approuvée, sinon "OUVERT" jusqu'à la relecture
approuvée

# Refus : circuit dont le quitus conformité est caduc -> 409
(CER_PREREQUISITE_QUITUS)
curl -s -o /dev/null -w "%{http_code}\n" -X POST "${BASE}/circuits/$
{CIRCUIT_ID_CADUC}/quitus/direction-generale" \
  -H "Authorization: Bearer ${TOKEN}" -H "X-Reauth-Assertion: ${REAUTH_ASSERTION:-
stub-ok}" \
  -H "Content-Type: application/json" -d
"{\"favorable\":true,\"motif\":\"Essai\",\"socleEmpreinte\":\"${SOCLE}\"}"
```

CER-05 | Refuser un quitus et renvoyer la liaison en homologation

Élément	Valeur
Points	3
Priorité	Must Have
Sprint cible	Sprint 4
Écrans	DES-11
Dépendances	CER-02, HOM-19

Objectif

Permettre à tout signataire du circuit d'émettre une décision défavorable signée, motivée et assortie de désignations d'opérations ou de services contrôlées contre le socle d'évaluation. Le refus annule les signatures non encore acquises, interrompt le circuit, laisse le dossier ouvert et renvoie la liaison en homologation par l'événement `certification.quitus-signed.v1` consommé par MS-HOMOLOGATION (HOM-19).

A. Implémentation

A1. Contrôle des désignations sur le socle d'évaluation

Choix explicites : les désignations sont facultatives (un refus peut porter sur une pièce extérieure au socle, cas nominal de CER-06), mais tout code désigné doit appartenir au socle ; une décision favorable ne désigne rien. Le motif est déjà exigé par le mécanisme de CER-02, qui renvoie, pour un refus, la liste des codes valides dans `details`.

```
// application/usecase/guard/DesignationGuard.java
@Component
@Order(30)
class DesignationGuard implements SignatureGuard {
    @Override
    public void check(SigningContext ctx) {
        SignQuitusCommand cmd = ctx.command();
        if (cmd.favorable()) {
            if (!cmd.operationsDesignees().isEmpty() || !
cmd.servicesDesignes().isEmpty()) {
                throw
CertificationRuleException.badRequest("CER_DESIGNATION_UNEXPECTED",
                    "Une décision favorable ne désigne aucune reprise.", Map.of());
            }
            return;
        }
        EvaluationBaseSnapshot socle = ctx.socle();
        List<String> invalides = Stream.concat(
                cmd.operationsDesignees().stream().filter(c -> !
socle.operationCodes().contains(c)),
                cmd.servicesDesignes().stream().filter(c -> !
socle.serviceCodes().contains(c)))
            .toList();
        if (!invalides.isEmpty()) {
            throw CertificationRuleException.conflict("CER_DESIGNATION_INVALID",
                "Codes absents du socle d'évaluation : " + invalides,
                Map.of("codesInvalides", invalides, "operationsValides",
socle.operationCodes(),
                       "servicesValides", socle.serviceCodes()));
        }
    }
```

```
}
```

A2. Annulation des signatures attendues

Ajout à l'automate de CER-03 : toute signature non encore acquise est annulée, les décisions favorables restent en l'état (elles seront soumises au contrôle de caducité lors de la nouvelle aptitude, CER-06). `circuitState` rend alors `INTERROMPU`. Aucune transition n'est appliquée au dossier, qui reste `OUVERT` et sera repris par le cycle suivant (CER-01).

```
// domain/service/CircuitStateMachine.java (ajout CER-05)
Map<NatureQuitus, EtatQuitus> cancelPending(EnumMap<NatureQuitus, EtatQuitus> next) {
    next.replaceAll((n, e) -> e == ATTENDU || e == NON_REQUIS ? ANNULE : e);
    return next;
}
```

A3. Notification du chef de projet de la liaison

La notification est écrite dans la transaction de la décision (`BEFORE_COMMIT`) : elle n'existe que si la décision est validée. Le chef de projet provient des intervenants lus par le mécanisme de signature.

```
// application/usecase/UnfavourableDecisionNotifier.java
@Component
@RequiredArgsConstructor
class UnfavourableDecisionNotifier {
    private final OutboxPublisher outboxPublisher;

    @TransactionalEventListener(phase = TransactionPhase.BEFORE_COMMIT)
    void onQuitusSigned(QuitusSignedDomainEvent e) {
        if (e.favorable()) {
            return;
        }
        outboxPublisher.publish("Notification", e.circuitId().toString(),
"certification.notification-requested.v1",
            NotificationRequestedV1.toUser(e.chefDeProjetId(), "CER_QUITUS_REFUSE",
Map.of(
                "circuitId", e.circuitId(), "nature", e.nature(), "motif",
e.content().motif(),
                "operations", e.content().operationsDesignees(), "services",
e.content().servicesDesignes())));
    }
}
```

La ressource est celle de CER-03 (`POST /circuits/{id}/quitus/{nature}` avec `"favorable": false`) ; la charge utile défavorable de `certification.quitus-signed.v1` porte `decision = "DEFAVORABLE"`, le motif et les listes `operationsDesignees` et `servicesDesignes`, que HOM-19 utilise pour rouvrir les opérations désignées.

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
// test : application/usecase/QuitusRefusalTest.java
@ExtendWith(MockitoExtension.class)
class QuitusRefusalTest {
    @Mock CertificationCircuitRepository circuits; @Mock SignedDecisionRepository
decisions;
    @Mock HomologationReaderPort homologation; @Mock ReauthenticationPort reauth;
@Mock KeyVaultSigningPort keyVault;
    @Mock MonitoringAlertPort alerts; @Mock OutboxPublisher outbox; @Mock
ApplicationEventPublisher domainEvents;
    CertificationCircuit circuit = TestCircuits.withStates(SIGNE_FAVORABLE, ATTENDU,
ATTENDU, NON_REQUIS);
```

```
    @Test
    @DisplayName("CER-05 : un refus sans motif est rejeté et propose les codes
valides du socle")
    void refusSansMotif() {

when(circuits.findByIdForUpdate(circuit.getId())).thenReturn(Optional.of(circuit));

when(homologation.readEvaluationBase(any())).thenReturn(Optional.of(TestCircuits.socl
e(circuit.getSocleEmpreinte())));
        QuitusSigningService service = new QuitusSigningService(circuits, decisions,
homologation, new CircuitStateMachine(),
            List.of(new DesignationGuard()), reauth, keyVault, alerts, outbox,
domainEvents, Clock.systemUTC());

        assertThatThrownBy(() -> service.sign(TestCircuits.refusal(circuit, SECURITE,
null, List.of("OP-12"))))
            .isInstanceOf(CertificationRuleException.class)
            .hasFieldOrPropertyWithValue("code", "CER_MOTIF_REQUIRED")
            .extracting("details").asInstanceOf(InstanceOfAssertFactories.MAP).contai
nsKey("operationsValides");
        verifyNoInteractions(keyVault, decisions);
    }

    @Test
    @DisplayName("CER-05 : un refus annule les signatures non acquises et interrompt
le circuit")
    void annulationDesSignaturesAttendues() {
        CircuitStateMachine machine = new CircuitStateMachine();
        var apres = machine.onDecision(circuit.quitusStates(), SECURITE, false);
        assertThat(apres).containsExactlyInAnyOrderEntriesOf(Map.of(
            TECHNIQUE, SIGNE_FAVORABLE, SECURITE, SIGNE_DEFAVORABLE, CONFORMITE,
ANNULE, DIRECTION_GENERALE, ANNULE));
        assertThat(machine.circuitState(apres)).isEqualTo(EtatCircuit.INTERROMPU);
    }

    @Test
    @DisplayName("CER-05 : la charge utile d'une décision défavorable respecte le
contrat publié")
    void contratChargeUtileDefavorable() throws Exception {
        ObjectMapper mapper = new ObjectMapper().findAndRegisterModules();
        JsonSchema schema =
JsonSchemaFactory.getInstance(SpecVersion.VersionFlag.V202012)
            .getSchema(getClass().getResourceAsStream("/contracts/certification/
quitus-signed.v1.schema.json"));
        QuitusSignedV1 payload = QuitusSignedV1.of(circuit,
            TestCircuits.content(circuit, SECURITE, false, "Journalisation
insuffisante", List.of("OP-12", "OP-17")), "c".repeat(64));

        assertThat(schema.validate(mapper.valueToTree(payload))).isEmpty();
        assertThat(payload.decision()).isEqualTo("DEFAVORABLE");
        assertThat(payload.operationsDesignees()).containsExactly("OP-12", "OP-17");
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

Le retour effectif de la liaison en homologation est constaté côté MS-HOMOLOGATION par le test cURL (section E) ; ici, on vérifie tout ce que MS-CERTIFICATION lui transmet.

```
// test : web/rest/QuitusRefusalIT.java
class QuitusRefusalIT extends AbstractCertificationIT {
    static final UUID RSSI = UUID.randomUUID();

    @Test
    @DisplayName("CER-05 : refus du quitus sécurité, désignations transmises et
dossier maintenu ouvert")
```

```
    void refusSecurite() throws Exception {
        CertificationCircuit c = fixtures.circuitWithFavourable(TECHNIQUE);   //
socle : OP-12, OP-17, OP-21
        UUID dossierId = fixtures.dossierOf(c, null);

        mockMvc.perform(post("/api/v1/circuits/{id}/quitus/securite", c.getId())
                .with(asUser(RSSI, "RSSI", "CER_QUITUS_SECURITE_SIGN")).header("X-
Reauth-Assertion", "ok")
                .contentType(MediaType.APPLICATION_JSON)
                .content(fixtures.refusalJson("Journalisation insuffisante",
c.getSocleEmpreinte(), List.of("OP-12", "OP-17"))))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.decisionFavorable").value(false));

        Map<String, Object> evt = jdbc.queryForMap("select payload->>'decision' as
decision, payload->'operationsDesignees' as ops"
            + " from outbox_event where event_type = 'certification.quitus-signed.v1'
and aggregate_id = ?", c.getId().toString());
        assertThat(evt.get("decision")).isEqualTo("DEFAVORABLE");
        assertThat(evt.get("ops").toString()).contains("OP-12", "OP-17");
        assertThat(fixtures.states(c.getId())).containsEntry(CONFORMITE,
ANNULE).containsEntry(DIRECTION_GENERALE, ANNULE);

assertThat(fixtures.circuitState(c.getId())).isEqualTo(EtatCircuit.INTERROMPU);
        assertThat(fixtures.dossierState(dossierId)).isEqualTo(EtatDossier.OUVERT);
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: CER-05 Refus d'un quitus et retour de la liaison en homologation

  Scénario: Nominal : refus du quitus sécurité
    Étant donné un circuit dont le quitus "TECHNIQUE" est "SIGNE_FAVORABLE" et dont
le socle contient "OP-12", "OP-17" et "OP-21"
    Quand le RSSI signe un quitus sécurité défavorable avec le motif "Journalisation
insuffisante" en désignant "OP-12" et "OP-17"
    Alors l'événement certification.quitus-signed.v1 portant la décision
"DEFAVORABLE", le motif et les opérations désignées est émis
    Et les quitus "CONFORMITE" et "DIRECTION_GENERALE" sont "ANNULE"
    Et le dossier reste à l'état "OUVERT" en attente d'une nouvelle aptitude

  Scénario: Alternatif : refus de la Direction Générale
    Étant donné un circuit dont les quitus "TECHNIQUE", "SECURITE" et "CONFORMITE"
sont favorables
    Quand la Direction Générale refuse avec le motif "Capacité de production
insuffisante" en désignant le service "RETRAIT"
    Alors la liaison retourne en homologation
    Et à la nouvelle aptitude, les trois quitus favorables sont soumis au contrôle de
caducité

  Scénario: Erreur : désignation invalide
    Étant donné un refus en préparation sur un socle contenant "OP-12", "OP-17" et
"OP-21"
    Quand le signataire désigne le code d'opération "OP-99" absent du socle
d'évaluation
    Alors le refus n'est pas enregistré et la réponse porte le code
"CER_DESIGNATION_INVALID"
    Et la liste des codes valides "OP-12", "OP-17" et "OP-21" est proposée
    Quand le signataire omet le motif
    Alors le refus n'est pas enregistré, la réponse porte le code
"CER_MOTIF_REQUIRED" et propose la même liste
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-rssi
BASE="${GATEWAY_URL}/bff/des/certification/circuits/${CIRCUIT_ID}"
SOCLE=$(curl -s "${BASE}" -H "Authorization: Bearer ${TOKEN}" | jq -
r .socleEmpreinte)

# Refus : code désigné absent du socle -> 409 et liste des codes valides
curl -s -w "\n%{http_code}\n" -X POST "${BASE}/quitus/securite" \
  -H "Authorization: Bearer ${TOKEN}" -H "X-Reauth-Assertion: ${REAUTH_ASSERTION:-
stub-ok}" -H "Content-Type: application/json" \
  -d "{\"favorable\":false,\"motif\":\"Journalisation
insuffisante\",\"socleEmpreinte\":\"${SOCLE}\",\"operationsDesignees\":[\"OP-99\"]}"

# Nominal : refus motivé avec deux opérations désignées
curl -s -X POST "${BASE}/quitus/securite" \
  -H "Authorization: Bearer ${TOKEN}" -H "X-Reauth-Assertion: ${REAUTH_ASSERTION:-
stub-ok}" -H "Content-Type: application/json" \
  -d "{\"favorable\":false,\"motif\":\"Journalisation
insuffisante\",\"socleEmpreinte\":\"${SOCLE}\",\"operationsDesignees\":[\"OP-
12\",\"OP-17\"]}" \
  | jq '{decisionFavorable, operationsDesignees}'
curl -s "${BASE}" -H "Authorization: Bearer ${TOKEN}" | jq '{etat, quitus: [.quitus[]
| {nature, etat}]}'
# Attendu : etat "INTERROMPU" ; CONFORMITE et DIRECTION_GENERALE "ANNULE"

# Retour en homologation constaté dans MS-HOMOLOGATION (HOM-19)
curl -s "${GATEWAY_URL}/bff/des/homologation/liaisons/${LIAISON_ID}" -H
"Authorization: Bearer ${TOKEN}" | jq '{etat, motifEtat}'
# Attendu : etat "EN_HOMOLOGATION", motifEtat "Journalisation insuffisante"
```

CER-06 | Rendre caducs les quitus antérieurs lorsque le socle d'évaluation change

Élément	Valeur
Points	5
Priorité	Must Have
Sprint cible	Sprint 5
Écrans	DES-11
Dépendances	CER-01, CER-05, HOM-11, HOM-18

Objectif

Conserver un quitus favorable tant que l'empreinte du socle d'évaluation est inchangée et le déclarer caduc, avec motif et événement `certification.quitus-lapsed.v1`, dès que ce socle change : entre deux cycles d'aptitude, sur réouverture d'une opération ou sur changement de couverture pendant le circuit. Toute signature portant une empreinte périmée est refusée (409) et le socle à jour est renvoyé.

A. Implémentation

A1. Règle de reconduction par comparaison d'empreintes (domaine pur)

```
// domain/service/QuitusCarryOverRule.java
public final class QuitusCarryOverRule {
    public record CarryOver(Set<NatureQuitus> lapsed, Set<NatureQuitus> carriedOver,
String motif) { }

    /** Empreinte identique : les favorables sont reconduits ; empreinte différente :
ils deviennent caducs. */
    public CarryOver decide(Map<NatureQuitus, EtatQuitus> previous, String
previousSocle, String newSocle) {
        Set<NatureQuitus> favorables = previous.entrySet().stream()
            .filter(e -> e.getValue() == SIGNE_FAVORABLE).map(Map.Entry::getKey)
            .collect(Collectors.toCollection(() ->
EnumSet.noneOf(NatureQuitus.class)));
        if (previousSocle.equals(newSocle)) {
            return new CarryOver(EnumSet.noneOf(NatureQuitus.class), favorables,
null);
        }
        return new CarryOver(favorables, EnumSet.noneOf(NatureQuitus.class),
            "Socle d'évaluation modifié : " + previousSocle.substring(0, 12) + "
remplacé par " + newSocle.substring(0, 12));
    }
}
```

A2. Service de caducité (remplace la version conservatrice de CER-01)

Un quitus reconduit passe à `SIGNE_FAVORABLE` dans le nouveau circuit ; la décision qui le fonde est retrouvée par la requête `SignedDecisionRepository.findFounding(liaisonId, nature, socleEmpreinte)`, sans champ supplémentaire au modèle. Pendant un circuit, seul un circuit `OUVERT` est concerné : après l'autorisation, un changement relève de la condition `QUATRE_QUITUS` du dossier (CER-08).

```
// application/usecase/QuitusLapseService.java
@Service
```

```
@RequiredArgsConstructor
public class QuitusLapseService {
    private final QuitusCarryOverRule rule = new QuitusCarryOverRule();
    private final CircuitStateMachine stateMachine;
    private final CertificationCircuitRepository circuits;
    private final HomologationReaderPort homologation;
    private final OutboxPublisher outboxPublisher;
    private final Clock clock;

    /** Nouveau cycle d'aptitude (appelé par CER-01 dans sa transaction). */
    public void onNewCycle(CertificationCircuit previous, CertificationCircuit next)
{
        Instant now = clock.instant();
        CarryOver d = rule.decide(previous.quitusStates(),
previous.getSocleEmpreinte(), next.getSocleEmpreinte());
        d.carriedOver().forEach(n -> next.quitus(n).changeState(SIGNE_FAVORABLE,
now));
        Map<NatureQuitus, EtatQuitus> states =
stateMachine.promote(next.quitusStates());
        next.applyStates(states, stateMachine.circuitState(states), now);
        lapse(previous, d.lapsed(), d.motif(), now);
    }

    /** Réouverture d'une opération : caducité inconditionnelle des favorables du
circuit ouvert. */
    @Transactional
    public void onLiaisonChanged(UUID liaisonId, String motif) {
        circuits.findOpenByLiaisonIdForUpdate(liaisonId).ifPresent(c -> {
            Instant now = clock.instant();
            Set<NatureQuitus> favorables = c.naturesIn(SIGNE_FAVORABLE);
            c.getQuitus().stream().filter(q -> q.getEtat() == ATTENDU || q.getEtat()
== NON_REQUIS)
                .forEach(q -> q.changeState(ANNULE, now));        // signatures
attendues annulées
            c.interrupt(now);
            lapse(c, favorables, motif, now);
        });
    }

    /** Changement de couverture : caducité seulement si l'empreinte du socle a
effectivement changé. */
    @Transactional
    public void onCoverageChanged(UUID liaisonId) {
        circuits.findOpenByLiaisonId(liaisonId).ifPresent(c ->
homologation.readEvaluationBase(liaisonId)
            .filter(s -> !s.empreinte().equals(c.getSocleEmpreinte()))
            .ifPresent(s -> onLiaisonChanged(liaisonId, "Couverture modifiée :
nouvelle empreinte " + s.empreinte().substring(0, 12))));
    }

    private void lapse(CertificationCircuit c, Set<NatureQuitus> natures, String
motif, Instant now) {
        if (natures.isEmpty()) {
            return;
        }
        natures.forEach(n -> c.quitus(n).lapse(motif));      // état CADUC et motif
conservés (historique)
        outboxPublisher.publish("Quitus", c.getId().toString(),
"certification.quitus-lapsed.v1",
            new QuitusLapsedV1(c.getId(), c.getLiaisonId(), c.getCycle(), natures,
motif, now));
        natures.forEach(n -> outboxPublisher.publish("Notification",
c.getId().toString(),
            "certification.notification-requested.v1",
            NotificationRequestedV1.toRole(QuitusRoles.roleOf(n), "CER_QUITUS_CADUC",
Map.of("circuitId", c.getId(), "motif", motif))));
    }
}
```

A3. Consommation des réouvertures et des changements de couverture

```
// infrastructure/messaging/consumer/HomologationChangeConsumer.java
@Component
@RequiredArgsConstructor
class HomologationChangeConsumer {
    static final String GROUP = "mscertification-caducite";
    private final IdempotentConsumer idempotentConsumer;
    private final QuitusLapseService quitusLapse;

    @KafkaListener(topics = {"homologation.operation", "homologation.coverage"},
groupId = GROUP)
    void onEvent(CloudEventEnvelope envelope) {
        switch (envelope.type()) {
            case "homologation.operation-reopened.v1" ->
idempotentConsumer.handleOnce(envelope.id(), GROUP, () -> {
                OperationReopenedV1 e = envelope.dataAs(OperationReopenedV1.class);
                quitusLapse.onLiaisonChanged(e.liaisonId(), "Opération " +
e.operationCode() + " rouverte : " + e.motif());
            });
            case "homologation.coverage-changed.v1" ->
idempotentConsumer.handleOnce(envelope.id(), GROUP,
                () ->
quitusLapse.onCoverageChanged(envelope.dataAs(CoverageChangedV1.class).liaisonId()));
            default -> { }   // autres types des agrégats opération et couverture :
sans objet
        }
    }
}
```

Après un refus (CER-05), le circuit est déjà `INTERROMPU` : les réouvertures émises par HOM-19 pour les opérations désignées n'ont donc aucun effet ici, et c'est la comparaison d'empreintes du cycle suivant qui tranche.

A4. Contrôle d'empreinte au moment de chaque signature

La commande porte `socleEmpreinte`, l'empreinte affichée au signataire (champ obligatoire depuis CER-02). Elle doit égaler celle du circuit et celle que MS-HOMOLOGATION restitue à l'instant de la signature.

```
// application/usecase/guard/SocleFreshnessGuard.java
@Component
@Order(20)
class SocleFreshnessGuard implements SignatureGuard {
    @Override
    public void check(SigningContext ctx) {
        String affiche = ctx.command().socleEmpreinte();
        String duCircuit = ctx.circuit().getSocleEmpreinte();
        EvaluationBaseSnapshot courant = ctx.socle();
        if (!duCircuit.equals(affiche) || !courant.empreinte().equals(duCircuit)) {
            throw CertificationRuleException.conflict("CER_SOCLE_OUTDATED",
                "Le socle d'évaluation a changé depuis son affichage.",
                Map.of("socleAffiche", affiche, "socleAJour", courant.empreinte(),
"dateCalcul", courant.dateCalcul(),
                       "couvertures", courant.couvertures()));
        }
    }
}
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
// test : domain/service/QuitusCarryOverRuleTest.java
class QuitusCarryOverRuleTest {
    final QuitusCarryOverRule rule = new QuitusCarryOverRule();
```

```
    final Map<NatureQuitus, EtatQuitus> precedent = Map.of(
        TECHNIQUE, SIGNE_FAVORABLE, SECURITE, ANNULE, CONFORMITE, SIGNE_DEFAVORABLE,
DIRECTION_GENERALE, ANNULE);

    @Test
    @DisplayName("CER-06 : une empreinte identique conserve le quitus technique
favorable")
    void conservationPourEmpreinteIdentique() {
        CarryOver d = rule.decide(precedent, "a".repeat(64), "a".repeat(64));
        assertThat(d.carriedOver()).containsExactly(TECHNIQUE);
        assertThat(d.lapsed()).isEmpty();

        var etats = new CircuitStateMachine().promote(Map.of(
            TECHNIQUE, SIGNE_FAVORABLE, SECURITE, NON_REQUIS, CONFORMITE, NON_REQUIS,
DIRECTION_GENERALE, NON_REQUIS));
        assertThat(etats).containsEntry(SECURITE, ATTENDU).containsEntry(CONFORMITE,
ATTENDU).containsEntry(DIRECTION_GENERALE, NON_REQUIS);
    }

    @Test
    @DisplayName("CER-06 : une empreinte différente rend caducs les favorables avec
un motif")
    void caduciteSurEmpreinteDifferente() {
        CarryOver d = rule.decide(precedent, "a".repeat(64), "b".repeat(64));
        assertThat(d.lapsed()).containsExactly(TECHNIQUE);
        assertThat(d.motif()).startsWith("Socle d'évaluation modifié");
    }

    @Test
    @DisplayName("CER-06 : une signature sur un socle périmé est refusée et le socle
à jour est renvoyé")
    void refusSurSoclePerime() {
        SigningContext ctx = TestCircuits.contextWithSocles("a".repeat(64),
"a".repeat(64), "b".repeat(64)); // affiché, circuit, courant
        assertThatThrownBy(() -> new SocleFreshnessGuard().check(ctx))
            .hasFieldOrPropertyWithValue("code", "CER_SOCLE_OUTDATED")
            .extracting("details").asInstanceOf(InstanceOfAssertFactories.MAP).contai
nsEntry("socleAJour", "b".repeat(64));
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
// test : infrastructure/messaging/consumer/QuitusLapseIT.java
class QuitusLapseIT extends AbstractCertificationIT {
    @Test
    @DisplayName("CER-06 : la réouverture d'une opération rend caducs les favorables
et annule les signatures attendues")
    void caduciteSurReouverture() {
        CertificationCircuit c = fixtures.circuitWithFavourable(TECHNIQUE, SECURITE);

        kafkaTemplate.send("homologation.operation", c.getLiaisonId().toString(),
            fixtures.operationReopenedEvent(c.getLiaisonId(), "OP-17", "Preuve
remplacée"));

        await().atMost(Duration.ofSeconds(20)).untilAsserted(() -> {
            assertThat(fixtures.states(c.getId())).containsEntry(TECHNIQUE,
CADUC).containsEntry(SECURITE, CADUC)
                .containsEntry(CONFORMITE, ANNULE).containsEntry(DIRECTION_GENERALE,
ANNULE);
            assertThat(outboxCount("certification.quitus-lapsed.v1",
c.getId())).isEqualTo(1);
        });
        assertThat(jdbc.queryForObject("select motif_caducite from quitus where
circuit_id = ? and nature = 'TECHNIQUE'",
            String.class, c.getId())).isEqualTo("Opération OP-17 rouverte : Preuve
remplacée");
```

```
assertThat(fixtures.circuitState(c.getId())).isEqualTo(EtatCircuit.INTERROMPU);
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: CER-06 Caducité des quitus antérieurs sur changement du socle
d'évaluation

  Scénario: Nominal : socle inchangé
    Étant donné un quitus "TECHNIQUE" favorable sur le socle "a3f1"
    Et un refus du quitus conformité motivé par une pièce légale de l'institution,
extérieure au socle d'évaluation
    Quand la liaison est redéclarée apte au cycle 2 avec la même empreinte "a3f1"
    Alors le quitus "TECHNIQUE" du nouveau circuit est "SIGNE_FAVORABLE" sans
nouvelle signature
    Et seuls les quitus "SECURITE" et "CONFORMITE" sont de nouveau "ATTENDU"

  Scénario: Alternatif : réouverture d'une opération pendant le circuit
    Étant donné un circuit ouvert comportant les quitus "TECHNIQUE" et "SECURITE"
favorables
    Quand l'événement homologation.operation-reopened.v1 de la liaison est consommé
pour l'opération "OP-17"
    Alors les quitus "TECHNIQUE" et "SECURITE" sont "CADUC" avec le motif de la
réouverture
    Et les signatures attendues sont "ANNULE"
    Et l'événement certification.quitus-lapsed.v1 est émis et les signataires sont
notifiés

  Scénario: Erreur : signature sur un socle périmé
    Étant donné un signataire consultant le socle "a3f1" dont l'empreinte est devenue
"c9d2" depuis l'affichage
    Quand il signe
    Alors la signature est refusée avec le code "CER_SOCLE_OUTDATED"
    Et le socle à jour "c9d2" lui est présenté
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-conformite
BASE="${GATEWAY_URL}/bff/des/certification/circuits/${CIRCUIT_ID}"

# Refus : empreinte affichée périmée -> 409 (CER_SOCLE_OUTDATED)
curl -s -o /dev/null -w "%{http_code}\n" -X POST "${BASE}/quitus/conformite" \
  -H "Authorization: Bearer ${TOKEN}" -H "X-Reauth-Assertion: ${REAUTH_ASSERTION:-
stub-ok}" -H "Content-Type: application/json" \
  -d "{\"favorable\":true,\"motif\":\"Pièces légales
vérifiées\",\"socleEmpreinte\":\"${SOCLE_PERIME}\"}"
curl -s -X POST "${BASE}/quitus/conformite" \
  -H "Authorization: Bearer ${TOKEN}" -H "X-Reauth-Assertion: ${REAUTH_ASSERTION:-
stub-ok}" -H "Content-Type: application/json" \
  -d "{\"favorable\":true,\"motif\":\"Pièces légales
vérifiées\",\"socleEmpreinte\":\"${SOCLE_PERIME}\"}" \
  | jq '{code, socleAJour: .details.socleAJour}'

# Nominal : après réouverture d'une opération, quitus caducs avec motif
curl -s "${BASE}/quitus" -H "Authorization: Bearer ${TOKEN}" | jq '[.[] | {nature,
etat, motifCaducite}]'
# Attendu : favorables antérieurs "CADUC" avec le motif ; signatures attendues
"ANNULE"
```

CER-07 | Contrôler l'impartialité des signataires

Élément	Valeur
Points	3
Priorité	Must Have
Sprint cible	Sprint 4
Écrans	DES-11
Dépendances	HOM-04, HOM-08, IAM-06

Objectif

Interdire à quiconque est chef de projet, responsable ou validateur d'une opération de la liaison de signer l'un de ses quitus, y compris sous délégation (délégataire et délégant contrôlés tous deux), conformément à l'exigence d'impartialité de l'ISO/IEC 17065. Chaque refus est journalisé vers MS-AUDIT et le responsable conformité est notifié afin qu'un suppléant soit désigné.

A. Implémentation

A1. Règle d'incompatibilité (domaine pur)

La règle est la même pour les quatre natures de quitus ; elle travaille sur l'objet-valeur `LiaisonParticipants` lu auprès de MS-HOMOLOGATION (`GET /liaisons/{id}/intervenants`, port de CER-01).

```
// domain/service/ImpartialityRule.java
public final class ImpartialityRule {
    public enum Motif { CHEF_DE_PROJET, RESPONSABLE_OPERATION, VALIDATEUR_OPERATION }

    public Set<Motif> incompatibilities(UUID personId, LiaisonParticipants p) {
        EnumSet<Motif> motifs = EnumSet.noneOf(Motif.class);
        if (personId.equals(p.chefDeProjetId())) motifs.add(Motif.CHEF_DE_PROJET);
        if (p.responsableIds().contains(personId))
motifs.add(Motif.RESPONSABLE_OPERATION);
        if (p.validateurIds().contains(personId))
motifs.add(Motif.VALIDATEUR_OPERATION);
        return motifs;
    }

    /** Sous délégation, l'incompatibilité est vérifiée pour le délégataire et pour
le délégant. */
    public Map<UUID, Set<Motif>> check(UUID signataireId, Optional<UUID> delegantId,
LiaisonParticipants p) {
        Map<UUID, Set<Motif>> conflits = new LinkedHashMap<>();
        Stream.concat(Stream.of(signataireId), delegantId.stream()).forEach(id -> {
            Set<Motif> m = incompatibilities(id, p);
            if (!m.isEmpty()) conflits.put(id, m);
        });
        return conflits;
    }
}
```

A2. Lecture des délégations de MS-IAM

```
// application/port/out/DelegationReaderPort.java
public interface DelegationReaderPort {
    /** GET /api/v1/delegations/{id} de MS-IAM (IAM-06), jeton de service. */
    Optional<Delegation> find(UUID delegationId);
```

```
    record Delegation(UUID id, UUID delegantId, UUID delegataireId, String
permission,
                      Instant debut, Instant fin, boolean approuvee) {
        public boolean activeFor(UUID user, String perm, Instant at) {
            return approuvee && delegataireId.equals(user) && permission.equals(perm)
                && !at.isBefore(debut) && at.isBefore(fin);
        }
    }
}
```

L'adaptateur réel `IamDelegationReaderAdapter` (`gimac.adapters.delegation-reader=real`) appelle MS-IAM avec le jeton de service et rend `Optional.empty()` sur 404 ; `DelegationReaderStubAdapter` (profils `dev` et `test`) sert des délégations enregistrées en mémoire.

A3. Garde d'impartialité et journalisation des refus

La garde s'insère en premier (`@Order(10)`) dans le mécanisme de signature de CER-02 ; elle est développée avant lui et branchée à sa livraison. Le refus annule la transaction de signature ; sa trace est donc écrite dans une transaction propre, afin qu'elle parvienne à MS-AUDIT (type `certification.signature-refused.v1` sur `certification.quitus`, consommé par MS-AUDIT comme tout événement).

```
// application/usecase/guard/ImpartialityGuard.java
@Component
@Order(10)
@RequiredArgsConstructor
class ImpartialityGuard implements SignatureGuard {
    private final ImpartialityRule rule = new ImpartialityRule();
    private final DelegationReaderPort delegations;
    private final SignatureRefusalJournal journal;
    private final Clock clock;

    @Override
    public void check(SigningContext ctx) {
        SignQuitusCommand cmd = ctx.command();
        Optional<UUID> delegant = Optional.ofNullable(cmd.delegationId()).map(id ->
delegations.find(id)
            .filter(d -> d.activeFor(cmd.signataireId(),
QuitusRoles.permissionOf(cmd.nature()), clock.instant()))
            .orElseThrow(() ->
CertificationRuleException.forbidden("CER_DELEGATION_INVALID",
                "Délégation absente, non approuvée ou expirée."))
            .delegantId());
        Map<UUID, Set<ImpartialityRule.Motif>> conflits =
rule.check(cmd.signataireId(), delegant, ctx.participants());
        if (!conflits.isEmpty()) {
            journal.record(ctx, conflits);
            throw CertificationRuleException.conflict("CER_IMPARTIALITY_CONFLICT",
                "Signature refusée pour incompatibilité avec la liaison.",
Map.of("motifs", conflits.values()));
        }
    }
}

// application/usecase/SignatureRefusalJournal.java
@Component
@RequiredArgsConstructor
public class SignatureRefusalJournal {
    private final OutboxPublisher outboxPublisher;

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void record(SigningContext ctx, Map<UUID, Set<ImpartialityRule.Motif>>
conflits) {
```

```
        CertificationCircuit c = ctx.circuit();
        outboxPublisher.publish("Quitus", c.getId().toString(),
"certification.signature-refused.v1",
            new SignatureRefusedV1(c.getId(), c.getLiaisonId(),
ctx.command().nature(), ctx.command().signataireId(),
                ctx.command().delegationId(), conflits, Instant.now()));
        outboxPublisher.publish("Notification", c.getId().toString(),
"certification.notification-requested.v1",
            NotificationRequestedV1.toRole("RESPONSABLE_CONFORMITE",
"CER_SUPPLEANT_A_DESIGNER",
                Map.of("circuitId", c.getId(), "nature", ctx.command().nature())));
    }
}
```

A4. Signataires admissibles

La ressource indique, pour chaque nature, si l'utilisateur courant peut signer et les identifiants des personnes exclues ; l'écran DES-11 masque le bouton de signature en conséquence.

```
// web/rest/CircuitResource.java (ajout CER-07)
@GetMapping("/circuits/{id}/signataires-admissibles")
@PreAuthorize("hasAuthority('CER_CIRCUIT_READ')")
public List<AdmissibilityDTO> admissibles(@PathVariable UUID id,
@AuthenticationPrincipal Jwt jwt) {
    CertificationCircuit c = circuits.findById(id).orElseThrow(() -> new
ResourceNotFoundException("circuit", id));
    LiaisonParticipants p = homologation.readParticipants(c.getLiaisonId());
    Set<ImpartialityRule.Motif> miens =
impartialityRule.incompatibilities(UUID.fromString(jwt.getSubject()), p);
    Set<UUID> exclus = new TreeSet<>(p.responsableIds());
    exclus.addAll(p.validateurIds());
    Optional.ofNullable(p.chefDeProjetId()).ifPresent(exclus::add);
    return Arrays.stream(NatureQuitus.values())
        .map(n -> new AdmissibilityDTO(n, QuitusRoles.roleOf(n), miens.isEmpty(),
miens, exclus)).toList();
}
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
// test : domain/service/ImpartialityRuleTest.java
class ImpartialityRuleTest {
    static final UUID CHEF = UUID.randomUUID(), RESP = UUID.randomUUID(), VALID =
UUID.randomUUID(), TIERS = UUID.randomUUID();
    static final LiaisonParticipants P = new LiaisonParticipants(CHEF, Set.of(RESP),
Set.of(VALID));
    final ImpartialityRule rule = new ImpartialityRule();

    static Stream<Arguments> motifs() {
        return Stream.of(Arguments.of(CHEF, Motif.CHEF_DE_PROJET),
                         Arguments.of(RESP, Motif.RESPONSABLE_OPERATION),
                         Arguments.of(VALID, Motif.VALIDATEUR_OPERATION));
    }

    @ParameterizedTest(name = "{1}")
    @MethodSource("motifs")
    @DisplayName("CER-07 : chacun des trois motifs d'incompatibilité est détecté")
    void chaqueMotif(UUID personne, Motif attendu) {
        assertThat(rule.check(personne, Optional.empty(),
P)).containsOnlyKeys(personne).containsEntry(personne, Set.of(attendu));
    }

    @Test
    @DisplayName("CER-07 : un signataire sans lien avec la liaison est admis")
    void signataireSansLien() {
        assertThat(rule.check(TIERS, Optional.empty(), P)).isEmpty();
    }
```

```
    @Test
    @DisplayName("CER-07 : sous délégation, le délégant incompatible est détecté même
si le délégataire est admissible")
    void delegation() {
        assertThat(rule.check(TIERS, Optional.of(VALID), P)).containsOnlyKeys(VALID);
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
// test : application/usecase/guard/ImpartialityDelegationIT.java
class ImpartialityDelegationIT extends AbstractCertificationIT {
    static final UUID DELEGATAIRE = UUID.randomUUID();
    static final UUID DELEGANT = UUID.randomUUID();      // RSSI titulaire,
validateur d'une opération de la liaison
    static final UUID DELEGATION = UUID.randomUUID();

    @Test
    @DisplayName("CER-07 : sous délégation, le délégant incompatible fait refuser la
signature et le refus est journalisé")
    void controleSousDelegation() throws Exception {
        CertificationCircuit c = fixtures.circuitWithFavourable(TECHNIQUE);
        fixtures.stubParticipants(c.getLiaisonId(), UUID.randomUUID(), Set.of(),
Set.of(DELEGANT));
        iam.stubFor(get(urlPathEqualTo("/api/v1/delegations/" +
DELEGATION)).willReturn(okJson(
            fixtures.delegationJson(DELEGATION, DELEGANT, DELEGATAIRE,
"CER_QUITUS_SECURITE_SIGN", true))));

        mockMvc.perform(post("/api/v1/circuits/{id}/quitus/securite", c.getId())
                .with(asUser(DELEGATAIRE, "RSSI",
"CER_QUITUS_SECURITE_SIGN")).header("X-Reauth-Assertion", "ok")
                .contentType(MediaType.APPLICATION_JSON)
                .content(fixtures.decisionJson(true, "Volet sécurité conforme",
c.getSocleEmpreinte(), DELEGATION)))
            .andExpect(status().isConflict())
            .andExpect(jsonPath("$.code").value("CER_IMPARTIALITY_CONFLICT"));

        assertThat(fixtures.decisionCount(c.getId())).isEqualTo(1);           // seul
le quitus technique existe
        assertThat(outboxCount("certification.signature-refused.v1",
c.getId())).isEqualTo(1);   // trace validée malgré le refus
        assertThat(jdbc.queryForObject("select count(*) from outbox_event where
aggregate_id = ?"
            + " and payload->>'gabarit' = 'CER_SUPPLEANT_A_DESIGNER'", Long.class,
c.getId().toString())).isEqualTo(1);
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: CER-07 Impartialité des signataires

  Scénario: Nominal : signataire sans lien avec la liaison
    Étant donné un RSSI qui n'est ni chef de projet, ni responsable, ni validateur
d'une opération de la liaison "L-BQB-02" de la Banque B
    Quand il ouvre la décision du quitus "SECURITE"
    Alors les signataires admissibles indiquent que la signature lui est permise

  Scénario: Alternatif : signature sous délégation
    Étant donné un délégataire signant le quitus "SECURITE" au titre d'une délégation
approuvée
    Et un délégant validateur d'une opération de la liaison
```

```
    Quand le contrôle s'exécute
    Alors l'incompatibilité est vérifiée pour le délégataire et pour le délégant
    Et la signature est refusée avec le code "CER_IMPARTIALITY_CONFLICT"

  Scénario: Erreur : chef de projet de la liaison
    Étant donné le chef du DES désigné chef de projet de la liaison "L-BQB-02"
    Quand il tente de signer le quitus "TECHNIQUE"
    Alors la signature est refusée pour incompatibilité et l'événement
certification.signature-refused.v1 est journalisé
    Et le responsable conformité est notifié afin qu'un suppléant soit désigné par
délégation
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-chef-des
BASE="${GATEWAY_URL}/bff/des/certification/circuits"

# Nominal : admissibilité de l'utilisateur courant par nature
curl -s "${BASE}/${CIRCUIT_ID}/signataires-admissibles" -H "Authorization: Bearer $
{TOKEN}" \
  | jq '[.[] | {nature, roleRequis, utilisateurCourantAdmissible, motifs}]'

# Refus : le chef du DES est chef de projet de la liaison du circuit
CIRCUIT_ID_CONFLIT -> 409
SOCLE=$(curl -s "${BASE}/${CIRCUIT_ID_CONFLIT}" -H "Authorization: Bearer ${TOKEN}" |
jq -r .socleEmpreinte)
curl -s -o /dev/null -w "%{http_code}\n" -X POST
"${BASE}/${CIRCUIT_ID_CONFLIT}/quitus/technique" \
  -H "Authorization: Bearer ${TOKEN}" -H "X-Reauth-Assertion: ${REAUTH_ASSERTION:-
stub-ok}" -H "Content-Type: application/json" \
  -d "{\"favorable\":true,\"motif\":\"Socle conforme\",\"socleEmpreinte\":\"$
{SOCLE}\"}"
```

CER-08 | Contrôler la complétude du dossier de mise en service

Élément	Valeur
Points	8
Priorité	Must Have
Sprint cible	Sprint 4
Écrans	DES-11
Dépendances	CER-01, HOM-17, HOM-22, DOC-05

Objectif

Évaluer en continu les six conditions de complétude du dossier, chacune avec sa source et la date de sa dernière vérification, et faire suivre au dossier un automate à cinq états (`OUVERT`, `AUTORISE`, `MIS_EN_SERVICE`, `COMPLET`, `ATTESTE`) dont la table de transitions est explicite. Le dossier passe à `COMPLET` quand la sixième condition est satisfaite, ce qui déclenche le scellement ; une empreinte divergente bloque le dossier et alerte le RSSI.

A. Implémentation

A1. Automate du dossier et entité de condition

Toute paire absente de la table est interdite : pas de saut d'état, pas de retour arrière. Chaque transition a un seul récit propriétaire, qui en vérifie la garde métier avant d'appeler l'automate.

```
// domain/service/DossierStateMachine.java
public final class DossierStateMachine {
    private static final Map<EtatDossier, Set<EtatDossier>> TRANSITIONS = Map.of(
        OUVERT, Set.of(AUTORISE),             // CER-04 : quitus final et fiche
approuvée
        AUTORISE, Set.of(MIS_EN_SERVICE),     // CER-10 : procès-verbal consigné
        MIS_EN_SERVICE, Set.of(COMPLET),      // CER-08 : six conditions satisfaites
        COMPLET, Set.of(ATTESTE),             // CER-12 : dossier scellé, attestation
émise
        ATTESTE, Set.of());

    public void assertTransition(EtatDossier from, EtatDossier to) {
        if (!TRANSITIONS.get(from).contains(to)) {
            throw new DossierTransitionViolation(from, to);   // traduit en 409
CER_DOSSIER_TRANSITION_FORBIDDEN
        }
    }
    public Set<EtatDossier> allowedFrom(EtatDossier from) { return
TRANSITIONS.get(from); }   // transitionsPermises
}
```

```
// domain/model/GoLiveDossier.java (ajout CER-08)
@OneToMany(mappedBy = "dossier", cascade = CascadeType.ALL, orphanRemoval = true)
private List<CompletenessCondition> conditions = new ArrayList<>();

public void transitionTo(EtatDossier target, DossierStateMachine machine, Instant
now) {
    if (isBlocked()) {
        throw new DossierTransitionViolation(etat, target, "Dossier bloqué :
empreinte divergente");
    }
    machine.assertTransition(etat, target);
    switch (target) {
```

```
        case AUTORISE -> dateAutorisation = now;
        case MIS_EN_SERVICE -> dateMiseEnService = now;
        case COMPLET -> { dateCompletude = now; etatScellement =
EtatScellement.EN_ATTENTE; }
        default -> { }
    }
    etat = target;
}

public boolean isBlocked() {
    String detail = condition(ConditionCode.EMPREINTES_VERIFIEES).getDetail();
    return detail != null && detail.startsWith(CompletenessEvaluator.DIVERGENCE);
}
public boolean allConditionsSatisfied() { return
conditions.stream().allMatch(CompletenessCondition::getSatisfaite); }
public CompletenessCondition condition(ConditionCode code) {
    return conditions.stream().filter(c -> c.getCode() ==
code).findFirst().orElseThrow();
}
```

`CompletenessCondition` (JDL : code, satisfaite, source, detail, dateVerification ; unicité (dossier, code)) expose `update(ConditionResult r, Instant now)` ; `GoLiveDossier.open` (CER-01) crée les six lignes non satisfaites.

A2. Évaluation des six conditions (domaine pur)

L'évaluateur ne lit rien : le cas d'utilisation lui fournit les faits. Chaque résultat porte sa source, affichée telle quelle par l'écran.

```
// domain/service/CompletenessEvaluator.java
public final class CompletenessEvaluator {
    public static final String DIVERGENCE = "EMPREINTE_DIVERGENTE";
    private static final BigDecimal CENT = new BigDecimal("100.00");

    public record Facts(List<CoverageLine> couvertures, Map<NatureQuitus, EtatQuitus>
quitus,
                        Optional<ChangeRecordFact> fiche, Optional<ReportFact>
procesVerbal, List<DocumentCheck> pieces) { }
    public record ChangeRecordFact(String reference, boolean approuvee, Instant
dateLecture) { }
    public record ReportFact(boolean dansFenetre, boolean regularise) { }
    public record DocumentCheck(String documentRef, String empreinteAttendue, String
empreinteRecalculee) {
        boolean conforme() { return empreinteAttendue.equals(empreinteRecalculee); }
    }
    public record ConditionResult(ConditionCode code, boolean satisfaite, String
source, String detail) { }

    public List<ConditionResult> evaluate(Facts f) {
        return List.of(coverage(COUVERTURE_CERTIFICATION, "CERTIFICATION",
f.couvertures()),
                       coverage(COUVERTURE_MISE_EN_SERVICE, "MISE_EN_SERVICE",
f.couvertures()),
                       quitus(f.quitus()), fiche(f.fiche()),
procesVerbal(f.procesVerbal()), empreintes(f.pieces()));
    }

    private ConditionResult coverage(ConditionCode code, String nature,
List<CoverageLine> all) {
        List<CoverageLine> l = all.stream().filter(c ->
c.nature().equals(nature)).toList();
        List<String> ko = l.stream().filter(c -> !c.calculable() ||
c.taux().compareTo(CENT) < 0).map(CoverageLine::serviceCode).toList();
        boolean ok = !l.isEmpty() && ko.isEmpty();
        return new ConditionResult(code, ok, "MS-HOMOLOGATION : couvertures de la
liaison",
            ok ? "100 % sur " + l.size() + " service(s)" : "Sous 100 % ou non
```

```
calculable : " + ko);
    }
    private ConditionResult quitus(Map<NatureQuitus, EtatQuitus> q) {
        List<String> ko = Arrays.stream(NatureQuitus.values()).filter(n -> q.get(n) !
= SIGNE_FAVORABLE).map(n -> n + " " + q.get(n)).toList();
        return new ConditionResult(QUATRE_QUITUS, ko.isEmpty(), "MS-CERTIFICATION :
circuit courant",
            ko.isEmpty() ? "Quatre quitus favorables non caducs" : "Non acquis : " +
ko);
    }
    private ConditionResult fiche(Optional<ChangeRecordFact> f) {
        return new ConditionResult(FICHE_CHANGEMENT_APPROUVEE,
f.map(ChangeRecordFact::approuvee).orElse(false),
            "Dispositif de gestion des changements (lecture seule, CER-09)",
            f.map(x -> x.reference() + (x.approuvee() ? " approuvée" : " non
approuvée") + ", lue le " + x.dateLecture())
             .orElse("Aucune fiche référencée"));
    }
    private ConditionResult procesVerbal(Optional<ReportFact> r) {
        return new ConditionResult(PROCES_VERBAL, r.map(x -> x.dansFenetre() ||
x.regularise()).orElse(false),
            "MS-DOCUMENT : procès-verbal déposé en WORM (CER-10)",
            r.map(x -> x.dansFenetre() ? "Bascule dans la fenêtre" : x.regularise() ?
"Hors fenêtre, régularisé" : "Hors fenêtre")
             .orElse("Procès-verbal absent"));
    }
    private ConditionResult empreintes(List<DocumentCheck> pieces) {
        List<String> divergentes = pieces.stream().filter(p -> !
p.conforme()).map(DocumentCheck::documentRef).toList();
        return new ConditionResult(EMPREINTES_VERIFIEES, !pieces.isEmpty() &&
divergentes.isEmpty(), "MS-DOCUMENT : recalcul SHA-256",
            !divergentes.isEmpty() ? DIVERGENCE + " : " + divergentes
                : pieces.isEmpty() ? "Aucune pièce vérifiée" : pieces.size() + "
pièce(s) conforme(s)");
    }
}
```

A3. Vérification des empreintes par MS-DOCUMENT

```
// application/port/out/DocumentVerificationPort.java
public interface DocumentVerificationPort {
    /** POST /api/v1/documents/{ref}/verification de MS-DOCUMENT (DOC-05) : empreinte
SHA-256 recalculée sur la pièce WORM. */
    String recomputeSha256(String documentRef);
}
```

`DocumentVerificationRestAdapter` (jeton de service, disjoncteur `msdocument`) lève `DocumentServiceUnavailableException` (503, aucune valeur par défaut) ; `DocumentVerificationStubAdapter` (profils `dev` et `test`) rend des empreintes enregistrées en mémoire, ce qui permet de simuler une divergence en recette. Les pièces vérifiées sont le procès-verbal (`GoLiveReport`, CER-10) et le dossier de preuves de la liaison (HOM-22), lu par une méthode ajoutée au port de CER-01 : `Optional<EvidenceFileRef> latestEvidenceFile(UUID liaisonId)` sur `GET /liaisons/{id}/dossier-de-preuves`.

A4. Cas d'utilisation de complétude, blocage et déclenchement du scellement

Choix explicite : les empreintes sont recalculées sur `POST /dossiers/{id}/verification` et à l'arrivée à `MIS_EN_SERVICE` (appel par CER-10) ; les autres déclencheurs (changement de couverture, signature, relecture de fiche) réévaluent les cinq autres conditions et conservent le dernier résultat de la sixième.

```
// application/usecase/DossierCompletenessService.java
@Service
```

```
@RequiredArgsConstructor
public class DossierCompletenessService {
    private final CompletenessEvaluator evaluator = new CompletenessEvaluator();
    private final GoLiveDossierRepository dossiers;
    private final ChangeRecordSnapshotRepository snapshots;
    private final GoLiveReportRepository reports;
    private final HomologationReaderPort homologation;
    private final DocumentVerificationPort documents;
    private final DossierStateMachine dossierStateMachine;
    private final MonitoringAlertPort alerts;
    private final OutboxPublisher outboxPublisher;
    private final ApplicationEventPublisher domainEvents;
    private final Clock clock;

    @Transactional
    public GoLiveDossier reevaluate(UUID dossierId, boolean verifyDocuments) {
        GoLiveDossier d = dossiers.findByIdForUpdate(dossierId).orElseThrow(() ->
CertificationRuleException.notFound("dossier", dossierId));
        Instant now = clock.instant();
        EvaluationBaseSnapshot socle =
homologation.readEvaluationBase(d.getLiaisonId())
            .orElseThrow(() ->
CertificationRuleException.unavailable("CER_SOCLE_UNAVAILABLE", "Socle d'évaluation
introuvable."));
        List<ConditionResult> results = evaluator.evaluate(new
Facts(socle.couvertures(), d.getCircuitCourant().quitusStates(),

snapshots.findFirstByDossierIdOrderByDateLectureDesc(dossierId).map(ChangeRecordSnaps
hot::toFact),
            reports.findByDossierId(dossierId).map(GoLiveReport::toFact),
            verifyDocuments ? verifyPieces(d) : List.of()));
        results.stream().filter(r -> verifyDocuments || r.code() !=
EMPREINTES_VERIFIEES)
            .forEach(r -> d.condition(r.code()).update(r, now));

        if (verifyDocuments && d.isBlocked()) {           // blocage et alerte RSSI
dans la même transaction
            outboxPublisher.publish("Notification", dossierId.toString(),
"certification.notification-requested.v1",
                NotificationRequestedV1.toRole("RSSI", "CER_EMPREINTE_DIVERGENTE",
                    Map.of("dossierId", dossierId, "detail",
d.condition(EMPREINTES_VERIFIEES).getDetail())));
            alerts.raise("CER_EMPREINTE_DIVERGENTE", dossierId.toString(),
Map.of("liaisonId", d.getLiaisonId()));
            return d;
        }
        if (d.getEtat() == EtatDossier.MIS_EN_SERVICE && d.allConditionsSatisfied())
{
            d.transitionTo(EtatDossier.COMPLET, dossierStateMachine, now);
            domainEvents.publishEvent(new DossierCompletedEvent(dossierId));
        }
        return d;
    }

    private List<DocumentCheck> verifyPieces(GoLiveDossier d) {
        List<DocumentCheck> checks = new ArrayList<>();
        reports.findByDossierId(d.getId()).ifPresent(r ->
checks.add(check(r.getDocumentRef(), r.getEmpreinte())));
        homologation.latestEvidenceFile(d.getLiaisonId()).ifPresent(e ->
checks.add(check(e.documentRef(), e.empreinte())));
        return checks;
    }
    private DocumentCheck check(String ref, String attendue) {
        return new DocumentCheck(ref, attendue, documents.recomputeSha256(ref));
    }
}

// application/usecase/DossierSealingTrigger.java
@Component
```

```
@RequiredArgsConstructor
class DossierSealingTrigger {
    private final DossierSealingService sealing;   // CER-11 : horodatage RFC 3161 et
inscription au registre d'audit

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    void onCompleted(DossierCompletedEvent e) { sealing.seal(e.dossierId()); }
}
```

A5. Ressources REST et consommation des changements de couverture

```
// web/rest/DossierResource.java (extrait CER-08)
@GetMapping("/dossiers/{id}")
@PreAuthorize("hasAuthority('CER_DOSSIER_READ')")
public DossierDTO get(@PathVariable UUID id, Authentication auth) { return
dossierQueries.dossier(id, auth); }

@GetMapping("/dossiers/{id}/conditions")
@PreAuthorize("hasAuthority('CER_DOSSIER_READ')")
public List<ConditionDTO> conditions(@PathVariable UUID id) { return
dossierQueries.conditions(id); }  // code, satisfaite, source, detail,
dateVerification

@PostMapping("/dossiers/{id}/verification")
@PreAuthorize("hasAuthority('CER_DOSSIER_VERIFY')")
public DossierDTO verify(@PathVariable UUID id, Authentication auth) {
    completeness.reevaluate(id, true);
    return dossierQueries.dossier(id, auth);   // etat, bloque, transitionsPermises,
actionsPermises
}

// infrastructure/messaging/consumer/CoverageChangedCompletenessConsumer.java
@KafkaListener(topics = "homologation.coverage", groupId = "mscertification-
completude")
void onEvent(CloudEventEnvelope envelope) {
    if (!"homologation.coverage-changed.v1".equals(envelope.type())) {
        return;
    }
    idempotentConsumer.handleOnce(envelope.id(), "mscertification-completude", () ->

dossiers.findFirstByLiaisonIdAndEtatNot(envelope.dataAs(CoverageChangedV1.class).liai
sonId(), EtatDossier.ATTESTE)
            .ifPresent(d -> completeness.reevaluate(d.getId(), false)));
}
```

L'écran du dossier (DES-11, fragment frontend) présente le tableau des six conditions et la barre d'états (`StateMachineBar`) à partir de ces réponses.

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
// test : domain/service/DossierStateMachineTest.java
class DossierStateMachineTest {
    final DossierStateMachine machine = new DossierStateMachine();

    @Test
    @DisplayName("CER-08 : seules les quatre transitions de la table sont permises
sur les 25 paires d'états")
    void tableDeTransitions() {
        Set<String> permises = Set.of("OUVERT>AUTORISE", "AUTORISE>MIS_EN_SERVICE",
"MIS_EN_SERVICE>COMPLET", "COMPLET>ATTESTE");
        for (EtatDossier from : EtatDossier.values()) {
            for (EtatDossier to : EtatDossier.values()) {
                Throwable t = catchThrowable(() -> machine.assertTransition(from,
to));
                assertThat(t == null).as(from + " vers " +
to).isEqualTo(permises.contains(from + ">" + to));
```

```
            }
        }
    }
}

// test : domain/service/CompletenessEvaluatorTest.java
class CompletenessEvaluatorTest {
    final CompletenessEvaluator evaluator = new CompletenessEvaluator();
    static final BigDecimal CENT = new BigDecimal("100.00");

    /** Faits complets, sauf la condition indiquée. */
    static Facts completsSauf(ConditionCode c) {
        var cov = List.of(
            new CoverageLine("RETRAIT", "CERTIFICATION", c ==
COUVERTURE_CERTIFICATION ? new BigDecimal("97.50") : CENT, true),
            new CoverageLine("RETRAIT", "MISE_EN_SERVICE", c ==
COUVERTURE_MISE_EN_SERVICE ? new BigDecimal("80.00") : CENT, true));
        var q = new EnumMap<NatureQuitus, EtatQuitus>(NatureQuitus.class);
        for (NatureQuitus n : NatureQuitus.values()) q.put(n, SIGNE_FAVORABLE);
        if (c == QUATRE_QUITUS) q.put(CONFORMITE, CADUC);
        return new Facts(cov, q,
            Optional.of(new ChangeRecordFact("CHG-2026-0042", c !=
FICHE_CHANGEMENT_APPROUVEE, Instant.now())),
            Optional.of(new ReportFact(c != PROCES_VERBAL, false)),
            List.of(new DocumentCheck("DOC-PV-1", "a".repeat(64), c ==
EMPREINTES_VERIFIEES ? "b".repeat(64) : "a".repeat(64))));
    }

    @ParameterizedTest(name = "{0}")
    @EnumSource(ConditionCode.class)
    @DisplayName("CER-08 : chacune des six conditions est évaluée isolément, avec sa
source")
    void chaqueCondition(ConditionCode cassee) {
        List<ConditionResult> r = evaluator.evaluate(completsSauf(cassee));
        assertThat(r).hasSize(6).allSatisfy(c ->
assertThat(c.source()).isNotBlank());
        assertThat(r).filteredOn(c -> !
c.satisfaite()).extracting(ConditionResult::code).containsExactly(cassee);
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
// test : application/usecase/DossierCompletenessIT.java
class DossierCompletenessIT extends AbstractCertificationIT {
    static final UUID INGENIEUR = UUID.randomUUID();

    @Test
    @DisplayName("CER-08 : une empreinte divergente bloque le dossier et alerte le
RSSI")
    void blocageSurEmpreinteDivergente() throws Exception {
        GoLiveDossier d = fixtures.dossierMisEnService("a".repeat(64));   // cinq
conditions réunies, procès-verbal d'empreinte "a..."
        documents.stubFor(post(urlPathMatching("/api/v1/documents/.*/verification"))
            .willReturn(okJson("{\"empreinte\":\"" + "f".repeat(64) + "\"}")));

        mockMvc.perform(post("/api/v1/dossiers/{id}/verification", d.getId())
                .with(asUser(INGENIEUR, "INGENIEUR_HOMOLOGATION",
"CER_DOSSIER_VERIFY")))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.etat").value("MIS_EN_SERVICE"))
            .andExpect(jsonPath("$.bloque").value(true));

        mockMvc.perform(get("/api/v1/dossiers/{id}/conditions", d.getId())
                .with(asUser(INGENIEUR, "INGENIEUR_HOMOLOGATION",
"CER_DOSSIER_READ")))
            .andExpect(jsonPath("$.length()").value(6))
```

```
            .andExpect(jsonPath("$[?
(@.code=='EMPREINTES_VERIFIEES')].satisfaite").value(false))
            .andExpect(jsonPath("$[?
(@.code=='EMPREINTES_VERIFIEES')].detail").value(startsWith("EMPREINTE_DIVERGENTE")))
;

        assertThat(jdbc.queryForObject("select count(*) from outbox_event where
aggregate_id = ?"
            + " and payload->>'gabarit' = 'CER_EMPREINTE_DIVERGENTE' and payload-
>>'role' = 'RSSI'",
            Long.class, d.getId().toString())).isEqualTo(1);
        assertThatThrownBy(() -> fixtures.transition(d.getId(), EtatDossier.COMPLET))
            .isInstanceOf(DossierTransitionViolation.class);
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: CER-08 Complétude du dossier de mise en service

  Scénario: Nominal : dossier complet
    Étant donné un dossier à l'état "MIS_EN_SERVICE" dont cinq conditions sont
satisfaites
    Quand la condition "EMPREINTES_VERIFIEES" est satisfaite par la vérification des
pièces
    Alors le dossier passe à l'état "COMPLET"
    Et le scellement horodaté du dossier est déclenché avec l'état de scellement
"EN_ATTENTE"

  Scénario: Alternatif : tableau des conditions
    Étant donné un dossier à l'état "OUVERT"
    Quand l'ingénieur d'homologation consulte ses conditions
    Alors les six conditions sont présentées, chacune avec son état, sa source et la
date de sa dernière vérification

  Scénario: Erreur : transition non prévue
    Étant donné un dossier à l'état "OUVERT"
    Quand un passage direct à l'état "COMPLET" est demandé
    Alors la transition est refusée par l'automate avec le code
"CER_DOSSIER_TRANSITION_FORBIDDEN"

  Scénario: Erreur : empreinte divergente
    Étant donné une pièce du dossier dont l'empreinte recalculée par MS-DOCUMENT
diffère de l'empreinte enregistrée
    Quand la condition des empreintes est vérifiée
    Alors la condition "EMPREINTES_VERIFIEES" est déclarée non satisfaite
    Et le dossier est bloqué
    Et une alerte est émise vers le RSSI
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-ingenieur-homologation
BASE="${GATEWAY_URL}/bff/des/certification/dossiers/${DOSSIER_ID}"

# Nominal : tableau des six conditions
curl -s "${BASE}/conditions" -H "Authorization: Bearer ${TOKEN}" \
  | jq '[.[] | {code, satisfaite, source, dateVerification}]'

# Nominal : vérification des empreintes, puis état du dossier
curl -s -X POST "${BASE}/verification" -H "Authorization: Bearer ${TOKEN}" \
  | jq '{etat, bloque, transitionsPermises, etatScellement}'
# Attendu : "COMPLET" et scellement "EN_ATTENTE" si les six conditions sont réunies ;
sinon état inchangé
```

```
# Refus : route du DES appelée avec un jeton de correspondant d'institution -> 403
(OPA)
source ./scripts/token.sh test-correspondant-a
curl -s -o /dev/null -w "%{http_code}\n" -X POST "${BASE}/verification" -H
"Authorization: Bearer ${TOKEN}"
```

CER-09 │ Lire la fiche de changement sans jamais écrire dans le dispositif de gestion des changements

Élément	Valeur
Points	5
Priorité	Must Have
Sprint cible	Sprint 4
Écrans	DES-11
Dépendances	CER-01, SOC-07, SOC-09

Objectif

Le dossier référence une fiche de changement qu'il lit en authentification mutuelle et n'écrit jamais ; toute décision appuyée sur la fiche exige une lecture de quinze minutes au plus, et une indisponibilité du dispositif bloque la décision (503) sans valeur par défaut.

A. Implémentation

A1. Port en lecture seule et règle de fraîcheur

Le port n'expose qu'une lecture : l'écriture est impossible par construction. `FreshnessRule` est un service de domaine pur, paramétré par `gimac.certification.change-record.freshness=PT15M`. Les erreurs passent par `DomainProblemException` du socle, traduite en `ProblemDetail`.

```
// application/port/out/ChangeRecordReaderPort.java
public interface ChangeRecordReaderPort {
    /** Lève ChangeRecordUnavailableException (503, CER_CHANGE_RECORD_UNAVAILABLE) si
le dispositif ne répond pas. */
    ChangeRecordView read(String reference);

    record ChangeRecordView(String reference, String etatApprobation, Instant
fenetreDebut,
                            Instant fenetreFin, String empreinteReponse) {
        public boolean approuvee() { return "APPROUVEE".equals(etatApprobation); }
    }
}

// domain/service/FreshnessRule.java
public final class FreshnessRule {
    private final Duration maxAge;
    public FreshnessRule(Duration maxAge) { this.maxAge =
Objects.requireNonNull(maxAge); }
    /** Fraîche si la lecture date de quinze minutes au plus ; au-delà, relecture
obligatoire. */
    public boolean isFresh(Instant dateLecture, Instant now) {
        return dateLecture != null && !dateLecture.plus(maxAge).isBefore(now);
    }
}
```

A2. Adaptateur mTLS, résilience et blocage des écritures

Le certificat client est lu dans le magasin monté par Kubernetes (`/etc/gimac/tls/client.p12`, émis par l'autorité interne SOC-09, mot de passe injecté par le coffre). Le réessai `changeRecord` (trois tentatives, 500 ms, multiplicateur 2) enveloppe le

disjoncteur (fenêtre de 10 appels, seuil 50 %, ouverture 30 s) ; le repli relève l'indisponibilité sans fabriquer de valeur. Un intercepteur refuse toute méthode autre que GET et journalise la tentative vers MS-AUDIT dans une transaction propre. `ChangeRecordStubAdapter` (profils `dev`, `test`) renvoie des fiches paramétrées.

```
// infrastructure/client/changerecord/ChangeRecordHttpAdapter.java
@Component
@ConditionalOnProperty(name = "gimac.adapters.change-record", havingValue = "real")
class ChangeRecordHttpAdapter implements ChangeRecordReaderPort {
    private final RestClient client;

    ChangeRecordHttpAdapter(ChangeRecordProperties p, ReadOnlyEgressInterceptor
readOnly) throws Exception {
        SSLContext ssl = SSLContextBuilder.create()
            .loadKeyMaterial(Path.of(p.keystorePath()), p.keystorePassword(),
p.keystorePassword())
            .loadTrustMaterial(Path.of(p.truststorePath()),
p.truststorePassword()).build();
        var pool = PoolingHttpClientConnectionManagerBuilder.create()
            .setSSLSocketFactory(SSLConnectionSocketFactoryBuilder.create().setSslCon
text(ssl).build()).build();
        this.client = RestClient.builder().baseUrl(p.baseUrl())          // $
{CHANGE_RECORD_BASE_URL}
            .requestFactory(new
HttpComponentsClientHttpRequestFactory(HttpClients.custom().setConnectionManager(pool
).build()))
            .requestInterceptor(readOnly).build();
    }

    @Override
    @CircuitBreaker(name = "changeRecord")
    @Retry(name = "changeRecord", fallbackMethod = "unavailable")
    public ChangeRecordView read(String reference) {
        byte[] body = client.get().uri("/changes/{ref}",
reference).retrieve().body(byte[].class);
        ChangeDto dto = JsonMapper.read(body, ChangeDto.class);
        return new ChangeRecordView(reference, dto.approvalState(),
dto.windowStart(), dto.windowEnd(), Sha256.hex(body));
    }

    private ChangeRecordView unavailable(String reference, Throwable cause) {
        throw new ChangeRecordUnavailableException(reference, cause);   // aucune
valeur par défaut
    }
}

// infrastructure/client/changerecord/ReadOnlyEgressInterceptor.java
@Component
class ReadOnlyEgressInterceptor implements ClientHttpRequestInterceptor {
    private final OutboxPublisher outboxPublisher;
    private final TransactionTemplate requiresNew;   // PROPAGATION_REQUIRES_NEW

    @Override
    public ClientHttpResponse intercept(HttpRequest req, byte[] body,
ClientHttpRequestExecution exec) throws IOException {
        if (req.getMethod() != HttpMethod.GET) {
            requiresNew.executeWithoutResult(s ->
outboxPublisher.publish("EgressPolicy", "change-record",
                "certification.egress-write-blocked.v1", Map.of("method",
req.getMethod().name(), "path", req.getURI().getPath())));
            throw new EgressWriteBlockedException(req.getMethod(),
req.getURI().getPath());
        }
        return exec.execute(req, body);
    }
}
```

La politique réseau (couches 3 et 4) limite l'egress au relais sortant de la zone d'échange (SOC-07), qui ne publie que la route de lecture ; le filtrage par méthode est porté par l'intercepteur et vérifié par ArchUnit.

```
# deploy/k8s/mscertification/networkpolicy-egress.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: mscertification-egress}
spec:
  podSelector: {matchLabels: {app: mscertification}}
  policyTypes: [Egress]
  egress:
    - to: [{namespaceSelector: {matchLabels: {gimac/zone: echange}}, podSelector:
{matchLabels: {gimac/flux: changements-lecture}}}]
      ports: [{protocol: TCP, port: 443}]
    - to: [{namespaceSelector: {matchLabels: {gimac/zone: services}}}]     # services
internes, Kafka, PostgreSQL
    - to: [{namespaceSelector: {}, podSelector: {matchLabels: {k8s-app: kube-dns}}}]
      ports: [{protocol: UDP, port: 53}]
```

A3. Cas d'utilisation et ressource

`requireFreshApproved` est appelé par l'autorisation (CER-04) et le procès-verbal (CER-10). Sa transaction propre conserve l'instantané relu et la condition `FICHE_CHANGEMENT_APPROUVEE` même quand la décision dépendante est suspendue.

```
// application/usecase/ChangeRecordService.java
@Service
@RequiredArgsConstructor
public class ChangeRecordService implements LinkChangeRecordUseCase,
RequireFreshChangeRecordUseCase {
    private final GoLiveDossierRepository dossiers;
    private final ChangeRecordSnapshotRepository snapshots;
    private final CompletenessConditionRepository conditions;
    private final ChangeRecordReaderPort reader;
    private final FreshnessRule freshness;
    private final Clock clock;

    @Transactional
    public ChangeRecordSnapshot link(UUID dossierId, String reference) {
        GoLiveDossier d = dossiers.findById(dossierId).orElseThrow(() ->
NotFound.dossier(dossierId));
        d.setReferenceFicheChangement(reference);                // seule la
référence est saisie
        return readAndStore(d);
    }

    @Transactional(propagation = Propagation.REQUIRES_NEW, noRollbackFor =
DomainProblemException.class)
    public ChangeRecordSnapshot requireFreshApproved(UUID dossierId, String usage) {
        GoLiveDossier d = dossiers.findById(dossierId).orElseThrow(() ->
NotFound.dossier(dossierId));
        ChangeRecordSnapshot last =
snapshots.findTopByDossierIdOrderByDateLectureDesc(dossierId)
            .orElseThrow(() -> new DomainProblemException(CONFLICT,
"CER_CHANGE_RECORD_MISSING", "Aucune fiche référencée"));
        ChangeRecordSnapshot cur = freshness.isFresh(last.getDateLecture(),
clock.instant()) ? last : readAndStore(d);
        if (!"APPROUVEE".equals(cur.getEtatApprobation())) {
            throw new DomainProblemException(CONFLICT,
"CER_CHANGE_RECORD_NOT_APPROVED",
                usage + " suspendu : fiche " + cur.getReference() + " à l'état " +
cur.getEtatApprobation());
        }
        return cur;
    }
```

```
    private ChangeRecordSnapshot readAndStore(GoLiveDossier d) {
        var v = reader.read(d.getReferenceFicheChangement());    // 503 remonte :
aucun instantané créé
        var s = new
ChangeRecordSnapshot().dossier(d).reference(v.reference()).etatApprobation(v.etatAppr
obation())
            .fenetreDebut(v.fenetreDebut()).fenetreFin(v.fenetreFin())
            .dateLecture(clock.instant()).empreinteReponse(v.empreinteReponse());
        conditions.mark(d.getId(), ConditionCode.FICHE_CHANGEMENT_APPROUVEE,
v.approuvee(),
            "dispositif de gestion des changements", "Lue le " + s.getDateLecture());
        return snapshots.save(s);
    }
}
```

`ChangeRecordResource` déclare `PUT /dossiers/{id}/fiche-de-changement` (`CER_CHANGE_RECORD_LINK`, corps `{"reference"}`) et `GET` (`CER_DOSSIER_READ`, dernier instantané avec `dateLecture` et `fraiche`) ; toute autre méthode renvoie 405.

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
class FreshnessRuleTest {
    FreshnessRule rule = new FreshnessRule(Duration.ofMinutes(15));
    Instant t0 = Instant.parse("2026-10-05T08:00:00Z");

    @Test @DisplayName("CER-09 : une lecture de quinze minutes exactement reste
fraîche")
    void boundary() { assertThat(rule.isFresh(t0, t0.plusSeconds(900))).isTrue(); }

    @Test @DisplayName("CER-09 : une lecture de plus de quinze minutes impose une
relecture")
    void stale() { assertThat(rule.isFresh(t0, t0.plusSeconds(901))).isFalse(); }
}

@ExtendWith(MockitoExtension.class)
class ChangeRecordServiceTest {
    @Mock GoLiveDossierRepository dossiers; @Mock ChangeRecordSnapshotRepository
snapshots;
    @Mock CompletenessConditionRepository conditions; @Mock ChangeRecordReaderPort
reader;
    Clock clock = Clock.fixed(Instant.parse("2026-10-05T08:30:00Z"), ZoneOffset.UTC);

    @Test @DisplayName("CER-09 : dispositif indisponible, aucune valeur par défaut et
décision bloquée")
    void unavailable() {
        var svc = new ChangeRecordService(dossiers, snapshots, conditions, reader,
new FreshnessRule(Duration.ofMinutes(15)), clock);
        UUID id = UUID.randomUUID();
        when(dossiers.findById(id)).thenReturn(Optional.of(new
GoLiveDossier().referenceFicheChangement("CHG-1")));

when(snapshots.findTopByDossierIdOrderByDateLectureDesc(id)).thenReturn(Optional.of(
            new
ChangeRecordSnapshot().etatApprobation("APPROUVEE").dateLecture(clock.instant().minus
Seconds(1200))));
        when(reader.read("CHG-1")).thenThrow(new
ChangeRecordUnavailableException("CHG-1", null));

        assertThatThrownBy(() -> svc.requireFreshApproved(id,
"Autorisation")).isInstanceOf(ChangeRecordUnavailableException.class);
        verify(snapshots, never()).save(any());
    }
}

class ChangeRecordAdapterArchTest {
```

```
    @Test @DisplayName("CER-09 : l'adaptateur n'appelle ni POST, ni PUT, ni PATCH, ni
DELETE")
    void readOnly() {
        JavaClasses c = new
ClassFileImporter().importPackages("org.gimac.des.certification.infrastructure.client
.changerecord");
        noClasses().should().callMethod(RestClient.class,
"post").orShould().callMethod(RestClient.class, "put")
            .orShould().callMethod(RestClient.class,
"patch").orShould().callMethod(RestClient.class, "delete")
            .orShould().callMethod(RestClient.class, "method",
HttpMethod.class).check(c);
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
@SpringBootTest @AutoConfigureMockMvc @Testcontainers @ActiveProfiles("test")
class ChangeRecordIT {
    @Container @ServiceConnection static PostgreSQLContainer<?> pg = new
PostgreSQLContainer<>("postgres:16-alpine");
    @Container @ServiceConnection static KafkaContainer kafka = new
KafkaContainer("apache/kafka:3.8.0");
    @RegisterExtension static WireMockExtension dispositif =
WireMockExtension.newInstance().options(wireMockConfig()
        .dynamicHttpsPort().needClientAuth(true).trustStorePath("src/test/resources/
tls/ca.p12")).build();
    @DynamicPropertySource static void props(DynamicPropertyRegistry r) {
        r.add("gimac.adapters.change-record", () -> "real");
        r.add("gimac.certification.change-record.base-url", () ->
dispositif.getRuntimeInfo().getHttpsBaseUrl());
    }
    @Autowired MockMvc mvc; @Autowired ChangeRecordService service; @Autowired
OutboxEventRepository outbox;
    @MockitoBean Clock clock;

    @Test @DisplayName("CER-09 : fiche retirée entre deux lectures, relecture puis
suspension avec motif")
    void withdrawnBetweenReads() throws Exception {
        UUID id = Fixtures.dossierAutorise();
        when(clock.instant()).thenReturn(Instant.parse("2026-10-05T08:00:00Z"));
        dispositif.stubFor(get("/changes/CHG-
7").inScenario("r").whenScenarioStateIs(STARTED)
            .willReturn(okJson(Fixtures.fiche("APPROUVEE"))).willSetStateTo("retiree"
));
        dispositif.stubFor(get("/changes/CHG-
7").inScenario("r").whenScenarioStateIs("retiree")
            .willReturn(okJson(Fixtures.fiche("RETIREE"))));
        mvc.perform(put("/api/v1/dossiers/{id}/fiche-de-changement",
id).with(jwtWith("CER_CHANGE_RECORD_LINK"))
                .contentType(APPLICATION_JSON).content("{\"reference\":\"CHG-7\"}"))
            .andExpect(status().isOk()).andExpect(jsonPath("$.etatApprobation").value
("APPROUVEE"));

        when(clock.instant()).thenReturn(Instant.parse("2026-10-05T08:16:00Z"));
        assertThatThrownBy(() -> service.requireFreshApproved(id, "Procès-verbal"))
            .hasFieldOrPropertyWithValue("code", "CER_CHANGE_RECORD_NOT_APPROVED");
        dispositif.verify(2, getRequestedFor(urlEqualTo("/changes/CHG-7")));
    }

    @Test @DisplayName("CER-09 : tentative d'écriture bloquée et journalisée vers MS-
AUDIT")
    void writeBlocked(@Autowired ReadOnlyEgressInterceptor interceptor) {
        RestClient rogue =
RestClient.builder().baseUrl(dispositif.baseUrl()).requestInterceptor(interceptor).bu
ild();
        assertThatThrownBy(() ->
```

```
rogue.post().uri("/changes").retrieve().toBodilessEntity())
            .isInstanceOf(EgressWriteBlockedException.class);
        dispositif.verify(0, postRequestedFor(anyUrl()));
        assertThat(outbox.findByEventType("certification.egress-write-
blocked.v1")).hasSize(1);
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: CER-09 Lecture seule de la fiche de changement

  Scénario: Nominal : lecture de la fiche
    Étant donné une référence de fiche de changement "CHG-7" saisie dans le dossier
    Quand la fiche est lue par l'interface en authentification mutuelle
    Alors l'état d'approbation et la fenêtre de changement (début, fin) sont
enregistrés avec l'instant de lecture

  Scénario: Alternatif : relecture avant usage
    Étant donné une lecture de la fiche "CHG-7" datant de 16 minutes
    Quand le procès-verbal doit s'appuyer sur la fiche
    Alors la fiche est relue avant tout usage
    Et si elle n'est plus approuvée, l'opération qui en dépend est suspendue avec le
code "CER_CHANGE_RECORD_NOT_APPROVED"

  Scénario: Erreur : dispositif indisponible
    Étant donné un dispositif de gestion des changements qui ne répond pas
    Quand une relecture est nécessaire
    Alors la réponse est 503 avec le code "CER_CHANGE_RECORD_UNAVAILABLE", aucune
valeur par défaut n'est retenue
    Et la décision dépendante reste bloquée jusqu'à une lecture réussie

  Scénario: Erreur : tentative d'écriture
    Étant donné la politique de flux sortants du service
    Quand une requête POST de création de fiche est émise vers le dispositif
    Alors elle est bloquée et l'événement "certification.egress-write-blocked.v1" est
journalisé vers MS-AUDIT
    Et le contrat du service ne déclare aucune route d'écriture vers le dispositif
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-conformite
# Nominal : référencer la fiche, puis consulter l'instantané lu
curl -s -X PUT "${GATEWAY_URL}/bff/des/certification/dossiers/${DOSSIER_ID}/fiche-de-
changement" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" \
  -d '{"reference":"CHG-2026-0412"}' | jq '{reference, etatApprobation, fenetreDebut,
fenetreFin, dateLecture}'

# Refus : aucune méthode d'écriture sur la fiche (attendu 405)
curl -s -o /dev/null -w "%{http_code}\n" -X POST -H "Authorization: Bearer $
{TOKEN}" \
  "${GATEWAY_URL}/bff/des/certification/dossiers/${DOSSIER_ID}/fiche-de-changement"

# Refus : un correspondant ne peut pas référencer une fiche (attendu 403)
source ./scripts/token.sh test-correspondant-a
curl -s -o /dev/null -w "%{http_code}\n" -X PUT -H "Authorization: Bearer ${TOKEN}" -
H "Content-Type: application/json" \
  -d '{"reference":"CHG-2026-0412"}' "${GATEWAY_URL}/bff/des/certification/dossiers/$
{DOSSIER_ID}/fiche-de-changement"
```

CER-10 | Consigner le procès-verbal de mise en service dans la fenêtre de changement

Élément	Valeur
Points	3
Priorité	Must Have
Sprint cible	Sprint 4
Écrans	DES-11
Dépendances	CER-04, CER-09, DOC-01

Objectif

Le procès-verbal de mise en service est déposé en WORM par MS-DOCUMENT, rattaché au dossier par un `GoLiveReport`, et son heure de bascule est comparée à la fenêtre d'une fiche relue depuis quinze minutes au plus ; le dossier passe de AUTORISE à MIS_EN_SERVICE et `certification.go-live-recorded.v1` est émis.

A. Implémentation

A1. Fenêtre de changement et port de dépôt

La fenêtre est un objet-valeur pur à bornes incluses. Le port `DocumentDepositPort` appelle l'interface interne de MS-DOCUMENT (`POST /api/v1/interne/documents`, mTLS, jeton de service, clé d'idempotence) ; `DocumentDepositStubAdapter` (profils `dev`, `test`) calcule l'empreinte et renvoie une référence locale. Ce port est réutilisé par CER-12 et CER-15.

```
// domain/service/ChangeWindow.java
public record ChangeWindow(Instant debut, Instant fin) {
    public ChangeWindow {
        if (debut == null || fin == null || fin.isBefore(debut)) throw new
IllegalArgumentException("Fenêtre invalide");
    }
    /** Bornes incluses : une bascule à l'instant exact du début ou de la fin est
dans la fenêtre. */
    public boolean contains(Instant heureBascule) {
        return !heureBascule.isBefore(debut) && !heureBascule.isAfter(fin);
    }
}

// application/port/out/DocumentDepositPort.java
public interface DocumentDepositPort {
    DepositedDocument deposit(DepositRequest request);
    record DepositRequest(String typePiece, UUID institutionId, String nomFichier,
String mediaType,
                          byte[] contenu, String cleIdempotence) {}
    record DepositedDocument(String documentRef, String empreinte) {}
}
```

A2. Cas d'utilisation : consigner le procès-verbal

L'état est contrôlé avant tout dépôt (aucune pièce orpheline). La fiche est relue par `requireFreshApproved` (CER-09) si la dernière lecture date de plus de quinze minutes. La transition passe par l'automate du dossier livré en CER-08.

```
// application/usecase/GoLiveReportService.java
```

```
@Service
@RequiredArgsConstructor
public class GoLiveReportService implements RecordGoLiveReportUseCase {
    private final GoLiveDossierRepository dossiers;
    private final GoLiveReportRepository reports;
    private final CompletenessConditionRepository conditions;
    private final RequireFreshChangeRecordUseCase changeRecords;
    private final DocumentDepositPort deposit;
    private final DossierStateMachine stateMachine;
    private final OutboxPublisher outboxPublisher;
    private final Clock clock;

    @Transactional
    public GoLiveReport record(UUID dossierId, Instant heureBascule, byte[] pdf, UUID
deposantId) {
        GoLiveDossier d = dossiers.findByIdForUpdate(dossierId).orElseThrow(() ->
NotFound.dossier(dossierId));
        if (d.getEtat() != EtatDossier.AUTORISE) {
            throw new DomainProblemException(CONFLICT, "CER_DOSSIER_NOT_AUTHORIZED",
                "Procès-verbal refusé : le dossier est à l'état " + d.getEtat());
        }
        ChangeRecordSnapshot fiche = changeRecords.requireFreshApproved(dossierId,
"Procès-verbal");
        boolean dansFenetre = new ChangeWindow(fiche.getFenetreDebut(),
fiche.getFenetreFin()).contains(heureBascule);

        var doc = deposit.deposit(new
DocumentDepositPort.DepositRequest("PROCES_VERBAL_MISE_EN_SERVICE",
            d.getInstitutionId(), "pv-" + dossierId + ".pdf", "application/pdf", pdf,
"pv-" + dossierId));
        GoLiveReport r = reports.save(new
GoLiveReport().dossier(d).documentRef(doc.documentRef()).empreinte(doc.empreinte())
            .heureBascule(heureBascule).dansFenetre(dansFenetre).regularise(false)
            .deposantId(deposantId).dateDepot(clock.instant()));

        stateMachine.transition(d, EtatDossier.MIS_EN_SERVICE);     // AUTORISE ->
MIS_EN_SERVICE
        d.setDateMiseEnService(heureBascule);
        conditions.mark(dossierId, ConditionCode.PROCES_VERBAL, dansFenetre, "procès-
verbal " + doc.documentRef(),
            dansFenetre ? "Bascule dans la fenêtre" : "Hors fenêtre : régularisation
attendue");
        outboxPublisher.publish("GoLiveDossier", dossierId.toString(),
"certification.go-live-recorded.v1",
            Map.of("dossierId", dossierId, "documentRef", doc.documentRef(),
"empreinte", doc.empreinte(),
                   "heureBascule", heureBascule, "dansFenetre", dansFenetre,
"ficheReference", fiche.getReference()));
        return r;
    }
}
```

A3. Régularisation d'une bascule hors fenêtre et ressource

Après chaque `link` (CER-09), `ChangeRecordService` appelle `GoLiveRegularizer.onNewSnapshot` : si un procès-verbal hors fenêtre existe et que la fiche nouvellement lue est approuvée et couvre l'heure de bascule, le procès-verbal est marqué régularisé et la condition `PROCES_VERBAL` devient satisfaite.

```
// application/usecase/GoLiveRegularizer.java
@Component
@RequiredArgsConstructor
class GoLiveRegularizer {
    private final GoLiveReportRepository reports;
    private final CompletenessConditionRepository conditions;
    private final OutboxPublisher outboxPublisher;
```

```
    @Transactional(propagation = Propagation.MANDATORY)
    void onNewSnapshot(UUID dossierId, ChangeRecordSnapshot s) {
        reports.findByDossierId(dossierId)
            .filter(r -> !r.getDansFenetre() && !r.getRegularise() &&
"APPROUVEE".equals(s.getEtatApprobation()))
            .filter(r -> new ChangeWindow(s.getFenetreDebut(),
s.getFenetreFin()).contains(r.getHeureBascule()))
            .ifPresent(r -> {
                r.setRegularise(true);
                conditions.mark(dossierId, ConditionCode.PROCES_VERBAL, true, "fiche
" + s.getReference(), "Régularisé");
                outboxPublisher.publish("GoLiveDossier", dossierId.toString(),
"certification.go-live-regularized.v1",
                    Map.of("dossierId", dossierId, "ficheReference",
s.getReference()));
            });
    }
}

// web/rest/GoLiveReportResource.java
@RestController
@RequestMapping("/api/v1/dossiers/{id}/proces-verbal")
@RequiredArgsConstructor
class GoLiveReportResource {
    private final RecordGoLiveReportUseCase useCase;
    private final CurrentUserProvider currentUser;
    private final GoLiveReportMapper mapper;

    @PostMapping(consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
    @PreAuthorize("hasAuthority('CER_GOLIVE_REPORT_SUBMIT')")
    ResponseEntity<GoLiveReportDTO> submit(@PathVariable UUID id,
@RequestPart("fichier") MultipartFile fichier,
                                           @RequestPart("heureBascule") String
heureBascule) throws IOException {
        var r = useCase.record(id, Instant.parse(heureBascule), fichier.getBytes(),
currentUser.current().id());
        return ResponseEntity.status(HttpStatus.CREATED).body(mapper.toDto(r));   //
porte transitionsPermises, actionsPermises
    }
}
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
class ChangeWindowTest {
    ChangeWindow w = new ChangeWindow(Instant.parse("2026-10-10T22:00:00Z"),
Instant.parse("2026-10-11T02:00:00Z"));

    @Test @DisplayName("CER-10 : les bornes de la fenêtre de changement sont
incluses")
    void boundsIncluded() {
        assertThat(w.contains(Instant.parse("2026-10-10T22:00:00Z"))).isTrue();
        assertThat(w.contains(Instant.parse("2026-10-11T02:00:00Z"))).isTrue();
    }

    @Test @DisplayName("CER-10 : une bascule une seconde après la fin est hors
fenêtre")
    void outside() { assertThat(w.contains(Instant.parse("2026-10-
11T02:00:01Z"))).isFalse(); }
}

@ExtendWith(MockitoExtension.class)
class GoLiveReportServiceTest {
    @Mock GoLiveDossierRepository dossiers; @Mock DocumentDepositPort deposit;
    @Mock RequireFreshChangeRecordUseCase changeRecords;
    @InjectMocks GoLiveReportService service;

    @Test @DisplayName("CER-10 : un dossier ouvert refuse le procès-verbal sans dépôt
```

```
ni relecture")
    void refusedWhenNotAuthorized() {
        UUID id = UUID.randomUUID();
        when(dossiers.findByIdForUpdate(id)).thenReturn(Optional.of(new
GoLiveDossier().etat(EtatDossier.OUVERT)));
        assertThatThrownBy(() -> service.record(id, Instant.now(), new byte[] {1},
UUID.randomUUID()))
            .hasFieldOrPropertyWithValue("code", "CER_DOSSIER_NOT_AUTHORIZED");
        verifyNoInteractions(deposit, changeRecords);
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
@SpringBootTest @AutoConfigureMockMvc @Testcontainers @ActiveProfiles("test")
class GoLiveReportIT {
    @Container @ServiceConnection static PostgreSQLContainer<?> pg = new
PostgreSQLContainer<>("postgres:16-alpine");
    @Container @ServiceConnection static KafkaContainer kafka = new
KafkaContainer("apache/kafka:3.8.0");
    @RegisterExtension static WireMockExtension msDocument =
WireMockExtension.newInstance().options(wireMockConfig().dynamicPort()).build();
    @DynamicPropertySource static void props(DynamicPropertyRegistry r) {
        r.add("gimac.adapters.document-deposit", () -> "real");
        r.add("gimac.clients.msdocument.base-url", msDocument::baseUrl);
    }
    @Autowired MockMvc mvc; @Autowired OutboxEventRepository outbox;

    @Test @DisplayName("CER-10 : procès-verbal refusé sur un dossier non autorisé,
sans dépôt ni événement")
    void refusedOnOpenDossier() throws Exception {
        UUID id = Fixtures.dossier(EtatDossier.OUVERT);
        mvc.perform(multipart("/api/v1/dossiers/{id}/proces-verbal", id)
                .file(new MockMultipartFile("fichier", "pv.pdf", "application/pdf",
Fixtures.pdfA()))
                .file(new MockMultipartFile("heureBascule", "", "text/plain", "2026-
10-10T23:00:00Z".getBytes()))
                .with(jwtWith("CER_GOLIVE_REPORT_SUBMIT")))
            .andExpect(status().isConflict()).andExpect(jsonPath("$.code").value("CER
_DOSSIER_NOT_AUTHORIZED"));
        msDocument.verify(0,
postRequestedFor(urlPathEqualTo("/api/v1/interne/documents")));
        assertThat(outbox.findByEventType("certification.go-live-
recorded.v1")).isEmpty();
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: CER-10 Procès-verbal de mise en service dans la fenêtre de changement

  Scénario: Nominal : procès-verbal dans la fenêtre
    Étant donné un dossier autorisé et une fiche de changement approuvée relue depuis
moins de quinze minutes
    Quand l'ingénieur d'homologation dépose le procès-verbal dont l'heure de bascule
est comprise dans la fenêtre
    Alors le dossier passe à l'état "MIS_EN_SERVICE"
    Et l'événement "certification.go-live-recorded.v1" est émis

  Scénario: Alternatif : bascule hors fenêtre
    Étant donné un procès-verbal dont l'heure de bascule est hors de la fenêtre
    Quand il est consigné
    Alors il est enregistré avec la mention « hors fenêtre »
    Et la condition "PROCES_VERBAL" reste non satisfaite jusqu'à la liaison d'une
```

```
fiche approuvée couvrant la bascule

  Scénario: Erreur : dossier non autorisé
    Étant donné un dossier à l'état "OUVERT"
    Quand un procès-verbal est déposé
    Alors le dépôt est refusé avec le code "CER_DOSSIER_NOT_AUTHORIZED" faute
d'autorisation de mise en service
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-ingenieur-homologation
# Nominal : dépôt du procès-verbal (attendu 201, dansFenetre=true,
etatDossier=MIS_EN_SERVICE)
curl -s -X POST "${GATEWAY_URL}/bff/des/certification/dossiers/${DOSSIER_ID}/proces-
verbal" \
  -H "Authorization: Bearer ${TOKEN}" \
  -F "fichier=@./fixtures/pv-mise-en-service.pdf;type=application/pdf" -F
"heureBascule=2026-10-10T23:00:00Z" \
  | jq '{documentRef, empreinte, dansFenetre, regularise, etatDossier}'

# Refus : dossier encore à l'état OUVERT (attendu 409)
curl -s -o /dev/null -w "%{http_code}\n" -X POST \
  "${GATEWAY_URL}/bff/des/certification/dossiers/${DOSSIER_OUVERT_ID}/proces-verbal"
-H "Authorization: Bearer ${TOKEN}" \
  -F "fichier=@./fixtures/pv-mise-en-service.pdf;type=application/pdf" -F
"heureBascule=2026-10-10T23:00:00Z"
```

CER-11 | Sceller et horodater les quitus et le dossier de mise en service

Élément	Valeur
Points	5
Priorité	Must Have
Sprint cible	Sprint 4
Écrans	DES-11
Dépendances	CER-02, CER-08, SOC-12, AUD-01

Objectif

Chaque quitus signé et chaque dossier complet reçoivent un jeton RFC 3161 demandé avec un nonce aléatoire sur leur empreinte SHA-256, vérifié intégralement (nonce, empreinte, chaîne jusqu'à l'ancre de confiance, cohérence temporelle) puis conservé dans `TimestampToken` ; `certification.dossier-sealed.v1` part par l'outbox vers MS-AUDIT, et un jeton invalide est rejeté avec alerte au RSSI.

A. Implémentation

A1. Port et vérification du jeton (Bouncy Castle)

L'adaptateur réel encode la `TimeStampReq` (SHA-256, nonce de 64 bits, `certReq=true`), l'envoie en `application/timestamp-query` à `${TSA_URL}` et confie la réponse à `TimestampTokenVerifier`. `TimestampAuthorityStubAdapter` (profils `dev`, `test`) signe localement avec une autorité de test. Toute erreur réseau devient `TsaUnavailableException` (réessai) ; toute incohérence devient `InvalidTimestampTokenException` (rejet).

```
// application/port/out/TimestampAuthorityPort.java
public interface TimestampAuthorityPort {
    VerifiedToken timestamp(byte[] sha256, BigInteger nonce);
    record VerifiedToken(byte[] encoded, Instant genTime, String autoriteChaine) {}
}

// infrastructure/client/tsa/Rfc3161TimestampAdapter.java
@Component
@ConditionalOnProperty(name = "gimac.adapters.timestamp-authority", havingValue =
"real")
@RequiredArgsConstructor
class Rfc3161TimestampAdapter implements TimestampAuthorityPort {
    private final RestClient tsaClient;                 // baseUrl ${TSA_URL}, flux
réseau unique déclaré (SOC-12)
    private final TimestampTokenVerifier verifier;
    private final Clock clock;

    @Override
    public VerifiedToken timestamp(byte[] sha256, BigInteger nonce) {
        TimeStampRequestGenerator gen = new TimeStampRequestGenerator();
        gen.setCertReq(true);
        TimeStampRequest req = gen.generate(TSPAlgorithms.SHA256, sha256, nonce);
        Instant sentAt = clock.instant();
        byte[] raw;
        try {
            raw =
tsaClient.post().contentType(MediaType.parseMediaType("application/timestamp-query"))
                .body(req.getEncoded()).retrieve().body(byte[].class);
        } catch (RestClientException | IOException e) {
```

```
            throw new TsaUnavailableException(e);
        }
        return verifier.verify(req, raw, sentAt, clock.instant());
    }
}

// infrastructure/client/tsa/TimestampTokenVerifier.java
public final class TimestampTokenVerifier {
    private final Set<TrustAnchor> anchors;             // ancre de l'autorité,
magasin monté par Kubernetes
    private final Duration tolerance;                   //
gimac.certification.tsa.tolerance=PT2M

    public VerifiedToken verify(TimeStampRequest req, byte[] raw, Instant sentAt,
Instant receivedAt) {
        try {
            TimeStampResponse resp = new TimeStampResponse(raw);
            resp.validate(req);                          // statut, nonce, empreinte,
algorithme
            TimeStampToken tok = resp.getTimeStampToken();
            var signer = (X509CertificateHolder)
tok.getCertificates().getMatches(tok.getSID()).iterator().next();
            tok.validate(new
JcaSimpleSignerInfoVerifierBuilder().setProvider("BC").build(signer)); // signature,
usage timeStamping
            Instant genTime = tok.getTimeStampInfo().getGenTime().toInstant();
            if (genTime.isBefore(sentAt.minus(tolerance)) ||
genTime.isAfter(receivedAt.plus(tolerance))) {
                throw new InvalidTimestampTokenException("COHERENCE_TEMPORELLE",
genTime.toString());
            }
            var jca = new JcaX509CertificateConverter().setProvider("BC");
            List<X509Certificate> chain = new ArrayList<>();   // du signataire vers
l'autorité, ancre exclue
            for (Object h : tok.getCertificates().getMatches(null))
chain.add(jca.getCertificate((X509CertificateHolder) h));
            PKIXParameters params = new PKIXParameters(anchors);
            params.setDate(Date.from(genTime));
            params.setRevocationEnabled(false);          // listes de révocation
conservées et contrôlées par SOC-12
            CertPathValidator.getInstance("PKIX").validate(
                CertificateFactory.getInstance("X.509").generateCertPath(chain),
params);
            return new VerifiedToken(tok.getEncoded(), genTime, signer.getSubject() +
" <- " + signer.getIssuer());
        } catch (TSPValidationException e) {
            throw new InvalidTimestampTokenException("NONCE_OU_EMPREINTE",
e.getMessage());
        } catch (TSPException | GeneralSecurityException | IOException |
OperatorCreationException e) {
            throw new InvalidTimestampTokenException("CHAINE_OU_SIGNATURE",
e.getMessage());
        }
    }
}
```

A2. Demandes en attente et réessai avec retard exponentiel

Une table technique `sealing_retry` (objet_type, objet_id, empreinte, tentatives, prochaine_tentative ; unicité objet_type + objet_id), créée par changelog Liquibase, porte l'état d'attente. CER-08 appelle `requestDossierSealing` au passage à COMPLET, CER-02 appelle `requestQuitusSealing` à la signature. Le planificateur traite les demandes échues toutes les dix secondes, chacune verrouillée dans sa transaction (`FOR UPDATE SKIP LOCKED`), avec un retard de 30 s doublé à chaque échec et plafonné à une heure.

```
// application/usecase/SealingService.java
```

```
@Service
@RequiredArgsConstructor
public class SealingService implements SealUseCase {
    private final SealingRetryRepository retries;
    private final TimestampTokenRepository tokens;
    private final GoLiveDossierRepository dossiers;
    private final TimestampAuthorityPort tsa;
    private final OutboxPublisher outboxPublisher;
    private final SecureRandom random = new SecureRandom();
    private final Clock clock;

    @Transactional(propagation = Propagation.MANDATORY)
    public void requestDossierSealing(GoLiveDossier d, String empreinteDossier) {
        d.setEtatScellement(EtatScellement.EN_ATTENTE);          // aucune
attestation possible avant SCELLE (CER-12)
        retries.save(new SealingRetry("DOSSIER", d.getId(), empreinteDossier, 0,
clock.instant()));
    }

    @Transactional
    public void attempt(UUID jobId) {
        SealingRetry job = retries.lockIfDue(jobId,
clock.instant()).orElse(null);   // FOR UPDATE SKIP LOCKED
        if (job == null) return;                                  // déjà pris par
une autre réplique
        BigInteger nonce = new BigInteger(64, random);
        try {
            var t = tsa.timestamp(HexFormat.of().parseHex(job.getEmpreinte()),
nonce);
            tokens.save(new
TimestampToken().objetType(job.getObjetType()).objetId(job.getObjetId())
                .empreinte(job.getEmpreinte()).nonce(nonce.toString(16)).jeton(Base64
.getEncoder().encodeToString(t.encoded()))
                .dateJeton(t.genTime()).verifie(true).autoriteChaine(t.autoriteChaine
()));
            retries.delete(job);
            if (job.isDossier()) {

dossiers.getReferenceById(job.getObjetId()).setEtatScellement(EtatScellement.SCELLE);
                outboxPublisher.publish("GoLiveDossier", job.getObjetId().toString(),
"certification.dossier-sealed.v1",
                    Map.of("dossierId", job.getObjetId(), "empreinte",
job.getEmpreinte(), "dateJeton", t.genTime()));
            } else {
                outboxPublisher.publish("Quitus", job.getObjetId().toString(),
"certification.quitus-sealed.v1",
                    Map.of("quitusId", job.getObjetId(), "empreinte",
job.getEmpreinte(), "dateJeton", t.genTime()));
            }
        } catch (TsaUnavailableException e) {
            long delay = Math.min(30L << Math.min(job.getTentatives(), 7), 3600L);
            job.setTentatives(job.getTentatives() + 1);
            job.setProchaineTentative(clock.instant().plusSeconds(delay));
        } catch (InvalidTimestampTokenException e) {
            retries.delete(job);
            if (job.isDossier())
dossiers.getReferenceById(job.getObjetId()).setEtatScellement(EtatScellement.REJETE);
            outboxPublisher.publish("GoLiveDossier", job.getObjetId().toString(),
"certification.timestamp-rejected.v1",
                Map.of("objetType", job.getObjetType(), "objetId", job.getObjetId(),
"motif", e.motif()));
            outboxPublisher.publish("Notification", job.getObjetId().toString(),
"certification.notification-requested.v1",
                Map.of("destinataireRole", "RSSI", "gravite", "HAUTE", "modele",
"JETON_HORODATAGE_INVALIDE",
                       "objetId", job.getObjetId(), "motif", e.motif()));
        }
    }
}
```

```
// infrastructure/scheduling/SealingScheduler.java
@Component
@RequiredArgsConstructor
class SealingScheduler {
    private final SealingRetryRepository retries;
    private final SealUseCase sealing;
    private final Clock clock;

    @Scheduled(fixedDelayString = "PT10S")
    void run() { retries.findDueIds(clock.instant(), 20).forEach(sealing::attempt); }
}
```

A3. Consultation des horodatages

`GET /dossiers/{id}/horodatages` (`CER_DOSSIER_READ`) renvoie les jetons du dossier et des quitus de son circuit : `objetType`, `objetId`, `empreinte`, `nonce`, `dateJeton`, `verifie`, `autoriteChaine`, plus `etatScellement` du dossier et, s'il est EN_ATTENTE, `prochaineTentative`.

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

La fixture `TestTsa` (sources de test) produit des réponses RFC 3161 signées par une autorité de test rattachée à une ancre de test, à l'aide de `TimeStampTokenGenerator`.

```
class TimestampTokenVerifierTest {
    TestTsa tsa = TestTsa.create();
    TimestampTokenVerifier verifier = new
TimestampTokenVerifier(Set.of(tsa.trustAnchor()), Duration.ofMinutes(2));
    byte[] digest = Sha256.of("dossier-42".getBytes());
    BigInteger nonce = BigInteger.valueOf(0x5EED);
    TimeStampRequest req = new
TimeStampRequestGenerator().generate(TSPAlgorithms.SHA256, digest, nonce);

    @Test @DisplayName("CER-11 : un jeton de référence est accepté (nonce, empreinte,
chaîne, date)")
    void referenceToken() {
        Instant now = Instant.now();
        var t = verifier.verify(req, tsa.respond(req), now.minusSeconds(1),
now.plusSeconds(1));
        assertThat(t.genTime()).isBetween(now.minusSeconds(120),
now.plusSeconds(120));
    }

    @Test @DisplayName("CER-11 : un jeton dont le nonce diffère de la demande est
rejeté")
    void alteredNonce() {
        var other = new TimeStampRequestGenerator().generate(TSPAlgorithms.SHA256,
digest, nonce.add(BigInteger.ONE));
        assertThatThrownBy(() -> verifier.verify(req, tsa.respond(other),
Instant.now(), Instant.now()))
            .isInstanceOf(InvalidTimestampTokenException.class).hasMessageContaining(
"NONCE_OU_EMPREINTE");
    }
}

@ExtendWith(MockitoExtension.class)
class SealingServiceTest {
    @Mock SealingRetryRepository retries; @Mock TimestampTokenRepository tokens;
@Mock GoLiveDossierRepository dossiers;
    @Mock TimestampAuthorityPort tsa; @Mock OutboxPublisher outboxPublisher;
    Clock clock = Clock.fixed(Instant.parse("2026-10-12T09:00:00Z"), ZoneOffset.UTC);

    @Test @DisplayName("CER-11 : autorité indisponible, la demande reste en attente
avec retard exponentiel")
```

```
    void unavailableKeepsPending() {
        var svc = new SealingService(retries, tokens, dossiers, tsa, outboxPublisher,
clock);
        var job = new SealingRetry("DOSSIER", UUID.randomUUID(), "ab".repeat(32), 2,
clock.instant());
        when(retries.lockIfDue(eq(job.getId()), any())).thenReturn(Optional.of(job));
        when(tsa.timestamp(any(), any())).thenThrow(new
TsaUnavailableException(null));
        svc.attempt(job.getId());
        assertThat(job.getTentatives()).isEqualTo(3);

assertThat(job.getProchaineTentative()).isEqualTo(clock.instant().plusSeconds(120));
        verifyNoInteractions(tokens, outboxPublisher);
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

L'autorité est simulée par WireMock avec un transformateur `TestTsaTransformer` qui signe la requête reçue (nonce et empreinte réels).

```
@SpringBootTest @Testcontainers @ActiveProfiles("test")
class DossierSealingIT {
    @Container @ServiceConnection static PostgreSQLContainer<?> pg = new
PostgreSQLContainer<>("postgres:16-alpine");
    @Container @ServiceConnection static KafkaContainer kafka = new
KafkaContainer("apache/kafka:3.8.0");
    @RegisterExtension static WireMockExtension tsa = WireMockExtension.newInstance()
        .options(wireMockConfig().dynamicPort().extensions(new
TestTsaTransformer())).build();
    @DynamicPropertySource static void props(DynamicPropertyRegistry r) {
        r.add("gimac.adapters.timestamp-authority", () -> "real");
        r.add("gimac.certification.tsa.url", tsa::baseUrl);
    }
    @Autowired SealingScheduler scheduler; @Autowired GoLiveDossierRepository
dossiers; @Autowired KafkaTestConsumer audit;

    @Test @DisplayName("CER-11 : indisponibilité de l'autorité puis reprise et
inscription au registre d'audit")
    void unavailableThenSealed() {
        UUID id = Fixtures.dossierCompletEnAttenteDeScellement();
        tsa.stubFor(post("/").willReturn(serviceUnavailable()));
        scheduler.run();

assertThat(dossiers.findById(id).orElseThrow().getEtatScellement()).isEqualTo(EtatSce
llement.EN_ATTENTE);

        tsa.stubFor(post("/").willReturn(aResponse().withTransformers("test-tsa")));
        Fixtures.rendreEchues();                                    // avance
prochaine_tentative
        scheduler.run();

assertThat(dossiers.findById(id).orElseThrow().getEtatScellement()).isEqualTo(EtatSce
llement.SCELLE);
        await().atMost(Duration.ofSeconds(10)).untilAsserted(() ->
            assertThat(audit.received("certification.dossier",
"certification.dossier-sealed.v1", id)).isTrue());
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: CER-11 Scellement et horodatage RFC 3161
```

```
  Scénario: Nominal : scellement du dossier
    Étant donné un dossier à l'état "COMPLET"
    Quand le scellement s'exécute
    Alors un jeton d'horodatage est demandé avec un nonce sur l'empreinte du dossier
puis vérifié (nonce, chaîne de certification, empreinte, cohérence temporelle)
    Et le jeton est conservé et l'événement "certification.dossier-sealed.v1" est
émis puis inscrit au registre d'audit

  Scénario: Alternatif : autorité d'horodatage indisponible
    Étant donné une autorité d'horodatage qui ne répond pas
    Quand le scellement est tenté
    Alors le dossier reste à l'état de scellement "EN_ATTENTE" et la demande est
réessayée avec retard exponentiel
    Et aucune attestation ne peut être émise avant l'obtention du jeton

  Scénario: Erreur : jeton invalide
    Étant donné un jeton dont le nonce ou l'empreinte ne correspond pas à la demande
    Quand il est vérifié
    Alors il est rejeté, l'état de scellement devient "REJETE" et une alerte est
émise vers le RSSI
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-chef-des
# Nominal : jetons du dossier et de ses quatre quitus
curl -s "${GATEWAY_URL}/bff/des/certification/dossiers/${DOSSIER_ID}/horodatages" \
  -H "Authorization: Bearer ${TOKEN}" | jq '{etatScellement, jetons: [.jetons[] |
{objetType, dateJeton, verifie}]}'

# Refus : attestation demandée alors que le scellement est en attente (attendu 409,
CER_DOSSIER_NOT_SEALED)
curl -s -o /dev/null -w "%{http_code}\n" -X POST -H "Authorization: Bearer $
{TOKEN}" \
  "${GATEWAY_URL}/bff/des/certification/dossiers/${DOSSIER_EN_ATTENTE_ID}/
attestation"

# Refus : sans jeton (attendu 401)
curl -s -o /dev/null -w "%{http_code}\n"
"${GATEWAY_URL}/bff/des/certification/dossiers/${DOSSIER_ID}/horodatages"
```

CER-12 | Émettre l'attestation de certification au format PDF/A-3 signée PAdES

Élément	Valeur
Points	8
Priorité	Must Have
Sprint cible	Sprint 4
Écrans	DES-11
Dépendances	CER-11, HOM-22, DOC-09, SOC-06

Objectif

Pour un dossier COMPLET et SCELLE, produire une seule attestation bilingue numérotée `GIMAC-CERT-AAAA-NNNNNN`, au format PDF/A-3 embarquant le dossier de preuves, les quitus et le procès-verbal, validée par veraPDF, signée PAdES-B-LT par le coffre de clés, déposée en WORM et référencée dans MS-AUDIT ; le dossier passe à ATTESTE et `certification.attestation-issued.v1` est émis.

A. Implémentation

A1. Numérotation unique

Le numéro combine l'année d'émission et une séquence PostgreSQL continue (`attestation_numero_seq`, créée par changelog Liquibase) ; les contraintes d'unicité portent sur `attestation.numero` et `attestation.dossier_id`. Un numéro consommé par une génération échouée n'est pas réattribué : un trou dans la séquence est admis et tracé par l'alerte.

```
// domain/service/AttestationNumber.java
public final class AttestationNumber {
    public static final Pattern FORMAT = Pattern.compile("GIMAC-CERT-\\d{4}-\\d{6}");
    private AttestationNumber() {}
    public static String of(int annee, long sequence) {
        if (sequence < 1 || sequence > 999_999) throw new
IllegalStateException("Séquence d'attestation épuisée");
        return "GIMAC-CERT-%04d-%06d".formatted(annee, sequence);
    }
}

// infrastructure/persistence/AttestationRepository.java (extrait)
@Query(value = "select nextval('attestation_numero_seq')", nativeQuery = true)
long nextSequence();
Optional<Attestation> findByDossierId(UUID dossierId);
Optional<Attestation> findByNumero(String numero);
```

A2. Ports sortants

`EvidenceFileExportPort` appelle le port interne d'export de MS-HOMOLOGATION (HOM-22) pour le socle identifié par son empreinte, puis récupère l'archive par MS-DOCUMENT ; `KeyVaultSigningPort` signe dans le coffre (SOC-06) sans jamais exporter la clé. Adaptateurs de substitution : export fictif composé à partir de fixtures, et signature par une clé de test PKCS#12 des sources de test.

```
// application/port/out/EvidenceFileExportPort.java
```

```
public interface EvidenceFileExportPort {
    ExportedEvidenceFile export(UUID liaisonId, String socleEmpreinte);
    record ExportedEvidenceFile(String documentRef, String empreinteManifeste, String
typeProjet,
                                List<String> servicesCertifies, byte[] archive) {}
}

// application/port/out/KeyVaultSigningPort.java
public interface KeyVaultSigningPort {
    List<X509Certificate> certificateChain();          // certificat de signature
des attestations et sa chaîne
    byte[] sign(byte[] dataToSign);                    // RSA PKCS#1 v1.5, SHA-256,
calculé dans le coffre
}

// infrastructure/client/keyvault/VaultTransitSigningAdapter.java
@Component
@ConditionalOnProperty(name = "gimac.adapters.key-vault-signing", havingValue =
"real")
@RequiredArgsConstructor
class VaultTransitSigningAdapter implements KeyVaultSigningPort {
    private final VaultOperations vault;                // authentification
Kubernetes, aucun secret en configuration
    private final SigningProperties props;              // nom de clé, chemin de la
chaîne publique

    @Override
    public byte[] sign(byte[] data) {
        Signature s = vault.opsForTransit().sign(props.keyName(),
VaultSignRequest.builder()
            .plaintext(Plaintext.of(data)).signatureAlgorithm("pkcs1v15").hashAlgorit
hm("sha2-256").build());
        return
Base64.getDecoder().decode(s.getSignature().substring(s.getSignature().lastIndexOf(':
') + 1));
    }

    @Override
    public List<X509Certificate> certificateChain() { return
PemChain.read(props.chainPath()); }
}
```

A3. Fabrique de PDF/A-3 signé (réutilisée par CER-15)

Le gabarit Thymeleaf bilingue (`templates/attestation.html`, clés `fr` et `en` en regard) est rendu en PDF/A-3b avec polices incorporées et profil ICC ; les pièces jointes sont incorporées avec `AFRelationship` et le tableau `/AF` exigés par PDF/A-3 ; veraPDF valide le document avant et après signature ; DSS produit une signature CMS détachée intégrée (PAdES-B-LT : jeton de signature par `${TSA_URL}`, données de révocation OCSP ou CRL incorporées).

```
// infrastructure/pdf/SignedPdfA3Factory.java
@Component
@RequiredArgsConstructor
public class SignedPdfA3Factory {
    private final TemplateEngine templates;
    private final PAdESService padesService;           // CertificateVerifier :
ancre GIMAC, OCSP, CRL, TSP ${TSA_URL}
    private final KeyVaultSigningPort keyVault;

    public record Attachment(String name, String mediaType, String relationship,
String description, byte[] content) {}

    public byte[] produce(String template, Map<String, Object> model,
List<Attachment> attachments) {
        byte[] pdf = embed(render(template, model), attachments);
        requirePdfA3(pdf);
```

```
        byte[] signed = sign(pdf);
        requirePdfA3(signed);                              // la révision incrémentale
doit rester conforme
        return signed;
    }

    private byte[] render(String template, Map<String, Object> model) {
        var out = new ByteArrayOutputStream();
        var b = new PdfRendererBuilder();
        b.usePdfAConformance(PdfRendererBuilder.PdfAConformance.PDFA_3_B);
        b.useFont(() -> getClass().getResourceAsStream("/fonts/LiberationSerif-
Regular.ttf"), "Times New Roman");
        b.useColorProfile(Resources.bytes("/icc/sRGB.icc"));
        b.withHtmlContent(templates.process(template, new Context(Locale.FRENCH,
model)), null).toStream(out);
        b.run();
        return out.toByteArray();
    }

    private byte[] embed(byte[] pdf, List<Attachment> attachments) {
        try (PDDocument doc = Loader.loadPDF(pdf); var out = new
ByteArrayOutputStream()) {
            Map<String, PDComplexFileSpecification> names = new LinkedHashMap<>();
            COSArray af = new COSArray();
            for (Attachment a : attachments) {
                PDEmbeddedFile ef = new PDEmbeddedFile(doc, new
ByteArrayInputStream(a.content()));
                ef.setSubtype(a.mediaType()); ef.setSize(a.content().length);
ef.setModDate(Calendar.getInstance());
                var fs = new PDComplexFileSpecification();
                fs.setFile(a.name()); fs.setFileUnicode(a.name());
fs.setFileDescription(a.description());
                fs.setEmbeddedFile(ef); fs.setEmbeddedFileUnicode(ef);
                fs.getCOSObject().setName(COSName.getPDFName("AFRelationship"),
a.relationship());
                names.put(a.name(), fs); af.add(fs);
            }
            var tree = new PDEmbeddedFilesNameTreeNode(); tree.setNames(names);
            var nd = new PDDocumentNameDictionary(doc.getDocumentCatalog());
nd.setEmbeddedFiles(tree);
            doc.getDocumentCatalog().setNames(nd);
            doc.getDocumentCatalog().getCOSObject().setItem(COSName.getPDFName("AF"),
af);
            doc.save(out);
            return out.toByteArray();
        } catch (IOException e) { throw new PdfAValidationException("Incorporation
impossible", e); }
    }

    private void requirePdfA3(byte[] pdf) {
        try (PDFAParser parser = Foundries.defaultInstance().createParser(new
ByteArrayInputStream(pdf), PDFAFlavour.PDFA_3_B);
             PDFAValidator validator =
Foundries.defaultInstance().createValidator(PDFAFlavour.PDFA_3_B, false)) {
            ValidationResult r = validator.validate(parser);
            if (!r.isCompliant()) throw new PdfAValidationException("Non conforme
PDF/A-3b : " + r.getTestAssertions().size() + " assertions");
        } catch (ModelParsingException | ValidationException | IOException e) { throw
new PdfAValidationException("veraPDF", e); }
    }

    private byte[] sign(byte[] pdf) {
        try {
            List<X509Certificate> chain = keyVault.certificateChain();
            var p = new PAdESSignatureParameters();
            p.setSignatureLevel(SignatureLevel.PAdES_BASELINE_LT);
            p.setDigestAlgorithm(DigestAlgorithm.SHA256);
            p.setSigningCertificate(new CertificateToken(chain.get(0)));
```

```
p.setCertificateChain(chain.stream().map(CertificateToken::new).toList());
            DSSDocument doc = new InMemoryDocument(pdf);
            ToBeSigned tbs = padesService.getDataToSign(doc, p);
            var value = new SignatureValue(SignatureAlgorithm.RSA_SHA256,
keyVault.sign(tbs.getBytes()));
            return DSSUtils.toByteArray(padesService.signDocument(doc, p, value));
        } catch (DSSException | KeyVaultUnavailableException e) { throw new
AttestationSigningException(e); }
    }
}
```

A4. Cas d'utilisation idempotent et ressource

Le dossier est verrouillé (`PESSIMISTIC_WRITE`) : deux demandes simultanées ne produisent qu'une attestation. En cas d'échec de validation (422, `CER_ATTESTATION_PDFA_INVALID`) ou de signature (503, `CER_ATTESTATION_SIGNING_FAILED`), rien n'est déposé, la transaction est annulée (le dossier reste COMPLET) et l'alerte est publiée dans une transaction propre.

```
// application/usecase/IssueAttestationService.java
@Service
@RequiredArgsConstructor
public class IssueAttestationService implements IssueAttestationUseCase {
    private final GoLiveDossierRepository dossiers;
    private final AttestationRepository attestations;
    private final QuitusRepository quitus;
    private final TimestampTokenRepository tokens;
    private final GoLiveReportRepository reports;
    private final EvidenceFileExportPort evidenceExport;
    private final DocumentContentPort documents;
    private final DocumentDepositPort deposit;
    private final SignedPdfA3Factory pdfFactory;
    private final DossierStateMachine stateMachine;
    private final AttestationAlertPublisher alerts;      //
@Transactional(REQUIRES_NEW)
    private final OutboxPublisher outboxPublisher;
    private final Clock clock;

    @Transactional
    public IssueResult issue(UUID dossierId) {
        GoLiveDossier d = dossiers.findByIdForUpdate(dossierId).orElseThrow(() ->
NotFound.dossier(dossierId));
        Optional<Attestation> existing = attestations.findByDossierId(dossierId);
        if (existing.isPresent()) return
IssueResult.existing(existing.get());      // ni numéro, ni dépôt
        if (d.getEtat() != EtatDossier.COMPLET || d.getEtatScellement() !=
EtatScellement.SCELLE) {
            throw new DomainProblemException(CONFLICT, "CER_DOSSIER_NOT_SEALED",
"Dossier non complet ou non scellé");
        }
        CertificationCircuit circuit = d.getCircuit();
        var preuves = evidenceExport.export(d.getLiaisonId(),
circuit.getSocleEmpreinte());
        String numero = AttestationNumber.of(Year.now(clock).getValue(),
attestations.nextSequence());
        Instant now = clock.instant();

        List<SignedPdfA3Factory.Attachment> pj = new ArrayList<>();
        pj.add(new SignedPdfA3Factory.Attachment("dossier-de-preuves.zip",
"application/zip", "Source",
            "Dossier de preuves, manifeste " + preuves.empreinteManifeste(),
preuves.archive()));
        quitus.findByCircuitId(circuit.getId()).forEach(q -> pj.add(new
SignedPdfA3Factory.Attachment(
            "quitus-" + q.getNature().name().toLowerCase() + ".json",
"application/json", "Supplement",
            "Quitus signé et jeton RFC 3161", QuitusExport.json(q,
tokens.findQuitusToken(q.getId()))));
```

```
        GoLiveReport pv = reports.findByDossierId(dossierId).orElseThrow();
        pj.add(new SignedPdfA3Factory.Attachment("proces-verbal.pdf",
"application/pdf", "Supplement",
            "Procès-verbal de mise en service",
documents.fetch(pv.getDocumentRef())));

        Map<String, Object> model = Map.of("numero", numero, "institutionId",
d.getInstitutionId(),
            "typeProjet", preuves.typeProjet(), "services",
preuves.servicesCertifies(),
            "dateEmission", now, "socleEmpreinte", circuit.getSocleEmpreinte());
        byte[] signed;
        try {
            signed = pdfFactory.produce("attestation", model, pj);
        } catch (PdfAValidationException | AttestationSigningException e) {
            alerts.generationFailed(dossierId, numero, e);
            throw e;
        }
        var doc = deposit.deposit(new
DocumentDepositPort.DepositRequest("ATTESTATION_CERTIFICATION",
            d.getInstitutionId(), numero + ".pdf", "application/pdf", signed,
"attestation-" + dossierId));
        Attestation a = attestations.save(new
Attestation().dossier(d).numero(numero).institutionId(d.getInstitutionId())
            .typeProjet(preuves.typeProjet()).servicesCertifies(Json.write(preuves.se
rvicesCertifies()))
            .dateEmission(now).socleEmpreinte(circuit.getSocleEmpreinte()).documentRe
f(doc.documentRef())
            .empreinteDocument(doc.empreinte()).etat(EtatAttestation.EN_VIGUEUR).refe
renceAudit("CER-ATT-" + numero));
        stateMachine.transition(d, EtatDossier.ATTESTE);
// COMPLET -> ATTESTE
        outboxPublisher.publish("Attestation", numero, "certification.attestation-
issued.v1", AttestationIssuedPayload.of(a));
        return IssueResult.created(a);
    }
}
```

`AttestationResource` : `POST /dossiers/{id}/attestation` (`CER_ATTESTATION_ISSUE`) répond 201 avec `Location: /api/v1/attestations/{numero}` à la première émission, 200 avec la même attestation ensuite ; `GET /attestations/{numero}` (`CER_DOSSIER_READ`) renvoie numéro, institution, type de projet, services, date, empreintes, état, `referenceAudit`, `transitionsPermises` et `actionsPermises`.

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
@ExtendWith(MockitoExtension.class)
class IssueAttestationServiceTest {
    @Mock GoLiveDossierRepository dossiers; @Mock AttestationRepository attestations;
    @Mock DocumentDepositPort deposit; @Mock SignedPdfA3Factory pdfFactory; @Mock
AttestationAlertPublisher alerts;
    @Mock EvidenceFileExportPort evidenceExport; @Mock OutboxPublisher
outboxPublisher;
    @Mock QuitusRepository quitus; @Mock TimestampTokenRepository tokens; @Mock
GoLiveReportRepository reports;
    @Mock DocumentContentPort documents; @Mock DossierStateMachine stateMachine;
@Mock Clock clock;
    @InjectMocks IssueAttestationService service;
    UUID id = UUID.randomUUID();

    @Test @DisplayName("CER-12 : une demande répétée renvoie l'attestation existante
sans numéro ni dépôt")
    void idempotent() {
        var existing = new Attestation().numero("GIMAC-CERT-2026-000042");
        when(dossiers.findByIdForUpdate(id)).thenReturn(Optional.of(new
```

```
GoLiveDossier().etat(EtatDossier.ATTESTE)));
        when(attestations.findByDossierId(id)).thenReturn(Optional.of(existing));
        assertThat(service.issue(id).attestation().getNumero()).isEqualTo("GIMAC-
CERT-2026-000042");
        verify(attestations, never()).nextSequence();
        verifyNoInteractions(deposit, pdfFactory, outboxPublisher);
    }

    @Test @DisplayName("CER-12 : le numéro respecte le format GIMAC-CERT-AAAA-
NNNNNN")
    void numberFormat() {
        assertThat(AttestationNumber.of(2026, 42)).isEqualTo("GIMAC-CERT-2026-
000042").matches(AttestationNumber.FORMAT);
        assertThatThrownBy(() -> AttestationNumber.of(2026,
1_000_000)).isInstanceOf(IllegalStateException.class);
    }

    @Test @DisplayName("CER-12 : un échec de validation PDF/A-3 n'entraîne aucun
dépôt et émet une alerte")
    void validationFailure() {

when(dossiers.findByIdForUpdate(id)).thenReturn(Optional.of(Fixtures.dossierCompletSc
elle(id)));
        when(attestations.findByDossierId(id)).thenReturn(Optional.empty());
        when(evidenceExport.export(any(),
any())).thenReturn(Fixtures.exportPreuves());
        when(attestations.nextSequence()).thenReturn(7L);
        when(clock.instant()).thenReturn(Instant.parse("2026-10-20T10:00:00Z"));
        when(clock.getZone()).thenReturn(ZoneOffset.UTC);
        when(reports.findByDossierId(id)).thenReturn(Optional.of(new
GoLiveReport().documentRef("DOC-PV")));
        when(pdfFactory.produce(any(), any(), any())).thenThrow(new
PdfAValidationException("polices non incorporées"));
        assertThatThrownBy(() ->
service.issue(id)).isInstanceOf(PdfAValidationException.class);
        verify(alerts).generationFailed(eq(id), anyString(), any());
        verifyNoInteractions(deposit, outboxPublisher);
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

Le coffre est remplacé par l'adaptateur de substitution (clé de test), MS-HOMOLOGATION et MS-DOCUMENT par WireMock ; la vérification est faite par le validateur DSS (`SignedDocumentValidator`), distinct de la chaîne de production, avec une source CRL hors ligne de test.

```
@SpringBootTest @AutoConfigureMockMvc @Testcontainers @ActiveProfiles("test")
class AttestationIssueIT {
    @Container @ServiceConnection static PostgreSQLContainer<?> pg = new
PostgreSQLContainer<>("postgres:16-alpine");
    @Container @ServiceConnection static KafkaContainer kafka = new
KafkaContainer("apache/kafka:3.8.0");
    @RegisterExtension static WireMockExtension tiers =
WireMockExtension.newInstance().options(wireMockConfig().dynamicPort()).build();
    @DynamicPropertySource static void props(DynamicPropertyRegistry r) {
        r.add("gimac.clients.mshomologation.base-url", tiers::baseUrl);
        r.add("gimac.clients.msdocument.base-url", tiers::baseUrl);
        r.add("gimac.adapters.document-deposit", () -> "real");
        r.add("gimac.adapters.evidence-export", () -> "real");
    }
    @Autowired MockMvc mvc; @Autowired CertificateVerifier testVerifier;

    @Test @DisplayName("CER-12 : attestation conforme PDF/A-3 et signature PAdES-B-LT
vérifiée par un validateur indépendant")
    void issueAndVerify() throws Exception {
```

```
        UUID id = Fixtures.dossierCompletScelle(tiers);                 // stubs
export, procès-verbal, dépôt
        mvc.perform(post("/api/v1/dossiers/{id}/attestation",
id).with(jwtWith("CER_ATTESTATION_ISSUE")))
            .andExpect(status().isCreated()).andExpect(jsonPath("$.numero").value(mat
chesPattern("GIMAC-CERT-\\d{4}-\\d{6}")));
        byte[] pdf = Fixtures.captureDepot(tiers, "ATTESTATION_CERTIFICATION");

        try (var parser = Foundries.defaultInstance().createParser(new
ByteArrayInputStream(pdf), PDFAFlavour.PDFA_3_B);
             var v =
Foundries.defaultInstance().createValidator(PDFAFlavour.PDFA_3_B, false)) {
            assertThat(v.validate(parser).isCompliant()).isTrue();
        }
        var validator = SignedDocumentValidator.fromDocument(new
InMemoryDocument(pdf));
        validator.setCertificateVerifier(testVerifier);
        var report = validator.validateDocument().getSimpleReport();

assertThat(report.getSignatureFormat(report.getFirstSignatureId())).isEqualTo(Signatu
reLevel.PAdES_BASELINE_LT);

assertThat(report.getIndication(report.getFirstSignatureId())).isEqualTo(Indication.T
OTAL_PASSED);
        try (PDDocument doc = Loader.loadPDF(pdf)) {

assertThat(doc.getDocumentCatalog().getNames().getEmbeddedFiles().getNames())
                .containsKeys("dossier-de-preuves.zip", "quitus-technique.json",
"proces-verbal.pdf");
        }
        mvc.perform(post("/api/v1/dossiers/{id}/attestation",
id).with(jwtWith("CER_ATTESTATION_ISSUE")))
            .andExpect(status().isOk());
        tiers.verify(1,
postRequestedFor(urlPathEqualTo("/api/v1/interne/documents")));
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: CER-12 Attestation de certification PDF/A-3 signée PAdES

  Scénario: Nominal : émission de l'attestation
    Étant donné un dossier complet et scellé pour l'Établissement de paiement A
    Quand l'attestation est générée
    Alors le document porte un numéro unique, l'institution, le type de projet, les
services certifiés, la date et l'empreinte du socle
    Et il intègre en pièces jointes le dossier de preuves, les quitus et le procès-
verbal
    Et il est validé PDF/A-3, signé PAdES au niveau long terme par le coffre de clés,
déposé via MS-DOCUMENT et référencé dans MS-AUDIT
    Et l'événement "certification.attestation-issued.v1" est émis et le dossier passe
à l'état "ATTESTE"

  Scénario: Alternatif : demande répétée
    Étant donné une attestation déjà émise pour le dossier
    Quand une nouvelle génération est demandée
    Alors la référence de l'attestation existante est renvoyée sans nouveau numéro ni
nouveau dépôt

  Scénario: Erreur : validation ou signature en échec
    Étant donné un document qui échoue à la validation PDF/A-3 ou à la signature
    Quand la génération s'achève
    Alors aucune pièce n'est déposée, le dossier reste à l'état "COMPLET" et une
alerte est émise
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-chef-des
# Nominal : émission (attendu 201) puis demande répétée (attendu 200, même numéro)
curl -s -X POST
"${GATEWAY_URL}/bff/des/certification/dossiers/${DOSSIER_ID}/attestation" \
  -H "Authorization: Bearer ${TOKEN}" | tee /tmp/attestation.json | jq '{numero,
etat, empreinteDocument, referenceAudit}'
NUMERO=$(jq -r .numero /tmp/attestation.json)
curl -s -X POST
"${GATEWAY_URL}/bff/des/certification/dossiers/${DOSSIER_ID}/attestation" \
  -H "Authorization: Bearer ${TOKEN}" | jq -r .numero                    # identique
à ${NUMERO}
curl -s "${GATEWAY_URL}/bff/des/certification/attestations/${NUMERO}" \
  -H "Authorization: Bearer ${TOKEN}" | jq '{numero, typeProjet, servicesCertifies,
dateEmission, socleEmpreinte}'

# Refus : l'ingénieur d'homologation n'émet pas d'attestation (attendu 403)
source ./scripts/token.sh test-ingenieur-homologation
curl -s -o /dev/null -w "%{http_code}\n" -X POST -H "Authorization: Bearer $
{TOKEN}" \
  "${GATEWAY_URL}/bff/des/certification/dossiers/${DOSSIER_ID}/attestation"
```

CER-13 | Consulter sa certification et télécharger l'attestation depuis le portail

Élément	Valeur
Points	3
Priorité	Should Have
Sprint cible	Sprint 4
Écrans	POR-03
Dépendances	CER-12, DOC-05, GW-13, IHM-07

Objectif

Le correspondant consulte, sous `/api/v1/portail/certifications`, les dossiers de sa seule institution (identifiée par le claim `institution_id` du jeton) dans une projection sans annotations internes, et télécharge l'attestation restituée par MS-DOCUMENT avec son empreinte, chaque téléchargement étant inscrit au journal de consultation ; toute ressource d'une autre institution répond 404 et la tentative est journalisée.

A. Implémentation

A1. Projection du portail par liste blanche

La projection est construite champ par champ : aucun identifiant de signataire, aucune référence d'audit, aucun détail de condition ni aucune opération désignée n'en sort. Le motif d'un quitus n'est exposé que s'il est défavorable : CER-02 impose que ce motif soit rédigé pour être communiqué au participant. La mention d'état de l'attestation (suspendue, révoquée) est alimentée par CER-15.

```
// web/rest/portail/PortalCertificationView.java
public record PortalCertificationView(UUID id, String typeProjet, EtatDossier
etatDossier, List<QuitusView> quitus,
                                      AttestationView attestation, List<String>
actionsPermises) {
    public record QuitusView(NatureQuitus nature, EtatQuitus etat, String
motifCommunicable) {}
    public record AttestationView(String numero, Instant dateEmission,
EtatAttestation etat, String empreinteDocument, String mention) {}
}

// web/rest/portail/PortalCertificationMapper.java
@Component
class PortalCertificationMapper {
    PortalCertificationView toView(GoLiveDossier d, List<Quitus> quitus,
Optional<Attestation> attestation) {
        var q = quitus.stream().map(x -> new
PortalCertificationView.QuitusView(x.getNature(), x.getEtat(),
            x.getEtat() == EtatQuitus.SIGNE_DEFAVORABLE ?
x.getDecision().getMotif() : null)).toList();
        var a = attestation.map(t -> new
PortalCertificationView.AttestationView(t.getNumero(), t.getDateEmission(),
            t.getEtat(), t.getEmpreinteDocument(), switch (t.getEtat()) {
                case SUSPENDUE -> "portail.attestation.mention.suspendue";
                case REVOQUEE -> "portail.attestation.mention.revoquee";
                default -> null;
            })).orElse(null);
        List<String> actions = a != null ? List.of("TELECHARGER_ATTESTATION") :
List.of();
```

```
        return new PortalCertificationView(d.getId(),
attestation.map(Attestation::getTypeProjet).orElse(null),
            d.getEtat(), q, a, actions);
    }
}
```

A2. Cloisonnement et restitution journalisée

L'institution vient exclusivement du jeton. Une ressource absente ou appartenant à une autre institution produit la même réponse 404 (`CER_NOT_FOUND`), et la tentative est journalisée par `certification.portal-access-denied.v1` dans une transaction propre. La restitution passe par `DocumentContentPort.stream`, qui appelle `GET /api/v1/documents/{ref}/contenu` de MS-DOCUMENT (DOC-05) au nom du correspondant : MS-DOCUMENT inscrit la consultation au journal avant toute remise et vérifie l'empreinte à la volée.

```
// application/usecase/PortalCertificationService.java
@Service
@RequiredArgsConstructor
@Transactional(readOnly = true)
public class PortalCertificationService implements PortalCertificationUseCase {
    private final GoLiveDossierRepository dossiers;
    private final QuitusRepository quitus;
    private final AttestationRepository attestations;
    private final DocumentContentPort documents;
    private final PortalAccessAuditor auditor;          //
@Transactional(REQUIRES_NEW) + outboxPublisher.publish(...)
    private final PortalCertificationMapper mapper;

    public List<PortalCertificationView> list(CurrentUser user) {
        return
dossiers.findByInstitutionId(user.institutionId()).stream().map(this::view).toList();
    }

    public DocumentStream downloadAttestation(UUID dossierId, CurrentUser user) {
        GoLiveDossier d = dossiers.findByIdAndInstitutionId(dossierId,
user.institutionId())
            .orElseThrow(() -> auditor.deniedAndNotFound(dossierId, user));
        Attestation a = attestations.findByDossierId(d.getId())
            .orElseThrow(() -> new DomainProblemException(NOT_FOUND, "CER_NOT_FOUND",
"Aucune attestation émise"));
        return documents.stream(a.getDocumentRef(), new
ConsultationContext(user.id(), user.institutionId(),
            "PORTAIL_ATTESTATION"), a.getEmpreinteDocument(), a.getNumero() +
".pdf");
    }

    private PortalCertificationView view(GoLiveDossier d) {
        return mapper.toView(d, quitus.findByCircuitId(d.getCircuit().getId()),
attestations.findByDossierId(d.getId()));
    }
}

// web/rest/portail/PortalCertificationResource.java
@RestController
@RequestMapping("/api/v1/portail/certifications")
@PreAuthorize("hasAuthority('CER_PORTAL_READ')")
@RequiredArgsConstructor
class PortalCertificationResource {
    private final PortalCertificationUseCase useCase;
    private final CurrentUserProvider currentUser;       // institutionId lu dans le
claim institution_id

    @GetMapping
    List<PortalCertificationView> list() { return
```

```
useCase.list(currentUser.current()); }

    @GetMapping("/{id}/attestation")
    ResponseEntity<StreamingResponseBody> attestation(@PathVariable UUID id) {
        DocumentStream s = useCase.downloadAttestation(id, currentUser.current());
        return ResponseEntity.ok().contentType(MediaType.APPLICATION_PDF)
            .header(HttpHeaders.CONTENT_DISPOSITION,
ContentDisposition.attachment().filename(s.fileName()).build().toString())
            .header("X-Content-SHA256", s.empreinte())
            .body(s::transferTo);
    }
}
```

A3. Notification du correspondant à l'émission

Un consommateur idempotent du sujet `certification.attestation` (groupe `mscertification-portail`) demande à MS-NOTIFICATION d'informer les correspondants de l'institution, en français et en anglais. L'écran bilingue POR-03 est traité dans la partie frontend du plan.

```
// infrastructure/messaging/consumer/AttestationIssuedPortalNotifier.java
@Component
@RequiredArgsConstructor
class AttestationIssuedPortalNotifier {
    private final IdempotentConsumer idempotentConsumer;
    private final OutboxPublisher outboxPublisher;

    @KafkaListener(topics = "certification.attestation", groupId = "mscertification-
portail")
    @Transactional
    void on(CloudEventEnvelope<AttestationIssuedPayload> e) {
        if (!"certification.attestation-issued.v1".equals(e.type())) return;
        idempotentConsumer.handleOnce(e.id(), "mscertification-portail", () ->
            outboxPublisher.publish("Notification", e.data().numero(),
"certification.notification-requested.v1",
                Map.of("destinataireRole", "CORRESPONDANT_INSTITUTION",
"institutionId", e.data().institutionId(),
                       "modele", "ATTESTATION_EMISE", "langues", List.of("fr", "en"),
"numero", e.data().numero())));
    }
}
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
class PortalCertificationMapperTest {
    PortalCertificationMapper mapper = new PortalCertificationMapper();

    @Test @DisplayName("CER-13 : la projection ne porte ni signataire ni annotation
interne, seul le motif défavorable est exposé")
    void noInternalAnnotations() {
        var defav = Fixtures.quitus(NatureQuitus.SECURITE,
EtatQuitus.SIGNE_DEFAVORABLE, "Correctif de chiffrement attendu");
        var fav = Fixtures.quitus(NatureQuitus.TECHNIQUE, EtatQuitus.SIGNE_FAVORABLE,
"Note interne du DES");
        var v = mapper.toView(Fixtures.dossier(EtatDossier.OUVERT), List.of(fav,
defav), Optional.empty());

assertThat(v.quitus()).extracting(PortalCertificationView.QuitusView::motifCommunicab
le)
            .containsExactly(null, "Correctif de chiffrement attendu");
        assertThat(Json.write(v)).doesNotContain("signataire", "referenceAudit",
"Note interne du DES");
    }
}
```

```
@ExtendWith(MockitoExtension.class)
class PortalCertificationServiceTest {
    @Mock GoLiveDossierRepository dossiers; @Mock PortalAccessAuditor auditor; @Mock
DocumentContentPort documents;
    @InjectMocks PortalCertificationService service;

    @Test @DisplayName("CER-13 : l'attestation d'une autre institution est
introuvable et la tentative est journalisée")
    void otherInstitution() {
        var banqueB = new CurrentUser(UUID.randomUUID(),
Set.of("CORRESPONDANT_INSTITUTION"), Fixtures.BANQUE_B);
        UUID dossierA = UUID.randomUUID();
        when(dossiers.findByIdAndInstitutionId(dossierA,
Fixtures.BANQUE_B)).thenReturn(Optional.empty());
        when(auditor.deniedAndNotFound(dossierA, banqueB))
            .thenReturn(new DomainProblemException(NOT_FOUND, "CER_NOT_FOUND",
"Introuvable"));
        assertThatThrownBy(() -> service.downloadAttestation(dossierA,
banqueB)).hasFieldOrPropertyWithValue("code", "CER_NOT_FOUND");
        verifyNoInteractions(documents);
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
@SpringBootTest @AutoConfigureMockMvc @Testcontainers @ActiveProfiles("test")
class PortalCertificationIT {
    @Container @ServiceConnection static PostgreSQLContainer<?> pg = new
PostgreSQLContainer<>("postgres:16-alpine");
    @Container @ServiceConnection static KafkaContainer kafka = new
KafkaContainer("apache/kafka:3.8.0");
    @RegisterExtension static WireMockExtension msDocument =
WireMockExtension.newInstance().options(wireMockConfig().dynamicPort()).build();
    @DynamicPropertySource static void props(DynamicPropertyRegistry r) {
        r.add("gimac.adapters.document-content", () -> "real");
        r.add("gimac.clients.msdocument.base-url", msDocument::baseUrl);
    }
    @Autowired MockMvc mvc; @Autowired OutboxEventRepository outbox;

    @Test @DisplayName("CER-13 : le téléchargement est inscrit au journal de
consultation et restitué avec son empreinte")
    void downloadJournaled() throws Exception {
        var att = Fixtures.attestationEmise(Fixtures.ETABLISSEMENT_A);
        msDocument.stubFor(get("/api/v1/documents/" + att.getDocumentRef() +
"/contenu")
            .willReturn(ok().withBody(Fixtures.pdfA()).withHeader("Content-Type",
"application/pdf")));
        mvc.perform(get("/api/v1/portail/certifications/{id}/attestation",
att.getDossier().getId())
                .with(correspondant(Fixtures.ETABLISSEMENT_A)))
            .andExpect(status().isOk()).andExpect(header().string("X-Content-SHA256",
att.getEmpreinteDocument()));
        msDocument.verify(getRequestedFor(urlEqualTo("/api/v1/documents/" +
att.getDocumentRef() + "/contenu"))
            .withHeader("X-Consultation-Motif", equalTo("PORTAIL_ATTESTATION"))
            .withHeader("X-On-Behalf-Of", matching("[0-9a-f-]{36}")));
    }

    @Test @DisplayName("CER-13 : cloisonnement par institution, 404 et tentative
journalisée")
    void isolation() throws Exception {
        var att = Fixtures.attestationEmise(Fixtures.ETABLISSEMENT_A);
        mvc.perform(get("/api/v1/portail/certifications/{id}/attestation",
att.getDossier().getId())
                .with(correspondant(Fixtures.BANQUE_B)))
            .andExpect(status().isNotFound()).andExpect(jsonPath("$.code").value("CER
_NOT_FOUND"));
```

```
        msDocument.verify(0, anyRequestedFor(anyUrl()));
        assertThat(outbox.findByEventType("certification.portal-access-
denied.v1")).hasSize(1);
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: CER-13 Certification consultée depuis le portail

  Scénario: Nominal : téléchargement journalisé
    Étant donné une attestation émise pour l'Établissement de paiement A
    Quand le correspondant de l'Établissement de paiement A la télécharge depuis le
portail
    Alors le document lui est restitué avec son empreinte dans l'en-tête "X-Content-
SHA256"
    Et le téléchargement est inscrit au journal de consultation

  Scénario: Alternatif : suivi du dossier en cours
    Étant donné un dossier de l'Établissement de paiement A au stade du circuit de
quitus avec un quitus sécurité défavorable
    Quand le correspondant le consulte
    Alors il voit l'état du dossier et le motif communicable du refus, sans les
annotations internes

  Scénario: Erreur : attestation d'une autre institution
    Étant donné un correspondant de la Banque B
    Quand il demande l'attestation de l'Établissement de paiement A
    Alors la ressource lui est présentée comme introuvable (404)
    Et la tentative est journalisée par l'événement "certification.portal-access-
denied.v1"
```

E. Tests cURL via la passerelle

La route portail de la passerelle réécrit `/bff/portail/certification/**` en `/api/v1/portail/**`.

```
source ./scripts/token.sh test-correspondant-a
# Nominal : liste des dossiers de l'institution, puis téléchargement avec contrôle
d'empreinte
curl -s "${GATEWAY_URL}/bff/portail/certification/certifications" -H "Authorization:
Bearer ${TOKEN}" \
  | jq '.[] | {id, etatDossier, attestation: .attestation.numero, quitus: [.quitus[]
| {nature, etat, motifCommunicable}]}'
curl -s -D /tmp/entetes.txt -o /tmp/attestation.pdf -H "Authorization: Bearer $
{TOKEN}" \
  "${GATEWAY_URL}/bff/portail/certification/certifications/${DOSSIER_A_ID}/
attestation"
grep -i "x-content-sha256" /tmp/entetes.txt; sha256sum /tmp/attestation.pdf     # les
deux empreintes coïncident

# Refus : la Banque B demande l'attestation de l'Établissement de paiement A (attendu
404)
source ./scripts/token.sh test-correspondant-b
curl -s -o /dev/null -w "%{http_code}\n" -H "Authorization: Bearer ${TOKEN}" \
  "${GATEWAY_URL}/bff/portail/certification/certifications/${DOSSIER_A_ID}/
attestation"
```

CER-14 | Tenir le registre des certifications délivrées et suivre leur renouvellement

Élément	Valeur
Points	5
Priorité	Should Have
Sprint cible	Sprint 5
Écrans	DES-11 (onglet registre)
Dépendances	CER-12, NOT-14

Objectif

Chaque `certification.attestation-issued.v1` crée une `CertificationRegisterEntry` liée à la précédente du même couple institution et type de projet (qui passe à REMPLACEE) ; le registre est consultable par filtres, exportable en CSV sous journal, et une veille quotidienne idempotente émet `certification.renewal-due.v1` une seule fois par seuil.

A. Implémentation

A1. Services de domaine : lignée et seuils d'échéance

L'échéance de renouvellement vaut la date d'émission augmentée de `gimac.certification.renewal-period` (valeur initiale `P3Y`, paramétrable). Les seuils de veille sont `gimac.certification.renewal-thresholds=90,30,7` (jours) ; `dernierSeuilNotifie` (entier, en jours) mémorise le plus petit seuil déjà notifié.

```
// domain/service/RenewalLineage.java
public final class RenewalLineage {
    private RenewalLineage() {}
    /** La nouvelle entrée pointe vers la précédente, qui prend l'état REMPLACEE. */
    public static CertificationRegisterEntry
link(Optional<CertificationRegisterEntry> previous, CertificationRegisterEntry next)
{
        previous.ifPresent(p -> {
            if (p.getEtat() == EtatAttestation.REVOQUEE || p.getEtat() ==
EtatAttestation.REMPLACEE) return;
            p.setEtat(EtatAttestation.REMPLACEE);
            next.setPrecedenteId(p.getId());
        });
        return next;
    }
}

// domain/service/RenewalThresholdPolicy.java
public final class RenewalThresholdPolicy {
    private final List<Integer> seuils;
    public RenewalThresholdPolicy(List<Integer> seuils) { this.seuils =
List.copyOf(seuils); }

    /** Seuil atteint le plus serré, s'il n'a pas déjà été notifié. */
    public Optional<Integer> due(LocalDate echeance, LocalDate today, Integer
dernierSeuilNotifie) {
        long restants = ChronoUnit.DAYS.between(today, echeance);
        return seuils.stream().filter(s -> restants <=
s).min(Comparator.naturalOrder())
            .filter(s -> dernierSeuilNotifie == null || s < dernierSeuilNotifie);
    }
```

```
}
```

A2. Inscription au registre par consommation interne

```
// infrastructure/messaging/consumer/AttestationIssuedRegisterConsumer.java
@Component
@RequiredArgsConstructor
class AttestationIssuedRegisterConsumer {
    private final IdempotentConsumer idempotentConsumer;
    private final CertificationRegisterService register;

    @KafkaListener(topics = "certification.attestation", groupId = "mscertification-
registre")
    @Transactional
    void on(CloudEventEnvelope<AttestationIssuedPayload> e) {
        if ("certification.attestation-issued.v1".equals(e.type())) {
            idempotentConsumer.handleOnce(e.id(), "mscertification-registre", () ->
register.inscribe(e.data()));
        }
    }
}

// application/usecase/CertificationRegisterService.java
@Service
@RequiredArgsConstructor
public class CertificationRegisterService {
    private final CertificationRegisterEntryRepository entries;
    private final AttestationRepository attestations;
    private final RegisterProperties props;

    @Transactional(propagation = Propagation.MANDATORY)
    public void inscribe(AttestationIssuedPayload p) {
        var previous = entries.findCurrentForUpdate(p.institutionId(),
p.typeProjet());   // EN_VIGUEUR ou SUSPENDUE
        var next = new
CertificationRegisterEntry().numeroAttestation(p.numero()).institutionId(p.institutio
nId())
            .typeProjet(p.typeProjet()).services(Json.write(p.servicesCertifies()))
            .dateEmission(p.dateEmission()).etat(EtatAttestation.EN_VIGUEUR)
            .echeanceRenouvellement(LocalDate.ofInstant(p.dateEmission(),
ZoneOffset.UTC).plus(props.renewalPeriod()));
        RenewalLineage.link(previous, next);
        previous.ifPresent(prev ->
attestations.findByNumero(prev.getNumeroAttestation())
            .ifPresent(a -> a.setEtat(EtatAttestation.REMPLACEE)));
        entries.save(next);
    }
}
```

A3. Veille quotidienne idempotente

Chaque entrée est traitée dans sa transaction ; la mise à jour conditionnelle de `dernierSeuilNotifie` garantit qu'un seuil n'est notifié qu'une fois, même si la tâche est rejouée ou exécutée par deux répliques.

```
// infrastructure/scheduling/RenewalWatchJob.java
@Component
@RequiredArgsConstructor
class RenewalWatchJob {
    private final CertificationRegisterEntryRepository entries;
    private final RenewalThresholdPolicy policy;
    private final OutboxPublisher outboxPublisher;
    private final TransactionTemplate tx;
    private final Clock clock;

    @Scheduled(cron = "0 50 1 * * *", zone = "Africa/Douala")
```

```
    public void run() {
        LocalDate today = LocalDate.now(clock);
        entries.findEnVigueurWithEcheanceBefore(today.plusDays(90)).forEach(e ->
            policy.due(e.getEcheanceRenouvellement(), today,
e.getDernierSeuilNotifie()).ifPresent(seuil ->
                tx.executeWithoutResult(s -> {
                    // update ... set dernier_seuil_notifie = :seuil where id = :id
and (dernier_seuil_notifie is null or dernier_seuil_notifie > :seuil)
                    if (entries.markNotified(e.getId(), seuil) == 1) {
                        Map<String, Object> data = Map.of("numero",
e.getNumeroAttestation(), "institutionId", e.getInstitutionId(),
                            "typeProjet", e.getTypeProjet(), "echeance",
e.getEcheanceRenouvellement(), "seuilJours", seuil);
                        outboxPublisher.publish("Attestation",
e.getNumeroAttestation(), "certification.renewal-due.v1", data);
                        outboxPublisher.publish("Notification",
e.getNumeroAttestation(), "certification.notification-requested.v1",
                            Map.of("destinataires", List.of(Map.of("role",
"CHEF_DES"),
                                    Map.of("role", "CORRESPONDANT_INSTITUTION",
"institutionId", e.getInstitutionId())),
                                "modele", "ECHEANCE_RENOUVELLEMENT", "donnees",
data));
                    }
                })));
    }
}
```

A4. Consultation filtrée et export contrôlé

`GET /registre` (`CER_REGISTER_READ`) accepte `institutionId`, `typeProjet`, `etat`, `echeanceAvant`, `page`, `size` (spécifications Spring Data). `GET /registre/export` (`CER_REGISTER_EXPORT`) applique les mêmes filtres, refuse au-delà de 10 000 lignes (409, `CER_EXPORT_TOO_LARGE`) et inscrit l'export au registre d'audit avant la remise.

```
// web/rest/RegisterResource.java
@RestController
@RequestMapping("/api/v1/registre")
@RequiredArgsConstructor
class RegisterResource {
    private final CertificationRegisterQueryService query;
    private final OutboxPublisher outboxPublisher;
    private final CurrentUserProvider currentUser;

    @GetMapping
    @PreAuthorize("hasAuthority('CER_REGISTER_READ')")
    Page<RegisterEntryDTO> list(RegisterFilter filter, Pageable pageable) { return
query.search(filter, pageable); }

    @GetMapping(value = "/export", produces = "text/csv")
    @PreAuthorize("hasAuthority('CER_REGISTER_EXPORT')")
    @Transactional
    ResponseEntity<byte[]> export(RegisterFilter filter) {
        List<RegisterEntryDTO> rows = query.searchForExport(filter, 10_000);   //
lève CER_EXPORT_TOO_LARGE au-delà
        byte[] csv = RegisterCsv.write(rows);                                   //
séparateur « ; », UTF-8 avec BOM
        String empreinte = Sha256.hex(csv);
        outboxPublisher.publish("Register", "export", "certification.register-
exported.v1", Map.of(
            "demandeurId", currentUser.current().id(), "filtres", filter, "lignes",
rows.size(), "empreinte", empreinte));
        return ResponseEntity.ok().header("X-Content-SHA256", empreinte)
            .header(HttpHeaders.CONTENT_DISPOSITION, "attachment; filename=registre-
certifications.csv").body(csv);
    }
}
```

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
class RenewalLineageTest {
    @Test @DisplayName("CER-14 : un renouvellement lie la nouvelle entrée à la
précédente, qui devient REMPLACEE")
    void lineage() {
        var prev = new
CertificationRegisterEntry().id(UUID.randomUUID()).etat(EtatAttestation.EN_VIGUEUR);
        var next = RenewalLineage.link(Optional.of(prev), new
CertificationRegisterEntry().etat(EtatAttestation.EN_VIGUEUR));
        assertThat(prev.getEtat()).isEqualTo(EtatAttestation.REMPLACEE);
        assertThat(next.getPrecedenteId()).isEqualTo(prev.getId());
    }

    @Test @DisplayName("CER-14 : une première attestation n'a pas de précédente")
    void first() {
        assertThat(RenewalLineage.link(Optional.empty(), new
CertificationRegisterEntry()).getPrecedenteId()).isNull();
    }
}

class RenewalThresholdPolicyTest {
    RenewalThresholdPolicy policy = new RenewalThresholdPolicy(List.of(90, 30, 7));
    LocalDate today = LocalDate.of(2026, 10, 1);

    @Test @DisplayName("CER-14 : le seuil de quatre-vingt-dix jours est dû une seule
fois")
    void ninetyOnce() {
        assertThat(policy.due(today.plusDays(90), today, null)).contains(90);
        assertThat(policy.due(today.plusDays(89), today.plusDays(1), 90)).isEmpty();
        assertThat(policy.due(today.plusDays(30), today, 90)).contains(30);
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

Aucun tiers HTTP n'intervient : les notifications partent par l'outbox vers MS-NOTIFICATION (NOT-14), d'où l'absence de WireMock.

```
@SpringBootTest @AutoConfigureMockMvc @Testcontainers @ActiveProfiles("test")
class RegisterRenewalIT {
    @Container @ServiceConnection static PostgreSQLContainer<?> pg = new
PostgreSQLContainer<>("postgres:16-alpine");
    @Container @ServiceConnection static KafkaContainer kafka = new
KafkaContainer("apache/kafka:3.8.0");
    @Autowired RenewalWatchJob job; @Autowired OutboxEventRepository outbox;
@Autowired KafkaTemplate<String, Object> kafkaTemplate;
    @Autowired CertificationRegisterEntryRepository entries; @MockitoBean Clock
clock;

    @Test @DisplayName("CER-14 : l'échéance est émise une seule fois par seuil, même
après deux passages")
    void singleEmissionPerThreshold() {
        when(clock.instant()).thenReturn(Instant.parse("2026-10-01T00:50:00Z"));
        when(clock.getZone()).thenReturn(ZoneOffset.UTC);
        Fixtures.registreEnVigueur(Fixtures.ETABLISSEMENT_A, LocalDate.of(2026, 12,
30));   // J-90
        job.run();
        job.run();
        assertThat(outbox.findByEventType("certification.renewal-
due.v1")).hasSize(1);
        assertThat(outbox.findByEventType("certification.notification-
requested.v1")).hasSize(1);
    }
```

```
    @Test @DisplayName("CER-14 : l'émission d'une attestation crée l'entrée et
remplace la précédente")
    void inscriptionAndRenewal() {
        var old = Fixtures.registreEnVigueur(Fixtures.ETABLISSEMENT_A,
LocalDate.of(2027, 1, 15));
        kafkaTemplate.send("certification.attestation",
Fixtures.attestationIssuedEvent(Fixtures.ETABLISSEMENT_A, "CARTE_INTERBANCAIRE"));
        await().atMost(Duration.ofSeconds(10)).untilAsserted(() -> {

assertThat(entries.findById(old.getId()).orElseThrow().getEtat()).isEqualTo(EtatAttes
tation.REMPLACEE);
            assertThat(entries.findByPrecedenteId(old.getId())).isPresent();
        });
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: CER-14 Registre des certifications et renouvellement

  Scénario: Nominal : inscription au registre
    Étant donné une attestation émise pour l'Établissement de paiement A
    Quand l'événement "certification.attestation-issued.v1" est traité
    Alors une entrée est créée au registre avec le numéro, l'institution, le type de
projet, les services, la date et l'échéance de renouvellement

  Scénario: Alternatif : renouvellement
    Étant donné une certification en vigueur pour la Banque B et le type de projet
"CARTE_INTERBANCAIRE"
    Quand une nouvelle attestation est émise pour le même couple
    Alors la nouvelle entrée est liée à la précédente, qui prend l'état "REMPLACEE"

  Scénario: Alternatif : échéance proche
    Étant donné une certification dont l'échéance de renouvellement tombe dans
quatre-vingt-dix jours
    Quand le passage quotidien de l'ordonnanceur temporel s'exécute deux fois
    Alors l'événement "certification.renewal-due.v1" est émis une seule fois pour le
seuil de 90 jours
    Et le chef du DES ainsi que le correspondant sont notifiés
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-chef-des
# Nominal : registre filtré et paginé
curl -s "${GATEWAY_URL}/bff/des/certification/registre?
typeProjet=CARTE_INTERBANCAIRE&etat=EN_VIGUEUR&echeanceAvant=2027-06-
30&page=0&size=20" \
  -H "Authorization: Bearer ${TOKEN}" | jq '.content[] | {numeroAttestation,
institutionId, echeanceRenouvellement, etat, precedenteId}'

# Nominal : export CSV journalisé, empreinte vérifiée
curl -s -D /tmp/entetes.txt -o /tmp/registre.csv -H "Authorization: Bearer $
{TOKEN}" \
  "${GATEWAY_URL}/bff/des/certification/registre/export?etat=EN_VIGUEUR"
grep -i "x-content-sha256" /tmp/entetes.txt; sha256sum /tmp/registre.csv

# Refus : un correspondant n'accède pas au registre (attendu 403)
source ./scripts/token.sh test-correspondant-a
curl -s -o /dev/null -w "%{http_code}\n" -H "Authorization: Bearer ${TOKEN}" "$
{GATEWAY_URL}/bff/des/certification/registre"
```

CER-15 | Révoquer ou suspendre une attestation de certification

Élément	Valeur
Points	5
Priorité	Should Have
Sprint cible	Sprint 6
Écrans	DES-11 (onglet registre)
Dépendances	CER-12, CER-14, IAM-07, PAR-02

Objectif

Une demande motivée du chef du DES, accompagnée d'une pièce justificative, est approuvée par la Direction Générale selon une procédure à quatre yeux avec réauthentification FIDO2 ; un avis PDF/A-3 signé est déposé en WORM, l'attestation d'origine reste inchangée, le registre et le portail reflètent le nouvel état, et l'événement de révocation, de suspension ou de levée est émis.

A. Implémentation

A1. Compléments au modèle, transitions et garde à quatre yeux

Compléments au JDL, ajoutés par changelog Liquibase : énumération `EtatDemandeStatut {EN_ATTENTE, APPROUVEE}` pour `AttestationStatusDecision.etat`, champ `motifApprobation` (TextBlob) exigé à l'approbation. Les transitions et la garde sont des services de domaine purs.

```
// domain/service/AttestationStatusTransitions.java
// import static org.gimac.des.certification.domain.enumeration.EtatAttestation.*;
public final class AttestationStatusTransitions {
    private AttestationStatusTransitions() {}
    public static EtatAttestation target(EtatAttestation courant, TypeDemandeStatut
type) {
        return switch (type) {
            case REVOCATION -> require(courant == EN_VIGUEUR || courant == SUSPENDUE,
REVOQUEE, courant, type);
            case SUSPENSION -> require(courant == EN_VIGUEUR, SUSPENDUE, courant,
type);
            case LEVEE -> require(courant == SUSPENDUE, EN_VIGUEUR, courant, type);
        };
    }
    private static EtatAttestation require(boolean ok, EtatAttestation cible,
EtatAttestation courant, TypeDemandeStatut type) {
        if (!ok) throw new StatusTransitionRefused(courant, type);            // 409
CER_STATUS_TRANSITION_REFUSED
        return cible;
    }
}

// domain/service/FourEyesGuard.java
public final class FourEyesGuard {
    private FourEyesGuard() {}
    public static void check(AttestationStatusDecision d, UUID approbateurId, String
motifApprobation) {
        if (d.getEtat() != EtatDemandeStatut.EN_ATTENTE) throw new
FourEyesViolation("CER_STATUS_REQUEST_CLOSED");
        if (d.getDemandeurId().equals(approbateurId)) throw new
FourEyesViolation("CER_FOUR_EYES_SAME_PERSON");
        if (motifApprobation == null || motifApprobation.isBlank()) throw new
```

```
FourEyesViolation("CER_APPROVAL_REASON_MISSING");
        if (d.getPieceRef() == null || d.getPieceRef().isBlank()) throw new
FourEyesViolation("CER_EVIDENCE_MISSING");
    }
}
```

A2. Demande et approbation

La demande exige un motif et une référence de pièce déposée au préalable dans MS-DOCUMENT (400 sinon), une condition de levée pour une suspension, et refuse une seconde demande en attente sur la même attestation (409, `CER_STATUS_REQUEST_PENDING`). L'approbation vérifie l'assertion FIDO2 par `ReauthenticationPort` (403, `CER_REAUTH_FAILED`), applique la garde, produit l'avis avec la fabrique PDF/A-3 de CER-12 (gabarit bilingue `avis-statut`), le dépose en WORM et ne modifie de l'attestation que son champ `etat` : `documentRef` et `empreinteDocument` restent ceux de l'original.

```
// application/usecase/AttestationStatusService.java
@Service
@RequiredArgsConstructor
public class AttestationStatusService implements RequestStatusChangeUseCase,
ApproveStatusChangeUseCase {
    private static final Map<TypeDemandeStatut, String> EVENTS = Map.of(
        TypeDemandeStatut.REVOCATION, "certification.attestation-revoked.v1",
        TypeDemandeStatut.SUSPENSION, "certification.attestation-suspended.v1",
        TypeDemandeStatut.LEVEE, "certification.attestation-reinstated.v1");

    private final AttestationRepository attestations;
    private final AttestationStatusDecisionRepository decisions;
    private final CertificationRegisterEntryRepository register;
    private final ReauthenticationPort reauth;
    private final SignedPdfA3Factory pdfFactory;
    private final DocumentDepositPort deposit;
    private final OutboxPublisher outboxPublisher;
    private final Clock clock;

    @Transactional
    public AttestationStatusDecision request(String numero, StatusRequestCommand c,
UUID demandeurId) {
        Attestation a = attestations.findByNumeroForUpdate(numero).orElseThrow(() ->
NotFound.attestation(numero));
        AttestationStatusTransitions.target(a.getEtat(), c.typeDemande());
// contrôle anticipé
        if (decisions.existsByAttestationIdAndEtat(a.getId(),
EtatDemandeStatut.EN_ATTENTE)) {
            throw new DomainProblemException(CONFLICT, "CER_STATUS_REQUEST_PENDING",
"Une demande est déjà en attente");
        }
        return decisions.save(new
AttestationStatusDecision().attestation(a).typeDemande(c.typeDemande())
            .motif(c.motif()).pieceRef(c.pieceRef()).conditionLevee(c.conditionLevee(
))
            .demandeurId(demandeurId).dateDemande(clock.instant()).etat(EtatDemandeSt
atut.EN_ATTENTE));
    }

    @Transactional
    public AttestationStatusDecision approve(UUID demandeId, UUID approbateurId,
String assertion, String motifApprobation) {
        if (!reauth.verify(assertion, approbateurId,
"CER_ATTESTATION_STATUS_APPROVE")) {
            throw new DomainProblemException(FORBIDDEN, "CER_REAUTH_FAILED",
"Réauthentification FIDO2 requise");
        }
        AttestationStatusDecision d =
decisions.findByIdForUpdate(demandeId).orElseThrow(() ->
```

```
NotFound.demande(demandeId));
        FourEyesGuard.check(d, approbateurId, motifApprobation);
        Attestation a = d.getAttestation();
        EtatAttestation cible = AttestationStatusTransitions.target(a.getEtat(),
d.getTypeDemande());

        byte[] avis = pdfFactory.produce("avis-statut", Map.of("numero",
a.getNumero(), "type", d.getTypeDemande(),
            "motif", d.getMotif(), "conditionLevee",
Objects.toString(d.getConditionLevee(), ""),
            "dateEffet", clock.instant(), "empreinteAttestation",
a.getEmpreinteDocument()), List.of());
        var doc = deposit.deposit(new
DocumentDepositPort.DepositRequest("AVIS_STATUT_ATTESTATION", a.getInstitutionId(),
            "avis-" + a.getNumero() + "-" + d.getId() + ".pdf", "application/pdf",
avis, "avis-" + d.getId()));

        d.setApprobateurId(approbateurId); d.setMotifApprobation(motifApprobation);
        d.setDateApprobation(clock.instant());
d.setEtat(EtatDemandeStatut.APPROUVEE); d.setAvisDocumentRef(doc.documentRef());
        a.setEtat(cible);                                                    //
l'original PDF n'est jamais réécrit
        register.findByNumeroAttestation(a.getNumero()).ifPresent(e ->
e.setEtat(cible));
        outboxPublisher.publish("Attestation", a.getNumero(),
EVENTS.get(d.getTypeDemande()), Map.of(
            "numero", a.getNumero(), "institutionId", a.getInstitutionId(),
"typeProjet", a.getTypeProjet(),
            "nouvelEtat", cible, "motif", d.getMotif(), "avisDocumentRef",
doc.documentRef(),
            "conditionLevee", Objects.toString(d.getConditionLevee(), "")));
        return d;
    }
}
```

A3. Ressources REST

```
// web/rest/AttestationStatusResource.java
@RestController
@RequestMapping("/api/v1")
@RequiredArgsConstructor
class AttestationStatusResource {
    private final RequestStatusChangeUseCase requestUseCase;
    private final ApproveStatusChangeUseCase approveUseCase;
    private final CurrentUserProvider currentUser;
    private final StatusDecisionMapper mapper;

    @PostMapping("/attestations/{numero}/demandes-de-statut")
    @PreAuthorize("hasAuthority('CER_ATTESTATION_STATUS_REQUEST')")
    ResponseEntity<StatusDecisionDTO> request(@PathVariable String numero, @Valid
@RequestBody StatusRequestCommand c) {
        var d = requestUseCase.request(numero, c, currentUser.current().id());   //
@NotBlank motif, pieceRef
        return ResponseEntity.status(HttpStatus.CREATED).body(mapper.toDto(d));
    }

    @PostMapping("/demandes-de-statut/{id}/approbation")
    @PreAuthorize("hasAuthority('CER_ATTESTATION_STATUS_APPROVE')")
    StatusDecisionDTO approve(@PathVariable UUID id, @RequestHeader("X-Reauth-
Assertion") String assertion,
                              @RequestBody ApprovalCommand body) {
        return mapper.toDto(approveUseCase.approve(id, currentUser.current().id(),
assertion, body.motifApprobation()));
    }
}
```

L'absence de l'en-tête `X-Reauth-Assertion` est traduite en 403 `CER_REAUTH_FAILED` par le gestionnaire d'erreurs. La mention de révocation ou de suspension apparaît sur le portail par la

projection de CER-13, qui lit `Attestation.etat` ; MS-PARTICIPANT et MS-NOTIFICATION consomment les trois événements sur `certification.attestation`.

B. Tests unitaires (JUnit 5 + Mockito + AssertJ)

```
class FourEyesGuardTest {
    UUID chefDes = UUID.randomUUID();
    AttestationStatusDecision demande = new
AttestationStatusDecision().typeDemande(TypeDemandeStatut.SUSPENSION)
        .demandeurId(chefDes).motif("Anomalie de sécurité constatée").pieceRef("DOC-
7781").etat(EtatDemandeStatut.EN_ATTENTE);

    @Test @DisplayName("CER-15 : l'approbation est refusée lorsque demandeur et
approbateur sont identiques")
    void samePerson() {
        assertThatThrownBy(() -> FourEyesGuard.check(demande, chefDes, "Confirmé"))
            .hasFieldOrPropertyWithValue("code", "CER_FOUR_EYES_SAME_PERSON");
    }

    @Test @DisplayName("CER-15 : l'approbation sans motif est refusée")
    void missingReason() {
        assertThatThrownBy(() -> FourEyesGuard.check(demande, UUID.randomUUID(), "
"))
            .hasFieldOrPropertyWithValue("code", "CER_APPROVAL_REASON_MISSING");
    }
}

class AttestationStatusTransitionsTest {
    @Test @DisplayName("CER-15 : suspension puis levée ramènent l'attestation à
EN_VIGUEUR ; une levée sans suspension est refusée")
    void suspendThenReinstate() {
        var suspendue =
AttestationStatusTransitions.target(EtatAttestation.EN_VIGUEUR,
TypeDemandeStatut.SUSPENSION);
        assertThat(AttestationStatusTransitions.target(suspendue,
TypeDemandeStatut.LEVEE)).isEqualTo(EtatAttestation.EN_VIGUEUR);
        assertThatThrownBy(() ->
AttestationStatusTransitions.target(EtatAttestation.EN_VIGUEUR,
TypeDemandeStatut.LEVEE))
            .isInstanceOf(StatusTransitionRefused.class);
    }
}
```

C. Tests d'intégration (@SpringBootTest + Testcontainers)

```
@SpringBootTest @AutoConfigureMockMvc @Testcontainers @ActiveProfiles("test")
class AttestationRevocationIT {
    @Container @ServiceConnection static PostgreSQLContainer<?> pg = new
PostgreSQLContainer<>("postgres:16-alpine");
    @Container @ServiceConnection static KafkaContainer kafka = new
KafkaContainer("apache/kafka:3.8.0");
    @RegisterExtension static WireMockExtension tiers =
WireMockExtension.newInstance().options(wireMockConfig().dynamicPort()).build();
    @DynamicPropertySource static void props(DynamicPropertyRegistry r) {
        r.add("gimac.adapters.document-deposit", () -> "real");
        r.add("gimac.adapters.reauthentication", () -> "real");
        r.add("gimac.clients.msdocument.base-url", tiers::baseUrl);
        r.add("gimac.clients.msiam.base-url", tiers::baseUrl);
    }
    @Autowired MockMvc mvc; @Autowired AttestationRepository attestations;
    @Autowired CertificationRegisterEntryRepository register; @Autowired
OutboxEventRepository outbox;

    @Test @DisplayName("CER-15 : l'attestation d'origine reste inchangée après
révocation")
    void originalUnchanged() throws Exception {
```

```
        Attestation a = Fixtures.attestationEmiseAuRegistre(Fixtures.BANQUE_B);
        String ref = a.getDocumentRef(), empreinte = a.getEmpreinteDocument();

tiers.stubFor(post("/api/v1/reauthentication/verify").willReturn(okJson("{\"valid\":t
rue}")));

tiers.stubFor(post("/api/v1/interne/documents").willReturn(okJson("{\"documentRef\":\
"DOC-AVIS-1\",\"empreinte\":\"" + "c".repeat(64) + "\"}")));

        String id =
JsonPath.read(mvc.perform(post("/api/v1/attestations/{n}/demandes-de-statut",
a.getNumero())
                .with(user(Fixtures.CHEF_DES,
"CER_ATTESTATION_STATUS_REQUEST")).contentType(APPLICATION_JSON)
                .content("{\"typeDemande\":\"REVOCATION\",\"motif\":\"Conditions de
certification non réunies\",\"pieceRef\":\"DOC-7781\"}"))
            .andExpect(status().isCreated()).andReturn().getResponse().getContentAsSt
ring(), "$.id");
        mvc.perform(post("/api/v1/demandes-de-statut/{id}/approbation", id)
                .with(user(Fixtures.DG, "CER_ATTESTATION_STATUS_APPROVE")).header("X-
Reauth-Assertion", "assertion-de-test")
                .contentType(APPLICATION_JSON).content("{\"motifApprobation\":\"Révoc
ation confirmée\"}"))
            .andExpect(status().isOk()).andExpect(jsonPath("$.avisDocumentRef").value
("DOC-AVIS-1"));

        Attestation after = attestations.findByNumero(a.getNumero()).orElseThrow();
        assertThat(after.getEtat()).isEqualTo(EtatAttestation.REVOQUEE);
        assertThat(after.getDocumentRef()).isEqualTo(ref);
        assertThat(after.getEmpreinteDocument()).isEqualTo(empreinte);

assertThat(register.findByNumeroAttestation(a.getNumero()).orElseThrow().getEtat()).i
sEqualTo(EtatAttestation.REVOQUEE);
        assertThat(outbox.findByEventType("certification.attestation-
revoked.v1")).hasSize(1);
        tiers.verify(1,
postRequestedFor(urlEqualTo("/api/v1/interne/documents")).withRequestBody(containing(
"AVIS_STATUT_ATTESTATION")));
    }
}
```

D. Tests Gherkin (Cucumber)

```
# language: fr
Fonctionnalité: CER-15 Révocation et suspension d'une attestation

  Scénario: Nominal : révocation
    Étant donné une demande de révocation motivée du chef du DES accompagnée d'une
pièce justificative pour l'attestation de la Banque B
    Quand la Direction Générale l'approuve après réauthentification FIDO2
    Alors un avis de révocation au format PDF/A-3 signé est déposé en WORM,
l'attestation d'origine restant inchangée
    Et l'entrée du registre prend l'état "REVOQUEE"
    Et l'événement "certification.attestation-revoked.v1" est émis, consommé par MS-
PARTICIPANT et MS-NOTIFICATION
    Et le portail de la Banque B affiche la mention de révocation

  Scénario: Alternatif : suspension puis levée
    Étant donné une attestation suspendue avec la condition de levée "Correctif
déployé et vérifié"
    Quand la levée est décidée par la même procédure à quatre yeux
    Alors l'attestation redevient "EN_VIGUEUR" et l'événement
"certification.attestation-reinstated.v1" est émis

  Scénario: Erreur : demandeur et approbateur confondus
    Étant donné une demande de suspension
    Quand son auteur tente de l'approuver, ou l'approbation est donnée sans motif
```

```
    Alors l'approbation est refusée avec le code "CER_FOUR_EYES_SAME_PERSON" ou
"CER_APPROVAL_REASON_MISSING"
```

E. Tests cURL via la passerelle

```
source ./scripts/token.sh test-chef-des
# Nominal : demande de révocation motivée avec pièce justificative
DEMANDE_ID=$(curl -s -X POST "${GATEWAY_URL}/bff/des/certification/attestations/$
{NUMERO}/demandes-de-statut" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" \
  -d '{"typeDemande":"REVOCATION","motif":"Conditions de certification non
réunies","pieceRef":"'"${PIECE_REF}"'"}' | jq -r .id)

# Refus : le demandeur n'a pas la permission d'approuver (attendu 403)
curl -s -o /dev/null -w "%{http_code}\n" -X POST -H "Authorization: Bearer ${TOKEN}"
-H "Content-Type: application/json" \
  -H "X-Reauth-Assertion: ${REAUTH_ASSERTION}" -d '{"motifApprobation":"Confirmé"}' \
  "${GATEWAY_URL}/bff/des/certification/demandes-de-statut/${DEMANDE_ID}/approbation"

source ./scripts/token.sh test-dg
# Refus : approbation sans réauthentification (attendu 403)
curl -s -o /dev/null -w "%{http_code}\n" -X POST -H "Authorization: Bearer ${TOKEN}"
-H "Content-Type: application/json" \
  -d '{"motifApprobation":"Révocation confirmée"}'
"${GATEWAY_URL}/bff/des/certification/demandes-de-statut/${DEMANDE_ID}/approbation"
# Nominal : approbation avec assertion FIDO2 (adaptateur de substitution en recette)
curl -s -X POST "${GATEWAY_URL}/bff/des/certification/demandes-de-statut/$
{DEMANDE_ID}/approbation" \
  -H "Authorization: Bearer ${TOKEN}" -H "Content-Type: application/json" -H "X-
Reauth-Assertion: ${REAUTH_ASSERTION}" \
  -d '{"motifApprobation":"Révocation confirmée"}' | jq '{etat, typeDemande,
avisDocumentRef, dateApprobation}'
curl -s "${GATEWAY_URL}/bff/des/certification/attestations/${NUMERO}" -H
"Authorization: Bearer ${TOKEN}" | jq '{numero, etat, empreinteDocument}'
```

INTERFACE — Socle d'interface du lot et écrans DES-08 à DES-12 et POR-03

La partie frontend suit le système de conception DS-GIMAC-01 : chaque agrégat est décrit une seule fois par un descripteur de ressource, les pages se réduisent à l'appel des composants universels, et les composants de domaine sont les seuls à connaître le vocabulaire métier. Le fragment F0 livre le socle commun du lot ; chaque écran est ensuite traité avec ses cinq sections.

F0 | Socle d'interface du lot 1 : descripteurs, composants de domaine et hooks

Élément	Valeur
Points	transverse
Priorité	Must Have
Sprint cible	Sprint 3
Gabarit	Enveloppe `AppShell` (DS-GIMAC-01 §6) ; socle des gabarits G2 à G6
Récits couverts	Prérequis des écrans DES-08 à DES-12 et POR-03 (HOM-01 à HOM-24, CER-01 à CER-15)

Objectif

Poser une seule fois ce que les six écrans du lot 1 consomment : arborescence, client HTTP vers la passerelle, canal poussé, réauthentification FIDO2 portée par `SignDialog`, contrats des composants de domaine, dictionnaires d'états et clés i18n. Le socle est vérifié lorsqu'un écran vide monté dans l'enveloppe passe axe sans violation et que la passerelle de recette confirme une session sans jeton accessible au client.

A. Implémentation

A1. Arborescence du lot 1

Les descripteurs sont des fichiers `*.ts` (`*.tsx` lorsqu'ils déclarent des rendus d'onglets). Les composants de domaine entrent au catalogue (`design-system/domain`), jamais dans `features/`, qui ne contient que des pages réduites à des compositions et des hooks de données.

```
src/lib/http/            client.ts, prefixes.ts, errors.ts
src/lib/live/            live.ts, useLiveTopic.ts
src/lib/reauth/          webauthn.ts
src/lib/resource/        types.ts (ResourceDescriptor et extensions du lot 1),
hooks.ts
src/design-system/domain/  LiaisonProgress, OperationNode, DagView, CoverageBar,
                           FourElementsChecklist, QuitusCircuit, SignDialog
src/design-system/data/    TreeView, DiffView, StateMachineBar
src/design-system/display/ ProgressRing, Hash
src/resources/homologation/  types.ts, dictionaries.ts, options.ts,
referentielOperations.tsx,
                             liaison.tsx, liaison.tabs.tsx, operation.tsx,
cloture.tsx,
                             campagne.tsx, scenarioInconnu.ts, portailLiaison.tsx
src/resources/certification/ types.ts, dictionaries.ts, dossierMiseEnService.tsx,
                             attestation.ts, registre.ts
src/features/referentiels/   ReferentielsOperationsPage.tsx, hooks.ts
src/features/liaisons/       LiaisonsPage.tsx, LiaisonPage.tsx,
ClotureOperationPage.tsx, hooks.ts
src/features/certification/  DossierMiseEnServicePage.tsx, hooks.ts
src/features/portail/        MaLiaisonPage.tsx, hooks.ts
src/test/                    render.tsx, factories/homologation.ts,
factories/certification.ts
e2e/                         features/*.feature, steps/*.ts, specs/*.spec.ts,
support/webauthn.ts
```

A2. Client HTTP : préfixes, erreur normalisée, 401 et 403

Le client est le seul point d'appel réseau. La passerelle retire `/bff/<couche>/<service>` et ajoute `/api/v1` (et, pour la couche portail, `/api/v1/portail`) : une route du portail s'écrit donc `${P.portailHomologation}/liaisons/{id}`. Le témoin de session `HttpOnly` part seul (`credentials: "same-origin"`) ; le client ne lit ni ne pose aucun jeton.

```
// src/lib/http/client.ts
export const P = {
  desHomologation: "/bff/des/homologation",
  desCertification: "/bff/des/certification",
  portailHomologation: "/bff/portail/homologation",
  portailCertification: "/bff/portail/certification",
} as const;

export class ApiError extends Error {
  constructor(readonly status: number, readonly code: string, message: string,
              readonly correlationId: string, readonly details?:
Readonly<Record<string, unknown>>) {
    super(message); this.name = "ApiError";
  }
}
type Problem = { code?: string; message?: string; detail?: string; details?:
Record<string, unknown>; correlationId?: string };
type Hooks = { onUnauthorized: () => Promise<void>; onForbidden: (e: ApiError) =>
void; onCorrelation: (id: string) => void };
export type RequestOptions = {
  query?: Record<string, string | number | boolean | undefined>;
  body?: unknown; reauthAssertion?: string; accept?: string; signal?: AbortSignal;
};
let hooks: Hooks = { onUnauthorized: async () => undefined, onForbidden: () =>
undefined, onCorrelation: () => undefined };
export const configureHttp = (h: Hooks): void => { hooks = h; };

async function send(method: string, path: string, o: RequestOptions, retried =
false): Promise<Response> {
  const url = new URL(path, window.location.origin);
  Object.entries(o.query ?? {}).forEach(([k, v]) => { if (v !== undefined)
url.searchParams.set(k, String(v)); });
  const headers = new Headers({ Accept: o.accept ?? "application/json" });
  if (o.body !== undefined && !(o.body instanceof FormData)) headers.set("Content-
Type", "application/json");
  if (o.reauthAssertion) headers.set("X-Reauth-Assertion", o.reauthAssertion); //
assertion FIDO2 (IAM-07)
  const res = await fetch(url, {
    method, headers, credentials: "same-origin", signal: o.signal,
    body: o.body === undefined ? undefined : o.body instanceof FormData ? o.body :
JSON.stringify(o.body),
  });
  const correlationId = res.headers.get("X-Correlation-Id") ?? "";
  hooks.onCorrelation(correlationId); // pied de page de l'enveloppe (DS-GIMAC-01
§6.3)
  if (res.status === 401 && !retried) { await hooks.onUnauthorized(); return
send(method, path, o, true); }
  if (res.ok) return res;
  const p: Problem = await res.json().catch(() => ({}));
  const err = new ApiError(res.status, p.code ?? `HTTP_${res.status}`, p.message ??
p.detail ?? "",
                           p.correlationId ?? correlationId, p.details);
  if (res.status === 403) hooks.onForbidden(err); // page dédiée TRA-03
  throw err;
}
const json = async <R>(r: Response): Promise<R> => (r.status === 204 ? (undefined as
R) : ((await r.json()) as R));
export const http = {
  get: <R>(path: string, o: RequestOptions = {}) => send("GET", path,
o).then(json<R>),
  post: <R>(path: string, o: RequestOptions = {}) => send("POST", path,
```

```
o).then(json<R>),
  put: <R>(path: string, o: RequestOptions = {}) => send("PUT", path,
o).then(json<R>),
  /** Téléchargement journalisé côté service ; l'empreinte vient de l'en-tête Repr-
Digest (RFC 9530). */
  blob: async (path: string, accept: string) => {
    const r = await send("GET", path, { accept });
    return { blob: await r.blob(), empreinte: reprDigestToHex(r.headers.get("Repr-
Digest")) };
  },
};
function reprDigestToHex(h: string | null): string | null {
  const b64 = h?.match(/sha-256=:([^:]+):/)?.[1];
  return b64 ? Array.from(atob(b64), c => c.charCodeAt(0).toString(16).padStart(2,
"0")).join("") : null;
}
```

A3. Canal poussé et réauthentification FIDO2

`lib/live` ouvre un flux d'événements par couche, s'abonne aux sujets `homologation` et `certification`, reprend depuis le dernier identifiant reçu et invalide les requêtes dont la clé commence par le sujet. Le filtrage par institution est fait par la passerelle pour le portail. `lib/reauth` obtient un défi auprès de MS-IAM, appelle `navigator.credentials.get` et sérialise l'assertion transmise ensuite dans `X-Reauth-Assertion`.

```
// src/lib/live/live.ts
import type { QueryClient } from "@tanstack/react-query";
export type LiveTopic = "homologation" | "certification";
export type LiveEvent = { id: string; type: string; subject: string; topic: LiveTopic
};
export function openLiveChannel(layer: "des" | "portail", topics: LiveTopic[], qc:
QueryClient,
                                onState: (s: "connected" | "lost") => void,
                                backoff: { baseMs: number; maxMs: number }): () =>
void { // valeurs lues de la configuration
  let source: EventSource | null = null; let attempt = 0; let lastId = ""; let closed
= false;
  const connect = () => {
    const q = new URLSearchParams({ sujets: topics.join(","), depuis: lastId });
    source = new EventSource(`/bff/${layer}/live?${q.toString()}`, { withCredentials:
true });
    source.onopen = () => { attempt = 0; onState("connected"); };
    source.onmessage = m => {
      const ev = JSON.parse(m.data) as LiveEvent; lastId = ev.id;
      void qc.invalidateQueries({ queryKey: [ev.topic] }); // clés de requête :
[sujet, ressource, id]
    };
    source.onerror = () => {
      source?.close(); onState("lost"); if (closed) return;
      window.setTimeout(connect, Math.min(backoff.maxMs, 2 ** attempt++ *
backoff.baseMs)); // plafond : 30 s
    };
  };
  connect();
  return () => { closed = true; source?.close(); };
}
```

```
// src/lib/reauth/webauthn.ts
import { http } from "../http/client";
export type SignPurpose = "OPERATION_VALIDATE" | "QUITUS_SIGN" | "STATUS_APPROVE";
type Challenge = { challengeId: string; challenge: string; rpId: string;
allowCredentials: string[]; timeoutMs: number };
const b64u = (b: ArrayBuffer) => btoa(String.fromCharCode(...new
Uint8Array(b))).replace(/\+/g, "-").replace(/\//g, "_").replace(/=+$/, "");
const fromB64u = (s: string) => Uint8Array.from(atob(s.replace(/-/g,
```

```
"+").replace(/_/g, "/")), c => c.charCodeAt(0));
export class ReauthCancelled extends Error {}

export async function getReauthAssertion(layer: "des" | "portail", purpose:
SignPurpose): Promise<string> {
  const ch = await http.post<Challenge>(`/bff/${layer}/iam/reauthentification/defi`,
{ body: { purpose } });
  const cred = await navigator.credentials.get({ publicKey: {
    challenge: fromB64u(ch.challenge), rpId: ch.rpId, timeout: ch.timeoutMs,
userVerification: "required",
    allowCredentials: ch.allowCredentials.map(id => ({ type: "public-key" as const,
id: fromB64u(id) })) } });
  if (!(cred instanceof PublicKeyCredential)) throw new ReauthCancelled();
  const r = cred.response as AuthenticatorAssertionResponse;
  return btoa(JSON.stringify({ challengeId: ch.challengeId, id: cred.id,
clientDataJSON: b64u(r.clientDataJSON),
    authenticatorData: b64u(r.authenticatorData), signature: b64u(r.signature),
    userHandle: r.userHandle ? b64u(r.userHandle) : null }));
}
```

A4. Contrats des composants de domaine et extensions du descripteur

Trois ajouts rétrocompatibles au type du DS-GIMAC-01 §8.2 sont portés au catalogue : `consequences` (contenu du `ConfirmDialog`), `sign` (objet de la réauthentification et empreinte affichée par `SignDialog`) et `steps` (étapes du `ResourceForm`, chacune pouvant se conclure par une action du descripteur). Dans `ResourceForm`, `ctx.row` porte les valeurs saisies fusionnées avec la ressource lue. Les libellés des descripteurs sont des clés i18n résolues par les composants universels. Le conteneur `QueryBoundary` impose les quatre états à toute donnée d'onglet. Aucun composant ne calcule une règle : les couches du graphe, les transitions et les actions permises viennent du service.

```
// src/design-system/domain/types.ts
import type { ReactNode } from "react";
import type { ApiError } from "@/lib/http/client";
export type ViewState = "loading" | "empty" | "error" | "ok";
export type Tone = "ok" | "warn" | "crit" | "neutral" | "info";
export type StatusDictionary = Record<string, { label: string; status: Tone }>;
export type ElementKey = "responsable" | "preuve" | "horodatage" | "validation";
export type ElementState = "REUNI" | "MANQUANT" | "EN_ATTENTE";
export type OperationNodeData = { id: string; code: string; libelle: string; phase:
string; partie: "DES" | "PARTICIPANT";
  etat: string; elements: Record<ElementKey, ElementState>; enRetard: boolean;
aRevoir: boolean };
export type OperationNodeProps = { node: OperationNodeData; dictionary:
StatusDictionary; selected: boolean; onSelect?: (id: string) => void };
export type DagViewProps = { nodes: OperationNodeData[]; edges: { from: string; to:
string }[];
  layers: string[][];                                  // couches topologiques fournies
par GET .../graphe
  // filtres, vue et sélection : contrôlés, ou à défaut synchronisés avec l'URL (?
phase=, ?etat=, ?vue=, ?detail=)
  filters?: { phases: string[]; etats: string[] }; onFiltersChange?: (f: { phases:
string[]; etats: string[] }) => void;
  selectedId?: string; onSelect?: (id: string) => void; view?: "graphe" | "liste";
onViewChange?: (v: "graphe" | "liste") => void;
  readOnly: boolean;                                   // aucune action proposée dans le
tiroir de l'opération
  renderDetail?: (id: string) => ReactNode;            // contenu du tiroir de
l'opération sélectionnée
  dictionary: StatusDictionary; state: ViewState; error?: ApiError };
export type CoverageValue = { taux: number | null; numerateur: number; denominateur:
number; calculable: boolean; motifNonCalculable?: string };
export type CoverageBarProps = { services: { serviceCode: string; serviceLibelle:
string; etatPerimetre: string;
  certification: CoverageValue; miseEnService: CoverageValue }[]; dictionary:
```

```
StatusDictionary; state: ViewState };
export type ProgressRingProps = { value: number | null; labelKey: string; status:
Tone; size: "sm" | "md" };
export type LiaisonProgressProps = { certification: CoverageValue; miseEnService:
CoverageValue; phaseCourante: string;
  services: CoverageBarProps["services"]; dictionary: StatusDictionary; state:
ViewState };
export type FourElementsChecklistProps = { elements: { key: ElementKey; etat:
ElementState; detail?: string;
  actionManquante?: { labelKey: string; href?: string } }[]; state: ViewState;
error?: ApiError };
export type QuitusView = { nature: "TECHNIQUE" | "SECURITE" | "CONFORMITE" |
"DIRECTION_GENERALE"; etat: string;
  signataireRole: string; dateSignature?: string; dateHorodatage?: string; motif?:
string; actionsPermises: string[] };
export type QuitusDecision = { decisionFavorable: boolean; motif: string;
operationsDesignees: string[]; servicesDesignes: string[]; assertion: string };
export type QuitusCircuitProps = { quitus: QuitusView[]; dictionary:
StatusDictionary; state: ViewState;
  socle: { empreinte: string; operations: { code: string; libelle: string }[];
services: string[] }; // désignations possibles
  onSubmit: (q: QuitusView, d: QuitusDecision) => Promise<void> };  // ouvre
SignDialog (QUITUS_SIGN) puis soumet
export type TreeNode = { id: string; label: string; badge?: { label: string; status:
Tone }; children?: TreeNode[] };
export type TreeViewProps = { nodes: TreeNode[]; selectedId?: string; onSelect: (id:
string) => void; labelKey: string; state: ViewState };
export type DiffViewProps<R> = { before: R[]; after: R[]; keyOf: (r: R) => string;
fields: (keyof R & string)[];
  labels: { before: string; after: string } };
export type HashProps = { value: string; onVerify?: () => Promise<boolean> };
/** Conteneur du catalogue : rend chargement, vide, erreur (identifiant de
corrélation, réessai) et nominal. */
export type QueryBoundaryProps<R> = { query: import("@tanstack/react-
query").UseQueryOptions<R, ApiError>;
  isEmpty?: (r: R) => boolean; emptyKey?: string; children: (r: R) => ReactNode };
export type StateMachineBarProps = { states: string[]; current: string; traversed:
string[]; transitionsPermises: string[]; dictionary: StatusDictionary };
export type SignDialogProps = { open: boolean; titleKey: string; empreinte: string;
reasonRequired: boolean;
  summary: { labelKey: string; value: ReactNode }[]; extra?: ReactNode;
  purpose: import("@/lib/reauth/webauthn").SignPurpose; layer: "des" | "portail";
  onSign: (input: { assertion: string; motif: string }) => Promise<void>; onClose: ()
=> void };
```

```
// src/lib/resource/types.ts (extrait : extensions du lot 1)
export type ResourceActionLot1<T> = ResourceAction<T> & {
  consequences?: (row: T) => React.ReactNode;                                    //
corps du ConfirmDialog
  sign?: { purpose: import("@/lib/reauth/webauthn").SignPurpose; empreinte: (row: T)
=> string }; // SignDialog
};
export type FormStep<T> = { key: string; labelKey: string; fields: (keyof T & string)
[]; action?: string }; // action de fin d'étape
export type ResourceDescriptorLot1<T> = Omit<ResourceDescriptor<T>, "actions"> & {
  actions: ResourceActionLot1<T>[]; steps?: FormStep<T>[] };
export type Permitted = { transitionsPermises: string[]; actionsPermises: string[] };
export type ActionContext<T> = { row: T; selection: T[]; motif?: string; assertion?:
string;
  navigate: (to: string) => void; openForm: (mode: "create" | "edit") => void;
openTab: (key: string) => void;
  invalidate: () => void };
```

A5. Dictionnaires d'états, catalogues i18n et conventions de test

```
// src/resources/homologation/dictionaries.ts
import type { StatusDictionary } from "@/design-system/domain/types";
const d = (p: string, e: Record<string, StatusDictionary[string]["status"]>):
```

```
StatusDictionary =>
  Object.fromEntries(Object.entries(e).map(([k, status]) => [k, { label: `${p}.${k}`,
status }]));
export const etatLiaison = d("liaison.etat", { OUVERTE: "neutral", EN_HOMOLOGATION:
"info", GELEE: "crit",
  APTE_CERTIFICATION: "info", MISE_EN_SERVICE_AUTORISEE: "ok", CERTIFIEE: "ok",
CLOSE: "neutral" });
export const etatOperation = d("operation.etat", { VERROUILLEE: "neutral", ELIGIBLE:
"info", PREUVE_DEPOSEE: "warn",
  VALIDEE_HORODATAGE_EN_ATTENTE: "warn", CLOSE: "ok", ROUVERTE: "crit",
HORS_PERIMETRE: "neutral" });
export const etatScope = d("perimetre.etat", { A_ENGAGER: "neutral", EN_COURS:
"info", COUVERT: "ok", HORS_PERIMETRE: "neutral" });
// src/resources/certification/dictionaries.ts (même fabrique d, importée de
lib/resource)
export const etatQuitus = d("quitus.etat", { ATTENDU: "info", SIGNE_FAVORABLE: "ok",
SIGNE_DEFAVORABLE: "crit",
  CADUC: "warn", ANNULE: "neutral", NON_REQUIS: "neutral" });
export const etatDossier = d("dossier.etat", { OUVERT: "info", AUTORISE: "ok",
MIS_EN_SERVICE: "ok", COMPLET: "ok", ATTESTE: "ok" });
export const etatAttestation = d("attestation.etat", { EN_VIGUEUR: "ok", SUSPENDUE:
"warn", REVOQUEE: "crit", REMPLACEE: "neutral" });
```

```
// src/i18n/fr.json (extrait ; en.json porte les mêmes clés)
{ "liaison.etat.GELEE": "gelée", "operation.etat.VALIDEE_HORODATAGE_EN_ATTENTE":
"validée, horodatage en attente",
  "quitus.etat.CADUC": "caduc", "dossier.etat.ATTESTE": "attesté",
"attestation.etat.REVOQUEE": "révoquée",
  "commun.erreur.correlation": "Identifiant de corrélation : {{id}}",
"commun.reessayer": "Réessayer",
  "sign.cle": "Confirmez avec votre clé de sécurité", "sign.empreinte": "Empreinte
signée",
  "live.perdu": "Mises à jour en temps réel interrompues, reconnexion en cours" }
```

Conventions de test : un fichier `*.test.tsx` à côté de chaque composant et descripteur ; `lib/http` est simulé par `vi.mock("@/lib/http/client")` (aucun serveur factice appelant le réseau) ; les fabriques de `src/test/factories` produisent « Établissement de paiement A » et « Banque B » ; `renderWithShell` monte l'enveloppe, i18n et un `QueryClient` sans réessai ; les tests portent l'identifiant de l'écran en tête de nom.

B. Tests unitaires (Vitest + Testing Library)

```
// src/lib/http/client.test.ts
import { configureHttp, http, ApiError } from "./client";
const reply = (status: number, body: object, cid = "cid-1") =>
  vi.spyOn(globalThis, "fetch").mockResolvedValueOnce(new
Response(JSON.stringify(body), { status, headers: { "X-Correlation-Id": cid } }));

it("F0 : une erreur 409 est normalisée en ApiError avec son identifiant de
corrélation", async () => {
  reply(409, { code: "HOM_CLOSURE_ELEMENT_MISSING", message: "preuve", correlationId:
"cid-409" });
  await
expect(http.post("/bff/des/homologation/operations/o1/validation")).rejects.toMatchOb
ject(
    { status: 409, code: "HOM_CLOSURE_ELEMENT_MISSING", correlationId: "cid-409" }
satisfies Partial<ApiError>);
});
it("F0 : un 401 déclenche la réauthentification puis rejoue la requête une seule
fois", async () => {
  const onUnauthorized = vi.fn(async () => undefined);
  configureHttp({ onUnauthorized, onForbidden: vi.fn(), onCorrelation: vi.fn() });
  reply(401, {}); reply(200, { ok: true });
  await expect(http.get("/bff/des/homologation/liaisons")).resolves.toEqual({ ok:
true });
```

```
  expect(onUnauthorized).toHaveBeenCalledTimes(1);
});
it("F0 : l'assertion est transmise dans X-Reauth-Assertion et aucun en-tête
Authorization n'est posé", async () => {
  const spy = reply(200, {});
  await http.post("/bff/des/homologation/operations/o1/validation", { body: {},
reauthAssertion: "YXNzZXJ0" });
  const h = new Headers(spy.mock.calls[0][1]?.headers);
  expect(h.get("X-Reauth-Assertion")).toBe("YXNzZXJ0");
expect(h.has("Authorization")).toBe(false);
});
```

C. Storybook et accessibilité (axe)

```
// src/design-system/data/StateMachineBar/StateMachineBar.stories.tsx
import type { Meta, StoryObj } from "@storybook/react";
import { StateMachineBar } from "./StateMachineBar";
import { etatLiaison } from "@/resources/homologation/dictionaries";
const states = Object.keys(etatLiaison);
export default { component: StateMachineBar, args: { states, dictionary:
etatLiaison } } satisfies Meta<typeof StateMachineBar>;
type S = StoryObj<typeof StateMachineBar>;
export const Ouverte: S = { args: { current: "OUVERTE", traversed: [],
transitionsPermises: ["EN_HOMOLOGATION"] } };
export const Gelee: S = { args: { current: "GELEE", traversed: ["OUVERTE",
"EN_HOMOLOGATION"], transitionsPermises: [] } };
export const Certifiee: S = { args: { current: "CERTIFIEE", traversed:
states.slice(0, 5), transitionsPermises: ["CLOSE"] } };
```

```
// src/test/a11y.test.tsx : balaye toutes les histoires du catalogue, clair et sombre
import { composeStories } from "@storybook/react";
import { axe } from "vitest-axe";
import { renderWithShell } from "./render";
const modules = import.meta.glob<Record<string,
unknown>>("../design-system/**/*.stories.tsx", { eager: true });
describe.each(Object.entries(modules))("%s", (_path, mod) => {
  it.each(Object.entries(composeStories(mod)))("F0 : %s sans violation axe", async
(_n, Story) => {
    for (const theme of ["light", "dark"]) {
      document.documentElement.dataset.theme = theme;
      const { container, unmount } = renderWithShell(<Story />);
      expect(await axe(container)).toHaveNoViolations(); unmount();
    }
  });
});
```

D. Tests Gherkin (Playwright BDD)

```
# language: fr
Fonctionnalité: F0 Enveloppe et session du lot 1
  Scénario: Session expirée pendant la saisie
    Étant donné un ingénieur d'homologation connecté sur l'écran des liaisons
    Quand la passerelle répond 401 à la requête suivante
    Alors le dialogue de réauthentification s'affiche
    Et la saisie en cours est conservée après reconnexion

  Scénario: Accès refusé
    Étant donné un correspondant d'institution connecté sur le portail
    Quand il ouvre l'adresse d'un écran de l'application du département
    Alors la page « Accès refusé » s'affiche avec un identifiant de corrélation

  Scénario: Perte du canal poussé
    Étant donné un utilisateur sur la vue d'une liaison
    Quand le canal poussé est interrompu
```

```
    Alors le bandeau « Mises à jour en temps réel interrompues, reconnexion en cours
» s'affiche
```

E. Vérification via la passerelle (Playwright e2e)

```
// e2e/specs/f0-enveloppe.spec.ts : baseURL = process.env.GATEWAY_URL ; adaptateurs
de substitution actifs
import { test, expect } from "@playwright/test";
test.use({ storageState: "e2e/.auth/test-ingenieur-homologation.json" });
const PREFERENCES = new Set(["gm.theme", "gm.density", "gm.filters.favoris",
"i18nextLng"]);

test("F0 : session HttpOnly, en-têtes de sécurité et stockage local limité aux
préférences", async ({ page, context }) => {
  const res = await page.goto("/des/liaisons");
  const csp = res?.headers()["content-security-policy"] ?? "";
  expect(csp).toContain("default-src 'self'"); expect(csp).toContain("frame-ancestors
'none'");
  const session = (await context.cookies()).filter(c =>
c.name.startsWith("GIMAC_SESSION"));
  expect(session.length).toBeGreaterThan(0);
  for (const c of session) { expect(c.httpOnly).toBe(true);
expect(c.secure).toBe(true); expect(c.sameSite).toBe("Strict"); }
  const keys = await page.evaluate(() =>
[...Object.keys(localStorage), ...Object.keys(sessionStorage)]);
  expect(keys.filter(k => !PREFERENCES.has(k) && !
k.startsWith("gm.draft."))).toEqual([]);
  expect(await page.evaluate(() =>
document.cookie)).not.toContain("GIMAC_SESSION"); // illisible par le script
});
test("F0 : une ressource introuvable affiche l'identifiant de corrélation", async
({ page }) => {
  await page.goto("/des/liaisons/00000000-0000-0000-0000-000000000000");
  await expect(page.getByTestId("correlation-id")).toHaveText(/[0-9a-f-]{8,}/);
});
```

DES-12 | Référentiels d'opérations

Élément	Valeur
Points	18 (HOM-01 : 5, HOM-02 : 5, HOM-03 : 8)
Priorité	Must Have (HOM-03 : Should Have)
Sprint cible	Sprint 3
Gabarit	G5 Console de paramétrage (`TreeView`, `ResourceForm`, `DiffView`)
Récits couverts	HOM-01, HOM-02, HOM-03

Objectif

Livrer l'écran `/des/referentiels/operations` : arbre par type de projet (tronc commun amont, branche, tronc commun aval), édition d'un brouillon en tableau avec tiroir par opération, comparaison avec la version active, contrôle d'acyclicité restitué tel que calculé par le service, publication confirmée et simulation de migration. L'écran est vérifié quand un cycle de trois opérations est restitué dans l'ordre et que le bouton de publication n'existe pas sur une version publiée.

A. Implémentation

A1. Types et descripteur `referentielOperationsDescriptor`

Le descripteur décrit la version (agrégat publié) ; un descripteur interne au fichier décrit ses opérations pour le tableau d'édition. Les actions ne sont affichées que si `actionsPermises` les contient : l'interface ne sait pas qu'une version publiée est figée, elle constate que `MODIFIER` est absent.

```
// src/resources/homologation/referentielOperations.tsx
import { http, P } from "@/lib/http/client";
import type { ResourceDescriptorLot1, Permitted } from "@/lib/resource/types";
import { DiffView, QueryBoundary, InlineAlert } from "@/design-system";
import { ResourceList } from "@/lib/resource/ResourceList";
import { ControleRestitution, ImpactMigrationTable } from "@/design-system/domain";
export type Segment = "TRONC_COMMUN_AMONT" | "BRANCHE" | "TRONC_COMMUN_AVAL";
export type ReferentielVersion = Permitted & { id: string; typeProjet:
"CARTE_INTERBANCAIRE" | "PAIEMENT_MOBILE" | "DELEGATAIRE";
  numero: number | null; libelle: string; etat: "BROUILLON" | "PUBLIEE" | "RETIREE";
  dateCreation: string; datePublication: string | null; versionSourceId: string |
null; versionActiveId: string | null };
export type ReferenceOperation = Permitted & { versionId: string; code: string;
libelle: string; phase: string; segment: Segment;
  partie: "DES" | "PARTICIPANT"; serviceCode: string; natureCouverture:
"CERTIFICATION" | "MISE_EN_SERVICE" | "AUCUNE";
  idScenario: string | null; predecesseurs: string[]; fonctionResponsable: string;
preuveAttendue: Record<string, string> };
export type ImpactMigration = { liaisonId: string; institutionNom: string;
conservees: number; ajoutees: number;
  retirees: number; modifiees: number; exclue: boolean; motifExclusion: string | null
};
export type ControleResultat = { acyclique: boolean; ordreTopologique: string[];
cycle: string[];
  incoherences: { code: string; predecesseur: string; motif: string }[]; manquants: {
code: string; attributs: string[] }[] };
const R = `${P.desHomologation}/referentiels`;
export const referentielQueries = {
  operations: (id: string) => ({ queryKey: ["homologation", "referentiel", id,
"operations"],
```

```
    queryFn: () => http.get<ReferenceOperation[]>(`${R}/${id}/operations`) }),
  // contrôle à blanc et simulation : sans effet, traités comme des lectures
(relancées par invalidation)
  controle: (id: string) => ({ queryKey: ["homologation", "referentiel", id,
"controle"],
    queryFn: () => http.post<ControleResultat>(`${R}/${id}/controle`) }),
  simulation: (id: string) => ({ queryKey: ["homologation", "referentiel", id,
"simulation"],
    queryFn: () =>
http.post<ImpactMigration[]>(`${R}/${id}/migrations/simulation`) }),
  liaisonsImpactees: (id: string) => ({ queryKey: ["homologation", "referentiel", id,
"impact"],
    queryFn: () => http.get<{ total: number }>(`${P.desHomologation}/liaisons`,
{ query: { versionId: id, actives: true, taille: 1 } }) }),
};

export const referentielOperationsDescriptor:
ResourceDescriptorLot1<ReferentielVersion> = {
  key: "referentielOperations",
  label: { singular: "referentiel.singulier", plural: "referentiel.pluriel", gender:
"m" },
  idField: "id", titleField: "libelle", statusField: "etat",
  endpoints: { list: R, get: `${R}/{id}`, create: R },
  fields: [
    { name: "typeProjet", label: "referentiel.typeProjet", kind: "select", required:
true, inList: true, inDetail: true, inForm: "create", filterable: true,
      options: ["CARTE_INTERBANCAIRE", "PAIEMENT_MOBILE", "DELEGATAIRE"].map(v =>
({ value: v, label: `typeProjet.${v}` })) },
    { name: "numero", label: "referentiel.numero", kind: "number", inList: true,
inDetail: true, readOnly: true },
    { name: "libelle", label: "referentiel.libelle", kind: "text", required: true,
inList: true, inDetail: true, inForm: true },
    { name: "etat", label: "referentiel.etat", kind: "status", inList: true,
inDetail: true, filterable: true },
    { name: "dateCreation", label: "commun.dateCreation", kind: "datetime", inDetail:
true, readOnly: true },
    { name: "datePublication", label: "referentiel.datePublication", kind:
"datetime", inList: true, inDetail: true, readOnly: true },
  ],
  actions: [
    { key: "creer", label: "referentiel.action.creer", icon: "file-plus", variant:
"primary", scope: "page",
      permission: "HOM_REFERENTIEL_EDIT", confirm: "confirm", run: async ctx =>
ctx.openForm("create") },
    { key: "deriver", label: "referentiel.action.deriver", icon: "git-branch", scope:
"row", permission: "HOM_REFERENTIEL_EDIT",
      confirm: "confirm", isAllowed: r => r.actionsPermises.includes("DERIVER"),
      run: async ({ row, navigate }) => navigate(`?version=${(await
http.post<ReferentielVersion>(R, { body: { versionSourceId: row.id } })).id}`) },
    { key: "controler", label: "referentiel.action.controler", icon: "shield-check",
scope: "row", permission: "HOM_REFERENTIEL_EDIT",
      confirm: "none", isAllowed: r => r.actionsPermises.includes("CONTROLER"),
      run: ({ openTab, invalidate }) => { openTab("controle"); invalidate(); return
Promise.resolve(); } },
    { key: "publier", label: "referentiel.action.publier", icon: "stamp", variant:
"primary", scope: "row",
      permission: "HOM_REFERENTIEL_PUBLISH", confirm: "confirm", isAllowed: r =>
r.actionsPermises.includes("PUBLIER"),
      consequences: r => <QueryBoundary
query={referentielQueries.liaisonsImpactees(r.versionActiveId ?? r.id)}>
        {i => <InlineAlert tone="info" messageKey="referentiel.publier.impact"
values={{ n: i.total }} />}</QueryBoundary>,
      run: ({ row, invalidate }) =>
http.post(`${R}/${row.id}/publication`).then(invalidate) },
    { key: "simuler", label: "referentiel.action.simulerMigration", icon: "flask-
conical", scope: "row",
      permission: "HOM_REFERENTIEL_MIGRATE", confirm: "none", isAllowed: r =>
r.actionsPermises.includes("MIGRER"),
      run: ({ openTab }) => { openTab("migration"); return Promise.resolve(); } },
```

```
  ],
  tabs: [
    { key: "operations", label: "referentiel.onglet.operations",
      render: r => <ResourceList descriptor={referenceOperationDescriptor} embedded
fixedFilters={{ versionId: r.id }} /> },
    { key: "comparaison", label: "referentiel.onglet.comparaison", render: r =>
r.versionActiveId === null ? null :
      <QueryBoundary query={referentielQueries.operations(r.versionActiveId)}>{active
=>
        <QueryBoundary query={referentielQueries.operations(r.id)}>{courante =>
          <DiffView before={active} after={courante} keyOf={o => o.code}
labels={{ before: "referentiel.active", after: "referentiel.courante" }}
            fields={["libelle", "phase", "segment", "partie", "serviceCode",
"predecesseurs", "idScenario"]} />}</QueryBoundary>}</QueryBoundary> },
    { key: "controle", label: "referentiel.onglet.controle", render: r =>
      <QueryBoundary query={referentielQueries.controle(r.id)}>{res =>
<ControleRestitution resultat={res} />}</QueryBoundary> },
    { key: "migration", label: "referentiel.onglet.migration", render: r =>
      <QueryBoundary query={referentielQueries.simulation(r.id)}
emptyKey="referentiel.migration.aucune">{impacts =>
        <ImpactMigrationTable impacts={impacts}
canApprove={r.actionsPermises.includes("MIGRER")}
          onApprove={ids => http.post(`${R}/${r.id}/migrations`, { body:
{ liaisonsRetenues: ids } })} />}</QueryBoundary> },
  ],
  timeline: r => `${R}/${r.id}/historique`,
  statusDictionary: { BROUILLON: { label: "referentiel.etat.BROUILLON", status:
"info" },
    PUBLIEE: { label: "referentiel.etat.PUBLIEE", status: "ok" }, RETIREE: { label:
"referentiel.etat.RETIREE", status: "neutral" } },
  permissions: { list: "HOM_REFERENTIEL_READ", read: "HOM_REFERENTIEL_READ", create:
"HOM_REFERENTIEL_EDIT", update: "HOM_REFERENTIEL_EDIT" },
  liveTopic: "homologation",
};
```

Le descripteur `referenceOperationDescriptor` (même fichier) déclare les colonnes du tableau d'édition (code en `mono`, libellé, phase, segment, partie, service, nature de couverture, prédécesseurs en `multiSelect`, identifiant de scénario, preuve attendue par catégorie en `custom`), l'endpoint `PUT ${R}/{versionId}/operations/{code}` et l'action `modifier` (`HOM_REFERENTIEL_EDIT`, `isAllowed: r => r.actionsPermises.includes("MODIFIER")`) qui ouvre le tiroir de détail en édition.

A2. Page G5 et arbre

La page compose le gabarit à deux volets ; l'arbre groupe les données reçues par type, version et segment (regroupement d'affichage, aucune règle).

```
// src/features/referentiels/ReferentielsOperationsPage.tsx
export default function ReferentielsOperationsPage() {
  const [versionId, setVersionId] = useSearchParam("version");
  const tree = useReferentielTree(); // features/referentiels/hooks.ts : GET
/referentiels puis groupement
  return (
    <SettingsTemplate titleKey="referentiel.titre"
      aside={<TreeView labelKey="referentiel.arbre" nodes={tree.nodes}
state={tree.state} selectedId={versionId} onSelect={setVersionId} />}
      main={versionId ? <ResourceDetail descriptor={referentielOperationsDescriptor}
id={versionId} mode="page" />
                      : <EmptyState cause="selection"
titleKey="referentiel.vide.titre" />} />
  );
}
```

A3. Restitution du contrôle et rapport de migration

Les deux composants de domaine ne font aucun appel réseau : ils reçoivent la réponse obtenue par `QueryBoundary`. `ControleRestitution` reçoit le résultat de `POST ${R}/{id}/controle` et affiche l'ordre topologique en `Tag` numérotés si le graphe est acyclique, sinon la suite ordonnée des codes du cycle (`A → B → C → A`), les incohérences entre segments (code, prédécesseur, motif) et les attributs manquants par opération. `ImpactMigrationTable` reçoit la simulation (`POST ${R}/{id}/migrations/simulation`), affiche par liaison les opérations conservées, ajoutées, retirées et modifiées, rend non sélectionnables les liaisons exclues avec leur motif (circuit de quitus ouvert), puis appelle `onApprove` avec les liaisons retenues après son `ConfirmDialog` (`POST ${R}/{id}/migrations`) ; le rapport reste consultable par `GET /migrations/{id}/rapport`.

```
// src/i18n/fr.json (extrait)
{ "referentiel.titre": "Référentiels d'opérations", "referentiel.action.creer":
"Créer un brouillon", "referentiel.action.publier": "Publier la version",
  "referentiel.action.deriver": "Dériver un brouillon",
"referentiel.action.controler": "Contrôler le graphe",
  "referentiel.publier.impact": "{{n}} liaisons en cours restent sur la version
active jusqu'à une migration décidée.",
  "referentiel.controle.cycle": "Cycle détecté : {{suite}}",
"referentiel.controle.incoherence": "Incohérence entre segments",
  "referentiel.migration.exclue": "Exclue : circuit de quitus ouvert" }
```

B. Tests unitaires (Vitest + Testing Library)

```
// src/resources/homologation/referentielOperations.test.tsx
vi.mock("@/lib/http/client");
it("DES-12 : le bouton de publication est absent sur une version publiée", async ()
=> {
  mockGet(versionFactory({ etat: "PUBLIEE", actionsPermises: ["DERIVER"] }));
  renderWithShell(<ResourceDetail descriptor={referentielOperationsDescriptor}
id="v1" mode="page" />);
  expect(await screen.findByRole("button", { name: "Dériver un
brouillon" })).toBeVisible();
  expect(screen.queryByRole("button", { name: "Publier la version" })).toBeNull();
});
it("DES-12 : le cycle est restitué sous la forme de la suite ordonnée des codes",
async () => {
  renderWithShell(<ControleRestitution resultat={{ acyclique: false,
ordreTopologique: [],
    cycle: ["OP-A", "OP-B", "OP-C", "OP-A"], incoherences: [], manquants: [] }} />);
  expect(await screen.findByText("Cycle détecté : OP-A → OP-B → OP-C → OP-
A")).toBeVisible();
});
it.each(["loading", "empty", "error", "ok"] as const)("DES-12 : l'arbre rend l'état
%s", async state => {
  renderWithShell(<TreeView labelKey="referentiel.arbre" nodes={state === "ok" ?
treeFactory() : []} state={state} onSelect={vi.fn()} />);
  expect(screen.getByTestId(`tree-${state}`)).toBeInTheDocument();
});
```

C. Storybook et accessibilité (axe)

```
// src/design-system/data/TreeView/TreeView.stories.tsx
export default { component: TreeView, args: { labelKey: "referentiel.arbre",
onSelect: fn() } } satisfies Meta<typeof TreeView>;
export const Chargement: S = { args: { nodes: [], state: "loading" } };
export const Vide: S = { args: { nodes: [], state: "empty" } };
```

```
export const TroncEtBranches: S = { args: { nodes: treeFactory(), state: "ok",
selectedId: "PAIEMENT_MOBILE/v3" } };
// src/design-system/data/DiffView/DiffView.stories.tsx
export const AjoutRetraitModification: S = { args: { before: opsV2, after: opsV3,
keyOf: o => o.code,
  fields: ["libelle", "predecesseurs"], labels: { before: "referentiel.active",
after: "referentiel.courante" } } };
```

Les deux composants sont couverts par le balayage axe de F0 ; `TreeView` est en plus testé au clavier (flèches, `Home`, `End`, rôle `tree` et `aria-expanded`).

D. Tests Gherkin (Playwright BDD)

```
# language: fr
Fonctionnalité: DES-12 Référentiels d'opérations (HOM-01 à HOM-03)
  Contexte:
    Étant donné un ingénieur d'homologation connecté sur l'écran des référentiels
d'opérations

  Scénario: Publication d'une version du référentiel
    Étant donné un brouillon complet du type paiement mobile
    Quand il clique sur « Publier la version » puis confirme
    Alors la version apparaît à l'état « publiée » avec un numéro et une date
    Et le bouton « Publier la version » n'est plus proposé

  Scénario: Évolution d'une version publiée
    Étant donné une version publiée du type carte interbancaire
    Quand il ouvre une opération de cette version
    Alors aucune action de modification n'est proposée
    Et l'action « Dériver un brouillon » est proposée

  Scénario: Opération incomplète
    Étant donné un brouillon dont une opération n'indique ni la partie ni la preuve
attendue
    Quand il demande la publication
    Alors un bandeau d'erreur liste l'opération et les attributs manquants avec
l'identifiant de corrélation

  Scénario: Cycle détecté
    Étant donné un brouillon où OP-A, OP-B et OP-C se désignent en boucle comme
prédécesseurs
    Quand il clique sur « Contrôler le graphe »
    Alors le cycle « OP-A → OP-B → OP-C → OP-A » est affiché

  Scénario: Liaison engagée dans le circuit de quitus
    Étant donné le chef du DES sur la simulation de migration d'une nouvelle version
    Quand une liaison au circuit de quitus ouvert figure au rapport
    Alors sa case de sélection est absente et le motif d'exclusion est affiché
```

E. Vérification via la passerelle (Playwright e2e)

```
// e2e/specs/des12-referentiels.spec.ts
test.use({ storageState: "e2e/.auth/test-ingenieur-homologation.json" });
test("DES-12 : du brouillon à la publication, avec refus tracé", async ({ page }) =>
{
  await page.goto("/des/referentiels/operations");
  await page.getByRole("button", { name: "Créer un brouillon" }).click();
  await fillForm(page, { "Type de projet": "Paiement mobile", "Libellé": "Parcours
paiement mobile" });
  await page.getByRole("button", { name: "Confirmer" }).click();
  await page.getByRole("button", { name: "Publier la version" }).click();
  await page.getByRole("button", { name: "Publier" }).click();          // brouillon
vide : refus 409 attendu
  await expect(page.getByTestId("correlation-id")).toBeVisible();
  await saisirOperations(page, "e2e/fixtures/referentiel-paiement-mobile.json"); //
```

```
saisie par le tableau et le tiroir
  await page.getByRole("button", { name: "Contrôler le graphe" }).click();
  await expect(page.getByText("Ordre topologique")).toBeVisible();
  await page.getByRole("button", { name: "Publier la version" }).click();
  await page.getByRole("button", { name: "Publier" }).click();
  await expect(page.getByTestId("status-badge")).toHaveText("publiée");
});
```

DES-08 | Liaisons et parcours

Élément	Valeur
Points	10 (HOM-04 : 5, HOM-20 : 5)
Priorité	Must Have (HOM-20 : Should Have)
Sprint cible	Sprint 3
Gabarit	G2 Liste et tiroir (`ResourceList`)
Récits couverts	HOM-04, HOM-20

Objectif

Livrer l'écran `/des/liaisons` : liste des liaisons avec couvertures et retards, ouverture d'une liaison (institution, type de projet, chef de projet, portage), désignation du chef de projet, dégel motivé, et onglet de pilotage (délais par phase, opérations en retard, tiroir de détail). L'écran est vérifié quand l'ouverture en double est refusée avec son motif et son identifiant de corrélation, et que le dégel n'est proposé que si le service le permet.

A. Implémentation

A1. Type et descripteur `liaisonDescriptor`

Le champ `chefDeProjetId` est le seul éditable après création : le formulaire d'édition se réduit donc à la désignation du chef de projet (`PUT /liaisons/{id}/chef-de-projet`). Les onglets du détail sont écrits par DES-09 dans `liaison.tabs.tsx`.

```
// src/resources/homologation/liaison.tsx
import { http, P } from "@/lib/http/client";
import type { ResourceDescriptorLot1, Permitted } from "@/lib/resource/types";
import type { CoverageValue } from "@/design-system/domain/types";
import { ProgressRing, StatusBadge, InlineAlert, Text } from "@/design-system";
import { etatLiaison } from "./dictionaries";
import { rechercheInstitutions } from "@/resources/participant/options";   // DS-
GIMAC-01 §8.4
import { rechercheCollaborateursHomologation, typeProjetOptions } from "./options";
import { liaisonTabs } from "./liaison.tabs";
export type EtatLiaison = "OUVERTE" | "EN_HOMOLOGATION" | "GELEE" |
"APTE_CERTIFICATION"
  | "MISE_EN_SERVICE_AUTORISEE" | "CERTIFIEE" | "CLOSE";
export type Liaison = Permitted & { id: string; typeProjet: "CARTE_INTERBANCAIRE" |
"PAIEMENT_MOBILE" | "DELEGATAIRE";
  institutionId: string; institutionNom: string; etat: EtatLiaison; chefDeProjetId:
string; chefDeProjetNom: string;
  banquePartenaireId: string | null; banquePartenaireNom: string | null;
dateOuverture: string; motifEtat: string | null;
  versionId: string; versionNumero: number; phaseCourante: string;
couvertureCertification: CoverageValue;
  couvertureMiseEnService: CoverageValue; operationsEnRetard: number; etatsParcourus:
EtatLiaison[] };
const L = `${P.desHomologation}/liaisons`;

export const liaisonDescriptor: ResourceDescriptorLot1<Liaison> = {
  key: "liaison",
  label: { singular: "liaison.singulier", plural: "liaison.pluriel", gender: "f" },
  idField: "id", titleField: "institutionNom", statusField: "etat",
  endpoints: { list: L, get: `${L}/{id}`, create: L, update: `${L}/{id}/chef-de-
projet` },
  fields: [
```

```
    { name: "institutionId", label: "liaison.institution", kind: "searchSelect",
required: true, options: rechercheInstitutions, inForm: "create" },
    { name: "institutionNom", label: "liaison.institution", kind: "text", inList:
true, inDetail: true, sortable: true, filterable: true },
    { name: "typeProjet", label: "liaison.typeProjet", kind: "radio", required: true,
options: typeProjetOptions,
      inList: true, inDetail: true, inForm: "create", filterable: true },
    { name: "chefDeProjetId", label: "liaison.chefDeProjet", kind: "searchSelect",
required: true,
      options: rechercheCollaborateursHomologation, inForm: true, help:
"liaison.chefDeProjet.aide" },
    { name: "chefDeProjetNom", label: "liaison.chefDeProjet", kind: "text", inList:
true, inDetail: true, filterable: true },
    { name: "banquePartenaireId", label: "liaison.portage", kind: "searchSelect",
options: rechercheInstitutions,
      inForm: "create", help: "liaison.portage.aide" },
    { name: "banquePartenaireNom", label: "liaison.portage", kind: "text", inDetail:
true },
    { name: "etat", label: "liaison.etat", kind: "status", inList: true, inDetail:
true, filterable: true },
    { name: "phaseCourante", label: "liaison.phase", kind: "text", inList: true,
inDetail: true },
    { name: "couvertureCertification", label: "couverture.certification", kind:
"custom", inList: true,
      render: (_v, r) => <ProgressRing value={r.couvertureCertification.taux}
labelKey="couverture.certification" status="info" size="sm" /> },
    { name: "couvertureMiseEnService", label: "couverture.miseEnService", kind:
"custom", inList: true,
      render: (_v, r) => <ProgressRing value={r.couvertureMiseEnService.taux}
labelKey="couverture.miseEnService" status="info" size="sm" /> },
    { name: "operationsEnRetard", label: "liaison.retards", kind: "number", inList:
true, sortable: true,
      render: (_v, r) => r.operationsEnRetard > 0 ? <StatusBadge status="warn"
label={<Text i18nKey="liaison.retards.n" values={{ n: r.operationsEnRetard }} />}
/> : null },
    { name: "versionNumero", label: "liaison.version", kind: "number", inDetail: true
},
    { name: "dateOuverture", label: "liaison.dateOuverture", kind: "date", inList:
true, inDetail: true, sortable: true },
    { name: "motifEtat", label: "liaison.motifEtat", kind: "text", inDetail: true },
  ],
  actions: [
    { key: "ouvrir", label: "liaison.action.ouvrir", icon: "folder-plus", variant:
"primary", scope: "page",
      permission: "HOM_LIAISON_OPEN", confirm: "confirm", run: async ctx =>
ctx.openForm("create") },
    { key: "chefDeProjet", label: "liaison.action.designerChef", icon: "user-cog",
scope: "row", permission: "HOM_LIAISON_OPEN",
      confirm: "confirm", isAllowed: r =>
r.actionsPermises.includes("DESIGNER_CHEF_DE_PROJET"), run: async ctx =>
ctx.openForm("edit") },
    { key: "degel", label: "liaison.action.degeler", icon: "lock-open", variant:
"danger", scope: "row",
      permission: "HOM_LIAISON_UNFREEZE", confirm: "confirm", reason: true,
isAllowed: r => r.actionsPermises.includes("DEGELER"),
      consequences: r => <InlineAlert tone="info"
messageKey="liaison.degel.consequence" values={{ motif: r.motifEtat ?? "" }} />,
      run: ({ row, motif, invalidate }) => http.post(`${L}/${row.id}/degel`, { body:
{ motif } }).then(invalidate) },
    { key: "exporterPreuves", label: "liaison.action.exporterPreuves", icon:
"archive", scope: "row",  // DES-09
      permission: "HOM_EVIDENCE_FILE_EXPORT", confirm: "confirm", isAllowed: r =>
r.actionsPermises.includes("EXPORTER_PREUVES"),
      run: ({ row, openTab }) => http.post(`${L}/${row.id}/dossier-de-
preuves`).then(() => openTab("preuves")) },
  ],
  tabs: liaisonTabs,
  timeline: r => `${L}/${r.id}/historique`,
  statusDictionary: etatLiaison,
```

```
  permissions: { list: "HOM_LIAISON_LIST", read: "HOM_LIAISON_READ", create:
"HOM_LIAISON_OPEN", update: "HOM_LIAISON_OPEN" },
  liveTopic: "homologation",
};
```

A2. Descripteur de pilotage (HOM-20)

La projection `GET /tableau-de-bord/liaisons` étend `Liaison` avec les délais de la phase courante ; le tiroir reprend les onglets graphe, périmètre et chronologie de DES-09 et ajoute un onglet des opérations en retard (jours de dépassement, partie qui en répond).

```
// src/resources/homologation/liaisonPilotage.tsx
import { operationDescriptor } from "./operation";
export type LiaisonPilotage = Liaison & { delaiConsomme: number; delaiStandard:
number };
export const tableauDeBordLiaisonsDescriptor: ResourceDescriptorLot1<LiaisonPilotage>
= {
  ...liaisonDescriptor, key: "liaisonPilotage", actions: [],
  endpoints: { list: `${P.desHomologation}/tableau-de-bord/liaisons`, get:
`${L}/{id}` },
  fields: [
    ...liaisonDescriptor.fields.filter(f => f.inList),
    { name: "delaiConsomme", label: "pilotage.delai", kind: "custom", inList: true,
sortable: true,
      render: (_v, r) => <Text i18nKey="pilotage.delai.valeur" values={{ consomme:
r.delaiConsomme, standard: r.delaiStandard }} /> },
  ],
  tabs: [
    { key: "retards", label: "pilotage.onglet.retards", render: r =>
      <ResourceList descriptor={operationDescriptor} embedded
fixedFilters={{ liaisonId: r.id, enRetard: true }} /> },
    ...liaisonTabs.filter(t => ["graphe", "perimetre"].includes(t.key)),
  ],
  permissions: { list: "HOM_DASHBOARD_READ", read: "HOM_LIAISON_READ" },
};
```

A3. Options de recherche et page

```
// src/resources/homologation/options.ts
export const typeProjetOptions = ["CARTE_INTERBANCAIRE", "PAIEMENT_MOBILE",
"DELEGATAIRE"].map(v => ({ value: v, label: `typeProjet.${v}` }));
/** Collaborateurs habilités : l'admissibilité est revérifiée par le service à la
soumission. */
export const rechercheCollaborateursHomologation = (recherche = "") =>
  http.get<{ contenu: { id: string; nom: string; role: string }[]
}>("/bff/des/iam/collaborateurs",
    { query: { recherche, roles: "INGENIEUR_HOMOLOGATION,CHEF_DES" } })
    .then(p => p.contenu.map(c => ({ value: c.id, label: c.nom, hint: `role.$
{c.role}` })));
```

```
// src/features/liaisons/LiaisonsPage.tsx
export default function LiaisonsPage() {
  const [onglet, setOnglet] = useSearchParam("onglet", "liste");
  return <Tabs value={onglet} onChange={setOnglet} items={[
    { key: "liste", labelKey: "liaison.onglet.liste", content: <ResourceList
descriptor={liaisonDescriptor} /> },
    { key: "pilotage", labelKey: "liaison.onglet.pilotage", permission:
"HOM_DASHBOARD_READ",
      content: <ResourceList descriptor={tableauDeBordLiaisonsDescriptor}
defaultQuery={{ tri: "operationsEnRetard,desc" }} /> },
  ]} />;
}
```

```
// src/i18n/fr.json (extrait)
{ "liaison.action.ouvrir": "Ouvrir une liaison", "liaison.action.degeler": "Dégeler
```

```
la liaison",
  "liaison.action.designerChef": "Désigner le chef de projet",
"liaison.portage.aide": "Banque partenaire qui porte le participant, le cas échéant",
  "liaison.degel.consequence": "Chaque opération retrouvera l'état qu'elle avait
avant le gel ({{motif}}).",
  "pilotage.delai.valeur": "{{consomme}} j sur {{standard}} j",
"erreur.HOM_LIAISON_DUPLICATE": "Une liaison est déjà ouverte pour cette institution
et ce type de projet.",
  "erreur.HOM_INSTITUTION_NOT_ELIGIBLE": "L'institution n'est pas à l'état « en
homologation »." }
```

B. Tests unitaires (Vitest + Testing Library)

```
// src/resources/homologation/liaison.test.tsx
vi.mock("@/lib/http/client");
it("DES-08 : le bouton de dégel est absent lorsque le service ne le permet pas",
async () => {
  mockList([liaisonFactory({ etat: "GELEE", actionsPermises: [] }),
liaisonFactory({ etat: "GELEE", actionsPermises: ["DEGELER"] })]);
  renderWithShell(<ResourceList descriptor={liaisonDescriptor} />, { permissions:
["HOM_LIAISON_LIST", "HOM_LIAISON_UNFREEZE"] });
  const rows = await screen.findAllByRole("row", { name: /Établissement de paiement
A/ });
  expect(within(rows[0]).queryByRole("button", { name: "Dégeler la
liaison" })).toBeNull();
  expect(within(rows[1]).getByRole("button", { name: "Dégeler la
liaison" })).toBeVisible();
});
it("DES-08 : l'ouverture en double affiche le motif et l'identifiant de corrélation",
async () => {
  mockPostError(new ApiError(409, "HOM_LIAISON_DUPLICATE", "", "cid-dup"));
  renderWithShell(<ResourceForm descriptor={liaisonDescriptor} mode="create"
onDone={vi.fn()} />);
  await remplirOuverture({ institution: "Établissement de paiement A", type:
"Paiement mobile" });
  await userEvent.click(screen.getByRole("button", { name: "Confirmer" }));
  expect(await screen.findByText(/déjà ouverte pour cette
institution/)).toBeVisible();
  expect(screen.getByTestId("correlation-id")).toHaveTextContent("cid-dup");
});
it.each(["loading", "empty", "error", "ok"] as const)("DES-08 : la liste rend l'état
%s", async state => {
  mockListState(state, [liaisonFactory()]);
  renderWithShell(<ResourceList descriptor={liaisonDescriptor} />);
  expect(await screen.findByTestId(`datatable-${state}`)).toBeInTheDocument();
});
```

C. Storybook et accessibilité (axe)

```
// src/design-system/domain/LiaisonProgress/LiaisonProgress.stories.tsx
export default { component: LiaisonProgress, args: { dictionary: etatScope } }
satisfies Meta<typeof LiaisonProgress>;
export const Chargement: S = { args: { state: "loading", services: [] } };
export const EnCours: S = { args: { state: "ok", phaseCourante: "Branche paiement
mobile", certification: cov(4, 10), miseEnService: cov(0, 6), services:
servicesFactory() } };
export const NonCalculable: S = { args: { state: "ok", phaseCourante: "Tronc commun
amont", certification: { ...cov(0, 0), calculable: false, motifNonCalculable:
"référentiel incomplet" }, miseEnService: cov(0, 6), services: [] } };
// src/design-system/display/ProgressRing/ProgressRing.stories.tsx : Vide (null),
Partiel (40), Complet (100), Petit (size "sm")
```

`ProgressRing` expose `role="img"` et un `aria-label` portant la valeur et le libellé ; une valeur `null` affiche « non calculable » et jamais « 100 % ». Le balayage axe de F0 couvre les deux composants.

D. Tests Gherkin (Playwright BDD)

```
# language: fr
Fonctionnalité: DES-08 Liaisons et parcours (HOM-04, HOM-20)
  Scénario: Ouverture d'une liaison
    Étant donné un ingénieur d'homologation sur l'écran des liaisons
    Et l'Établissement de paiement A à l'état « en homologation »
    Quand il ouvre une liaison de type paiement mobile en désignant un chef de projet
habilité
    Alors la liaison apparaît à l'état « ouverte » avec la version en vigueur du
référentiel

  Scénario: Participant porté par une banque partenaire
    Quand il ouvre une liaison en renseignant la Banque B comme portage
    Alors le tiroir de la liaison affiche la Banque B en portage et le type de projet
du participant

  Scénario: Institution non éligible ou liaison en double
    Étant donné une liaison déjà ouverte pour l'Établissement de paiement A en
paiement mobile
    Quand il demande une seconde ouverture pour le même couple
    Alors le formulaire affiche « Une liaison est déjà ouverte pour cette institution
et ce type de projet »
    Et aucune nouvelle ligne n'apparaît dans la liste

  Scénario: Vue consolidée et opérations en retard
    Étant donné le chef du DES sur l'onglet « Pilotage »
    Quand il filtre par type de projet paiement mobile
    Alors chaque ligne affiche la phase courante, le délai consommé et le délai
standard, les couvertures et les retards
    Et l'onglet « Retards » du tiroir indique les jours de dépassement et la partie
qui en répond
```

E. Vérification via la passerelle (Playwright e2e)

```
// e2e/specs/des08-liaisons.spec.ts
test.use({ storageState: "e2e/.auth/test-ingenieur-homologation.json" });
test("DES-08 : ouverture, refus du doublon tracé, session sans jeton", async
({ page }) => {
  await page.goto("/des/liaisons");
  await ouvrirLiaison(page, { institution: "Établissement de paiement A", type:
"Paiement mobile", chef: "test-chef-des" });
  await expect(page.getByRole("row", { name: /Établissement de paiement
A.*ouverte/ })).toBeVisible();
  await ouvrirLiaison(page, { institution: "Établissement de paiement A", type:
"Paiement mobile", chef: "test-chef-des" });
  await expect(page.getByTestId("form-banner")).toContainText("déjà ouverte");
  await expect(page.getByTestId("correlation-id")).toHaveText(/[0-9a-f-]{8,}/);
  const cookies = await page.context().cookies();
  expect(cookies.every(c => !c.name.startsWith("GIMAC_SESSION") ||
c.httpOnly)).toBe(true);
});
```

DES-09 | Vue d'une liaison

Élément	Valeur
Points	74 (HOM-05 à HOM-19)
Priorité	Must Have (HOM-06, HOM-12 : Should Have)
Sprint cible	Sprint 3
Gabarit	G3 Fiche à onglets + G6 Plan de travail (onglet graphe)
Récits couverts	HOM-05, HOM-06, HOM-07, HOM-08, HOM-09, HOM-10, HOM-11, HOM-12, HOM-13, HOM-14, HOM-15, HOM-16, HOM-17, HOM-18, HOM-19

Objectif

Livrer l'écran `/des/liaisons/{id}` : fiche de la liaison avec `StateMachineBar` à sept états, onglets graphe (`DagView` et vue liste équivalente), périmètre et couvertures en lecture seule, opérations avec leurs actions, résultats du simulateur repris tels quels, preuves et chronologie, bandeau de reprise suspendue et export du dossier de preuves. L'écran est vérifié quand aucun verdict repris n'offre d'action de modification et que la vue liste restitue exactement les opérations du graphe.

A. Implémentation

A1. Onglets de la liaison

`ResourceDetail` compose l'en-tête (titre, statut, actions, champs `inDetail`), la `StateMachineBar` (états du `statusDictionary`, états parcourus `etatsParcourus`, `transitionsPermises`) et les onglets ci-dessous ; la chronologie vient de `timeline` (`GET /liaisons/{id}/historique`). Les couches du graphe sont celles du service : `DagView` ne trie rien.

```
// src/resources/homologation/liaison.tabs.tsx
import { http, P } from "@/lib/http/client";
import { DagView, LiaisonProgress, InlineAlert, KeyValueList, Hash, QueryBoundary }
from "@/design-system";
import { ResourceList, ResourceDetail } from "@/lib/resource";
import { operationDescriptor } from "./operation";
import { campagneDescriptor } from "./campagne";
import { scenarioInconnuDescriptor } from "./scenarioInconnu";
import { etatOperation, etatScope } from "./dictionaries";
import type { Liaison } from "./liaison";
import type { OperationNodeData, CoverageBarProps } from
"@/design-system/domain/types";
const H = P.desHomologation;
type Graphe = { noeuds: OperationNodeData[]; aretes: { from: string; to: string }[];
couches: string[][] };
type ServiceCouvert = CoverageBarProps["services"][number];
type ExportPreuves = { id: string; etat: string; dateExport: string;
empreinteManifeste: string };
export const liaisonQueries = {
  graphe: (id: string) => ({ queryKey: ["homologation", "liaison", id, "graphe"],
queryFn: () => http.get<Graphe>(`${H}/liaisons/${id}/graphe`) }),
  perimetre: (id: string) => ({ queryKey: ["homologation", "liaison", id,
"perimetre"], queryFn: () =>
http.get<ServiceCouvert[]>(`${H}/liaisons/${id}/couvertures`) }),
  dernierExport: (id: string) => ({ queryKey: ["homologation", "liaison", id,
"export"], queryFn: () => http.get<ExportPreuves |
null>(`${H}/liaisons/${id}/dossier-de-preuves`) }),
```

```
};
export const liaisonTabs: { key: string; label: string; render: (r: Liaison) =>
React.ReactNode }[] = [
  { key: "graphe", label: "liaison.onglet.graphe", render: r =>
    <QueryBoundary query={liaisonQueries.graphe(r.id)} isEmpty={g => g.noeuds.length
=== 0}>{g =>
      <DagView nodes={g.noeuds} edges={g.aretes} layers={g.couches}
dictionary={etatOperation} state="ok"
        readOnly={false} renderDetail={opId => <ResourceDetail
descriptor={operationDescriptor} id={opId} mode="drawer" />} />}
    </QueryBoundary> },
  { key: "perimetre", label: "liaison.onglet.perimetre", render: r => <>
    <InlineAlert tone="info" messageKey="perimetre.lectureSeule" />
    <QueryBoundary query={liaisonQueries.perimetre(r.id)}>{services =>
      <LiaisonProgress state="ok" phaseCourante={r.phaseCourante}
certification={r.couvertureCertification}
        miseEnService={r.couvertureMiseEnService} services={services}
dictionary={etatScope} />}</QueryBoundary></> },
  { key: "operations", label: "liaison.onglet.operations", render: r =>
    <ResourceList descriptor={operationDescriptor} embedded
fixedFilters={{ liaisonId: r.id }} /> },
  { key: "resultats", label: "liaison.onglet.resultats", render: r => <>
    <InlineAlert tone="info" messageKey="resultats.reprisNonRecalcules" />
    <ResourceList descriptor={campagneDescriptor} embedded fixedFilters={{ liaisonId:
r.id }} />
    <ResourceList descriptor={scenarioInconnuDescriptor} embedded
fixedFilters={{ liaisonId: r.id }} /></> },
  { key: "preuves", label: "liaison.onglet.preuves", render: r => <>
    <QueryBoundary query={liaisonQueries.dernierExport(r.id)} isEmpty={e => e ===
null} emptyKey="preuves.aucunExport">{e => e &&
      <KeyValueList items={[{ key: "preuves.etat", value: e.etat }, { key:
"preuves.date", value: e.dateExport },
        { key: "preuves.empreinte", value: <Hash value={e.empreinteManifeste}
/> }]} />}</QueryBoundary>
    <ResourceList descriptor={operationDescriptor} embedded
fixedFilters={{ liaisonId: r.id, avecPreuve: true }} /></> },
];
```

A2. Descripteur `operationDescriptor` embarqué

Les gabarits `{clé}` d'un endpoint sont résolus par les filtres fixes (listes) ou par les valeurs initiales (formulaires). Déposer et clore mènent à l'assistant DES-10, où l'effet est confirmé ou signé ; attribuer et rouvrir sont confirmés ici avec motif.

```
// src/resources/homologation/operation.tsx
export type OperationInstance = Permitted & { id: string; liaisonId: string; code:
string; libelle: string; segment: Segment;
  partie: "DES" | "PARTICIPANT"; serviceCode: string; etat: string; responsableId:
string | null; responsableNom: string | null;
  echeance: string | null; joursDepassement: number; predecesseursNonClos: string[];
aRevoir: boolean };
const O = `${P.desHomologation}/operations`;
const vers = (r: OperationInstance, etape: string) =>
`/des/liaisons/${r.liaisonId}/operations/${r.id}/cloture?etape=${etape}`;
export const operationDescriptor: ResourceDescriptorLot1<OperationInstance> = {
  key: "operation", label: { singular: "operation.singulier", plural:
"operation.pluriel", gender: "f" },
  idField: "id", titleField: "libelle", statusField: "etat",
  endpoints: { list: `${P.desHomologation}/liaisons/{liaisonId}/operations`, get: `$
{O}/{id}`, update: `${O}/{id}/responsable` },
  fields: [
    { name: "code", label: "operation.code", kind: "mono", inList: true, inDetail:
true, sortable: true },
    { name: "libelle", label: "operation.libelle", kind: "text", inList: true,
inDetail: true },
    { name: "segment", label: "operation.segment", kind: "select", inList: true,
filterable: true, options: segmentOptions },
```

```
    { name: "partie", label: "operation.partie", kind: "select", inList: true,
inDetail: true, filterable: true, options: partieOptions },
    { name: "etat", label: "operation.etat", kind: "status", inList: true, inDetail:
true, filterable: true },
    { name: "responsableId", label: "operation.responsable", kind: "searchSelect",
inForm: "edit", required: true, options: rechercheResponsables },
    { name: "responsableNom", label: "operation.responsable", kind: "text", inList:
true, inDetail: true },
    { name: "echeance", label: "operation.echeance", kind: "date", inList: true,
sortable: true },
    { name: "joursDepassement", label: "operation.retard", kind: "number", inList:
true,
      render: (_v, r) => r.joursDepassement > 0 ? <StatusBadge status="warn"
label={<Text i18nKey="operation.retard.j" values={{ n: r.joursDepassement }} />} /> :
null },
    { name: "predecesseursNonClos", label: "operation.predecesseursNonClos", kind:
"custom", inDetail: true,
      render: (_v, r) => r.predecesseursNonClos.map(c => <Tag key={c} label={c}
/>) },
  ],
  actions: [
    { key: "attribuer", label: "operation.action.attribuer", icon: "user-check",
scope: "row", permission: "HOM_OPERATION_ASSIGN",
      confirm: "confirm", reason: true, isAllowed: r =>
r.actionsPermises.includes("ATTRIBUER"), run: async ctx => ctx.openForm("edit") },
    { key: "deposer", label: "operation.action.deposer", icon: "file-up", scope:
"row", permission: "HOM_EVIDENCE_SUBMIT",
      confirm: "none", isAllowed: r => r.actionsPermises.includes("DEPOSER_PREUVE"),
run: async ({ row, navigate }) => navigate(vers(row, "preuve")) },
    { key: "cloturer", label: "operation.action.cloturer", icon: "pen-line", variant:
"primary", scope: "row", permission: "HOM_OPERATION_VALIDATE",
      confirm: "none", isAllowed: r => r.actionsPermises.includes("VALIDER"), run:
async ({ row, navigate }) => navigate(vers(row, "elements")) },
    { key: "grille", label: "operation.action.grille", icon: "list-checks", scope:
"row", permission: "HOM_GRID_EDIT",
      confirm: "confirm", isAllowed: r =>
r.actionsPermises.includes("RENSEIGNER_GRILLE"), run: async ({ openTab }) =>
openTab("grille") },
    { key: "rouvrir", label: "operation.action.rouvrir", icon: "rotate-ccw", variant:
"danger", scope: "row", permission: "HOM_OPERATION_REOPEN",
      confirm: "confirm", reason: true, isAllowed: r =>
r.actionsPermises.includes("ROUVRIR"),
      consequences: () => <InlineAlert tone="warn"
messageKey="operation.rouvrir.consequence" />,
      run: ({ row, motif, invalidate }) => http.post(`${O}/${row.id}/reouverture`,
{ body: { motif } }).then(invalidate) },
  ],
  tabs: [
    { key: "preuves", label: "operation.onglet.preuves", render: r => <QueryBoundary
query={operationQueries.preuves(r.id)}>{v =>
      <DataTable rows={v} total={v.length} page={0} pageSize={v.length} state="ok"
onChange={() => undefined} rowKey={p => p.documentRef} columns={preuveColumns}
/>}</QueryBoundary> },
    { key: "attributions", label: "operation.onglet.attributions", render: r =>
<QueryBoundary query={operationQueries.attributions(r.id)}>{a => <Timeline
items={a} />}</QueryBoundary> },
    { key: "grille", label: "operation.onglet.grille", render: r => <QueryBoundary
query={operationQueries.grille(r.id)}>{g =>
      <ControlGrid points={g.points} readOnly={!
r.actionsPermises.includes("RENSEIGNER_GRILLE")}
        onSubmit={rep => http.put(`${O}/${r.id}/grille`, { body: rep })}
/>}</QueryBoundary> },
  ],
  timeline: r => `${O}/${r.id}/historique`,
  statusDictionary: etatOperation,
  permissions: { list: "HOM_LIAISON_READ", read: "HOM_LIAISON_READ", update:
"HOM_OPERATION_ASSIGN" },
  liveTopic: "homologation",
};
```

`preuveColumns` affiche la version, le type de pièce, l'empreinte (`Hash`), la date de dépôt, le scellement et la validité : les anciennes versions restent consultables (HOM-09). `ControlGrid` (catalogue, domaine) rend chaque point de contrôle en `RadioGroup` conforme ou non conforme ; un point non conforme restitué par le service (HOM-12) est signalé sous le point.

A3. Résultats du simulateur et scénarios inconnus

`campagneDescriptor` n'a aucune action et aucun champ `inForm` : la lecture seule est structurelle, pas un masquage. `scenarioInconnuDescriptor` porte la seule action de l'onglet : qualifier (« à déclarer au référentiel » ou « hors parcours »), avec motif, par `POST /scenarios-inconnus/{id}/qualification` (`HOM_SCENARIO_QUALIFY`).

```
// src/resources/homologation/campagne.tsx
export type CampaignResult = { id: string; campagneId: string; executionId: string;
idScenario: string; operationCode: string | null;
  verdict: "FAVORABLE" | "DEFAVORABLE" | "NON_EXECUTE"; dateExecution: string;
versionScenario: string;
  referenceTrace: string; empreinteReponse: string; dateReprise: string };
export const campagneDescriptor: ResourceDescriptorLot1<CampaignResult> = {
  key: "campagne", label: { singular: "campagne.singulier", plural:
"campagne.pluriel", gender: "m" },
  idField: "id", titleField: "idScenario", statusField: "verdict",
  endpoints: { list: `${P.desHomologation}/liaisons/{liaisonId}/campagnes`, get: `$
{P.desHomologation}/liaisons/{liaisonId}/campagnes/{id}` },
  fields: [
    { name: "idScenario", label: "campagne.scenario", kind: "mono", inList: true,
filterable: true },
    { name: "operationCode", label: "campagne.operation", kind: "mono", inList:
true },
    { name: "verdict", label: "campagne.verdict", kind: "status", inList: true,
filterable: true },
    { name: "dateExecution", label: "campagne.dateExecution", kind: "datetime",
inList: true, sortable: true },
    { name: "versionScenario", label: "campagne.versionScenario", kind: "mono",
inList: true },
    { name: "referenceTrace", label: "campagne.referenceTrace", kind: "mono", inList:
true },
    { name: "empreinteReponse", label: "campagne.empreinte", kind: "custom",
inDetail: true, render: v => <Hash value={String(v)} /> },
    { name: "dateReprise", label: "campagne.dateReprise", kind: "datetime", inDetail:
true },
  ],
  actions: [],
  statusDictionary: { FAVORABLE: { label: "verdict.FAVORABLE", status: "ok" },
DEFAVORABLE: { label: "verdict.DEFAVORABLE", status: "crit" },
    NON_EXECUTE: { label: "verdict.NON_EXECUTE", status: "neutral" } },
  permissions: { list: "HOM_LIAISON_READ", read: "HOM_LIAISON_READ" },
  liveTopic: "homologation",
};
```

A4. Page et bandeau de reprise

```
// src/features/liaisons/LiaisonPage.tsx
export default function LiaisonPage() {
  const { id = "" } = useParams();
  return <><RepriseBanner /><ResourceDetail descriptor={liaisonDescriptor} id={id}
mode="page" /></>;
}
// src/features/liaisons/RepriseBanner.tsx : GET /reprise/etat, invalidé par le canal
poussé
export function RepriseBanner() {
  const { data } = useQuery({ queryKey: ["homologation", "reprise"], queryFn: () =>
http.get<{ etatDisjoncteur: string; depuis: string | null
```

```
}>(`${P.desHomologation}/reprise/etat`) });
  return data && data.etatDisjoncteur !== "FERME" ? <Banner tone="warn"
messageKey="reprise.suspendue" values={{ depuis: data.depuis }} /> : null;
}
```

```
// src/i18n/fr.json (extrait)
{ "resultats.reprisNonRecalcules": "Résultats repris du simulateur, jamais
recalculés.",
  "perimetre.lectureSeule": "Périmètre en lecture seule : toute correction passe par
les services souscrits de l'institution (MS-PARTICIPANT).",
  "reprise.suspendue": "Reprise suspendue depuis {{depuis, datetime}} : les autres
fonctions restent disponibles.",
  "couverture.nonCalculable": "Non calculable : {{motif}}",
"operation.action.rouvrir": "Rouvrir l'opération",
  "liaison.action.exporterPreuves": "Exporter le dossier de preuves" }
```

B. Tests unitaires (Vitest + Testing Library)

```
// src/features/liaisons/LiaisonPage.test.tsx
vi.mock("@/lib/http/client");
it("DES-09 : l'onglet des résultats n'offre aucune action sur un verdict repris",
async () => {
  expect(campagneDescriptor.actions).toEqual([]);
  expect(campagneDescriptor.fields.some(f => f.inForm)).toBe(false);
  mockList([campagneFactory({ verdict: "DEFAVORABLE", referenceTrace: "TRC-
0042" })]);
  renderWithShell(<ResourceList descriptor={campagneDescriptor} embedded
fixedFilters={{ liaisonId: "l1" }} />);
  const row = await screen.findByRole("row", { name: /TRC-0042/ });
  expect(within(row).queryAllByRole("button")).toHaveLength(0);
});
it("DES-09 : la vue liste restitue les opérations du graphe dans l'ordre des
couches", async () => {
  const g = grapheFactory({ couches: [["OP-01"], ["OP-02", "OP-03"]] });
  renderWithShell(<DagView nodes={g.noeuds} edges={g.aretes} layers={g.couches}
dictionary={etatOperation} state="ok" readOnly view="liste" />);
  expect(screen.getAllByRole("row").slice(1).map(r => within(r).getAllByRole("cell")
[0].textContent)).toEqual(["OP-01", "OP-02", "OP-03"]);
});
it("DES-09 : le bandeau s'affiche quand le disjoncteur de reprise est ouvert", async
() => {
  mockGet({ etatDisjoncteur: "OUVERT", depuis: "2026-03-02T08:00:00Z" });
  renderWithShell(<RepriseBanner />);
  expect(await screen.findByRole("status")).toHaveTextContent("Reprise suspendue
depuis");
});
it.each(["loading", "empty", "error", "ok"] as const)("DES-09 : DagView rend l'état
%s", state => {
  const g = grapheFactory();
  renderWithShell(<DagView nodes={g.noeuds} edges={g.aretes} layers={g.couches}
dictionary={etatOperation} state={state} readOnly />);
  expect(screen.getByTestId(`dag-${state}`)).toBeInTheDocument();
});
```

C. Storybook et accessibilité (axe)

```
// src/design-system/domain/OperationNode/OperationNode.stories.tsx
const base = operationNodeFactory({ code: "PM-07", libelle: "Vérification du code QR
statique", partie: "DES" });
export const Eligible: S = { args: { node: { ...base, etat: "ELIGIBLE" } } };
export const PreuveDeposee: S = { args: { node: { ...base, etat: "PREUVE_DEPOSEE",
elements: els("REUNI", "REUNI", "MANQUANT", "MANQUANT") } } };
export const HorodatageEnAttente: S = { args: { node: { ...base, etat:
"VALIDEE_HORODATAGE_EN_ATTENTE", elements: els("REUNI", "REUNI", "EN_ATTENTE",
```

```
"REUNI") } } };
export const Close: S = { args: { node: { ...base, etat: "CLOSE", elements:
els("REUNI", "REUNI", "REUNI", "REUNI") } } };
export const EnRetardARevoir: S = { args: { node: { ...base, etat: "ROUVERTE",
enRetard: true, aRevoir: true } } };
// DagView.stories.tsx : Chargement, Vide, Erreur (ApiError avec correlationId),
Graphe, Liste, Filtre « phase branche »
// CoverageBar.stories.tsx : Nominal, NonCalculable (« référentiel incomplet »),
ServiceHorsPerimetre
```

Chaque pictogramme des quatre éléments porte un libellé accessible (« preuve : réunie ») ; le graphe est navigable aux flèches entre couches et la vue liste est un tableau à en-têtes, conformément au DS-GIMAC-01 §11.1.

D. Tests Gherkin (Playwright BDD)

```
# language: fr
Fonctionnalité: DES-09 Vue d'une liaison (HOM-05 à HOM-19)
  Contexte:
    Étant donné un ingénieur d'homologation sur la vue de la liaison de
l'Établissement de paiement A

  Scénario: Périmètre en lecture seule
    Quand il ouvre l'onglet « Périmètre »
    Alors chaque service affiche son état et ses deux couvertures sans aucun champ
modifiable
    Et la mention « toute correction passe par les services souscrits de
l'institution » est affichée

  Scénario: Liaison gelée
    Étant donné une liaison gelée pour suspension de l'institution
    Quand il ouvre une opération éligible
    Alors les actions de dépôt et de clôture sont absentes et le motif du gel est
affiché

  Scénario: Opération verrouillée
    Quand il sélectionne une opération verrouillée dans le graphe
    Alors le tiroir liste les prédécesseurs non clos et ne propose aucun dépôt

  Scénario: Réouverture sans motif
    Quand il demande la réouverture d'une opération close sans saisir de motif
    Alors le bouton de confirmation reste inactif avec l'explication « motif
obligatoire »

  Scénario: Tentative de modification d'un verdict
    Quand il ouvre l'onglet « Résultats du simulateur »
    Alors aucun résultat ne propose d'action et la mention « repris du simulateur,
jamais recalculés » est affichée

  Scénario: Simulateur indisponible
    Étant donné le disjoncteur de reprise ouvert
    Alors le bandeau « Reprise suspendue depuis » est affiché et les autres onglets
restent utilisables

  Scénario: Scénario inconnu
    Quand il qualifie un scénario inconnu « hors parcours » avec un motif
    Alors le scénario quitte la file de qualification sans effet sur les couvertures

  Scénario: Service sans opération de référence
    Alors la couverture du service concerné affiche « Non calculable : référentiel
incomplet » et jamais « 100 % »
```

E. Vérification via la passerelle (Playwright e2e)

```
// e2e/specs/des09-liaison.spec.ts
test.use({ storageState: "e2e/.auth/test-ingenieur-homologation.json" });
test("DES-09 : graphe, vue liste, export du dossier de preuves et erreur tracée",
async ({ page }) => {
  const id = await creerLiaisonDeRecette(page.request, { institution: "Établissement
de paiement A", type: "PAIEMENT_MOBILE" });
  await page.goto(`/des/liaisons/${id}?onglet=graphe`);
  const nb = await page.getByTestId(/^op-/).count();
  await page.getByRole("radio", { name: "Vue liste" }).check();
  await expect(page.getByRole("row")).toHaveCount(nb + 1);
  await page.getByRole("button", { name: "Exporter le dossier de preuves" }).click();
  await page.getByRole("button", { name: "Confirmer" }).click();
  await expect(page.getByText("Empreinte du manifeste")).toBeVisible();
  const r = await page.request.post(`/bff/des/homologation/operations/${await
premiereOperationVerrouillee(page.request, id)}/preuves`, { multipart: pieceFactice()
});
  expect(r.status()).toBe(409); expect((await r.json()).correlationId).toMatch(/[0-
9a-f-]{8,}/);
});
```

DES-10 | Clore une opération

Élément	Valeur
Points	13 (HOM-09 : 5, HOM-10 : 8)
Priorité	Must Have
Sprint cible	Sprint 3
Gabarit	G4 Assistant en étapes (`ResourceForm` en mode étapes)
Récits couverts	HOM-09, HOM-10

Objectif

Livrer l'assistant `/des/liaisons/{id}/operations/{op}/cloture` en trois étapes : dépôt de la preuve via MS-DOCUMENT avec le résultat de la chaîne de contrôle, état des quatre éléments avec l'action manquante nommée, puis validation signée après réauthentification FIDO2. L'écran est vérifié quand le responsable et le déposant ne voient aucun bouton de validation mais le rappel de la règle, et que l'assertion part dans `X-Reauth-Assertion`.

A. Implémentation

A1. Type et descripteur `clotureOperationDescriptor`

La ressource lue est `GET /operations/{id}/elements-de-cloture` : le service y renvoie l'état des quatre éléments, l'empreinte à valider, les actions permises et, le cas échéant, la restriction qui interdit la validation à l'utilisateur courant. L'interface n'évalue jamais la distinction entre validateur, responsable et déposant : elle affiche ce que le service lui dit.

```
// src/resources/homologation/cloture.tsx
import { http, P } from "@/lib/http/client";
import type { ResourceDescriptorLot1, Permitted } from "@/lib/resource/types";
import type { FourElementsChecklistProps } from "@/design-system/domain/types";
import { FourElementsChecklist, MaskingResult, InlineAlert, Hash } from "@/design-
system";
import { etatOperation } from "./dictionaries";
export type DepotResultat = { documentRef: string; empreinte: string; version:
number; scelle: boolean;
  controle: { resultat: "ACCEPTEE" | "MASQUEE" | "REFUSEE"; motif?: string;
sequencesMasquees?: number } };
export type ClotureOperation = Permitted & { id: string; liaisonId: string; code:
string; libelle: string; etat: string;
  preuveAttendue: string; preuve: File | null; dernierDepot: DepotResultat | null;
  elements: FourElementsChecklistProps["elements"]; empreinteAValider: string | null;
  restrictionValidation: "EST_RESPONSABLE" | "EST_DEPOSANT" | null };
const O = `${P.desHomologation}/operations`;

export const clotureOperationDescriptor: ResourceDescriptorLot1<ClotureOperation> = {
  key: "clotureOperation", label: { singular: "cloture.singulier", plural:
"cloture.pluriel", gender: "f" },
  idField: "id", titleField: "libelle", statusField: "etat",
  endpoints: { list: `${O}`, get: `${O}/{id}/elements-de-cloture` },
  steps: [
    { key: "preuve", labelKey: "cloture.etape.preuve", fields: ["preuveAttendue",
"preuve", "dernierDepot"], action: "deposer" },
    { key: "elements", labelKey: "cloture.etape.elements", fields: ["elements"] },
    { key: "signature", labelKey: "cloture.etape.signature", fields: ["etat",
"restrictionValidation"], action: "valider" },
  ],
  fields: [
```

```
    { name: "preuveAttendue", label: "cloture.preuveAttendue", kind: "text", inForm:
"edit", readOnly: true },
    { name: "preuve", label: "cloture.fichier", kind: "file", inForm: "edit", help:
"cloture.fichier.aide" }, // FileDropzone
    { name: "dernierDepot", label: "cloture.dernierDepot", kind: "custom", inForm:
"edit", readOnly: true,
      render: (_v, r) => r.dernierDepot && <>
        <MaskingResult result={r.dernierDepot.controle.resultat}
motif={r.dernierDepot.controle.motif}
          count={r.dernierDepot.controle.sequencesMasquees} />
        <Hash value={r.dernierDepot.empreinte} /></> },
    { name: "elements", label: "cloture.quatreElements", kind: "custom", inForm:
"edit", readOnly: true,
      render: (_v, r) => <FourElementsChecklist elements={r.elements} state="ok"
/> },
    { name: "etat", label: "operation.etat", kind: "status", inForm: "edit",
readOnly: true,
      render: (_v, r) => r.etat === "VALIDEE_HORODATAGE_EN_ATTENTE" ? <InlineAlert
tone="info" messageKey="cloture.horodatageEnAttente" /> : null },
    { name: "restrictionValidation", label: "cloture.regle", kind: "custom", inForm:
"edit", readOnly: true,
      render: v => v ? <InlineAlert tone="info" messageKey={`cloture.restriction.$
{String(v)}`} /> : null },
  ],
  actions: [
    { key: "deposer", label: "cloture.action.deposer", icon: "file-up", variant:
"primary", scope: "page",
      permission: "HOM_EVIDENCE_SUBMIT", confirm: "confirm", isAllowed: r =>
r.actionsPermises.includes("DEPOSER_PREUVE"),
      run: async ({ row, invalidate }) => {
        if (!row.preuve) return;                              // le format est
contrôlé par zod ; le contenu par MS-DOCUMENT
        const body = new FormData(); body.set("fichier", row.preuve);
body.set("typePiece", row.preuveAttendue);
        await http.post<DepotResultat>(`${O}/${row.id}/preuves`, { body }); // 422 :
refus explicite sous le champ « preuve »
        invalidate();
      } },
    { key: "valider", label: "cloture.action.valider", icon: "pen-line", variant:
"primary", scope: "page",
      permission: "HOM_OPERATION_VALIDATE", confirm: "sign", reason: true, isAllowed:
r => r.actionsPermises.includes("VALIDER"),
      sign: { purpose: "OPERATION_VALIDATE", empreinte: r => r.empreinteAValider ??
"" },
      run: ({ row, motif, assertion, invalidate }) =>
http.post(`${O}/${row.id}/validation`,
        { body: { motif, empreinte: row.empreinteAValider }, reauthAssertion:
assertion }).then(invalidate) },
  ],
  statusDictionary: etatOperation,
  permissions: { list: "HOM_LIAISON_READ", read: "HOM_LIAISON_READ", update:
"HOM_EVIDENCE_SUBMIT" },
  liveTopic: "homologation",
};
```

A2. Composant de domaine `FourElementsChecklist`

```
// src/design-system/domain/FourElementsChecklist/FourElementsChecklist.tsx
import css from "./FourElementsChecklist.module.css";
const TONE = { REUNI: "ok", MANQUANT: "crit", EN_ATTENTE: "warn" } as const;
const ICON = { responsable: "user-check", preuve: "file-check", horodatage: "clock",
validation: "pen-line" } as const;
export function FourElementsChecklist({ elements, state, error }:
FourElementsChecklistProps) {
  const { t } = useTranslation();
  if (state !== "ok") return <StateSlot state={state} error={error} testId="four-
elements" />;
  return (
```

```
    <ol className={css.list} aria-label={t("cloture.quatreElements")} data-
testid="four-elements-ok">
      {elements.map(e => (
        <li key={e.key} className={css.item} data-etat={e.etat}>
          <StatusBadge status={TONE[e.etat]} icon={ICON[e.key]} label={t(`element.$
{e.key}.${e.etat}`)} />
          {e.detail && <span className={css.detail}>{e.detail}</span>}
          {e.etat !== "REUNI" && e.actionManquante && (e.actionManquante.href
            ? <Link
href={e.actionManquante.href}>{t(e.actionManquante.labelKey)}</Link>
            : <span className={css.action}>{t(e.actionManquante.labelKey)}</span>)}
        </li>))}
    </ol>);
}
```

Le module CSS n'emploie que `--gm-space-*`, `--gm-color-border` et `--gm-color-surface` ; l'état n'est jamais porté par la seule couleur (pictogramme et libellé dans `StatusBadge`).

A3. Page, étapes et `SignDialog`

La page se réduit à l'appel du composant universel ; l'étape courante est dans l'URL (`?etape=preuve|elements|signature`). À l'étape 3, `ResourceForm` ouvre `SignDialog` avec le résumé (code, libellé, empreinte de la preuve), l'empreinte à valider, le motif et le bloc `SignatureField` ; il appelle `getReauthAssertion("des", "OPERATION_VALIDATE")` puis l'action `valider`. Un échec ou une annulation de la clé n'envoie aucune requête de validation. Une réponse 202 laisse l'opération « validée, horodatage en attente » : le service réessaie seul et le canal poussé fait passer l'écran à « close ».

```
// src/features/liaisons/ClotureOperationPage.tsx
export default function ClotureOperationPage() {
  const { id = "", op = "" } = useParams();
  const navigate = useNavigate();
  return <ResourceForm descriptor={clotureOperationDescriptor} mode="edit" id={op}
    onDone={() => navigate(`/des/liaisons/${id}?onglet=operations&detail=${op}`)} />;
}
```

```
// src/i18n/fr.json (extrait)
{ "cloture.etape.preuve": "Preuve", "cloture.etape.elements": "Quatre éléments",
"cloture.etape.signature": "Validation",
  "cloture.action.valider": "Valider et clore l'opération", "cloture.action.deposer":
"Déposer la preuve",
  "cloture.restriction.EST_RESPONSABLE": "Vous êtes responsable de cette opération :
la validation revient à un tiers distinct du responsable et du déposant.",
  "cloture.restriction.EST_DEPOSANT": "Vous avez déposé la preuve : la validation
revient à un tiers distinct du responsable et du déposant.",
  "cloture.horodatageEnAttente": "Validation enregistrée ; horodatage en attente,
réessayé automatiquement sans nouvelle validation.",
  "element.preuve.MANQUANT": "preuve : manquante", "element.horodatage.EN_ATTENTE":
"horodatage : en attente",
  "erreur.HOM_EVIDENCE_REJECTED": "Pièce refusée par la chaîne de contrôle :
{{motif}}. Aucune référence n'a été rattachée." }
```

B. Tests unitaires (Vitest + Testing Library)

```
// src/resources/homologation/cloture.test.tsx
vi.mock("@/lib/http/client");
vi.mock("@/lib/reauth/webauthn", () => ({ getReauthAssertion: vi.fn(async () =>
"YXNzZXJ0aW9u") }));
const ouvrir = (etape: string, c: Partial<ClotureOperation>) => {
  mockGet(clotureFactory(c));
  return renderWithShell(<ResourceForm descriptor={clotureOperationDescriptor}
mode="edit" id="op1" onDone={vi.fn()} />,
```

```
    { route: `/des/liaisons/l1/operations/op1/cloture?etape=${etape}` });
};
it("DES-10 : le bouton de validation est absent pour le responsable", async () => {
  ouvrir("signature", { actionsPermises: [], restrictionValidation: "EST_RESPONSABLE"
});
  expect(await screen.findByText(/validation revient à un tiers
distinct/)).toBeVisible();
  expect(screen.queryByRole("button", { name: "Valider et clore
l'opération" })).toBeNull();
});
it("DES-10 : l'élément manquant est nommé avec l'action à mener", async () => {
  ouvrir("elements", { elements: [el("responsable", "REUNI"), el("preuve",
"MANQUANT", "cloture.action.deposer"),
    el("horodatage", "MANQUANT"), el("validation", "MANQUANT")] });
  const item = (await screen.findByText("preuve : manquante")).closest("li");
  expect(item).toHaveTextContent("Déposer la preuve");
});
it("DES-10 : une pièce refusée affiche le motif sans référence rattachée", async ()
=> {
  mockPostError(new ApiError(422, "HOM_EVIDENCE_REJECTED", "", "cid-422", { champ:
"preuve", motif: "numéro de carte détecté" }));
  ouvrir("preuve", { actionsPermises: ["DEPOSER_PREUVE"], dernierDepot: null });
  await deposerFichier("rapport-parametrage.pdf"); await confirmer();
  expect(await screen.findByText(/numéro de carte détecté/)).toBeVisible();
  expect(screen.queryByTestId("hash")).toBeNull();
});
it("DES-10 : la validation transmet l'assertion dans X-Reauth-Assertion", async () =>
{
  ouvrir("signature", { actionsPermises: ["VALIDER"], empreinteAValider:
"ab".repeat(32) });
  await userEvent.click(await screen.findByRole("button", { name: "Valider et clore
l'opération" }));
  await userEvent.type(screen.getByLabelText("Motif"), "Paramétrage conforme");
  await userEvent.click(screen.getByRole("button", { name: "Signer" }));
  expect(http.post).toHaveBeenCalledWith("/bff/des/homologation/operations/op1/
validation",
    expect.objectContaining({ reauthAssertion: "YXNzZXJ0aW9u" }));
});
```

C. Storybook et accessibilité (axe)

```
// src/design-system/domain/FourElementsChecklist/FourElementsChecklist.stories.tsx
export const Chargement: S = { args: { state: "loading", elements: [] } };
export const Erreur: S = { args: { state: "error", elements: [], error: new
ApiError(503, "HOM_TSA_UNAVAILABLE", "", "cid-503") } };
export const PreuveManquante: S = { args: { state: "ok", elements: quatre("REUNI",
"MANQUANT", "MANQUANT", "MANQUANT") } };
export const HorodatageEnAttente: S = { args: { state: "ok", elements:
quatre("REUNI", "REUNI", "EN_ATTENTE", "REUNI") } };
export const Reunis: S = { args: { state: "ok", elements: quatre("REUNI", "REUNI",
"REUNI", "REUNI") } };
// src/design-system/domain/SignDialog/SignDialog.stories.tsx : Repos, MotifManquant,
ReauthEnCours, ReauthEchouee, Signee
```

`SignDialog` est testé au clavier (piège de focus, `Échap`, retour au déclencheur) ; l'empreinte est lisible par lecteur d'écran en entier malgré l'affichage tronqué de `Hash`.

D. Tests Gherkin (Playwright BDD)

```
# language: fr
Fonctionnalité: DES-10 Clore une opération (HOM-09, HOM-10)
  Scénario: Dépôt accepté
    Étant donné un ingénieur d'homologation sur l'étape « Preuve » d'une opération
éligible attendant un rapport de paramétrage
    Quand il dépose la pièce du type attendu et confirme
```

```
    Alors le résultat « acceptée » et l'empreinte de la pièce sont affichés
    Et l'opération passe à l'état « preuve déposée »

  Scénario: Nouvelle version de la preuve
    Quand il dépose une nouvelle version de la pièce
    Alors l'onglet des preuves liste les deux versions et l'élément « validation »
redevient manquant

  Scénario: Pièce rejetée par la chaîne de contrôle
    Quand il dépose une pièce contenant un numéro de carte
    Alors le motif du rejet est affiché sous le champ et aucune empreinte n'est
rattachée

  Scénario: Clôture avec les quatre éléments
    Étant donné un validateur distinct du responsable et du déposant sur l'étape «
Validation »
    Quand il signe avec un motif et sa clé de sécurité
    Alors l'opération passe à l'état « close » et ses successeurs deviennent
éligibles dans le graphe

  Scénario: Validateur non distinct
    Étant donné le responsable de l'opération sur l'étape « Validation »
    Alors le bouton « Valider et clore l'opération » est absent
    Et le rappel « la validation revient à un tiers distinct du responsable et du
déposant » est affiché

  Scénario: Élément manquant
    Étant donné une opération sans preuve scellée
    Alors l'étape « Quatre éléments » nomme la preuve manquante et propose « Déposer
la preuve »

  Scénario: Autorité d'horodatage indisponible
    Étant donné l'adaptateur de substitution de l'autorité d'horodatage configuré en
indisponibilité
    Quand le validateur signe
    Alors l'écran affiche « validée, horodatage en attente » sans demander de
nouvelle validation
```

E. Vérification via la passerelle (Playwright e2e)

```
// e2e/specs/des10-cloture.spec.ts
import { ajouterCleVirtuelle } from "../support/webauthn"; // CDP
WebAuthn.addVirtualAuthenticator (ctap2, vérification utilisateur)
test("DES-10 : dépôt par le responsable, validation par un tiers distinct", async
({ browser }) => {
  const ing = await browser.newContext({ storageState: "e2e/.auth/test-ingenieur-
homologation.json" });
  const p1 = await ing.newPage(); const { liaisonId, opId } = await
operationEligibleDeRecette(p1.request);
  await p1.goto(`/des/liaisons/${liaisonId}/operations/${opId}/cloture?
etape=preuve`);
  await p1.getByLabel("Fichier").setInputFiles("e2e/fixtures/rapport-
parametrage.pdf");
  await p1.getByRole("button", { name: "Déposer la preuve" }).click(); await
p1.getByRole("button", { name: "Confirmer" }).click();
  await expect(p1.getByTestId("hash")).toBeVisible();
  await p1.goto(`/des/liaisons/${liaisonId}/operations/${opId}/cloture?
etape=signature`);
  await expect(p1.getByRole("button", { name: "Valider et clore
l'opération" })).toHaveCount(0);

  const chef = await browser.newContext({ storageState: "e2e/.auth/test-chef-
des.json" });
  const p2 = await chef.newPage(); await ajouterCleVirtuelle(p2);
  await p2.goto(`/des/liaisons/${liaisonId}/operations/${opId}/cloture?
etape=signature`);
  await p2.getByRole("button", { name: "Valider et clore l'opération" }).click();
```

```
  await p2.getByLabel("Motif").fill("Rapport conforme au paramétrage attendu");
  const req = p2.waitForRequest(r =>
r.url().endsWith(`/operations/${opId}/validation`));
  await p2.getByRole("button", { name: "Signer" }).click();
  const h = (await req).headers();
  expect(h["x-reauth-assertion"]).toBeTruthy();
expect(h["authorization"]).toBeUndefined();
  await expect(p2.getByTestId("status-badge")).toHaveText(/close|validée, horodatage
en attente/);
});
```

```
// e2e/support/webauthn.ts
import type { Page } from "@playwright/test";
/** Clé FIDO2 virtuelle ; la clé de recette est enrôlée par e2e/setup/enrolement.ts
(IAM-07). */
export async function ajouterCleVirtuelle(page: Page): Promise<void> {
  const cdp = await page.context().newCDPSession(page);
  await cdp.send("WebAuthn.enable");
  await cdp.send("WebAuthn.addVirtualAuthenticator", { options: { protocol: "ctap2",
transport: "usb",
    hasResidentKey: true, hasUserVerification: true, isUserVerified: true } });
}
```

DES-11 | Dossier de mise en service

Élément	Valeur
Points	73 (CER-01 à CER-12, CER-14, CER-15)
Priorité	Must Have (CER-14, CER-15 : Should Have)
Sprint cible	Sprint 4
Gabarit	G3 Fiche à onglets (`ResourceDetail`)
Récits couverts	CER-01, CER-02, CER-03, CER-04, CER-05, CER-06, CER-07, CER-08, CER-09, CER-10, CER-11, CER-12, CER-14, CER-15

Objectif

Livrer l'écran `/des/liaisons/{id}/mise-en-service` : circuit des quatre quitus dans l'ordre imposé avec signature réauthentifiée et refus motivé, six conditions de complétude vérifiables, fiche de changement lue (jamais saisie), procès-verbal, horodatages, attestation, registre et demandes de statut à quatre yeux. L'écran est vérifié quand le demandeur d'un changement de statut ne voit pas le bouton d'approbation et qu'un refus sans motif ne peut pas être signé.

A. Implémentation

A1. Types et descripteur `dossierMiseEnServiceDescriptor`

```
// src/resources/certification/dossierMiseEnService.tsx
import { http, P } from "@/lib/http/client";
import type { ResourceDescriptorLot1, Permitted } from "@/lib/resource/types";
import type { QuitusView, QuitusCircuitProps } from "@/design-system/domain/types";
import { QuitusCircuit, QueryBoundary, DataTable, KeyValueList, Hash, InlineAlert,
StatusBadge } from "@/design-system";
import { ResourceList, ResourceForm } from "@/lib/resource";
import { etatDossier, etatQuitus } from "./dictionaries";
import { attestationPanel } from "./attestation";
import { registreDescriptor } from "./registre";
import { procesVerbalDescriptor } from "./procesVerbal";
export type ConditionCode = "COUVERTURE_CERTIFICATION" | "COUVERTURE_MISE_EN_SERVICE"
| "QUATRE_QUITUS"
  | "FICHE_CHANGEMENT_APPROUVEE" | "PROCES_VERBAL" | "EMPREINTES_VERIFIEES";
export type Dossier = Permitted & { id: string; liaisonId: string; circuitId: string;
institutionNom: string; cycle: number;
  etat: "OUVERT" | "AUTORISE" | "MIS_EN_SERVICE" | "COMPLET" | "ATTESTE";
etatsParcourus: string[];
  etatScellement: "EN_ATTENTE" | "SCELLE" | "REJETE" | null; socleEmpreinte: string;
dateOuverture: string;
  dateAutorisation: string | null; referenceFicheChangement: string | null;
numeroAttestation: string | null };
type Condition = Permitted & { code: ConditionCode; satisfaite: boolean; source:
string; detail: string | null; dateVerification: string | null };
type FicheLue = { reference: string; etatApprobation: string; fenetreDebut: string;
fenetreFin: string; dateLecture: string; aRelire: boolean };
type Circuit = { id: string; quitus: QuitusView[]; socle: QuitusCircuitProps["socle"]
};
type Horodatage = { id: string; objetType: "QUITUS" | "DOSSIER"; empreinte: string;
dateJeton: string; verifie: boolean; autoriteChaine: string };
const C = P.desCertification;
const NATURE = { TECHNIQUE: "technique", SECURITE: "securite", CONFORMITE:
"conformite", DIRECTION_GENERALE: "direction-generale" } as const;
export const certQueries = {
  circuit: (id: string) => ({ queryKey: ["certification", "circuit", id], queryFn: ()
=> http.get<Circuit>(`${C}/circuits/${id}`) }),
```

```
  conditions: (id: string) => ({ queryKey: ["certification", "dossier", id,
"conditions"], queryFn: () => http.get<Condition[]>(`${C}/dossiers/${id}/conditions`)
}),
  fiche: (id: string) => ({ queryKey: ["certification", "dossier", id, "fiche"],
queryFn: () => http.get<FicheLue | null>(`${C}/dossiers/${id}/fiche-de-
changement`) }),
  horodatages: (id: string) => ({ queryKey: ["certification", "dossier", id,
"horodatages"], queryFn: () =>
http.get<Horodatage[]>(`${C}/dossiers/${id}/horodatages`) }),
};

export const dossierMiseEnServiceDescriptor: ResourceDescriptorLot1<Dossier> = {
  key: "dossierMiseEnService", label: { singular: "dossier.singulier", plural:
"dossier.pluriel", gender: "m" },
  idField: "id", titleField: "institutionNom", statusField: "etat",
  endpoints: { list: `${C}/dossiers`, get: `${C}/dossiers/{id}`, update:
`${C}/dossiers/{id}/fiche-de-changement` },
  fields: [
    { name: "institutionNom", label: "dossier.institution", kind: "text", inDetail:
true },
    { name: "etat", label: "dossier.etat", kind: "status", inDetail: true },
    { name: "cycle", label: "dossier.cycle", kind: "number", inDetail: true },
    { name: "socleEmpreinte", label: "dossier.socle", kind: "custom", inDetail: true,
render: v => <Hash value={String(v)} /> },
    { name: "etatScellement", label: "dossier.scellement", kind: "status", inDetail:
true },
    { name: "referenceFicheChangement", label: "fiche.reference", kind: "mono",
inDetail: true, inForm: "edit", required: true,
      help: "fiche.reference.aide" },                                  // seule
saisie : la référence, jamais le contenu
  ],
  actions: [
    { key: "verifier", label: "dossier.action.verifier", icon: "refresh-cw", scope:
"row", permission: "CER_DOSSIER_VERIFY",
      confirm: "none", isAllowed: r => r.actionsPermises.includes("VERIFIER"),
      run: ({ row, invalidate }) => http.post(`${C}/dossiers/${row.id}/verification`,
{ body: {} }).then(invalidate) },
    { key: "lierFiche", label: "fiche.action.lier", icon: "link", scope: "row",
permission: "CER_CHANGE_RECORD_LINK",
      confirm: "confirm", isAllowed: r => r.actionsPermises.includes("LIER_FICHE"),
run: async ctx => ctx.openForm("edit") },
    { key: "relireFiche", label: "fiche.action.relire", icon: "rotate-cw", scope:
"row", permission: "CER_CHANGE_RECORD_LINK",
      confirm: "none", isAllowed: r => r.actionsPermises.includes("RELIRE_FICHE"),
      run: ({ row, invalidate }) => http.put(`${C}/dossiers/${row.id}/fiche-de-
changement`,
        { body: { reference:
row.referenceFicheChangement } }).then(invalidate) },   // relecture : lecture seule
du dispositif
    { key: "emettreAttestation", label: "attestation.action.emettre", icon: "award",
variant: "primary", scope: "row",
      permission: "CER_ATTESTATION_ISSUE", confirm: "confirm", isAllowed: r =>
r.actionsPermises.includes("EMETTRE_ATTESTATION"),
      run: ({ row, invalidate }) =>
http.post(`${C}/dossiers/${row.id}/attestation`).then(invalidate) },
  ],
  tabs: [
    { key: "circuit", label: "dossier.onglet.circuit", render: r => <QueryBoundary
query={certQueries.circuit(r.circuitId)}>{c =>
      <QuitusCircuit quitus={c.quitus} dictionary={etatQuitus} state="ok"
socle={c.socle}
        onSubmit={(q, d) => http.post(`${C}/circuits/${c.id}/quitus/$
{NATURE[q.nature]}`, {
          body: { decisionFavorable: d.decisionFavorable, motif: d.motif,
operationsDesignees: d.operationsDesignees,
                  servicesDesignes: d.servicesDesignes, socleEmpreinte:
c.socle.empreinte }, reauthAssertion: d.assertion })} />}
    </QueryBoundary> },
    { key: "completude", label: "dossier.onglet.completude", render: r =>
```

```
<QueryBoundary query={certQueries.conditions(r.id)}>{cs =>
      <DataTable rows={cs} total={cs.length} page={0} pageSize={cs.length} state="ok"
onChange={() => undefined} rowKey={c => c.code}
        columns={conditionColumns} rowActions={c =>
c.actionsPermises.includes("VERIFIER") ? [{ key: "verifier", labelKey:
"condition.verifier",
          run: () => http.post(`${C}/dossiers/${r.id}/verification`, { body:
{ condition: c.code } }) }] : []} />}</QueryBoundary> },
    { key: "fiche", label: "dossier.onglet.fiche", render: r => <QueryBoundary
query={certQueries.fiche(r.id)}
      isEmpty={f => f === null} emptyKey="fiche.aucune">{f => f && <>
      {f.aRelire && <InlineAlert tone="warn" messageKey="fiche.aRelire" />}
      <KeyValueList items={[{ key: "fiche.reference", value: f.reference }, { key:
"fiche.etat", value: f.etatApprobation },
        { key: "fiche.fenetre", value: `${f.fenetreDebut} / ${f.fenetreFin}` },
{ key: "fiche.dateLecture", value: f.dateLecture }]} /></>}
    </QueryBoundary> },
    { key: "procesVerbal", label: "dossier.onglet.procesVerbal", render: r =>
      <ResourceForm descriptor={procesVerbalDescriptor} mode="edit" id={r.id}
onDone={() => undefined} /> },
    { key: "horodatages", label: "dossier.onglet.horodatages", render: r =>
<QueryBoundary query={certQueries.horodatages(r.id)}>{hs =>
      <DataTable rows={hs} total={hs.length} page={0} pageSize={hs.length} state="ok"
onChange={() => undefined}
        rowKey={h => h.id} columns={horodatageColumns} />}</QueryBoundary> },
    { key: "attestation", label: "dossier.onglet.attestation", render: r =>
r.numeroAttestation ? attestationPanel(r.numeroAttestation) : null },
    { key: "registre", label: "dossier.onglet.registre", render: () => <ResourceList
descriptor={registreDescriptor} embedded /> },
  ],
  timeline: r => `${C}/dossiers/${r.id}/historique`,
  statusDictionary: etatDossier,
  permissions: { list: "CER_DOSSIER_READ", read: "CER_DOSSIER_READ", update:
"CER_CHANGE_RECORD_LINK" },
  liveTopic: "certification",
};
```

`conditionColumns` : condition (libellé i18n `condition.<code>`), état (`StatusBadge` satisfaite ou non), source, détail, date de dernière vérification ; `horodatageColumns` : objet, empreinte (`Hash`), date du jeton, vérification (`StatusBadge`), chaîne de l'autorité. `procesVerbalDescriptor` porte le fichier (`file`), l'heure de bascule (`datetime`) et l'action `deposer` (`CER_GOLIVE_REPORT_SUBMIT`, `confirm: "confirm"`, `POST /dossiers/{id}/proces-verbal`) ; une fois déposé, il affiche l'empreinte, l'heure de bascule et la mention « hors fenêtre » (`StatusBadge` à surveiller) tant que la régularisation n'est pas constatée par le service.

A2. `QuitusCircuit` : ordre imposé et décision signée

Le composant range les quitus en trois colonnes (technique ; sécurité et conformité ; Direction Générale). Le bouton « Signer » d'un quitus n'existe que si `actionsPermises` contient `SIGNER` : le service y a déjà appliqué l'ordre, la fonction du signataire et l'impartialité (CER-03, CER-07). La décision défavorable exige un motif et au moins une désignation parmi les opérations ou services du socle.

```
// src/design-system/domain/QuitusCircuit/QuitusCircuit.tsx
const COLONNES: QuitusView["nature"][][] = [["TECHNIQUE"], ["SECURITE",
"CONFORMITE"], ["DIRECTION_GENERALE"]];
export function QuitusCircuit({ quitus, dictionary, state, socle, onSubmit }:
QuitusCircuitProps) {
  const { t } = useTranslation();
  const [cible, setCible] = useState<QuitusView | null>(null);
  const [favorable, setFavorable] = useState(true);
  const [designees, setDesignees] = useState<string[]>([]);
```

```
  if (state !== "ok") return <StateSlot state={state} testId="quitus-circuit" />;
  const parNature = new Map(quitus.map(q => [q.nature, q]));
  return (<>
    <ol className={css.circuit} aria-label={t("quitus.circuit")}>
      {COLONNES.map((col, i) => <li key={i} className={css.etape}>{col.map(n =>
parNature.get(n)).filter(Boolean).map(q => q &&
        <Card key={q.nature} titleKey={`quitus.nature.${q.nature}`} data-
testid={`quitus-${q.nature}`}>
          <StatusBadge status={dictionary[q.etat].status}
label={t(dictionary[q.etat].label)} />
          <KeyValueList items={[{ key: "quitus.signataire", value: t(`role.$
{q.signataireRole}`) },
            { key: "quitus.horodatage", value: q.dateHorodatage ??
t("commun.nonDisponible") }, { key: "quitus.motif", value: q.motif ?? "" }]} />
          {q.actionsPermises.includes("SIGNER") && <Button icon="pen-line"
onClick={() => setCible(q)}>{t("quitus.action.signer")}</Button>}
        </Card>)}</li>)}
    </ol>
    {cible && <SignDialog open layer="des" purpose="QUITUS_SIGN"
titleKey={`quitus.signer.${cible.nature}`} empreinte={socle.empreinte}
      reasonRequired summary={[{ labelKey: "quitus.nature", value: t(`quitus.nature.$
{cible.nature}`) }]}
      extra={<>
        <RadioGroup name="decision" labelKey="quitus.decision" value={favorable ? "F"
: "D"} onChange={v => setFavorable(v === "F")}
          options={[{ value: "F", label: t("quitus.favorable") }, { value: "D",
label: t("quitus.defavorable") }]} />
        {!favorable && <MultiSelect name="designations"
labelKey="quitus.designations" required value={designees} onChange={setDesignees}
          options={[...socle.operations.map(o => ({ value: `OP:${o.code}`, label: `$
{o.code} ${o.libelle}` })),
                    ...socle.services.map(s => ({ value: `SV:${s}`, label: s }))]}
/>}</>}
      onClose={() => setCible(null)}
      onSign={({ assertion, motif }) => onSubmit(cible, { decisionFavorable:
favorable, motif, assertion,
        operationsDesignees: designees.filter(d => d.startsWith("OP:")).map(d =>
d.slice(3)),
        servicesDesignes: designees.filter(d => d.startsWith("SV:")).map(d =>
d.slice(3)) }).then(() => setCible(null))} />}
  </>);
}
```

Une réponse 409 `CER_SOCLE_PERIME` (CER-06) est affichée dans `SignDialog` avec l'identifiant de corrélation ; l'invalidation par le canal poussé présente le socle à jour avant toute nouvelle signature.

A3. Attestation, registre et demandes de statut

```
// src/resources/certification/attestation.tsx
export type Attestation = Permitted & { numero: string; institutionNom: string;
typeProjet: string; servicesCertifies: string[];
  dateEmission: string; socleEmpreinte: string; empreinteDocument: string; etat:
string;
  demandeEnAttente: { id: string; typeDemande: "REVOCATION" | "SUSPENSION" | "LEVEE";
motif: string; empreinte: string; demandeeParMoi: boolean } | null;
  typeDemande: "REVOCATION" | "SUSPENSION" | "LEVEE" | null; piece: File | null;
conditionLevee: string | null };
const A = `${P.desCertification}/attestations`;
export const attestationDescriptor: ResourceDescriptorLot1<Attestation> = {
  key: "attestation", label: { singular: "attestation.singulier", plural:
"attestation.pluriel", gender: "f" },
  idField: "numero", titleField: "numero", statusField: "etat", endpoints: { list: `$
{P.desCertification}/registre`, get: `${A}/{id}` },
  steps: [{ key: "demande", labelKey: "statut.demande", fields: ["typeDemande",
"piece", "conditionLevee"], action: "soumettreDemande" }],
  fields: [
```

```
    { name: "typeDemande", label: "statut.type", kind: "radio", inForm: "edit",
required: true,
      options: ["REVOCATION", "SUSPENSION", "LEVEE"].map(v => ({ value: v, label:
`statut.${v}` })) },
    { name: "piece", label: "statut.piece", kind: "file", inForm: "edit", required:
true },
    { name: "conditionLevee", label: "statut.conditionLevee", kind: "textarea",
inForm: "edit" },
    { name: "demandeEnAttente", label: "statut.enAttente", kind: "custom", inDetail:
true, render: (_v, r) => r.demandeEnAttente?.demandeeParMoi
      ? <InlineAlert tone="info" messageKey="statut.quatreYeux" /> : null },
  ],
  actions: [
    { key: "telecharger", label: "attestation.action.telecharger", icon: "download",
scope: "row", permission: "CER_DOSSIER_READ",
      confirm: "none", isAllowed: r => r.actionsPermises.includes("TELECHARGER"),
      run: async ({ row }) => { const f = await http.blob(`${A}/${row.numero}`,
"application/pdf");
        enregistrer(f.blob, `${row.numero}.pdf`, f.empreinte ===
row.empreinteDocument); } },  // journalisé côté service
    { key: "demanderStatut", label: "statut.action.demander", icon: "flag", scope:
"row", permission: "CER_ATTESTATION_STATUS_REQUEST",
      confirm: "none", isAllowed: r => r.actionsPermises.includes("DEMANDER_STATUT"),
run: async ctx => ctx.openForm("edit") },
    { key: "soumettreDemande", label: "statut.action.soumettre", scope: "page",
permission: "CER_ATTESTATION_STATUS_REQUEST",
      confirm: "confirm", reason: true, run: async ({ row, motif, invalidate }) => {
        const body = new FormData(); body.set("typeDemande", row.typeDemande ?? "");
body.set("motif", motif ?? "");
        if (row.piece) body.set("piece", row.piece); if (row.conditionLevee)
body.set("conditionLevee", row.conditionLevee);
        await http.post(`${A}/${row.numero}/demandes-de-statut`, { body });
invalidate(); } },
    { key: "approuver", label: "statut.action.approuver", icon: "pen-line", variant:
"primary", scope: "row",
      permission: "CER_ATTESTATION_STATUS_APPROVE", confirm: "sign", reason: true,
isAllowed: r => r.actionsPermises.includes("APPROUVER_STATUT"),
      sign: { purpose: "STATUS_APPROVE", empreinte: r =>
r.demandeEnAttente?.empreinte ?? "" },
      run: ({ row, motif, assertion, invalidate }) => http.post(`$
{P.desCertification}/demandes-de-statut/${row.demandeEnAttente?.id ??
""}/approbation`,
        { body: { motif }, reauthAssertion: assertion }).then(invalidate) },
  ],
  statusDictionary: etatAttestation,
  permissions: { list: "CER_REGISTER_READ", read: "CER_DOSSIER_READ", update:
"CER_ATTESTATION_STATUS_REQUEST" },
  liveTopic: "certification",
};
export const attestationPanel = (numero: string) => <ResourceDetail
descriptor={attestationDescriptor} id={numero} mode="drawer" />;
```

`registreDescriptor` (`GET /registre`, `CER_REGISTER_READ`) liste numéro, institution, type de projet, services, date d'émission, échéance de renouvellement et état (`etatAttestation`), filtrables par institution, type, état et échéance ; son action de page `exporter` (`CER_REGISTER_EXPORT`, `confirm: "confirm"`) appelle `http.blob("/bff/des/certification/registre/export", "text/csv")` avec les filtres courants ; son tiroir affiche `attestationPanel(numeroAttestation)` et la lignée des renouvellements (entrée précédente remplacée).

```
// src/i18n/fr.json (extrait)
{ "quitus.action.signer": "Signer le quitus", "quitus.defavorable": "Défavorable :
renvoyer en homologation",
  "quitus.designations": "Opérations ou services à reprendre", "fiche.action.relire":
"Relire la fiche",
```

```
  "fiche.aRelire": "Dernière lecture de plus de quinze minutes : la fiche sera relue
avant tout usage.",
  "fiche.reference.aide": "Référence de la fiche dans le dispositif de gestion des
changements ; son contenu n'est jamais saisi ici.",
  "pv.horsFenetre": "hors fenêtre", "statut.quatreYeux": "Vous avez formulé cette
demande : son approbation revient à un autre utilisateur.",
  "condition.EMPREINTES_VERIFIEES": "Empreintes des pièces vérifiées" }
```

B. Tests unitaires (Vitest + Testing Library)

```
// src/resources/certification/dossierMiseEnService.test.tsx
vi.mock("@/lib/http/client");
it("DES-11 : le bouton d'approbation est absent pour le demandeur", async () => {
  mockGet(attestationFactory({ actionsPermises: ["TELECHARGER"], demandeEnAttente:
demandeFactory({ demandeeParMoi: true }) }));
  renderWithShell(attestationPanel("GIMAC-CERT-2026-000017"));
  expect(await screen.findByText(/son approbation revient à un autre
utilisateur/)).toBeVisible();
  expect(screen.queryByRole("button", { name: "Approuver la demande" })).toBeNull();
});
it("DES-11 : un refus exige un motif et ne propose que les opérations du socle",
async () => {
  const onSubmit = vi.fn(async () => undefined);
  renderWithShell(<QuitusCircuit state="ok" dictionary={etatQuitus}
onSubmit={onSubmit}
    quitus={[quitusFactory({ nature: "SECURITE", actionsPermises: ["SIGNER"] })]}
    socle={{ empreinte: "cd".repeat(32), operations: [{ code: "PM-07", libelle: "Code
QR statique" }], services: [] }} />);
  await userEvent.click(screen.getByRole("button", { name: "Signer le quitus" }));
  await userEvent.click(screen.getByRole("radio", { name: /Défavorable/ }));
  await userEvent.click(screen.getByLabelText("Opérations ou services à reprendre"));
  expect(screen.getAllByRole("option").map(o => o.textContent)).toEqual(["PM-07 Code
QR statique"]);
  expect(screen.getByRole("button", { name: "Signer" })).toBeDisabled();
  expect(onSubmit).not.toHaveBeenCalled();
});
it("DES-11 : la fiche de changement n'offre aucune saisie de son contenu", () => {
  const saisissables = dossierMiseEnServiceDescriptor.fields.filter(f => f.inForm);
  expect(saisissables.map(f => f.name)).toEqual(["referenceFicheChangement"]);
});
it.each(["loading", "empty", "error", "ok"] as const)("DES-11 : QuitusCircuit rend
l'état %s", state => {
  renderWithShell(<QuitusCircuit state={state} dictionary={etatQuitus} quitus={state
=== "ok" ? circuitFactory().quitus : []}
    socle={socleFactory()} onSubmit={vi.fn()} />);
  expect(screen.getByTestId(state === "ok" ? "quitus-TECHNIQUE" : `quitus-circuit-$
{state}`)).toBeInTheDocument();
});
```

C. Storybook et accessibilité (axe)

```
// src/design-system/domain/QuitusCircuit/QuitusCircuit.stories.tsx
export const Chargement: S = { args: { state: "loading", quitus: [] } };
export const TechniqueAttendu: S = { args: { state: "ok", quitus:
circuit({ TECHNIQUE: "ATTENDU" }) } };
export const ParalleleSecuriteSignee: S = { args: { state: "ok", quitus:
circuit({ TECHNIQUE: "SIGNE_FAVORABLE", SECURITE: "SIGNE_FAVORABLE", CONFORMITE:
"ATTENDU" }) } };
export const InterrompuCaducs: S = { args: { state: "ok", quitus:
circuit({ TECHNIQUE: "CADUC", SECURITE: "SIGNE_DEFAVORABLE", CONFORMITE:
"ANNULE" }) } };
export const Acheve: S = { args: { state: "ok", quitus: circuit({ TECHNIQUE:
"SIGNE_FAVORABLE", SECURITE: "SIGNE_FAVORABLE", CONFORMITE: "SIGNE_FAVORABLE",
DIRECTION_GENERALE: "SIGNE_FAVORABLE" }) } };
// src/design-system/display/Hash/Hash.stories.tsx : Tronquee, Copiee,
```

```
VerificationReussie, VerificationEchouee
```

Le circuit est une liste ordonnée dont chaque étape est annoncée (« étape 2 sur 3 : sécurité et conformité, en parallèle ») ; le balayage axe de F0 couvre chaque histoire en clair et en sombre.

D. Tests Gherkin (Playwright BDD)

```
# language: fr
Fonctionnalité: DES-11 Dossier de mise en service (CER-01 à CER-12, CER-14, CER-15)
  Scénario: Quitus technique favorable
    Étant donné le chef du DES sur le circuit d'une liaison apte de l'Établissement
de paiement A
    Quand il signe le quitus technique favorable avec un motif et sa clé de sécurité
    Alors le quitus technique est « signé favorable » avec son horodatage
    Et les boutons de signature des quitus sécurité et conformité apparaissent pour
leurs signataires

  Scénario: Réauthentification échouée
    Quand la clé de sécurité est refusée
    Alors aucune décision n'est enregistrée et le quitus reste « attendu »

  Scénario: Quitus technique non acquis
    Étant donné le RSSI sur un circuit dont le quitus technique est attendu
    Alors aucun bouton « Signer le quitus » n'est proposé pour le quitus sécurité

  Scénario: Refus avec désignation
    Quand le RSSI signe un quitus sécurité défavorable avec un motif et deux
opérations du socle
    Alors les signatures encore attendues passent à « annulé » et le dossier reste «
ouvert »

  Scénario: Signature sur un socle périmé
    Étant donné un socle modifié depuis l'affichage
    Quand le signataire signe
    Alors le refus est affiché avec l'identifiant de corrélation et le socle à jour
est présenté

  Scénario: Tableau des conditions et relecture de la fiche
    Quand l'ingénieur d'homologation ouvre l'onglet « Complétude »
    Alors les six conditions sont affichées avec leur état, leur source et leur date
de vérification
    Et l'onglet « Fiche de changement » propose « Relire la fiche » sans aucun champ
de contenu

  Scénario: Bascule hors fenêtre
    Quand il consigne un procès-verbal dont l'heure de bascule est hors de la fenêtre
    Alors le procès-verbal porte la mention « hors fenêtre » et la condition reste
non satisfaite

  Scénario: Demandeur et approbateur confondus
    Étant donné une demande de suspension formulée par le chef du DES
    Quand il ouvre l'attestation concernée
    Alors le bouton « Approuver la demande » est absent et le rappel des quatre yeux
est affiché
```

E. Vérification via la passerelle (Playwright e2e)

```
// e2e/specs/des11-mise-en-service.spec.ts
const signer = async (browser: Browser, role: string, url: string, nature: string) =>
{
  const ctx = await browser.newContext({ storageState: `e2e/.auth/${role}.json` });
  const page = await ctx.newPage(); await ajouterCleVirtuelle(page); await
page.goto(url);
```

```
  await page.getByTestId(`quitus-${nature}`).getByRole("button", { name: "Signer le
quitus" }).click();
  await page.getByLabel("Motif").fill("Socle d'évaluation examiné");
  const req = page.waitForRequest(r => r.url().includes("/quitus/"));
  await page.getByRole("button", { name: "Signer" }).click();
  expect((await req).headers()["x-reauth-assertion"]).toBeTruthy();
  await expect(page.getByTestId(`quitus-${nature}`)).toContainText("signé
favorable");
  return page;
};
test("DES-11 : circuit complet dans l'ordre imposé, refus tracé hors ordre", async ({
browser }) => {
  const url = `/des/liaisons/${await liaisonApteDeRecette(browser)}/mise-en-service?
onglet=circuit`;
  const dg = await (await browser.newContext({ storageState: "e2e/.auth/test-dg.json"
})).newPage();
  const r = await dg.request.post(`/bff/des/certification/circuits/${await
circuitDe(dg.request, url)}/quitus/direction-generale`,
    { data: { decisionFavorable: true, motif: "essai hors ordre" } });
  expect(r.status()).toBeGreaterThanOrEqual(400); expect((await
r.json()).correlationId).toBeTruthy();
  await signer(browser, "test-chef-des", url, "TECHNIQUE");
  await signer(browser, "test-conformite", url, "CONFORMITE");
  await signer(browser, "test-rssi", url, "SECURITE");
  const fin = await signer(browser, "test-dg", url, "DIRECTION_GENERALE");
  await expect(fin.getByTestId("status-badge").first()).toHaveText(/autorisé/);
});
```

POR-03 | Ma liaison (portail des participants)

Élément	Valeur
Points	13 (HOM-23 : 5, HOM-24 : 5, CER-13 : 3)
Priorité	Must Have (CER-13 : Should Have)
Sprint cible	Sprint 4
Gabarit	G3 Fiche à onglets ; G4 réduit pour le dépôt de preuve
Récits couverts	HOM-23, HOM-24, CER-13

Objectif

Livrer l'écran `/portail/liaisons/{id}` pour le correspondant d'une institution : ce qui incombe à son institution en tête avec les échéances, parcours en lecture, dépôt de preuve limité aux opérations dont il est responsable, résultats du simulateur tels que repris, couvertures, état du dossier de mise en service et attestation téléchargeable avec son empreinte ; mobile dès 360 px, français et anglais. L'écran est vérifié quand une liaison d'une autre institution est présentée comme introuvable et qu'aucune annotation interne n'apparaît dans les réponses.

A. Implémentation

A1. Descripteur `portailLiaisonDescriptor`

La projection du portail ne porte aucun champ interne (motif d'état interne, chef de projet, annotations) : l'absence est structurelle, le type ne les déclare pas. Toutes les routes passent par `/bff/portail/...` ; l'institution vient du jeton côté service, jamais de la requête.

```
// src/resources/homologation/portailLiaison.tsx
import { http, P } from "@/lib/http/client";
import type { ResourceDescriptorLot1, Permitted } from "@/lib/resource/types";
import type { OperationNodeData, CoverageBarProps } from
"@/design-system/domain/types";
import { DagView, DataTable, LiaisonProgress, InlineAlert, QueryBoundary,
KeyValueList, StatusBadge, Text, AttestationDownload } from "@/design-system";
import { ResourceList, columnsOf } from "@/lib/resource";
import { campagneDescriptor, type CampaignResult } from "./campagne";
import { portailOperationDescriptor } from "./portailOperation";
import { etatDossier } from "@/resources/certification/dictionaries";
import { etatLiaison, etatOperation, etatScope } from "./dictionaries";
export type PortailLiaison = Permitted & { id: string; typeProjet: string; etat:
string; etatsParcourus: string[]; phaseCourante: string;
  versionNumero: number; couvertureCertification: CoverageBarProps["services"]
[number]["certification"];
  couvertureMiseEnService: CoverageBarProps["services"][number]["miseEnService"];
services: CoverageBarProps["services"];
  graphe: { noeuds: OperationNodeData[]; aretes: { from: string; to: string }[];
couches: string[][] }; resultats: CampaignResult[] };
export type PortailCertification = { id: string; liaisonId: string; etatDossier:
string; motifCommunicable: string | null;
  attestation: { numero: string; etat: string; empreinteDocument: string } | null };
const PH = P.portailHomologation;      // passerelle : /bff/portail/homologation/**
vers /api/v1/portail/**
const PC = P.portailCertification;     // passerelle : /bff/portail/certification/**
vers /api/v1/portail/**
export const portailQueries = {
  certification: (liaisonId: string) => ({ queryKey: ["certification", "portail",
liaisonId],
    queryFn: () => http.get<PortailCertification[]>(`${PC}/certifications`).then(l =>
```

```
l.find(c => c.liaisonId === liaisonId) ?? null) }),
};

export const portailLiaisonDescriptor: ResourceDescriptorLot1<PortailLiaison> = {
  key: "portailLiaison", label: { singular: "portail.liaison", plural:
"portail.liaisons", gender: "f" },
  idField: "id", titleField: "typeProjet", statusField: "etat",
  endpoints: { list: `${PH}/liaisons`, get: `${PH}/liaisons/{id}` },
  fields: [
    { name: "typeProjet", label: "liaison.typeProjet", kind: "text", inList: true,
inDetail: true },
    { name: "etat", label: "liaison.etat", kind: "status", inList: true, inDetail:
true },
    { name: "phaseCourante", label: "liaison.phase", kind: "text", inList: true,
inDetail: true },
    { name: "versionNumero", label: "liaison.version", kind: "number", inDetail: true
},
  ],
  actions: [],
  tabs: [
    { key: "a-faire", label: "portail.onglet.aFaire", render: r =>
      <ResourceList descriptor={portailOperationDescriptor} embedded
fixedFilters={{ liaisonId: r.id }} defaultQuery={{ tri: "echeance,asc" }} /> },
    { key: "parcours", label: "portail.onglet.parcours", render: r =>
      <DagView nodes={r.graphe.noeuds} edges={r.graphe.aretes}
layers={r.graphe.couches} dictionary={etatOperation}
        state={r.graphe.noeuds.length ? "ok" : "empty"} readOnly /> },
    { key: "resultats", label: "portail.onglet.resultats", render: r => <>
      <InlineAlert tone="info" messageKey="resultats.reprisNonRecalcules" />
      <DataTable rows={r.resultats} total={r.resultats.length} page={0}
pageSize={r.resultats.length} onChange={() => undefined}
        state={r.resultats.length ? "ok" : "empty"} rowKey={c => c.id}
columns={columnsOf(campagneDescriptor)} /></> },
    { key: "couvertures", label: "portail.onglet.couvertures", render: r =>
      <LiaisonProgress state="ok" phaseCourante={r.phaseCourante}
certification={r.couvertureCertification}
        miseEnService={r.couvertureMiseEnService} services={r.services}
dictionary={etatScope} /> },
    { key: "certification", label: "portail.onglet.certification", render: r =>
      <QueryBoundary query={portailQueries.certification(r.id)} isEmpty={c => c ===
null} emptyKey="portail.certification.aucune">{c => c && <>
        <KeyValueList items={[{ key: "dossier.etat", value: <StatusBadge
status={etatDossier[c.etatDossier]?.status ?? "neutral"}
          label={<Text i18nKey={`dossier.etat.${c.etatDossier}`} />} /> },
          ...(c.motifCommunicable ? [{ key: "portail.motifRefus", value:
c.motifCommunicable }] : [])]} />
        {c.attestation && <AttestationDownload numero={c.attestation.numero}
empreinte={c.attestation.empreinteDocument}
          download={() => http.blob(`${PC}/certifications/${c.id}/attestation`,
"application/pdf")} />}</>}
      </QueryBoundary> },
  ],
  statusDictionary: etatLiaison,
  permissions: { list: "HOM_PORTAL_READ", read: "HOM_PORTAL_READ" },
  liveTopic: "homologation",
};
```

`AttestationDownload` (catalogue, domaine) affiche le numéro et l'empreinte (`Hash`), télécharge le document (journalisé par le service), compare l'empreinte de l'en-tête `Repr-Digest` à celle annoncée et affiche « empreinte vérifiée » ou un `InlineAlert` critique. `columnsOf` dérive les colonnes d'un descripteur : les résultats s'affichent donc exactement comme dans DES-09.

A2. Ce qui incombe à l'institution et dépôt G4 réduit

```
// src/resources/homologation/portailOperation.tsx
```

```
export type PortailOperation = Permitted & { id: string; liaisonId: string; code:
string; libelle: string; etat: string;
  echeance: string | null; joursRestants: number | null; preuveAttendue: string;
preuve: File | null;
  accuse: { empreinte: string; dateDepot: string } | null };
export const portailOperationDescriptor: ResourceDescriptorLot1<PortailOperation> = {
  key: "portailOperation", label: { singular: "portail.operation", plural:
"portail.operations", gender: "f" },
  idField: "id", titleField: "libelle", statusField: "etat",
  endpoints: { list: `${PH}/liaisons/{liaisonId}/a-faire`, get:
`${PH}/liaisons/{liaisonId}/a-faire/{id}` },
  steps: [{ key: "preuve", labelKey: "cloture.etape.preuve", fields:
["preuveAttendue", "preuve", "accuse"], action: "deposer" }],
  fields: [
    { name: "code", label: "operation.code", kind: "mono", inList: true },
    { name: "libelle", label: "operation.libelle", kind: "text", inList: true },
    { name: "etat", label: "operation.etat", kind: "status", inList: true },
    { name: "echeance", label: "operation.echeance", kind: "date", inList: true,
sortable: true,
      render: (_v, r) => r.joursRestants !== null && r.joursRestants < 0
        ? <StatusBadge status="warn" label={<Text i18nKey="portail.echeanceDepassee"
values={{ n: -r.joursRestants }} />} /> : r.echeance },
    { name: "preuveAttendue", label: "cloture.preuveAttendue", kind: "text", inForm:
"edit", readOnly: true },
    { name: "preuve", label: "cloture.fichier", kind: "file", inForm: "edit",
required: true },
    { name: "accuse", label: "portail.accuse", kind: "custom", inForm: "edit",
readOnly: true,
      render: (_v, r) => r.accuse && <KeyValueList items={[{ key:
"portail.accuse.empreinte", value: <Hash value={r.accuse.empreinte} /> },
        { key: "portail.accuse.date", value: r.accuse.dateDepot }]} /> },
  ],
  actions: [
    { key: "ouvrirDepot", label: "portail.action.deposer", icon: "file-up", variant:
"primary", scope: "row", permission: "HOM_PORTAL_SUBMIT",
      confirm: "none", isAllowed: r => r.actionsPermises.includes("DEPOSER_PREUVE"),
      run: async ({ row, navigate }) => navigate(`/portail/liaisons/$
{row.liaisonId}/operations/${row.id}/preuve`) },
    { key: "deposer", label: "portail.action.confirmerDepot", icon: "file-up",
variant: "primary", scope: "page", permission: "HOM_PORTAL_SUBMIT",
      confirm: "confirm", isAllowed: r =>
r.actionsPermises.includes("DEPOSER_PREUVE"),
      run: async ({ row, invalidate }) => { if (!row.preuve) return;
        const body = new FormData(); body.set("fichier", row.preuve);
body.set("typePiece", row.preuveAttendue);
        await http.post(`${PH}/operations/${row.id}/preuves`, { body });
invalidate(); } },
  ],
  statusDictionary: etatOperation,
  permissions: { list: "HOM_PORTAL_READ", read: "HOM_PORTAL_READ", update:
"HOM_PORTAL_SUBMIT" },
  liveTopic: "homologation",
};
```

A3. Pages, canal poussé filtré et adaptation mobile

```
// src/features/portail/MaLiaisonPage.tsx
export default function MaLiaisonPage() {
  const { id = "" } = useParams();
  useLiveTopic("portail", ["homologation", "certification"]); // la passerelle ne
diffuse que les événements de l'institution du jeton
  return <ResourceDetail descriptor={portailLiaisonDescriptor} id={id} mode="page"
/>;
}
// src/features/portail/DepotPreuvePage.tsx :
/portail/liaisons/{id}/operations/{op}/preuve
export default function DepotPreuvePage() {
  const { id = "", op = "" } = useParams(); const navigate = useNavigate();
```

```
  return <ResourceForm descriptor={portailOperationDescriptor} mode="edit" id={op}
initial={{ liaisonId: id }}
    onDone={() => navigate(`/portail/liaisons/${id}?onglet=a-faire`)} />;
}
```

L'enveloppe `layer="portail"` applique les règles du DS-GIMAC-01 §10.2 sans code d'écran : navigation en tiroir, `DataTable` en cartes empilées sous le point de rupture `--gm-bp-sm`, boutons `lg`, `FileDropzone` ouvrant le sélecteur de l'appareil, `DagView` basculé par défaut en vue liste sous ce point de rupture.

```
// src/i18n/en.json (extrait ; fr.json porte les mêmes clés)
{ "portail.onglet.aFaire": "What my institution must do", "portail.onglet.parcours":
"Pathway",
  "portail.onglet.resultats": "Simulator results", "portail.action.deposer": "Submit
the evidence",
  "portail.echeanceDepassee": "Overdue by {{n}} days", "portail.accuse.empreinte":
"Fingerprint of the submitted file",
  "resultats.reprisNonRecalcules": "Results taken from the simulator, never
recalculated.",
  "portail.motifRefus": "Reason given", "portail.introuvable": "This pathway cannot
be found." }
```

B. Tests unitaires (Vitest + Testing Library)

```
// src/resources/homologation/portailLiaison.test.tsx
vi.mock("@/lib/http/client");
it("POR-03 : aucune action de dépôt sur une opération qui incombe au DES", async ()
=> {
  mockList([portailOperationFactory({ code: "TC-02", actionsPermises: [] }),
portailOperationFactory({ code: "PM-07", actionsPermises: ["DEPOSER_PREUVE"] })]);
  renderWithShell(<ResourceList descriptor={portailOperationDescriptor} embedded
fixedFilters={{ liaisonId: "l1" }} />, { layer: "portail" });
  expect(within(await screen.findByRole("row", { name: /TC-
02/ })).queryByRole("button")).toBeNull();
  expect(within(screen.getByRole("row", { name: /PM-07/ })).getByRole("button",
{ name: "Déposer la preuve" })).toBeVisible();
});
it("POR-03 : la projection ne déclare aucun champ interne et n'appelle que le
portail", () => {
  const noms = portailLiaisonDescriptor.fields.map(f => f.name as string);
  ["motifEtat", "chefDeProjetNom", "annotations"].forEach(n =>
expect(noms).not.toContain(n));
  Object.values(portailLiaisonDescriptor.endpoints).forEach(e =>
expect(e).toMatch(/^\/bff\/portail\//));
});
it("POR-03 : l'onglet de tête s'affiche en anglais sans rechargement", async () => {
  mockGet(portailLiaisonFactory());
  renderWithShell(<ResourceDetail descriptor={portailLiaisonDescriptor} id="l1"
mode="page" />, { layer: "portail" });
  await userEvent.click(await screen.findByRole("button", { name: "EN" }));
  expect(await screen.findByRole("tab", { name: "What my institution must do",
selected: true })).toBeVisible();
});
it.each(["loading", "empty", "error", "ok"] as const)("POR-03 : la liste de ce qui
m'incombe rend l'état %s", async state => {
  mockListState(state, [portailOperationFactory()]);
  renderWithShell(<ResourceList descriptor={portailOperationDescriptor} embedded
fixedFilters={{ liaisonId: "l1" }} />, { layer: "portail" });
  expect(await screen.findByTestId(`datatable-${state}`)).toBeInTheDocument();
});
```

C. Storybook et accessibilité (axe)

```
// src/design-system/domain/DagView/DagView.portail.stories.tsx
export default { component: DagView, parameters: { viewport: { defaultViewport:
"mobile360" }, layer: "portail" } } satisfies Meta<typeof DagView>;
export const LectureMobile: S = { args: { ...grapheFactory(), readOnly: true, state:
"ok", dictionary: etatOperation } };
export const LectureMobileAnglais: S = { ...LectureMobile, parameters: { locale: "en"
} };
// src/design-system/domain/AttestationDownload/AttestationDownload.stories.tsx :
Disponible, EmpreinteVerifiee, EmpreinteDivergente
```

Le balayage axe de F0 s'exécute aussi sur le viewport `mobile360` pour les histoires du portail ; les cibles tactiles sont contrôlées à 48 px par la règle `target-size` d'axe.

D. Tests Gherkin (Playwright BDD)

```
# language: fr
Fonctionnalité: POR-03 Ma liaison (HOM-23, HOM-24, CER-13)
  Contexte:
    Étant donné le correspondant de l'Établissement de paiement A connecté au portail

  Scénario: Vue de la liaison
    Quand il ouvre la liaison de son institution
    Alors l'onglet « Ce qui incombe à mon institution » est ouvert en tête avec les
échéances
    Et l'onglet des résultats affiche les verdicts tels que repris avec leur
référence de trace

  Scénario: Dépôt par le correspondant
    Étant donné une opération éligible incombant à son institution dont il est
responsable
    Quand il dépose la pièce du type attendu et confirme
    Alors l'accusé de dépôt affiche l'empreinte de la pièce et l'opération passe à «
preuve déposée »

  Scénario: Opération incombant au DES
    Quand il consulte une opération de sa liaison incombant au DES
    Alors aucune action de dépôt ne lui est proposée

  Scénario: Changement d'état poussé
    Quand une opération qui lui incombe devient éligible
    Alors la liste se met à jour sans rechargement et la ligne est surlignée

  Scénario: Liaison ou attestation d'une autre institution
    Étant donné le correspondant de la Banque B connecté au portail
    Quand il ouvre l'adresse de la liaison de l'Établissement de paiement A
    Alors la page « introuvable » est affichée avec un identifiant de corrélation

  Scénario: Téléchargement de l'attestation
    Étant donné une attestation émise pour son institution
    Quand il la télécharge depuis l'onglet « Certification »
    Alors le document est enregistré et la mention « empreinte vérifiée » est
affichée
```

E. Vérification via la passerelle (Playwright e2e)

```
// e2e/specs/por03-ma-liaison.spec.ts : projet Playwright « portail-mobile »
(viewport 360 x 740)
test.use({ storageState: "e2e/.auth/test-correspondant-a.json", viewport: { width:
360, height: 740 } });
const INTERNES = ["motifEtat", "chefDeProjetId", "chefDeProjetNom", "annotations",
"annotationsInternes"];
test("POR-03 : parcours mobile, cloisonnement et absence d'annotation interne", async
```

```
({ page, browser }) => {
  const id = await liaisonDuCorrespondant(page.request);
  const reponse = page.waitForResponse(r => r.url().endsWith(`/portail/liaisons/$
{id}`));
  await page.goto(`/portail/liaisons/${id}`);
  const corps = JSON.stringify(await (await reponse).json());
  INTERNES.forEach(k => expect(corps).not.toContain(`"${k}"`));
  expect(await page.evaluate(() => document.documentElement.scrollWidth <=
window.innerWidth)).toBe(true);
  await expect(page.getByRole("tab", { name: "Ce qui incombe à mon institution",
selected: true })).toBeVisible();
  await page.getByRole("button", { name: "Déposer la preuve" }).first().click();
  await page.getByLabel("Fichier").setInputFiles("e2e/fixtures/capture-code-qr.png");
  await page.getByRole("button", { name: "Confirmer le dépôt" }).click(); await
page.getByRole("button", { name: "Confirmer" }).click();
  await expect(page.getByTestId("hash")).toBeVisible();
  const b = await (await browser.newContext({ storageState: "e2e/.auth/test-
correspondant-b.json" })).newPage();
  await b.goto(`/portail/liaisons/${id}`);
  await expect(b.getByText("introuvable")).toBeVisible(); await
expect(b.getByTestId("correlation-id")).toBeVisible();
  const cookies = await page.context().cookies();
  expect(cookies.filter(c => c.name.startsWith("GIMAC_SESSION")).every(c =>
c.httpOnly && c.sameSite === "Strict")).toBe(true);
});
```

INTÉGRATIONS TRANSVERSES | Observabilité, résilience et sécurité du lot 1

Métriques métier exposées à la pile d'observabilité

Chaque service expose `/management/prometheus` (déjà collecté par le socle). Les métriques ci-dessous sont ajoutées par Micrometer, avec les étiquettes `service`, `type_projet` et, quand il a un sens, `institution_id` haché (jamais en clair dans les étiquettes).

Métrique	Type	Service	Usage
`homologation_liaisons_par_etat`	jauge	MS-HOMOLOGATION	Tableau mural et cockpit : nombre de liaisons par état
`homologation_operations_closes_total`	compteur	MS-HOMOLOGATION	Débit de clôture ; alerte si nul sur cinq jours ouvrés pour une liaison ouverte
`homologation_cloture_horodatage_attente`	jauge	MS-HOMOLOGATION	Opérations validées en attente de jeton RFC 3161 ; alerte au-delà de 15 minutes
`homologation_reprise_disjoncteur_etat`	jauge (0, 1, 2)	MS-HOMOLOGATION	État du disjoncteur du simulateur ; alerte quand ouvert
`homologation_reprise_resultats_total`	compteur	MS-HOMOLOGATION	Résultats repris ; étiquette `verdict`
`homologation_reprise_rebut_total`	compteur	MS-HOMOLOGATION	Réponses mal formées placées au rebut
`homologation_scenarios_inconnus`	jauge	MS-HOMOLOGATION	File de qualification ; alerte au-delà de dix
`homologation_couverture_certification`	jauge	MS-HOMOLOGATION	Couverture par liaison et service (pour le cockpit du chef du DES)
`certification_circuits_ouverts`	jauge	MS-CERTIFICATION	Circuits en cours par étape attendue
`certification_quitus_delai_secondes`	histogramme	MS-CERTIFICATION	Délai entre attente et signature par nature de quitus
`certification_quitus_caducs_total`	compteur	MS-CERTIFICATION	Caducités déclarées ; étiquette `cause`
`certification_scellement_attente`	jauge	MS-CERTIFICATION	Dossiers en attente de jeton d'horodatage
`certification_attestations_emises_total`	compteur	MS-CERTIFICATION	Attestations émises par type de projet
`certification_fiche_changement_lecture_echecs_total`	compteur	MS-CERTIFICATION	Échecs de lecture du dispositif de gestion des changements
`outbox_en_attente`	jauge	les deux	Lignes PENDING ; alerte

Métrique	Type	Service	Usage
			au-delà de 1 000 ou d'un âge de 60 s
`outbox_echecs_total`	compteur	les deux	Lignes FAILED ; alerte dès la première
`consommateur_rebut_total`	compteur	les deux	Messages envoyés en file de rebut ; alerte dès le premier

```
// infrastructure/observability/HomologationMetrics.java
@Component
public class HomologationMetrics {
    private final MeterRegistry registry; private final LiaisonRepository liaisons;
private final ReplayCursorRepository cursors;
    public HomologationMetrics(MeterRegistry registry, LiaisonRepository liaisons,
ReplayCursorRepository cursors) {
        this.registry = registry; this.liaisons = liaisons; this.cursors = cursors;
        for (EtatLiaison e : EtatLiaison.values()) {
            Gauge.builder("homologation_liaisons_par_etat", () ->
liaisons.countByEtat(e)).tag("etat", e.name()).register(registry);
        }
        Gauge.builder("homologation_reprise_disjoncteur_etat", () ->
cursors.findFirstByOrderByDateDernierSuccesDesc()
            .map(c -> (double)
c.getEtatDisjoncteur().ordinal()).orElse(0d)).register(registry);
    }
    public void operationClosed(TypeProjet type)
{ registry.counter("homologation_operations_closes_total", "type_projet",
type.name()).increment(); }
    public void resultImported(Verdict v)
{ registry.counter("homologation_reprise_resultats_total", "verdict",
v.name()).increment(); }
}
```

Tableau de bord d'exploitation (Grafana, dossier `infra/observability/dashboards/lot1.json`) : quatre lignes de panneaux, dans l'ordre : santé des deux services et de l'outbox ; reprise du simulateur (disjoncteur, résultats, rebut) ; circuit de certification (circuits ouverts par étape, délais par quitus, caducités) ; attestations et scellements en attente. Règles d'alerte associées dans `infra/observability/alerts/lot1.yml`, routées vers MS-MONITORING par le récepteur d'alertes du socle.

Résilience des appels sortants

Port	Tiers ou service	Délai	Réessai	Disjoncteur	Comportement en panne
`ParticipantReaderPort`	MS-PARTICIPANT	2 s	2 tentatives	50 % d'échecs sur 20 appels	Ouverture de liaison refusée (503) ; aucune valeur par défaut
`DocumentDepositPort`	MS-DOCUMENT	30 s (dépôt)	aucun (non idempotent)	50 % sur 10 appels	Dépôt refusé, opération inchangée
`ReauthenticationPort`	MS-IAM	3 s	aucun	50 % sur 10 appels	Signature impossible ;

Port	Tiers ou service	Délai	Réessai	Disjoncteur	Comportement en panne
					message explicite
`SimulatorReaderPort`	Simulateur (lecture seule)	10 s	3 tentatives, retard 1 s, 2 s, 4 s	5 échecs consécutifs, demi-ouverture après 2 min	Reprise suspendue, bandeau, alerte ; autres fonctions intactes
`ChangeRecordReaderPort`	Dispositif de gestion des changements (lecture seule)	5 s	3 tentatives, retard exponentiel	5 échecs consécutifs	Décision dépendante bloquée (503) ; aucune valeur par défaut
`TimestampAuthorityPort`	Autorité d'horodatage RFC 3161	5 s	planifié : 30 s, 1 min, 2 min, 4 min, puis toutes les 10 min	aucun (planificateur)	État « horodatage en attente » ou « scellement en attente »
`KeyVaultSigningPort`	Coffre de clés	5 s	aucun	50 % sur 10 appels	Aucune décision partielle ; signataire invité à réessayer
`HomologationReaderPort`	MS-HOMOLOGATION (interne)	3 s	2 tentatives	50 % sur 20 appels	Ouverture de circuit suspendue, événement au rebut

Configuration Resilience4j commune, un nom d'instance par port :

```
# src/main/resources/config/application.yml (extrait)
resilience4j:
  circuitbreaker.instances:
    simulator-reader: { slidingWindowType: COUNT_BASED, slidingWindowSize: 5,
failureRateThreshold: 100, waitDurationInOpenState: 2m,
permittedNumberOfCallsInHalfOpenState: 1 }
    change-record-reader: { slidingWindowSize: 5, failureRateThreshold: 100,
waitDurationInOpenState: 2m }
    participant-reader: { slidingWindowSize: 20, failureRateThreshold: 50,
waitDurationInOpenState: 30s }
  retry.instances:
    simulator-reader: { maxAttempts: 3, waitDuration: 1s, enableExponentialBackoff:
true, exponentialBackoffMultiplier: 2 }
    change-record-reader: { maxAttempts: 3, waitDuration: 1s,
enableExponentialBackoff: true, exponentialBackoffMultiplier: 2 }
  timelimiter.instances:
    simulator-reader: { timeoutDuration: 10s }
    change-record-reader: { timeoutDuration: 5s }
```

Politique réseau et non-intrusion

Les deux adaptateurs de lecture vers des tiers (simulateur, dispositif de gestion des changements) sont encadrés par une politique réseau Kubernetes qui limite les flux sortants du pod au seul point d'accès de lecture, et par un test d'architecture qui interdit toute méthode

HTTP autre que GET dans ces adaptateurs. Les deux mécanismes sont détaillés dans HOM-16 et CER-09 ; ils sont rappelés ici parce qu'ils sont vérifiés à chaque revue d'itération.

```
// src/test/java/.../architecture/NonIntrusionArchTest.java (commun aux deux
services)
@AnalyzeClasses(packages = "org.gimac.des")
class NonIntrusionArchTest {
    @ArchTest
    static final ArchRule lecture_seule_vers_les_tiers = noClasses()
        .that().resideInAnyPackage("..infrastructure.client.simulator..",
"..infrastructure.client.changerecord..")
        .should().callMethodWhere(target(nameMatching("post|put|patch|
delete")).and(target(owner(assignableTo(RestClient.class)))))
        .because("la plateforme ne fait que lire le simulateur et le dispositif de
gestion des changements");

    @ArchTest
    static final ArchRule domaine_sans_infrastructure =
noClasses().that().resideInAPackage("..domain.service..")
        .should().dependOnClassesThat().resideInAnyPackage("org.springframework..",
"jakarta.persistence..", "org.apache.kafka..")
        .because("les services de domaine sont purs et testables sans conteneur");
}
```

Contrôles de sécurité de fin d'itération

Contrôle	Outil	Critère de passage
Refus sans jeton et refus par OPA sur chaque route du lot	Script `scripts/controle-acces-lot1.sh` (cURL, une ligne par route et par rôle non autorisé)	100 % des routes renvoient 401 sans jeton et 403 avec un rôle non autorisé
Cloisonnement du portail	Tests Playwright et cURL avec `test-correspondant-b` sur les ressources de l'institution A	404 systématique, tentative journalisée dans MS-AUDIT
Réauthentification forte	Tests d'intégration des commandes à signature sans en-tête `X-Reauth-Assertion`	401 avec code `REAUTH_REQUIRED`, aucune écriture
Masquage des journaux	Rejeu du contrôle de masquage du socle sur les journaux des deux services après la recette	Aucun numéro de carte, jeton, courriel ni téléphone en clair
Dépendances	Analyse des dépendances (OWASP Dependency-Check) et image (Trivy) dans la chaîne de construction	Aucune vulnérabilité critique ou élevée non justifiée
Non-intrusion	`NonIntrusionArchTest` et politique réseau	Tests verts ; `kubectl describe networkpolicy` conforme
Immuabilité	Tentative de mise à jour SQL directe d'une `signed_decision` et d'une version publiée	Refus par le déclencheur

CARTOGRAPHIE │ Contrats REST, événements et permissions du lot 1

Les tableaux ci-dessous consolident, récit par récit, les chemins REST (préfixe interne `/api/v1`, exposés par la passerelle sous `/bff/des/<service>` ou `/bff/portail/<service>`), les événements produits et consommés, et les appels internes. Ils servent de liste de contrôle à la revue de chaque itération : un chemin ou un événement absent du code à la fin de l'itération est une anomalie.

Contrats de MS-HOMOLOGATION

Récit	Chemins REST et appels internes	Événements	Permissions
HOM-01	POST /referentiels (création d'un brouillon) ; PUT /referentiels/{id}/operations/{code} ; POST /referentiels/{id}/publication ; GET /referentiels?type=	publie homologation.referential-published.v1	HOM_REFERENTIEL_EDIT, HOM_REFERENTIEL_PUBLISH
HOM-02	POST /referentiels/{id}/controle (contrôle à blanc) ; GET /referentiels/{id}/graphe ; port interne : contrôle d'acyclicité invoqué par la publication	aucun	HOM_REFERENTIEL_EDIT
HOM-03	POST /referentiels/{id}/migrations/simulation ; POST /referentiels/{id}/migrations (liaisons retenues) ; GET /migrations/{id}/rapport	publie homologation.liaison-migrated.v1 ; publie homologation.operation-reopened.v1	HOM_REFERENTIEL_MIGRATE
HOM-04	POST /liaisons ; GET /liaisons/{id} ; PUT /liaisons/{id}/chef-de-projet ; appelle GET /institutions/{id} de MS-PARTICIPANT	publie homologation.liaison-opened.v1	HOM_LIAISON_OPEN, HOM_LIAISON_READ
HOM-05	GET /liaisons/{id}/perimetre	consomme participant.subscription-changed.v1 ; publie homologation.coverage-changed.v1 ; publie homologation.notification-requested.v1	HOM_LIAISON_READ
HOM-06	POST /liaisons/{id}/degel	consomme	HOM_LIAISON_UNFR

Récit	Chemins REST et appels internes	Événements	Permissions
		participant.status-changed.v1 ; publie homologation.liaison-frozen.v1 ; publie homologation.liaison-unfrozen.v1	EEZE
HOM-07	GET /liaisons/{id}/operations (filtrable par état, segment, partie) ; GET /liaisons/{id}/graphe	publie homologation.operation-unlocked.v1	HOM_LIAISON_READ
HOM-08	PUT /operations/{id}/responsable ; GET /operations/{id}/attributions	publie homologation.operation-assigned.v1	HOM_OPERATION_ASSIGN
HOM-09	POST /operations/{id}/preuves ; GET /operations/{id}/preuves ; appelle le dépôt interne de MS-DOCUMENT	consomme document.document-sealed.v1 ; publie homologation.evidence-submitted.v1	HOM_EVIDENCE_SUBMIT
HOM-10	POST /operations/{id}/validation (réauthentification exigée) ; GET /operations/{id}/elements-de-cloture ; appelle l'autorité d'horodatage RFC 3161	publie homologation.operation-closed.v1	HOM_OPERATION_VALIDATE
HOM-11	POST /operations/{id}/reouverture	publie homologation.operation-reopened.v1 ; publie homologation.coverage-changed.v1	HOM_OPERATION_REOPEN
HOM-12	PUT /operations/{id}/grille ; GET /operations/{id}/grille	publie homologation.operation-closed.v1	HOM_GRID_EDIT
HOM-13	port sortant : lecture des campagnes et résultats du simulateur (lecture seule) ; POST /simulateur/notifications (réception seule) ; GET /liaisons/{id}/campagnes	publie homologation.campaign-result-imported.v1	HOM_LIAISON_READ
HOM-14	GET /reprise/etat (état du disjoncteur et dernière position) ; GET /reprise/rebut ; POST	aucun	HOM_REPLAY_ADMIN

Récit	Chemins REST et appels internes	Événements	Permissions
	/reprise/rebut/{id}/relance ; métriques exposées à la pile d'observabilité		
HOM-15	GET /liaisons/{id}/scenarios-inconnus ; POST /scenarios-inconnus/{id}/qualification	consomme homologation.campaign-result-imported.v1 (interne) ; publie homologation.coverage-changed.v1	HOM_SCENARIO_QUALIFY
HOM-16	port sortant : lecture seule vers le simulateur ; aucune route d'écriture vers le simulateur	publie les tentatives bloquées vers MS-AUDIT	aucune (contrôle d'architecture)
HOM-17	GET /liaisons/{id}/couvertures	publie homologation.coverage-changed.v1	HOM_LIAISON_READ
HOM-18	GET /liaisons/{id}/socle-evaluation (interne, MS-CERTIFICATION) ; GET /liaisons/{id}/intervenants (interne, MS-CERTIFICATION)	publie homologation.liaison-ready-for-certification.v1	INTERNAL_HOMOLOGATION_READ
HOM-19		consomme certification.quitus-signed.v1 ; consomme certification.go-live-authorized.v1 ; consomme certification.attestation-issued.v1 ; publie homologation.liaison-returned.v1 ; publie homologation.operation-reopened.v1	aucune (consommateur)
HOM-20	GET /tableau-de-bord/liaisons (filtré, paginé) ; GET /liaisons/{id}/historique	aucun	HOM_DASHBOARD_READ
HOM-21	PUT /parametres/echeances	publie homologation.notification-requested.v1	HOM_DEADLINE_CONFIG
HOM-22	POST /liaisons/{id}/dossier-de-preuves ; GET /dossiers-de-preuves/{id} ; port interne : export pour MS-CERTIFICATION par empreinte de socle	publie homologation.evidence-file-exported.v1	HOM_EVIDENCE_FILE_EXPORT, INTERNAL_HOMOLOGATION_READ
HOM-23	POST	publie	HOM_PORTAL_SUBMI

Récit	Chemins REST et appels internes	Événements	Permissions
	/portail/operations/{id}/preuves	homologation.evidence-submitted.v1 ; publie homologation.notification-requested.v1	T
HOM-24	GET /portail/liaisons ; GET /portail/liaisons/{id} ; GET /portail/liaisons/{id}/a-faire ; diffusion sur le canal poussé via MS-GATEWAY	aucun	HOM_PORTAL_READ

Événements produits par MS-HOMOLOGATION

Type d'événement	Sujet Kafka	Récits émetteurs
homologation.referential-published.v1	homologation.referential	HOM-01
homologation.liaison-opened.v1	homologation.liaison	HOM-04
homologation.liaison-frozen.v1	homologation.liaison	HOM-06
homologation.liaison-unfrozen.v1	homologation.liaison	HOM-06
homologation.liaison-migrated.v1	homologation.liaison	HOM-03
homologation.liaison-returned.v1	homologation.liaison	HOM-19
homologation.operation-unlocked.v1	homologation.operation	HOM-07
homologation.operation-assigned.v1	homologation.operation	HOM-08
homologation.evidence-submitted.v1	homologation.evidence	HOM-09, HOM-23
homologation.operation-closed.v1	homologation.operation	HOM-10, HOM-12
homologation.operation-reopened.v1	homologation.operation	HOM-03, HOM-11, HOM-19
homologation.campaign-result-imported.v1	homologation.campaign	HOM-13
homologation.coverage-changed.v1	homologation.coverage	HOM-05, HOM-11, HOM-15, HOM-17
homologation.liaison-ready-for-certification.v1	homologation.liaison	HOM-18
homologation.evidence-file-exported.v1	homologation.evidence	HOM-22
homologation.notification-requested.v1	homologation.notification	HOM-05, HOM-21, HOM-23

Événements consommés par MS-HOMOLOGATION

Type d'événement	Sujet Kafka	Récits consommateurs
`participant.subscription-changed.v1`	`participant.subscription`	HOM-05
`participant.status-changed.v1`	`participant.status`	HOM-06
`certification.quitus-signed.v1`	`certification.quitus`	HOM-19
`certification.go-live-authorized.v1`	`certification.go`	HOM-19
`certification.attestation-issued.v1`	`certification.attestation`	HOM-19
`document.document-sealed.v1`	`document.document`	HOM-09
`iam.user-changed.v1`	`iam.user`	(prévu par la conception, sans récit dédié : traité par le socle d'audit ou en itération C)
`homologation.campaign-result-imported.v1`	`homologation.campaign`	HOM-15

Contrats de MS-CERTIFICATION

Récit	Chemins REST et appels internes	Événements	Permissions
CER-01	appelle GET /liaisons/{id}/socle-evaluation de MS-HOMOLOGATION ; GET /circuits/{id}	consomme homologation.liaison-ready-for-certification.v1 ; publie certification.circuit-opened.v1	CER_CIRCUIT_READ
CER-02	POST /circuits/{id}/quitus/technique (réauthentification exigée) ; GET /circuits/{id}/quitus ; port interne : signature par le coffre de clés	publie certification.quitus-signed.v1	CER_QUITUS_TECHNIQUE_SIGN
CER-03	POST /circuits/{id}/quitus/securite (réauthentification exigée) ; POST /circuits/{id}/quitus/conformite (réauthentification exigée)	publie certification.quitus-signed.v1	CER_QUITUS_SECURITE_SIGN, CER_QUITUS_CONFORMITE_SIGN
CER-04	POST /circuits/{id}/quitus/direction-generale (réauthentification exigée) ; GET /dossiers/{id}/decision	publie certification.quitus-signed.v1 ; publie certification.go-live-authorized.v1	CER_QUITUS_DG_SIGN
CER-05	POST /circuits/{id}/quitus/{natu	publie certification.quitus-	CER_QUITUS_*_SIGN

Récit	Chemins REST et appels internes	Événements	Permissions
	re} (décision défavorable)	signed.v1	
CER-06		consomme homologation.operation-reopened.v1 ; consomme homologation.coverage-changed.v1 ; publie certification.quitus-lapsed.v1	aucune (consommateur)
CER-07	appelle GET /liaisons/{id}/intervenants de MS-HOMOLOGATION ; GET /circuits/{id}/signataires-admissibles	aucun	CER_CIRCUIT_READ
CER-08	GET /dossiers/{id} ; GET /dossiers/{id}/conditions ; POST /dossiers/{id}/verification	consomme homologation.coverage-changed.v1	CER_DOSSIER_READ, CER_DOSSIER_VERIFY
CER-09	port sortant : lecture seule des fiches de changement ; PUT /dossiers/{id}/fiche-de-changement (référence uniquement) ; GET /dossiers/{id}/fiche-de-changement ; aucune route d'écriture vers le dispositif de gestion des changements	aucun	CER_CHANGE_RECORD_LINK, CER_DOSSIER_READ
CER-10	POST /dossiers/{id}/proces-verbal ; appelle le dépôt interne de MS-DOCUMENT	publie certification.go-live-recorded.v1	CER_GOLIVE_REPORT_SUBMIT
CER-11	appelle l'autorité d'horodatage RFC 3161 ; GET /dossiers/{id}/horodatages	publie certification.dossier-sealed.v1	CER_DOSSIER_READ
CER-12	POST /dossiers/{id}/attestation ; GET /attestations/{numero} ; appelle le port interne d'export du dossier de preuves de MS-HOMOLOGATION ; appelle le dépôt interne de MS-DOCUMENT	publie certification.attestation-issued.v1	CER_ATTESTATION_ISSUE

Récit	Chemins REST et appels internes	Événements	Permissions
CER-13	GET /portail/certifications ; GET /portail/certifications/{id}/attestation	aucun	CER_PORTAL_READ
CER-14	GET /registre (filtré, paginé) ; GET /registre/export (journalisé)	publie certification.renewal-due.v1 ; publie certification.notification-requested.v1	CER_REGISTER_READ, CER_REGISTER_EXPORT
CER-15	POST /attestations/{numero}/demandes-de-statut ; POST /demandes-de-statut/{id}/approbation (réauthentification exigée)	publie certification.attestation-revoked.v1 ; publie certification.attestation-suspended.v1 ; publie certification.attestation-reinstated.v1	CER_ATTESTATION_STATUS_REQUEST, CER_ATTESTATION_STATUS_APPROVE

Événements produits par MS-CERTIFICATION

Type d'événement	Sujet Kafka	Récits émetteurs
`certification.circuit-opened.v1`	`certification.circuit`	CER-01
`certification.quitus-signed.v1`	`certification.quitus`	CER-02, CER-03, CER-04, CER-05
`certification.quitus-lapsed.v1`	`certification.quitus`	CER-06
`certification.go-live-authorized.v1`	`certification.go`	CER-04
`certification.go-live-recorded.v1`	`certification.go`	CER-10
`certification.dossier-sealed.v1`	`certification.dossier`	CER-11
`certification.attestation-issued.v1`	`certification.attestation`	CER-12
`certification.attestation-suspended.v1`	`certification.attestation`	CER-15
`certification.attestation-reinstated.v1`	`certification.attestation`	CER-15
`certification.attestation-revoked.v1`	`certification.attestation`	CER-15
`certification.renewal-due.v1`	`certification.renewal`	CER-14
`certification.notification-requested.v1`	`certification.notification`	CER-14

Événements consommés par MS-CERTIFICATION

Type d'événement	Sujet Kafka	Récits consommateurs
`homologation.liaison-ready-for-certification.v1`	`homologation.liaison`	CER-01

Type d'événement	Sujet Kafka	Récits consommateurs
homologation.operation-reopened.v1	homologation.operation	CER-06
homologation.coverage-changed.v1	homologation.coverage	CER-06, CER-08
homologation.liaison-frozen.v1	homologation.liaison	(prévu par la conception, sans récit dédié : traité par le socle d'audit ou en itération C)
participant.status-changed.v1	participant.status	(prévu par la conception, sans récit dédié : traité par le socle d'audit ou en itération C)
document.document-sealed.v1	document.document	(prévu par la conception, sans récit dédié : traité par le socle d'audit ou en itération C)
iam.user-changed.v1	iam.user	(prévu par la conception, sans récit dédié : traité par le socle d'audit ou en itération C)
iam.role-assignment-changed.v1	iam.role	(prévu par la conception, sans récit dédié : traité par le socle d'audit ou en itération C)

RÉCAPITULATIF — Fichiers, ordre d'implémentation, calendrier et Definition of Done

Fichiers à créer ou modifier dans MS-HOMOLOGATION

Fichier (relatif à `org/gimac/des/homologation`)	Action	Récits
`jdl/lot1-homologation.jdl`, `config/liquibase/changelog/20261005000001_lot1_post_generation.xml`	créer	Phase 0
`infrastructure/messaging/{outbox,consumer}/*`, `infrastructure/messaging/{CloudEventEnvelope,TopicResolver}.java`	créer si absents du socle	Phase 0
`config/{SecurityConfiguration,AdaptersConfiguration,KafkaErrorHandlingConfiguration}.java`	compléter	Phase 0
`domain/service/TopologicalSorter.java`, `CycleFinder.java`, `ReferentialConsistencyChecker.java`	créer	HOM-02, HOM-07
`domain/service/ReferentialDiff.java`, `application/usecase/MigrateLiaisonsUseCase.java`	créer	HOM-03
`application/usecase/{CreateReferentialDraft,PublishReferential,DeriveDraft}UseCase.java`, `web/rest/ReferentialResource.java`	créer	HOM-01
`application/usecase/OpenLiaisonUseCase.java`, `web/rest/LiaisonResource.java`, `infrastructure/client/participant/*`	créer	HOM-04
`infrastructure/messaging/consumer/{SubscriptionChanged,StatusChanged}Consumer.java`, `domain/service/ScopeDeriver.java`	créer	HOM-05, HOM-06
`domain/service/OperationInstantiator.java`, `application/usecase/InstantiateOperationsUseCase.java`, `web/rest/OperationResource.java`	créer	HOM-07
`domain/service/AssignmentRule.java`, `application/usecase/AssignResponsibleUseCase.java`	créer	HOM-08

Fichier (relatif à `org/gimac/des/homologation`)	Action	Récits
`application/port/out/DocumentDepositPort.java`, `infrastructure/client/document/*`, `infrastructure/messaging/consumer/DocumentSealedConsumer.java`	créer	HOM-09
`domain/service/ClosureGuard.java`, `ClosureHasher.java`, `application/port/out/{Reauthentication,TimestampAuthority}Port.java`, `infrastructure/client/{iam,tsa}/*`, `infrastructure/scheduling/TimestampRetryJob.java`, `application/usecase/{Validate,Reopen}OperationUseCase.java`	créer	HOM-10, HOM-11
`domain/service/ControlGridRule.java`, `web/rest/ControlGridResource.java`	créer	HOM-12
`application/port/out/SimulatorReaderPort.java`, `infrastructure/client/simulator/*`, `infrastructure/scheduling/ReplayJob.java`, `domain/service/{ReplayIdempotence,ScenarioMatcher,ReplayCircuitBreaker}.java`, `web/rest/{ReplayResource,SimulatorNotificationResource,UnmatchedScenarioResource}.java`	créer	HOM-13 à HOM-15
`infrastructure/security/OutboundPolicy`, `kube/networkpolicy-mshomologation.yaml`, `src/test/.../NonIntrusionArchTest.java`	créer	HOM-16
`domain/service/{CoverageCalculator,ReadinessRule,EvaluationBaseHasher}.java`, `application/usecase/RecalculateCoverageUseCase.java`, `web/rest/InternalLiaisonResource.java`	créer	HOM-17, HOM-18
`domain/service/LiaisonStateMachine.java`, `infrastructure/messaging/consumer/Certification*Consumer.java`	créer	HOM-19
`infrastructure/persistence/LiaisonDashboardProjection.java`, `web/rest/DashboardResource.ja`	créer	HOM-20

Fichier (relatif à `org/gimac/des/homologation`)	Action	Récits
`va`		
`infrastructure/scheduling/DeadlineJob.java`, `web/rest/DeadlineParametersResource.java`	créer	HOM-21
`domain/service/EvidenceManifestBuilder.java`, `application/usecase/ExportEvidenceFileUseCase.java`, `web/rest/EvidenceFileResource.java`	créer	HOM-22
`web/rest/portail/{PortalLiaisonResource,PortalEvidenceResource}.java`, `infrastructure/persistence/PortalLiaisonProjection.java`, `infrastructure/security/InstitutionScope.java`	créer	HOM-23, HOM-24
`infrastructure/observability/HomologationMetrics.java`	créer	transverse

Fichiers à créer ou modifier dans MS-CERTIFICATION

Fichier (relatif à `org/gimac/des/certification`)	Action	Récits
`jdl/lot1-certification.jdl`, changelog `lot1_post_generation.xml` (jsonb, unicités, séquence des numéros, déclencheur d'immuabilité)	créer	Phase 0
`application/port/out/HomologationReaderPort.java`, `infrastructure/client/homologation/*`, `infrastructure/messaging/consumer/LiaisonReadyConsumer.java`, `application/usecase/OpenCircuitUseCase.java`, `web/rest/CircuitResource.java`	créer	CER-01
`domain/service/{DecisionHasher,CircuitStateMachine,ImpartialityRule}.java`, `application/usecase/SignQuitusUseCase.java`, `application/port/out/{Reauthentication,KeyVaultSigning,DelegationReader}Port.java`, `infrastructure/client/{iam,vault}/*`	créer	CER-02, CER-03, CER-05, CER-07
`domain/service/GoLivePrerequisitesGuard.java`, `application/usecase/Authorize`	créer	CER-04

Fichier (relatif à `org/gimac/des/certification`)	Action	Récits
`GoLiveUseCase.java`		
`domain/service/LapseRule.java`, `infrastructure/messaging/consumer/{OperationReopened,CoverageChanged}Consumer.java`	créer	CER-06
`domain/service/{CompletenessEvaluator,DossierStateMachine}.java`, `application/port/out/DocumentVerificationPort.java`, `web/rest/DossierResource.java`	créer	CER-08
`application/port/out/ChangeRecordReaderPort.java`, `infrastructure/client/changerecord/*`, `domain/service/FreshnessRule.java`, `kube/networkpolicy-mscertification.yaml`	créer	CER-09
`application/usecase/RecordGoLiveReportUseCase.java`, `web/rest/GoLiveReportResource.java`	créer	CER-10
`application/port/out/TimestampAuthorityPort.java`, `infrastructure/client/tsa/Rfc3161Client.java`, `infrastructure/scheduling/SealingRetryJob.java`, `application/usecase/SealDossierUseCase.java`	créer	CER-11
`domain/service/AttestationNumbering.java`, `infrastructure/document/{AttestationPdfBuilder,PdfA3Validator,PadesSigner}.java`, `application/usecase/IssueAttestationUseCase.java`, `web/rest/AttestationResource.java`	créer	CER-12
`web/rest/portail/PortalCertificationResource.java`, `infrastructure/persistence/PortalDossierProjection.java`	créer	CER-13
`infrastructure/messaging/consumer/AttestationIssuedConsumer.java`, `infrastructure/scheduling/RenewalJob.java`, `web/rest/RegisterResource.java`	créer	CER-14
`application/usecase/{RequestStatusChange,ApproveStatusChange}UseCase.java`, `web/rest/StatusDecisionResour`	créer	CER-15

Fichier (relatif à `org/gimac/des/certification`)	Action	Récits
`ce.java`		
`infrastructure/observability/CertificationMetrics.java`	créer	transverse

Fichiers à créer dans l'application d'interface

Fichier (relatif à `src/`)	Action	Écrans
`lib/http/client.ts`, `lib/live/index.ts`, `lib/reauth/webauthn.ts`, `design-system/components/SignDialog.tsx`	créer ou compléter	F0
`design-system/domain/{LiaisonProgress,OperationNode,DagView,CoverageBar,FourElementsChecklist,QuitusCircuit,TreeView,DiffView,ProgressRing,Hash,StateMachineBar}.tsx` et leurs `*.stories.tsx`	créer	F0
`resources/homologation/{referentielOperations,referenceOperation,liaison,operation,campagne,scenarioInconnu,tableauDeBordLiaisons,clotureOperation}.ts`	créer	DES-08 à DES-12
`resources/certification/{dossierMiseEnService,attestation,registre,procesVerbal}.ts`	créer	DES-11
`resources/portail/{portailLiaison,portailOperation}.ts`	créer	POR-03
`features/referentiels/*`, `features/liaisons/*`, `features/certification/*`, `features/portail/*` (pages réduites aux composants universels)	créer	tous
`i18n/fr/homologation.json`, `i18n/en/homologation.json`, `i18n/fr/certification.json`, `i18n/en/certification.json`	créer	tous
`test/e2e/lot1/*.feature`, `test/e2e/lot1/*.spec.ts`	créer	tous

Ordre d'implémentation

Phase 0 (relevé, JDL, Liquibase, messagerie, permissions, routes, activation, structure, exécution à vide ; F0) → HOM-01 → HOM-02 → HOM-04 → HOM-05 → HOM-07 → HOM-08 → HOM-09 → HOM-10 → HOM-11 → HOM-06 → HOM-12 → DES-12 → DES-08 → DES-09 → DES-10 (jalon A) → HOM-13 → HOM-14 → HOM-15 → HOM-16

→ HOM-17 → HOM-18 → CER-01 → CER-07 → CER-02 → CER-03 → CER-09 → CER-08 → CER-04 → CER-05 → HOM-19 → CER-10 → CER-11 → HOM-22 → CER-12 → HOM-20 → HOM-23 → HOM-24 → CER-13 → DES-11 → POR-03 (jalon B) → CER-06 → HOM-21 → CER-14 → CER-15 → HOM-03 → compléments DES-11 → recette du lot (jalon C).

Le frontend suit le backend avec un décalage d'une semaine par écran : un écran n'est commencé que lorsque les contrats REST qu'il consomme sont fusionnés et exposés par la passerelle de recette avec les adaptateurs de substitution.

Calendrier indicatif

Étape	Début	Fin	Contenu	Livrable de sortie
Phase 0	lundi 5 octobre 2026	vendredi 9 octobre 2026	Relevé, JDL, Liquibase, messagerie, permissions, routes, F0	Deux services à vide, chaîne verte, tableau de correspondance
Itération A	lundi 12 octobre 2026	vendredi 30 octobre 2026	HOM-01 à HOM-12 (hors HOM-03), DES-12, DES-08, DES-09, DES-10	Démonstration : ouverture d'une liaison et clôture d'une opération par la règle des quatre éléments
Itération B	lundi 2 novembre 2026	vendredi 27 novembre 2026	HOM-13 à HOM-20, HOM-22 à HOM-24, CER-01 à CER-05, CER-07 à CER-13, DES-11, POR-03	Démonstration : circuit des quatre quitus, attestation téléchargée depuis le portail
Itération C	lundi 30 novembre 2026	vendredi 11 décembre 2026	HOM-03, HOM-21, CER-06, CER-14, CER-15, compléments DES-11	Migration, échéances, caducité, registre, révocation
Recette du lot	lundi 14 décembre 2026	vendredi 18 décembre 2026	Rejeu des features Gherkin, contrôles de sécurité, revue d'ergonomie, rapport de recette	Procès-verbal de recette du lot 1, rapport de couverture, demande de mise en pilote

Les dates sont calées sur des sprints de deux semaines et sur la capacité de référence ; elles sont revues à la fin de la Phase 0 et à chaque revue d'itération. Les jours fériés du siège ne sont pas décomptés ici et le sont dans l'outil de suivi.

Definition of Done du lot 1 (40 points)

1. JDL des deux services aligné, régénéré et versionné ; tableau de correspondance des noms publié.

2. Changelogs Liquibase du lot appliqués : colonnes `jsonb`, unicités composites, index, séquence des numéros d'attestation, déclencheurs d'immuabilité.
3. Classes communes de messagerie présentes (socle ou locales) ; relais d'outbox avec `SKIP LOCKED` ; table `processed_event`.
4. Sujets Kafka du lot et files de rebut créés ; schémas JSON des événements versionnés et vérifiés par des tests de contrat.
5. Permissions du lot déclarées dans MS-IAM ; politique OPA du lot chargée ; `@PreAuthorize` sur chaque route.
6. Quatre routes de la passerelle ouvertes ; refus sans jeton (401) et refus par OPA (403) vérifiés sur chaque route par script.
7. Jeton de service et file de rebut Kafka activés ; mTLS configuré pour les adaptateurs réels des tiers.
8. Arborescence hexagonale en place ; `NonIntrusionArchTest` vert dans les deux services.
9. Chaque port sortant possède un adaptateur réel et un adaptateur de substitution sélectionnés par propriété.
10. Référentiel d'opérations versionné, publié, immuable ; acyclicité contrôlée par tri de Kahn avec cycle restitué (HOM-01, HOM-02).
11. Liaison ouverte avec unicité institution et type de projet, chef de projet et portage ; périmètre dérivé des services souscrits ; gel et dégel (HOM-04 à HOM-06).
12. Opérations instanciées par tri topologique, déblocage automatique, prédécesseurs hors périmètre neutres (HOM-07).
13. Responsable unique par opération avec contrôle de partie et d'institution (HOM-08).
14. Preuve déposée via MS-DOCUMENT, référence et empreinte rattachées, versions et annulation des validations (HOM-09).
15. Clôture refusée tant qu'un des quatre éléments manque, l'élément manquant étant nommé ; validateur distinct ; jeton RFC 3161 obtenu, vérifié et conservé ; réessai sur indisponibilité (HOM-10).
16. Réouverture motivée avec historique, reverrouillage des successeurs, recalcul des couvertures (HOM-11) ; grilles de contrôle du code QR (HOM-12).
17. Reprise du simulateur en lecture seule, idempotente, avec empreinte de réponse ; aucune route de modification d'un verdict (HOM-13).
18. Disjoncteur à trois états, curseur de reprise, file de rebut consultable et relançable (HOM-14).
19. Rapprochement par identifiant de scénario, règle de l'exécution la plus récente, file de qualification (HOM-15).
20. Politique réseau de lecture seule vers le simulateur ; tentatives d'écriture bloquées et journalisées (HOM-16).
21. Deux couvertures distinctes par service, cas non calculable explicite, recalcul incrémental (HOM-17).
22. Aptitude détectée, empreinte du socle stable, émission unique par cycle, interfaces internes protégées (HOM-18).

23. Décisions de certification reflétées dans l'automate de la liaison ; réouverture groupée ; anomalies signalées (HOM-19).
24. Tableau de bord des liaisons avec délais par phase et retards ; échéances notifiées sans doublon (HOM-20, HOM-21).
25. Dossier de preuves exporté avec manifeste, empreintes vérifiées, interruption et alerte sur divergence (HOM-22).
26. Portail : dépôt et suivi cloisonnés par institution, 404 hors institution, aucune annotation interne, canal poussé filtré (HOM-23, HOM-24).
27. Circuit ouvert automatiquement sur aptitude, socle vérifié par empreinte, redélivrance sans doublon (CER-01).
28. Quitus signés après réauthentification FIDO2, motif obligatoire, signature par le coffre, décision immuable (CER-02, CER-03, CER-04).
29. Refus signé avec désignations validées ; caducité sur changement de socle ; impartialité contrôlée avec délégations (CER-05 à CER-07).
30. Six conditions de complétude évaluées avec leur source ; automate à cinq états avec table de transitions ; blocage sur empreinte divergente (CER-08).
31. Fiche de changement lue en authentification mutuelle, fraîcheur de quinze minutes, aucune valeur par défaut, aucune route d'écriture (CER-09).
32. Procès-verbal consigné dans la fenêtre, mention hors fenêtre et régularisation (CER-10).
33. Quitus et dossier scellés par jeton RFC 3161 vérifié (nonce, chaîne, empreinte, temps) ; réessai exponentiel (CER-11).
34. Attestation PDF/A-3 validée, pièces jointes, signature PAdES de niveau long terme, dépôt WORM, référence d'audit, idempotence (CER-12).
35. Portail des certifications cloisonné, téléchargement journalisé ; registre avec lignée et veille des échéances ; révocation et suspension à quatre yeux (CER-13 à CER-15).
36. Interface : descripteurs du lot, composants de domaine avec histoires Storybook et test axe, pages réduites aux composants universels, aucune couleur ni taille en dur, aucun `any`, aucun appel réseau hors `lib/http`.
37. Interface : quatre états de chaque écran testés ; actions absentes quand non permises ; `SignDialog` avec WebAuthn et en-tête `X-Reauth-Assertion` ; motif obligatoire ; identifiant de corrélation affiché sur erreur.
38. Métriques du lot exposées, tableau de bord et règles d'alerte livrés ; disjoncteurs et délais configurés selon le tableau de résilience.
39. Couverture de tests : unitaires supérieure à 85 % sur `domain` et `application`, intégration verte avec Testcontainers, features Cucumber vertes pour les 39 récits, features Playwright vertes pour les 6 écrans, chaîne de construction verte.
40. Contrôles de sécurité de fin d'itération passés ; rapport de recette du lot signé ; état du projet mis à jour dans le dossier de suivi.

Registre des risques du lot

Risque	Probabilité	Impact	Parade
Noms du JDL du socle	élevée	faible	Tableau de

Risque	Probabilité	Impact	Parade
différents de ceux du plan			correspondance dès la Phase 0 ; le plan ne change pas
IAM-07 (réauthentification FIDO2) non livré à temps	moyenne	élevé	Adaptateur de substitution en profil `dev` ; passage à `real` sans changement de code ; aucune mise en pilote sans l'adaptateur réel
Format des résultats du simulateur inconnu avant l'accès de lecture	moyenne	moyen	Couche anticorruption : seul l'adaptateur change ; contrat de lecture figé sur un exemple fourni par l'exploitant du simulateur
Autorité d'horodatage ou coffre de clés indisponibles en recette	moyenne	moyen	Substituts conformes RFC 3161 et PAdES pour la recette ; tests de bascule vers les réels avant le pilote
Signature PAdES de niveau long terme plus complexe que prévu (validation d'horodatage, chaînes)	moyenne	moyen	Réserver quatre jours en itération B ; vérification par outil indépendant dès le premier document
Dérive de charge : 254 points sur cinq sprints	élevée	moyen	Deux binômes en parallèle ; HOM-03, HOM-21, CER-14, CER-15 reportables sans casser le jalon B
Divergence entre contrats REST et descripteurs d'interface	moyenne	faible	Cartographie du présent plan comme liste de contrôle ; tests de contrat partagés

Prompts d'amorçage pour l'agent codeur

Chaque itération est ouverte par un prompt de session qui fixe le cadre ; les prompts de récit sont ensuite construits à partir du fragment correspondant du présent plan (sections A à E) en y ajoutant les fichiers du dépôt à lire.

Prompt de session, itération A (backend MS-HOMOLOGATION)

Tu travailles sur le microservice `mshomologation` de la plateforme intégrée du DES du GIMAC (Java 21, Spring Boot 3, JHipster, PostgreSQL, Kafka, Keycloak, OPA). Lis d'abord `docs/lot1/PI-GIMAC-01.md` (convention transversale, Phase 0, cartographie) et `docs/lot1/correspondance.md`. Règles : couches hexagonales de la Phase 0 ; services de domaine purs sans Spring ; toute émission d'événement par `OutboxPublisher.publish` dans la transaction ; toute consommation par `IdempotentConsumer.handleOnce` ; un port sortant a un adaptateur réel et un adaptateur de substitution ; aucune écriture vers le simulateur ; aucune donnée de carte ; erreurs en `ProblemDetail` avec `code` et `correlationId` ; `@PreAuthorize` avec les

permissions de la cartographie. Pour chaque récit que je te donne, produis dans l'ordre : le code de la section A, les tests B, C et D, puis exécute `./mvnw verify` et le bloc E contre l'environnement de développement ; ne passe au récit suivant que si tout est vert. Ne modifie jamais un contrat REST ou un événement sans me le signaler.

Prompt de session, itération B (backend MS-CERTIFICATION)

Même cadre que l'itération A, sur `mscertification`. Points spécifiques : les quitus sont des décisions immuables signées par le coffre de clés après réauthentification (`ReauthenticationPort`, en-tête `X-Reauth-Assertion`) ; l'ordre des quitus est porté par `CircuitStateMachine` et ne se contourne pas ; la fiche de changement est lue, jamais écrite, et relue au-delà de quinze minutes ; le dossier suit `DossierStateMachine` à cinq états ; le scellement passe par `TimestampAuthorityPort` (Bouncy Castle) et l'attestation par le générateur PDF/A-3 signé PAdES ; aucune décision partielle n'est enregistrée si un tiers échoue. Vérifie chaque signature et chaque jeton avec un outil indépendant dans les tests d'intégration.

Prompt de session, itération A et B (frontend)

Tu travailles sur l'application d'interface (React 18, TypeScript strict, Vite). Lis `DS-GIMAC-01` en entier, puis les fragments F0 et de l'écran demandé dans `docs/lot1/PI-GIMAC-01.md`. Règles : une fonctionnalité commence par un descripteur `ResourceDescriptor<T>` puis, si nécessaire, un composant de domaine ; une page est réduite à l'appel d'un composant universel ; les transitions et actions permises viennent du service ; toute action à effet passe par `confirm` ou `sign` ; aucune couleur ni taille en dur ; aucun `any` ; aucun `fetch` hors `lib/http` ; catalogues fr et en complétés dans la même modification ; chaque composant de domaine a ses histoires Storybook et son test axe ; chaque écran a ses quatre états testés et sa feature Playwright. Termine par la grille de revue en trente points du système de conception et corrige avant de livrer.

Conclusion

Le présent plan transforme les trente-neuf récits du module de gestion de la certification et ses six écrans en un chemin d'exécution unique, ordonné par les dépendances et vérifié à chaque pas par quatre familles de tests. Il s'appuie sur un socle déjà éprouvé et n'y ajoute que ce que le lot exige : deux schémas de données, onze sujets d'événements, trente-cinq permissions, quatre routes et l'activation du jeton de service et du rebut. À l'issue du jalon B, le GIMAC dispose d'un suivi de l'homologation opposable, d'un circuit de certification signé et horodaté et d'une attestation vérifiable hors de la plateforme, remis au participant par son portail. Le plan d'implémentation du lot 2 (MS-INCIDENT et MS-MONITORING, écrans DES-13 à DES-18) sera établi sur le même schéma à la revue de fin d'itération B.